Networking

K19G

2026-10-31

Table of contents

Networking **45**
Curriculum map . . . 45
Journey map . . . 46
What "done" looks like . . . 46
Lab ground rules . . . 47
Related books . . . 47
Formats . . . 47

Lab Platform **48**

Lab Host — macOS **49**

Lab Host — macOS **50**
Goals . . . 50
Prerequisites . . . 50
Virtualization and container runtime . . . 50
Options . . . 50
Checklist . . . 51
Install Containerlab . . . 51
Networking notes on macOS . . . 51
First smoke test . . . 51
Next . . . 51

Lab Host — Linux **52**

Lab Host — Linux **53**
Goals . . . 53
Prerequisites . . . 53
Container runtime . . . 53
Typical path (Debian/Ubuntu-style) . . . 53
Kernel / modules . . . 54
Install Containerlab . . . 54
Host networking and firewall . . . 54
First smoke test . . . 54
Next . . . 54

Containerlab Fundamentals **55**

Containerlab Fundamentals **56**
Learning goals . . . 56

Prerequisites . . . 56
How Containerlab fits together . . . 57
Minimal topology: two Linux nodes . . . 58
Triangle of FRR routers (book default pattern) . . . 58
Startup config sketch (`config/r1/daemons` + `frr.conf`) . . . 59
Lifecycle . . . 60
Topology YAML anatomy . . . 60
Kinds you will actually use . . . 60
Links and endpoints . . . 60
Multipoint / LAN segments . . . 61
Management network . . . 61
Inspect, exec, save . . . 61
Lab directory layout (recommended) . . . 62
Graph and documentation . . . 62
Partial failure recovery . . . 62
Predict → observe → fix lab . . . 63
Setup . . . 63
Predict . . . 63
Observe . . . 63
Fix inject . . . 64
Harden . . . 64
Common mistakes . . . 64
Version awareness (mid-2026) . . . 64
Security and shared hosts . . . 65
Summary . . . 65

Netlab Fundamentals **66**

Netlab Fundamentals **67**
Learning goals . . . 67
The Dual-Engine Model: Why Both Tools Matter . . . 68
Installation & Setup . . . 69
Prerequisites . . . 69
Automated Platform Provisioning . . . 69
The Anatomy of a Netlab Topology . . . 69
Minimal Triangle Topology with OSPF . . . 69
What Happens Behind the Scenes? . . . 70
Operating Netlab Labs . . . 71
1. Starting and Stopping Labs . . . 71
2. Connecting to Devices . . . 71
3. Graphing Topologies . . . 71
Automated Verification with `netlab validate` . . . 72
Simulating Real-World Links with `netlab tc` . . . 73
Multi-NOS Portability: The Ultimate Learning Superpower . . . 73
Summary & What Comes Next . . . 74

Persistent State in Containerlab Labs **75**

Persistent State in Containerlab Labs **76**
Thesis . . . 76
Mental model . . . 76
Worked examples . . . 76
Trap . . . 77
Boring rule . . . 77
Try this . . . 77
Sources . . . 77

A First BGP Lab with netlab **78**

A First BGP Lab with netlab **79**
Thesis . . . 79
Mental model . . . 79
Worked example . . . 79
Trap . . . 80
Boring rule . . . 80
Try this . . . 80
Sources . . . 80

Free Image Matrix & Resources **81**

Free Image Matrix & Resources **82**
Learning goals . . . 82
Principles . . . 82
Role matrix . . . 82
What we deliberately de-emphasize . . . 83
FRR as the workhorse . . . 83
Linux endpoints . . . 84
SR Linux community (optional track) . . . 84
SONiC and VyOS (optional) . . . 85
Sizing the lab host . . . 85
RAM savers . . . 85
Disk . . . 86
Version pinning strategy . . . 86
Resource planning worksheet . . . 86
Network resource limits on the host . . . 87
Documentation & community map . . . 87
RFCs worth bookmarking (foundations) . . . 87
Predict → observe: image sanity lab . . . 88
Multi-implementation mindset . . . 88
Ethics and redistribution . . . 88
Summary . . . 89

Linux Networking for Lab Hosts **90**

Linux Networking for Lab Hosts **91**
Learning goals . . . 91
Learning goals in practice . . . 91

Core objects . . . 91
Network namespaces . . . 92
veth pairs: virtual cables . . . 92
Bridges: software switches . . . 93
Routing between namespaces . . . 94
Failure drill: disable forwarding . . . 95
Failure drill: missing return route . . . 95
VLAN subinterfaces (preview of trunks) . . . 95
Policy leftovers that break labs . . . 95
rp_filter (reverse path filtering) . . . 95
Forwarding and firewall . . . 96
Disable offloads when captures look weird (optional) . . . 96
Mapping to Containerlab . . . 96
Clean teardown habits . . . 97
Example `down.sh` . . . 97
Predict → observe → fix drill (checklist) . . . 97
Useful one-liners cheatsheet . . . 98
Security note for shared hosts . . . 98
Summary . . . 98

Lab Hygiene **99**

Lab Hygiene **100**
Learning goals . . . 100
Why hygiene is a skill . . . 100
Addressing plans . . . 100
Template . . . 101
Allocation strategy (personal lab supernets) . . . 101
/30 vs /31 vs /24 on links . . . 101
Naming conventions . . . 102
Lab name . . . 102
Node names . . . 102
Interface names . . . 102
Git branches / folders . . . 102
Directory layout (canonical) . . . 103
README skeleton . . . 103
Git habits . . . 104
Never commit . . . 104
Diagram habit . . . 104
Configuration hygiene . . . 105
FRR example: comment intent . . . 105
verify.sh hygiene . . . 105
Cross-lab contamination . . . 106
Time hygiene . . . 106
Definition of done (lab) . . . 106
Predict → observe → fix: hygiene drill . . . 107
Collaboration norms (optional) . . . 107
Summary . . . 108

Networking Foundations **109**

What Is Networking? **110**

What Is Networking? **111**
Learning goals . . . 111
Building blocks . . . 111
What problem networking solves . . . 111
Switching vs routing (first cut) . . . 112
Protocols as contracts . . . 112
Why labs beat slideware . . . 112
Observation toolkit (preview) . . . 113
Checkpoint . . . 113
Next . . . 113

TCP/IP and the Layered Stack **114**

TCP/IP and the Layered Stack **115**
Learning goals . . . 115
TCP/IP model (working map) . . . 115
OSI as a legend (not a religion) . . . 115
What each layer "owns" . . . 116
Link . . . 116
Internet (IP) . . . 116
Transport . . . 116
Application . . . 116
Demultiplexing (up the stack) . . . 117
Layered troubleshooting . . . 117
Encapsulation (preview) . . . 117
Checkpoint . . . 117
Next . . . 117

Packets, Frames, Encapsulation **118**

Packets, Frames, Encapsulation **119**
Learning goals . . . 119
Vocabulary . . . 119
Encapsulation path (send) . . . 119
Decapsulation path (receive) . . . 120
Maximum sizes and MTU . . . 120
Captures: reading the onion . . . 120
Tunnels and overlays (preview) . . . 120
Lab sketch (two nodes) . . . 121
Checkpoint . . . 121
Next . . . 121

Link Layer and Ethernet **122**

Link Layer and Ethernet **123**
Learning goals . . . 123
Why link layer matters . . . 123
Ethernet frame (simplified) . . . 123
MAC addresses . . . 124
Broadcast domain vs collision domain . . . 124
Switch vs router (again) . . . 124
MAC learning (preview) . . . 124
Virtual Ethernet in labs . . . 125
Common L2 failures (foundations list) . . . 125
Checkpoint . . . 125
Next . . . 126

IPv4 Addressing and Subnetting **127**

IPv4 Addressing and Subnetting **128**
Learning goals . . . 128
IPv4 structure . . . 128
CIDR and masks . . . 129
On-link decision . . . 129
Private and documentation ranges . . . 129
Subnetting for multi-tier labs . . . 129
Point-to-point links . . . 130
Summarization (intro) . . . 130
VLSM . . . 130
Host configuration checklist . . . 132
Worked subnet: `/26` from a `/24` . . . 132
Private vs public (RFC 1918) . . . 132
Paper drills . . . 133
Checkpoint . . . 133
Next . . . 133

IPv6 Essentials **134**

IPv6 Essentials **135**
Learning goals . . . 135
Why IPv6 in this book . . . 135
Address format . . . 135
Address kinds (must know) . . . 136
Interface identifiers and /64 . . . 136
Linux static example . . . 136
Neighbor Discovery (preview) . . . 136
Dual-stack thinking . . . 137
Checkpoint . . . 137
Next . . . 137

SLAAC, the RA M/O Flags and DHCPv6: What a Linux Host Actually Does **138**

SLAAC, the RA M/O Flags and DHCPv6: What a Linux Host Actually Does **139**
Learning goals . . . 139
The three ways an IPv6 host gets an address . . . 139
Lab . . . 140
Reading the output . . . 142
What the kernel knobs mean . . . 143
Practical rules . . . 143
Try this . . . 143
Sources . . . 143

ARP, ND, and Local Resolution **145**

ARP, ND, and Local Resolution **146**
Learning goals . . . 146
When local resolution happens . . . 146
ARP (IPv4) . . . 146
Incomplete / failed ARP . . . 147
Neighbor Discovery (IPv6) . . . 147
Gratuitous ARP and updates . . . 147
Proxy ARP (awareness) . . . 147
Lab sketch . . . 147
Checkpoint . . . 148
Next . . . 148

ICMP and Path Diagnostics **149**

ICMP and Path Diagnostics **150**
Learning goals . . . 150
ICMPv4 types (operator set) . . . 150
ICMPv6 . . . 150
Interpreting ping . . . 151
Traceroute mechanics (conceptual) . . . 151
Path MTU Discovery (preview) . . . 151
Filters and policy . . . 151
Lab sketch . . . 152
Checkpoint . . . 152
Next . . . 152

Transport: TCP and UDP **153**

Transport: TCP and UDP **154**
Learning goals . . . 154
Ports and sockets . . . 154
UDP . . . 154
TCP . . . 155
Three-way handshake (must know) . . . 155
States (operator view) . . . 155
TCP vs UDP selection . . . 155
NAT and transports (preview) . . . 156

Security basics . . . 156
TCP state machine (operator view) . . . 156
Three-way handshake and teardown (why captures look busy) . . . 157
Lab sketch . . . 157
Checkpoint . . . 157
Next . . . 157

Small Writes in TCP: Nagle's Algorithm, Delayed ACKs, and the 40 ms Stall 158

Small Writes in TCP: Nagle's Algorithm, Delayed ACKs, and the 40 ms Stall 159
Learning goals . . . 159
Mental model . . . 159
Concepts — the knobs . . . 161
Lab — reproduce the stall and measure the fixes . . . 162
Lab — catch the stall in `ss` . . . 165
Lab — what Nagle is for . . . 166
Case — choosing the fix . . . 167
The trap . . . 167
The boring rule . . . 168
Try this . . . 168
Next . . . 168
Verification . . . 169
Sources . . . 169

TCP Fast Open 170

TCP Fast Open 171
Thesis . . . 171
Mental model . . . 171
Worked examples . . . 171
Trap . . . 172
Boring rule . . . 172
Try this . . . 172
Sources . . . 172

TIME-WAIT on Linux: What the Timers Really Do 173

TIME-WAIT on Linux: What the Timers Really Do 174
Learning goals . . . 174
Thesis . . . 174
Mental model . . . 174
Why the wait exists (RFC 9293, §3.6) . . . 174
What the kernel source says . . . 176
The knobs, and what each one really does . . . 176
Lab 1: watch the timers (run, no sudo) . . . 177
Lab 2: port exhaustion and the fixes (run, no sudo) . . . 179
Case: the proxy that ran out of ports . . . 182
Trap . . . 182
Boring rule . . . 182

Try this . . . 182
Verification . . . 183
Sources . . . 183

Zero-Copy Sends with MSG_ZEROCOPY: Completions, Deferred Copies, and When It Loses 184

Zero-Copy Sends with MSG_ZEROCOPY: Completions, Deferred Copies, and When It Loses 185
Learning goals . . . 185
The contract . . . 186
Lab: measure it on loopback . . . 186
Reading the numbers . . . 188
What changes on a real NIC (not run here) . . . 188
The boring rule . . . 188
Try this . . . 189
Sources . . . 189

Common Application Protocols 190

Common Application Protocols 191
Learning goals . . . 191
Map (operator cheat sheet) . . . 191
DNS . . . 191
Roles . . . 191
Query types (common) . . . 192
Failure patterns . . . 192
DHCP (IPv4) . . . 192
HTTP and HTTPS . . . 193
SSH . . . 193
TLS in one paragraph . . . 193
NTP and time . . . 193
Lab sketch . . . 194
Checkpoint . . . 194
Next . . . 194

Host Forwarding and Routing Intro 195

Host Forwarding and Routing Intro 196
Learning goals . . . 196
Connected vs remote . . . 196
Linux route table essentials . . . 196
Longest prefix match . . . 197
Default gateway . . . 197
Static routes (host or simple router) . . . 197
IP forwarding (Linux as router) . . . 197
Policy routing (awareness) . . . 198
Firewall vs routing . . . 198
Lab sketch . . . 198

Checkpoint . . . 198
Next . . . 198

Control, Data, and Management Planes **199**

Control, Data, and Management Planes **200**
Learning goals . . . 200
Data plane . . . 200
Control plane . . . 200
Management plane . . . 201
Interaction . . . 201
Troubleshooting order (habit) . . . 202
Lab habit . . . 202
Checkpoint . . . 202
Next . . . 202

Encrypted DNS — DoH and DoT **203**

Encrypted DNS — DoH and DoT **204**
Thesis . . . 204
Mental model . . . 204
Worked examples . . . 204
Trap . . . 204
Boring rule . . . 204
Try this . . . 204
Sources . . . 205

Layer 2 **206**

Ethernet & MAC Learning **207**

Ethernet & MAC Learning **208**
Learning goals . . . 208
Concepts . . . 209
Frame essentials . . . 209
Learning and flooding . . . 209
MAC addresses . . . 209
ARP: glue for IPv4 on Ethernet . . . 210
Linux bridge FDB . . . 211
Lab A: two hosts, one bridge (hand or Containerlab) . . . 211
Containerlab topology . . . 211
Predict . . . 212
Observe . . . 212
Fix inject . . . 212
Harden . . . 213
Lab B: three hosts — flooding visibility . . . 213
MAC moves and flaps . . . 213
Common L2 failures (pre-VLAN) . . . 213

Promiscuous mode and labs . . . 213
Security digression (short) . . . 214
Predict worksheet (paper) . . . 214
FRR / NOS note . . . 214
Summary . . . 214

VLANs & Trunks **216**

VLANs & Trunks **217**
Learning goals . . . 217
Concepts . . . 218
Broadcast domains . . . 218
802.1Q tag . . . 218
Router roles . . . 218
Addressing plan example . . . 218
Lab topology (conceptual) . . . 219
Containerlab sketch using Linux VLAN-aware bridge . . . 219
`config/sw1.sh` (VLAN filtering bridge) . . . 220
Predict → observe → fix . . . 221
Predict . . . 221
Observe . . . 221
Failure drill: native / PVID mismatch . . . 221
Failure drill: missing allowed VLAN . . . 222
Router-on-a-stick preview . . . 222
Verification checklist . . . 222
verify.sh sketch . . . 222
Design tips . . . 223
Common mistakes . . . 223
Summary . . . 223

Loop Prevention **224**

Loop Prevention **225**
Learning goals . . . 225
Why loops kill L2 . . . 225
Spanning tree behavior . . . 225
Priority and ties . . . 226
Port states (classic story) . . . 226
Design alternatives (model maturity) . . . 226
Lab: three switches, intentional loop . . . 227
Topology YAML (Linux bridges) . . . 227
Dangerous mode: dumb bridges without STP . . . 228
Observe storm (careful) . . . 228
Safer mode: enable STP on the bridge . . . 228
Root placement drill . . . 229
Failure drill: link down on active tree edge . . . 229
BPDUs and protection concepts (awareness) . . . 230
When not to extend L2 . . . 230

Predict worksheet . . . 230
Verification checklist for STP labs . . . 230
Summary . . . 231

Link Aggregation **232**

Link Aggregation **233**
Learning goals . . . 233
Concepts . . . 233
One logical link . . . 233
Static vs LACP . . . 233
Hashing . . . 234
Split brain / misconfig . . . 234
Linux bonding quickstart . . . 234
Containerlab topology (two paths, bond on routers) . . . 235
`config/r1-bond.sh` . . . 236
Predict → observe → fix . . . 236
Predict . . . 236
Observe . . . 237
Failover drill . . . 237
Mismatch drill . . . 237
Hashing awareness drill . . . 237
STP interaction . . . 237
Operational checks (NOS-neutral) . . . 238
Common mistakes . . . 238
Summary . . . 238

L2 Failure Drills **239**

L2 Failure Drills **240**
Learning goals . . . 240
Baseline lab . . . 240
Evidence kit . . . 241
Drill catalog . . . 241
Drill 1 — Access port wrong VLAN . . . 241
Drill 2 — Trunk missing VLAN . . . 241
Drill 3 — Native VLAN mismatch . . . 242
Drill 4 — Cable / interface down . . . 242
Drill 5 — Broadcast storm (controlled) . . . 242
Drill 6 — STP blocked port confusion . . . 243
Drill 7 — MAC move . . . 243
Drill 8 — Duplicate IP . . . 243
Drill 9 — Duplicate MAC (advanced) . . . 244
Drill 10 — LAG member down . . . 244
Drill 11 — Inter-VLAN path mistaken for L2 . . . 244
Drill 12 — Host firewall . . . 245
Incident note template . . . 245
Scorecard . . . 245

Combined verify.sh ideas . . . 246
Checkpoint: L2 competence . . . 246
Summary . . . 246

Layer 3 **248**

IP Addressing **249**

IP Addressing **250**
Learning goals . . . 250
IPv4 structure . . . 250
On-link decision . . . 251
Subnetting for labs . . . 251
Summarization . . . 251
Paper drill . . . 252
Special-use ranges (know them) . . . 252
IPv6 essentials for dual-stack labs . . . 252
Dual-stack habit . . . 253
Linux configuration patterns . . . 253
Persistent lab style . . . 253
Multiple addresses . . . 253
Lab: addressing plan + dual-stack hosts . . . 253
Predict . . . 254
Observe . . . 254
Failure: overlapping subnets . . . 255
Failure: bad summary (static preview) . . . 255
Calculation practice (quick) . . . 255
Host vs router addressing habits . . . 255
Verification checklist . . . 255
Summary . . . 256

Routing Fundamentals **257**

Routing Fundamentals **258**
Learning goals . . . 258
Concepts . . . 258
Forwarding equation . . . 258
RIB vs FIB . . . 259
Longest prefix match . . . 259
Metrics and distance (ideas) . . . 259
Connected routes . . . 259
Host vs router . . . 260
Gateway ARP . . . 260
Next-hop resolution failures . . . 260
Asymmetric routing . . . 260
Lab: competing prefixes . . . 261
Predict / observe . . . 262
Lab: router with two interfaces . . . 262

ARP/ND deep enough for routing . . . 263
TTL and traceroute . . . 263
Policy preview . . . 263
Predict → observe → fix drills . . . 264
Drill: missing return route . . . 264
Drill: wrong gateway MAC path . . . 264
Drill: more-specific shadow . . . 264
Cheatsheet . . . 264
Summary . . . 264

Static Routing Labs **266**

Static Routing Labs **267**
Learning goals . . . 267
Topology: triangle with hosts . . . 267
Containerlab topology . . . 269
FRR config sketches . . . 270
Linux-only alternative . . . 270
Deploy and baseline . . . 271
Predict . . . 271
Observe . . . 271
Floating statics . . . 271
Failover drill . . . 271
Return path drill (mandatory) . . . 272
Blackhole and reject . . . 272
verify.sh . . . 272
Addressing.md excerpt . . . 272
Default routes at edge . . . 273
IPv6 statics (optional parallel) . . . 273
Common mistakes . . . 273
Summary . . . 273

ICMP & Path MTU **275**

ICMP & Path MTU **276**
Learning goals . . . 276
ICMP essentials (IPv4) . . . 276
Path MTU discovery (idea) . . . 277
Lab: MTU mismatch . . . 277
Topology . . . 277
Predict . . . 278
Observe . . . 278
Inject: drop ICMP unreachable on r1 . . . 279
Restore . . . 279
Harden . . . 279
Traceroute and ICMP filters . . . 279
Capture workflow . . . 279
iperf3 large transfers . . . 280

IPv6 notes . . . 280
Operational checklist . . . 280
Predict worksheet . . . 280
verify.sh additions . . . 281
Summary . . . 281

BGP no_export and Well-Known Communities **282**

BGP no_export and Well-Known Communities **283**
Thesis . . . 283
Mental model . . . 283
Worked example . . . 283
Trap . . . 284
Boring rule . . . 284
Try this . . . 284
Sources . . . 284

RPKI ROAs and the maxLength Trap **285**

RPKI ROAs and the maxLength Trap **286**
Thesis . . . 286
Mental model . . . 286
Worked example . . . 286
Trap . . . 287
Boring rule . . . 287
Try this . . . 287
Sources . . . 287

Interior Routing **288**

OSPF Basics **289**

OSPF Basics **290**
Learning goals . . . 290
Concepts . . . 290
Distance-vector vs link-state . . . 290
OSPF adjacency state machine (short) . . . 290
Areas . . . 291
Cost . . . 291
Network types (awareness) . . . 291
Addressing plan . . . 291
Method A: The Fast Netlab Workflow (Recommended) . . . 291
Deploy and Validate with Netlab . . . 292
Method B: Raw Containerlab & Manual FRR Configurations . . . 293
Topology YAML (`ospf-tri.clab.yml`) . . . 293
FRR Daemons File (`config/r1/daemons`) . . . 294
FRR OSPF Config (`config/r1/frr.conf`) . . . 294
Deploy and Verify Manually . . . 295

Predict . . . 295
Neighbor not Full? . . . 295
Cost change drill . . . 295
Link failure drill . . . 296
Passive interfaces / LAN notes . . . 296
Verification checklist . . . 296
verify.sh . . . 296
Common mistakes . . . 297
Beyond this chapter . . . 297
Summary . . . 297

BGP Basics **298**

BGP Basics **299**
Learning goals . . . 299
Concepts . . . 299
Autonomous systems . . . 299
eBGP vs iBGP . . . 300
Session state . . . 300
Path selection overview . . . 300
NLRI and attributes . . . 300
Lab A: two AS eBGP . . . 300
Topology . . . 301
daemons . . . 301
r1 frr.conf . . . 302
r2 . . . 302
Verify session . . . 302
Predict . . . 303
Lab B: iBGP sketch (optional same day) . . . 303
Session failure drill . . . 303
Next-hop reachability drill . . . 303
Dual-home teaser . . . 304
verify.sh . . . 304
Safety habits . . . 304
Common mistakes . . . 304
Summary . . . 305

Policy & Filters **306**

Policy & Filters **307**
Learning goals . . . 307
Why policy exists . . . 307
Prefix lists (FRR) . . . 308
Route-maps as idea . . . 308
Lab: filter unauthorized prefixes . . . 308
Customer r1 advertises extra junk . . . 309
ISP r2 policy . . . 309
Predict . . . 309

Observe . . . 309
Harden . . . 309
Local-pref steer drill . . . 310
Outbound preference (customer) . . . 310
Redistribution loop awareness . . . 310
Example static→OSPF with filter (sketch) . . . 311
ACL-style packet filters vs routing policy . . . 311
Distribute-list / filter-list ideas . . . 311
Failure drills . . . 311
Drill 1 — over-broad permit any . . . 311
Drill 2 — accidental deny all . . . 312
Drill 3 — next-hop-self missing (iBGP) . . . 312
verify.sh policy checks . . . 312
Policy design worksheet . . . 312
Summary . . . 313

BFD — Fast Failure Detection **314**

BFD — Fast Failure Detection **315**
Learning goals . . . 315
Why carrier is not enough . . . 315
Concepts — BFD in one page . . . 316
Addressing plan (BFD lab) . . . 317
Lab build (Linux namespaces + FRR) . . . 317
Drill 1 — the silent failure (no BFD) . . . 320
Predict . . . 320
Observe . . . 320
Explain . . . 321
Drill 2 — add BFD and repeat . . . 321
Drill 3 — the slower side wins . . . 323
Predict . . . 323
Observe . . . 323
Drill 4 — BFD for a static route . . . 324
Drill 5 — one-sided BFD (predict, then try) . . . 324
Choosing timers . . . 325
Verification checklist . . . 326
Common mistakes . . . 326
Summary . . . 326
Sources . . . 327

RPKI — Route Origin Validation **328**

RPKI — Route Origin Validation **329**
Learning goals . . . 329
Concepts — RPKI in one page . . . 329
Addressing plan (RPKI lab) . . . 332
Lab build (Linux namespaces + FRR + StayRTR) . . . 333

Drill 1 — validation is a label, not a filter . . . 336
Predict . . . 336
Observe . . . 336
Explain . . . 337
Drill 2 — drop Invalid, de-prefer NotFound . . . 337
Drill 3 — the maxLength trap . . . 338
Predict . . . 338
Explain . . . 339
Drill 4 — state changes under accepted routes . . . 339
Predict . . . 339
Observe . . . 339
Explain . . . 340
Drill 5 — the cache goes away (predict, then try) . . . 340
Checking real-world origins . . . 341
Deploying it for real . . . 341
Verification checklist . . . 342
Common mistakes . . . 342
Summary . . . 343
Sources . . . 343

Segment Routing, SRv6, and TI-LFA **345**

Segment Routing, SRv6, and TI-LFA **346**
Learning goals . . . 346
Thesis . . . 346
Mental model . . . 346
SR data planes . . . 347
Worked examples . . . 348
Case 1: Architecture in one page (RFC 8402) . . . 348
Case 2: TI-LFA repair flavours (RFC 9855 §5) . . . 348
Case 3: What the PLR does to the packet (RFC 9855 §6–7) . . . 348
Case 4: Numbers from RFC 9855 Appendix B (SR-MPLS measurements) . . . 349
The trap . . . 349
The boring rule . . . 349
Try this . . . 349
Sources . . . 350

BGP Route Reflectors: Escaping the iBGP Full Mesh **351**

BGP Route Reflectors: Escaping the iBGP Full Mesh **352**
Learning goals . . . 352
Thesis . . . 352
Mental model . . . 353
Concepts . . . 354
The reflection rules (RFC 4456 §6) . . . 354
Two loop checks . . . 355
Route selection (RFC 4456 §9) . . . 355
One cluster ID or two? . . . 355

Lab 1: what the full mesh costs . . . 356
Lab 2: reflection rules in a model you can see all of . . . 356
Lab 3: the same network in netlab (FRR) . . . 359
Compile it (run here) . . . 361
Start it and verify (not run here) . . . 362
Case: placing reflectors . . . 363
The trap . . . 363
The boring rule . . . 364
Try this . . . 364
Next . . . 364
Verification . . . 365
Sources . . . 365

IPv4 Routes over IPv6 Next Hops: BGP Extended Next Hop (RFC 8950) 366

IPv4 Routes over IPv6 Next Hops: BGP Extended Next Hop (RFC 8950) 367
Learning goals . . . 367
Thesis . . . 367
Mental model . . . 367
Concepts . . . 368
Why the classic NEXT_HOP can't do it . . . 368
The length rule (RFC 8950 §3) . . . 369
What changed from RFC 5549 . . . 369
The capability (RFC 8950 §4) . . . 369
Route reflectors and mixed peers (RFC 8950 §5) . . . 369
Lab 1: the bytes, by hand (run here) . . . 370
Lab 2: an IPv4 route with an IPv6 gateway (run here, no sudo) . . . 372
Lab 3: two FRR routers in containerlab (from the docs, not run here) . . . 373
Case: an unnumbered leaf-spine . . . 375
The trap . . . 375
The boring rule . . . 375
Try this . . . 376
Next . . . 376
Verification . . . 376
Sources . . . 376

Ops and Automation 378

Observability for Networks 379

Observability for Networks 380
Learning goals . . . 380
Observability layers . . . 380
Baseline: what "healthy" looks like . . . 380
Interface and error signals . . . 381
Control-plane signals . . . 381
OSPF . . . 381
BGP . . . 382

Path quality . . . 382
Packet capture practice . . . 382
Capture points . . . 382
Logging . . . 383
FRR . . . 383
Kernel . . . 383
Metrics sketch (Prometheus-shaped thinking) . . . 383
Minimal blackbox idea . . . 383
Telemetry awareness (2026) . . . 384
Incident timeline template . . . 384
Lab drill: observe a BGP flap . . . 384
Lab drill: silent MTU blackhole . . . 385
Runbook snippet library . . . 385
Privacy and safety . . . 385
Summary . . . 385

Automation & Lab-as-Code **386**

Automation & Lab-as-Code **387**
Learning goals . . . 387
Principles . . . 387
Directory layout (automation-ready) . . . 387
Makefile pattern . . . 388
up.sh / down.sh . . . 388
Config strategies . . . 389
Safe interactive explore → promote . . . 389
Light Ansible (optional) . . . 389
Templating addressing: DIY vs. Netlab . . . 390
The DIY Approach (Jinja2 + custom scripts) . . . 390
The Netlab Declarative Approach . . . 390
Verification: verify.sh vs. netlab validate . . . 391
Custom Shell Script (`verify.sh`) . . . 391
Netlab Automated Validation . . . 392
CI smoke (GitHub Actions sketch) . . . 392
Image pins in automation . . . 392
Idempotency drills . . . 392
Secret hygiene . . . 393
Graph and inventory export . . . 393
Predict → observe → fix . . . 393
Anti-patterns . . . 393
Summary . . . 394

Capstone Outline **395**

Capstone Outline **396**
Learning goals . . . 396
Definition of done (all capstones) . . . 396

Capstone C1 — Campus-style dual core (open images) . . . 397
Intent . . . 397
Skills exercised . . . 397
Success metrics . . . 397
Suggested stack . . . 397
Capstone C2 — Dual-homed site to provider edge . . . 398
Intent . . . 398
Skills exercised . . . 398
Success metrics . . . 398
verify extras . . . 398
Capstone C3 — Mini fabric snack . . . 398
Intent . . . 398
Skills exercised . . . 399
Stretch (optional) . . . 399
Capstone C4 — Incident week . . . 399
Intent . . . 399
Skills exercised . . . 400
Success metrics . . . 400
How to pick . . . 400
Project template (copy) . . . 400
DESIGN.md skeleton . . . 401
RETRO.md prompts . . . 401
Integration with the book spine . . . 401
Beyond this book (next horizons) . . . 401
Final checkpoint: "expert" as defined in the syllabus . . . 402
Summary . . . 402

Edge and Services **403**

First-Hop Redundancy **404**

First-Hop Redundancy **405**
Learning goals . . . 405
Concepts . . . 405
The problem . . . 405
First-hop redundancy idea . . . 405
Protocol family (vendor-agnostic) . . . 406
Priority, preempt, timers . . . 406
What FHRP is *not* . . . 406
ARP and the VIP . . . 407
Addressing plan . . . 407
Topology YAML . . . 407
FRR daemons . . . 408
FRR VRRP sketch (r1) . . . 409
Linux alternative (keepalived) . . . 409
Deploy and baseline . . . 410
Predict (baseline) . . . 410

Observe . . . 410
Drill 1 — Master failure . . . 410
Predict → observe → fix . . . 410
Drill 2 — Split brain / LAN partition (concept) . . . 411
Predict . . . 411
Observe . . . 411
Fix / harden . . . 411
Drill 3 — Priority change live . . . 412
Predict . . . 412
Observe . . . 412
Return path and asymmetric edges . . . 412
Verification checklist . . . 412
verify.sh sketch . . . 413
Common mistakes . . . 413
Optional SR Linux note (community) . . . 413
Summary . . . 413

DHCP Relay and NAT **415**

DHCP Relay and NAT **416**
Learning goals . . . 416
Concepts — DHCP . . . 416
Why not static everything? . . . 416
Four-way dance (v4) . . . 416
Critical fields . . . 417
Relay failure modes . . . 417
Concepts — NAT family . . . 417
When NAT is appropriate in this book . . . 417
Addressing plan (combined edge lab) . . . 418
Topology YAML . . . 418
Edge addressing and forward . . . 419
DHCP server (dnsmasq) on dhcpsrv . . . 420
Minimal same-subnet lab (sanity first) . . . 420
Relay path (concept + command) . . . 420
Predict (DHCP) . . . 420
Observe . . . 421
Fix . . . 421
NAT — MASQUERADE on edge . . . 421
Predict . . . 421
Observe . . . 422
Fix . . . 422
Drill — break DHCP then NAT . . . 422
FRR on the edge (optional co-location) . . . 422
Verification checklist . . . 423
Common mistakes . . . 423
Summary . . . 423

Name, Time, and Log Hygiene **424**

Name, Time, and Log Hygiene **425**
Learning goals . . . 425
Why this chapter exists . . . 425
Concepts — naming . . . 426
Layers of DNS use . . . 426
Records you actually need . . . 426
Split horizon awareness . . . 426
Concepts — time . . . 426
Concepts — logs . . . 427
Addressing plan (hygiene lab) . . . 427
Topology YAML . . . 427
Bring-up addressing (example) . . . 428
DNS lab — dnsmasq authoritative snack . . . 429
Predict . . . 429
Observe / fix . . . 429
Drill — DNS outage looks like network outage . . . 429
Predict → observe → fix . . . 429
Time sync — chrony sketch . . . 430
Skew drill . . . 430
Predict . . . 430
Observe . . . 430
Logging hygiene . . . 431
Local journal pattern . . . 431
syslog-style forward (concept) . . . 431
What to log for network incidents . . . 431
FRR logging (when routers are FRR) . . . 431
Hygiene checklist (use every serious lab) . . . 432
Predict → observe → fix mini-scenarios . . . 432
Scenario A — "Git clone fails" in a lab VM . . . 432
Scenario B — "Certificate expired" on brand-new lab CA . . . 432
Scenario C — "It failed an hour ago" with no trace . . . 432
Optional: name the lab graph . . . 433
Common mistakes . . . 433
Summary . . . 433

Edge Dual-Home Lab **434**

Edge Dual-Home Lab **435**
Learning goals . . . 435
Design choices (pick one primary) . . . 435
Pattern A — Single CE, two eBGP uplinks (simpler) . . . 435
Pattern B — Dual CE + VRRP (more realistic campus edge) . . . 435
Addressing plan . . . 436
Topology YAML . . . 436
FRR daemons . . . 437
CE1 config sketch . . . 438
CE2 config differences . . . 439
PE1 / PE2 sketch . . . 439

Deploy and baseline verify . . . 440
Predict (baseline) . . . 440
Drill 1 — Primary PE session loss . . . 441
Predict → observe → fix . . . 441
Drill 2 — Force VRRP failover . . . 441
Predict . . . 441
Drill 3 — Policy rejection . . . 442
Predict . . . 442
Drill 4 — Name/time smoke (hygiene) . . . 442
verify.sh . . . 442
Definition of done (this lab) . . . 442
Optional SR Linux PE . . . 443
Common mistakes . . . 443
Summary . . . 443

DNS Authoritative and Recursive **444**

DNS Authoritative and Recursive **445**
Learning goals . . . 445
Why roles matter . . . 445
Concepts — recursive vs authoritative . . . 446
Authoritative . . . 446
Recursive . . . 446
Stub . . . 446
Concepts — SOA, NS, glue . . . 447
Concepts — caching . . . 447
Concepts — DNSSEC at operator level . . . 448
Addressing plan (DNS roles lab) . . . 448
Topology YAML . . . 448
Authoritative zone (CoreDNS sketch) . . . 449
Direct auth checks (no recursion) . . . 450
Recursive (Unbound) sketch . . . 450
dig verification workflows . . . 451
1. Happy path via recursive . . . 451
2. Prove auth is the source of truth . . . 451
3. Trace mindset (public names) . . . 451
4. Glue / NS consistency . . . 451
5. DNSSEC operator glance (public) . . . 452
Predict → observe → fix . . . 452
Scenario A — client NXDOMAIN for `www.lab.example` . . . 452
Scenario B — auth works, recursive SERVFAIL . . . 452
Scenario C — “update not visible” . . . 452
Scenario D — AA vs AD confusion . . . 452
Verification checklist . . . 453
Common mistakes . . . 453
Summary . . . 453

QUIC Version 2 and Compatible Version Negotiation **454**

QUIC Version 2 and Compatible Version Negotiation **455**
Learning goals . . . 455
Concepts — why a "version 2" that does nothing new? . . . 455
Concepts — Compatible VN vs Incompatible VN . . . 456
Version Information (transport parameter 0x11) . . . 457
Packet-version rules after a Compatible switch (v1 v2) . . . 457
When v2 matters vs when it does not . . . 458
Worked observation — Firefox and Compatible VN . . . 458
Try this (browser) . . . 459
Mental model — one diagram . . . 459
The trap . . . 459
The boring rule . . . 460
Sources . . . 460

Happy Eyeballs: Racing IPv6 and IPv4 (RFC 8305 and the v3 Draft) **461**

Happy Eyeballs: Racing IPv6 and IPv4 (RFC 8305 and the v3 Draft) **462**
Learning goals . . . 462
Concepts in one page . . . 462
Build the lab . . . 463
Drill 1: three kinds of AAAA, one client that races . . . 463
Drill 2: the same name, a client that does not race . . . 464
Drill 3: read the race on the wire . . . 465
What version 3 changes . . . 465
The trap . . . 466
The boring rule . . . 466
Try this . . . 467
Sources . . . 467

QUIC Load Balancing with nftables: Routing on Connection IDs **468**

QUIC Load Balancing with nftables: Routing on Connection IDs **469**
Learning goals . . . 469
Mental model . . . 469
The CID layout . . . 470
Build the lab . . . 471
Why not paste the published ruleset? . . . 477
Drill 1: handshakes stick to a client address . . . 479
Drill 2: established packets follow the server ID . . . 479
Drill 3: two connections, one 4-tuple, and why zones exist . . . 480
Drill 4: real QUIC, including migration . . . 481
Drill 5: drain be3, add be4, deregister be3, reset its clients . . . 482
The trap . . . 483
Notes and pitfalls . . . 485
The boring rule . . . 485
Try this . . . 485
Sources . . . 485

HTTP/3 Stream Cancellation: STOP_SENDING, RESET_STREAM, and 499 **487**

HTTP/3 Stream Cancellation: STOP_SENDING, RESET_STREAM, and 499 **488**
Learning goals . . . 488
Mental model . . . 488
Concepts — two halves of a QUIC stream . . . 490
Concepts — which HTTP/3 error code . . . 491
Case — what a proxy must do . . . 492
Case — cancel-safe origins . . . 493
Lab — cancel mid-response on localhost . . . 493
The trap . . . 502
The boring rule . . . 502
Try this . . . 502
Sources . . . 503

Inside the TLS 1.3 Key Schedule: HKDF, Three Secrets, and Every Key in a Handshake **504**

Inside the TLS 1.3 Key Schedule: HKDF, Three Secrets, and Every Key in a Handshake **505**
Learning goals . . . 505
Mental model . . . 505
Concepts . . . 507
HKDF in two functions . . . 507
The two TLS wrappers . . . 507
Leaf keys, nonces and Finished . . . 508
Lab 1: HKDF from the standard library . . . 508
Lab 2: the whole schedule against RFC 8448 . . . 509
Lab 3: open RFC 8448's records with those keys . . . 513
Lab 4: a live OpenSSL 3.5 connection, through its key log . . . 515
Lab 5: one-line bugs and where they surface . . . 518
Case: what the proposed XOF key schedule would change . . . 519
The trap . . . 520
The boring rule . . . 520
Try this . . . 520
Next . . . 521
Verification . . . 521
Sources . . . 521

The 2026 Root KSK Rollover: Trust Anchors, RFC 5011, and What Breaks When Keys Are Hard-Coded **523**

The 2026 Root KSK Rollover: Trust Anchors, RFC 5011, and What Breaks When Keys Are Hard-Coded **524**
Learning goals . . . 524
Mental model . . . 524
Concepts . . . 525
Key tags and flags . . . 525
Trust anchors in the wild . . . 525
RFC 5011 in one paragraph . . . 526
Lab 1: the root keys right now, against IANA's anchors . . . 526
Lab 2: what each trust-anchor configuration sees today . . . 528

Lab 3 (operator checklist, from vendor docs; not run here) . . . 529
The trap . . . 529
The boring rule . . . 530
Try this . . . 530
Next . . . 530
Verification . . . 530
Sources . . . 531

TLS 1.3 0-RTT: Replaying Early Data, and What Actually Stops It **532**

TLS 1.3 0-RTT: Replaying Early Data, and What Actually Stops It **533**
Learning goals . . . 533
Mental model . . . 534
Lab: record one 0-RTT request, replay it . . . 534
Where the protection ends . . . 536
HTTP's answer: `Early-Data: 1` and `425 Too Early` (not run here) . . . 537
The boring rule . . . 537
Try this . . . 537
Sources . . . 537

Encrypting IP Addresses in Logs with IPCrypt **539**

Encrypting IP Addresses in Logs with IPCrypt **540**
Learning goals . . . 540
Why not just hash or truncate? . . . 541
The modes in one table . . . 541
Lab 1: implement it and check the test vectors . . . 541
Lab 2: anonymise a log, then ask analyst questions . . . 544
Pitfalls the lab shows . . . 547
The boring rule . . . 547
Try this . . . 547
Sources . . . 547

Policy, QoS, and Hardening **549**

ACL Thinking **550**

ACL Thinking **551**
Learning goals . . . 551
Concepts . . . 551
Three different "filters" people mix up . . . 551
Policy statement template . . . 551
First-match vs best-match . . . 552
Stateful vs stateless . . . 552
Direction and attachment . . . 552
Written policy for the lab . . . 552
Topology YAML . . . 553
Addressing . . . 554

Implement with iptables (readable teaching form) . . . 554
nftables sketch (same policy) . . . 555
Predict → observe → fix drills . . . 556
Drill 1 — Allowed path . . . 556
Drill 2 — Denied path . . . 556
Drill 3 — SSH source restriction . . . 556
Drill 4 — Asymmetric paths . . . 556
Drill 5 — Wrong plane . . . 557
FRR: route policy is still "ACL thinking" . . . 557
Object-group idea (vendor-agnostic) . . . 557
Change control for ACLs . . . 557
Verification checklist . . . 558
Common mistakes . . . 558
Summary . . . 558

Control-Plane Protection **559**

Control-Plane Protection **560**
Learning goals . . . 560
Concepts . . . 560
Two paths through a router . . . 560
Why unprotected CPUs melt . . . 561
Policy matrix (start simple) . . . 561
Hardening adjacent controls . . . 561
Lab topology . . . 562
Addressing . . . 563
Baseline routing (static snack) . . . 563
Linux receive-path protection (teaching CoPP) . . . 563
Predict → observe → fix . . . 564
Drill 1 — Transit still works under "attack" to RE . . . 564
Drill 2 — SSH from non-mgmt . . . 565
Drill 3 — Over-tight BGP protection . . . 565
Drill 4 — Peer-only OSPF . . . 565
FRR-side levers (awareness) . . . 565
Building a CoPP document . . . 565
IPv6 notes . . . 566
Optional SR Linux / NOS CoPP . . . 566
Verification checklist . . . 566
Common mistakes . . . 566
Summary . . . 567

QoS Vocabulary **568**

QoS Vocabulary **569**
Learning goals . . . 569
Concepts — the pipeline . . . 569
Vocabulary table . . . 570
DSCP / PHB (operator view) . . . 570

Policing vs shaping . . . 571
Linux lab topology . . . 571
Addressing . . . 571
Experiment 1 — bottleneck without QoS . . . 572
Predict . . . 572
Observe . . . 572
Experiment 2 — two classes with HTB . . . 572
Predict . . . 573
Observe . . . 573
Fix . . . 573
Experiment 3 — police vs shape feel . . . 573
Marking sketch (DSCP) . . . 573
Predict . . . 574
Observe . . . 574
What lab QoS cannot prove . . . 574
FRR / NOS note . . . 574
Design mini-checklist . . . 574
Predict → observe → fix template . . . 575
Common mistakes . . . 575
Summary . . . 575

Hardening Checklists **576**

Hardening Checklists **577**
Learning goals . . . 577
How to use these lists . . . 577
Checklist 0 — Lab hygiene (every topology) . . . 577
Checklist 1 — Management plane . . . 578
Linux probes . . . 578
Checklist 2 — Control plane & routing . . . 578
FRR probes . . . 579
Checklist 3 — Data plane filters . . . 579
Checklist 4 — Edge services . . . 579
Checklist 5 — Secrets, users, supply chain . . . 580
Checklist 6 — Resilience & abuse . . . 580
Role profiles (quick) . . . 580
Lab host (Containerlab hypervisor) . . . 580
FRR edge CE . . . 580
FRR P / fabric leaf . . . 581
Linux bridge “switch” . . . 581
Exception register template . . . 581
Automating gates — `verify-hard.sh` . . . 581
Predict → observe → fix (meta-drill) . . . 582
Mapping to prior chapters . . . 582
Optional SR Linux . . . 582
Definition of done — hardened lab . . . 583
Common mistakes . . . 583
Summary . . . 583

SYN Floods and Listen Queues: SYN_RECV, SYN Cookies, and Accept Overflow 584

SYN Floods and Listen Queues: SYN_RECV, SYN Cookies, and Accept Overflow 585
Learning goals . . . 585
Mental model . . . 586
Concepts — what a SYN cookie is . . . 587
Lab — setup: a private network namespace with a void . . . 587
Lab — accept queue: a server that never calls `accept()` . . . 589
Lab — `somaxconn` silently caps the backlog . . . 591
Lab — a spoofed SYN flood, three ways . . . 591
Lab — what a cookie connection keeps . . . 594
Case — reading a production listener . . . 595
The trap . . . 596
The boring rule . . . 596
Try this . . . 597
Verification . . . 597
Sources . . . 597

FQ-CoDel and CAKE: How Linux Fights Bufferbloat, and How to Check It Is Actually On 599

FQ-CoDel and CAKE: How Linux Fights Bufferbloat, and How to Check It Is Actually On 600
Learning goals . . . 600
Mental model . . . 600
Concepts . . . 601
CoDel in four lines (RFC 8289) . . . 601
FQ-CoDel (RFC 8290) . . . 601
CAKE (tc-cake(8)) . . . 602
Who is on by default . . . 602
Lab 1: what this host actually runs . . . 602
Lab 2: the algorithms, simulated . . . 603
Lab 3: configuring it (from the man pages; not run here) . . . 606
The trap . . . 607
The boring rule . . . 607
Try this . . . 608
Next . . . 608
Verification . . . 608
Sources . . . 608

Overlays and Tunnels 610

Why Encapsulate 611

Why Encapsulate 612
Learning goals . . . 612
The core picture . . . 612
Why people encapsulate . . . 613
Why *not* to encapsulate . . . 613
Encapsulation as Russian dolls . . . 614

Identifiers you must name . . . 614
Failure domains . . . 615
MTU tax . . . 615
ECMP and entropy . . . 616
Control plane vs data plane (again) . . . 616
Mini lab — see encapsulation with tcpdump . . . 616
GRE between a and b across under . . . 617
Predict → observe → fix . . . 617
Design questions checklist . . . 617
Mapping to the journey map . . . 618
Common mistakes . . . 618
Summary . . . 618

Site Tunnels **619**

Site Tunnels **620**
Learning goals . . . 620
When site tunnels are the right tool . . . 620
Topology — two sites + "Internet" . . . 621
Addressing plan . . . 621
Lab 1 — GRE + static routes . . . 622
Predict . . . 623
Observe . . . 623
Fix . . . 623
Lab 2 — MTU blackhole drill . . . 623
Predict . . . 623
Lab 3 — Underlay failure . . . 624
Predict . . . 624
Route-based vs policy-based VPN . . . 624
Lab 4 — WireGuard route-based (if packages available) . . . 624
Predict → observe . . . 625
FRR over GRE (dynamic optional) . . . 625
Policy-based IPsec awareness . . . 625
Security notes (vendor-agnostic) . . . 626
verify.sh sketch . . . 626
Common mistakes . . . 626
Summary . . . 626

WireGuard: Cryptokey Routing and the Pre-Shared Key **627**

WireGuard: Cryptokey Routing and the Pre-Shared Key **628**
Learning goals . . . 628
Concepts in one page . . . 628
Lab build . . . 629
Drill 1: cryptokey routing drops a perfectly encrypted packet . . . 631
Drill 2: a PSK mismatch, seen from the wire . . . 632
Drill 3: fix and rotate the PSK live . . . 633
Deploying it for real . . . 633

Verification checklist . . . 634
Common mistakes . . . 634
Summary . . . 634
Sources . . . 635

OpenVPN Data Channel Offload (DCO) 636

OpenVPN Data Channel Offload (DCO) 637
Thesis . . . 637
Mental model . . . 637
Worked examples . . . 638
Case 1: Confirm the kernel side before you touch configs . . . 638
Case 2: Let OpenVPN auto-enable DCO (and force it off for A/B) . . . 639
Case 3: One-sided DCO still helps . . . 640
Case 4: Routing mental shift under DCO . . . 640
The trap . . . 640
The boring rule . . . 641
Try this . . . 641
Sources . . . 641

WireGuard Multi-Tunnel: AllowedIPs, Policy Routing, Kill Switch 642

WireGuard Multi-Tunnel: AllowedIPs, Policy Routing, Kill Switch 643
Learning goals . . . 643
Concepts in one page . . . 643
Worked examples . . . 644
Case 1: Two tunnels, one default — the collision . . . 644
Case 2: `fwmark` + policy routing (single full tunnel) . . . 644
Case 3: Multi-tunnel with distinct marks . . . 645
Case 4: Kill switch = firewall, not WireGuard . . . 646
Case 5: Namespace alternative for "all traffic through one tunnel" . . . 646
The trap . . . 646
The boring rule . . . 647
Try this . . . 647
Sources . . . 647

WireGuard and MTU: Where the 1420 Comes From, Padding, and Silent Fragmentation 648

WireGuard and MTU: Where the 1420 Comes From, Padding, and Silent Fragmentation 649
Learning goals . . . 649
Where 1420 comes from . . . 650
Lab: a real tunnel between two namespaces . . . 650
Reading the numbers . . . 652
When fragmentation stops saving you (not run here) . . . 653
The boring rule . . . 653
Try this . . . 653
Sources . . . 654

VXLAN Data Plane 655

VXLAN Data Plane **656**
Learning goals . . . 656
Concepts . . . 656
Roles . . . 656
Header (mental) . . . 657
Data-plane only learning . . . 657
Multicast vs head-end replication . . . 657
When VXLAN is appropriate . . . 657
Topology — two VTEPs + underlay + hosts . . . 657
Addressing . . . 658
VXLAN + bridge on leaf1 . . . 659
Deploy checks . . . 660
Predict . . . 660
Observe . . . 660
Fix . . . 660
Drill — MTU . . . 661
Predict . . . 661
Drill — wrong VNI isolation . . . 661
Predict . . . 661
Drill — MAC move / learning . . . 661
Predict . . . 661
Flood scope risk . . . 662
FRR / SR Linux awareness . . . 662
Verification checklist . . . 662
Common mistakes . . . 662
Summary . . . 662

EVPN Control-Plane Intro **664**

EVPN Control-Plane Intro **665**
Learning goals . . . 665
Why EVPN exists . . . 665
Objects (vendor-agnostic) . . . 666
Control vs data verification split . . . 666
Minimal design story — 2 leaf . . . 666
FRR capability note (mid-2026) . . . 667
Topology sketch (EVPN lab) . . . 667
RR config idea . . . 668
Predict → observe → fix drills . . . 669
Drill 1 — Session before service . . . 669
Drill 2 — Host MAC appears as EVPN route . . . 669
Drill 3 — Kill EVPN, keep underlay . . . 669
Drill 4 — RT mismatch (concept) . . . 669
L2 VNI vs L3 VNI (awareness) . . . 669
Optional SR Linux community path . . . 670
Security / isolation checklist . . . 670
What this intro deliberately skips . . . 670
Common mistakes . . . 671

Summary . . . 671

Overlay Lab **672**

Overlay Lab **673**
Learning goals . . . 673
Pick a track . . . 673
Definition of done (all tracks) . . . 673
Track A — Dual-site GRE (reference build) . . . 674
Success metrics . . . 674
verify.sh . . . 674
Failure matrix A . . . 675
Track B — VXLAN two-leaf (reference build) . . . 675
Success metrics . . . 675
bringup excerpt . . . 675
verify.sh B . . . 675
Failure matrix B . . . 676
Capture helper . . . 676
Track C — EVPN intro (stretch) . . . 676
Option 1: Netlab EVPN-VXLAN Topology (Recommended) . . . 676
Option 2: Manual FRR Configuration . . . 677
Success metrics . . . 677
Failure matrix C . . . 678
Integrated “mini fabric snack” (optional merge) . . . 678
Observability expectations . . . 678
Hardening lite (overlay) . . . 678
Lab report template . . . 679
Predict → observe → fix (capstone habit) . . . 679
Time-box guidance . . . 679
Common mistakes . . . 679
Summary . . . 680

Head Office and Branch Office **681**

Head Office and Branch Office **682**
Learning goals . . . 682
Why this shape . . . 682
Roles . . . 683
Topology . . . 683
Addressing . . . 686
Install and address the lab Internet . . . 686
Start Headscale and join both offices . . . 687
Predict . . . 688
VXLAN on the tailnet . . . 689
Prove the offices share a LAN . . . 690
Predict . . . 690
Observe . . . 690

Drill — MTU . . . 690
Predict . . . 690
Drill — wrong VNI . . . 691
Predict . . . 691
A third office . . . 691
Offices that should stay two networks . . . 691
Same exercise on a tailnet you already run . . . 692
Failure table . . . 692
Tear down . . . 693
Checklist . . . 693
Summary . . . 693

Fabrics and Multi-Area **694**

Clos Leaf-Spine **695**

Clos Leaf-Spine **696**
Learning goals . . . 696
Why Clos-style fabrics . . . 696
Roles . . . 697
Oversubscription (awareness) . . . 697
Underlay protocol choices . . . 698
Addressing plan (2×2) . . . 698
Topology YAML . . . 698
FRR daemons . . . 700
OSPF underlay sketch (l1) . . . 700
eBGP underlay sketch (optional alternate) . . . 701
Deploy and ECMP verify . . . 701
Predict . . . 701
Observe multipath . . . 701
Drill 1 — Spine loss . . . 702
Predict → observe → fix . . . 702
Drill 2 — One leaf uplink down . . . 702
Predict . . . 702
Drill 3 — Polarization awareness . . . 702
East-west vs north-south . . . 703
Overlay placement (preview) . . . 703
Verification checklist . . . 703
Common mistakes . . . 703
Summary . . . 704

Multi-Area OSPF **705**

Multi-Area OSPF **706**
Learning goals . . . 706
Concepts . . . 706
Why areas . . . 706
Backbone rule (practical) . . . 706

Roles . . . 707
LSA awareness (not flash cards) . . . 707
Summarization . . . 707
Topology . . . 707
Topology YAML . . . 708
FRR configs . . . 709
r1 (area 1) . . . 709
r3 (area 2) . . . 709
r2 (ABR) . . . 709
Deploy and baseline . . . 710
Predict . . . 710
Observe . . . 710
Drill 1 — Summarization blackhole . . . 711
Predict . . . 711
Fix . . . 711
Drill 2 — ABR failure . . . 711
Predict . . . 711
Drill 3 — Area mismatch . . . 712
Predict . . . 712
Stub area ideas (awareness) . . . 712
Multi-area vs fabric BGP . . . 712
Verification checklist . . . 713
Common mistakes . . . 713
Summary . . . 713

Fabric with FRR **714**

Fabric with FRR **715**
Learning goals . . . 715
Lab repo layout . . . 715
Design defaults for this book . . . 716
eBGP unnumbered (modern fabric style) . . . 716
Numbered eBGP fabric (reliable teaching) . . . 716
Spine s1 excerpt . . . 717
Leaf l1 excerpt . . . 717
OSPF alternate (same wiring) . . . 718
Declarative Fabric with Netlab (Fast & Scalable) . . . 718
Deploy and Verify with Netlab . . . 719
Manual Containerlab & FRR Implementation . . . 719
Containerlab topology (`clos22.clab.yml`) . . . 720
Operational show kit . . . 720
verify.sh . . . 720
Failure scripts . . . 721
Predict → observe → fix . . . 721
Policy on fabric (light) . . . 721
Adding overlay later (hook points) . . . 721
Scale checklist 2×2 → 4×4 . . . 722
Observability . . . 722

Optional SONiC / SR Linux leaf . . . 722
Common mistakes . . . 722
Summary . . . 723

Multi-Implementation Lab **724**

Multi-Implementation Lab **725**
Learning goals . . . 725
Design principle . . . 725
Recommended topology . . . 726
Addressing / ASN (fixed) . . . 726
Topology YAML (FRR + optional srl) . . . 726
FRR side . . . 727
SR Linux mapping (conceptual) . . . 728
Verification questions (dialect-free) . . . 728
FRR answers . . . 728
SR Linux answers (illustrative) . . . 729
predict → observe → fix matrix . . . 729
verify.sh (heterogeneous) . . . 729
Overlay optional add-on . . . 729
Hardening gates (reuse part 08) . . . 730
Lab report template . . . 730
When interop hurts . . . 730
Common mistakes . . . 731
Summary . . . 731

BGP Security 732

Why Routing Security Is Harder Than It Looks **733**

Why Routing Security Is Harder Than It Looks **734**
The Architectural Paradox . . . 734
Why Cryptographic Patches Fall Short . . . 735
Economic Incentives and Asymmetry . . . 735
Roadmap of This Deep Dive . . . 736

The Architecture of Global Connectivity **737**

The Architecture of Global Connectivity **738**
IP Addressing and the Prefix Hierarchy . . . 738
Autonomous Systems and Administrative Boundaries . . . 738
The Transit-Peering Model and Its Economics . . . 739
1. Transit (Customer-Provider) . . . 739
2. Peering (Settlement-Free or Paid) . . . 739
Delegation: How Addresses and ASNs Become Someone's Responsibility . . . 740
Scalability Through Hierarchical Aggregation . . . 741

Interdomain Routing Fundamentals **742**

Interdomain Routing Fundamentals **743**
What Interdomain Routing Must Achieve . . . 743
Direct Connectivity: iBGP and eBGP Sessions . . . 743
Route Advertisements and Withdrawals in Practice . . . 744
Advertisements (UPDATE with NLRI) . . . 744
Withdrawals (UPDATE with Withdrawn Routes) . . . 745
Implicit Withdrawals . . . 745
Import and Export Policies: The Operator's Control Point . . . 745
Propagation Through the Transit Web . . . 746

BGP Protocol Architecture **747**

BGP Protocol Architecture **748**
The BGP Message Types: OPEN, UPDATE, NOTIFICATION, KEEPALIVE . . . 748
The BGP Finite State Machine: From Idle to Established . . . 749
TCP Port 179 and Transport Considerations . . . 750
BGP-4 UPDATE Message Structure and TLV Encoding . . . 751
Capabilities Negotiation and Modern Extensions . . . 751

BGP Path Attributes and Policy Expression **752**

BGP Path Attributes and Policy Expression **753**
Attribute Categories . . . 753
AS_PATH: Loop Detection and Policy Signal . . . 754
1. Loop Prevention . . . 754
2. Traffic Engineering (AS-Path Prepending) . . . 754
NEXT_HOP, LOCAL_PREF, MED and ORIGIN: Comparative Selection Logic . . . 754
BGP Communities and Extended Communities: Encoding Policy at Scale . . . 755
Well-Known Standard Communities . . . 755
Extended (RFC 4360) and Large Communities (RFC 8092) . . . 756
ADD-PATH, GTSM and Modern Security Attributes . . . 756

The BGP Decision Process and Route Selection **757**

The BGP Decision Process and Route Selection **758**
The Step-by-Step BGP Decision Process . . . 758
Weight, LOCAL_PREF and Administrative Distance . . . 760
AS_PATH Length, ORIGIN and MED Comparisons . . . 760
Route Preference and IGP Influence: eBGP vs iBGP vs Static vs IGP . . . 760
Multi-Exit Discrimination, Tie-Breakers and Stability . . . 761

Convergence, Stability and Propagation Dynamics **762**

Convergence, Stability and Propagation Dynamics **763**
Convergence Time and Global Propagation Delays . . . 763
Route Flapping and Dampening (RFC 2439) . . . 763
Hot Potato Routing and Path Instability . . . 764
Slow Convergence as an Attack Surface . . . 764
Measurement Data on Convergence Behavior . . . 765

Historical Design Decisions and Their Security Consequences **766**

Historical Design Decisions and Their Security Consequences **767**
EGP, CGE and the Origins of Interdomain Routing Protocols . . . 767
The Scalability Imperative: Why End-to-End Verification Was Abandoned . . . 767
The Trust Assumptions Embedded in BGP-4's Design . . . 768
Operational Flexibility Over Consistency: A Deliberate Choice . . . 768
Why Security Was Added as an Afterthought . . . 768

The Threat Landscape: Accidents and Intentional Attacks **770**

The Threat Landscape: Accidents and Intentional Attacks **771**
Accidental Misconfiguration: The Most Common Disruption . . . 771
Origin Prefix Hijacking and Impersonation . . . 771
Subprefix Hijacking and Splitting . . . 772
Route Leaks: Transit Rules Violated . . . 772
Blackholing and Traffic Interception . . . 773
AS_PATH Manipulation and Policy Exploitation . . . 773
Control-Plane Exhaustion and Volumetric Attacks . . . 773
Compromised Routers and Network Operators . . . 774

Origin Hijacking Attacks: Mechanics, Impact and Detection **775**

Origin Hijacking Attacks: Mechanics, Impact and Detection **776**
Attack Prerequisites and Execution Steps . . . 776
Step 1: Upstream Peering Access . . . 776
Step 2: Identification of Unfiltered Upstream . . . 776
Step 3: Prefix Injection . . . 777
Step 4: Upstream Propagation . . . 777
Complete vs. Partial Hijacking . . . 777
1. Complete Hijacking (Subprefix / More-Specific) . . . 777
2. Partial Hijacking (Exact-Match) . . . 777
Confidentiality and Integrity Implications . . . 777
How Hijacking Enables Phishing and MITM . . . 778
Detection via Route Monitors, Looking Glasses and Real-Time Alerts . . . 778
The Role of RPKI and ROV in Origin Hijacking Prevention . . . 779

Route Leaks and Policy Abuse **780**

Route Leaks and Policy Abuse **781**
What Is a Route Leak and How It Differs From Hijacking . . . 781
The Customer-Concentrator-Provider Leak Pattern . . . 781
Accidental vs. Intentional Policy Violations . . . 782
Propagation Patterns and Impact on Reachability . . . 782
Prevention Through Policy, Filtering and Community Tags . . . 782
1. Strict Outbound Prefix Filtering . . . 783
2. BGP Communities for Leak Prevention . . . 783
3. RFC 9234: Autonomous System Provider Authorization (ASPA) and Only to Customer (OTC) . . . 783

Detection via Monitoring and Analysis . . . 783

Advanced and Emerging Routing Threats 784

Advanced and Emerging Routing Threats 785
Compromised Routers and Rogue Operators . . . 785
Competitive Manipulation: Steering Traffic for Advantage . . . 785
Attacks on Route Collectors and Validation Infrastructure . . . 785
Long-Term Reconnaissance and Persistent Manipulation . . . 786
Threats Specific to Large Networks and Cloud Providers . . . 786

The Routing Registries: IRRs and Their Limitations 787

The Routing Registries: IRRs and Their Limitations 788
What Routing Registries Are and How They Store Route Objects . . . 788
Creating Route Objects: Route, Route6, AS-SET and Related Classes . . . 788
1. `route` (IPv4) and `route6` (IPv6) Objects . . . 788
2. `aut-num` Objects . . . 789
3. `as-set` Objects . . . 789
Policy-Based Filtering: Using IRR Data to Build Filters . . . 789
Weaknesses: No Authorization Checking, No Cryptographic Binding . . . 790
The Transition from IRR Filtering to RPKI-Based Validation . . . 790

The RPKI Architecture: Resource Certificates and Trust Anchors 791

The RPKI Architecture: Resource Certificates and Trust Anchors 792
The RPKI Trust Model: Hierarchical Certification from RIRs . . . 792
Trust Anchors and Root Certificates at Each RIR . . . 793
Resource Certificates: Structure, Extensions and Resource Holders . . . 793
CA Certificates and Chain Validation . . . 793
Manifests and Signed Object Lists: What They Protect . . . 794

Route Origin Authorizations: The Core Authorization Object 795

Route Origin Authorizations: The Core Authorization Object 796
What a ROA Is and What It Proves . . . 796
Creating a ROA: Prefix, AS Number and Maximum Length . . . 796
Max-Length: How It Controls Subprefix Authorization . . . 797
The `maxLength` Security Vulnerability (RFC 9319) . . . 797
ROA Lifespan, Expiration and Revocation . . . 797
Common Mistakes: Over-Permissive, Under-Covered and Conflicting ROAs . . . 798

RPKI Validation: Validators, VRPs and Relying Parties 799

RPKI Validation: Validators, VRPs and Relying Parties 800
What an RPKI Validator Does: Download, Verify, Extract . . . 800
Validator Architecture and Repository Sync: RRDP vs. rsync . . . 801
Validated ROA Payloads (VRPs): The Validator's Output . . . 801
CRL and SKI/IAI: Revocation and Validation Accuracy . . . 802
Validator Outages, Stale Data and Recovery Behavior . . . 802

RPKI-to-Router (RTR) and Route Origin Validation (ROV) 803

RPKI-to-Router (RTR) and Route Origin Validation (ROV) 804
The RPKI-to-Router Protocol: Messages and Mechanics . . . 804
Session Establishment, Keepalive and Data Transfer . . . 805
How VRPs Become Route Decision Criteria: Route Origin Validation (ROV) . . . 806
ROV Integration with the BGP Decision Process . . . 807
Handling State Transitions: Valid, Invalid, NotFound and Pending . . . 807

Operational Consequences of ROV: Invalid, Valid, NotFound and Real-World Effects 808

Operational Consequences of ROV: Invalid, Valid, NotFound and Real-World Effects 809
Rejecting Invalid Routes vs. De-Prioritizing Them . . . 809
Option A: Hard Dropping (Drop Invalids) . . . 809
Option B: Soft De-prioritizing (Deprecate Invalids) . . . 809
The Risk of Rejecting Legitimate Traffic . . . 810
NotFound: The Ambiguous Middle State . . . 810
Max-Length Policy Errors and Collateral Damage . . . 810
Documented Incidents: When ROV Went Wrong . . . 811
1. The Cloudflare / AWS Route 53 Incident (2018) . . . 811
2. The Orange España RPKI Account Takeover (January 2024) . . . 811

The Scope and Limits of RPKI and ROV 812

The Scope and Limits of RPKI and ROV 813
What Origin Validation Proves and What It Does Not . . . 813
Why RPKI Cannot Prevent All Route Leaks . . . 813
The Dependency on Accurate, Complete ROA Data . . . 814
Cryptographic Soundness vs. Operational Fragility . . . 814
Misconceptions About What RPKI "Solves" . . . 814

Complementary Mechanisms: IRR Filtering, Prefix Filters and AS_PATH Filters 815

Complementary Mechanisms: IRR Filtering, Prefix Filters and AS_PATH Filters 816
Prefix Filters: What Prefixes Should Be Announced . . . 816
1. Bogon and Special-Purpose Filtering . . . 816
2. Prefix Length Boundaries . . . 816
AS_PATH Filters: Detecting and Rejecting Suspicious Paths . . . 816
IRR-Based Filtering: Still Relevant After RPKI . . . 817
Maximum-Prefix Limits: Protection Against Table Poisoning . . . 817
BGP Communities for Policy Signaling and Leak Prevention . . . 817

Route Leak Detection and Prevention Mechanisms 819

Route Leak Detection and Prevention Mechanisms 820
Community-Based Leak Prevention and RFC 9234 . . . 820
Automated Leak Detection via Route Collectors and Monitors . . . 821
BGPsec Route Leak Prevention Potential . . . 821
MANRS Principles and Their Implementation Status . . . 821

Coordination and Incident Response When Leaks Occur . . . 822

BGPsec: Architecture, Security Guarantees and Deployment Reality 823

BGPsec: Architecture, Security Guarantees and Deployment Reality 824
BGPsec's Threat Model: Protecting AS_PATH Integrity . . . 824
The BGPsec Path Attribute and Signature Chain . . . 824
Key Management and Certificate Infrastructure . . . 825
Operational Barriers: Complexity, Interoperability and Performance . . . 825
1. The Asymmetric Cryptographic Compute Penalty . . . 825
2. Message Size and MTU Fragmentation . . . 825
3. The All-or-Nothing Deployment Trap . . . 825
Current Deployment Status and Future Prospects . . . 826

ASPA: Autonomous System Provider Authorizations 827

ASPA: Autonomous System Provider Authorizations 828
What ASPA Is and the Problem It Solves . . . 828
ASPA Structure: Customer, Provider and Peer Relationships . . . 828
Integration with the RPKI Trust Infrastructure . . . 829
How ASPA Validation Affects Route Selection . . . 829
Current Deployment Status and Standardization Progress . . . 830

Designing and Implementing Routing Security in Your Network 831

Designing and Implementing Routing Security in Your Network 832
Step 1: Inventory and Audit Your Address Space and Announcements . . . 832
Step 2: Verify Ownership and Create or Update ROAs . . . 833
Step 3: Design and Deploy Import Filters . . . 833
Step 4: Design and Deploy Export Filters . . . 833
Step 5: Enable ROV with Staged Enforcement . . . 833
Stage A: Validation Without Enforcement (Telemetry Mode) . . . 833
Stage B: Hard Drop Enforcement . . . 834
Step 6: Implement Monitoring, Logging and Alerting . . . 834
Step 7: Document Policies, Plan Rollback and Test Recovery . . . 834

Configuration Examples and Vendor-Specific Considerations 835

Configuration Examples and Vendor-Specific Considerations 836
Cisco IOS XR: ROV, Filtering and RTR Configuration . . . 836
Juniper Junos: ROV, Filtering and RPKI Integration . . . 837
Nokia SR OS: Routing Security Configuration . . . 838
Open Source and Research Platforms: FRR and BIRD . . . 839
FRRouting (FRR) . . . 839
BIRD 2 (Internet Routing Daemon) . . . 839
Common Pitfalls Across Platforms . . . 840

Troubleshooting Routing Security Controls 841

Troubleshooting Routing Security Controls **842**
Unexpected Invalid Routes: Systematic Diagnosis . . . 842
RTR Session Failures and Stale Validation Data . . . 843
ROA Lookup: Verifying What Validators See . . . 843
Legitimate Routes Rejected by Filters . . . 844
Cross-Validator Consistency Checks . . . 844
Post-Incident Forensics: Reconstructing What Happened . . . 844

Monitoring, Measurement and Testing **845**

Monitoring, Measurement and Testing **846**
Real-Time Route Monitoring: BGPStream, RIS-Live and RouteViews . . . 846
1. RIPE RIS and RIS-Live . . . 847
2. RouteViews . . . 847
3. BGPStream and BGPalerter . . . 847
Looking Glasses and Prefix Monitoring Services . . . 847
Safe Testing: Lab Environments and Controlled Announcements . . . 847
1. Containerlab Network Topologies . . . 847
2. The PEERING Testbed . . . 848
Public Incident Detection Systems and Alerts . . . 848
Metrics That Matter: Convergence, Stability and Coverage . . . 848

Case Studies: Documented Routing Incidents **849**

Case Studies: Documented Routing Incidents **850**
Pakistan Telecom YouTube Hijack (February 24, 2008) . . . 850
Safe Host / China Telecom Route Leak (June 6, 2019) . . . 851
Cloudflare 1.1.1.1 Incident (June 27, 2024) . . . 851
3ve BGP Hijacking Campaign (2018) . . . 852
Comparative Analysis: What Changed, What Remained Vulnerable . . . 852

Economics, Governance and the Human Dimension **853**

Economics, Governance and the Human Dimension **854**
Who Is Responsible for What: A Stakeholder Map . . . 854
Economic Incentives and Disincentives for Adoption . . . 855
The Role of RIRs, Regional Communities and IETF . . . 855
MANRS, IRTF and Industry Self-Regulation . . . 855
Why Technically Sound Mechanisms Fail to Deploy . . . 855

The Current State and Future of Internet Routing Security **857**

The Current State and Future of Internet Routing Security **858**
The Baseline: How Secure Is Global Routing Today? . . . 858
Deployment Gaps: Who Has Not Adopted and Why? . . . 858
Open Research Problems and Protocol-Level Unknowns . . . 859
Plausible Developments: ASPA, BGPsec Revival, or New Directions? . . . 859
A Grounded Assessment: Progress Made, Work Remaining . . . 860

Networking

Personal notes on learning **computer networking from first principles to expert NetOps**, with labs built as **code** in Containerlab using **open-source and free network images** only (FRR, Linux, optional SR Linux / SONiC / VyOS).

> **Note**
>
> This book is **vendor-agnostic**. CLIs differ; protocols do not. Start with the **lab platform** (macOS/Linux + Containerlab), then a deep **networking foundations** pass (TCP/IP, addressing, core protocols), then L2 → L3 → routing → edge → policy → overlays → fabrics → ops. Topology figures are illustrated SVGs under `images/`.

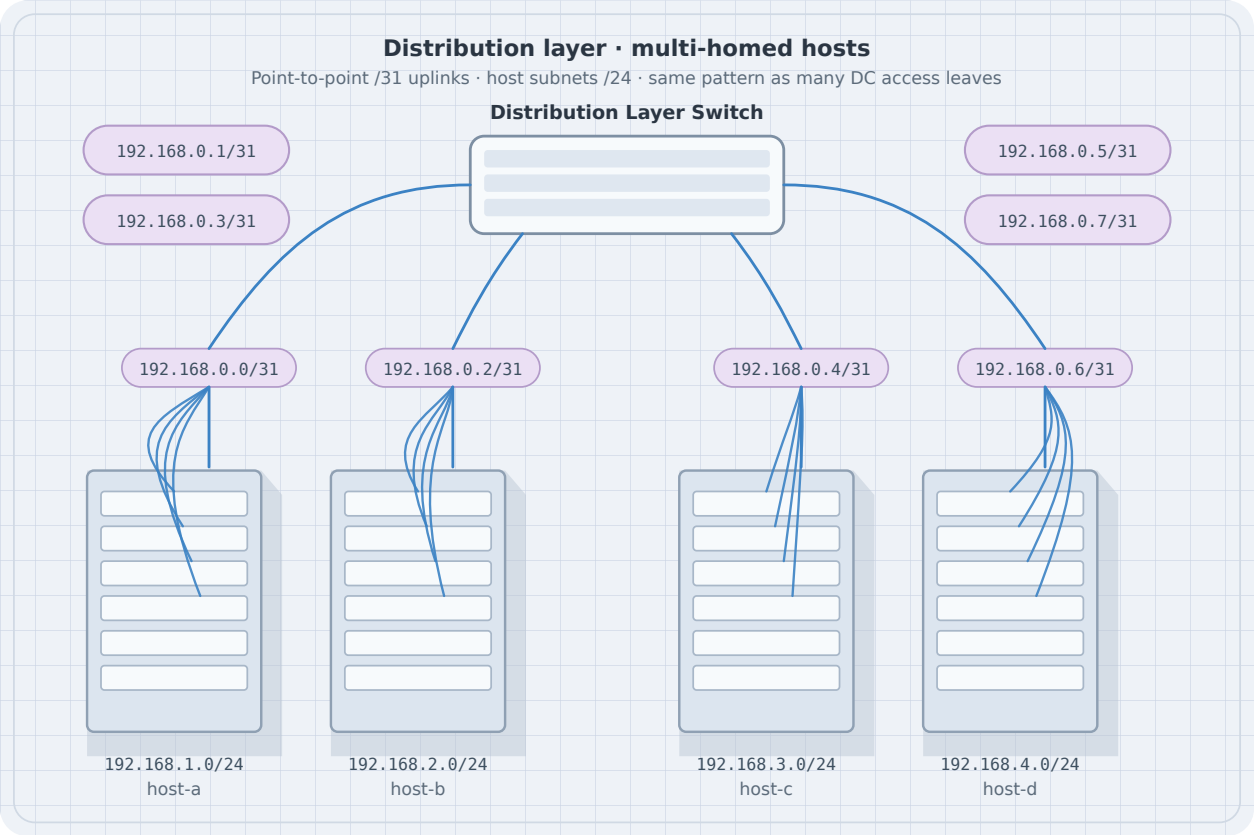


Figure 1: Distribution switch with multi-homed servers

Curriculum map

Part	Focus
01 Lab platform	macOS/Linux host setup, Containerlab, free images, host networking, hygiene
02 Networking foundations	TCP/IP stack, packets, Ethernet intro, IPv4/IPv6, ARP/ND, ICMP, TCP/UDP, common apps, host routing, planes
03 Layer 2	Ethernet/MAC deep, VLANs, loops, LAG, failure drills
04 Layer 3	Advanced addressing/routing labs, statics, ICMP/PMTU in multi-hop designs
05 Interior routing	OSPF, BGP, policy & filters
06 Ops & automation	Observability, lab-as-code, capstone outline
07 Edge & services	First-hop HA, DHCP/NAT in edge designs, hygiene, dual-home lab
08 Policy, QoS, hardening	ACLs, CoPP, QoS vocabulary, hardening
09 Overlays & tunnels	Encap, site tunnels, VXLAN, Tailscale underlay between offices, EVPN intro
10 Fabrics & multi-area	Clos, multi-area OSPF, FRR fabric, multi-NOS
11 BGP security	A deep dive into BGP, RPKI, routing attacks, and global Internet defense

Journey map

```
Lab platform (macOS / Linux + Containerlab)
  → Networking foundations (TCP/IP, addressing, protocols)
  → Layer 2 depth
  → Layer 3 / routing depth
  → Interior routing (OSPF, BGP)
  → Edge & services
  → Policy / QoS / hardening
  → Overlays & tunnels
  → Fabrics
  → Ops & automation
  → BGP security at production scale
```

What “done” looks like

- Design multi-site L2/L3 networks with clear underlay vs overlay roles
- Implement them in Containerlab with free images
- Prove behavior with tables, neighbors, and packet captures

- Break links and policies on purpose; restore and document
- Automate lab lifecycle

Lab ground rules

- Prefer **free/open** node kinds (Containerlab + FRR + Alpine/Debian)
- Every serious lab: topology file + addressing plan + failure scenario
- Capture evidence when it teaches something tables do not

Related books

Book	Overlap
Linux	Host networking commands (`ip`, `ss`, `tcpdump`, firewalls)
NixOS	Declarative network config on NixOS hosts
Go	`net/http`, probes, and load-test projects
VCS	Lab-as-code repos and CI for topology files

Formats

HTML, PDF, and EPUB via the monorepo portal when built.

Lab Platform

Lab Host — macOS

Lab Host — macOS

Prepare a **macOS** machine as the Containerlab control host: virtualization, container runtime, resources, and networking gotchas before you deploy topologies.

> **Note**
>
> **Stub chapter** — outline for the lab-platform on-ramp. Expand with exact install commands and verified versions as you freeze a lab baseline.

Goals

- Run Docker-compatible containers with enough CPU/RAM for FRR + Linux nodes
- Install Containerlab and pull free lab images
- Understand macOS-specific limits (VM layer, file sharing, port publishing)

Prerequisites

- Recent macOS with virtualization support (Apple Silicon or Intel)
- Admin rights for Docker/Colima and Homebrew
- Disk space for images (plan multi-GB for NOS images)

Virtualization and container runtime

Options

Approach	Notes
Docker Desktop	Simple GUI; license/terms apply for some orgs
Colima + Docker CLI	Lightweight VM; common for CLI-first labs
Lima / other	Possible; keep one path documented for this book

Checklist

1. Install runtime and confirm `docker version` / `docker info`
2. Allocate CPU and memory for multi-node labs (start modest; scale up)
3. Confirm architecture (`linux/arm64` vs `linux/amd64`) matches images you pull

Install Containerlab

- Prefer the Containerlab install docs for the current macOS method
- Verify: `containerlab version`
- Confirm the binary can talk to your Docker socket

Networking notes on macOS

- Lab nodes run **inside a Linux VM** — host `localhost` port maps differ from native Linux
- Bridge and macvlan behaviors are not identical to bare-metal Linux
- Prefer topologies that do not assume bare-metal SR-IOV or host netns tricks on day one

First smoke test

1. Deploy a **two-node** FRR or Linux topology from Containerlab examples
2. `containerlab inspect` / `docker ps` — nodes up
3. `docker exec` into a node; `ping` the peer
4. Destroy the lab; confirm no orphan containers

Next

- Parallel path: Lab Host — Linux
- Then Containerlab Fundamentals

Lab Host — Linux

Lab Host — Linux

Prepare a **Linux** machine (bare metal or VM) as the Containerlab control host: kernel/container runtime, permissions, and host networking.

> **Note**
>
> **Stub chapter** — outline for the lab-platform on-ramp. Expand with distro-specific package names as you freeze a baseline (Debian/Ubuntu recommended for docs).

Goals

- Install Docker or a Containerlab-supported runtime
- Install Containerlab and free images
- Avoid permission and firewall traps that break lab deploy

Prerequisites

- 64-bit Linux with virtualization if nested (for some images)
- Root or passwordless sudo for install steps
- Enough RAM/disk for your target topology scale

Container runtime

Typical path (Debian/Ubuntu-style)

1. Install Docker Engine (or Podman only if you know Containerlab support for your version)
2. Add your user to the `docker` group (or use root consistently in labs)
3. Enable and start the service; `docker info` succeeds

Kernel / modules

- Bridge, veth, and overlay-related features must work
- Nested virt: confirm if you need it for chosen NOS images

Install Containerlab

- Follow Containerlab install for Linux
- Verify: `containerlab version`
- Optional: bash completion

Host networking and firewall

- Local firewalls (ufw, firewalld, nftables) can block management or published ports
- Don't disable security blindly — open what the lab needs
- IP forwarding may be required for some designs later

First smoke test

1. Deploy a minimal two-node topology
2. Confirm links and management connectivity
3. Capture with `tcpdump` on a veth if useful
4. `containerlab destroy -t <file>` — clean teardown

Next

- macOS path: Lab Host — macOS
- Then Containerlab Fundamentals

Containerlab Fundamentals

Containerlab Fundamentals

Containerlab turns a YAML topology into runnable network labs: create nodes, wire virtual links, attach a management network, and destroy everything cleanly. This chapter is the operator's manual for the rest of the book's labs.

Learning goals

By the end of this chapter you can:

- Read and write a minimal `*.clab.yml` topology
- Deploy, inspect, connect, and destroy labs
- Choose free node **kinds** (FRR, Linux, etc.)
- Understand management network vs data-plane links
- Export a graph and use consistent naming
- Recover from partial deploys and leftover state

Prerequisites

- Linux host or VM with nested virt as needed (Docker or podman per Containerlab docs)
- `containerlab` installed (install guide)
- Permission to run privileged containers / sudo as required
- Images pullable from public registries for free kinds

Check:

```
containerlab version
docker version   # or podman
```

How Containerlab fits together

```
topology.clab.yml


 containerlab deploy

         create containers (nodes)
         create veth/links between interfaces
         attach management network (SSH/mgmt IPs)
         optional startup configs / binds
```

Destroy reverses the graph. **Source of truth is the YAML** (plus mounted configs), not hand edits inside nodes.

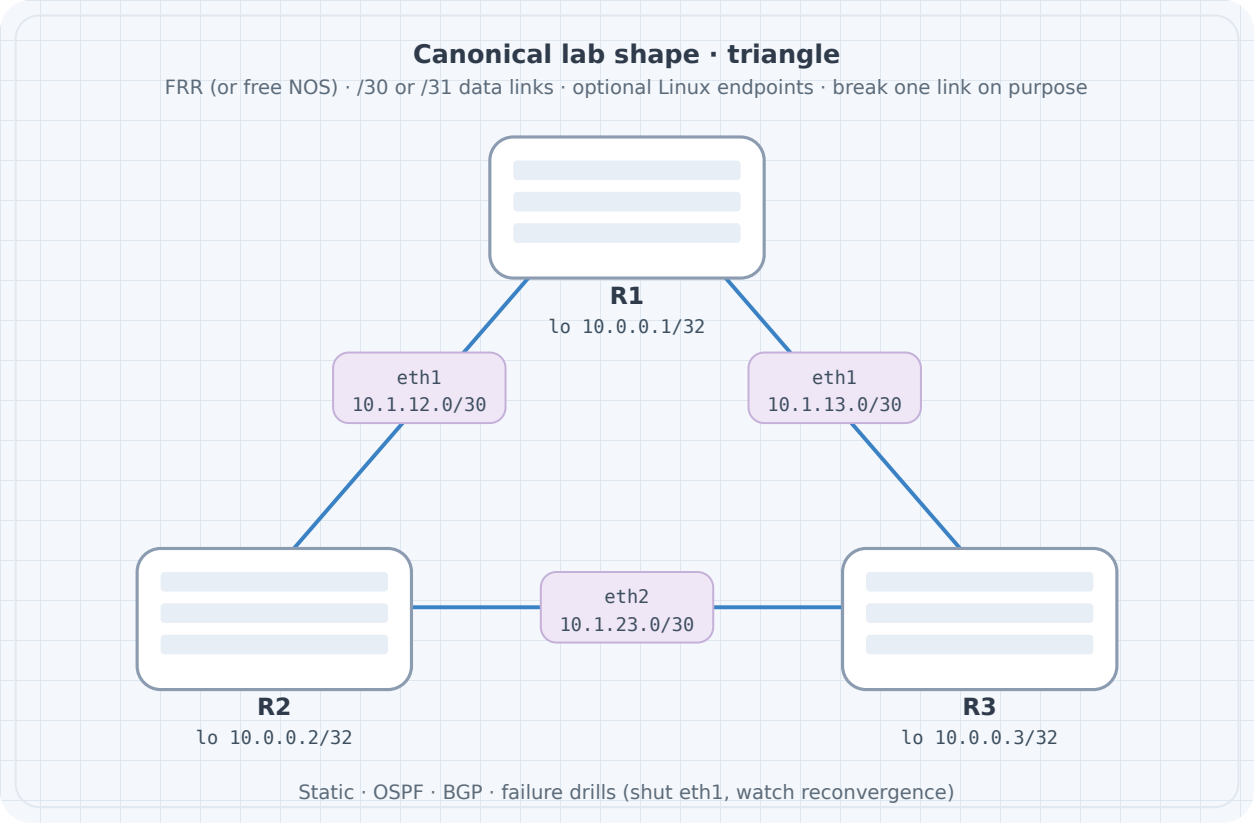


Figure 1: Triangle lab

Minimal topology: two Linux nodes

```
name: twoping

topology:
  nodes:
    h1:
      kind: linux
      image: alpine:3.20
    h2:
      kind: linux
      image: alpine:3.20

  links:
    - endpoints: ["h1:eth1", "h2:eth1"]
```

Save as `twoping.clab.yml`.

```
sudo containerlab deploy -t twoping.clab.yml
sudo containerlab inspect -t twoping.clab.yml
```

Address the link and test (interfaces may need addressing—Alpine is raw):

```
# Names often: clab-twoping-h1
docker exec -it clab-twoping-h1 sh
# inside:
ip addr add 10.0.0.1/24 dev eth1
ip link set eth1 up
# on h2:
# ip addr add 10.0.0.2/24 dev eth1 && ip link set eth1 up
# ping 10.0.0.1
```

For real labs you bake addressing into startup scripts or use NOS images that apply configs. Pattern: **deploy → configure via files → verify**.

Triangle of FRR routers (book default pattern)

FRR containers give you a real control plane without licenses.

```
name: triangle

topology:
  nodes:
    r1:
      kind: linux
```

```
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/r1:/etc/frr
    r2:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/r2:/etc/frr
    r3:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/r3:/etc/frr

  links:
    - endpoints: ["r1:eth1", "r2:eth1"]
    - endpoints: ["r2:eth2", "r3:eth1"]
    - endpoints: ["r3:eth2", "r1:eth2"]
```

Note: Pin image tags deliberately (example tag may differ—check current FRR container tags). Each node mounts its own `./config/rN` to `/etc/frr`.

Startup config sketch (`config/r1/daemons` + `frr.conf`)

FRR images expect daemons enabled. Example `frr.conf` ideas (interface names must match):

```
frr version 10.2.1
frr defaults traditional
hostname r1
!
interface eth1
 ip address 10.0.12.1/24
!
interface eth2
 ip address 10.0.13.1/24
!
ip route 0.0.0.0/0 10.0.12.2
!
line vty
```

Exact FRR container entrypoint behavior evolves—follow the image README. The **topology pattern** (binds + links) is the durable skill.

Lifecycle

```
sudo containerlab deploy -t triangle.clab.yml
sudo containerlab graph -t triangle.clab.yml   # if supported in your version
sudo containerlab inspect -t triangle.clab.yml
docker exec -it clab-triangle-r1 vtysh
sudo containerlab destroy -t triangle.clab.yml --cleanup
```

Topology YAML anatomy

Key	Purpose
`name`	Lab name prefix for containers/namespaces
`topology.nodes`	Node inventory
`topology.links`	Point-to-point (or multipoint via extras)
`kind`	How Containerlab treats the node
`image`	Container image
`binds`	Host path mounts
`exec`	Commands after start (use sparingly; prefer files)
`env`	Environment variables
`ports`	Publish container ports to host (mgmt tools)
`mgmt-ipv4`	Static management address (optional)

Kinds you will actually use

Kind / image pattern	Role
`linux` + Alpine/Debian	Hosts, simple routers with FRR image
FRR container on `linux` kind	OSPF/BGP labs
`srl` (SR Linux community)	Modern NOS practice (free image path)
VyOS / SONiC images	Optional diversity

Always confirm **license-free** use for the image you pull. This book never requires paid virtual NOS licenses.

Links and endpoints

```
  links:
    - endpoints: ["r1:eth1", "r2:eth1"]
```

Rules of thumb:

- Interface names must match what the NOS expects or what you configure
- Point-to-point links are veth pairs under the hood
- Do not reuse the same interface name twice on one node
- Leave `eth0` for management if the kind uses it for mgmt

Multipoint / LAN segments

Patterns include attaching multiple nodes via a bridge node or kind-specific multipoint links. Early labs stick to **point-to-point** for clarity; L2 chapters introduce switch nodes.

Management network

Containerlab creates a management network so you can SSH/API to nodes without using data-plane links.

```
sudo containerlab inspect -t triangle.clab.yml
# Note mgmt IPv4 addresses
ssh admin@<mgmt-ip>   # credentials depend on image
```

Predict: Breaking a data-plane link does not remove SSH via mgmt (unless you broke the host docker network). That separation mirrors production OOB vs in-band—use it.

Inspect, exec, save

```
# Table of nodes and mgmt IPs
sudo containerlab inspect -t triangle.clab.yml

# Shell
docker exec -it clab-triangle-r1 bash

# FRR
docker exec -it clab-triangle-r1 vtysh -c 'show interface brief'

# Destroy
sudo containerlab destroy -t triangle.clab.yml --cleanup
```

`--cleanup` removes lab containers and lab-specific wiring; use it to avoid ghost interfaces.

Lab directory layout (recommended)

```
labs/triangle/
  triangle.clab.yml
  README.md
  addressing.md
  verify.sh
  config/
    r1/
    r2/
    r3/
  captures/
```

```
cd labs/triangle
sudo containerlab deploy -t triangle.clab.yml
./verify.sh
```

Graph and documentation

```
sudo containerlab graph -t triangle.clab.yml
```

Import diagrams into your journal. Combine with a hand-drawn addressing plan—graph alone lacks subnets.

Partial failure recovery

If deploy dies halfway:

```
sudo containerlab destroy -t triangle.clab.yml --cleanup
docker ps -a | grep clab
# remove stragglers if needed
sudo containerlab deploy -t triangle.clab.yml
```

Also check:

```
docker images | head
df -h          # disk full breaks pulls
ip link | wc -l  # too many leftover veths
```

Predict → observe → fix lab

Setup

Deploy `twoping` with addresses applied via `exec` for speed (acceptable for learning; later prefer binds):

```
name: twoping

topology:
  nodes:
    h1:
      kind: linux
      image: alpine:3.20
      exec:
        - ip addr add 10.0.0.1/24 dev eth1
        - ip link set eth1 up
    h2:
      kind: linux
      image: alpine:3.20
      exec:
        - ip addr add 10.0.0.2/24 dev eth1
        - ip link set eth1 up
  links:
    - endpoints: ["h1:eth1", "h2:eth1"]
```

Note: `exec` timing can race interface creation on some versions—if eth1 missing, retry commands or use a small sleep in a script bind.

Predict

- `ping 10.0.0.2` from h1 succeeds
- `ip neigh` shows REACHABLE for peer

Observe

```
docker exec clab-twoping-h1 ping -c 3 10.0.0.2
docker exec clab-twoping-h1 ip neigh
docker exec clab-twoping-h1 ip -br a
```

Fix inject

```
docker exec clab-twoping-h2 ip link set eth1 down
docker exec clab-twoping-h1 ping -c 2 10.0.0.2  # fail
docker exec clab-twoping-h2 ip link set eth1 up
```

Harden

```
#!/usr/bin/env bash
set -euo pipefail
docker exec clab-twoping-h1 ping -c 2 -W 1 10.0.0.2
echo OK
```

Re-deploy from clean destroy and ensure `verify.sh` still passes.

Common mistakes

Mistake	Symptom	Fix
Wrong interface name in links	Deploy error or silent no-path	Match kind docs
Editing live node only	Lost on destroy	Binds + git
Forgetting `--cleanup`	Ghost links/names	Always cleanup when done
Huge images, tiny laptop	OOM kills	Smaller kinds, fewer nodes
Mixing lab names	Container name collisions	Unique `name:` per active lab

Version awareness (mid-2026)

Containerlab releases steadily. Skills that stay:

- Topology as code
- kinds + images + links + binds
- deploy / inspect / destroy lifecycle

Syntax extras (stages, components, interfaces block style) may appear in newer docs—read containerlab.dev for your installed version when YAML fails validation.

```
containerlab version
containerlab help deploy
```

Security and shared hosts

- Labs often run privileged networking—isolate lab hosts
- Do not expose mgmt SSH to the internet
- Pull images from trusted registries; pin digests for CI later

Summary

- Containerlab materializes YAML into namespaces, containers, and links
- Master deploy → inspect → exec → destroy –cleanup
- Prefer binds/startup configs over irreproducible exec-only hacks
- Keep per-lab directories with verify scripts
- FRR + Linux free images cover most of this book's spine

Next: **Netlab Fundamentals**—how to layer a declarative intent compiler on top of Containerlab to automate IP addressing, protocol configurations, and validation suites.

Netlab Fundamentals

Netlab Fundamentals

While Containerlab excels at spinning up containers and wiring virtual Ethernet pairs, running real multi-node routing topologies requires hundreds of lines of boilerplate device configurations, IP address spreadsheets, and manual routing daemon setup.

Netlab (developed by Ivan Pepelnjak) solves this problem by providing a **declarative network intent and configuration compiler**. Rather than replacing Containerlab, Netlab uses Containerlab as its primary virtualization provider (`provider: clab`). You describe *what* you want in a high-level YAML topology; Netlab automatically plans IP subnets, assigns loopback and transit addresses, generates vendor-specific configurations (FRR, Arista cEOS, Nokia SR Linux, Cisco XRd), and instructs Containerlab to deploy the lab.

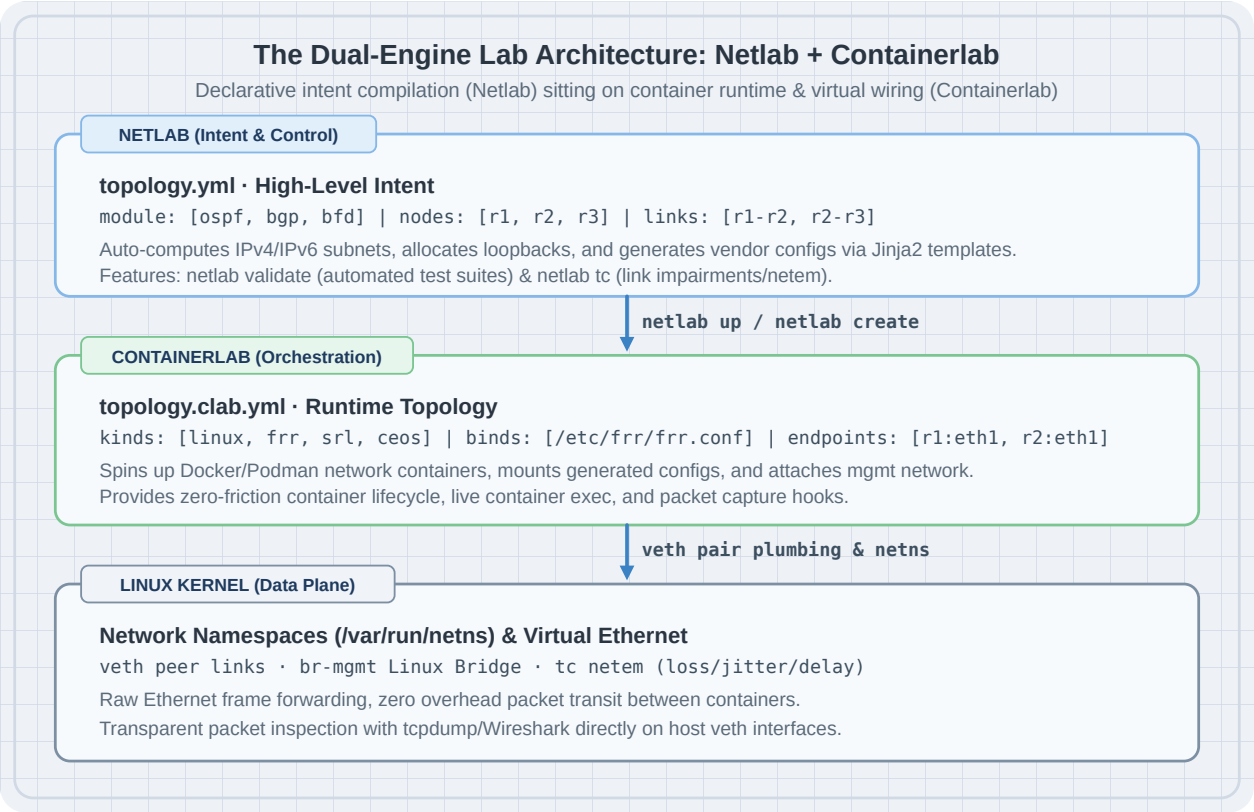


Figure 1: Dual-Engine Lab Architecture: Netlab + Containerlab

Learning goals

By the end of this chapter, you will be able to:

- Articulate the distinction between the orchestration layer (Containerlab) and the intent/configuration layer (Netlab).
- Install and configure Netlab with Containerlab on a Linux system.
- Write concise declarative topology files (`topology.yml`) with automated module activation (OSPF, BGP, BFD, VLANs).
- Inspect compiled artifacts (`netlab create`) to understand the underlying Containerlab topology and generated NOS configuration files.
- Operate the full lifecycle: `netlab up`, `netlab connect`, `netlab inspect`, and `netlab down`.
- Run automated verification test suites using `netlab validate`.
- Inject realistic network impairments (latency, jitter, packet loss) using `netlab tc`.
- Swap network operating systems (e.g., FRR to Arista cEOS or Nokia SR Linux) with a single configuration parameter.

The Dual-Engine Model: Why Both Tools Matter

When learning networking from scratch, choosing between Containerlab and Netlab is not an either/or dilemma. They serve distinct, complementary purposes across your learning journey:

Capability	Containerlab Direct (`*.clab.yml`)	Netlab with Containerlab (`topology.yml`)
Mental Model	Physical / Virtual link plumbing (`veth` pairs & namespaces)	Network architecture & protocol intent
IP Addressing	Completely manual; you must assign every IP on every interface	Fully automated IPAM; computes /30, /31, or /24 transit and /32 loopbacks
Protocol Config	Manual; you author and mount `/etc/frr/frr.conf` or vendor configs	Automated; Jinja2 templates render native NOS configs from modules
Ideal For	L1/L2 mechanics, raw ARP, custom bridges, packet crafts	OSPF, BGP, EVPN-VXLAN, multi-area fabrics, failure drills
Validation	Manual CLI checks and `docker exec` commands	Automated testing assertions with `netlab validate`
NOS Portability	Low; changing NOS requires rewriting all config files	High; change `device: frr` to `device: eos` or `device: srlinux`

Curriculum Rule of Thumb:
Use **raw Containerlab** for visceral wire-level understanding in early foundation chapters (inspecting ARP frames, Linux bridges, MTU fragmentation). Transition to **Netlab** when studying dynamic routing protocols, transit policies, and data center fabrics to eliminate syntax fatigue and focus on architectural principles.

Installation & Setup

Netlab is distributed as a Python package (`network-lab`) that interacts directly with Docker and Containerlab.

Prerequisites

Netlab requires Python 3.9+, Docker (or Podman), and Containerlab. On Ubuntu/Debian:

```
# Ensure Python virtual environment tools and Ansible core are installed
sudo apt update
sudo apt install -y python3-pip python3-venv git

# Create a dedicated virtual environment or install via pipx / pip
python3 -m pip install --upgrade network-lab
```

Automated Platform Provisioning

Netlab includes a self-bootstrapping command that verifies system requirements, installs Containerlab, configures Docker, and sets the required user groups (`docker` and `clab_admins`):

```
# Verify installation prerequisites and dependencies
netlab check

# Automatically install or update Containerlab and dependencies
sudo netlab install containerlab
```

Verify your active environment:

```
netlab test clab
```

This command deploys a two-node lightweight container lab, validates bidirectional connectivity over a virtual link, and tears down the lab cleanly.

The Anatomy of a Netlab Topology

A Netlab topology is saved by default as `topology.yml` in your working directory.

Minimal Triangle Topology with OSPF

Here is the exact equivalent of the three-router OSPF triangle that required three separate configuration files and complex volume binds in raw Containerlab:

```
# topology.yml
---
provider: clab
defaults.device: frr

module: [ ospf ]

nodes:
  r1:
  r2:
  r3:

links:
  - r1:
    r2:
  - r2:
    r3:
  - r3:
    r1:
```

What Happens Behind the Scenes?

When you run `netlab up`, the Netlab compiler executes a multi-step pipeline:

```
topology.yml


1. Topology Transformation (Assign IP subnets, loopbacks, router IDs, interface names)


2. Jinja2 Config Generation (Render /etc/frr/frr.conf and daemons for r1, r2, r3)


3. Containerlab File Generation (Emit clab.yml with nodes, kinds, mounts, and links)


4. Containerlab Orchestration (sudo containerlab deploy -t clab.yml)


5. Initial Configuration Push (Mount configs or apply via daemon sockets)
```

To see the compiled Containerlab YAML without starting containers, run:

```
netlab create
```

Inspect the generated artifacts in the current directory: - `clab.yml`: The generated Containerlab topology. - `hosts.yml` / `ansible.cfg`: Automation inventory for Ansible and Python scripts. -

`config/`: The auto-generated initial configurations for each device (e.g., `config/r1/frr.conf`).

Operating Netlab Labs

Netlab abstracts away container naming conventions (`clab-<labname>-<nodename>`) and provides uniform commands across any operating system.

1. Starting and Stopping Labs

```
# Create configuration, deploy containers, and configure devices in one step
netlab up

# Inspect running nodes, interfaces, and IPv4/IPv6 address assignments
netlab inspect

# Cleanly stop and destroy containers, veth pairs, and management links
netlab down
```

2. Connecting to Devices

You do not need to run `docker exec -it clab-... vtysh`. Netlab connects directly to the device CLI:

```
# Open an interactive CLI session on router r1 (starts vtysh on FRR)
netlab connect r1

# Run a non-interactive show command across a node
netlab connect r1 vtysh -c "show ip ospf neighbor"
```

3. Graphing Topologies

Netlab can automatically render visual diagrams of your topology in SVG or ASCII format:

```
# Generate a graphical visualization of the lab topology
netlab graph -o topology.png
```

Automated Verification with `netlab validate`

In enterprise NetOps, verifying that a network functions according to specification is done through automated tests rather than manual pings. Netlab provides an automated validation engine built directly into the topology file.

You can declare expected states under the `validate` key:

```
# topology.yml with validation rules
---
provider: clab
defaults.device: frr
module: [ ospf ]

nodes:
  r1:
  r2:
  r3:

links:
  - r1:
    r2:
  - r2:
    r3:
  - r3:
    r1:

validate:
  ospf_neighbors:
    description: "Verify all routers reach Full OSPF adjacency"
    nodes: [ r1, r2, r3 ]
    plugin: ospf_neighbor(count=2, state='Full')

  loopback_reachability:
    description: "Verify end-to-end routing between loopbacks"
    nodes: [ r1 ]
    exec:
      r1: ping -c 2 10.0.0.2
      r1_to_r3: ping -c 2 10.0.0.3
```

Execute the test suite with a single command:

```
netlab validate
```

If an adjacency is stuck in `ExStart` due to an MTU mismatch or a broken link, `netlab validate` reports a red failure with actionable diagnostic outputs.

Simulating Real-World Links with `netlab tc`

Production networks suffer from latency, jitter, packet reordering, and drop rates. Netlab integrates Linux Traffic Control (`tc` / `netem`) directly into links.

You can define impairments declaratively in `topology.yml`:

```
links:
  - r1:
    r2:
    # Emulate an intercontinental WAN link
    bandwidth: 100mbps
    latency: 45ms
    jitter: 5ms
    loss: 0.5%
```

Or inject impairments dynamically during a live lab drill using the `netlab tc` CLI tool:

```
# Add 50ms latency to the link between r1 and r2
netlab tc set r1 eth1 --delay 50ms

# Inject 10% packet drop to observe BFD or OSPF dead timer behavior
netlab tc set r2 eth1 --loss 10%

# Clear all impairments
netlab tc clear r1 eth1
```

Multi-NOS Portability: The Ultimate Learning Superpower

One of the greatest advantages of Netlab is device abstraction. Dynamic routing protocols are vendor-agnostic standards, but vendor CLIs differ dramatically.

In Netlab, changing your entire lab from FRR to Arista EOS, Cisco IOS-XR, or Nokia SR Linux requires modifying one line:

```
# Change default device from FRR to Arista cEOS
defaults.device: eos
```

Or mixing different operating systems in the same topology:

```
nodes:
  r1:
    device: frr       # Linux FRR routing daemon
  r2:
    device: eos       # Arista cEOS
```

```
  r3:
    device: srlinux   # Nokia SR Linux
```

When you execute `netlab up`, Netlab applies standard OSPF configurations across all three distinct operating systems automatically. This enables you to focus on protocol interoperability, RFC compliance, and packet behavior without getting derailed by syntax differences.

Summary & What Comes Next

- **Containerlab** is your runtime foundation: it wires containers, creates Linux bridges, and provisions network namespaces.
- **Netlab** is your network compiler: it automates IP addressing, configures routing protocols, conducts automated testing, and simulates link conditions.
- Throughout this book:
  - Early chapters (Ethernet, ARP, VLANs, static routes) use **raw Containerlab and Linux containers** so you understand the raw kernel plumbing.
  - Advanced routing chapters (OSPF, BGP, EVPN-VXLAN, Clos fabrics) provide **Netlab topologies** as the primary recipe, alongside the compiled Containerlab configurations for complete architectural clarity.

Persistent State in Containerlab Labs

Persistent State in Containerlab Labs

Thesis

Containerlab nodes are disposable until you **bind-mount** configs and state. This chapter focuses on what to persist. Containerlab was **not executed** here (no root / no installs); patterns cite containerlab.dev.

Mental model

```
deploy with binds → destroy → bind sources remain on host
```

Persist	Typical bind
NOS config	`./configs/<node>:/etc/...`
Scripts	`./scripts:/scripts:ro`
Avoid	secrets world-readable; unreproducible captures

Worked examples

```yaml
name: persist-demo
topology:
  nodes:
    r1:
      kind: linux
      image: ghcr.io/nokia/srlinux
      binds:
        - ./configs/r1:/etc/opt/srlinux:ro
    h1:
      kind: linux
      image: ghcr.io/srl-labs/network-multitool
      binds:
        - ./scripts:/scripts:ro
  links:
    - endpoints: ["r1:e1-1", "h1:eth1"]
```

Use `containerlab save` when the kind supports dumping running config back to the bind source.

Trap

Weak modes on shared lab hosts. Prefer `:ro` for configs that should not be rewritten.

Boring rule

If you care about a file after `destroy`, it lives on the host via `binds` (or documented save tooling).

Try this

1. Deploy/save/destroy/redeploy on a machine with Containerlab.
2. Mirror with **netlab** (netlab.tools).
3. Makefile target that diffs `./configs` against golden files.

Sources

- https://containerlab.dev
- https://netlab.tools

A First BGP Lab with netlab

A First BGP Lab with netlab

Thesis

netlab (ipspace/netlab) turns a YAML topology into a runnable BGP lab — addressing, ASNs, and device configs generated for you. It is the fastest way to feel eBGP/iBGP before touching production IRRs.

Mental model

```
topology.yml  --netlab up-->  VMs/containers + wiring + BGP configs

       nodes: routers with bgp.as
       links: stub / lan / p2p
       module: [ bgp ]
```

Companion: **containerlab** for container-native NOS images; netlab can use multiple virtualisation providers.

Worked example

Docs-based lab sketch (not executed here — needs provider images / elevated networking; policy forbids sudo/installs on this box).

```
# topology.yml
defaults.device: frr
module: [ bgp ]
nodes:
  r1:
    bgp.as: 65001
  r2:
    bgp.as: 65002
  r3:
    bgp.as: 65001
links:
  - r1-r2
  - r1-r3
```

```
netlab up
netlab connect r1
# vtysh
show bgp summary
show ip bgp
netlab down
```

Expected shape: `r1` runs eBGP to `r2` and iBGP to `r3` (same ASN), with netlab filling router-ids and sessions from the link graph.

Trap

Editing generated configs by hand and then re-running `netlab up`. Your edits vanish. Change the YAML (or documented extra config hooks), not the rendered files. Also: forgetting `module: [ bgp ]` yields a wiring lab with no protocol.

Boring rule

Keep topology in YAML, destroy/recreate liberally, and graduate to containerlab when you need vendor NOS images or CI-friendly containers.

Try this

1. Add `bgp.originate` prefixes on `r1` and watch them on `r2`.
2. Tag a prefix with `no_export` (previous chapter) and confirm `r2`'s downstream.
3. Compare the same graph under containerlab + FRR.

Sources

- netlab.tools documentation
- github.com/ipspace/netlab
- containerlab.dev (companion lab tool)

Free Image Matrix & Resources

Free Image Matrix & Resources

The main path of this book uses **open-source and freely available** lab images only. No paid virtual NOS license is required to finish the curriculum. This chapter catalogs practical choices (as of mid-2026 practice) and how to size a lab host.

Learning goals

By the end of this chapter you can:

- Pick images for endpoints, routers, and optional modern NOS practice
- Estimate RAM/CPU for small vs medium topologies
- Avoid accidental license walls
- Pin versions for reproducibility
- Know where official docs and communities live

Principles

1. **Free to pull and run for learning** on the main path
2. **Documented interfaces** (Linux, FRR vtysh, open NOS models)
3. **Containerlab-friendly** kinds where possible
4. **Pin tags**—`latest` is for tourists

Always re-check the image's license and Containerlab kind docs before classroom or corporate use; policies change.

Role matrix

Role	Recommended default	Alternatives (still free/open path)
Endpoint host	`alpine:3.20` or Debian slim	Ubuntu minimal
Software router / FRR	`quay.io/frrouting/frr:<tag>`	FRR on Debian + manual setup
Modern NOS feel	Nokia **SR Linux** community image via Containerlab kind	SONiC community images
Firewall / edge Swiss army	VyOS rolling (confirm current license terms)	Linux nftables node
Switch-like L2	Linux bridge / OVS container; FRR + bridge	SR Linux L2 features
Traffic tools	Alpine/Debian + `iperf3`, `tcpdump`, `mtr`	Dedicated trafficgen images
Capture analysis	Host `tshark` / Wireshark	Container with tshark
Automation control	Host Python/Ansible	CI runner container

What we deliberately de-emphasize

Avoid on main path	Why
Paid IOS/NX-OS/JUNOS virts	License friction; concepts unchanged
Exam dump topologies	Wrong optimization target
Huge multi-vendor museums	RAM death; low learning density

Paid images can be a **later footnote** if your employer provides them—not a gate.

FRR as the workhorse

FRRouting implements OSPF, BGP, IS-IS, statics, and more with a Cisco-like vtysh. Container images are the fastest path:

```
# Example - check quay for current stable tags
docker pull quay.io/frrouting/frr:10.2.1
```

Strengths: control-plane labs, policy, dual-stack
Limits: not a full merchant silicon simulator; L2 features depend on how you combine Linux bridging

Lab pattern:

```
# snippet
r1:
  kind: linux
  image: quay.io/frrouting/frr:10.2.1
  binds:
    - ./config/r1:/etc/frr
```

Enable only needed daemons (`bgpd=yes`, `ospfd=yes`) in `daemons` file to save RAM.

Linux endpoints

Alpine is small and fast:

```
h1:
  kind: linux
  image: alpine:3.20
  exec:
    - apk add --no-cache iproute2 iputils busybox-extras
```

Debian if you need a richer package set:

```
h1:
  kind: linux
  image: debian:bookworm-slim
  exec:
    - apt-get update && apt-get install -y iproute2 iputils-ping tcpdump iperf3 mtr-tiny
```

Tip: bake a personal lab image with tools preinstalled to save time:

```
FROM alpine:3.20
RUN apk add --no-cache iproute2 iputils tcpdump iperf3 mtr bind-tools
```

```
docker build -t local/lab-endpoint:1.0 .
```

Use `local/lab-endpoint:1.0` in topologies.

SR Linux community (optional track)

Nokia SR Linux community images are commonly used with Containerlab for modern streaming telemetry and CLI/model exposure—without classic paid chassis IOS requirements. Follow current Containerlab **kind: srl** documentation for image names and credentials.

Use SR Linux when you want:

- YANG-ish / modern NOS workflows
- Leaf-spine demos closer to DC ops
- Comparison against FRR ("same OSPF idea, different dialect")

Do not block L2/L3 fundamentals waiting for SR Linux. FRR+Linux is enough for competence.

SONiC and VyOS (optional)

Image family	Good for	Watch for
SONiC	DC NOS ideas, BGP underlay practice	Image size, platform assumptions
VyOS	Edge router feature bundle	Release/license wording over time

Treat both as **breadth**, not blockers.

Sizing the lab host

Rough starting points (conservative; measure yours):

Lab scale	Nodes (example)	RAM ballpark	CPU
Tiny	2–3 Alpine	2–4 GB free	2
Small	3× FRR + 2 hosts	8 GB system	4
Medium	6–8 FRR/SRL + hosts	16 GB	6–8
Fabric snack	2 spine + 4 leaf + hosts	32 GB happier	8+

```
free -h
nproc
docker stats --no-stream
```

RAM savers

- Prefer Alpine endpoints over full Ubuntu desktops
- Disable unused FRR daemons
- Destroy labs when idle

- Avoid multiple concurrent large labs
- Do not run heavy desktop + large lab + browser zoo on 8 GB

Disk

Images accumulate:

```
docker system df
docker image prune
# careful:
# docker system prune
```

Pin and keep only tags you need.

Version pinning strategy

```
# Good
image: quay.io/frrouting/frr:10.2.1
image: alpine:3.20

# Risky for books/CI
image: quay.io/frrouting/frr:latest
image: alpine:latest
```

In CI (later chapter), pin digests:

```
image: quay.io/frrouting/frr:10.2.1@sha256:....
```

Record pins in each lab's README.

Resource planning worksheet

Before a new topology, write:

```
## Sizing
Nodes:
- 3 x FRR (~X MB each estimated)
- 2 x Alpine
Total RAM budget: ...
Host free RAM: ...
Fallback if OOM: remove r3, collapse to chain
```

If deploy dies with OOM kills:

```
dmesg -T | tail -50
docker ps -a
```

Network resource limits on the host

Resource	Symptom when exhausted
RAM	Random node death
CPU	Protocol timers flaky, loss
Inotify/watches	Rare editor/CI issues
Interface count	Fail to create veth
Docker IP pool	Mgmt network allocate fail

```
ip link | wc -l
docker network ls
```

Destroy cleaned labs free veths.

Documentation & community map

Resource	Use
containerlab.dev	Topology reference, kinds
docs.frrouting.org	OSPF/BGP/commands
RFCs (IETF)	Authoritative protocol behavior
Linux `man ip`, `man bridge`	Host dataplane
Lab repos using Containerlab	Inspiration—verify licenses

RFCs worth bookmarking (foundations)

Topic	Starting points
IP	RFC 791 (historic), RFC 8200 (IPv6)
ICMP	RFC 792, RFC 4443
ARP	RFC 826
OSPF	RFC 2328 (v2)
BGP	RFC 4271
Ethernet/VLAN	IEEE world + RFC 5517 context notes

You do not memorize RFCs; you **use them when models conflict**.

Predict → observe: image sanity lab

```yaml
name: imgcheck

topology:
  nodes:
    frr1:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
    a1:
      kind: linux
      image: alpine:3.20
  links:
    - endpoints: ["frr1:eth1", "a1:eth1"]
```

Predict: Both containers running; `vtysh` answers on frr1; Alpine has `ip` after apk or use busybox ip if present.

```
sudo containerlab deploy -t imgcheck.clab.yml
docker exec clab-imgcheck-frr1 vtysh -c 'show version'
docker exec clab-imgcheck-a1 uname -a
sudo containerlab destroy -t imgcheck.clab.yml --cleanup
```

Harden: Document the exact tags that work on your machine in a personal `IMAGES.md`.

Multi-implementation mindset

Later design labs may mix FRR + Linux + optional SR Linux. Same OSPF adjacency model; different show commands. Keep a translation scrap:

Intent	FRR	Linux
Addresses	`show interface`	`ip -br a`
Routes	`show ip route`	`ip route`
BGP summary	`show bgp summary`	n/a or bird/etc

Ethics and redistribution

- Do not redistribute proprietary images
- Respect registry rate limits and ToS
- Prefer building lab content others can run **without** secrets or licenses

Summary

- Default stack: Containerlab + FRR + Alpine/Debian
- Optional free NOS: SR Linux community, SONiC, VyOS—with eyes open
- Size hosts honestly; destroy idle labs
- Pin image tags; record what works
- Official docs + RFCs beat random CLI folklore

Next: **lab hygiene**—addressing plans, naming, git layout, and diagram habits so multi-lab life stays sane.

Linux Networking for Lab Hosts

Linux Networking for Lab Hosts

Containerlab sits on Linux networking primitives: namespaces, veth pairs, bridges, routes, and basic filtering. If you only drive YAML without understanding the host, debugging feels like magic. This chapter builds the host-side intuition you need before (and during) Containerlab work.

Learning goals

By the end of this chapter you can:

- Explain how network namespaces isolate stacks
- Create veth pairs and bridges by hand
- Add addresses and routes; verify with `ip route get`
- Use `tcpdump` inside a namespace
- Map Containerlab links to veth/bridge concepts
- Apply minimal filter awareness (`nft`/`iptables`, `rp_filter`)

Learning goals in practice

You will build a **two-namespace router lab** without Containerlab, then tear it down. That is the same physics Containerlab orchestrates for you.

Core objects

Object	Role
Interface	L2 endpoint (physical, veth, bridge, vlan subint)
Network namespace	Isolated network stack (interfaces, routes, sockets)
veth pair	Virtual cable: two ends, often in different namespaces
Bridge	Software switch in the kernel

Object	Role
Route	FIB entry: destination → device / next hop
Rule	Policy routing (advanced; optional here)

```
ip -br link
ip -br addr
ip netns list
ip route
```

Network namespaces

A namespace process sees the **root** namespace. Lab nodes (containers) each get a namespace.

```
# Create two namespaces
sudo ip netns add ns1
sudo ip netns add ns2
ip netns list

# Run a command inside
sudo ip netns exec ns1 ip link
sudo ip netns exec ns1 ping -c1 127.0.0.1
```

Loopback exists per namespace; bring it up if you care about local sockets:

```
sudo ip netns exec ns1 ip link set lo up
```

veth pairs: virtual cables

```
# Create pair veth-a <-> veth-b
sudo ip link add veth-a type veth peer name veth-b

# Move ends into namespaces
sudo ip link set veth-a netns ns1
sudo ip link set veth-b netns ns2

# Name + address inside each ns
sudo ip netns exec ns1 ip link set veth-a name eth0
sudo ip netns exec ns2 ip link set veth-b name eth0

sudo ip netns exec ns1 ip addr add 10.0.0.1/24 dev eth0
sudo ip netns exec ns2 ip addr add 10.0.0.2/24 dev eth0
```

```
sudo ip netns exec ns1 ip link set eth0 up
sudo ip netns exec ns2 ip link set eth0 up

# Test L3 on-link
sudo ip netns exec ns1 ping -c 2 10.0.0.2
```

Predict: ARP resolves; ICMP works; no router required because same subnet.

Observe:

```
sudo ip netns exec ns1 ip neigh
sudo ip netns exec ns1 tcpdump -ni eth0 -c 10 arp or icmp &
sudo ip netns exec ns1 ping -c 2 10.0.0.2
```

Bridges: software switches

Connect multiple endpoints as if on a switch:

```
sudo ip netns add h1
sudo ip netns add h2
sudo ip netns add h3

sudo ip link add br0 type bridge
sudo ip link set br0 up

# veth for each host: host side in ns, peer on host attached to bridge
for n in 1 2 3; do
  sudo ip link add veth-h$n type veth peer name veth-b$n
  sudo ip link set veth-h$n netns h$n
  sudo ip netns exec h$n ip link set veth-h$n name eth0
  sudo ip netns exec h$n ip addr add 10.10.0.$n/24 dev eth0
  sudo ip netns exec h$n ip link set eth0 up
  sudo ip netns exec h$n ip link set lo up
  sudo ip link set veth-b$n master br0
  sudo ip link set veth-b$n up
done

sudo ip netns exec h1 ping -c 2 10.10.0.2
sudo ip netns exec h1 ping -c 2 10.10.0.3
```

Bridge FDB (MAC table):

```
bridge fdb show br br0
# or
ip neigh  # not the same; FDB is L2 forward database
```

```
bridge link show
```

After pings, you should see learned MACs on the bridge ports.

Routing between namespaces

Classic: two LANs and a router namespace.

```
h1 (10.1.0.10/24) -- veth -- r (10.1.0.1 + 10.2.0.1) -- veth -- h2 (10.2.0.10/24)
```

```
sudo ip netns add h1
sudo ip netns add h2
sudo ip netns add r

# h1 <-> r
sudo ip link add v1 type veth peer name r1
sudo ip link set v1 netns h1
sudo ip link set r1 netns r
sudo ip netns exec h1 ip link set v1 name eth0
sudo ip netns exec r  ip link set r1 name eth1
sudo ip netns exec h1 ip addr add 10.1.0.10/24 dev eth0
sudo ip netns exec r  ip addr add 10.1.0.1/24  dev eth1
sudo ip netns exec h1 ip link set eth0 up
sudo ip netns exec r  ip link set eth1 up
sudo ip netns exec h1 ip link set lo up
sudo ip netns exec r  ip link set lo up

# h2 <-> r
sudo ip link add v2 type veth peer name r2
sudo ip link set v2 netns h2
sudo ip link set r2 netns r
sudo ip netns exec h2 ip link set v2 name eth0
sudo ip netns exec r  ip link set r2 name eth2
sudo ip netns exec h2 ip addr add 10.2.0.10/24 dev eth0
sudo ip netns exec r  ip addr add 10.2.0.1/24  dev eth2
sudo ip netns exec h2 ip link set eth0 up
sudo ip netns exec r  ip link set eth2 up
sudo ip netns exec h2 ip link set lo up

# Enable forwarding on router ns
sudo ip netns exec r sysctl -w net.ipv4.ip_forward=1

# Default routes on hosts
sudo ip netns exec h1 ip route add default via 10.1.0.1
sudo ip netns exec h2 ip route add default via 10.2.0.1
```

```
# Test
sudo ip netns exec h1 ping -c 3 10.2.0.10
sudo ip netns exec h1 traceroute -n 10.2.0.10
```

Predict → observe:

```
sudo ip netns exec h1 ip route get 10.2.0.10
sudo ip netns exec r  ip route get 10.2.0.10 from 10.1.0.10 iif eth1
sudo ip netns exec r  ip -s link show eth1
```

Failure drill: disable forwarding

```
sudo ip netns exec r sysctl -w net.ipv4.ip_forward=0
sudo ip netns exec h1 ping -c 2 10.2.0.10   # expect fail
sudo ip netns exec r sysctl -w net.ipv4.ip_forward=1
```

Harden note: container routers must allow forwarding; images often set this, but verify.

Failure drill: missing return route

If the "router" only had a connected route on one side (misconfigured static elsewhere), one-way traffic appears. With connected routes on both interfaces above, both directions work. Later static chapters amplify return-path bugs.

VLAN subinterfaces (preview of trunks)

```
# On a namespace with eth0
sudo ip netns exec h1 ip link add link eth0 name eth0.10 type vlan id 10
sudo ip netns exec h1 ip addr add 10.10.10.1/24 dev eth0.10
sudo ip netns exec h1 ip link set eth0.10 up
```

Both ends need matching VLAN IDs. Bridges can be VLAN-aware (`vlan_filtering`); Containerlab L2 labs often use OVS or NOS switches—concepts transfer.

Policy leftovers that break labs

rp_filter (reverse path filtering)

Strict reverse-path checks drop asymmetric legitimate lab traffic.

```
sysctl net.ipv4.conf.all.rp_filter
sysctl net.ipv4.conf.default.rp_filter
# Per-iface:
# sysctl net.ipv4.conf.eth0.rp_filter
```

If mysterious one-way drops appear on Linux routers, check `rp_filter` and counters.

Forwarding and firewall

```
# nftables quick view
sudo nft list ruleset
# legacy
sudo iptables -L -n -v
sudo iptables -t nat -L -n -v
```

Host firewalls (especially Docker's chains) can interact with lab bridges. When a lab "should work" and does not, inspect host filter tables—not only node configs.

Disable offloads when captures look weird (optional)

Checksum offload can make `tcpdump` show wrong checksums on local veth. Prefer understanding this over panicking:

```
sudo ethtool -k veth0 | grep sum || true
```

Mapping to Containerlab

You did by hand	Containerlab roughly does
`ip netns add`	Creates container network namespaces
veth pairs	Links between nodes
bridges / OVS	Multipoint wiring, mgmt network
addressing	From topology + image startup
`ip_forward`	Inside router nodes

When Containerlab deploy fails or links are dark, drop to:

```
ip netns list
# Containerlab often names netsns like clab-<lab>-<node>
sudo ip netns exec clab-mylab-r1 ip -br a
```

Exact naming depends on lab name and version; `containerlab inspect` helps.

```
sudo containerlab inspect -t topology.clab.yml
```

Clean teardown habits

Namespaces and veths litter a host if you exit mid-lab.

```
# Example cleanup for the objects you created
sudo ip netns del h1 2>/dev/null || true
sudo ip netns del h2 2>/dev/null || true
sudo ip netns del r  2>/dev/null || true
sudo ip netns del ns1 2>/dev/null || true
sudo ip netns del ns2 2>/dev/null || true
sudo ip link del br0 2>/dev/null || true
# dangling veths usually die with netns; if not:
ip -br link | grep veth || true
```

Prefer scripted `up.sh` / `down.sh` even for hand labs.

Example `down.sh`

```
#!/usr/bin/env bash
set -euo pipefail
for ns in h1 h2 r ns1 ns2; do
  sudo ip netns del "$ns" 2>/dev/null || true
done
sudo ip link del br0 2>/dev/null || true
echo "cleaned"
```

Predict → observe → fix drill (checklist)

Lab: three hosts on `br0` as above.

1. **Predict:** h1→h3 uses bridge flood then learn; FDB populates.
2. **Observe:** `bridge fdb show`, tcpdump on `veth-b1`.
3. **Fix inject:** `sudo ip link set veth-b3 down` — h3 unreachable; FDB ages/fails.
4. **Harden:** document “access port down vs cable unplug” equivalence in labs.

Useful one-liners cheatsheet

```
# Watch neigh events
ip monitor neigh

# Route decision
ip route get 1.1.1.1 from 10.1.0.10 iif eth0

# Stats
ip -s -s link show eth0

# All addresses every ns (rough)
for ns in $(ip netns list | awk '{print $1}'); do
  echo "===== $ns ====="
  sudo ip netns exec "$ns" ip -br a
done
```

Security note for shared hosts

Namespace labs are powerful. On a shared machine:

- Do not bridge lab interfaces onto trusted production networks without filters
- Clean up after sessions
- Avoid enabling ip_forward on the **root** namespace unless you intend the host to route

Containerlab management networks still need isolation discipline on corporate laptops.

Summary

- Namespaces isolate full network stacks—foundation of container networking
- veth pairs are virtual cables; bridges are virtual switches
- Router namespaces need addressing, up state, forwarding, and routes both ways
- Host firewall and `rp_filter` are common "invisible" lab breakers
- Hand-building once makes Containerlab YAML readable and debuggable

Next: **Containerlab fundamentals**—topology YAML, kinds, links, lifecycle—on top of these primitives.

Lab Hygiene

Lab Hygiene

Brilliant topologies rot without hygiene: colliding subnets, random hostnames, uncommitted click-ops, and mystery cables. This chapter is the professional layer on top of Containerlab—how to keep labs reproducible, reviewable, and kind to future you.

Learning goals

By the end of this chapter you can:

- Write an addressing plan before deploy
- Apply consistent node and interface naming
- Structure a lab repo/directory for git
- Keep diagrams and inventories in sync
- Run a personal definition-of-done for every lab
- Avoid cross-lab contamination on one host

Why hygiene is a skill

Without hygiene	With hygiene
"Which 10.0.0.0/24 is this?"	Documented per-lab blocks
Works once after 40 hand edits	Destroy/deploy/verify loop
Cannot explain a failure	Journal + captures
Peer cannot run your lab	README + pins + scripts

Hygiene is not bureaucracy; it is how lab craft scales.

Addressing plans

Never invent IPs mid-SSH.

Template

```
# Addressing - lab triangle-ospf

| Node | Interface | Address | Notes |
|------|-----------|---------|-------|
| r1 | eth1 | 10.0.12.1/24 | to r2 |
| r2 | eth1 | 10.0.12.2/24 | to r1 |
| r1 | eth2 | 10.0.13.1/24 | to r3 |
| r3 | eth2 | 10.0.13.2/24 | to r1 |
| r2 | eth2 | 10.0.23.1/24 | to r3 |
| r3 | eth1 | 10.0.23.2/24 | to r2 |
| h1 | eth1 | 192.168.1.10/24 | gw 192.168.1.1 (r1 lo/lan) |

Loopbacks:
| Node | lo |
|------|-----|
| r1 | 1.1.1.1/32 |
| r2 | 2.2.2.2/32 |
| r3 | 3.3.3.3/32 |

VLANs: n/a
ASNs: n/a
Mgmt: Containerlab default pool
```

Allocation strategy (personal lab supernets)

Pick a **lab supernet** you will not confuse with home LAN:

Use	Example block
Home LAN (real)	192.168.1.0/24 (do not reuse in labs if VPN splits break)
Lab underlay P2P	10.0.0.0/16 carved into /24 or /30
Lab customer VRFs	10.10.0.0/16
Loopbacks	10.255.0.0/24 as /32s
“Internet” sim	203.0.113.0/24 (documentation range) / 192.0.2.0/24

Documentation ranges (RFC 5737) are great for examples in prose; inside isolated labs private space is fine—just **write it down**.

/30 vs /31 vs /24 on links

Choice	When
/24	Beginners, lots of headroom, wasteful

Choice	When
/30	Classic P2P
/31	Efficient P2P (ensure both NOS support)

Be consistent within a lab.

Naming conventions

Lab name

```
name: l2-vlan-trunk   # short, hyphenated, unique among active labs
```

Container names become `clab-<name>-<node>`.

Node names

Pattern	Examples
Role + number	`r1`, `r2`, `sw1`, `h1`
Role + site	`edge-a`, `core-1`
Avoid	`test`, `node`, `foo`, `router`

Interface names

Stick to what the image uses (`eth1`, `eth2`… for many Linux kinds). Document any breakout or rename.

Git branches / folders

```
labs/
  03-l2-vlan-trunk/
  04-static-triangle/
  05-ospf-single-area/
```

Numeric prefixes match book order when useful.

Directory layout (canonical)

```
labs/04-static-triangle/
  README.md           # intent, pins, how to run
  topology.clab.yml
  addressing.md
  journal.md
  verify.sh
  up.sh               # optional deploy wrapper
  down.sh             # destroy --cleanup
  config/
    r1/
    r2/
    h1/
  captures/
  diagrams/
    logical.png       # or .svg / .drawio
```

README skeleton

```
# Static triangle

## Intent
Three FRR routers, static routes, dual hosts ping.

## Images
- quay.io/frrouting/frr:10.2.1
- alpine:3.20

## Run
sudo containerlab deploy -t topology.clab.yml
./verify.sh
sudo containerlab destroy -t topology.clab.yml --cleanup

## Failure drills
- shut transit link r1-r2
- remove return route on r3
```

Git habits

```
git status
git add labs/04-static-triangle
git commit -m "lab: static triangle with verify.sh"
```

Commit:

- Topology YAML
- Configs
- addressing.md
- verify.sh
- journal highlights (not necessarily huge pcaps)

Usually ignore:

- Large pcaps (store sparse samples or regenerate)
- Ephemeral logs

```
# example snippet
labs/**/captures/*.pcap
!labs/**/captures/.gitkeep
```

Never commit

- Production credentials
- Private keys
- Proprietary images

Diagram habit

Minimum viable diagram set:

1. **Logical** — nodes and links
2. **Addressing** — subnets/VLAN IDs on links
3. **Failure** — which link you shut for the drill

ASCII is valid:

```
h1--r1------r2--h2
     \      /
      \    /
        r3
```

Illustrated topology SVGs (self-contained light canvas) match this book's lab figures under `images/`; conceptual models may stay monochrome. See `images/README.md`.

Configuration hygiene

Do	Don't
Edit files under `config/`	Treasure unique vtysh-only state
Reload/restart from files	Assume memory is durable
Diff configs in git	Copy-paste from chat without review
Comment tricky policy lines	Leave "magic" prefix-lists

FRR example: comment intent

```
! block lab RFC1918 from leaking to "isp" peer
ip prefix-list LAB-PRIV seq 5 permit 10.0.0.0/8 le 32
```

verify.sh hygiene

- `set -euo pipefail`
- Explicit container names
- Timeouts on ping
- Exit non-zero on failure
- Print which check failed

```
#!/usr/bin/env bash
set -euo pipefail
PASS=0
fail() { echo "FAIL: $*" >&2; exit 1; }

docker exec clab-static-h1 ping -c 2 -W 1 10.2.0.10 \
  || fail "h1 cannot reach h2"
```

```
echo "OK: all checks passed"
```

Cross-lab contamination

Symptoms:

- Wrong SSH target (same hostname different lab)
- Stale docker networks
- IP confusion in your brain

Mitigations:

```
sudo containerlab inspect --all 2>/dev/null || true
docker ps --format 'table {{.Names}}\t{{.Status}}' | grep clab || true
# Only one "heavy" lab running when learning
sudo containerlab destroy -t old.clab.yml --cleanup
```

Use unique lab `name:` fields always.

Time hygiene

Put dates in journals; note Containerlab and image versions:

```
Date: 2026-08-04
containerlab: 0.xx.x
frr image: 10.2.1
kernel: $(uname -r)
```

Future breakage is often a version drift story.

Definition of done (lab)

- ☐ Intent one-liner in README
- ☐ addressing.md complete
- ☐ topology deploys clean on fresh destroy
- ☐ verify.sh green

☐ One failure drill documented

☐ Diagrams match YAML

☐ Secrets absent

☐ Image tags pinned

☐ Committed to git

Predict → observe → fix: hygiene drill

1. **Predict:** A peer can clone your lab folder and pass `verify.sh` without asking you questions.
2. **Observe:** Remove your personal aliases; follow only README.
3. **Fix:** Gaps become README bullets.
4. **Harden:** Add `up.sh`/`down.sh`.

```
#!/usr/bin/env bash
# up.sh
set -euo pipefail
sudo containerlab deploy -t topology.clab.yml
./verify.sh
```

```
#!/usr/bin/env bash
# down.sh
set -euo pipefail
sudo containerlab destroy -t topology.clab.yml --cleanup
```

Collaboration norms (optional)

If sharing labs:

- Prefer open images only
- Include expected verify output sample
- Note host requirements (RAM)
- License your configs clearly (e.g. MIT notes)

Summary

- Address first, deploy second
- Name labs and nodes for humans and `docker ps`
- Canonical directory: topology, configs, verify, journal, diagrams
- Git is part of networking practice
- One clean destroy/deploy is worth ten fragile demos
- Definition of done prevents "almost labs"

Part checkpoint: topology-as-code should feel like muscle memory. Next part enters **layer 2**—Ethernet, VLANs, loops, aggregation, and failure drills.

Networking Foundations

What Is Networking?

What Is Networking?

A network moves **data** between **endpoints** over **links**, using **protocols**—agreed rules for format, addressing, and behavior. Everything later in this book (VLANs, OSPF, VXLAN) is a specialization of that idea.

Learning goals

- Name the main building blocks of a network and what problem each solves
- Distinguish local delivery (same L2 domain) from routed delivery (across networks)
- Explain why we use layered protocols instead of one monolithic format
- Tie lab tools (Containerlab, `ip`, captures) to those ideas

Building blocks

Piece	Role	Lab example
Endpoint / host	Source or sink of traffic	Alpine container, your laptop
Interface	Attachment to a link	`eth0`, `eth1`
Link	Path between interfaces	Cable, veth pair, Wi-Fi
Switch / bridge	Forward within an L2 domain by MAC	Linux bridge, virtual switch
Router	Forward between L3 networks by IP	FRR node, Linux with forwarding
Protocol	Rules and message formats	Ethernet, IPv4, TCP, DNS

What problem networking solves

Without a network, processes on different machines cannot cooperate. Networking provides:

1. **Reachability** — a path exists (or is reported as unreachable)

2. **Naming / addressing** — who is who (MAC, IP, names via DNS)
3. **Multiplexing** — many conversations share links (ports, VLANs, queues)
4. **Reliability options** — best-effort (UDP/IP) vs recovered streams (TCP)
5. **Policy** — who may talk to whom, with what priority

Switching vs routing (first cut)

```
Same subnet / VLAN          Different subnets

  Host A  switch  Host B    Host A → gateway → … → Host B
  Frame uses MAC             Packet uses IP; each hop rewrites L2
```

- **Switching (L2):** deliver a frame inside one broadcast domain; learn MACs; flood unknown unicasts.
- **Routing (L3):** deliver a packet across networks; each router decrements TTL and chooses a next hop.

You will refine both in the Layer 2 and Layer 3 parts. Foundations give vocabulary and host-level truth.

Protocols as contracts

A protocol answers:

- What bits mean (headers, fields)
- Who may send what next (state machines)
- How errors are signaled (ICMP, TCP RST, DHCP NAK)

Vendors invent CLIs; **RFCs and open implementations** define the contracts this book cares about.

Why labs beat slideware

Networks fail in partial, asymmetric, and time-dependent ways. A Containerlab topology lets you:

- Change **one** variable (shut a link, wrong mask, bad gateway)
- Observe **tables** (ARP, routes, MAC) and **packets** (tcpdump)
- Reset and try again

Observation toolkit (preview)

Tool	Use
`ip addr` / `ip route` / `ip neigh`	Host addressing and neighbors
`ping` / `traceroute` / `mtr`	Reachability and path hints
`ss` / `netstat`	Sockets and listeners
`tcpdump` / `tshark`	On-the-wire proof
Containerlab	Multi-node topologies as code

Checkpoint

Before deeper chapters, you should be able to point at a simple two-host lab and say: *these are endpoints, this is a link, delivery is L2 or L3 for this destination, and here is how I would prove it.*

Next

- TCP/IP and the Layered Stack

TCP/IP and the Layered Stack

TCP/IP and the Layered Stack

Networks are built as a **stack of layers**. Each layer solves one class of problem and offers a service to the layer above. This book uses the **TCP/IP** (Internet) model in day-to-day work; the **OSI** model remains a useful legend for vendor docs and troubleshooting language.

Learning goals

- Map problems and tools to the correct TCP/IP layer
- Translate common OSI layer numbers into TCP/IP practice
- Explain demultiplexing (how a host hands data up the stack)
- Avoid "wrong layer" diagnoses

TCP/IP model (working map)

Layer	Examples	Responsibility
Application	HTTP, DNS, SSH, DHCP, BGP (as app of TCP/UDP)	Meaning for programs; often client/server
Transport	TCP, UDP	Process-to-process; ports; reliability options
Internet	IPv4, IPv6, ICMP	Host-to-host logical addressing; routing; error signaling
Link / network access	Ethernet, Wi-Fi, PPP	Adjacent-node framing; media access

Some texts split "host-to-network" further; for labs, **Ethernet + ARP/ND** is the daily link reality.

OSI as a legend (not a religion)

OSI #	Name	Rough TCP/IP home
7	Application	Application
6	Presentation	Often folded into app (TLS, encodings)

OSI #	Name	Rough TCP/IP home
5	Session	Often folded into app/TCP
4	Transport	TCP/UDP
3	Network	IP / ICMP
2	Data link	Ethernet frames, switching
1	Physical	Cables, optics, signaling (usually out of scope here)

Practical rule: know *where the address lives* and *which header you are looking at in a capture*, not seven acronyms for a quiz.

What each layer "owns"

Link

- Local delivery between neighbors
- MAC addresses, Ethertype, FCS
- Switches forward here without caring about IP

Internet (IP)

- Global-ish logical addressing
- Best-effort packet forwarding
- TTL/hop limit; fragmentation (IPv4) or reliance on PMTU (IPv6)

Transport

- Which **process** on the host
- Ports; TCP connection state vs UDP datagrams

Application

- Protocol semantics (DNS query/response, HTTP methods)
- Often implements its own retries on top of UDP

Demultiplexing (up the stack)

On receive:

1. NIC / driver accepts a frame (MAC filter)
2. Ethertype → IP vs ARP vs other
3. IP protocol field → TCP / UDP / ICMP / …
4. Port numbers → socket / application

```
Frame [Eth | IP | TCP | HTTP ]
          ^    ^    ^     ^
          |    |    |     application
          |    |    transport demux (ports)
          |    internet demux (protocol #)
          link demux (Ethertype)
```

Layered troubleshooting

Symptom	Look first at
No link light / veth missing	Link / lab cabling
ARP incomplete	Link + IP on-link config
Ping fails, ARP OK	Routing, firewall, ICMP policy
TCP SYN no reply	ACL, listening socket, path, RST
TLS fails after TCP	Application / certificates

Encapsulation (preview)

Each layer **adds a header** when sending and **strips it** when receiving. Details: Packets, Frames, Encapsulation.

Checkpoint

Given a failure description, name the **layer** you will inspect first and **one command or table** that proves or disproves that theory.

Next

- Packets, Frames, Encapsulation

Packets, Frames, Encapsulation

Packets, Frames, Encapsulation

Data is **wrapped** in headers as it descends the stack and **unwrapped** on the way up. Clear names prevent confusion in captures and design talks.

Learning goals

- Use correct terms: data, segment/datagram, packet, frame
- Sketch encapsulation for a simple TCP and UDP flow
- Read a tcpdump line and identify outer headers
- Explain how tunnels add outer encapsulations

Vocabulary

Term	Layer (typical)	Meaning
Data / payload	Application	What the app cares about
Segment	TCP	TCP header + payload
Datagram	UDP (and sometimes IP)	UDP header + payload
Packet	IP	IP header + payload
Frame	Link	Ethernet (or other) header + payload + FCS

People say "packet" loosely; in precise talk, **frame** is L2 and **packet** is L3.

Encapsulation path (send)

1. Application writes bytes
2. Transport adds **src/dst ports** (TCP adds seq/ack/flags)
3. IP adds **src/dst IP**, TTL, protocol
4. Ethernet adds **src/dst MAC**, Ethertype

5. Bits go on the wire (or veth)

```
 Application data
+------------------+
| TCP/UDP header   |
+------------------+
| IP header        |
+------------------+
| Ethernet header  |
+------------------+
```

Decapsulation path (receive)

Reverse order: FCS check → MAC → Ethertype → IP → protocol → ports → app.

Maximum sizes and MTU

- **MTU**: max IP packet size a link will accept (commonly 1500 on Ethernet)
- Larger packets must fragment (IPv4) or fail with Packet Too Big (IPv6 + PMTUD)
- Extra encapsulation (VLAN tag, VXLAN, IPsec) **reduces** effective payload size

Details and labs: Layer 3 ICMP/PMTU chapter later; foundations need the idea.

Captures: reading the onion

```
# On a Linux lab node
tcpdump -ni eth1 -vv
tcpdump -ni eth1 icmp
tcpdump -ni eth1 'tcp port 22'
```

What you see first is the **outer** header. Inner headers appear after decap or when capturing on the right interface.

Tunnels and overlays (preview)

Overlays wrap an **inner** packet in an **outer** IP/UDP (or GRE) packet. Underlay routers only need to forward the outer packet. Full treatment: Overlays part.

```
[ Outer Eth | Outer IP | Outer UDP | VXLAN | Inner Eth | Inner IP | … ]
```

Lab sketch (two nodes)

1. Deploy two Linux nodes on one L2 segment
2. Capture while `ping` runs — ICMP inside IP inside Ethernet
3. Capture `nc` or `curl` — TCP three-way handshake fields

Checkpoint

Draw encapsulation for **DNS over UDP** and **SSH over TCP** from host A to B on the same Ethernet. Label every header.

Next

- Link Layer and Ethernet

Link Layer and Ethernet

Link Layer and Ethernet

The **link layer** moves frames between neighbors on a shared medium or point-to-point link. In almost every lab in this book, that means **Ethernet** (physical or virtual).

Learning goals

- Describe Ethernet frame fields used in daily troubleshooting
- Explain unicast, multicast, broadcast at L2
- Contrast hubs (obsolete), switches, and bridges
- Preview MAC learning (full drills in Layer 2 part)

Why link layer matters

IP cannot deliver alone: on multi-access Ethernet, the next hop is identified by **MAC**. Wrong VLAN, wrong MAC, or broken L2 path makes "routing" look broken when the problem is local.

Ethernet frame (simplified)

Field	Role
Dest MAC	Who should accept the frame
Src MAC	Who sent it (learning key on switches)
Ethertype / length	Payload type (e.g. `0x0800` IPv4, `0x86DD` IPv6, `0x0806` ARP)
Payload	IP packet, ARP, etc.
FCS	Error detection

802.1Q VLAN tags insert **TPID + TCI** (including VLAN ID) between SA and Ethertype on trunks—details in VLANs chapter.

MAC addresses

- 48-bit; written `aa:bb:cc:dd:ee:ff`
- Universally administered vs locally administered
- **Broadcast** `ff:ff:ff:ff:ff:ff`
- Multicast: group bit set

```
ip -br link
ip link show eth1
```

Broadcast domain vs collision domain

- **Collision domain:** historically shared half-duplex media; switched full-duplex Ethernet largely eliminates classic collisions
- **Broadcast domain:** extent of L2 broadcast/unknown flood—usually one VLAN

Routers **stop** L2 broadcasts; switches do not.

Switch vs router (again)

Device	Forwarding key	Changes MAC?	Changes IP?
L2 switch	Dest MAC + FDB	No (transparent)	No
Router	Dest IP + FIB	**Yes** (new Ethernet hop)	No (until NAT)

MAC learning (preview)

1. Learn **src MAC → port** on ingress
2. If dest MAC known, forward to that port
3. If unknown unicast, **flood** in the VLAN
4. Broadcast/multicast: flood per rules

Deep labs: Ethernet & MAC Learning.

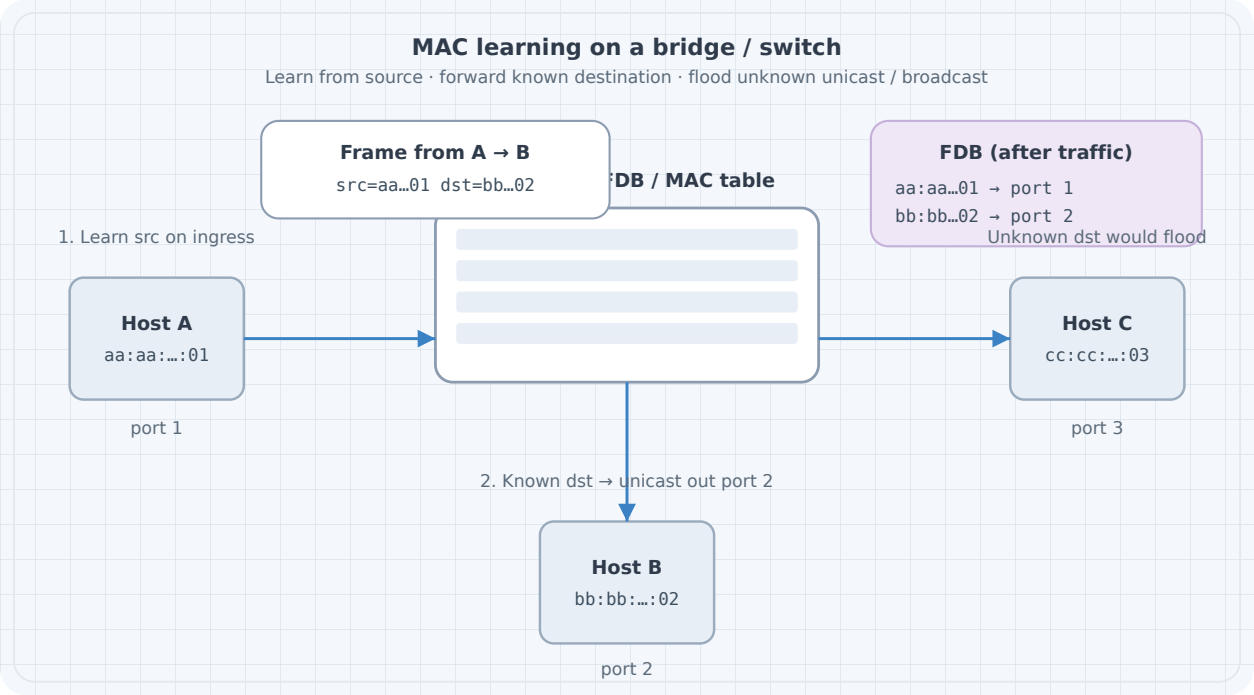


Figure 1: MAC learning

Virtual Ethernet in labs

Containerlab connects nodes with **veth pairs** and bridges—same Ethernet model, software implementation:

```
ip link
bridge link   # if bridge-utils / iproute2 bridge
```

Common L2 failures (foundations list)

Symptom	Typical cause
Incomplete ARP	Wrong subnet, wrong VLAN, silent host
Flapping MAC	Loops or dual-homing without design
No frame at all	Down interface, wrong cable/veth, admin down

Checkpoint

Explain what a switch does when it receives a frame with an **unknown** destination MAC, and what it does with the **source** MAC.

Next

- IPv4 Addressing and Subnetting

IPv4 Addressing and Subnetting

IPv4 Addressing and Subnetting

IPv4 addressing is the daily language of lab design: **prefixes**, **masks**, **subnets**, and **who is on-link**. Master this before multi-router labs.

Learning goals

- Read and write CIDR prefixes (`a.b.c.d/nn`) fluently
- Convert between prefix length and dotted masks
- Design non-overlapping lab subnets and point-to-point links
- Identify special-use and documentation ranges
- Configure addresses on Linux and verify with `ip`

IPv4 structure

- **32 bits**, usually dotted decimal (four 0–255 octets)
- **Prefix length** `/N` = number of network bits; host bits = `32 - N`
- **Network address**: host bits zero
- **Broadcast address** (classical Ethernet/IPv4): host bits one (except special cases like /31)

Prefix	Mask	Classical usable hosts	Common use
/32	255.255.255.255	1	Host route, loopback
/31	255.255.255.254	2	Point-to-point (RFC 3021)
/30	255.255.255.252	2	Classic P2P
/29	255.255.255.248	6	Small segment
/24	255.255.255.0	254	Common LAN
/16	255.255.0.0	many	Summaries / big labs
/8	255.0.0.0	huge	Rare in labs

CIDR and masks

```
192.168.10.0/24
network: 192.168.10.0
mask:    255.255.255.0
hosts:   192.168.10.1 - 192.168.10.254
bcast:   192.168.10.255
```

```
# Linux: add address
ip addr add 192.168.10.10/24 dev eth1
ip -br addr
```

On-link decision

If the destination is inside a **connected** prefix, the host resolves L2 (ARP) on that interface. Otherwise it looks up a **route** (often default via gateway).

```
ip route get 192.168.10.50
ip route get 8.8.8.8
```

Private and documentation ranges

Range	Role
`10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`	Private (RFC 1918)—default lab space
`192.0.2.0/24`, `198.51.100.0/24`, `203.0.113.0/24`	Documentation (TEST-NET)—safe in examples
`127.0.0.0/8`	Loopback
`169.254.0.0/16`	Link-local (APIPA)
`224.0.0.0/4`	Multicast

Lab hygiene: pick a documented plan (`10.20.0.0/16` etc.) and never invent overlapping subnets "for now."

Subnetting for multi-tier labs

Example allocation from `10.20.0.0/16`:

Use	Block
Access / users A	`10.20.1.0/24`
Access / users B	`10.20.2.0/24`

Use	Block
Servers	`10.20.10.0/24`
P2P core	`10.20.255.0/24` carved as `/31` or `/30`
Loopbacks	`10.20.254.0/24` as `/32`s

Rules:

1. No accidental overlap
2. Leave growth room
3. Write the plan **before** deploy
4. Align future summaries with real topology

Point-to-point links

Style	Example	Notes
/31	`10.20.255.0/31` → `.0` and `.1`	Preferred in modern designs
/30	`10.20.255.0/30` → `.1` and `.2` usable	Classic textbooks

Summarization (intro)

Advertising `10.20.0.0/22` for four /24s only works if you **own** the whole /22. Bad summaries **blackhole** traffic for space you do not have.

Action	Risk
Summary too broad	Blackholes
No aggregation ever	Table growth later (IGP/BGP scale)

Layer 3 part continues summarization drills; foundations need the danger signal.

VLSM

Variable Length Subnet Masking: different prefix lengths in one design (`/24` LAN + `/31` core). Required for efficient labs—do not force everything to /24.

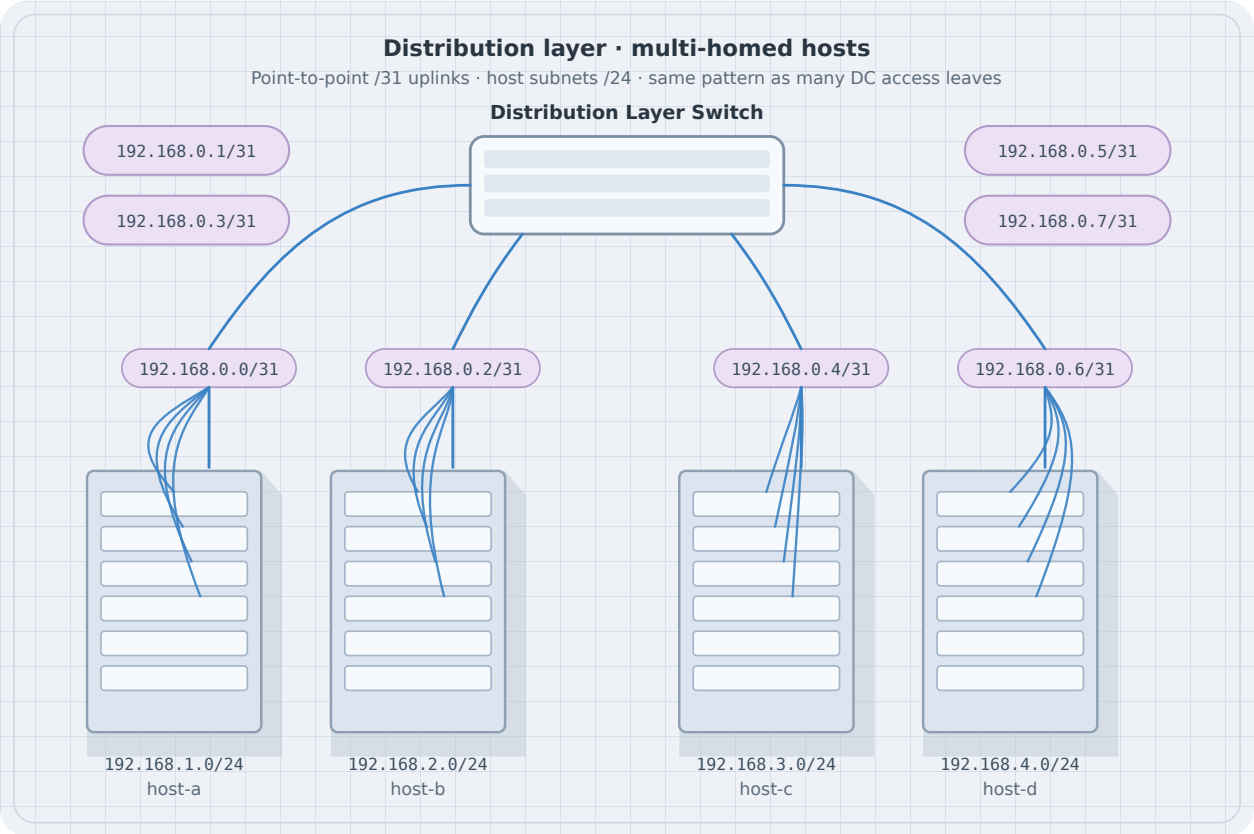


Figure 1: Dist switch multi-home

Host configuration checklist

1. Address + prefix on the correct interface
2. Up state (`ip link set eth1 up`)
3. Gateway if leaving the subnet
4. DNS only if apps need names (separate from pure L3 reachability)

```
ip addr add 10.20.1.10/24 dev eth1
ip link set eth1 up
ip route add default via 10.20.1.1
ip -br a
ip route
```

Worked subnet: /26 from a /24

Parent: `192.168.10.0/24` (256 addresses). Split into four `/26`s (64 addresses each):

Block	Network	Usable hosts	Broadcast
0	`192.168.10.0/26`	`.1`–`.62`	`.63`
1	`192.168.10.64/26`	`.65`–`.126`	`.127`
2	`192.168.10.128/26`	`.129`–`.190`	`.191`
3	`192.168.10.192/26`	`.193`–`.254`	`.255`

Host bits: `/26` → 6 host bits → $2^6 = 64$ addresses → 62 usable (reserve network + broadcast) on Ethernet-style LANs. Point-to-point often uses `/31` (RFC 3021) or `/30`.

Longest-prefix match: if a router has both `10.0.0.0/16` and `10.1.2.0/24`, a packet to `10.1.2.5` uses the `/24`. More specific wins—plan summarization so you do not punch accidental holes.

Private vs public (RFC 1918)

Range	Typical use
`10.0.0.0/8`	Large private / DC
`172.16.0.0/12`	Mid private
`192.168.0.0/16`	Home/lab LANs
`100.64.0.0/10`	CGNAT (ISP shared) — not “your” private LAN plan

Lab rule: pick one private block for the whole topology and **document** the plan before cabling Containerlab.

Paper drills

1. Network, first host, last host, broadcast for `172.16.5.0/26`
2. How many /24s fit in `10.0.0.0/16`?
3. Can `10.0.1.0/24` and `10.0.1.128/25` coexist without overlap?
4. From `10.50.0.0/16`, carve three `/24`s and one `/28` for a mgmt VLAN without overlap

Checkpoint

Design addresses for: two LANs (/24 each), one router-router link (/31), two router loopbacks (/32). No overlaps; list every interface IP.

Next

- IPv6 Essentials

IPv6 Essentials

IPv6 Essentials

IPv6 is not "IPv4 with longer addresses only." Foundations need **address types**, **prefix notation**, **link-local**, and enough config to run dual-stack labs.

Learning goals

- Read compressed IPv6 addresses and `/prefix` lengths
- Distinguish link-local, unique local, global unicast
- Configure a static IPv6 address on Linux
- Know what Neighbor Discovery replaces (ARP, parts of ICMP discovery)

Why IPv6 in this book

- Dual-stack labs mirror real networks
- ND and SLAAC appear in operations
- Many overlays and DC fabrics are IPv6-capable or IPv6-underlay

You do not need to memorize every transition technology here.

Address format

- **128 bits**, eight hextets, e.g. `2001:db8:1:2:3:4:5:6`
- Compress longest zero run with `::` (once)
- Prefix: `2001:db8:1::/64` (common LAN length is **/64**)

Documentation prefix: `2001:db8::/32` (use in examples).

Address kinds (must know)

Kind	Pattern / notes	Role
Link-local	`fe80::/10`	On-link only; required on every interface
Unique local (ULA)	`fc00::/7` (commonly `fd00::/8`)	Private-like lab space
Global unicast	`2000::/3` (in practice)	Internet-routable space
Multicast	`ff00::/8`	One-to-many
Loopback	`::1`	Local host
Unspecified	`::`	"Any" / not set

```
ip -6 addr
ip -6 route
```

Interface identifiers and /64

LAN segments almost always use **/64**. Host bits can come from:

- Manual config
- SLAAC (router advertisements)
- DHCPv6 (where used)

EUI-64 and privacy extensions exist; labs often set **static** addresses for clarity.

Linux static example

```
ip -6 addr add 2001:db8:1::10/64 dev eth1
ip link set eth1 up
ip -6 route add default via 2001:db8:1::1 dev eth1
ping -6 2001:db8:1::1
```

Neighbor Discovery (preview)

IPv6 uses **ND** (ICMPv6) instead of ARP:

- Neighbor Solicitation / Advertisement
- Router Solicitation / Advertisement

Details: ARP, ND, and Local Resolution.

Dual-stack thinking

Stack	Failure isolation
IPv4 works, IPv6 fails	v6 route, RA, ND, firewall ip6tables/nft
Prefer AAAA vs A	Happy Eyeballs / app order

Test **both** when dual-stack is in scope:

```
ping -4 <host>
ping -6 <host>
```

Checkpoint

Write a dual-stack plan for one LAN: IPv4 `/24`, IPv6 `/64` from `2001:db8:10::/64`, gateway `::1` / `.1`, two hosts.

Next

- ARP, ND, and Local Resolution

SLAAC, the RA M/O Flags and DHCPv6: What a Linux Host Actually Does

SLAAC, the RA M/O Flags and DHCPv6: What a Linux Host Actually Does

Learning goals

- Read a Router Advertisement (RA): the M and O flags, router lifetime, and the A and L flags plus lifetimes in each Prefix Information option.
- Know which parts the **Linux kernel** acts on (default route, on-link prefix, SLAAC address) and which it leaves to a **userspace client** (M, O, RDNSS → DHCPv6 and DNS).
- Predict what happens when a router deprecates a prefix or retires itself, including RFC 4862's two-hour rule.

Version note: run on Linux **6.12** inside an unprivileged user and network namespace (`unshare -rn`), with a veth pair between a "router" namespace and a "host" namespace. RAs were crafted with a 30-line Python script. There is no DHCPv6 server or client in the lab. The output is pasted verbatim.

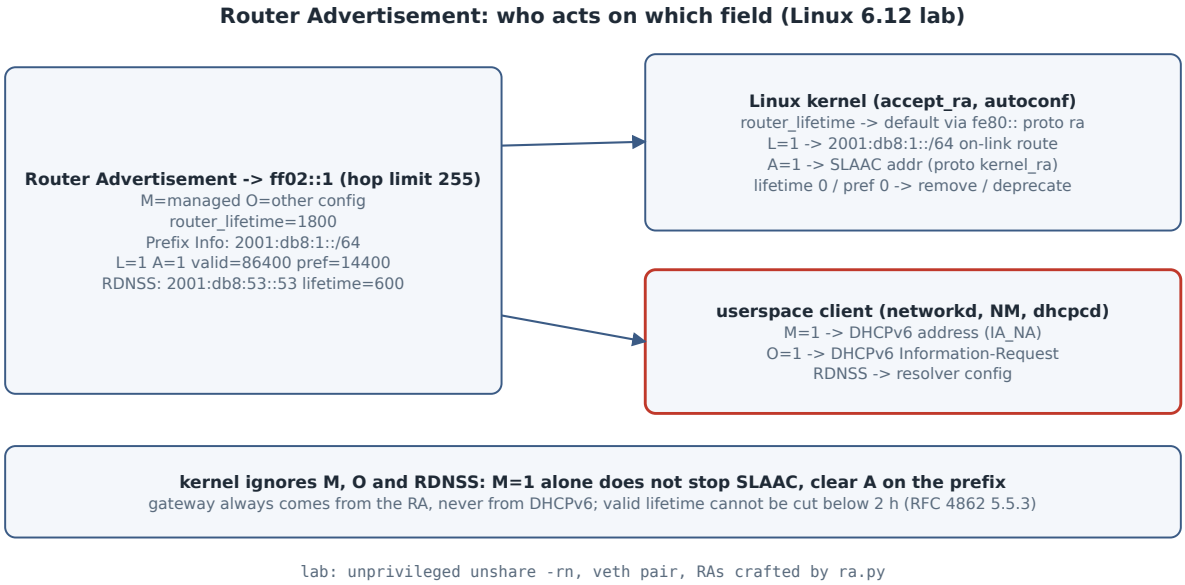


Figure 1: Who consumes each part of a Router Advertisement on a Linux host.

The three ways an IPv6 host gets an address

RA says	Meaning (RFC 4861 §4.2, RFC 4862)	Typical client behaviour
prefix `A=1`, `M=0 O=0`	SLAAC only	kernel forms address; DNS from RDNSS (RFC 8106)
prefix `A=1`, `M=0 O=1`	SLAAC + stateless DHCPv6	address via SLAAC, DNS/NTP etc. via DHCPv6 Information-Request
`M=1` (prefix `A=0` or no prefix)	stateful DHCPv6	address from DHCPv6 IA_NA. RFC 4861: "If the M flag is set, the O flag is redundant"

Two facts trip people up. First, the default gateway **never** comes from DHCPv6. It only comes from the RA router lifetime, so a "DHCPv6-only" network still needs RAs. Second, the M and O flags are hints to a userspace process. The kernel does not read them, and setting `M=1` does not stop SLAAC. Only `A=0` on the prefix does that.

Lab

`ra.py` sends one RA from the router side, with hop limit 255 as RFC 4861 requires:

```
# Send one Router Advertisement (RFC 4861 4.2) out of IFACE to ff02::1.
# usage: ra.py IFACE M O router_lifetime prefix/len A valid preferred [rdnss]
import socket, struct, sys, ipaddress
ifc, M, O, rl, pfx, A, vl, pl = sys.argv[1:9]; rdnss = sys.argv[9] if len(sys.argv) > 9 else
s = socket.socket(socket.AF_INET6, socket.SOCK_RAW, socket.IPPROTO_ICMPV6)
s.setsockopt(socket.IPPROTO_IPV6, socket.IPV6_MULTICAST_HOPS, 255)
s.setsockopt(socket.SOL_SOCKET, socket.SO_BINDTODEVICE, ifc.encode())
flags = (int(M) << 7) | (int(O) << 6)
msg = struct.pack("!BBHBBHII", 134, 0, 0, 64, flags, int(rl), 0, 0)
net = ipaddress.ip_network(pfx)
msg += struct.pack("!BBBBIII", 3, 4, net.prefixlen, 0x80 | (int(A) << 6), int(vl), int(pl),
if rdnss:
    msg += struct.pack("!BBHI", 25, 3, 0, 600) + ipaddress.ip_address(rdnss).packed
s.sendto(msg, ("ff02::1", 0, 0, socket.if_nametoindex(ifc)))
```

`rdisc.py` runs on the host and decodes every RA it receives, which is what systemd-networkd, NetworkManager or dhcpcd see:

```
# What a userspace client (NetworkManager, systemd-networkd, dhcpcd) sees: decode RAs.
import socket, struct, sys, ipaddress, select
s = socket.socket(socket.AF_INET6, socket.SOCK_RAW, socket.IPPROTO_ICMPV6)
s.setsockopt(socket.SOL_SOCKET, socket.SO_BINDTODEVICE, sys.argv[1].encode())
n = int(sys.argv[2])
while n:
    if not select.select([s], [], [], 8)[0]: break
```

```
    d, a = s.recvfrom(2000)
    if d[0] != 134: continue
    n -= 1
    _, _, _, hl, fl, rl, _, _ = struct.unpack("!BBHBBHII", d[:16])
    out = [f"RA from {a[0].split('%')[0]}: M={fl>>7&1} O={fl>>6&1} router_lifetime={rl}"]
    o = d[16:]
    while o:
        t, ln = o[0], o[1] * 8
        if t == 3:
            plen, pf, vl, pl = o[2], o[3], *struct.unpack("!II", o[4:12])
            out.append(f"prefix {ipaddress.ip_address(o[16:32])}/{plen} L={pf>>7&1} A={pf>>6
        elif t == 25:
            out.append("RDNSS " + " ".join(str(ipaddress.ip_address(o[i:i+16])) for i in ran
        o = o[ln:]
    print("   client sees: " + "; ".join(out), flush=True)
```

`lab.sh` wires the namespaces together and sends three RAs. In the host namespace it sets `forwarding=0`, `accept_ra=1` and `autoconf=1` (the defaults), plus `accept_dad=0` to make it faster:

```
#!/bin/sh
# Run as: unshare -rn ./lab.sh   (unprivileged: user + net namespace, no sudo)
set -e
unshare -n sleep 60 >/dev/null 2>&1 & H=$!; sleep 0.2
inH() { nsenter -t $H -n --preserve-credentials "$@"; }
ip link set lo up; inH ip link set lo up
ip link add r0 type veth peer name h0; ip link set h0 netns /proc/$H/ns/net
sc() { echo $2 > /proc/sys/net/ipv6/conf/$1; }
for kv in h0/accept_dad:0 h0/forwarding:0 h0/accept_ra:1 h0/autoconf:1; do inH sh -c "echo $
sc r0/accept_dad 0
ip link set r0 up; inH ip link set h0 up; sleep 1
echo "== host h0 before any RA"; inH ip -6 addr show dev h0 | grep inet6; inH ip -6 route sh
inH python3 rdisc.py h0 3 & C=$!; sleep 0.3
show() { sleep 0.5; inH ip -6 addr show dev h0 | grep -A1 'inet6 2001' | sed 's/^/   /'; inH
echo "-- 1) SLAAC only: M=0 O=0, prefix A=1, RDNSS 2001:db8:53::53"
python3 ra.py r0 0 0 1800 2001:db8:1::/64 1 86400 14400 2001:db8:53::53; show
echo "-- 2) stateful DHCPv6 signalled: M=1 O=1, new prefix 2001:db8:2::/64 with A=0 (on-link
python3 ra.py r0 1 1 1800 2001:db8:2::/64 0 86400 14400; show
echo "-- 3) router retires itself: router_lifetime=0, prefix 2001:db8:1::/64 valid=7200 pref
python3 ra.py r0 0 0 0 2001:db8:1::/64 1 7200 0; show
echo "== resolv.conf inside host netns is untouched by the kernel:"; inH grep -c 2001:db8:53
wait $C; kill $H
```

Output of `unshare -rn ./lab.sh`:

```
== host h0 before any RA
    inet6 fe80::181a:3cff:fe93:9810/64 scope link proto kernel_ll
fe80::/64 proto kernel metric 256 pref medium
-- 1) SLAAC only: M=0 O=0, prefix A=1, RDNSS 2001:db8:53::53
   client sees: RA from fe80::4873:c0ff:feb0:3069: M=0 O=0 router_lifetime=1800; prefix 2001
       inet6 2001:db8:1:0:181a:3cff:fe93:9810/64 scope global dynamic mngtmpaddr proto kerne
          valid_lft 86400sec preferred_lft 14400sec
   2001:db8:1::/64 dev h0 proto kernel metric 256 expires 86399sec pref medium
   default via fe80::4873:c0ff:feb0:3069 dev h0 proto ra metric 1024 expires 1799sec hoplimi
-- 2) stateful DHCPv6 signalled: M=1 O=1, new prefix 2001:db8:2::/64 with A=0 (on-link only)
   client sees: RA from fe80::4873:c0ff:feb0:3069: M=1 O=1 router_lifetime=1800; prefix 2001
       inet6 2001:db8:1:0:181a:3cff:fe93:9810/64 scope global dynamic mngtmpaddr proto kerne
          valid_lft 86399sec preferred_lft 14399sec
   2001:db8:1::/64 dev h0 proto kernel metric 256 expires 86398sec pref medium
   2001:db8:2::/64 dev h0 proto kernel metric 256 expires 86399sec pref medium
   default via fe80::4873:c0ff:feb0:3069 dev h0 proto ra metric 1024 expires 1799sec hoplimi
-- 3) router retires itself: router_lifetime=0, prefix 2001:db8:1::/64 valid=7200 pref=0 (de
   client sees: RA from fe80::4873:c0ff:feb0:3069: M=0 O=0 router_lifetime=0; prefix 2001:db
       inet6 2001:db8:1:0:181a:3cff:fe93:9810/64 scope global deprecated dynamic mngtmpaddr
          valid_lft 7200sec preferred_lft 0sec
   2001:db8:1::/64 dev h0 proto kernel metric 256 expires 7199sec pref medium
   2001:db8:2::/64 dev h0 proto kernel metric 256 expires 86398sec pref medium
== resolv.conf inside host netns is untouched by the kernel:
0
```

Reading the output

- **RA 1 (SLAAC).** The kernel did three things, each tagged with its origin. It formed `2001:db8:1:0:181a:3cff:fe93:9810` (`proto kernel_ra`, `mngtmpaddr`) from the prefix plus a modified EUI-64 interface ID. That is the `ff:fe` in the middle, because `addr_gen_mode` defaults to 0 in a fresh namespace. It added the on-link route `2001:db8:1::/64` from `L=1`, and it added `default via fe80::… proto ra expires 1799sec` from the router lifetime. Lifetimes are copied straight across (`valid_lft 86400sec preferred_lft 14400sec`).
- **The kernel ignored RDNSS.** The client decoded `RDNSS 2001:db8:53::53 lifetime=600`, but the host's `/etc/resolv.conf` has zero mentions of it. DNS from RAs is entirely a userspace job (RFC 8106), so a minimal system with no RA-aware daemon has IPv6 addresses and routes but no IPv6 DNS servers.
- **RA 2 (M=1 O=1).** No address was formed for `2001:db8:2::/64` (`A=0`), but its on-link route was installed (`L=1`). The existing SLAAC address stayed. Nothing in the kernel reacted to `M=1`. Starting DHCPv6 is up to the client daemon, which saw `M=1 O=1`.
- **RA 3 (retire and deprecate).** `router_lifetime=0` removed the default route immediately: it is missing from the last listing. `pref=0` made the address `deprecated` (`preferred_lft 0sec`). It still works for existing connections but is not chosen for new ones. The valid lifetime went to `7200sec`. That matches RFC 4862 §5.5.3(e): an unauthenticated RA cannot cut a remaining valid lifetime below **2 hours**. A received value above 2 h, or above the time

remaining, is taken as-is. Otherwise the valid lifetime is set to 2 hours. This is what stops a single forged RA from wiping addresses.

What the kernel knobs mean

From the kernel ip-sysctl documentation:

- `accept_ra`: "Accept Router Advertisements; autoconfigure using them." With 1, RAs are accepted only if forwarding is disabled. With 2, they are accepted even if forwarding is on, which is what a router that also learns its upstream prefix from an RA needs.
- `autoconf`: "Autoconfigure addresses using Prefix Information in Router Advertisements." Turning it off stops SLAAC addresses but keeps the RA default route.
- `addr_gen_mode`, `use_tempaddr`: choose EUI-64, stable-privacy (RFC 7217) or temporary (RFC 8981) interface IDs. Most distributions set stable-privacy through their network manager.

Practical rules

- When debugging "no IPv6", check the RA first. Run `ip -6 route show default` and look for `proto ra`: if it is missing, no RA arrived or `accept_ra` is off (forwarding enabled?).
- When debugging "IPv6 address but no IPv6 DNS", check whether an RDNSS-aware daemon is running. The kernel will not help.
- To get DHCPv6-only addressing, send `M=1` **and** clear `A` on the prefix, and keep RAs with a non-zero router lifetime for the gateway.
- When renumbering, send `preferred=0` for the old prefix and expect it to linger for up to 2 hours, by design.

Try this

1. Re-run with `echo 2 > /proc/sys/net/ipv6/conf/h0/accept_ra` and `forwarding` set to 1 in the host namespace, then compare with `accept_ra=1`.
2. Set `addr_gen_mode` to 2 (stable-privacy, which needs `stable_secret`) or 3 (random) on `h0` before the RA. How does the interface ID change?
3. Send RA 3 with `valid=60` instead of 7200. What valid lifetime does the address end up with, and why?

Sources

- RFC 4861, Neighbor Discovery for IPv6, §4.2 Router Advertisement: https://www.rfc-editor.org/rfc/rfc4861#section-4.2
- RFC 4862, IPv6 Stateless Address Autoconfiguration, §5.5.3: https://www.rfc-editor.org/rfc/rfc4862#section-5.5.3

- RFC 8106, IPv6 RA Options for DNS Configuration (RDNSS): https://www.rfc-editor.org/rfc/rfc8106
- RFC 8415, DHCPv6: https://www.rfc-editor.org/rfc/rfc8415
- Linux kernel documentation, IP sysctl (`accept_ra`, `autoconf`, `addr_gen_mode`): https://docs.kernel.org/networking/ip-sysctl.html

ARP, ND, and Local Resolution

ARP, ND, and Local Resolution

On multi-access links, IP needs a **local** next-hop mapping: IPv4 uses **ARP**; IPv6 uses **Neighbor Discovery (ND)**. If resolution fails, routing never gets a chance.

Learning goals

- Explain when ARP/ND runs (on-link destinations and gateways)
- Read and clear neighbor tables on Linux
- Spot incomplete entries and wrong-subnet mistakes
- Capture ARP or NS/NA in a lab

When local resolution happens

1. Host decides destination is **on-link** (connected route), or
2. Host needs the **gateway's MAC** for an off-link destination

Then: resolve next-hop IP → MAC; build Ethernet header; send.

ARP (IPv4)

Message	Role
Who-has (request)	Broadcast: "Who has IP X? Tell Y"
Is-at (reply)	Unicast: "X is at MAC M"

```
ip neigh show
ip neigh show dev eth1
ping -c1 192.168.10.1
ip neigh show
# force re-resolve
ip neigh flush dev eth1
```

Incomplete / failed ARP

Cause	What you see
Target down	Incomplete, then failed
Wrong VLAN / segment	No reply
Wrong mask (thinking on-link when not)	ARP for remote IPs on LAN
Firewall drop	Silence

Neighbor Discovery (IPv6)

Message (ICMPv6)	Role
Neighbor Solicitation	Resolve or probe neighbor
Neighbor Advertisement	Provide or confirm MAC
Router Solicitation / Advertisement	Find routers; SLAAC info

```
ip -6 neigh show
ping -6 2001:db8:1::1
```

Gratuitous ARP and updates

Hosts may announce their own IP/MAC (gratuitous ARP) on address change. Useful for failover designs later (first-hop redundancy).

Proxy ARP (awareness)

A router answering ARP for non-local addresses can hide topology mistakes—or create blackholes. Prefer **correct masks and routes** over proxy ARP as a habit.

Lab sketch

1. Two hosts + gateway on `/24`
2. `tcpdump -ni eth1 arp` while first ping runs
3. Wrong mask on host: watch ARP for remote destinations

Checkpoint

Host `10.0.0.10/24`, gateway `10.0.0.1`. Destination `10.0.1.10`. Which IP is ARP'd for? Why?

Next

- ICMP and Path Diagnostics

ICMP and Path Diagnostics

ICMP and Path Diagnostics

ICMP (and ICMPv6) signal errors and support diagnostics. `ping` and `traceroute` are applications of those messages—not separate magic protocols.

Learning goals

- Map common ICMP types to meaning
- Use ping/traceroute with clear interpretation limits
- Understand TTL expiry and "why traceroute hops appear"
- Preview Path MTU Discovery

ICMPv4 types (operator set)

Type	Name	Typical meaning
0 / 8	Echo reply / request	Ping
3	Destination unreachable	No route, port unreachable, admin prohibited, …
11	Time exceeded	TTL expired (traceroute hops)

Codes refine type 3 (network unreachable, host unreachable, etc.).

ICMPv6

Similar roles: echo, destination unreachable, **packet too big**, time exceeded, ND messages (see previous chapter).

```
ping -c3 192.0.2.1
ping -6 -c3 2001:db8::1
```

Interpreting ping

Result	Possible meaning
Reply	Forward path + return path + ICMP allowed
Timeout	Drop, filter, wrong route, silent host
Unreachable	Explicit ICMP error from some node

Ping success application success. TCP port filters can still block SSH/HTTP.

Traceroute mechanics (conceptual)

1. Send probes with TTL=1, 2, 3, …
2. Each hop that decrements TTL to 0 should send **time exceeded**
3. Destination eventually responds (port unreachable / echo)

```
traceroute 192.0.2.1
# or
mtr -r -c 5 192.0.2.1
```

Limits: load balancers, ICMP rate limits, firewalls that hide hops, asymmetric return paths.

Path MTU Discovery (preview)

- Senders prefer **not** to fragment (IPv6: no in-path fragmentation)
- Hop returns **Packet Too Big** / frag-needed if DF set and MTU smaller
- Broken PMTUD → hangs for large transfers while small pings work

Deep labs: Layer 3 ICMP & Path MTU chapter.

Filters and policy

Many networks rate-limit or block ICMP. That improves some attacks and **hurts** diagnostics and PMTUD. Lab hygiene: know your nftables/iptables ICMP rules.

```
# Example observability only-do not paste blindly into production
nft list ruleset
iptables -L -n
```

Lab sketch

1. Three-node line (A—B—C)
2. Ping A→C; traceroute A→C
3. ACL drop ICMP on B: compare symptoms

Checkpoint

Why can `ping` work while `curl http://…` fails? Why can `traceroute` show `* * *` on middle hops while the destination still answers?

Next

- Transport: TCP and UDP

Transport: TCP and UDP

Transport: TCP and UDP

The **transport layer** delivers data to the correct **process** on a host using **ports**, with different reliability and connection models.

Learning goals

- Explain 5-tuples and port roles
- Contrast TCP and UDP services
- Read basic TCP handshake and teardown signals
- Use `ss` to inspect listeners and connections

Ports and sockets

- Port: 16-bit number (0–65535)
- **Well-known** ports (e.g. 22 SSH, 53 DNS, 80 HTTP) vs **ephemeral** client ports
- Socket: local address + port (+ protocol), often with peer

5-tuple: protocol, src IP, src port, dst IP, dst port — identifies a conversation.

```
ss -tulpn
ss -tan
ss -uan
```

UDP

Property	UDP
Connection	None
Reliability	None (app may add)
Ordering	Not guaranteed
Header	Lightweight
Use cases	DNS, DHCP, many tunnels, telemetry, real-time

```
# one-shot listener / client ideas
nc -u -l -p 9999
nc -u <peer> 9999
```

TCP

Property	TCP
Connection	Yes (stateful)
Reliability	Ack, retransmit
Ordering	Byte stream reassembly
Flow / congestion control	Yes
Use cases	SSH, HTTP, BGP, config APIs

Three-way handshake (must know)

1. Client → Server: **SYN**
2. Server → Client: **SYN-ACK**
3. Client → Server: **ACK**

Then data can flow. Teardown uses **FIN/ACK** (or **RST** for abort).

```
tcpdump -ni eth1 'tcp port 22'
```

States (operator view)

State	Meaning
LISTEN	Waiting for connections
SYN-SENT / SYN-RECV	Handshake in progress
ESTABLISHED	Data transfer
TIME-WAIT	Waiting to be sure old segments die
CLOSE-WAIT / FIN-WAIT	Teardown in progress

TCP vs UDP selection

Need	Prefer
Reliable byte stream	TCP

Need	Prefer
Simple request/response, app handles loss	UDP
Multicast / lightweight tunnel outer	Often UDP
Head-of-line sensitivity	Design carefully (HTTP/2 vs QUIC later topics)

NAT and transports (preview)

NATS track **transport** state (especially TCP). UDP mappings time out differently. Edge/NAT chapter expands this.

Security basics

- **Do not** expose admin TCP ports to untrusted networks without policy
- Spoofing and scan noise are normal on open nets
- TLS usually rides **on TCP** (or QUIC on UDP)—app layer crypto

TCP state machine (operator view)

```
CLOSED → SYN_SENT → ESTABLISHED → FIN_WAIT_* → TIME_WAIT → CLOSED
                (listen) LISTEN → SYN_RCVD → ESTABLISHED → CLOSE_WAIT → LAST_ACK → CLOSED
```

State you see (`ss` / `netstat`)	Meaning
`LISTEN`	Server bound, waiting for SYN
`ESTAB` / `ESTABLISHED`	Data can flow
`TIME-WAIT`	Local side closed; holding tuple so delayed segments die (normal after busy services)
`CLOSE-WAIT`	Peer closed; **your app** must close its side (often a bug if stuck)
`SYN-SENT`	Client waiting for SYN-ACK (firewall/blackhole if stuck)

Flow control vs congestion control: window advertisements limit *receiver* buffer pressure; congestion control (slow start, Reno/CUBIC, …) limits *network* load. Both shrink effective throughput; do not "tune sysctls" before measuring.

UDP: no handshake, no stream—each datagram is independent. Good for DNS, DHCP, QUIC's outer path, and lab pings of custom apps. Reliability, if needed, is **your** application's job.

Three-way handshake and teardown (why captures look busy)

1. Client → Server: `SYN` (seq=c)
2. Server → Client: `SYN-ACK` (seq=s, ack=c+1)
3. Client → Server: `ACK` (ack=s+1)

Teardown is often four segments (`FIN/ACK` each way) or `RST` on abort. In labs, filter `tcp.flags.syn==1 or tcp.flags.fin==1 or tcp.flags.reset==1` in Wireshark/tshark to see only control.

Lab sketch

1. `ss -tulpn` on a node before/after starting SSH
2. Capture handshake for SSH (`tcpdump -ni any port 22`)
3. `nc` UDP echo between two lab hosts
4. Force a stuck `CLOSE-WAIT` by killing only one side of a netcat TCP session; observe with `ss -tan`
5. Note `TIME-WAIT` count after many short connections (`ss -tan state time-wait | wc -l`)

Checkpoint

A client opens HTTPS to a server. Name protocol numbers/fields for: Ethernet type, IP protocol, TCP ports (client ephemeral + 443), and where TLS sits.

Next

- Deep dive: Small Writes in TCP: Nagle's Algorithm, Delayed ACKs, and the 40 ms Stall
- Common Application Protocols

Small Writes in TCP: Nagle's Algorithm, Delayed ACKs, and the 40 ms Stall

Small Writes in TCP: Nagle's Algorithm, Delayed ACKs, and the 40 ms Stall

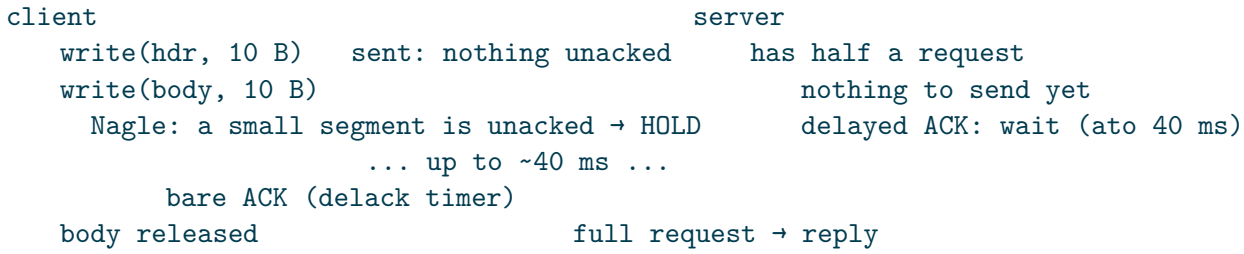
Two TCP features that each save packets can together add a fixed **40 ms** to every request. **Nagle's algorithm** (sender side) holds a small segment while an earlier small segment is still unacknowledged. **Delayed ACK** (receiver side) waits, up to a timer, before acknowledging, hoping to piggyback the ACK on a reply. When a client sends one request as *two* small writes and then waits for the answer, the server cannot reply (it has only half the request), so it delays its ACK; the client cannot send the second half until that ACK arrives. Both sides wait for the timer. Nothing is lost, nothing is retransmitted, CPU is idle, and p50 latency is 40 ms on a link with a 20 µs RTT. This chapter reproduces the stall on loopback, catches it in `ss`, and measures every fix, including the one people reach for first (`TCP_NODELAY`) and what it costs.

Learning goals

- State Nagle's rule (RFC 896, RFC 9293 §3.7.4) and Linux's Minshall variant, and the delayed-ACK rule (RFC 9293 §3.8.6.3)
- Recognise the write-write-read pattern and predict the stall from it
- Read `ato`, `unacked` and `notsent` in `ss -ti` while the stall is happening
- Compare the fixes: one write (`writev`/`sendmsg`), `TCP_CORK`, `MSG_MORE`, `TCP_NODELAY` and `TCP_QUICKACK`, and say which side each one acts on
- Explain what Nagle protects against, and measure what `TCP_NODELAY` costs in segments

Lab platform: Linux 6.12, iproute2 6.15 (`ss`, `nstat`), Python 3.13.5, run as an unprivileged user over `127.0.0.1`. No namespaces, no `tc`, no root. Every listing output below is pasted from those runs. Kernel behaviour is quoted from `include/net/tcp.h` and `net/ipv4/tcp_output.c` in Linux 6.12; the same constants and `tcp_nagle_check()` are unchanged in Linux 7.2, the current stable series.

Mental model



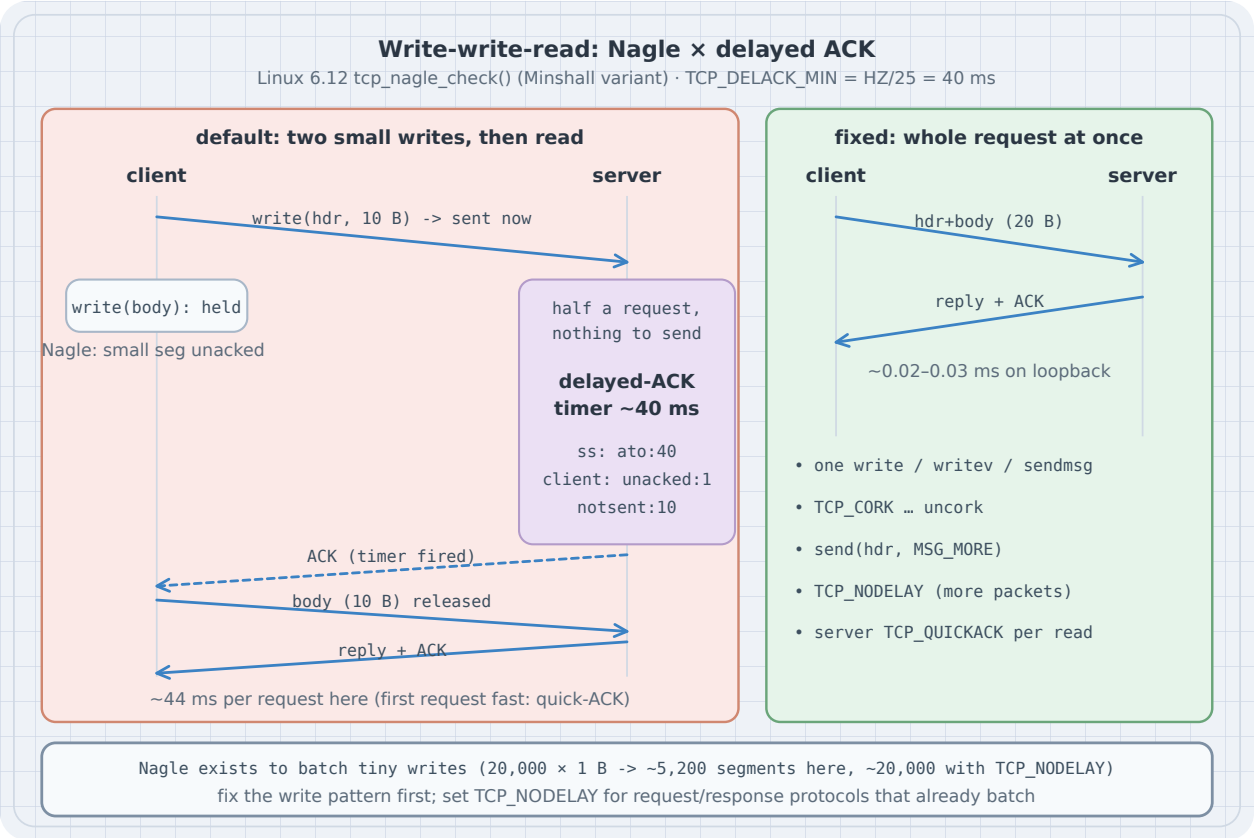


Figure 1: Write-write-read: Nagle × delayed ACK

```
                reply (+ ACK)
```

The two rules:

Rule	Where	What it says	Linux
Nagle (RFC 896; RFC 9293 §3.7.4, SHLD-7, with a per-connection off switch, MUST-17)	Sender	While data is unacknowledged, buffer small writes until the ACK arrives or a full MSS is ready	`tcp_nagle_check()` with **Minshall's variant** (RFC 9293 App. A.3): hold a sub-MSS segment only while a previous *sub-MSS* segment is unacknowledged
Delayed ACK (RFC 1122 §4.2.3.2; RFC 9293 §3.8.6.3, SHLD-18/19, MUST-40)	Receiver	ACK at least every second full-sized segment; otherwise wait (less than 0.5 s) to piggyback the ACK on data	`TCP_DELACK_MIN` = `TCP_ATO_MIN` = `HZ/25` = **40 ms**, `TCP_DELACK_MAX` = `HZ/5` = 200 ms; a new connection starts in **quick-ACK** mode (up to `TCP_MAX_QUICKACKS` = 16 immediate ACKs)

Neither rule is a bug. Together they deadlock until the timer fires, whenever all of these hold:

1. The sender writes one logical message as **two or more sub-MSS writes**.
2. The receiver **cannot respond** until it has the whole message.
3. The sender **waits for the response** before writing more.

HTTP/1.x clients that write headers and body separately, RPC frameworks that write a length prefix and then the payload, database drivers, Redis/memcached clients and SSH-like interactive protocols all fit this pattern unless they batch their writes or set `TCP_NODELAY`.

Concepts — the knobs

Knob	Side	Scope	Effect
One write: build the message in user space, or `writev()`/`sendmsg()` with an iovec	Sender	Per message	The message leaves as one segment; Nagle has nothing to hold. Portable.

Knob	Side	Scope	Effect
`TCP_CORK` (Linux)	Sender	Until cleared	Hold partial segments; clearing it sends them now. 200 ms ceiling. Overrides `TCP_NODELAY`.
`MSG_MORE` flag on `send()` (Linux)	Sender	One call	Same as `TCP_CORK` for that call: "more is coming"; the next send without the flag pushes.
`TCP_NODELAY`	Sender	Socket	Disable Nagle: every write is sent as soon as cwnd/rwnd allow. Portable (RFC 9293 MUST-17).
`TCP_QUICKACK` (Linux)	Receiver	**Not sticky**	Enter quick-ACK mode now. The stack drops back to delayed ACKs on its own (`tcp(7)`: "This flag is not permanent").
`net.ipv4.tcp_autocorking` (default 1)	Sender	Host	Coalesce small writes while an earlier packet is still queued in the host. Counter `TcpExtTCPAutoCorking`.
`ip route ... quickack 1`	Receiver	Route	Quick-ACK for every connection on that route (needs root; not run here).

Lab — reproduce the stall and measure the fixes

```
# nagle_lab.py - write-write-read over loopback: the Nagle x delayed-ACK stall, and the ways
# usage: python3 nagle_lab.py [REQUESTS]
import socket, statistics, struct, sys, threading, time

HDR, BODY = b"H" * 10, b"B" * 10          # one request, sent as two small writes
REQ, RESP = len(HDR) + len(BODY), b"R" * 20
N = int(sys.argv[1]) if len(sys.argv) > 1 else 50

def data_segs_out(s):                     # struct tcp_info -> tcpi_data_segs_out
    f = struct.unpack("8B24I4Q6I", s.getsockopt(socket.IPPROTO_TCP, socket.TCP_INFO, 160))
```

```
    return f[41]

def recv_exact(s, n):
    buf = b""
    while len(buf) < n:
        chunk = s.recv(n - len(buf))
        if not chunk:
            raise ConnectionError("peer closed")
        buf += chunk
    return buf

def server(lsock, quickack):
    c, _ = lsock.accept()
    if quickack == "once":
        c.setsockopt(socket.IPPROTO_TCP, socket.TCP_QUICKACK, 1)
    for _ in range(N):
        if quickack == "every read":      # not sticky: re-arm before every read
            c.setsockopt(socket.IPPROTO_TCP, socket.TCP_QUICKACK, 1)
        recv_exact(c, REQ)
        c.sendall(RESP)
    c.close()

def run(mode):
    lsock = socket.create_server(("127.0.0.1", 0))
    quickack = {"QUICKACK once": "once", "QUICKACK every read": "every read"}.get(mode)
    t = threading.Thread(target=server, args=(lsock, quickack))
    t.start()
    s = socket.create_connection(lsock.getsockname())
    if mode == "TCP_NODELAY":
        s.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, 1)
    lat = []
    for _ in range(N):
        t0 = time.perf_counter()
        if mode == "one write":
            s.sendall(HDR + BODY)
        elif mode == "TCP_CORK":
            s.setsockopt(socket.IPPROTO_TCP, socket.TCP_CORK, 1)
            s.sendall(HDR); s.sendall(BODY)
            s.setsockopt(socket.IPPROTO_TCP, socket.TCP_CORK, 0)   # uncork = send now
        elif mode == "MSG_MORE":
            s.send(HDR, socket.MSG_MORE); s.sendall(BODY)
        else:                             # default, TCP_NODELAY, server-side QUICKACK
            s.sendall(HDR); s.sendall(BODY)
        recv_exact(s, len(RESP))
        lat.append((time.perf_counter() - t0) * 1e3)
    segs = data_segs_out(s)
```

```
    s.close(); t.join(); lsock.close()
    first = " ".join(f"{x:.2f}" for x in lat[:3])
    lat.sort()
    print(f"{mode:19} first3 [{first}]  median {statistics.median(lat):6.3f} ms"
          f"  p90 {lat[int(0.9 * N)]:6.3f} ms  total {sum(lat):6.1f} ms  data segs {segs:3}"

print(f"{N} requests of {REQ} bytes (two writes of 10), {len(RESP)}-byte reply, 127.0.0.1")
for mode in ("Nagle (default)", "TCP_NODELAY", "one write", "TCP_CORK", "MSG_MORE",
             "QUICKACK once", "QUICKACK every read"):
    run(mode)
```

```
nstat -n                  # reset the nstat history for this network namespace
python3 nagle_lab.py
nstat TcpExtDelayedACKs   # delayed ACKs sent since the reset
```

```
50 requests of 20 bytes (two writes of 10), 20-byte reply, 127.0.0.1
Nagle (default)     first3 [0.26 42.26 44.12]  median 44.014 ms  p90 44.163 ms  total 2154.4
TCP_NODELAY         first3 [0.43 0.38 0.49]  median  0.031 ms  p90  0.070 ms  total    2.9 m
one write           first3 [0.25 0.04 0.06]  median  0.029 ms  p90  0.032 ms  total    1.7 m
TCP_CORK            first3 [0.10 0.04 0.03]  median  0.031 ms  p90  0.034 ms  total    1.6 m
MSG_MORE            first3 [0.08 0.04 0.03]  median  0.030 ms  p90  0.033 ms  total    1.6 m
QUICKACK once       first3 [0.10 40.85 43.98]  median 43.990 ms  p90 44.201 ms  total 2152.9
QUICKACK every read first3 [0.10 0.06 0.04]  median  0.036 ms  p90  0.039 ms  total    1.9 m
#kernel
TcpExtDelayedACKs               134                0.0
```

Read it line by line:

- **Nagle (default):** the first request takes 0.26 ms: the connection is fresh and in quick-ACK mode, so the server ACKs the header at once. From the second request on, every request takes about **44 ms**: the 40 ms minimum delayed-ACK timeout plus a few milliseconds of timer granularity. Fifty requests take 2.15 s instead of 2 ms. The client sent 100 data segments: header and body separately.
- **TCP_NODELAY:** same two writes, same 100 segments, but the body is not held: median 31 µs.
- **one write / TCP_CORK / MSG_MORE:** the request leaves as **one** segment (50 data segments for 50 requests) and the stall is gone. This is the best result: fast *and* half the packets.
- **QUICKACK once:** setting `TCP_QUICKACK` on the server right after `accept()` does nothing useful. The connection is already in quick-ACK mode then, and it leaves that mode by itself after a few exchanges: the stall starts at request two, exactly like the default.
- **QUICKACK every read:** re-arming it before every `recv()` fixes the stall from the receiver side, with 100 segments. That works when you own the server but not the client; it is also Linux-only and costs a system call per read.
- `TcpExtDelayedACKs` counted 134: about 98 from the two stalling runs (49 stalled requests each), and the rest from other traffic in this shared network namespace. `nstat` counters are per network namespace, not per process.

Lab — catch the stall in ss

```
# catch_stall.py - freeze the moment of the stall and look at both sockets with ss -ti
import socket, subprocess, threading, time

lsock = socket.create_server(("127.0.0.1", 0))
port = lsock.getsockname()[1]
ready = threading.Event()

def server():
    c, _ = lsock.accept()
    for _ in range(3):                      # warm up: leave the initial quick-ACK phase
        c.recv(20); c.sendall(b"R" * 20)
    ready.set()
    c.recv(10)                              # got the header, still waiting for the body
    time.sleep(0.2); c.close()

t = threading.Thread(target=server); t.start()
s = socket.create_connection(("127.0.0.1", port))
for _ in range(3):
    s.sendall(b"Q" * 20); s.recv(20)
ready.wait()
s.sendall(b"H" * 10)                        # goes out at once: nothing unacknowledged
s.sendall(b"B" * 10)                        # Nagle holds this: a small segment is unacked
time.sleep(0.005)                           # 5 ms into the up-to-40 ms wait
out = subprocess.run(["ss", "-tniH", "state", "established", f"( sport = :{port} or dport =
                      capture_output=True, text=True).stdout
keep = ("ato", "mss", "unacked", "notsent", "lastsnd", "lastack", "bytes_sent", "bytes_recei
lines = out.splitlines()
for addrs, info in zip(lines[0::2], lines[1::2]):
    local, peer = addrs.split()[2:4]
    who = "server" if local.endswith(f":{port}") else "client"
    print(f"{who}  {local} -> {peer}")
    print("        " + " ".join(w for w in info.split() if w.split(":")[0] in keep))
s.close(); t.join()
```

```
python3 catch_stall.py
```

```
server  127.0.0.1:45499 -> 127.0.0.1:45560
        ato:40 mss:32768 bytes_sent:60 bytes_received:70 lastsnd:8 lastack:8
client  127.0.0.1:45560 -> 127.0.0.1:45499
        ato:40 mss:32768 bytes_sent:70 bytes_received:60 lastsnd:8 lastack:8 unacked:1 notse
```

Eight milliseconds into the stall the evidence is in two fields. On the **client**: `unacked:1` (the header segment is in flight, unacknowledged) and **`notsent:10`** (the 10-byte body is in the send queue and Nagle has not sent it). On the **server**: `ato:40`, the delayed-ACK timeout, and nothing to

send. `bytes_sent:70` on the client is three 20-byte warm-up requests plus the header; the body is not counted until it leaves. In production, `ss -tni` showing a small, persistent `notsent` on a request/response socket whose peer shows `ato:40` is the stall. A packet capture shows it as a 40 ms gap between a bare ACK from the server and the client's next small segment.

Lab — what Nagle is for

```
# tiny_writes.py - what Nagle is FOR: 20,000 one-byte writes, counted in segments on the wir
import socket, struct, threading, time

WRITES = 20_000

def data_segs_out(s):
    f = struct.unpack("8B24I4Q6I", s.getsockopt(socket.IPPROTO_TCP, socket.TCP_INFO, 160))
    return f[41]

def drain(lsock, got):
    c, _ = lsock.accept()
    n = 0
    while chunk := c.recv(65536):
        n += len(chunk)
    got.append(n); c.close()

for nodelay in (0, 1, 0, 1):
    lsock = socket.create_server(("127.0.0.1", 0)); got = []
    t = threading.Thread(target=drain, args=(lsock, got)); t.start()
    s = socket.create_connection(lsock.getsockname())
    s.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, nodelay)
    t0 = time.perf_counter()
    for _ in range(WRITES):
        s.send(b"x")
    dt = time.perf_counter() - t0
    segs = data_segs_out(s)
    s.close(); t.join(); lsock.close()
    print(f"TCP_NODELAY={nodelay}  {WRITES} writes -> {segs:6} data segments"
          f"  ({WRITES / segs:5.1f} bytes/segment)  {dt * 1e3:6.1f} ms  received {got[0]}")
```

```
python3 tiny_writes.py
```

```
TCP_NODELAY=0  20000 writes ->   5418 data segments  (  3.7 bytes/segment)    36.4 ms  recei
TCP_NODELAY=1  20000 writes ->  19135 data segments  (  1.0 bytes/segment)    79.1 ms  recei
TCP_NODELAY=0  20000 writes ->   5175 data segments  (  3.9 bytes/segment)    39.8 ms  recei
TCP_NODELAY=1  20000 writes ->  20000 data segments  (  1.0 bytes/segment)    81.4 ms  recei
```

An application that writes one byte at a time (unbuffered logging, keystrokes, a naive serializer)

is what Nagle was designed for in 1984. With Nagle on, 20,000 writes left as about 5,200–5,400 segments; with `TCP_NODELAY` as 19,135–20,000 segments (the few merges come from autocorking). Each segment carries 40+ bytes of IP and TCP headers for 1 byte of data. On loopback the ACK comes back in microseconds, so Nagle can only batch about 4 bytes per segment. On a real path with a 20 ms RTT Nagle allows at most one small segment in flight per round trip, so the same loop (about 40 ms of writes) would leave as a handful of segments, each carrying everything written during one round trip (not run here: adding delay needs `tc netem`, see Try this). That is the cost of setting `TCP_NODELAY` everywhere without also batching writes: more packets, more interrupts, more per-packet CPU on every hop, and on lossy or metered links (cellular, satellite, IoT) a visible bill.

Case — choosing the fix

Situation	Fix
You own the sender code	Build the whole message and write it once (`writev`/`sendmsg`, a buffered writer you flush per message). Then decide on `TCP_NODELAY` separately.
Request/response protocol whose library already writes each message once (gRPC, HTTP/2, most modern RPC and DB drivers)	`TCP_NODELAY` on. Most such libraries already set it: Go's `net` package sets it on every TCP connection by default; Node.js `http.createServer` defaults to `noDelay: true` since v18, while a raw `net.Socket` still starts with Nagle on. Check yours.
Header + `sendfile()` body (static file servers)	`TCP_CORK` around the header write and the `sendfile()`, then uncork. Or `MSG_MORE` on the header.
Bulk transfer (backups, replication streams)	Nothing: writes are MSS, Nagle never holds them.
You own only the receiver and cannot change clients	`TCP_QUICKACK` before every read, or a `quickack 1` route for those clients' prefix. Treat it as a workaround.
Interactive byte streams (terminals)	Depends on what matters more: latency (SSH sets `TCP_NODELAY` for interactive sessions) or packet count.

The trap

Trap: "it's the network." The stall is entirely host-side: no loss, no retransmission, no queueing. Pings are fast, `ss` shows a tiny RTT (tens of microseconds on loopback), yet requests take 40 ms. The tell is latency that clusters at ~**40 ms** (or ~200 ms on stacks whose delayed-ACK timer is 200 ms) regardless of path RTT or load.

Trap: "`TCP_NODELAY` everywhere." It removes the stall but not the cause, two writes per message. You now send twice the packets (100 vs 50 data segments in the first lab), and a chatty writer can send one packet per byte (third lab). Batch first; then `TCP_NODELAY` is cheap.

Trap: "set `TCP_QUICKACK` once." It is not a mode you turn on; it is a nudge. The first lab shows "QUICKACK once" stalls exactly like the default.

Trap: "`TCP_CORK` and forget." A corked socket holds partial data up to 200 ms. Forgetting to uncork turns a 40 ms stall into a 200 ms one. `MSG_MORE` cannot be forgotten: it applies to one call.

Trap: "the first request was fast, so we're fine." Quick-ACK mode at connection start hides the stall for the first exchange. Benchmarks that open a new connection per request never see it; connection pools see it on every request.

The boring rule

Write each message with one system call. Then turn on `TCP_NODELAY` for request/response protocols, and leave Nagle on for streams of tiny writes you cannot batch. When latency clusters at 40 ms, look for `notsent` on the sender and `ato:40` on the receiver before you look at the network.

Try this

1. Change `HDR` to 1,000 bytes and `BODY` to 70,000 bytes in `nagle_lab.py`. Does the default still stall? Why does a body larger than one MSS change the answer?
2. Make the server write its reply as two 10-byte writes too. Which side stalls now, and does client-side `TCP_NODELAY` still fix it?
3. Run `nagle_lab.py` with `N=200` and plot the latency of each request. How many fast requests do you get before the stall starts on this kernel?
4. With a capture tool (`tcpdump -ni lo port PORT`) where one is available, find the bare ACK and the body segment around each 40 ms gap.
5. On a machine you control, add a 20 ms delay to a test interface (`tc qdisc add dev IF root netem delay 20ms`, as root or in a network namespace) and re-run `tiny_writes.py`. How many bytes per segment does Nagle batch now?

Next

- Common Application Protocols

Verification

- Run unprivileged on this box: Debian 13, Linux **6.12.94+**, iproute2 **6.15.0** (`ss`, `nstat`), Python **3.13.5**, `tcp_congestion_control=cubic`, `tcp_autocorking=1`, loopback MTU 65536. All outputs above are pasted from those runs; timings vary by a few percent between runs.
- Kernel constants and logic checked in Linux 6.12 source: `TCP_DELACK_MIN`/`TCP_ATO_MIN` = `HZ/25`, `TCP_DELACK_MAX` = `HZ/5`, `TCP_MAX_QUICKACKS` = 16 (`include/net/tcp.h`); `tcp_nagle_check()` and `tcp_minshall_check()` (`net/ipv4/tcp_output.c`), and re-checked unchanged in the `linux-7.2.y` tree. Linux 7.2.11 is current stable (kernel.org, 2026-10-18).
- **Not run:** `tcpdump` (not installed; nothing was installed for this chapter), the `ip route ... quickack 1` route attribute and `tc netem` delay (both need root or a separate network namespace, which this run did not use). These are described from `ip-route(8)`, `tc-netem(8)` and the kernel source.
- `nstat` counters are per network namespace; the delta above includes unrelated traffic in a shared namespace, as noted.
- No packages installed, no `sudo`, no sysctl changed.

Sources

- RFC 896 — Congestion Control in IP/TCP Internetworks (Nagle, 1984)
- RFC 9293 — Transmission Control Protocol: §3.7.4 Nagle Algorithm (SHLD-7, MUST-17), §3.8.6.3 Delayed Acknowledgments (SHLD-18, SHLD-19, MUST-40), Appendix A.3 Nagle Modification
- RFC 1122 — Requirements for Internet Hosts §4.2.3.2 (delayed ACK), §4.2.3.4 (Nagle)
- RFC 5681 — TCP Congestion Control §4.2 (generating acknowledgments)
- G. Minshall, *A Suggested Modification to Nagle's Algorithm* (Internet-Draft, 1998), reference [68] in RFC 9293
- Linux 6.12 `include/net/tcp.h` and `net/ipv4/tcp_output.c` (`tcp_nagle_check`, `tcp_minshall_check`): https://git.kernel.org/pub/scm/linux/kernel/git/stable/linux.git/tree/net/ipv4/tcp_output.c?h=linux-6.12.y
- Kernel docs — IP sysctl: `tcp_autocorking`
- man7 — `tcp(7)` (`TCP_NODELAY`, `TCP_CORK`, `TCP_QUICKACK`, `TCP_INFO`), `send(2)` (`MSG_MORE`), `ss(8)`, `ip-route(8)` (`quickack`)
- Go — `net.TCPConn.SetNoDelay` ("The default is true (no delay)"); Node.js — `http.createServer noDelay`, `net.Socket noDelay`

TCP Fast Open

TCP Fast Open

Thesis

TCP Fast Open (TFO) lets a client send data in the first SYN (with a cookie) so short RPCs avoid a full handshake RTT. Linux exposes it via `net.ipv4.tcp_fastopen` bits: client=1, server=2, both=3. This chapter reads the sysctl from procfs.

Mental model

```
first connection: SYN + cookie request
later: SYN + data + cookie
```

Value	Meaning
0	off
1	client
2	server
3	client+server

Worked examples

```
=== /proc/sys net.ipv4.tcp_fastopen ===
1
/proc/sys/net/ipv4/tcp_fastopen
/proc/sys/net/ipv4/tcp_fastopen_blackhole_timeout_sec
/proc/sys/net/ipv4/tcp_fastopen_key
/proc/sys/net/ipv4/tcp_fastopen=1
/proc/sys/net/ipv4/tcp_fastopen_blackhole_timeout_sec=0
/proc/sys/net/ipv4/tcp_fastopen_key=
=== ss ===
State  Recv-Q Send-Q Local Address:Port  Peer Address:Port
LISTEN 0      128        127.0.0.1:8791       0.0.0.0:*
LISTEN 0      100          0.0.0.0:6080       0.0.0.0:*
LISTEN 0      100          0.0.0.0:6081       0.0.0.0:*
LISTEN 0      32         127.0.0.1:5904       0.0.0.0:*
LISTEN 0      32         127.0.0.1:5900       0.0.0.0:*
LISTEN 0      32         127.0.0.1:5902       0.0.0.0:*
```

```
LISTEN 0      5            0.0.0.0:2375       0.0.0.0:*
6.12.94+
```

This guest has **`tcp_fastopen=1`** (client enabled).

Trap

Enabling server TFO without measuring. Some middleboxes blackhole TFO (`tcp_fastopen_blackhole_time`

Boring rule

Keep client TFO on if the distro default says so. Turn on server TFO only behind metrics and with app support.

Try this

1. Compare app docs for `TCP_FASTOPEN`.
2. Watch TCPFastOpen counters under load.
3. Read RFC 7413.

Sources

- RFC 7413
- `tcp(7)`

TIME-WAIT on Linux: What the Timers Really Do

TIME-WAIT on Linux: What the Timers Really Do

Learning goals

- Say which end of a connection holds `TIME-WAIT`, and why
- Read the `timer:(timewait,…)` field in `ss -o` and tell `TIME-WAIT` from an orphaned `FIN-WAIT-2`
- Explain why `net.ipv4.tcp_fin_timeout` does **not** shorten `TIME-WAIT` on Linux
- Recognise source-port exhaustion (`EADDRNOTAVAIL`) and fix it in the right order: pooling first, then `tcp_tw_reuse`, never `tcp_max_tw_buckets`

Thesis

`TIME-WAIT` is the price the **active closer** pays so that old segments of a connection die before the same four-tuple is used again. RFC 9293 asks for 2×MSL. Linux pays a fixed 60 seconds: `TCP_TIMEWAIT_LEN (60*HZ)` in `include/net/tcp.h`. No sysctl changes it. The sysctl most people reach for, `tcp_fin_timeout`, times a *different* state. If short connections run you out of ports, the fix is fewer connections (pooling, keep-alive), not a shorter wait.

Mental model

Three facts carry the whole chapter:

1. **Whoever sends the first FIN waits.** It is a role, not "the client" or "the server". An HTTP server that closes idle keep-alive connections collects `TIME-WAIT` itself.
2. **`TIME-WAIT` costs a four-tuple, not much memory.** Linux swaps the full socket for a small `tcp_timewait_sock`. The scarce thing is the *local port* toward one destination `(dst IP, dst port)`.
3. **Two timers, two states.** `TIME-WAIT` is always 60 s. An *orphaned* `FIN-WAIT-2` (the app called `close()`, the peer never sent its FIN) is limited by `tcp_fin_timeout`.

Why the wait exists (RFC 9293, §3.6)

- **Late duplicates.** A segment delayed in the network must not be accepted by a *new* connection that reuses the same four-tuple and happens to have an overlapping sequence space.

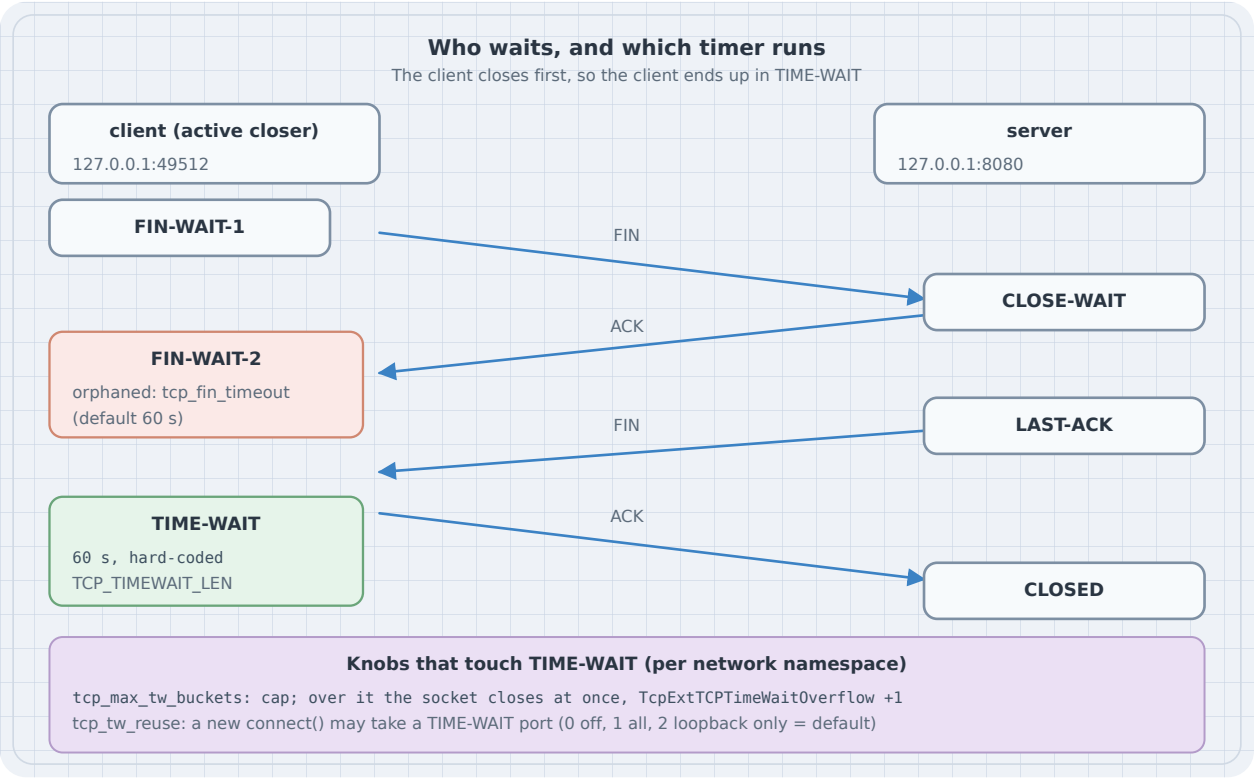


Figure 1: A TCP close: the side that sends the first FIN ends in TIME-WAIT for a hard-coded 60 seconds. tcp_fin_timeout only limits FIN-WAIT-2 for sockets the application has already closed.

- **The last ACK can be lost.** If the final ACK is lost, the peer resends its FIN. The side in `TIME-WAIT` still has state and re-ACKs it. Without that state it would answer with RST, and the peer would log an error for a connection that actually closed cleanly.

RFC 9293 sets the wait at 2×MSL, with MSL "arbitrarily defined to be 2 minutes" (4 minutes in total). Linux chose 60 s long ago and wrote it as a compile-time constant.

What the kernel source says

From Linux 6.12 (`include/net/tcp.h`):

```
#define TCP_TIMEWAIT_LEN (60*HZ) /* how long to wait to destroy TIME-WAIT
                  * state, about 60 seconds */
#define TCP_FIN_TIMEOUT TCP_TIMEWAIT_LEN
```

In `tcp_time_wait()` (`net/ipv4/tcp_minisocks.c`), a real `TIME-WAIT` socket always gets the constant:

```
        if (state == TCP_TIME_WAIT)
            timeo = TCP_TIMEWAIT_LEN;
```

`tcp_close()` (`net/ipv4/tcp.c`) is where `tcp_fin_timeout` comes in. When an application closes a socket that is in `FIN-WAIT-2`, the kernel parks it as a time-wait mini-socket *in substate* ***`FIN-WAIT-2`***, with the `tcp_fin_timeout` value as its timer:

```
            const int tmo = tcp_fin_time(sk);

            if (tmo > TCP_TIMEWAIT_LEN) {
                inet_csk_reset_keepalive_timer(sk,
                        tmo - TCP_TIMEWAIT_LEN);
            } else {
                tcp_time_wait(sk, TCP_FIN_WAIT2, tmo);
```

That explains the odd `ss` line you will see in Lab 1: state `fin-wait-2` with a `timewait` timer.

The ip-sysctl documentation says the same thing in prose: `tcp_fin_timeout` is "the length of time an orphaned (no longer referenced by any application) connection will remain in the FIN_WAIT_2 state before it is aborted at the local end."

The knobs, and what each one really does

Sysctl (per network namespace)	Default	Effect on `TIME-WAIT`
`net.ipv4.tcp_fin_timeout`	60	None. Limits orphaned `FIN-WAIT-2` only

Sysctl (per network namespace)	Default	Effect on `TIME-WAIT`
`net.ipv4.tcp_max_tw_buckets`	depends on memory (65536 in the lab below)	A cap. Over it, a closing socket skips `TIME-WAIT` and is destroyed at once (`TcpExtTCPTimeWaitOverflow`). The docs: "you *must* not lower the limit artificially"
`net.ipv4.tcp_tw_reuse`	2	A new **outgoing** `connect()` may take over a `TIME-WAIT` four-tuple if both ends used TCP timestamps and the newest timestamp is from an earlier second. 0 = off, 1 = all, 2 = loopback only
`net.ipv4.tcp_tw_recycle`	removed	Removed in Linux 4.12. It broke clients behind NAT. Old tuning guides that set it now just fail
`net.ipv4.ip_local_port_range`	32768 60999	How many source ports a client has per destination

Lab 1: watch the timers (run, no sudo)

Everything below runs inside `unshare -rn`: a private network namespace in which you are root, so its TCP sysctls are yours to change and the host is not affected. You need Python 3 and `ss`/`nstat` from iproute2. These are genuine outputs from Linux 6.12.

`tw.py` opens N connections on loopback and always closes the client side first:

```
"""Make TCP connections on loopback and close them from the client side.

usage: python3 tw.py PORT N {both-close|server-holds} SECONDS
  both-close    the client closes first, the server closes on EOF  -> client in TIME-WAIT
  server-holds  the client closes, the server never does           -> client in FIN-WAIT-2
The server sockets stay alive for SECONDS so ss can look at them.
"""
import socket, sys, time

port, n, mode, hold = int(sys.argv[1]), int(sys.argv[2]), sys.argv[3], float(sys.argv[4])
srv = socket.socket(); srv.bind(("127.0.0.1", port)); srv.listen(64)
kept = []
for _ in range(n):
    c = socket.create_connection(("127.0.0.1", port))
    s, _ = srv.accept()
```

```
    c.close()                       # the client sends FIN first: the active closer
    if mode == "both-close":
        s.recv(1)                   # read the EOF
        s.close()                   # the server's FIN completes the close
    else:
        kept.append(s)              # never close: the client waits in FIN-WAIT-2
time.sleep(hold)
```

`lab.sh` drives it and reads the timers back with `ss -o`:

```
#!/bin/sh
# Run as: unshare -rn sh lab.sh   (a private network namespace; its TCP sysctls are its own)
ip link set lo up
P=/proc/sys/net/ipv4
show() { ss -tano state "$1" "( dport = :$2 )" | sed 1d | sed "s/^/$1  /"; }

echo "== 1. defaults in a fresh namespace"
for k in tcp_fin_timeout tcp_max_tw_buckets tcp_tw_reuse; do echo "$k = $(cat $P/$k)"; done

echo "== 2. client closes first: who holds TIME-WAIT, for how long?"
python3 tw.py 8001 3 both-close 0; show time-wait 8001
ss -tanH "( sport = :8001 )" | wc -l | sed 's/^/server-side sockets left: /'

echo "== 3. tcp_fin_timeout=5 does not shorten TIME-WAIT"
echo 5 > $P/tcp_fin_timeout
python3 tw.py 8002 2 both-close 0; show time-wait 8002

echo "== 4. ...it times FIN-WAIT-2 for sockets the app already closed"
python3 tw.py 8003 2 server-holds 7 & sleep 0.5; show fin-wait-2 8003
sleep 5; echo "after 5.5 s:"; ss -tanH "( dport = :8003 )" | wc -l | sed 's/^/client sockets
```

```
echo "== 5. tcp_max_tw_buckets=2: the third socket skips TIME-WAIT"
echo 60 > $P/tcp_fin_timeout
sleep 75                                    # let every earlier TIME-WAIT socket be reaped
echo "TIME-WAIT sockets before: $(ss -tanH state time-wait | wc -l)"
echo 2 > $P/tcp_max_tw_buckets
before=$(nstat -az TcpExtTCPTimeWaitOverflow | awk '/Overflow/{print $2}')
python3 tw.py 8005 5 both-close 0; show time-wait 8005
after=$(nstat -az TcpExtTCPTimeWaitOverflow | awk '/Overflow/{print $2}')
echo "TcpExtTCPTimeWaitOverflow: $before -> $after"
```

```
$ unshare -rn sh lab.sh
== 1. defaults in a fresh namespace
tcp_fin_timeout = 60
tcp_max_tw_buckets = 65536
tcp_tw_reuse = 2
```

```
== 2. client closes first: who holds TIME-WAIT, for how long?
time-wait  0      0          127.0.0.1:37370    127.0.0.1:8001 timer:(timewait,59sec,0)
time-wait  0      0          127.0.0.1:37366    127.0.0.1:8001 timer:(timewait,59sec,0)
time-wait  0      0          127.0.0.1:37386    127.0.0.1:8001 timer:(timewait,59sec,0)
server-side sockets left: 0
== 3. tcp_fin_timeout=5 does not shorten TIME-WAIT
time-wait  0      0          127.0.0.1:47914    127.0.0.1:8002 timer:(timewait,59sec,0)
time-wait  0      0          127.0.0.1:47930    127.0.0.1:8002 timer:(timewait,59sec,0)
== 4. ...it times FIN-WAIT-2 for sockets the app already closed
fin-wait-2  0      0          127.0.0.1:38132    127.0.0.1:8003 timer:(timewait,4.516sec,0)
fin-wait-2  0      0          127.0.0.1:38116    127.0.0.1:8003 timer:(timewait,4.516sec,0)
after 5.5 s:
client sockets left: 0
== 5. tcp_max_tw_buckets=2: the third socket skips TIME-WAIT
TIME-WAIT sockets before: 0
time-wait  0      0          127.0.0.1:56858    127.0.0.1:8005 timer:(timewait,59sec,0)
time-wait  0      0          127.0.0.1:56848    127.0.0.1:8005 timer:(timewait,59sec,0)
TcpExtTCPTimeWaitOverflow: 0 -> 3
```

Reading it:

- **Step 2.** Only the client's sockets (destination port 8001) are in `TIME-WAIT`, each with `timewait,59sec`. The server, the passive closer, kept nothing.
- **Step 3.** With `tcp_fin_timeout` at 5, new `TIME-WAIT` sockets still start at 59 s. The constant wins.
- **Step 4.** The server never closes, so the client (already closed by the app) sits in `FIN-WAIT-2`. *This* timer is ~4.5 s, the 5 s `tcp_fin_timeout`, and 5.5 s later the client side is gone. The kernel stores it as a time-wait mini-socket, which is why `ss` prints `timewait` as the timer name for a `fin-wait-2` row.
- **Step 5.** With the cap at 2, two sockets enter `TIME-WAIT` and three are destroyed on the spot, counted by `TcpExtTCPTimeWaitOverflow`. That is the lost-ACK protection gone for those three. The only trace is that counter (6.12 prints no log line), so this setting is a DoS brake, not a tuning knob.

A detail you will hit in your own experiments: a `TIME-WAIT` socket can linger a few seconds past 60 s, shown as `timer:(timewait,,0)`. That's because long kernel timers are batched on a coarse timer wheel. Wait about 70 s between runs, or use a new port each time as the scripts above do.

Lab 2: port exhaustion and the fixes (run, no sudo)

Each `TIME-WAIT` socket holds one local port toward one `(dst IP, dst port)`. A client that opens a new connection per request to the same backend uses up `ip_local_port_range` within the 60 s. `exhaust.py` makes it quick by shrinking the range to four ports:

```
"""Open-and-close short connections until connect() runs out of source ports.
```

```
usage: python3 exhaust.py ATTEMPTS [PAUSE]
Each round: connect, server closes on EOF after the client closes (client keeps TIME-WAIT).
"""
import errno, socket, sys, time

attempts = int(sys.argv[1]); pause = float(sys.argv[2]) if len(sys.argv) > 2 else 0
srv = socket.socket(); srv.bind(("127.0.0.1", 8080)); srv.listen(64)
for i in range(1, attempts + 1):
    try:
        c = socket.create_connection(("127.0.0.1", 8080))
    except OSError as e:
        print(f"connect {i}: {errno.errorcode[e.errno]} ({e.strerror})")
        continue
    port = c.getsockname()[1]
    s, _ = srv.accept(); c.close(); s.recv(1); s.close()
    print(f"connect {i}: ok, source port {port}")
    time.sleep(pause)
```

`pool.py` sends 1000 small requests, either with a new connection each or over one kept-open connection:

```
"""The same 1000 requests, once with a new connection each, once over one kept connection.""
import socket, subprocess, sys

def tw():
    out = subprocess.run(["ss", "-tanH", "state", "time-wait", f"( dport = :{port} )"], capt
    return len(out.splitlines())

mode, port = sys.argv[1], int(sys.argv[2])
srv = socket.socket(); srv.bind(("127.0.0.1", port)); srv.listen(64)
if mode == "new-per-request":
    for _ in range(1000):
        c = socket.create_connection(("127.0.0.1", port)); s, _ = srv.accept()
        c.sendall(b"GET\n"); s.recv(4); s.sendall(b"OK\n"); c.recv(3)
        c.close(); s.recv(1); s.close()
else:  # pooled
    c = socket.create_connection(("127.0.0.1", port)); s, _ = srv.accept()
    for _ in range(1000):
        c.sendall(b"GET\n"); s.recv(4); s.sendall(b"OK\n"); c.recv(3)
    c.close(); s.recv(1); s.close()
print(f"{mode:16} 1000 requests -> {tw()} sockets in TIME-WAIT")
```

```
#!/bin/sh
# Run as: unshare -rn sh lab2.sh
ip link set lo up
P=/proc/sys/net/ipv4
```

```
echo 40000 40003 > $P/ip_local_port_range      # four source ports, to make exhaustion quick
echo "== 6. tcp_tw_reuse=0: four ports, six connections"
echo 0 > $P/tcp_tw_reuse
python3 exhaust.py 6
sleep 75
echo "== 7. tcp_tw_reuse=2 (loopback): same six, 1.1 s apart"
echo 2 > $P/tcp_tw_reuse
python3 exhaust.py 6 1.1
echo "== 8. one connection per request vs one pooled connection"
echo "32768 60999" > $P/ip_local_port_range
python3 pool.py new-per-request 9001
python3 pool.py pooled 9002
```

```
$ unshare -rn sh lab2.sh
== 6. tcp_tw_reuse=0: four ports, six connections
connect 1: ok, source port 40002
connect 2: ok, source port 40000
connect 3: ok, source port 40001
connect 4: ok, source port 40003
connect 5: EADDRNOTAVAIL (Cannot assign requested address)
connect 6: EADDRNOTAVAIL (Cannot assign requested address)
== 7. tcp_tw_reuse=2 (loopback): same six, 1.1 s apart
connect 1: ok, source port 40000
connect 2: ok, source port 40002
connect 3: ok, source port 40002
connect 4: ok, source port 40002
connect 5: ok, source port 40002
connect 6: ok, source port 40000
== 8. one connection per request vs one pooled connection
new-per-request  1000 requests -> 1000 sockets in TIME-WAIT
pooled           1000 requests -> 1 sockets in TIME-WAIT
```

- **Step 6.** With reuse off, four connections fit and the fifth `connect()` fails with `EADDRNOTAVAIL`. That error, not "too many open files", is what port exhaustion looks like to the application.
- **Step 7.** With `tcp_tw_reuse=2` and loopback, all six work: the kernel takes over a `TIME-WAIT` port (40002 is used four times). It does this only because Linux sends TCP timestamps by default, which lets PAWS (RFC 7323) reject old duplicates. The source (`tcp_twsk_unique()` in `net/ipv4/tcp_ipv4.c`) also requires the last timestamp to be from an earlier second; that is why the script pauses 1.1 s.
- **Step 8.** The real fix. The same 1000 requests leave 1000 `TIME-WAIT` sockets, or 1 when they share a connection.

The default `2` explains a common surprise: reuse "works" in a laptop benchmark against `localhost` and does nothing against a remote backend. For non-loopback traffic you have to set `1` yourself.

Case: the proxy that ran out of ports

A reverse proxy forwards to a single backend `10.0.0.5:8080` and opens a new upstream connection per request, closing it after the response. At 600 requests/s it holds about $600 \times 60 = 36{,}000$ `TIME-WAIT` sockets toward one destination. That is more than the 28,232 ports in the default range, so upstream `connect()` calls begin to fail with `EADDRNOTAVAIL`.

The fixes, in order:

1. **Keep upstream connections alive and pool them.** For nginx, use `keepalive` in the `upstream` block with `proxy_http_version 1.1` and an empty `Connection` header. For an HTTP client library, reuse the client object. This removes the problem instead of hiding it.
2. **Let the side that can afford it close.** If the backend closes, its `TIME-WAIT` is keyed on *its* listening port with many client ports, which never exhausts anything.
3. **Spread the four-tuple.** More backend IPs or ports, or more proxy source IPs (`proxy_bind`), each give a fresh 28k ports.
4. **Then, maybe, `tcp_tw_reuse=1`** on the client host, knowing it relies on timestamps on both ends.
5. **Widen `ip_local_port_range`** if you must, but avoid ports that local services listen on.

Never: lower `tcp_max_tw_buckets`, set the removed `tcp_tw_recycle`, or "fix" it with `SO_LINGER` timeout 0. That last one sends RST instead of FIN and throws away unsent data.

Trap

Counting `TIME-WAIT` sockets and panicking. Thousands of them on a busy web server are normal and cheap. Measure the symptom instead: `EADDRNOTAVAIL` in client logs, or `TIME-WAIT` grouped by destination nearing your port range:

```
ss -tanH state time-wait | awk '{print $4}' | sort | uniq -c | sort -rn | head
```

Boring rule

Leave the `TIME-WAIT` sysctls alone. Pool connections to the same destination, and only if a measured shortage remains, turn on `tcp_tw_reuse=1` on the side that connects.

Try this

1. In Lab 1, make the *server* close first (call `s.close()` before `c.close()`) and check which port now shows up in `TIME-WAIT`.
2. Set `tcp_fin_timeout` to 90 and repeat step 4. `tcp_close()` now uses a keepalive timer for the first 30 s, so a `fin-wait-2` row should show a different timer name. Check it with `ss -o`.
3. Turn timestamps off (`echo 0 > /proc/sys/net/ipv4/tcp_timestamps` inside the namespace) and rerun step 7. Predict the result first.

4. Change `pool.py` so that the server closes each connection first, then count `TIME-WAIT` by local port instead of by destination port.

Verification

- Both labs were run on Linux 6.12 with iproute2 `ss`/`nstat` and Python 3, inside `unshare -rn` as an unprivileged user. The outputs above are genuine and unedited.
- The kernel excerpts are from the v6.12 tag. `tcp_tw_reuse_delay` (a later addition for tuning the one-second rule) is not present on 6.12 and is not covered here.

Sources

- RFC 9293, *Transmission Control Protocol*, §3.3.2 (state diagram) and §3.6 (closing a connection): https://www.rfc-editor.org/rfc/rfc9293
- RFC 7323, *TCP Extensions for High Performance* (timestamps, PAWS): https://www.rfc-editor.org/rfc/rfc7323
- Linux ip-sysctl documentation (`tcp_fin_timeout`, `tcp_max_tw_buckets`, `tcp_tw_reuse`): https://docs.kernel.org/networking/ip-sysctl.html
- Linux v6.12 source: `include/net/tcp.h`, `net/ipv4/tcp.c` (`tcp_close`), `net/ipv4/tcp_minisocks.c` (`tcp_time_wait`), `net/ipv4/tcp_ipv4.c` (`tcp_twsk_unique`): https://github.com/torvalds/linux/tree/v6.12/net/ipv4
- Removal of `tcp_tw_recycle` (Linux 4.12): https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=4396e46187ca5070219b81773c4e65088dac50cc
- nginx `upstream keepalive`: https://nginx.org/en/docs/http/ngx_http_upstream_module.html#keepalive
- ss(8): https://man7.org/linux/man-pages/man8/ss.8.html

Zero-Copy Sends with MSG_ZEROCOPY: Completions, Deferred Copies, and When It Loses

Zero-Copy Sends with MSG_ZEROCOPY: Completions, Deferred Copies, and When It Loses

Learning goals

- Explain what `MSG_ZEROCOPY` changes about `send()`. The kernel pins your pages instead of copying them, which means **you may not reuse the buffer until the kernel says so**.
- Turn the feature on correctly. `SO_ZEROCOPY` has to be set first, because without it the flag is silently ignored.
- Read completion notifications from the socket **error queue** (`MSG_ERRQUEUE`), including the coalesced ranges and the `SO_EE_CODE_ZEROCOPY_COPIED` flag.
- Predict when zero-copy is slower than copying (small writes, loopback and veth, NICs without scatter-gather), and measure it rather than assume.

Version note: the lab ran on Linux **6.12** with plain Python 3 sockets, with no root and no extra packages. The constants come from the kernel UAPI headers because Python's `socket` module does not export them. Every number below comes from the pasted run. Throughput on a shared box varies by tens of percent between runs, so read the shape of the results, not the digits.

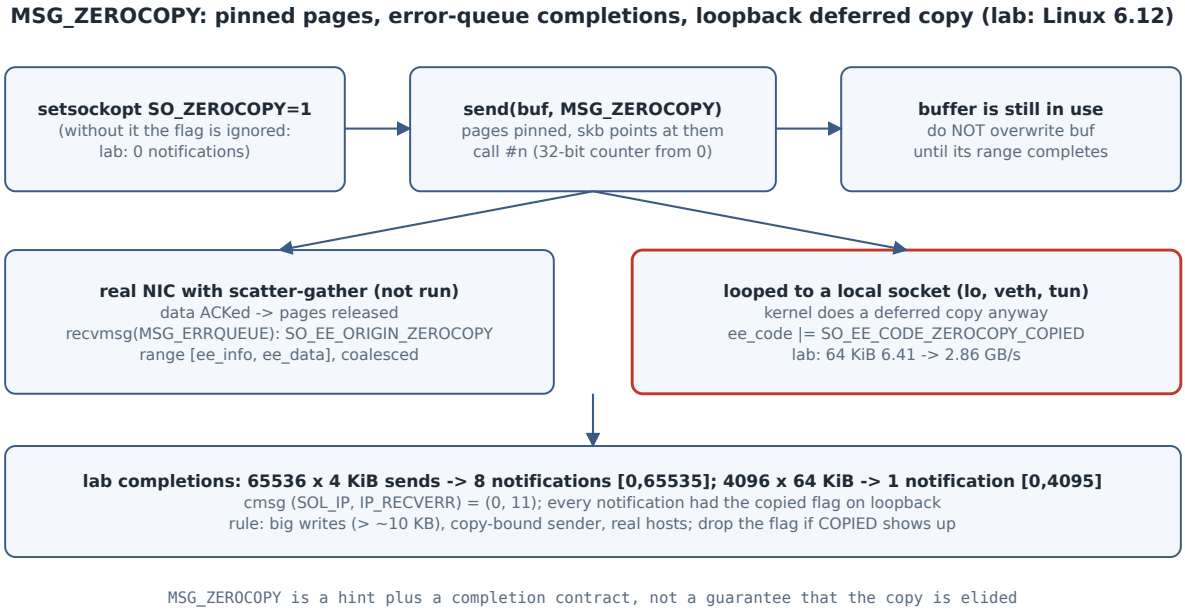


Figure 1: MSG_ZEROCOPY send path, completion notifications, and the loopback deferred copy.

The contract

Per the kernel's MSG_ZEROCOPY documentation:

- **Socket setup.** `send()` ignores flags it does not know, so the kernel will not enable copy avoidance just because a legacy program happens to pass the bit. You must first set `setsockopt(fd, SOL_SOCKET, SO_ZEROCOPY, 1)`.
- **Transmission.** `send(fd, buf, len, MSG_ZEROCOPY)`. The call can fail with `ENOBUFS` if the socket exceeds its optmem limit or the user exceeds the locked-pages ulimit. Calls with and without the flag can be mixed on the same socket.
- **Completion.** Each successful zero-copy send call increments an unsigned 32-bit counter, starting at 0. It counts system calls, not bytes. When the kernel is done with the pages, it queues a `sock_extended_err` with `ee_origin = SO_EE_ORIGIN_ZEROCOPY`. `ee_info` holds the first number of the completed range and `ee_data` the last. Consecutive completions are **coalesced**, and for TCP there is usually at most one outstanding. The notification has a zero error code, so it does not block normal `send`/`recv`.
- **Deferred copies.** The flag is "a hint … not a guarantee". If the kernel had to copy after all, it sets `SO_EE_CODE_ZEROCOPY_COPIED` in `ee_code`, and a program can then stop passing the flag.
- **Loopback.** All `MSG_ZEROCOPY` packets "looped to a local socket will incur a deferred copy", because a local receiver might never read and the pages would stay pinned indefinitely.
- **Size.** Page pinning and notification costs mean it is "generally only effective at writes over around 10 KB".

Lab: measure it on loopback

```
#!/usr/bin/env python3
"""MSG_ZEROCOPY over TCP: send, then read completions from the error queue."""
import socket, struct, sys, threading, time

SO_ZEROCOPY = 60          # asm-generic/socket.h
MSG_ZEROCOPY = 0x4000000  # linux/socket.h
SO_EE_ORIGIN_ZEROCOPY = 5
SO_EE_CODE_ZEROCOPY_COPIED = 1

def sink(srv):
    c, _ = srv.accept()
    while c.recv(1 << 20):
        pass

def run(addr, size, count, zc, flag_only=False):
    fam = socket.AF_INET6 if ':' in addr else socket.AF_INET
    srv = socket.socket(fam, socket.SOCK_STREAM); srv.bind((addr, 0)); srv.listen()
    threading.Thread(target=sink, args=(srv,), daemon=True).start()
    s = socket.socket(fam, socket.SOCK_STREAM)
```

```
    if zc and not flag_only:
        s.setsockopt(socket.SOL_SOCKET, SO_ZEROCOPY, 1)
    s.connect(srv.getsockname()[:2])
    buf = bytearray(size)
    t = time.perf_counter(); c0 = time.thread_time()
    for _ in range(count):
        s.sendall(buf, MSG_ZEROCOPY if zc else 0)
    # drain notifications
    lo = hi = None; copied = 0; notes = 0; first = None
    if zc:
        deadline = time.time() + (0.5 if flag_only else 2)
        while time.time() < deadline and (hi is None or hi < count - 1):
            try:
                _, anc, _, _ = s.recvmsg(0, 512, socket.MSG_ERRQUEUE | socket.MSG_DONTWAIT)
            except BlockingIOError:
                time.sleep(0.001); continue
            for lvl, typ, data in anc:
                first = first or (lvl, typ)
                errno_, origin, _t, code, _p, info, data_ = struct.unpack('IBBBBII', data[:1
                if origin == SO_EE_ORIGIN_ZEROCOPY:
                    notes += 1
                    lo = info if lo is None else min(lo, info)
                    hi = data_ if hi is None else max(hi, data_)
                    copied += code & SO_EE_CODE_ZEROCOPY_COPIED
    el = time.perf_counter() - t; cpu = time.thread_time() - c0
    s.close()
    print(f"  {addr:9} size={size:>7} zc={int(zc)} sends={count} "
          f"{size*count/el/1e9:6.2f} GB/s sender_cpu={cpu:5.2f}s"
          + (f" notifications={notes} range=[{lo},{hi}] copied_flag={copied} cmsg={first}" i
          + (" (MSG_ZEROCOPY without SO_ZEROCOPY)" if flag_only else ""))

print("== loopback TCP, 256 MiB per run (per-run sizes below)")
for addr in ('127.0.0.1',):
    for size in (4096, 65536, 1048576):
        cnt = max(200, (256 << 20) // size)
        run(addr, size, cnt, False); run(addr, size, cnt, True)
print("== flag passed but socket option not set")
run('127.0.0.1', 65536, 64, True, flag_only=True)
```

```
$ python3 zc.py
== loopback TCP, 256 MiB per run (per-run sizes below)
  127.0.0.1 size=   4096 zc=0 sends=65536   2.46 GB/s sender_cpu= 0.10s
  127.0.0.1 size=   4096 zc=1 sends=65536   2.47 GB/s sender_cpu= 0.11s notifications=8 rang
  127.0.0.1 size=  65536 zc=0 sends=4096   6.41 GB/s sender_cpu= 0.04s
  127.0.0.1 size=  65536 zc=1 sends=4096   2.86 GB/s sender_cpu= 0.08s notifications=1 range
  127.0.0.1 size=1048576 zc=0 sends=256   8.80 GB/s sender_cpu= 0.03s
```

```
  127.0.0.1 size=1048576 zc=1 sends=256   5.45 GB/s sender_cpu= 0.05s notifications=1 range=
== flag passed but socket option not set
  127.0.0.1 size=  65536 zc=1 sends=64   0.01 GB/s sender_cpu= 0.01s notifications=0 range=[
```

Reading the numbers

- **The flag without the socket option does nothing.** There were no notifications and no error, so the last run behaved exactly like a normal copy. Its GB/s figure is meaningless because it includes the script's 0.5-second wait for notifications that never come. This is the documented legacy-safety behaviour, and it is also the most common bug: "we enabled zero-copy" when nothing changed.
- **Completions really are coalesced.** 65,536 sends of 4 KiB produced only **8** notifications covering `[0, 65535]`. The 64 KiB and 1 MiB runs produced a **single** notification for the whole run. Your reclaim loop must handle ranges, not one completion per send. The ranges end exactly at `sends - 1`, which confirms the counter counts calls: every Python `sendall()` here was satisfied by a single `send` syscall.
- **The control message is `(0, 11)`**, which is `SOL_IP`/`IP_RECVERR`. That matches the documentation (IPv6 sockets report `IPV6_RECVERR`).
- **Every notification carried `copied_flag`.** On loopback the kernel copied anyway, later. The result is no gain at 4 KiB, and **a loss** at 64 KiB (6.41 → 2.86 GB/s) and 1 MiB (8.80 → 5.45 GB/s), with more sender CPU. You pay for pinning, notifications **and** a copy that is now cold in cache.

What changes on a real NIC (not run here)

The box has no physical NIC to test against. Between two hosts, the selftest `tools/testing/selftests/net/msg_z` (driven by `msg_zerocopy.sh`) is the reference measurement. The documentation warns that running it between namespaces over veth "will not show any improvement", for the same loopback reason seen above. Expect gains only when all of these hold:

- writes are large (well above about 10 KB) and the sender is CPU-bound on copies
- the NIC does scatter-gather, so the kernel can send headers and user pages together
- the application can afford to hold buffers until completion, which is easiest with a ring of buffers reclaimed on each notification range

`ss -tmi` and `perf top` (look for `copy_user_*` / `_copy_from_iter`) on the sender are the quickest ways to see whether copying was the bottleneck in the first place.

The boring rule

- Default to copying. Try `MSG_ZEROCOPY` only for big writes on a sender proven to be copy-bound.

- Always set `SO_ZEROCOPY`, always drain `MSG_ERRQUEUE` (non-blocking, every few sends), and never touch a buffer before its range completes.
- Watch `SO_EE_CODE_ZEROCOPY_COPIED`. If it shows up on a path, turn the flag off for that socket.
- Benchmark between real hosts, never over loopback or veth.

Try this

1. Change the address to `::1`. What `cmsg` level and type do you see now?
2. Drain the error queue after every 16 sends instead of at the end. Does the notification count go up?
3. Lower the locked-memory limit (`ulimit -l 64`) and try 1 MiB sends. Do you get `ENOBUFS`, or does the deferred-copy path on loopback avoid pinning altogether?

Sources

- Linux kernel documentation, *MSG_ZEROCOPY*: https://docs.kernel.org/networking/msg_zerocopy.html
- Kernel selftest `tools/testing/selftests/net/msg_zerocopy.c`: https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/tree/tools/testing/selftests/net/msg_zerocopy.c
- UAPI constants: `include/uapi/asm-generic/socket.h` (`SO_ZEROCOPY` 60), `include/linux/socket.h` (`MSG_ZEROCOPY`), `include/uapi/linux/errqueue.h` (`sock_extended_err`, `SO_EE_ORIGIN_ZEROCOPY`, `SO_EE_CODE_ZEROCOPY_COPIED`)
- W. de Bruijn, *sendmsg copy avoidance with MSG_ZEROCOPY* (netdev 2.1, 2017; paper, slides, video): https://netdevconf.org/2.1/session.html?debruijn
- recvmsg(2) and ip(7), `MSG_ERRQUEUE` / `IP_RECVERR`: https://man7.org/linux/man-pages/man7/ip.7.html

Common Application Protocols

Common Application Protocols

Application protocols give **meaning** to payloads. Operators still need a working vocabulary: DNS, DHCP, HTTP, SSH, and TLS appear in almost every network design.

Learning goals

- Place common apps on TCP/UDP and well-known ports
- Describe DNS resolution flow at a high level
- Describe DHCP DORA and where relays fit
- Separate HTTP from TLS and from TCP failures
- Know what SSH needs from the network

Map (operator cheat sheet)

Protocol	Transport	Port(s)	Role
DNS	UDP/TCP	53	Name → address (and more)
DHCP	UDP	67/68	IPv4 host config
DHCPv6	UDP	546/547	IPv6 host config
HTTP	TCP	80	Web (cleartext)
HTTPS	TCP	443	HTTP over TLS
SSH	TCP	22	Secure shell / SFTP
NTP	UDP	123	Time
BGP	TCP	179	Routing (control plane app)
SNMP	UDP	161/162	Legacy management

DNS

Roles

Role	Function
Stub resolver	On the host; asks a recursive resolver
Recursive resolver	Walks the tree for the answer
Authoritative server	Holds zone data

Query types (common)

Type	Meaning
A	IPv4 address
AAAA	IPv6 address
CNAME	Alias
MX / TXT / SRV	Mail, text, services

```
# Linux lab host examples
getent hosts example.com
# if dig/nslookup installed:
# dig +short A example.com
# dig +short AAAA example.com
```

Failure patterns

Symptom	Check
Names fail, IPs work	DNS config, UDP/53 filters, resolver reachability
Slow apps	Timeouts to dead resolvers
Wrong address	Cache, split DNS, hosts file

DHCP (IPv4)

DORA: Discover → Offer → Request → Ack

- Client uses broadcasts initially (needs L2 reachability to server or **relay**)
- Server assigns IP, mask, gateway, DNS, lease time

```
Client --L2 domain-- DHCP server
Client --L2-- Relay --L3-- DHCP server
```

Edge designs with helpers/relays: later Edge & services part.

HTTP and HTTPS

```
HTTP:  App  TCP:80  Server
HTTPS: App  TCP:443  TLS   HTTP   Server
```

Failure	Layer to inspect
TCP SYN timeout	Path, ACL, listen socket
TCP up, TLS error	Certificates, SNI, protocol mismatch
TLS up, HTTP 5xx	Application

```
curl -v http://192.0.2.10/
curl -vk https://192.0.2.10/
```

SSH

- TCP/22 (by default)
- Needs bidirectional path and allowed security policy
- Used heavily for lab and production management

```
ssh user@192.0.2.10
ss -tlnp | grep 22
```

TLS in one paragraph

TLS provides encryption and authentication **above** TCP (classically). Network engineers care when: MTU/PMTUD breaks large records, middleboxes interfere, or certificates expire. Crypto suite deep-dives are optional here.

NTP and time

Broken time breaks auth (Kerberos, cert validation) and log correlation. Lab VMs often need chrony/ntp or host sync.

Lab sketch

1. DNS: break `/etc/resolv.conf`; compare `ping` by name vs by IP
2. HTTP: capture TCP/80 three-way + first GET
3. SSH: confirm LISTEN on 22; ACL drop and observe client timeout

Checkpoint

User says “the network is down” but `ping` by IP works and `curl` by name fails. List the top three protocol checks in order.

Next

- Host Forwarding and Routing Intro

Host Forwarding and Routing Intro

Host Forwarding and Routing Intro

Every host is a mini-router for its **own** traffic: connected routes, default gateway, and policy policy later. This chapter is host-centric; multi-router IGP/BGP comes in later parts.

Learning goals

- Read a Linux routing table
- Explain longest-prefix match (LPM)
- Configure a default route and a static host route
- Distinguish RIB vs what `ip route get` uses
- Enable IP forwarding on a Linux router node (preview)

Connected vs remote

Destination	Host behavior
In a connected prefix	ARP/ND on that interface; no gateway
Not connected	Lookup route → next hop → ARP/ND for next hop

Linux route table essentials

```
ip route
ip -6 route
ip route get 1.1.1.1
ip route get 192.168.10.50
```

Common entry types:

Entry	Meaning
`default via G dev IF`	Default gateway
`P.P.P.P/nn dev IF`	Connected or direct
`H via G`	Host route (/32)

Entry	Meaning
metric / proto	Preference and source (dhcp, static, …)

Longest prefix match

More specific prefixes win:

```
0.0.0.0/0        via 10.0.0.1
10.0.0.0/8       via 10.0.0.2
10.1.1.0/24      dev eth2
```

Destination `10.1.1.5` → `eth2` connected.
Destination `10.2.0.1` → via `10.0.0.2`.
Destination `8.8.8.8` → default via `10.0.0.1`.

Default gateway

Most endpoints need one:

```
ip route add default via 10.20.1.1 dev eth1
```

Without it, on-link works and off-link fails—classic "I can ping the gateway but not the internet."

Static routes (host or simple router)

```
ip route add 10.30.0.0/16 via 10.20.1.2
ip route add 192.0.2.10/32 via 10.20.1.3
```

Multi-router static designs and floating statics: Layer 3 static routing labs.

IP forwarding (Linux as router)

```
sysctl net.ipv4.ip_forward
sysctl -w net.ipv4.ip_forward=1
sysctl -w net.ipv6.conf.all.forwarding=1
```

With forwarding on, Linux can move packets between interfaces per the FIB—foundation of FRR data plane coexistence in labs.

Policy routing (awareness)

Multiple tables and `ip rule` exist for VRFs and advanced designs. Foundations: know they exist; do not start there.

Firewall vs routing

A **correct route** with a **drop rule** still fails. Always separate:

1. Is there a route? (`ip route get`)
2. Does L2 resolve? (`ip neigh`)
3. Does policy allow? (nft/iptables, security groups)

Lab sketch

1. Two subnets, one Linux router
2. Hosts with defaults pointing at the router
3. `ip route get` on host and router for east-west traffic
4. Break default on host; compare symptoms

Checkpoint

Host `10.1.1.10/24`, gateway `10.1.1.1`. Route table has only the connected /24. What works? What fails? What single command fixes off-subnet reachability?

Next

- Control, Data, and Management Planes

Control, Data, and Management Planes

Control, Data, and Management Planes

Separate **how the network decides**, **how it forwards**, and **how you operate it**. This split keeps troubleshooting honest from first lab to fabric design.

Learning goals

- Define data, control, and management planes with examples
- Classify failures into the right plane
- Pick evidence (tables vs protocols vs access path) per plane
- Connect planes to tools used in this book

Data plane

Forwards user and transit packets using programmed tables.

Examples	Evidence
Ethernet FDB / MAC table	`bridge fdb`, switch MAC table
IP FIB / routes used for forward	`ip route get`, CEF/FIB show
ACL counters / nft hits	Counters increment on matches
Queues / drops	Interface drop stats

Failures: blackhole route, wrong next hop, MTU drop, ACL drop, congestion.

Control plane

Builds and maintains the information data plane uses.

Examples	Evidence
ARP/ND	`ip neigh`
OSPF/BGP adjacencies	protocol show commands
MAC learning	dynamic FDB entries
Route computation	RIB entries, SPF/BGP path choice

Failures: adjacency down, authentication mismatch, policy filter, CPU overload, CoPP drops.

Management plane

How humans and automation configure and observe devices.

Examples	Evidence
SSH, HTTPS API, NETCONF	Reachability to mgmt VRF/IP
Telemetry, syslog, SNMP	Streams arrive at collectors
Console / OOB	Still works when in-band dies

Failures: ACL on vty, expired credentials, mgmt routing broken, "network down" that is only mgmt path down.

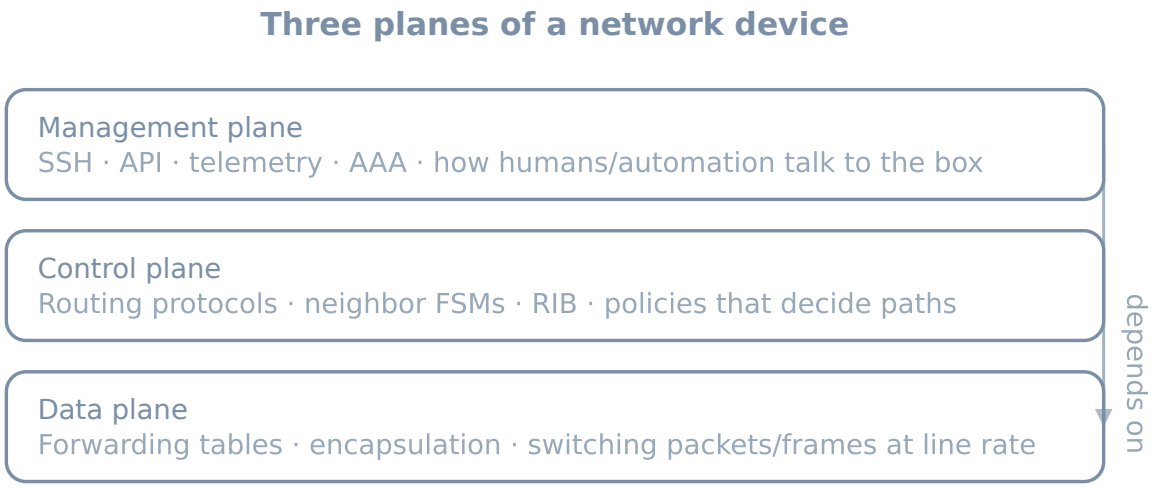


Figure 1: Planes

Interaction

```
Management  →  configures  →  Control
Control     →  programs    →  Data
Data        →  forwards       user packets
```

A BGP session (control) programs the FIB (data). You establish BGP over TCP (data plane for the control session—meta but real).

Troubleshooting order (habit)

1. **Management:** Can I access the device?
2. **Data:** Does `ping`/`curl` of interest work? What does `ip route get` say?
3. **Control:** Are neighbors up? Are routes present in the RIB?
4. Reconcile: RIB has route but FIB/policy drops?

Lab habit

When something fails, write one sentence: *"I think this is a **data / control / management** plane fault because …; I will prove it with …."*

Checkpoint

SSH to a router fails, but the router still forwards traffic between two lab hosts. Which plane is most likely broken for you, and which is still working?

Next

Depth continues in Ethernet & MAC Learning. Host and Containerlab skills remain in Lab platform.

Encrypted DNS — DoH and DoT

Encrypted DNS — DoH and DoT

Thesis

Plain DNS on port 53 is readable on path. **DoT (RFC 7858)** wraps DNS in TLS on 853. **DoH (RFC 8484)** carries DNS in HTTPS. This chapter resolves via the OS stub and via Cloudflare DoH JSON. DoT is marked not run (`dig` absent).

Mental model

```
app → stub → plain :53 | DoT :853 | DoH :443
```

Worked examples

```
getaddrinfo example.com: 2.91 ms -> ('198.18.0.1', 443)
DoH cloudflare-dns.com: 86.50 ms status=0 answers=[{'name': 'example.com', 'type': 1, 'TTL': 19
DoT: dig +tls not available on this host; chapter shows the wire form from RFC 7858.
```

Local stub returned `198.18.0.0/15` lab space; DoH returned public A records (~86 ms).

Trap

Assuming DoH always matches corporate DNS. Split-horizon breaks when browsers bypass the stub.

Boring rule

Prefer OS-managed DoT/DoH for fleets over browser-only DoH.

Try this

1. `dig +tls @1.1.1.1 example.com` where dig exists.
2. Compare browser DoH vs OS resolver.
3. Read RFC 8484.

Sources

- RFC 7858, RFC 8484

Layer 2

Ethernet & MAC Learning

Ethernet & MAC Learning

Layer 2 is where frames move inside a broadcast domain. Before VLANs and spanning tree, master **Ethernet addressing, flooding, and MAC learning**—the behaviors every switch and bridge still implements.

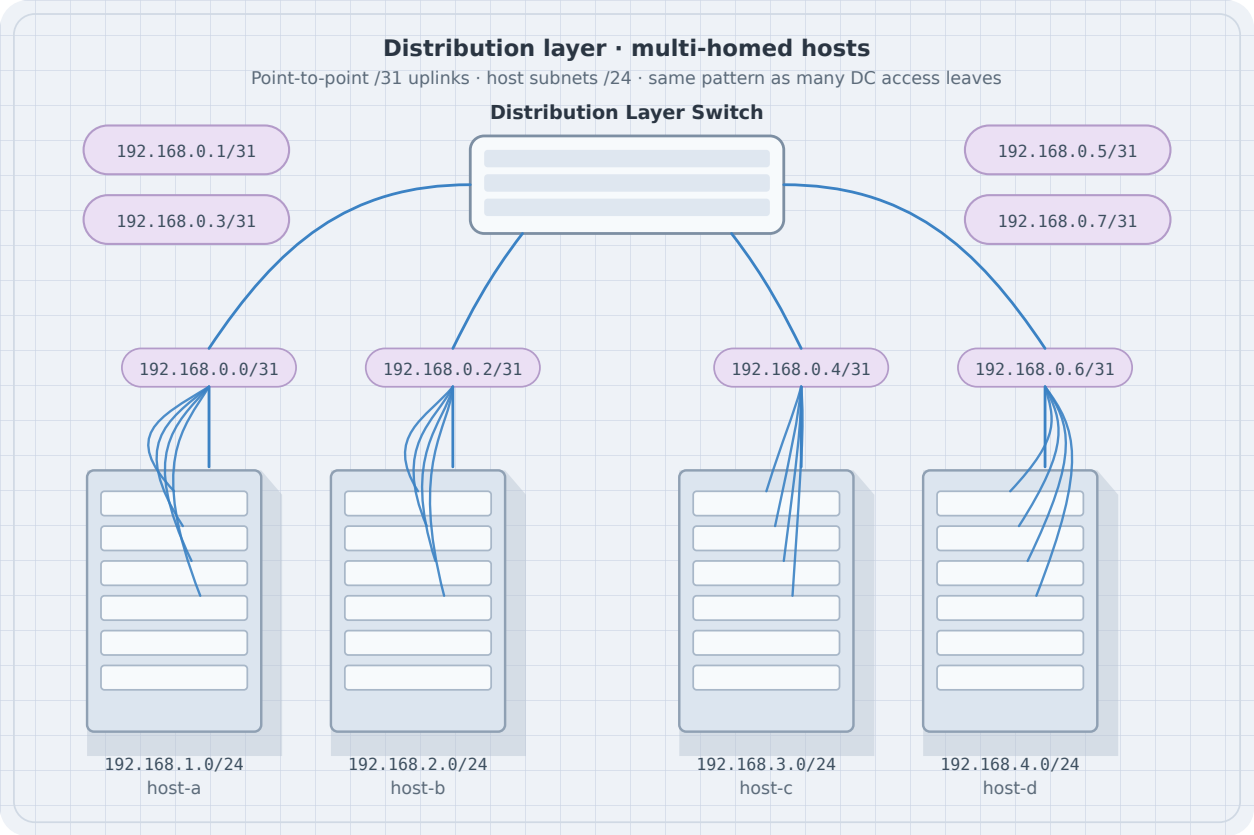


Figure 1: Distribution switch multi-home topology

Learning goals

By the end of this chapter you can:

- Explain source learning and unknown unicast flooding
- Read and interpret a MAC/FDB table

- Predict ARP's role binding IP to MAC on-link
- Build a small bridged lab with Linux or Containerlab
- Diagnose "same subnet but no ping" with neigh + FDB + capture

Concepts

Frame essentials

An Ethernet frame carries:

- Destination MAC
- Source MAC
- Ethertype (e.g. IPv4 `0x0800`, ARP `0x0806`, IPv6 `0x86DD`, 802.1Q tag)
- Payload
- FCS (hardware)

Switches forward using **destination MAC** and the VLAN context (next chapter). They **learn** from **source MAC**.

Learning and flooding

Event	Switch action
Frame arrives with source MAC S on port P	Learn S → P (for that VLAN)
Destination MAC known	Forward out that port only (not ingress)
Destination unknown unicast	Flood within VLAN
Broadcast (ff:ff:ff:ff:ff:ff)	Flood within VLAN
Multicast	Flood or constrain (IGMP snooping later world)

Stale entries age out (aging timer). Moves update the port mapping—important when you relocate a VM/container.

MAC addresses

- 48-bit, usually written `aa:bb:cc:dd:ee:ff`
- Locally administered vs universal bits exist; labs often show container random MACs

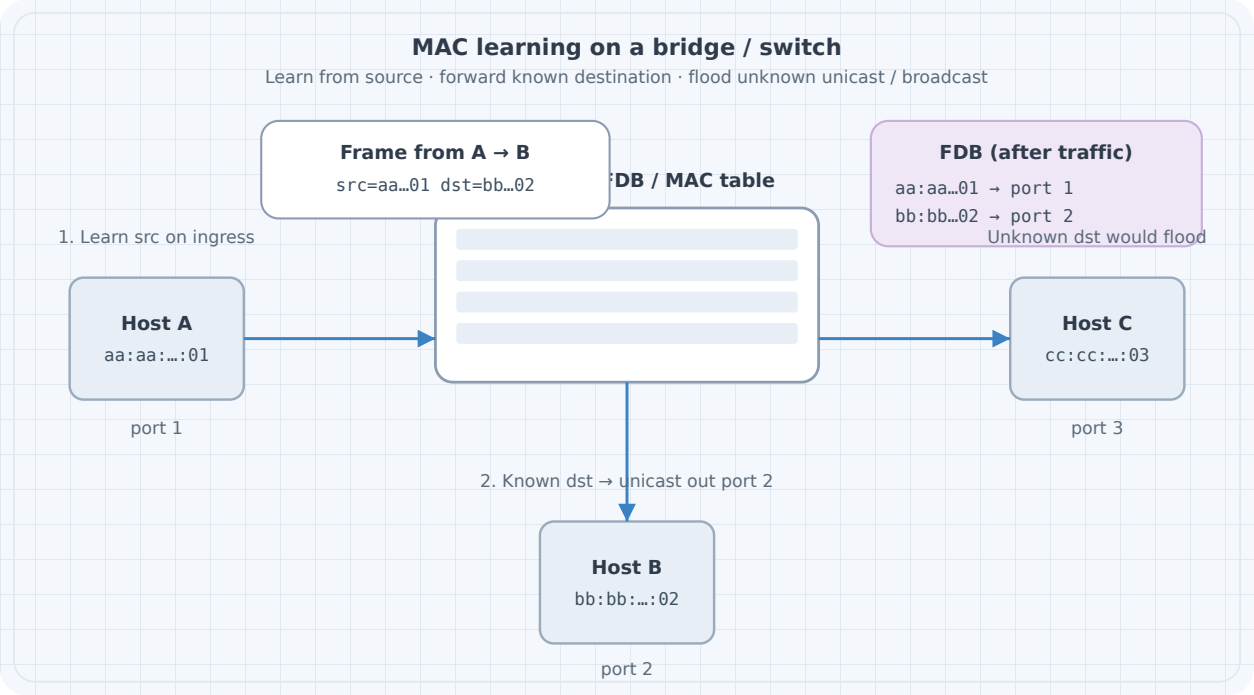


Figure 2: MAC learning: learn source, forward or flood destination

- Do not design security that only trusts MAC equality (easy to spoof)

```
ip link show eth1
# link/ether xx:xx:xx:xx:xx:xx
```

ARP: glue for IPv4 on Ethernet

On-link IPv4 needs ARP:

1. Host knows dest IP is on-link (same subnet / connected route)
2. Who has IP X? Tell Y — broadcast
3. Reply unicast with MAC
4. Neigh cache stores IP MAC

```
ip neigh show
ip neigh flush dev eth1   # lab use carefully
```

IPv6 uses Neighbor Discovery (ICMPv6) instead of ARP—same role, different messages.

Linux bridge FDB

```
# Create bridge and show FDB after traffic
bridge fdb show br br0
bridge link show
```

Dynamic entries learn from traffic; static entries can be added for experiments.

Lab A: two hosts, one bridge (hand or Containerlab)

Containerlab topology

```
name: l2-mac

topology:
  nodes:
    br1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 bridge
        - ip link add name br0 type bridge
        - ip link set br0 up
        # attach eth1/eth2 after links exist - see note below
    h1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 10.10.0.1/24 dev eth1
        - ip link set eth1 up
    h2:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 10.10.0.2/24 dev eth1
        - ip link set eth1 up

  links:
    - endpoints: ["h1:eth1", "br1:eth1"]
    - endpoints: ["h2:eth1", "br1:eth2"]
```

Operational note: Alpine bridge setup via `exec` may race; a mounted `entrypoint.sh` is more reliable:

```
#!/bin/sh
set -e
apk add --no-cache iproute2 bridge >/dev/null
ip link add name br0 type bridge 2>/dev/null || true
ip link set br0 up
ip link set eth1 master br0
ip link set eth2 master br0
ip link set eth1 up
ip link set eth2 up
sleep infinity
```

Bind and set as command for `br1`. Hosts keep simple addressing execs.

Predict

- First ping causes ARP broadcast flooded to the other host
- FDB on `br0` learns h1 and h2 MACs on respective ports
- Subsequent unicast ICMP is switched, not broadcast

Observe

```
docker exec clab-l2-mac-h1 ping -c 3 10.10.0.2
docker exec clab-l2-mac-br1 bridge fdb show br br0
docker exec clab-l2-mac-h1 ip neigh
docker exec clab-l2-mac-h1 tcpdump -ni eth1 -e -c 20 arp or icmp
```

Fix inject

```
# Wrong mask on h2 - still "same numbers" but off-link behavior
docker exec clab-l2-mac-h2 ip addr flush dev eth1
docker exec clab-l2-mac-h2 ip addr add 10.10.0.2/32 dev eth1
docker exec clab-l2-mac-h2 ip link set eth1 up
docker exec clab-l2-mac-h1 ping -c 2 10.10.0.2
```

Restore `/24`. Journal the symptom.

Harden

```
#!/usr/bin/env bash
set -euo pipefail
docker exec clab-l2-mac-h1 ping -c 2 -W 1 10.10.0.2
docker exec clab-l2-mac-br1 bridge fdb show br br0 | grep -q .
echo OK
```

Lab B: three hosts — flooding visibility

Add `h3` on `10.10.0.3/24` linked to `br1:eth3`.

Predict: Unknown unicast to a silent host floods; after traffic, FDB constrains.

Observe: tcpdump on h3 while h1 pings h2—after learning, h3 should **not** see unicast ICMP (ideal case). During ARP storms or failures, you may still see broadcasts.

```
docker exec clab-l2-mac-h3 tcpdump -ni eth1 -c 30 &
docker exec clab-l2-mac-h1 ping -c 5 10.10.0.2
```

MAC moves and flaps

```
# Simulate "move": shut one path (in multi-path labs) or change container
# Observe FDB port update and short loss
bridge fdb show br br0
```

Duplicate MACs on two ports (misconfig or bridge loops) cause flapping—spanning tree chapter addresses loops.

Common L2 failures (pre-VLAN)

Symptom	Likely cause
No ARP reply	Far host down, wrong cable/port, filter
ARP ok, no ping	Local firewall, wrong IP on host
Intermittent	MAC flap, duplex issues (physical), loop
Works after first flood only	Misread; check FDB aging and silent nodes

Promiscuous mode and labs

Bridges and captures may require promisc on interfaces:

```
ip link set eth1 promisc on
tcpdump -ni eth1
```

Containerlab links typically allow node-local captures on data interfaces.

Security digression (short)

- Port security / sticky MAC exist on enterprise gear—concept: limit learned MANs
- DHCP snooping / dynamic ARP inspection—later ops world
- For this book: understand **why** spoofing works at L2 so you do not mythologize the LAN

Predict worksheet (paper)

Domain with switch S and hosts A,B,C. A sends to B's IP first time.

1. Is the first frame after ARP unicast or might B's MAC still be unknown?
2. Does C receive the ARP request?
3. Does C receive the ICMP echo request once FDB learned?

Answers: (1) after ARP, A knows B's MAC—frame is unicast; switch floods only if FDB miss on B. (2) yes, ARP request is broadcast. (3) no, not if learning completed and no loop/mirror.

FRR / NOS note

Hardware switches expose `show mac address-table` equivalents. Linux `bridge fdb` is the same idea. When you use SR Linux or others later, translate:

Intent	Linux bridge	Typical NOS language
Show learned MACs	`bridge fdb show`	show mac table
Clear dynamic	`bridge fdb del ...`	clear mac

Summary

- Switches learn source MACs, forward known unicasts, flood unknowns/broadcasts
- ARP/ND bind IP to MAC on-link

- FDB + neigh + tcpdump diagnose most “L2 should work” failures
- Build bridge labs with Containerlab Linux nodes before complex VLAN designs

Next: **VLANs and trunks**—splitting broadcast domains without new cables.

VLANs & Trunks

VLANs & Trunks

VLANs split one physical switching fabric into multiple **broadcast domains**. Trunks carry many VLANs between switches with 802.1Q tags. Mis-matched tags and native VLANs are classic outage factories—this chapter makes them deliberate.

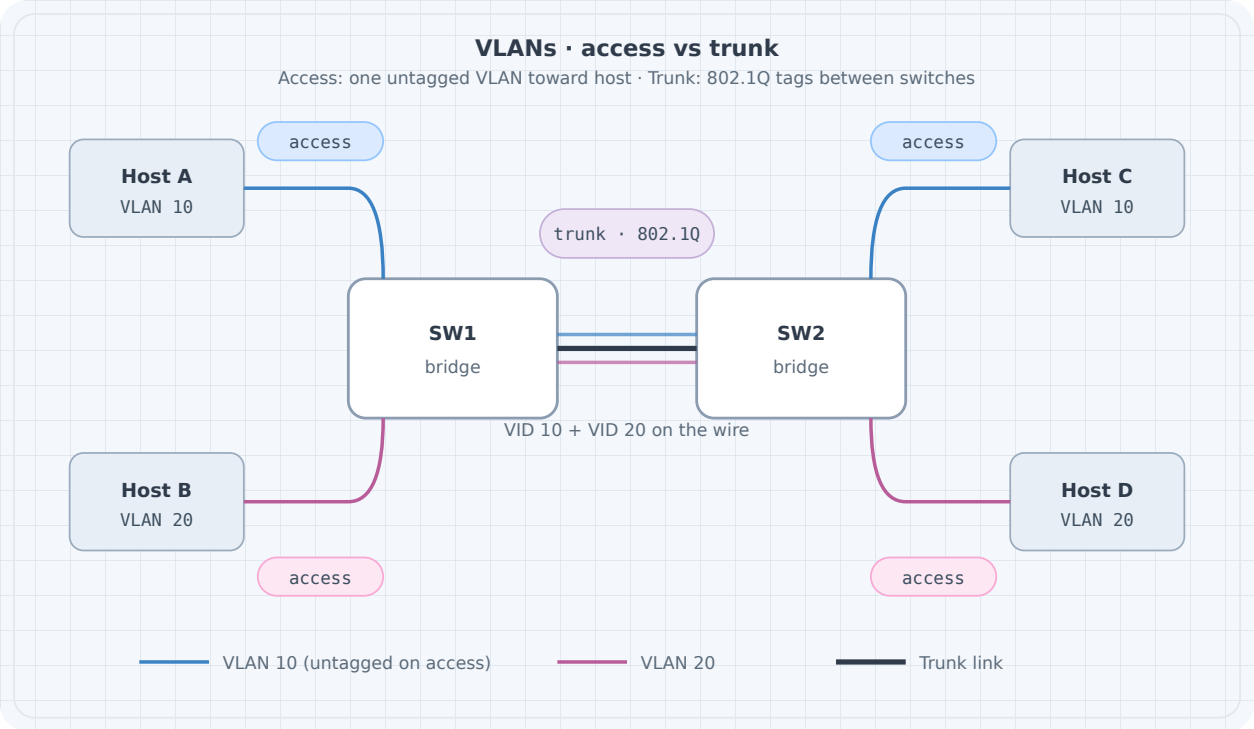


Figure 1: Access ports versus 802.1Q trunk

Learning goals

By the end of this chapter you can:

- Explain access vs trunk ports
- Design a multi-VLAN lab with consistent IDs
- Predict tag behavior on the wire
- Spot native VLAN mismatch symptoms

- Verify isolation between VLANs without a router

Concepts

Broadcast domains

Without VLANs, one bridge = one flood domain. VLAN-aware bridging tags frames with a VLAN ID (VID) and keeps separate FDBs per VLAN (implementation details vary, model holds).

Port mode	Behavior
Access	One VLAN; frames untagged on the wire toward the host
Trunk	Multiple VLANs; tags on the wire (except optional native)
Native VLAN	Untagged traffic on a trunk is mapped to this VLAN

802.1Q tag

Inserts TPID `0x8100` + PCP/DEI + VID. Capture shows tagged frames between switches:

```
tcpdump -ni eth1 -e vlan
# or
tcpdump -ni eth1 -e | head
```

Router roles

Inter-VLAN traffic needs an L3 device (router-on-a-stick, SVI, separate routed ports). Same switch + different VLANs **no IP connectivity** by design.

Addressing plan example

VLAN	Name	Subnet	Access ports
10	users	10.10.10.0/24	h1
20	iot	10.10.20.0/24	h2
99	native/mgmt lab	10.10.99.0/24	optional

Trunk between `sw1` and `sw2` allows 10,20 (and 99 if used).

Lab topology (conceptual)

```
h1 --(access VLAN 10)-- sw1 ==trunk== sw2 --(access VLAN 10)-- h3
h2 --(access VLAN 20)-- sw1 ==trunk== sw2 --(access VLAN 20)-- h4
```

Containerlab sketch using Linux VLAN-aware bridge

Full production-grade switch images are optional; Linux can teach tags.

```
name: l2-vlan

topology:
  nodes:
    sw1:
      kind: linux
      image: alpine:3.20
      binds:
        - ./config/sw1.sh:/startup.sh
      cmd: /bin/sh /startup.sh
    sw2:
      kind: linux
      image: alpine:3.20
      binds:
        - ./config/sw2.sh:/startup.sh
      cmd: /bin/sh /startup.sh
    h1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 10.10.10.1/24 dev eth1
        - ip link set eth1 up
    h2:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 10.10.20.1/24 dev eth1
        - ip link set eth1 up
    h3:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 10.10.10.3/24 dev eth1
```

```
        - ip link set eth1 up
    h4:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 10.10.20.4/24 dev eth1
        - ip link set eth1 up

  links:
    - endpoints: ["h1:eth1", "sw1:eth1"]
    - endpoints: ["h2:eth1", "sw1:eth2"]
    - endpoints: ["h3:eth1", "sw2:eth1"]
    - endpoints: ["h4:eth1", "sw2:eth2"]
    - endpoints: ["sw1:eth3", "sw2:eth3"]
```

`config/sw1.sh` (VLAN filtering bridge)

```
#!/bin/sh
set -e
apk add --no-cache iproute2 bridge >/dev/null
ip link add br0 type bridge vlan_filtering 1
ip link set br0 up

# access ports
for i in 1 2; do ip link set eth$i up; ip link set eth$i master br0; done
ip link set eth3 up
ip link set eth3 master br0

# clear default VLAN 1 membership as needed; set access VLANs
bridge vlan add vid 10 dev eth1 pvid untagged
bridge vlan del vid 1 dev eth1 2>/dev/null || true
bridge vlan add vid 20 dev eth2 pvid untagged
bridge vlan del vid 1 dev eth2 2>/dev/null || true

# trunk eth3: tagged 10,20
bridge vlan add vid 10 dev eth3
bridge vlan add vid 20 dev eth3
bridge vlan del vid 1 dev eth3 2>/dev/null || true

# bridge self VLANs
bridge vlan add vid 10 dev br0 self
bridge vlan add vid 20 dev br0 self

sleep infinity
```

Mirror on `sw2` with eth1→VLAN10, eth2→VLAN20, eth3 trunk.

Kernel/`bridge` command details vary slightly by version—if a flag fails, check `man bridge` on the image and adjust. The **model** (pvid untagged access, tagged trunk) is the learning target.

Predict → observe → fix

Predict

- h1 h3 (VLAN 10) ping works
- h2 h4 (VLAN 20) ping works
- h1 h2 fails (no L3 inter-VLAN)
- Capture on trunk shows VLAN tags for inter-switch frames

Observe

```
docker exec clab-l2-vlan-h1 ping -c 2 10.10.10.3
docker exec clab-l2-vlan-h2 ping -c 2 10.10.20.4
docker exec clab-l2-vlan-h1 ping -c 1 -W 1 10.10.20.1  # expect fail
docker exec clab-l2-vlan-sw1 bridge vlan show
docker exec clab-l2-vlan-sw1 tcpdump -ni eth3 -e -c 20
```

Failure drill: native / PVID mismatch

On sw1 trunk, map untagged to VLAN 10; on sw2 map untagged to VLAN 20. Send untagged (or mis-access) traffic.

Symptoms often seen:

- Silent discard
- Hosts “leak” into wrong segment
- Intermittent weirdness depending on tagging

Harden: Explicitly tag all lab VLANs on trunks; avoid relying on native VLAN for user data. Document native VLAN ID identically on both ends if you must use it.

Failure drill: missing allowed VLAN

Remove VLAN 20 from sw2 trunk membership.

Predict: VLAN 10 still works; VLAN 20 across switches dies.

```
# on sw2, delete vid 20 from eth3, retest h2->h4
```

Router-on-a-stick preview

```
sw trunk --- r1 eth1.10 + eth1.20
```

```
ip link add link eth1 name eth1.10 type vlan id 10
ip link add link eth1 name eth1.20 type vlan id 20
ip addr add 10.10.10.254/24 dev eth1.10
ip addr add 10.10.20.254/24 dev eth1.20
ip link set eth1.10 up
ip link set eth1.20 up
sysctl -w net.ipv4.ip_forward=1
```

Hosts use `.254` as gateway. Full L3 chapters expand routing; here you only prove **VLAN separation needs L3 to merge**.

Verification checklist

```
# Per switch
bridge vlan show
bridge fdb show

# Per host
ip -br a
ip neigh
ping -c 2 <same-vlan-far-host>
ping -c 1 -W 1 <other-vlan-host>   # should fail without router
```

verify.sh sketch

```
#!/usr/bin/env bash
set -euo pipefail
docker exec clab-l2-vlan-h1 ping -c 2 -W 1 10.10.10.3
docker exec clab-l2-vlan-h2 ping -c 2 -W 1 10.10.20.4
if docker exec clab-l2-vlan-h1 ping -c 1 -W 1 10.10.20.1; then
```

```
  echo "FAIL: inter-VLAN should be blocked" >&2
  exit 1
fi
echo OK
```

Negative checks are gold.

Design tips

Tip	Why
Plan VLAN IDs globally in the lab	Avoid "VLAN 10 means different things"
Do not reuse VLAN 1 for user data	Habit for real gear hygiene
Match trunk allow lists both ends	One-way or partial connectivity
Document native VLAN	Mismatch class of bugs
Separate voice/data IDs even in labs	Practice real patterns

Common mistakes

Mistake	Result
Access port in wrong VLAN	ARP never reaches partner
Trunk missing VID	Works locally, fails across switches
Same subnet on two VLANs	Broken design + asymmetric mess
Expecting VLANs to route	Need L3

Summary

- VLANs partition broadcast domains; trunks carry tagged sets between switches
- Access = untagged one VID; trunk = tagged many VIDs
- Align allow lists and native VLAN on both ends
- Prove isolation with intentional negative pings
- Inter-VLAN = router/SVI job

Next: **loop prevention**—what happens when redundant L2 paths exist without a control protocol.

Loop Prevention

Loop Prevention

Redundant L2 links are good for resilience and catastrophic without loop control. Broadcasts and unknown unicasts circulate forever; MAC tables flap; the LAN melts. This chapter covers the **spanning tree family** as a model (not a vendor certification deep dive) and how to see loops in the lab.

Learning goals

By the end of this chapter you can:

- Explain why Ethernet needs loop prevention
- Describe root bridge election and port roles at a conceptual level
- Predict a blocked port in a simple triangle of switches
- Build a loop, observe the blast, then enable prevention
- Prefer design alternatives when L2 redundancy is the wrong tool

Why loops kill L2

Ethernet has no TTL on frames. In a loop:

1. Broadcast / unknown unicast is flooded
2. Copies multiply around the cycle
3. CPUs and links saturate
4. Learning oscillates (MAC seen on alternating ports)

IP routers do not save you inside a pure L2 domain.

Spanning tree behavior

Classic STP/RSTP/MSTP ideas:

Idea	Meaning
Root bridge	Reference switch (lowest priority/BID wins)
Root port	Each non-root switch's best path toward root
Designated port	Best port on a segment toward downstream
Blocked / alternate	Redundant path held in reserve
BPDU	Control frames that build the tree

Modern networks often use **RSTP** or vendor rapid variants; data centers may avoid large L2 domains entirely (L3 fabric). Still, you must recognize STP language on campus gear and many NOS features.

Priority and ties

Lower bridge priority wins. Ties break by MAC. In labs, set priorities explicitly so root placement is intentional:

```
sw1 priority better (numerically lower) → root
sw2/sw3 farther
```

Port states (classic story)

Blocking → Listening → Learning → Forwarding (RSTP simplifies roles/states). During convergence, expect transient blackholes—measure them in drills.

Design alternatives (model maturity)

Approach	Loop story
STP/RSTP on bridged triangle	Protocol blocks one link
L3 access / routed triangle	No L2 loop; ECMP at L3
MC-LAG / bundling	Multi-chassis tricks (complex)
Single homing	No redundancy, no loop

For many greenfield labs after this chapter, **prefer L3 redundancy**. Learn STP so you can operate brownfield and understand failure domains.

Lab: three switches, intentional loop

```
    sw1
   /   \
 sw2---sw3
```

Hosts on sw2 and sw3.

Topology YAML (Linux bridges)

```yaml
name: l2-loop

topology:
  nodes:
    sw1:
      kind: linux
      image: alpine:3.20
      binds: ["./config/sw-loop.sh:/startup.sh"]
      cmd: /bin/sh /startup.sh
    sw2:
      kind: linux
      image: alpine:3.20
      binds: ["./config/sw-loop.sh:/startup.sh"]
      cmd: /bin/sh /startup.sh
    sw3:
      kind: linux
      image: alpine:3.20
      binds: ["./config/sw-loop.sh:/startup.sh"]
      cmd: /bin/sh /startup.sh
    h2:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 10.10.0.2/24 dev eth1 && ip link set eth1 up
    h3:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 10.10.0.3/24 dev eth1 && ip link set eth1 up

  links:
    - endpoints: ["sw1:eth1", "sw2:eth1"]
    - endpoints: ["sw1:eth2", "sw3:eth1"]
```

```
    - endpoints: ["sw2:eth2", "sw3:eth2"]
    - endpoints: ["h2:eth1", "sw2:eth3"]
    - endpoints: ["h3:eth1", "sw3:eth3"]
```

Dangerous mode: dumb bridges without STP

```
#!/bin/sh
# config/sw-loop.sh - NO STP (for short controlled drill only!)
set -e
apk add --no-cache iproute2 bridge >/dev/null
ip link add br0 type bridge
ip link set br0 up
for i in 1 2 3; do
  ip link set eth$i up 2>/dev/null || true
  ip link set eth$i master br0 2>/dev/null || true
done
sleep infinity
```

Warning: Run the loop experiment **briefly** on an isolated lab host. Broadcast storms can stress CPU.

Observe storm (careful)

```
# terminal 1
docker exec clab-l2-loop-h2 ping -f 10.10.0.3 &
# or send broadcasts
docker exec clab-l2-loop-sw1 ip -s link
# counters explode
```

Destroy promptly:

```
sudo containerlab destroy -t l2-loop.clab.yml --cleanup
```

Safer mode: enable STP on the bridge

Linux bridge STP:

```
ip link add br0 type bridge
ip link set br0 type bridge stp_state 1
ip link set br0 up
# attach ports...
```

```
# inspect
docker exec clab-l2-loop-sw1 cat /sys/class/net/br0/bridge/stp_state
# port states under /sys/class/net/br0/brif/*/state
```

Predict: One of the triangle links is blocked; ping between h2 and h3 still works via remaining tree; storm does not persist.

Observe:

```
docker exec clab-l2-loop-h2 ping -c 5 10.10.0.3
# check which port is blocking on each sw
for s in sw1 sw2 sw3; do
  echo "== $s =="
  docker exec clab-l2-loop-$s sh -c 'for p in /sys/class/net/br0/brif/*/state; do echo $p $(
done
```

Root placement drill

Influence root by bridge priority (when using STP implementations that expose it):

```
# example sysfs / tools depend on stack; concept: make sw1 root
ip link set br0 type bridge priority 0x1000
```

On NOS images (SR Linux, etc.), use vendor STP priority knobs—same election idea.

Predict: With sw1 root, blocked port is likely on the sw2–sw3 link (common triangle outcome with symmetric costs).

Verify: Diagram the tree; mark root ports and blocked port.

Failure drill: link down on active tree edge

1. Baseline ping
2. Shut a forwarding uplink
3. Measure time until ping recovers
4. Confirm previously blocked port becomes forwarding

```
docker exec clab-l2-loop-sw1 ip link set eth1 down
# time recovery
docker exec clab-l2-loop-h2 ping -c 20 10.10.0.3
docker exec clab-l2-loop-sw1 ip link set eth1 up
```

Journal convergence feel (RSTP vs classic differs a lot).

BPDUs and protection concepts (awareness)

Feature idea	Intent
BPDU guard	Err-disable access port if BPDU seen
Root guard	Prevent downstream from becoming root
Loop guard	Protect against unidirectional link issues

You may not implement all in Alpine labs; know the **jobs** so production configs make sense.

When not to extend L2

Smell	Prefer
L2 stretched across sites	L3 + overlay if needed
Huge broadcast domain	Segment VLANs + route
STP diameter anxiety	Routed access / leaf-spine

Spanning tree is a safety net, not an excuse for continent-sized broadcast domains.

Predict worksheet

Triangle swA–swB–swC, all costs equal, swA root.

1. Which link is most likely blocked?
2. Host on swB to host on swC: path?
3. If swA dies, what must happen?

Answers: (1) often B–C; (2) B–A–C; (3) re-election, former blocked may forward, connectivity among survivors if links remain.

Verification checklist for STP labs

- Never leave an intentional storm lab running unattended
- Record before/after port states
- Compare L2 redundancy recovery to L3 ECMP recovery later—feel the difference

Summary

- L2 loops multiply floods; no frame TTL
- Spanning tree elects a root and blocks redundant ports
- Practice triangle labs with STP on; only briefly demo storms
- Root placement is a design choice
- Mature designs often push redundancy to L3

Next: **link aggregation**—bundling links for bandwidth and failover without creating an independent loop story (when configured correctly).

Link Aggregation

Link Aggregation

Link aggregation (LAG / bonding / etherchannel-class ideas) combines multiple physical links into one logical link: more bandwidth, faster failover, and fewer STP blocked paths between the same pair of devices—when both ends agree.

Learning goals

By the end of this chapter you can:

- Explain when LAG helps vs when it hides problems
- Contrast static bundle vs LACP
- Configure a Linux bond in a lab
- Predict traffic behavior and failover
- Verify member health and hashing limitations

Concepts

One logical link

```
     +---- eth1 ----+
sw1 -+              +- sw2
     +---- eth2 ----+
           \____ bond0 / lag0 ____/
```

Upper protocols (STP, L3 adjacency) see **one** link if bundling is correct.

Static vs LACP

Mode	Pros	Cons
Static / on	Simple	Mis-wiring may blackhole; no negotiation

LACP	Detects partner; protects against some mistakes	Both ends must support/enable

Labs may start static; production almost always prefers LACP.

Hashing

Flows distribute across members by a hash (MAC/IP/L4 ports depending on config). **One TCP flow** typically stays on one member—do not expect a single iperf stream to fill N× bandwidth.

Split brain / misconfig

Classic fail: one side bundles, the other has independent ports—loops or blackholes. Always verify both ends.

Linux bonding quickstart

Kernel bonding modes (selection depends on goal):

Mode	Name	Notes
0	balance-rr	Round-robin
1	active-backup	Simple failover
4	802.3ad	LACP

```
# Example active-backup (conceptual host commands)
modprobe bonding
ip link add bond0 type bond mode active-backup miimon 100
ip link set eth1 down
ip link set eth2 down
ip link set eth1 master bond0
ip link set eth2 master bond0
ip addr add 10.0.0.1/24 dev bond0
ip link set bond0 up
ip link set eth1 up
ip link set eth2 up
cat /proc/net/bonding/bond0
```

In containers, bonding support depends on privileges and kernel modules on the **host**. Containerlab labs sometimes simulate aggregation at the NOS layer instead. If bonding is blocked in your environment, run this chapter on a Linux VM/nested host and keep Containerlab for endpoints.

Containerlab topology (two paths, bond on routers)

```
name: l2-lag

topology:
  nodes:
    r1:
      kind: linux
      image: alpine:3.20
      binds:
        - ./config/r1-bond.sh:/startup.sh
      cmd: /bin/sh /startup.sh
      # may need privileged: true depending on clab version/kind options
    r2:
      kind: linux
      image: alpine:3.20
      binds:
        - ./config/r2-bond.sh:/startup.sh
      cmd: /bin/sh /startup.sh
    h1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 192.168.1.10/24 dev eth1 && ip link set eth1 up
        - ip route add default via 192.168.1.1
    h2:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 192.168.2.10/24 dev eth1 && ip link set eth1 up
        - ip route add default via 192.168.2.1

  links:
    - endpoints: ["h1:eth1", "r1:eth3"]
    - endpoints: ["h2:eth1", "r2:eth3"]
    - endpoints: ["r1:eth1", "r2:eth1"]
    - endpoints: ["r1:eth2", "r2:eth2"]
```

config/r1-bond.sh

```
#!/bin/sh
set -e
apk add --no-cache iproute2 iputils >/dev/null
# fallback if bonding unavailable: sequential routes demo only
if ip link add bond0 type bond mode 802.3ad 2>/dev/null; then
  ip link set eth1 down
  ip link set eth2 down
  ip link set eth1 master bond0
  ip link set eth2 master bond0
  ip addr add 10.0.0.1/24 dev bond0
  ip link set bond0 up
  ip link set eth1 up
  ip link set eth2 up
else
  echo "bonding unavailable; configure two links manually for failover drill" >&2
  ip addr add 10.0.0.1/24 dev eth1
  ip link set eth1 up
  ip link set eth2 up
fi
ip addr add 192.168.1.1/24 dev eth3
ip link set eth3 up
sysctl -w net.ipv4.ip_forward=1
ip route add 192.168.2.0/24 via 10.0.0.2
sleep infinity
```

Symmetric on r2 with `10.0.0.2`, LAN `192.168.2.1/24`, route to `192.168.1.0/24 via 10.0.0.1`.

If LACP mode fails without a partner daemon, use `mode active-backup` for reliable lab failover learning.

Predict → observe → fix

Predict

- h1→h2 ping works through logical path
- Shutting **one** member keeps session alive (active-backup or LACP with remaining member)
- Single flow throughput one member speed

Observe

```
docker exec clab-l2-lag-h1 ping -c 5 192.168.2.10
docker exec clab-l2-lag-r1 cat /proc/net/bonding/bond0 2>/dev/null || true
docker exec clab-l2-lag-r1 ip -br link
```

Failover drill

```
docker exec clab-l2-lag-r1 ip link set eth1 down
docker exec clab-l2-lag-h1 ping -c 10 192.168.2.10
docker exec clab-l2-lag-r1 ip link set eth1 up
```

Measure loss count. Journal recovery time.

Mismatch drill

On r2, do **not** enslave eth2; leave as separate interface with another IP.

Predict: loops, flaps, or partial connectivity—document what **your** kernel does; fix by matching configs.

Hashing awareness drill

```
# multiple parallel streams may use more members than one stream
# on hosts with iperf3
iperf3 -s   # h2
iperf3 -c 192.168.2.10 -P 8
```

Compare `-P 1` vs `-P 8`.

STP interaction

Between two switches, a multi-link LAG is one logical link STP sees one edge. Independent unbundled links STP blocks all but one (or you loop). **Bundle before you celebrate "redundant cables."**

Check	Intent

Operational checks (NOS-neutral)

Check	Intent
Members all up	Physical/L1
LACP PDUs both ways	Negotiation
Same speed/duplex	Bundle eligibility
Counters increment on members	Actual use
Control plane adjacency single	Logical link view

Common mistakes

Mistake	Result
One end LACP, other static only	Stuck negotation / down
Different VLANs on members	Inconsistent bundle
Expect one flow = N x bandwidth	Disappointed benchmarks
Forget host firewall on bond MAC	Odd ARP

Summary

- LAG = one logical link from multiple physical members
- Prefer LACP in real life; static only with tight control
- Hashing limits single-flow throughput
- Verify both ends; failover-test by shutting members
- Unbundled parallel L2 links reintroduce STP/loop problems

Next: **L2 failure drills**—deliberate breakage across MAC, VLAN, loop, and bundle scenarios with evidence.

L2 Failure Drills

L2 Failure Drills

This chapter is a **practice arena**. You combine Ethernet, VLANs, loop prevention, and aggregation under intentional failure. Competence is measured by evidence and recovery, not by green first pings.

Learning goals

By the end of this chapter you can:

- Run a structured failure catalog against an L2 lab
- Collect the minimum evidence set for each class of break
- Recover cleanly and re-verify
- Write short incident notes a peer can follow
- Know when an "L2 issue" is actually L3 or host firewall

Baseline lab

Use a multi-switch VLAN lab (from the VLANs chapter) with:

- Two switches, trunk between them
- VLAN 10 and VLAN 20
- Hosts h1/h3 in VLAN 10, h2/h4 in VLAN 20
- Optional inter-VLAN router for a subset of drills

Document addressing in `addressing.md`. Ensure `./verify.sh` is green before any inject.

Evidence kit

```
# Host
ip -br a
ip neigh
ping -c 3 -W 1 <target>
tcpdump -ni eth1 -e -c 30 arp or icmp

# Switch / bridge
bridge vlan show
bridge fdb show
ip -s link
# STP port states if enabled
```

Save outputs under `journal.md` with timestamps.

Drill catalog

For each drill: **predict → inject → observe → restore → harden**.

Drill 1 — Access port wrong VLAN

Inject: Move h1's switch port from VLAN 10 to VLAN 20 (PVID change).

Predict: h1 cannot ARP h3; may see wrong subnet peers if any; verify.sh fails.

Observe: `bridge vlan show`, tcpdump for ARP who-has never answered by h3.

Restore: Correct PVID/untagged VLAN.

Harden: Label ports in diagrams; negative test in verify (optional).

Drill 2 — Trunk missing VLAN

Inject: Remove VLAN 10 from trunk allow list on one switch only.

Predict: Local VLAN 10 still works on that switch; cross-switch VLAN 10 dies; VLAN 20 may still work.

Observe: Asymmetric "works on same switch only."

Restore: Re-add VID on both ends.

Harden: Checklist "both ends allow list."

Drill 3 — Native VLAN mismatch

Inject: Native/PVID untagged mapping differs on trunk ends.

Predict: Untagged frames land in different broadcast domains; confusing intermittent leaks.

Observe: Captures show untagged where you expected tags; wrong FDB.

Restore: Align native VLAN; prefer tagged-only user VLANs.

Harden: Explicit lab policy: native VLAN unused for data.

Drill 4 — Cable / interface down

Inject:

```
docker exec clab-l2-vlan-sw1 ip link set eth3 down
```

Predict: Cross-switch traffic dies; FDB ages; local switching may live.

Observe: ping loss; interface counters; STP reconvergence if redundant path exists.

Restore: `ip link set eth3 up`; wait for learning/STP.

Harden: If no redundant path, document single point of failure.

Drill 5 — Broadcast storm (controlled)

Inject: Temporary bridging loop **without** STP on an isolated lab (very short).

Predict: CPU spikes, loss of management responsiveness, exploding counters.

Observe: `ip -s link`, host load.

Restore: `containerlab destroy --cleanup` immediately if unstable.

Harden: STP on; never lab this on a shared prod-like bridge.

Drill 6 — STP blocked port confusion

Inject: With STP triangle, shut the active root port path.

Predict: Alternate port activates; brief loss; new tree.

Observe: Port state sysfs or NOS show; ping `-c 50` loss burst.

Restore: Bring link up; tree may revert depending on costs.

Harden: Note expected reconvergence budget.

Drill 7 — MAC move

Inject: Move host container link from sw1 to sw2 (redeploy link or migrate veth).

Predict: FDB updates; short blackhole until relearn; ARP may refresh.

Observe: `bridge fdb show` before/after; neigh on peers.

Restore: Stable attachment.

Harden: In virtualization, understand port channels / bond moves.

Drill 8 — Duplicate IP

Inject: Configure h4 temporarily with h3's IP on VLAN 10 (careful).

Predict: Flapping ARP, intermittent delivery, wrong neigh.

Observe: `ip neigh` oscillating; tcpdump conflicting ARP replies.

Restore: Unique IPs.

Harden: Addressing plan ownership; IPAM habit.

Drill 9 — Duplicate MAC (advanced)

Inject: Spoof MAC on a second host (lab only).

```
ip link set eth1 down
ip link set eth1 address aa:bb:cc:dd:ee:ff
ip link set eth1 up
```

Predict: FDB flap; unstable delivery.

Restore: Original MAC; destroy/recreate if messy.

Harden: Understand L2 trust boundaries.

Drill 10 — LAG member down

Inject: On a bond/LAG lab, shut one member.

Predict: Traffic continues if min-links satisfied; single-flow rate unchanged.

Observe: `/proc/net/bonding/bond0`; ping continuity.

Restore: Member up.

Harden: Alert on member-down even if bond stays up.

Drill 11 — Inter-VLAN path mistaken for L2

Inject: Delete router inter-VLAN route or shut subinterface.

Predict: Same-VLAN ok; cross-VLAN fails—looks like "VLAN broken" if you misread.

Observe: traceroute never leaves gateway; ARP to gateway fails or gateway unreachable.

Restore: Router config.

Harden: Always check same-VLAN baseline before blaming trunk.

Drill 12 — Host firewall

Inject: On Linux host, drop ICMP.

```
# if nft/iptables available
iptables -A INPUT -p icmp -j DROP
```

Predict: Others cannot ping in; egress may still work.

Observe: Asymmetric ping stories.

Restore: Flush rule.

Harden: Include host endpoint in the failure picture.

Incident note template

```
## Incident: trunk missing VLAN 10 on sw2
Date:
Lab: l2-vlan

### User symptom
h1 cannot reach h3; h1 reaches local same-switch hosts.

### Prediction before fix
Trunk allow list asymmetry.

### Evidence
- bridge vlan show sw1/sw2 (paste)
- tcpdump trunk: no VLAN 10 tags toward sw2
- ping results

### Root cause
vid 10 deleted from sw2 eth3

### Fix
bridge vlan add vid 10 dev eth3

### Prevent
verify.sh checks cross-switch VLAN 10 and 20
README allow-list both ends
```

Scorecard

Run at least **six** drills across different classes. Check:

Class	Drills done
VLAN membership	
Trunk tagging	
Link down	
STP / loop	
Aggregation	
Host/IP mistakes	

Combined verify.sh ideas

```
#!/usr/bin/env bash
set -euo pipefail
# positive
docker exec clab-l2-vlan-h1 ping -c 2 -W 1 10.10.10.3
docker exec clab-l2-vlan-h2 ping -c 2 -W 1 10.10.20.4
# negative isolation
if docker exec clab-l2-vlan-h1 ping -c 1 -W 1 10.10.20.4 2>/dev/null; then
  echo "unexpected inter-VLAN without router" >&2
  exit 1
fi
echo OK
```

After each restore, re-run.

Checkpoint: L2 competence

You pass the part when you can:

1. Deploy multi-switch VLANs from code
2. Produce evidence for tag/allow-list bugs
3. Explain and demo loop risk
4. Fail over a link or LAG member deliberately
5. Write an incident note with root cause and prevention

Summary

- Failure drills convert L2 theory into ops judgment

- Always baseline with verify.sh
- Predict before inject; restore and re-verify after
- Separate L2, L3, and host causes with evidence
- Incident notes are part of the curriculum artifact

Next part: **layer 3**—addressing, routing fundamentals, static labs, ICMP and path MTU.

Layer 3

IP Addressing

IP Addressing

Forwarding starts with **addressing**: structure, masks, subnets, summarization, and enough IPv6 to run dual-stack labs. This chapter is calculation plus lab verification—not exam trick puzzles for their own sake.

Learning goals

By the end of this chapter you can:

- Convert between prefix length and masks fluently
- Design subnets for a multi-tier lab without overlaps
- Summarize contiguous blocks and predict blackholes from bad summaries
- Configure IPv4 and basic IPv6 on Linux
- Use documentation ranges and lab hygiene ranges deliberately

IPv4 structure

An IPv4 address is 32 bits, usually dotted decimal. A **prefix** `address/len` identifies network bits vs host bits.

Prefix	Mask	Hosts (approx usable)
/32	255.255.255.255	1 (host route)
/31	255.255.255.254	2 (P2P often)
/30	255.255.255.252	2 usable classic
/24	255.255.255.0	254 usable classic
/16	255.255.0.0	large
/8	255.0.0.0	huge

```
# Linux
ip addr add 192.0.2.10/24 dev eth1
ip -br a
ip route
```

On-link decision

If destination is inside a connected prefix, the host uses ARP/ND on that interface. Otherwise it sends to a gateway (route lookup).

```
ip route get 192.0.2.50
ip route get 8.8.8.8
```

Subnetting for labs

Example campus-ish allocation from `10.20.0.0/16`:

Use	Block
Users site A	10.20.1.0/24
Users site B	10.20.2.0/24
Servers	10.20.10.0/24
P2P core	10.20.255.0/24 as /30s
Loopbacks	10.20.254.0/24 as /32s

Rules:

1. No overlapping prefixes on different segments unless deliberate anycast
2. Leave growth room
3. Write the plan before deploy
4. Align summaries with actual aggregation topology

Summarization

If you own `10.20.1.0/24` and `10.20.2.0/24`, a summary `10.20.0.0/22` may or may not be correct depending on other contents of that /22.

Action	Risk
Summary advertises space you do not have	Blackhole for missing parts
Too-specific everywhere	Table bloat (later IGP/BGP scale)
Forgetting more-specific local routes	Traffic steers wrong

Paper drill

You have:

- 10.0.0.0/25
- 10.0.0.128/25

Correct tight summary? `10.0.0.0/24`.

You have only `10.0.0.0/25` but advertise `10.0.0.0/24` upstream without a sink. What happens to 10.0.0.200? **Blackhole** toward you.

Special-use ranges (know them)

Range	Role
10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16	Private (RFC 1918)
127.0.0.0/8	Loopback
169.254.0.0/16	Link-local
192.0.2.0/24, 198.51.100.0/24, 203.0.113.0/24	Documentation (RFC 5737)
224.0.0.0/4	Multicast

IPv6 essentials for dual-stack labs

IPv6 address is 128 bits; common lab prefix `/64` on LANs.

```
ip -6 addr add 2001:db8:1::10/64 dev eth1
ip link set eth1 up
ip -6 route
ip -6 neigh
ping -6 -c 2 2001:db8:1::1
```

Prefix idea	Example
Documentation	`2001:db8::/32`
Unique local	`fd00::/8` (with local generation rules)
Link-local	`fe80::/10` auto on interfaces

ND replaces ARP. Router Advertisements may autoconfigure hosts—labs often use statics for control.

Dual-stack habit

Give nodes both families when learning:

```
ip addr add 10.1.0.10/24 dev eth1
ip -6 addr add 2001:db8:1::10/64 dev eth1
```

Test both paths; do not assume v4 success means v6 works.

Linux configuration patterns

Persistent lab style

In Containerlab, prefer startup scripts or image config over manual `ip addr` that vanishes.

```
#!/bin/sh
ip addr flush dev eth1
ip addr add 10.1.0.10/24 dev eth1
ip -6 addr add 2001:db8:1::10/64 dev eth1
ip link set eth1 up
ip route add default via 10.1.0.1
ip -6 route add default via 2001:db8:1::1
```

Multiple addresses

```
ip addr add 10.1.0.10/24 dev eth1
ip addr add 10.1.0.11/24 dev eth1
```

Secondary addresses appear in ARP; know who answers.

Lab: addressing plan + dual-stack hosts

```
name: l3-addr

topology:
  nodes:
    r1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - sysctl -w net.ipv4.ip_forward=1
```

```
      - sysctl -w net.ipv6.conf.all.forwarding=1
      - ip addr add 10.1.0.1/24 dev eth1
      - ip addr add 10.2.0.1/24 dev eth2
      - ip -6 addr add 2001:db8:1::1/64 dev eth1
      - ip -6 addr add 2001:db8:2::1/64 dev eth2
      - ip link set eth1 up
      - ip link set eth2 up
  h1:
    kind: linux
    image: alpine:3.20
    exec:
      - apk add --no-cache iproute2 iputils
      - ip addr add 10.1.0.10/24 dev eth1
      - ip -6 addr add 2001:db8:1::10/64 dev eth1
      - ip link set eth1 up
      - ip route add default via 10.1.0.1
      - ip -6 route add default via 2001:db8:1::1
  h2:
    kind: linux
    image: alpine:3.20
    exec:
      - apk add --no-cache iproute2 iputils
      - ip addr add 10.2.0.10/24 dev eth1
      - ip -6 addr add 2001:db8:2::10/64 dev eth1
      - ip link set eth1 up
      - ip route add default via 10.2.0.1
      - ip -6 route add default via 2001:db8:2::1

links:
  - endpoints: ["h1:eth1", "r1:eth1"]
  - endpoints: ["h2:eth1", "r1:eth2"]
```

Predict

- h1 pings h2 v4 and v6 through r1
- Wrong mask on h1 (e.g. /32 only) breaks on-link perception of gateway

Observe

```
docker exec clab-l3-addr-h1 ping -c 2 10.2.0.10
docker exec clab-l3-addr-h1 ping -6 -c 2 2001:db8:2::10
docker exec clab-l3-addr-h1 ip route get 10.2.0.10
docker exec clab-l3-addr-r1 ip -br a
```

Failure: overlapping subnets

Put h2 also in `10.1.0.0/24` while connected to r1 eth2 differently—create intentional overlap and journal chaos (ARP confusion, asymmetric paths). Restore clean plan.

Failure: bad summary (static preview)

On a third router path (optional), advertise/install `10.0.0.0/8` via a dead next hop while more specifics missing—observe blackhole for unused space.

Calculation practice (quick)

1. How many /24s in a /16? → 256
2. First/last usable in 10.0.0.0/30 classic? → 10.0.0.1 and 10.0.0.2
3. Is 10.0.0.64/26 inside 10.0.0.0/24? → yes
4. Can you summarize 10.1.0.0/24 and 10.2.0.0/24 as a single /23? → no (not contiguous bit boundary as one /23 covering both cleanly without others)

Work these on paper until automatic.

Host vs router addressing habits

Role	Typical
Host	One address + default route
Router	Addresses on each L3 interface + routing table
Loopback	Stable /32 or /128 for protocols later

```
ip addr add 1.1.1.1/32 dev lo
```

Verification checklist

```
ip -br a
ip route
ip -6 route
ip route get <dst>
ping -c 2 <dst>
ping -6 -c 2 <dst6>
```

Summary

- Prefixes define on-link vs routed behavior
- Plan non-overlapping lab allocations; document them
- Summaries must match reality or they blackhole
- Dual-stack labs need explicit v6 config and tests
- `ip route get` is your friend

Next: **routing fundamentals**—RIB/FIB, longest match, next hops, and host vs router behavior including ARP/ND.

Routing Fundamentals

Routing Fundamentals

Routing is **longest-prefix match** to a next hop or interface, plus the glue that resolves the next hop to L2. This chapter ties RIB/FIB, metrics, and host vs router behavior into one model you will reuse for statics and dynamic protocols.

Learning goals

By the end of this chapter you can:

- Explain longest prefix match with competing routes
- Distinguish connected, static, and protocol routes
- Resolve next hops (ARP/ND) and see failures when resolution dies
- Use `ip route get` as a decision debugger
- Describe host default-route behavior vs multi-interface routers

Concepts

Forwarding equation

For each packet:

1. Look up destination in FIB
2. Select winning route (longest prefix, then tie-breakers)
3. Find egress interface and L2 rewrite target
4. Decrement TTL / hop limit; drop if zero

Store	Role

RIB vs FIB

Store	Role
RIB	Routing information: all candidates, protocol owners
FIB	What data plane uses for forward

On Linux, `ip route` shows kernel FIB. FRR holds protocol RIB and installs selected routes into the kernel via zebra.

```
ip route
vtysh -c 'show ip route'   # FRR RIB view
```

Longest prefix match

Routes:

- `0.0.0.0/0` via gwA
- `10.0.0.0/8` via gwB
- `10.1.2.0/24` via gwC

Destination `10.1.2.5` → **gwC**. Destination `10.9.0.1` → **gwB**. Destination `1.2.3.4` → **gwA**.

```
ip route add 10.0.0.0/8 via 192.0.2.1
ip route add 10.1.2.0/24 via 192.0.2.2
ip route get 10.1.2.5
```

Metrics and distance (ideas)

Multiple protocols may offer the same prefix. Selection uses administrative preference (distance) then metric. Linux has metrics on routes; FRR has AD per protocol. Exact numbers differ—**model** is ranked sources.

Connected routes

Configuring `10.1.0.1/24` on eth1 installs connected route `10.1.0.0/24 dev eth1`. Connected usually beats static/default for that space.

Host vs router

Host	Router
Often one default route	Many specific routes
Forwards? usually no	`ip_forward=1`
ARP for on-link and gateway	ARP for each next hop per interface
Care about DNS, apps	Care about RIB scale, policy

Gateway ARP

When host sends off-link:

- Destination IP stays the remote
- Ethernet dest is **gateway MAC**

```
# host
ip route get 8.8.8.8
ip neigh show
tcpdump -ni eth1 -e host 8.8.8.8
```

Next-hop resolution failures

Symptom: route exists, ping fails.

```
ip route get 10.2.0.10
# shows via 10.0.0.2 dev eth1
ip neigh show dev eth1
# if FAILED/INCOMPLETE for 10.0.0.2 → L2 problem or peer down
```

Control vs data: Static route "up" in config does not guarantee next hop resolves.

Asymmetric routing

Forward path R1→R2→R3, return R3→R4→R1. Stateless ping may work; stateful firewalls and some reverse-path filters break.

Always verify both directions.

```
# from each side
ping -c 2 <other>
traceroute -n <other>
```

Lab: competing prefixes

```
name: l3-lpm

topology:
  nodes:
    h1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 192.168.0.10/24 dev eth1
        - ip link set eth1 up
        - ip route add default via 192.168.0.1
        - ip route add 10.1.2.0/24 via 192.168.0.2
    r1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 192.168.0.1/24 dev eth1
        - ip link set eth1 up
    r2:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 192.168.0.2/24 dev eth1
        - ip link set eth1 up

  links:
    - endpoints: ["h1:eth1", "r1:eth1"]
    # NOTE: for a real multi-router demo, use a bridge multipoint or
    # connect h1-r1 and h1-r2 carefully; simplest teaching variant below.
```

Cleaner multipoint teaching lab: one LAN bridge with h1, gw1, gw2:

```
name: l3-lpm

topology:
  nodes:
    br0:
      kind: linux
      image: alpine:3.20
      binds: ["./config/br.sh:/startup.sh"]
      cmd: /bin/sh /startup.sh
```

```
    h1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 192.168.0.10/24 dev eth1 && ip link set eth1 up
        - ip route add default via 192.168.0.1
        - ip route add 10.1.2.0/24 via 192.168.0.2
    gw1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 192.168.0.1/24 dev eth1 && ip link set eth1 up
    gw2:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 192.168.0.2/24 dev eth1 && ip link set eth1 up

  links:
    - endpoints: ["h1:eth1", "br0:eth1"]
    - endpoints: ["gw1:eth1", "br0:eth2"]
    - endpoints: ["gw2:eth1", "br0:eth3"]
```

Predict / observe

```
docker exec clab-l3-lpm-h1 ip route get 10.1.2.5
# expect via 192.168.0.2
docker exec clab-l3-lpm-h1 ip route get 10.9.0.1
# expect via 192.168.0.1 (default) if no better route
```

Lab: router with two interfaces

```
# r1
sysctl -w net.ipv4.ip_forward=1
ip addr add 10.1.0.1/24 dev eth1
ip addr add 10.2.0.1/24 dev eth2
# hosts with defaults via r1
```

Predict: Connected routes cover both LANs; hosts need default only; r1 needs no static for inter-LAN.

Observe:

```
ip route
ip route get 10.2.0.10 from 10.1.0.10 iif eth1
```

ARP/ND deep enough for routing

IPv4	IPv6
ARP request/reply	NS/NA
`ip neigh`	`ip -6 neigh`
Broadcast	Multicast solicited-node

```
ip monitor neigh
```

Flush for drills:

```
ip neigh flush dev eth1
```

TTL and traceroute

Each hop decrements TTL. Traceroute infers path from ICMP time exceeded.

```
traceroute -n 10.2.0.10
# or
mtr -rwzc 20 10.2.0.10
```

Filters that drop ICMP break traceroute but not always data—note for ops.

Policy preview

Policy routing (`ip rule`) can select alternate tables. Out of scope for depth here; know it exists when "ip route get" surprises you with marks/fwmasks.

```
ip rule list
```

Predict → observe → fix drills

Drill: missing return route

Two routers between hosts; only one direction has statics.

Symptom: requests arrive, replies die.

Fix: install return route.

Harden: verify.sh pings both ways.

Drill: wrong gateway MAC path

Shut gateway interface; host neigh becomes FAILED.

```
ip neigh show
ping -c 2 <offlink>
```

Drill: more-specific shadow

Add junk more-specific blackhole:

```
ip route add 10.2.0.10/32 via 192.0.2.254
# 192.0.2.254 dead
ping 10.2.0.10  # fail despite broader working route
ip route del 10.2.0.10/32
```

Cheatsheet

```
ip route
ip route get 1.2.3.4
ip route add 10.0.0.0/8 via 192.168.0.1 metric 100
ip route del 10.0.0.0/8 via 192.168.0.1
ip neigh show
sysctl net.ipv4.ip_forward
```

Summary

- Longest prefix match drives selection
- Connected, static, and dynamic routes feed RIB→FIB

- Next hop must resolve at L2
- Hosts mostly default; routers interconnect prefixes
- Bidirectional reachability is the real goal

Next: **static routing labs**—triangles, floating statics, and deliberate failover.

Static Routing Labs

Static Routing Labs

Static routes are the control plane you can see with your eyeballs. Master them in Containerlab with FRR or Linux before OSPF/BGP hide the decisions inside protocols.

Learning goals

By the end of this chapter you can:

- Build a multi-router static topology as code
- Install and verify routes in Linux and FRR
- Implement floating static failover with metrics
- Break and restore return paths deliberately
- Write verify scripts that catch one-way routing

Topology: triangle with hosts

```
h1--r1-------r2--h2
     \       /
      \     /
        r3
```

Link	Subnet
r1–r2	10.0.12.0/24
r2–r3	10.0.23.0/24
r3–r1	10.0.13.0/24
h1 LAN	192.168.1.0/24 (r1 gateway)
h2 LAN	192.168.2.0/24 (r2 gateway)
r3 LAN optional	192.168.3.0/24

Loopbacks: r1 `1.1.1.1/32`, r2 `2.2.2.2/32`, r3 `3.3.3.3/32`.

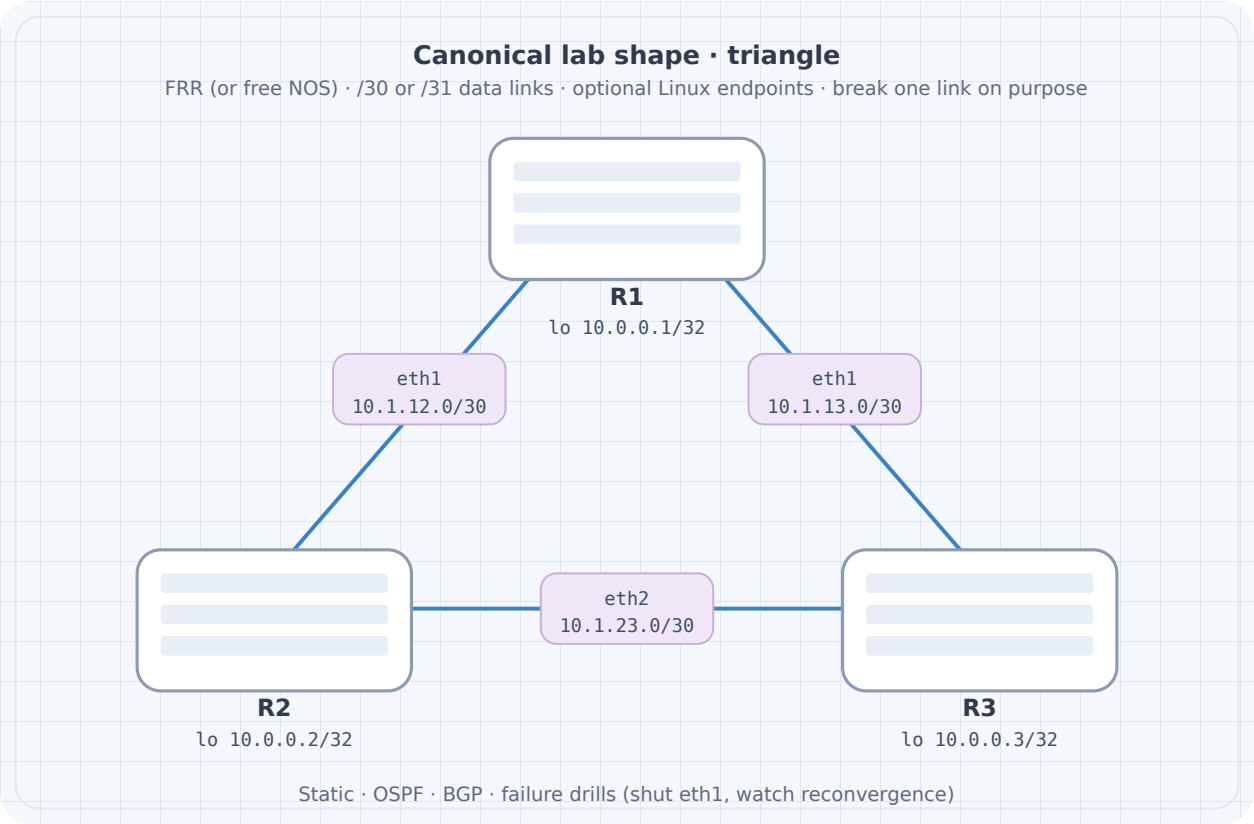


Figure 1: Triangle

Containerlab topology

```
name: static-tri

topology:
  nodes:
    r1:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/r1:/etc/frr
    r2:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/r2:/etc/frr
    r3:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/r3:/etc/frr
    h1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 192.168.1.10/24 dev eth1
        - ip link set eth1 up
        - ip route add default via 192.168.1.1
    h2:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 192.168.2.10/24 dev eth1
        - ip link set eth1 up
        - ip route add default via 192.168.2.1

  links:
    - endpoints: ["h1:eth1", "r1:eth3"]
    - endpoints: ["h2:eth1", "r2:eth3"]
    - endpoints: ["r1:eth1", "r2:eth1"]
    - endpoints: ["r2:eth2", "r3:eth1"]
    - endpoints: ["r3:eth2", "r1:eth2"]
```

FRR config sketches

Enable `zebra` in `daemons`. Example `frr.conf` for **r1**:

```
frr version 10.2.1
frr defaults traditional
hostname r1
!
interface lo
 ip address 1.1.1.1/32
!
interface eth1
 ip address 10.0.12.1/24
!
interface eth2
 ip address 10.0.13.1/24
!
interface eth3
 ip address 192.168.1.1/24
!
ip route 192.168.2.0/24 10.0.12.2
ip route 2.2.2.2/32 10.0.12.2
ip route 3.3.3.3/32 10.0.13.2
!
line vty
```

r2 mirrors with LAN `192.168.2.1/24`, route to `192.168.1.0/24` via `10.0.12.1`, etc.

r3 needs routes to both LANs if used as alternate path:

```
ip route 192.168.1.0/24 10.0.13.1
ip route 192.168.2.0/24 10.0.23.1
```

Adjust interface IPs for r3 on `10.0.13.2` and `10.0.23.2`.

> Exact FRR container startup (vtysh config load) depends on image—follow image docs for `/etc/frr` layout (`frr.conf`, `daemons`, permissions).

Linux-only alternative

If FRR image friction appears, pure Alpine routers work:

```
sysctl -w net.ipv4.ip_forward=1
ip addr add ...
ip route add 192.168.2.0/24 via 10.0.12.2
```

Deploy and baseline

```
sudo containerlab deploy -t static-tri.clab.yml
docker exec clab-static-tri-r1 vtysh -c 'show ip route'
docker exec clab-static-tri-h1 ping -c 3 192.168.2.10
docker exec clab-static-tri-h1 traceroute -n 192.168.2.10
```

Predict

- Path h1→h2 is h1–r1–r2–h2 (direct static)
- Loopback 2.2.2.2 reachable from r1

Observe

```
docker exec clab-static-tri-r1 ip route get 192.168.2.10
docker exec clab-static-tri-r2 ip route get 192.168.1.10
```

Floating statics

On r1, prefer r2 path, backup via r3:

```
ip route 192.168.2.0/24 10.0.12.2
ip route 192.168.2.0/24 10.0.13.2 200
```

In Linux:

```
ip route add 192.168.2.0/24 via 10.0.12.2 metric 10
ip route add 192.168.2.0/24 via 10.0.13.2 metric 200
```

Caveat: Floating statics failover when the **route is withdrawn**—often when the interface or next hop tracking says down. A silent next hop that stays "up" but blackholes may **not** fail over without tracking/BFD (advanced). In labs, shut the interface to force failover.

Failover drill

```
docker exec clab-static-tri-h1 ping -c 100 192.168.2.10 &
docker exec clab-static-tri-r1 ip link set eth1 down
# observe loss then recovery via r3 if floating installed end-to-end
docker exec clab-static-tri-r1 ip route get 192.168.2.10
docker exec clab-static-tri-r1 ip link set eth1 up
```

Ensure r3 and r2 have consistent return paths for the backup trajectory.

Return path drill (mandatory)

Inject on r2:

```
# remove route to h1 LAN
vtysh -c 'configure terminal' -c 'no ip route 192.168.1.0/24 10.0.12.1' -c 'end'
# or linux: ip route del 192.168.1.0/24
```

Predict: One-way behavior; h1→h2 may fail depending on path; debugging must check reverse.

Restore from config files + reload, not memory folklore.

Blackhole and reject

```
ip route add blackhole 10.66.0.0/16
# or
ip route add unreachable 10.66.0.0/16
```

Useful when summarizing: explicit sink for unallocated space you advertise.

verify.sh

```
#!/usr/bin/env bash
set -euo pipefail
fail() { echo "FAIL: $*" >&2; exit 1; }

docker exec clab-static-tri-h1 ping -c 2 -W 1 192.168.2.10 || fail "h1->h2"
docker exec clab-static-tri-h2 ping -c 2 -W 1 192.168.1.10 || fail "h2->h1"
docker exec clab-static-tri-r1 vtysh -c 'show ip route' | grep -q 192.168.2.0 || fail "r1 ro

echo OK
```

Addressing.md excerpt

```
| Node | Iface | IP |
|------|-------|-----|
| r1 | eth1 | 10.0.12.1/24 |
| r2 | eth1 | 10.0.12.2/24 |
| r1 | eth2 | 10.0.13.1/24 |
```

```
| r3 | eth2 | 10.0.13.2/24 |
| r2 | eth2 | 10.0.23.1/24 |
| r3 | eth1 | 10.0.23.2/24 |
| r1 | eth3 | 192.168.1.1/24 |
| r2 | eth3 | 192.168.2.1/24 |
| h1 | eth1 | 192.168.1.10/24 |
| h2 | eth1 | 192.168.2.10/24 |
```

Default routes at edge

Simulating "internet":

```
# on r1
ip route 0.0.0.0/0 10.0.12.2
```

Only if r2 has a path onward. Avoid default loops (r1 defaults to r2, r2 defaults to r1).

IPv6 statics (optional parallel)

```
ipv6 route 2001:db8:2::/64 2001:db8:12::2
```

```
ip -6 route add 2001:db8:2::/64 via 2001:db8:12::2
```

Dual-stack the triangle when comfortable.

Common mistakes

Mistake	Symptom
Missing return route	One-way ping
Wrong next hop IP	Route present, unresolved neigh
Floating without interface down	No failover on silent blackhole
Overlapping LAN IPs	Chaos
Forgetting ip_forward	Router becomes host

Summary

- Static labs teach bidirectional design and verification

- Triangle + floating statics build failover intuition
- Interface shutdown is the honest failover inject
- FRR and Linux both work; keep configs in git
- verify.sh must test **both** directions

Next: **ICMP and path MTU**—when small pings lie and large packets die.

ICMP & Path MTU

ICMP & Path MTU

Small pings succeed while large transfers hang: classic **MTU / PMTUD** failure. This chapter treats ICMP as a control helper and shows you how to find black holes with deliberate packet sizes.

Learning goals

By the end of this chapter you can:

- Explain key ICMP types used in operations
- Test reachability with controlled sizes and DF bit
- Predict PMTUD behavior when ICMP "fragmentation needed" is filtered
- Clamp MTU on a lab link and measure the symptom
- Document a harden checklist for MTU in topologies

ICMP essentials (IPv4)

Type	Role
Echo request/reply	ping
Destination unreachable	no route, port unreachable, **frag needed**
Time exceeded	TTL expired (traceroute)
Redirect	host guidance (often ignored/noisy)

```
ping -c 3 10.0.0.1
ping -c 3 -s 1400 10.0.0.1
ping -M do -s 1472 10.0.0.1   # DF set; 1472 + 28 = 1500
```

`-M do` (Linux) sets Don't Fragment. Adjust payload so total IP packet hits MTU boundary.

IPv6 uses ICMPv6 extensively (ND, PMTUD); similar lab methods with `ping -6`.

Path MTU discovery (idea)

1. Sender assumes path MTU (often interface MTU)
2. Sends DF packets
3. Hop with smaller MTU sends ICMP frag needed + next-hop MTU
4. Sender caches smaller PMTU

If step 3 is **filtered**, large packets blackhole; small pings work.

Lab: MTU mismatch

```
h1 -- r1 ======== r2 -- h2
         MTU 1400 on r1-r2 link
```

Topology

```
name: mtu-lab

topology:
  nodes:
    r1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils iptables
        - sysctl -w net.ipv4.ip_forward=1
        - ip addr add 192.168.1.1/24 dev eth1
        - ip addr add 10.0.0.1/24 dev eth2
        - ip link set eth1 up
        - ip link set eth2 up
        - ip link set eth2 mtu 1400
        - ip route add 192.168.2.0/24 via 10.0.0.2
    r2:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils iptables
        - sysctl -w net.ipv4.ip_forward=1
        - ip addr add 192.168.2.1/24 dev eth1
        - ip addr add 10.0.0.2/24 dev eth2
        - ip link set eth1 up
```

```
        - ip link set eth2 up
        - ip link set eth2 mtu 1400
        - ip route add 192.168.1.0/24 via 10.0.0.1
    h1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 192.168.1.10/24 dev eth1
        - ip link set eth1 up
        - ip route add default via 192.168.1.1
    h2:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 192.168.2.10/24 dev eth1
        - ip link set eth1 up
        - ip route add default via 192.168.2.1

  links:
    - endpoints: ["h1:eth1", "r1:eth1"]
    - endpoints: ["h2:eth1", "r2:eth1"]
    - endpoints: ["r1:eth2", "r2:eth2"]
```

Predict

- `ping -c 2 192.168.2.10` small default works
- Large DF ping across path may fail or trigger PMTUD depending on sizes
- Capture may show ICMP unreachable frag needed if not filtered

Observe

```
docker exec clab-mtu-lab-h1 ping -c 2 192.168.2.10
docker exec clab-mtu-lab-h1 ping -M do -s 1472 -c 2 192.168.2.10
docker exec clab-mtu-lab-h1 ping -M do -s 1200 -c 2 192.168.2.10
docker exec clab-mtu-lab-r1 ip link show eth2
```

Inject: drop ICMP unreachable on r1

```
docker exec clab-mtu-lab-r1 iptables -A FORWARD -p icmp --icmp-type fragmentation-needed -j
# also output path variants may need INPUT/OUTPUT depending on generation point
docker exec clab-mtu-lab-r1 iptables -A OUTPUT -p icmp --icmp-type fragmentation-needed -j D
```

Retest large DF pings / large TCP if iperf available.

Predict: hard blackhole for large DF packets.

Restore

```
docker exec clab-mtu-lab-r1 iptables -F
```

Harden

- Match MTUs both ends of a link
- Do not filter all ICMP on transit routers
- For TCP, MSS clamping on edges sometimes used as workaround—prefer fixing MTU/ICMP

```
# example TCP MSS clamp (awareness; use carefully)
iptables -A FORWARD -p tcp --tcp-flags SYN,RST SYN -j TCPMSS --set-mss 1360
```

Traceroute and ICMP filters

```
traceroute -n 192.168.2.10
```

If time-exceeded filtered, traceroute incomplete while data works. Journal both ICMP classes separately: **PMTUD** vs **traceroute**.

Capture workflow

```
docker exec clab-mtu-lab-r1 tcpdump -ni eth2 -c 40 icmp or '(ip[6:2] & 0x1fff) != 0'
docker exec clab-mtu-lab-h1 ping -M do -s 1472 -c 3 192.168.2.10
```

Look for:

- Echo requests size

- ICMP type 3 code 4 (frag needed)
- Whether sender reduces size after

iperf3 large transfers

```
# on h2
apk add iperf3
iperf3 -s
# on h1
iperf3 -c 192.168.2.10
```

Compare before/after MTU clamp and ICMP drop. TCP may stall if PMTUD blackhole.

IPv6 notes

IPv6 routers do not fragment in the same way; PMTUD is even more critical. Minimum link MTU is 1280.

```
ping -6 -M do -s 1400 <dst>
ip -6 route get <dst>
```

Operational checklist

Check	Command / action
Interface MTU	`ip link`
End-to-end small ping	`ping -c 3`
DF large ping	`ping -M do -s ...`
ICMP allowed	policy review
TCP test	iperf3 / curl large
Tunnel overhead	account for encapsulation

Tunnels (later chapters) reduce effective MTU—plan overhead.

Predict worksheet

Path MTU 1500 except one hop 1400; ICMP filtered.

1. Ping default size 56 payload — ?

2. DF packet size 1500 — ?

3. TCP bulk transfer — ?

Answers: (1) usually ok; (2) blackhole; (3) often hang or extreme slowness depending on stack workarounds.

verify.sh additions

```
#!/usr/bin/env bash
set -euo pipefail
docker exec clab-mtu-lab-h1 ping -c 2 -W 1 192.168.2.10
# expect success for moderate DF size under 1400 path
docker exec clab-mtu-lab-h1 ping -M do -s 1200 -c 2 -W 1 192.168.2.10
echo OK
```

Optional: assert large 1472 DF fails when MTU 1400—can be a negative test in failure mode only.

Summary

- ICMP is part of healthy networks (PMTUD, traceroute), not pure “attack surface” to zero
- Size-controlled pings expose MTU issues small pings hide
- Filtering frag-needed creates blackholes
- Match link MTUs; account for encapsulation overhead
- Always pair ping with a bulk TCP test when performance is in question

Next part: **interior routing**—OSPF and BGP basics plus policy/filters on free FRR images.

BGP no_export and Well-Known Communities

BGP no_export and Well-Known Communities

Thesis

Well-known BGP communities are a portable policy language. `no_export` (65535:65281) means "do not announce this route to eBGP peers"; `no_advertise` keeps it off all peers; `no_export_subconfed` stops it at confederation boundaries. Mis-applying them is a classic route-leak footgun.

Mental model

```
route with community no_export

       iBGP: still propagated inside the AS (usual)
       eBGP: stripped from announcements to external peers
```

Vendor knobs differ: some require `send-community` on the neighbor before communities travel at all.

Worked example

Conceptual FRR / policy sketch (lab with netlab/containerlab — not executed on this box; no sudo/lab installs):

```
! tag a prefix so upstreams do not re-advertise it onward
ip prefix-list LOCAL-ONLY permit 192.0.2.0/24
route-map TAG-NO-EXPORT permit 10
 match ip address prefix-list LOCAL-ONLY
 set community no-export
!
router bgp 65001
 neighbor 203.0.113.1 remote-as 65002
 address-family ipv4 unicast
  neighbor 203.0.113.1 route-map TAG-NO-EXPORT out
  neighbor 203.0.113.1 send-community
```

Verification on a lab router after the session is up:

```
show bgp ipv4 uni 192.0.2.0/24
# expect Community: no-export
# on the eBGP peer: prefix absent (or present only if they ignore well-known - treat as bug)
```

Trap

Assuming communities are sent by default. Many implementations require an explicit `send-community [both]`. Also: `no_export` does **not** replace `NO_PEER` / RFC 7999 peer communities — pick the community that matches the business intent.

Boring rule

Document each tagged prefix: who may see it, which community encodes that, and whether `send-community` is on. Test with an eBGP session in the lab before production.

Try this

1. In netlab/containerlab, announce a prefix with and without `no_export` and watch the second-hop AS.
2. Compare `no_export` vs `no_advertise` on an iBGP mesh.
3. Read IANA well-known community registry values.

Sources

- RFC 1997 (BGP Communities Attribute)
- RFC 3765 (`NOPEER`) / RFC 7999 (BLACKHOLE) as related well-knowns
- FRR docs: BGP communities / route maps

RPKI ROAs and the maxLength Trap

RPKI ROAs and the maxLength Trap

Thesis

A **ROA** (Route Origin Authorization) signs that an ASN may originate a prefix. The optional `maxLength` widens which more-specifics are also valid. Broad `maxLength` (for example `/8` ROA with `maxLength=24`) makes hijacks of punched-out space look RPKI-valid — RFC 9319 warns against that.

Mental model

```
ROA: ASN 64500 may originate 192.0.2.0/24, maxLength=24
     → /24 only (tight)

ROA: ASN 64500 may originate 192.0.2.0/24, maxLength=32
     → any more-specific through /32 also "valid" under that ASN
```

Validators classify announcements as **Valid / Invalid / NotFound**. Invalid should be dropped (Route Origin Validation).

Worked example

Operational checklist (docs-based; no RPKI validator installed here):

```
1. Inventory prefixes you originate.
2. For each, publish a ROA with ASN = your origin AS.
3. Set maxLength to the longest prefix you actually announce -
   not "32 just in case".
4. After every IPv4 transfer, re-check ROAs (stale ROAs → NotFound/Invalid pain).
```

Example of a tight vs loose posture:

Prefix	ASN	maxLength	Comment
192.0.2.0/24	64500	24	Only the /24
198.51.100.0/22	64500	24	Allows /23–/24 you plan to announce
203.0.113.0/8	64500	24	**Avoid** — huge attack surface

Trap

Copy-pasting `maxLength=24` on large aggregates "for flexibility". Attackers who can somehow announce more-specifics under your ASN (or who compromise your CA workflow) get free cover. Prefer multiple tight ROAs over one loose one.

Boring rule

ROA maxLength = maximum length you **intentionally** announce today. Re-issue when that policy changes. Monitor Valid/Invalid/NotFound with a relying party (routinator, rpki-client, vendor RPKI RTR).

Try this

1. Look up your ASN's ROAs on a public RPKI viewer.
2. Read RFC 9319 §3 on maxLength.
3. In a lab, announce a /25 covered by a /24 ROA with maxLength=24 and watch it become Invalid.

Sources

- RFC 6482 (ROA format)
- RFC 6811 (BGP Prefix Origin Validation)
- RFC 9319 (RPKI best practices — maxLength guidance)
- NRO / RIR RPKI documentation

Interior Routing

OSPF Basics

OSPF Basics

OSPF is a **link-state IGP**: routers flood topology information within an area, run SPF, and install routes. This chapter builds a single-area triangle on FRR in Containerlab—the highest-ROI dynamic routing lab in the book.

Learning goals

By the end of this chapter you can:

- Contrast link-state vs distance-vector at a model level
- Bring up OSPF adjacencies on FRR and read neighbor states
- Advertise loopbacks and LANs into OSPF
- Predict reconvergence when a link costs out or dies
- Verify RIB/FIB and traceroute paths

Concepts

Distance-vector vs link-state

	Distance-vector (idea)	Link-state (OSPF)
What is shared	Vectors to prefixes	Topology (LSAs)
Loop risk	Needs careful rules	SPF on common map
Visibility	Partial	Area-wide topology view
Example	Classic RIP	OSPF, IS-IS

OSPF adjacency state machine (short)

Down → Init → 2-Way → ExStart → Exchange → Loading → **Full**

For learning: **Full** with expected neighbors on each transit link. Stuck states often mean MTU mismatch, authentication, layer-2, or network type mismatch.

Areas

Single-area (0) is enough here. Multi-area adds ABR summarization later in your journey; do not rush.

Cost

OSPF picks lowest cost path. Cost often derives from reference bandwidth / interface bandwidth. In FRR you can set `ip ospf cost` on interfaces for drills.

Network types (awareness)

Point-to-point vs broadcast affects adjacency count and DR/BDR. P2P links in labs are simple; Ethernet multipoint may elect DR.

Addressing plan

Node	lo	eth1 (to peer)	eth2	LAN eth3
r1	1.1.1.1/32	10.0.12.1/24 (r2)	10.0.13.1/24 (r3)	192.168.1.1/24
r2	2.2.2.2/32	10.0.12.2/24	10.0.23.1/24	192.168.2.1/24
r3	3.3.3.3/32	10.0.23.2/24	10.0.13.2/24	optional

Hosts: `192.168.1.10`, `192.168.2.10` with defaults to local r.

Method A: The Fast Netlab Workflow (Recommended)

Using Netlab (with `provider: clab`), you declare only the protocol intent and node interconnections. Netlab automatically plans IP subnets, allocates loopbacks, writes the FRR configurations, and stands up the Containerlab environment:

```
# topology.yml
---
provider: clab
defaults.device: frr
module: [ ospf ]

nodes:
  r1:
  r2:
```

```
  r3:
  h1:
    device: linux
  h2:
    device: linux

links:
  - r1:
    r2:
  - r2:
    r3:
  - r3:
    r1:
  - h1:
    r1:
  - h2:
    r2:

validate:
  ospf_neighbors:
    description: "Verify all routers reach Full OSPF adjacency with 2 neighbors"
    nodes: [ r1, r2, r3 ]
    plugin: ospf_neighbor(count=2, state='Full')
  end_to_end_ping:
    description: "Verify end-to-end connectivity between endpoints h1 and h2"
    nodes: [ h1 ]
    exec:
      ping_h2: ping -c 3 10.0.0.5
```

Deploy and Validate with Netlab

```
# Compile configurations and bring up the Containerlab environment
netlab up

# Run automated OSPF adjacency and ping assertions
netlab validate

# Connect to any router CLI directly without memorizing container IDs
netlab connect r1
```

Method B: Raw Containerlab & Manual FRR Configurations

If you are inspecting low-level container plumbing or deploying on a system without Python/Netlab, you can author the Containerlab topology and FRR configurations by hand.

Topology YAML (`ospf-tri.clab.yml`)

```yaml
name: ospf-tri

topology:
  nodes:
    r1:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/r1:/etc/frr
    r2:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/r2:/etc/frr
    r3:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/r3:/etc/frr
    h1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 192.168.1.10/24 dev eth1 && ip link set eth1 up
        - ip route add default via 192.168.1.1
    h2:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 192.168.2.10/24 dev eth1 && ip link set eth1 up
        - ip route add default via 192.168.2.1

  links:
    - endpoints: ["r1:eth1", "r2:eth1"]
    - endpoints: ["r2:eth2", "r3:eth1"]
    - endpoints: ["r3:eth2", "r1:eth2"]
    - endpoints: ["h1:eth1", "r1:eth3"]
```

```
    - endpoints: ["h2:eth1", "r2:eth3"]
```

FRR Daemons File (`config/r1/daemons`)

`daemons` file must enable:

```
zebra=yes
ospfd=yes
```

FRR OSPF Config (`config/r1/frr.conf`)

```
frr version 10.2.1
frr defaults traditional
hostname r1
!
interface lo
 ip address 1.1.1.1/32
 ip ospf area 0
!
interface eth1
 ip address 10.0.12.1/24
 ip ospf area 0
!
interface eth2
 ip address 10.0.13.1/24
 ip ospf area 0
!
interface eth3
 ip address 192.168.1.1/24
 ip ospf area 0
!
router ospf
 ospf router-id 1.1.1.1
!
line vty
```

Advertise by putting interfaces in area 0 (common FRR style). Equivalent network statements exist; be consistent across nodes.

r2/r3: unique router-ids `2.2.2.2`, `3.3.3.3`; matching interface IPs; LANs in OSPF so remote LANs appear in RIB.

Deploy and Verify Manually

```
sudo containerlab deploy -t ospf-tri.clab.yml

docker exec clab-ospf-tri-r1 vtysh -c 'show ip ospf neighbor'
docker exec clab-ospf-tri-r1 vtysh -c 'show ip ospf route'
docker exec clab-ospf-tri-r1 vtysh -c 'show ip route'
docker exec clab-ospf-tri-r1 vtysh -c 'show ip ospf database'

docker exec clab-ospf-tri-h1 ping -c 3 192.168.2.10
docker exec clab-ospf-tri-h1 traceroute -n 192.168.2.10
```

Predict

- Each router: 2 neighbors Full (triangle)
- RIB contains remote LAN and loopbacks via OSPF
- Path prefers lower cost sides

Neighbor not Full?

Check:

```
ip link
ip addr
# MTU match
vtysh -c 'show ip ospf interface'
# timers, area mismatch, passive interfaces accidental
```

Cost change drill

```
# on r1 eth1 increase cost so traffic prefers other way
docker exec -it clab-ospf-tri-r1 vtysh
configure terminal
interface eth1
 ip ospf cost 1000
end
write
# promote back to frr.conf in git after experiment
```

```
docker exec clab-ospf-tri-r1 vtysh -c 'show ip route 192.168.2.0/24'
docker exec clab-ospf-tri-h1 traceroute -n 192.168.2.10
```

Predict: path shifts if alternate exists with lower total cost.

Link failure drill

```
docker exec clab-ospf-tri-r1 ip link set eth1 down
sleep 5
docker exec clab-ospf-tri-r1 vtysh -c 'show ip ospf neighbor'
docker exec clab-ospf-tri-r1 vtysh -c 'show ip route 192.168.2.0'
docker exec clab-ospf-tri-h1 ping -c 20 192.168.2.10
docker exec clab-ospf-tri-r1 ip link set eth1 up
```

Journal reconvergence behavior and whether ping loss matches expectations.

Passive interfaces / LAN notes

Sometimes you put a LAN in OSPF for advertisement but want **no** adjacency on user ports:

```
router ospf
 passive-interface eth3
```

Loopbacks are often passive. Transit links remain non-passive.

Verification checklist

Check	Command
Neighbors	`show ip ospf neighbor`
Database	`show ip ospf database`
OSPF RIB	`show ip ospf route`
Kernel/FIB	`show ip route` / `ip route`
Data plane	ping / traceroute

verify.sh

```
#!/usr/bin/env bash
set -euo pipefail
fail() { echo "FAIL: $*" >&2; exit 1; }
```

```
docker exec clab-ospf-tri-r1 vtysh -c 'show ip ospf neighbor' | grep -q Full \
  || fail "no Full neighbor on r1"
docker exec clab-ospf-tri-h1 ping -c 2 -W 1 192.168.2.10 || fail "h1->h2"
docker exec clab-ospf-tri-h2 ping -c 2 -W 1 192.168.1.10 || fail "h2->h1"
echo OK
```

Common mistakes

Mistake	Symptom
ospfd not enabled	no neighbors
Mismatched areas	stuck
Different IP subnets on link	no adjacency
MTU mismatch	ExStart/Exchange issues
router-id collisions	confusing DB
Forgetting LAN in OSPF	hosts missing remote routes

Beyond this chapter

- Multi-area + summarization at ABR
- Stub/NSSA ideas
- OSPFv3 for IPv6
- Authentication

Master single-area Full adjacencies and failover first.

Summary

- OSPF floods link state, computes SPF, installs routes
- FRR triangle lab is the default IGP workout
- Always verify neighbors Full **and** FIB **and** ping
- Cost and link-down drills teach reconvergence
- Keep configs in binds; promote experimental vtysh changes to git

Next: **BGP basics**—sessions, path selection process, and a dual-router lab without vendor exam noise.

BGP Basics

BGP Basics

BGP is the inter-domain workhorse and also appears inside modern DC fabrics. This chapter stays vendor-agnostic and free-image-only: **eBGP and iBGP ideas**, sessions, advertisements, and path selection at a practitioner level—implemented with FRR.

Learning goals

By the end of this chapter you can:

- Explain ASNs, eBGP vs iBGP roles
- Bring up a BGP session on FRR to Established
- Advertise a prefix and see it on the peer
- Apply the path selection process at a high level
- Break a session and observe withdrawal

Concepts

Autonomous systems

An **AS** is a policy domain with an ASN. Labs use private ASNs:

Range	Notes
64512–65534	16-bit private
4200000000+	32-bit private space (use docs for exact ranges)

Example: AS65001 and AS65002 as "two sites" or "customer and ISP."

	eBGP	iBGP

eBGP vs iBGP

	eBGP	iBGP
Between	Different ASNs	Same ASN
Typical TTL	1 (direct)	multi-hop ok with care
Default next-hop	Peer address	Needs next-hop-self often for reachability
Scale	Edge policy	Route reflection / confed for large iBGP

Session state

Idle → Connect → Active → OpenSent → OpenConfirm → **Established**

Only Established exchanges routes.

Path selection overview

BGP is policy-heavy. Simplified order of preference ideas:

1. Prefer higher **local preference** (inbound influence)
2. Prefer shorter **AS path**
3. Origin, MED, eBGP over iBGP, IGP metric to next hop, router-id, etc.

You will not memorize every tie-breaker today—**know that policy beats pure shortest path**.

NLRI and attributes

Routes carry attributes (AS_PATH, NEXT_HOP, LOCAL_PREF, MED, communities…). Filters rewrite attributes; that is policy.

Lab A: two AS eBGP

```
h1--r1 (AS65001) -------- r2 (AS65002)--h2
         10.0.0.1/30   10.0.0.2/30
```

Advertise LANs (or loopbacks) across eBGP.

Topology

```
name: bgp-ebgp

topology:
  nodes:
    r1:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/r1:/etc/frr
    r2:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/r2:/etc/frr
    h1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 192.168.1.10/24 dev eth1 && ip link set eth1 up
        - ip route add default via 192.168.1.1
    h2:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 192.168.2.10/24 dev eth1 && ip link set eth1 up
        - ip route add default via 192.168.2.1

  links:
    - endpoints: ["r1:eth1", "r2:eth1"]
    - endpoints: ["h1:eth1", "r1:eth2"]
    - endpoints: ["h2:eth1", "r2:eth2"]
```

daemons

```
zebra=yes
bgpd=yes
```

r1 frr.conf

```
frr version 10.2.1
frr defaults traditional
hostname r1
!
interface eth1
 ip address 10.0.0.1/30
!
interface eth2
 ip address 192.168.1.1/24
!
interface lo
 ip address 1.1.1.1/32
!
router bgp 65001
 bgp router-id 1.1.1.1
 no bgp ebgp-requires-policy
 neighbor 10.0.0.2 remote-as 65002
 !
 address-family ipv4 unicast
  network 192.168.1.0/24
  network 1.1.1.1/32
  neighbor 10.0.0.2 activate
 exit-address-family
!
line vty
```

> `no bgp ebgp-requires-policy` eases labs on modern FRR that otherwise require explicit export policy. In real life, **prefer explicit policies** (next chapter).

r2

ASN 65002, IP `10.0.0.2/30`, LAN `192.168.2.0/24`, networks advertised symmetrically, neighbor `10.0.0.1 remote-as 65001`.

Verify session

```
sudo containerlab deploy -t bgp-ebgp.clab.yml
docker exec clab-bgp-ebgp-r1 vtysh -c 'show bgp summary'
docker exec clab-bgp-ebgp-r1 vtysh -c 'show bgp ipv4 unicast'
docker exec clab-bgp-ebgp-r1 vtysh -c 'show ip route'
docker exec clab-bgp-ebgp-h1 ping -c 3 192.168.2.10
```

Predict

- `show bgp summary` state Established, prefixes received > 0
- Kernel has remote LAN via BGP next hop 10.0.0.2
- Ping works both ways

Lab B: iBGP sketch (optional same day)

Two routers same ASN, loopbacks as session endpoints, IGP or statics for loopback reachability first.

```
router bgp 65001
 neighbor 2.2.2.2 remote-as 65001
 neighbor 2.2.2.2 update-source lo
 neighbor 2.2.2.2 next-hop-self
```

Rule: iBGP needs **underlying reachability** to the session address. OSPF underlay + iBGP overlay is a common pattern (underlay vs overlay planes!).

Session failure drill

```
docker exec clab-bgp-ebgp-r1 ip link set eth1 down
docker exec clab-bgp-ebgp-r1 vtysh -c 'show bgp summary'
docker exec clab-bgp-ebgp-r2 vtysh -c 'show bgp summary'
docker exec clab-bgp-ebgp-r2 vtysh -c 'show ip route 192.168.1.0'
# routes withdraw after hold timers - observe timing
docker exec clab-bgp-ebgp-r1 ip link set eth1 up
```

Predict: session drops; prefixes withdrawn; ping fails until re-established.

Optional: `neighbor x shutdown` in vtysh for cleaner control-plane inject.

Next-hop reachability drill

Install BGP route whose next hop is unreachable (misconfigured).

Symptom: BGP table shows path but not installed in FIB / not usable.

```
vtysh -c 'show bgp ipv4 unicast'
vtysh -c 'show ip route'
```

Compare “known in BGP” vs “installed.”

Dual-home teaser

Customer with two eBGP uplinks steers with local-pref / AS-path prepend—full treatment lives in policy chapter. Topology idea:

```
      isp1
     /
 cust
     \
      isp2
```

verify.sh

```
#!/usr/bin/env bash
set -euo pipefail
fail() { echo "FAIL: $*" >&2; exit 1; }

docker exec clab-bgp-ebgp-r1 vtysh -c 'show bgp summary' | grep -q Established \
  || fail "session not Established"
docker exec clab-bgp-ebgp-h1 ping -c 2 -W 1 192.168.2.10 || fail "ping"
docker exec clab-bgp-ebgp-h2 ping -c 2 -W 1 192.168.1.10 || fail "reverse ping"
echo OK
```

Safety habits

- Do not experiment with BGP on real Internet edges without change control
- Private ASNs in labs only unless you know what you are doing
- Explicit export policy in anything beyond throwaway labs
- Cap lab prefixes; never leak real full tables into tiny FRR VMs by accident

Common mistakes

Mistake	Result
wrong remote-as	session fails
no underlay to iBGP peer	idle/active
missing activate / AF	Established but no routes
ebgp-requires-policy deny	no prefixes without route-maps
next hop unreachable	BGP path not used

Summary

- BGP sessions must be Established before routes flow
- eBGP links ASNs; iBGP needs underlay reachability
- Path selection is policy-centric, not pure SPF
- FRR lab: advertise LANs, ping through, shut session
- Prefer explicit policies as you leave pure lab mode

Next: **policy and filters**—prefix lists, route-maps-as-idea, and preventing loops/leaks.

Policy & Filters

Policy & Filters

Routing without policy is a demo; routing with policy is operations. This chapter covers **prefix filters, distribute ideas, route-maps as a pattern, and loop-prevention instincts** using FRR—still free and vendor-agnostic.

Learning goals

By the end of this chapter you can:

- Write prefix-lists that match intended space only
- Apply import/export filters on BGP neighbors
- Use route-map style logic (match → set → permit/deny)
- Predict leaks and loops from redistribution mistakes
- Build a lab that rejects unauthorized prefixes

Why policy exists

Goal	Mechanism ideas
Do not accept bogons / your own space from customers	Inbound prefix filter
Do not advertise internal-only nets to Internet	Outbound filter
Prefer one uplink	local-pref on import
De-prefer backup path	AS-path prepend on export
Redistribute without feedback loops	Tags, filters, distance

Prefix lists (FRR)

```
ip prefix-list OWN-NETS seq 10 permit 192.168.1.0/24
ip prefix-list OWN-NETS seq 20 permit 1.1.1.1/32
ip prefix-list OWN-NETS seq 30 deny any
```

```
ip prefix-list CUST-ALLOW seq 10 permit 198.51.100.0/24
ip prefix-list CUST-ALLOW seq 20 deny any
```

Order matters: first match wins. Always know your implicit/default at the end.

Route-maps as idea

Regardless of vendor syntax, the pattern is:

```
if match prefix-list X:
  set local-preference 200
  permit
else deny
```

FRR example:

```
route-map FROM-CUST permit 10
 match ip address prefix-list CUST-ALLOW
 set local-preference 200
route-map FROM-CUST deny 99
!
route-map TO-CUST permit 10
 match ip address prefix-list OWN-NETS
```

Attach:

```
router bgp 65002
 neighbor 10.0.0.1 route-map FROM-CUST in
 neighbor 10.0.0.1 route-map TO-CUST out
```

Lab: filter unauthorized prefixes

Extend eBGP lab: r2 (ISP) should **only** accept `192.168.1.0/24` from r1 (customer), not a hijack prefix.

Customer r1 advertises extra junk

```
address-family ipv4 unicast
 network 192.168.1.0/24
 network 203.0.113.0/24
```

ISP r2 policy

```
ip prefix-list CUST-R1 seq 5 permit 192.168.1.0/24
ip prefix-list CUST-R1 seq 10 deny any
!
route-map FROM-R1 permit 10
 match ip address prefix-list CUST-R1
route-map FROM-R1 deny 20
!
router bgp 65002
 neighbor 10.0.0.1 route-map FROM-R1 in
```

Predict

- ISP installs `192.168.1.0/24`
- `203.0.113.0/24` is rejected
- `show bgp` on r2 lacks the junk

Observe

```
docker exec clab-bgp-ebgp-r2 vtysh -c 'show bgp ipv4 unicast'
docker exec clab-bgp-ebgp-r2 vtysh -c 'show ip route 203.0.113.0'
docker exec clab-bgp-ebgp-r1 vtysh -c 'show bgp ipv4 unicast neighbors 10.0.0.2 advertised-r
```

Harden

Return to explicit export on customer too—only advertise owned space.

```
route-map EXPORT-OWN permit 10
 match ip address prefix-list OWN-NETS
!
neighbor 10.0.0.2 route-map EXPORT-OWN out
```

Remove `no bgp ebgp-requires-policy` when policies are in place if you want FRR to enforce the discipline.

Local-pref steer drill

Two ISPs: isp1 and isp2; customer prefers isp1 for inbound to customer… actually:

- **Local pref** (on customer) influences **outbound** exit from customer AS
- **AS prepend / MED / communities** influence how others send **inbound**

Outbound preference (customer)

```
route-map PREFER-ISP1 permit 10
 set local-preference 300
route-map PREFER-ISP2 permit 10
 set local-preference 100
!
neighbor <isp1> route-map PREFER-ISP1 in
neighbor <isp2> route-map PREFER-ISP2 in
```

Predict: traffic leaves via isp1 when both paths exist.

```
vtysh -c 'show ip route 0.0.0.0'
vtysh -c 'show bgp ipv4 unicast'
```

Redistribution loop awareness

Redistributing OSPF BGP (or static OSPF) without filters can:

- Re-inject prefixes with changed metrics
- Create loops or suboptimal paths
- Explode tables

Rules of thumb:

1. Redistribute **on purpose**, on as few points as possible
2. Tag routes; filter on tags
3. Never blindly mutual redistribute two IGPs
4. Prefer BGP for policy domains, IGP for underlay topology

Example static→OSPF with filter (sketch)

```
ip prefix-list STATICS seq 5 permit 192.168.9.0/24
route-map REDIST-STATIC permit 10
 match ip address prefix-list STATICS
!
router ospf
 redistribute static route-map REDIST-STATIC
```

ACL-style packet filters vs routing policy

Do not confuse:

Routing policy	Packet filter
Which **routes** enter RIB	Which **packets** pass
BGP/OSPF route-maps	nftables/iptables/ACLs
Control plane	Data plane

Both appear in hardening. A permit route does not mean packets are allowed through a firewall mid-path.

Distribute-list / filter-list ideas

Older or alternate forms filter by ACL or AS-path lists:

```
bgp as-path access-list 1 permit _65001$
neighbor x filter-list 1 in
```

Same intent: **accept only what policy allows**.

Failure drills

Drill 1 — over-broad permit any

Export `permit any` to a lab "ISP."

Observe: internal lab nets leave the AS.

Fix: prefix-list OWN only.

Harden: default-deny export mindset.

Drill 2 — accidental deny all

Route-map deny first line matches everything.

Symptom: Established but zero prefixes.

Debug: `show bgp neighbors ... routes` / policy counters if available; simplify map.

Drill 3 — next-hop-self missing (iBGP)

Learned eBGP route reflected iBGP with remote next hop unreachable.

Symptom: BGP has path; FIB lacks usable route.

Fix: `next-hop-self` or IGP reachability to next hop.

verify.sh policy checks

```
#!/usr/bin/env bash
set -euo pipefail
fail() { echo "FAIL: $*" >&2; exit 1; }

docker exec clab-bgp-ebgp-r2 vtysh -c 'show bgp ipv4 unicast' | grep -q '192.168.1.0/24' \
  || fail "missing customer prefix"
if docker exec clab-bgp-ebgp-r2 vtysh -c 'show bgp ipv4 unicast' | grep -q '203.0.113.0/24';
  fail "hijack prefix should be filtered"
fi
echo OK
```

Policy design worksheet

Before applying:

```
## Neighbor: r1-r2 eBGP
Import accept: ...
Import deny: ...
Export advertise: ...
Export never: ...
Preference intent: ...
Failure mode if map wrong: ...
```

Summary

- Policy decides which routes and with which attributes
- Prefix-lists + route-maps are the portable pattern
- Default-deny posture on Internet edges
- Redistribution needs filters/tags or it will bite
- Verify with BGP table **and** negative tests for rejected prefixes

Next part: **ops & automation**—observability, lab-as-code automation, and a capstone outline that pulls the book together.

BFD — Fast Failure Detection

BFD — Fast Failure Detection

Routing protocols are good at choosing paths and bad at noticing that a path died. When the failure is **not** a local link-down — a switch in the middle, a provider handoff, a hung line card, a VM whose host froze — BGP waits for its hold timer and OSPF for its dead interval. That is **minutes** of blackholed traffic with every session looking healthy. **Bidirectional Forwarding Detection (BFD)** is the boring fix: a tiny, protocol-independent hello that proves the forwarding path works and tells its clients (BGP, OSPF, static routes) within a second when it does not. This chapter runs the failure first without BFD, then with it, on two FRR routers behind a Linux bridge — and measures both.

Learning goals

By the end of this chapter you can:

- Explain why carrier loss on *one* side does not reach the other side through a switch
- Predict BGP failure detection time from keepalive/hold timers (and why it is a range)
- Describe BFD control packets, the session states, and the detection-time formula
- Enable BFD for an FRR BGP neighbor with a profile and verify the negotiated timers
- Show that the **slower side wins** timer negotiation
- Attach BFD to a static route so it is withdrawn when the next hop dies
- Choose timers that do not turn CPU hiccups into routing flaps

Why carrier is not enough

```
          r1-eth1                        r2-eth1
r1 (AS65001)      sw (bridge)      r2 (AS65002)   h2
10.0.12.1/24        sw-r1      sw-r2       10.0.12.2/24     203.0.113.10/24

                    failure here: r2 sees NO-CARRIER,
                    r1's port to the switch stays UP
```

Direct cable: either end losing light usually drops the link at both ends, and routing reacts to the interface event in milliseconds. Through a switch, a media converter, a provider's L2 service, or a virtual switch, **each side only sees its own port**. r2 notices instantly and tears its session down; r1 still has carrier, still has an Established BGP session, and still forwards toward a next hop that will never answer.

Mechanism	Default (FRR "traditional")	Detects
Local carrier	Immediate	Only *your* link
BGP hold timer	keepalive 60 s, hold 180 s	Peer silent for hold time
OSPF dead interval	hello 10 s, dead 40 s	Neighbor silent for dead interval
BFD (async)	300 ms × 3 = 900 ms	Forwarding path silent for detect time

Tuning BGP to keepalive 3 / hold 9 helps, but it puts liveness inside a TCP session handled by a busy control-plane process, it is per-protocol, and you still get several seconds. BFD exists so that every protocol can share **one** cheap, sub-second liveness check.

Concepts — BFD in one page

What it is. RFC 5880 defines BFD; RFC 5881 maps it onto single-hop IPv4/IPv6 (UDP destination port **3784**, sent with TTL/hop-limit **255** and checked on receipt so only a directly connected peer can inject packets); RFC 5883 covers multihop sessions (UDP **4784**, e.g., between loopbacks); RFC 5882 describes how clients such as BGP and OSPF should react.

Asynchronous mode. Both sides send control packets periodically. Each packet carries the sender's *My Discriminator* and the receiver's *Your Discriminator* (session IDs), the state, a diagnostic code, and three timer values: **Desired Min TX**, **Required Min RX**, and **Detect Mult**.

States. `AdminDown` → `Down` → `Init` → `Up`. A three-way handshake brings a session up; while not Up, packets are sent at most once per second (slow start), so a flapping peer cannot generate a burst.

Timer negotiation — the slower side wins.

```
r1 transmit interval = max(r1 Desired Min TX, r2 Required Min RX)
r1 detection time    = r2 Detect Mult × max(r1 Required Min RX, r2 Desired Min TX)
```

Each side declares what it *wants* to send and what it *can* receive; nobody is forced to receive faster than it said. Transmit times are jittered (75–100% of the interval) so sessions do not synchronize.

Diagnostics. When a session goes down the reason is in the packet and in `show bfd peer`: `control detection time expired` (no packets in time — the classic failure), `neighbor signaled session down`, `administratively down` (the peer disabled it on purpose — RFC 5882 says clients should *not* treat this as a path failure), and others.

Echo mode (optional) sends packets that the peer loops back in its forwarding plane, testing the data path rather than the peer's BFD process. Single-hop only. In FRR it works only when the peer is also FRR, unless you have hardware-offloaded BFD.

Clients. BFD itself changes nothing in the RIB. A client registers interest in a peer; when BFD declares it Down, the client acts — BGP closes the session and withdraws routes, OSPF drops the adjacency, a static route is removed. Several clients share one session to the same peer.

Addressing plan (BFD lab)

Node	Interface	Address	Role
r1	r1-eth1	10.0.12.1/24	AS 65001
r2	r2-eth1	10.0.12.2/24	AS 65002
r2	r2-eth2	203.0.113.1/24	Customer LAN gateway
r2	lo	198.51.100.1/24	Static-route target (Drill 4)
h2	h2-eth0	203.0.113.10/24	Customer host
sw	br0 (sw-r1, sw-r2)	—	L2 switch

Lab build (Linux namespaces + FRR)

This lab needs no container images: four network namespaces, veth pairs, a Linux bridge as the switch, and one FRR instance per router namespace using FRR's **pathspace** option (`-N`), which gives each instance its own sockets and PID files under `/var/run/frr/<name>/` and its own `vtysh -N <name>`. It was run on **Debian 13 with the distro `frr` package (FRR 10.3)**, as root. Outputs below are copied from those runs; ages, IDs, and PIDs will differ on yours.

The same `frr.conf` snippets drop unchanged into the FRR containers used in other labs in this book — add `bfdd=yes` to their `daemons` file.

Install FRR and helpers (Debian/Ubuntu), and make sure the system-wide FRR service is not running (it would use the default pathspace, not these):

```
sudo apt-get install -y frr iputils-ping bc
```

```
#!/usr/bin/env bash
# lab.sh - r1 and r2 (FRR) behind a Linux-bridge "switch", h2 on r2's LAN
# usage: sudo ./lab.sh up | down
set -euo pipefail
FRR=/usr/lib/frr
DAEMONS="mgmtd zebra bgpd bfdd staticd"

up() {
  for ns in r1 r2 sw h2; do ip netns add "$ns"; ip -n "$ns" link set lo up; done
```

```
  # the switch: one bridge, two access ports
  ip -n sw link add br0 type bridge
  ip -n sw link set br0 up
  for r in r1 r2; do
    ip link add "$r-eth1" netns "$r" type veth peer name "sw-$r" netns sw
    ip -n sw link set "sw-$r" master br0 up
    ip -n "$r" link set "$r-eth1" up
  done
  ip -n r1 addr add 10.0.12.1/24 dev r1-eth1
  ip -n r2 addr add 10.0.12.2/24 dev r2-eth1

  # r2's customer LAN with one host
  ip link add r2-eth2 netns r2 type veth peer name h2-eth0 netns h2
  ip -n r2 addr add 203.0.113.1/24 dev r2-eth2
  ip -n h2 addr add 203.0.113.10/24 dev h2-eth0
  ip -n r2 link set r2-eth2 up
  ip -n h2 link set h2-eth0 up
  ip -n h2 route add default via 203.0.113.1
  ip netns exec r2 sysctl -qw net.ipv4.ip_forward=1
  ip netns exec r1 sysctl -qw net.ipv4.ip_forward=1

  for ns in r1 r2; do
    install -d -o frr -g frr "/var/run/frr/$ns" "/etc/frr/$ns"
    : > "/etc/frr/$ns/vtysh.conf"
    for d in $DAEMONS; do
      ip netns exec "$ns" "$FRR/$d" -d -N "$ns" -F traditional \
        -u frr -g frr --limit-fds 1024 >/dev/null 2>&1
    done
    vtysh -N "$ns" -f "$ns.conf" >/dev/null
  done
}

down() {
  for ns in r1 r2; do
    for d in $DAEMONS; do
      pidf="/var/run/frr/$ns/$d.pid"
      [ -f "$pidf" ] && kill "$(cat "$pidf")" 2>/dev/null || true
    done
  done
  for ns in r1 r2 sw h2; do ip netns del "$ns" 2>/dev/null || true; done
}

"$@"
```

Router configs — plain eBGP first, **no BFD**:

```
! r1.conf - AS 65001, plain eBGP (no BFD yet)
hostname r1
router bgp 65001
 bgp router-id 10.0.12.1
 no bgp ebgp-requires-policy
 neighbor 10.0.12.2 remote-as 65002
```

```
! r2.conf - AS 65002, originates the customer LAN
hostname r2
router bgp 65002
 bgp router-id 10.0.12.2
 no bgp ebgp-requires-policy
 neighbor 10.0.12.1 remote-as 65001
 address-family ipv4 unicast
  network 203.0.113.0/24
 exit-address-family
```

The failure drill — it fails r2's switch port, confirms r1's port is still up, and times how long r1 keeps the dead route in its kernel FIB:

```
#!/usr/bin/env bash
# drill.sh - fail r2's switch port, time how long r1 keeps a dead route
# usage: sudo ./drill.sh [prefix]   (default 203.0.113.0/24)
set -euo pipefail
PFX=${1:-203.0.113.0/24}

ip -n r1 route show "$PFX" | grep -q . || { echo "no route to $PFX on r1"; exit 1; }
echo "before: $(ip -n r1 route show "$PFX")"

start=$(date +%s.%N)
ip -n sw link set sw-r2 down            # r2 loses carrier; r1's port stays up
echo "r1-eth1 state after failure: $(ip -n r1 -br link show r1-eth1 | awk '{print $2}')"

while ip -n r1 route show "$PFX" | grep -q .; do sleep 0.05; done
end=$(date +%s.%N)
printf 'route withdrawn on r1 after %.2f s\n' "$(echo "$end - $start" | bc)"

ip -n sw link set sw-r2 up              # heal for the next run
```

Bring it up and check the baseline:

```
chmod +x lab.sh drill.sh
sudo ./lab.sh up
sleep 10
sudo vtysh -N r1 -c 'show ip route bgp'
sudo ip netns exec r1 ping -c2 -W1 203.0.113.10
```

Output (tails):

```
IPv4 unicast VRF default:
B>* 203.0.113.0/24 [20/0] via 10.0.12.2, r1-eth1, weight 1, 00:00:08
2 packets transmitted, 2 received, 0% packet loss, time 1023ms
```

Each daemon prints a `[pid|daemon] done` line as `vtysh -f` applies configuration; that is normal.

Drill 1 — the silent failure (no BFD)

Predict

r1's hold time is 180 s and keepalives arrive every 60 s. The failure happens at a random point in the keepalive cycle. How long will r1 keep the route?

```
sudo vtysh -N r1 -c 'show ip bgp neighbors 10.0.12.2' | grep -i 'hold time'
```

```
  Hold time is 180 seconds, keepalive interval is 60 seconds
  Configured hold time is 180 seconds, keepalive interval is 60 seconds
```

Observe

```
sudo ./drill.sh
```

While it waits, in a second terminal:

```
sudo ip netns exec r1 ping -c3 -W1 203.0.113.10 | tail -2
sudo vtysh -N r1 -c 'show ip bgp summary' | grep -A1 Neighbor
sudo vtysh -N r2 -c 'show ip bgp summary' | grep -A1 Neighbor
```

```
3 packets transmitted, 0 received, 100% packet loss, time 2039ms

Neighbor        V         AS   MsgRcvd   MsgSent   TblVer  InQ OutQ  Up/Down State/PfxRcd
10.0.12.2       4      65002         4         5        1    0    0 00:00:35            1
Neighbor        V         AS   MsgRcvd   MsgSent   TblVer  InQ OutQ  Up/Down State/PfxRcd
10.0.12.1       4      65001         4         4        0    0    0 00:00:13       Active
```

r2 already knows (its port lost carrier). r1 shows **Established with 1 prefix** and drops every packet. The drill finishes:

```
before: 203.0.113.0/24 nhid 6 via 10.0.12.2 dev r1-eth1 proto bgp metric 20
r1-eth1 state after failure: UP
route withdrawn on r1 after 158.87 s
```

Explain

Not 180 s: the hold timer restarts on every keepalive, so detection is anywhere from **hold − keepalive** (120 s) to **hold** (180 s), depending on when the last keepalive landed. Two and a half minutes of blackhole with a green dashboard.

Drill 2 — add BFD and repeat

Define a profile on each router and attach it to the BGP neighbor. The profile asks for 100 ms intervals with multiplier 3 (300 ms detection). FRR's defaults are 300 ms × 3.

```
! r1-bfd.conf - add a BFD profile and attach it to the BGP neighbor
bfd
 profile fast
  detect-multiplier 3
  receive-interval 100
  transmit-interval 100
 exit
exit
router bgp 65001
 neighbor 10.0.12.2 bfd profile fast
```

```
! r2-bfd.conf - add a BFD profile and attach it to the BGP neighbor
bfd
 profile fast
  detect-multiplier 3
  receive-interval 100
  transmit-interval 100
 exit
exit
router bgp 65002
 neighbor 10.0.12.1 bfd profile fast
```

```
sudo vtysh -N r1 -f r1-bfd.conf
sudo vtysh -N r2 -f r2-bfd.conf
sleep 3
sudo vtysh -N r1 -c 'show bfd peers brief'
sudo vtysh -N r1 -c 'show bfd peer 10.0.12.2'
sudo vtysh -N r1 -c 'show ip bgp neighbors 10.0.12.2' | grep -A3 'BFD'
```

Output (session IDs will differ):

```
Session count: 1
SessionId  LocalAddress                             PeerAddress
=========  ============                             ===========
```

```
3526470594 10.0.12.1                                    10.0.12.2
```

```
BFD Peer:
    peer 10.0.12.2 local-address 10.0.12.1 vrf default interface r1-eth1
        ID: 3526470594
        Remote ID: 1869221485
        Active mode
        Status: up
        Uptime: 2 second(s)
        Diagnostics: ok
        Remote diagnostics: ok
        Peer Type: dynamic
        RTT min/avg/max: 0/0/0 usec
        Local timers:
            Detect-multiplier: 3
            Receive interval: 100ms
            Transmission interval: 100ms
            Echo receive interval: 50ms
            Echo transmission interval: disabled
        Remote timers:
            Detect-multiplier: 3
            Receive interval: 100ms
            Transmission interval: 100ms
            Echo receive interval: 50ms
```

```
  BFD: Type: single hop
  Detect Multiplier: 3, Min Rx interval: 300, Min Tx interval: 300
  Status: Up, Last update: 0:00:00:02
```

(That last block was captured right after the first apply, before the 100 ms profile values were in place — BGP's neighbor view shows what the session had at registration. Trust `show bfd peer` for live timers.) `Peer Type: dynamic` means a client (BGP) created the session; you did not declare a `peer` under `bfd`.

Run the drill again, a few times:

```
sudo ./drill.sh
```

```
before: 203.0.113.0/24 nhid 6 via 10.0.12.2 dev r1-eth1 proto bgp metric 20
r1-eth1 state after failure: UP
route withdrawn on r1 after 0.38 s
```

Repeated runs: `0.33 s`, `0.33 s`, `0.33 s` — the 300 ms detection time plus BGP and zebra doing their work. After a failure the session records why:

```
sudo vtysh -N r1 -c 'show bfd peer 10.0.12.2' | grep -E 'Status|Diagnostics|Downtime'
```

```
        Status: down
        Downtime: 0 second(s)
        Diagnostics: control detection time expired
```

From 158.87 s to 0.33 s, with the same BGP timers.

Drill 3 — the slower side wins

Put r2 back to 300 ms while r1 stays at 100 ms:

```
sed 's/interval 100/interval 300/g' r2-bfd.conf > r2-slow.conf
sudo vtysh -N r2 -f r2-slow.conf
sleep 3
sudo vtysh -N r1 -c 'show bfd peer 10.0.12.2 json' | python3 -m json.tool | grep -E 'interva
```

```
    "receive-interval": 100,
    "transmit-interval": 100,
    "echo-receive-interval": 50,
    "echo-transmit-interval": 0,
    "detect-multiplier": 3,
    "remote-receive-interval": 300,
    "remote-transmit-interval": 300,
    "remote-echo-receive-interval": 50,
    "remote-detect-multiplier": 3,
```

Predict

r1 detection time = r2 Detect Mult × max(r1 Required Min RX, r2 Desired Min TX) = 3 × max(100, 300) = **900 ms**.

Observe

```
sudo ./drill.sh | tail -1
```

```
route withdrawn on r1 after 0.86 s
```

Configuring one end aggressively buys nothing; detection follows the slower declaration. Restore r2 with `sudo vtysh -N r2 -f r2-bfd.conf`.

Drill 4 — BFD for a static route

Static routes are the other place silent failures hurt: a static next hop "stays up" as long as the interface does. FRR's `staticd` can make a static route a BFD client — installed only while the session is Up.

```
sudo ip -n r2 addr add 198.51.100.1/24 dev lo
```

```
! r1-static.conf - a static route that is only installed while BFD says the next hop is aliv
ip route 198.51.100.0/24 10.0.12.2 bfd profile fast
```

```
! r2-static.conf - answer BFD for r1 even without a routing protocol
bfd
 peer 10.0.12.1 interface r2-eth1
  profile fast
 exit
exit
```

r2 has no routing protocol for this prefix, so it gets an explicit `peer` under `bfd` to answer the session (in this lab BGP's session to the same address already exists and is shared — one session, two clients).

```
sudo vtysh -N r1 -f r1-static.conf
sudo vtysh -N r2 -f r2-static.conf
sleep 3
sudo vtysh -N r1 -c 'show ip route 198.51.100.0/24'
sudo ./drill.sh 198.51.100.0/24
```

```
Routing entry for 198.51.100.0/24
  Known via "static", distance 1, metric 0, best
  Last update 00:00:03 ago
  * 10.0.12.2, via r1-eth1, weight 1
```

```
before: 198.51.100.0/24 nhid 6 via 10.0.12.2 dev r1-eth1 proto static metric 20
r1-eth1 state after failure: UP
route withdrawn on r1 after 0.27 s
```

A few seconds after the drill heals the port, both the static and the BGP route are back. While a monitored static is uninstalled it disappears from `show ip route`; use `show bfd static route` to see it. This is what makes a floating static (a backup with a higher distance) actually float when the primary next hop dies behind a switch.

Drill 5 — one-sided BFD (predict, then try)

Tear the lab down and up again, and apply `r1-bfd.conf` **only on r1**.

```
sudo ./lab.sh down && sudo ./lab.sh up
sudo vtysh -N r1 -f r1-bfd.conf
sleep 10
sudo vtysh -N r1 -c 'show bfd peers brief'
sudo vtysh -N r1 -c 'show ip bgp summary'
```

Predict before you look: does BGP come up? Is the BFD session Up? Does the drill now take 0.3 s or ~150 s? (FRR's default BGP–BFD integration reacts to a session going **down**; a session that never came up does not hold BGP back. Newer FRR releases add `neighbor … bfd strict`, which refuses to bring BGP up until BFD is Up — check `neighbor 10.0.12.2 bfd ?` on your build.) The lesson either way: BFD must be configured on **both** ends, and your verification must check BFD state, not just BGP state.

Clean up when done:

```
sudo ./lab.sh down
```

Choosing timers

Environment	Starting point	Why
Physical routers, hardware/offloaded BFD	50–100 ms × 3	Packets handled off the main CPU
Software routers (FRR on servers)	300 ms × 3 (default)	Survives scheduler hiccups and GC pauses
VMs / containers / cloud	300–1000 ms × 3–5	Noisy neighbors, live migration, steal time
Over a provider L2/L3 service	Agree with the provider	Their policers may drop small UDP

Rules of thumb:

- **Detection time is the number you choose**; interval and multiplier are how you get there. 900 ms that never false-positives beats 150 ms that flaps every afternoon.
- A false BFD-down is not free: it tears down BGP, withdraws routes, and triggers re-convergence across the network. Combine with BGP **graceful restart** awareness (`bfd check-control-plane-failure` in FRR) when routers restart their control planes.

- Protect BFD from your own filters and QoS: allow UDP 3784/4784 (and the echo port 3785 if used) in control-plane policy, and do not let it queue behind bulk traffic.
- Use **multihop** BFD (RFC 5883) for loopback-to-loopback iBGP; single-hop rules (TTL 255) do not apply, so set `minimum-ttl` sensibly.

Verification checklist

Check	Command	Intent
Session up on both ends	`show bfd peers brief` (r1 and r2)	No one-sided config
Live timers	`show bfd peer <ip>`	Negotiated values, not just configured
Client attached	`show ip bgp neighbors <ip>` → `BFD:` block	BGP actually registered
Monitored statics	`show bfd static route`	Uninstalled statics are visible here
Last failure reason	`show bfd peer <ip>` → `Diagnostics`	Detection expired vs admin down
Filters	Control-plane ACL permits UDP 3784/4784	BFD not dropped by your own policy
Drill	`drill.sh` or equivalent	Measured, not assumed, detection time

Common mistakes

Mistake	Symptom
BFD on one side only	BFD never Up; failures still take hold-timer time
Aggressive timers on VMs	Random BGP flaps under load; `control detection time expired` with no link event
Tuning only one end	Detection follows the slower side (Drill 3)
Filtering UDP 3784 in control-plane policy	BFD Down, BGP Up — or worse, BGP reset by strict modes
Treating `AdminDown` as a failure	Maintenance on the peer causes an outage
Single-hop BFD between loopbacks	Session never forms; needs multihop
Trusting `show ip bgp summary` alone	"Established" while forwarding is dead (Drill 1)

Summary

- Through a switch or L2 service, carrier loss is **local**: the far router keeps its port, its session, and its route

- BGP detection is **hold — keepalive ... hold** (120–180 s by default) — measured here at **158.87 s**
- BFD is a shared sub-second liveness check; clients (BGP, OSPF, statics) act on its Down event — measured here at **0.33 s**
- Detection time = remote multiplier × max(local Required Min RX, remote Desired Min TX): **the slower side wins**
- Configure both ends, verify negotiated timers, size timers for your platform, and drill the failure

Sources

- RFC 5880 — Bidirectional Forwarding Detection (BFD): https://www.rfc-editor.org/rfc/rfc5880
- RFC 5881 — BFD for IPv4 and IPv6 (Single Hop): https://www.rfc-editor.org/rfc/rfc5881
- RFC 5882 — Generic Application of BFD: https://www.rfc-editor.org/rfc/rfc5882
- RFC 5883 — BFD for Multihop Paths: https://www.rfc-editor.org/rfc/rfc5883
- FRRouting user guide — BFD (`bfdd`, profiles, BGP and static integration): https://docs.frrouting.org/en/latest/bfd.html
- FRRouting user guide — BGP neighbor timers: https://docs.frrouting.org/en/latest/bgp.html

RPKI — Route Origin Validation

RPKI — Route Origin Validation

BGP believes whatever its neighbors say. If a peer announces your prefix with someone else's AS at the end of the path, your router installs it, prefers it if it is more specific, and forwards your customers' traffic into the wrong network. Prefix lists help for the handful of customers you know; they do nothing for the 1,000,000 routes arriving from a transit provider. **RPKI route origin validation (ROV)** is the boring, widely deployed answer: address holders publish signed statements of *which AS may originate which prefix, up to which length*, a validator turns those into a flat table, and your router labels every route **Valid**, **Invalid**, or **NotFound**. Policy then drops the Invalids. This chapter builds the whole chain on one Linux box — a cache, a validating FRR router, and a misbehaving peer — and then breaks it the three ways it breaks in production.

Learning goals

By the end of this chapter you can:

- Explain the chain ROA → validator → VRP → RTR → router, and what each piece is trusted for
- Apply the RFC 6811 algorithm by hand: covering VRPs, origin AS, maxLength
- Enable RPKI in FRR, connect it to an RTR cache, and read validation state per route
- Show that validation without policy changes **nothing** about forwarding
- Write an inbound route-map that drops Invalid and de-prefers NotFound
- Explain why a loose `maxLength` turns a sub-prefix hijack into a **Valid** route
- Predict what happens to routing when the cache is unreachable, and when VRPs change underneath accepted routes

Concepts — RPKI in one page

Who signs what. The Resource Public Key Infrastructure (RFC 6480) mirrors the address allocation hierarchy: the five RIRs hold trust anchors, they certify the address blocks they hand out, and a holder can publish a **Route Origin Authorization (ROA)** for its space. A ROA says: *prefix P, origin AS A, maximum length M.* It says nothing about the rest of the AS path.

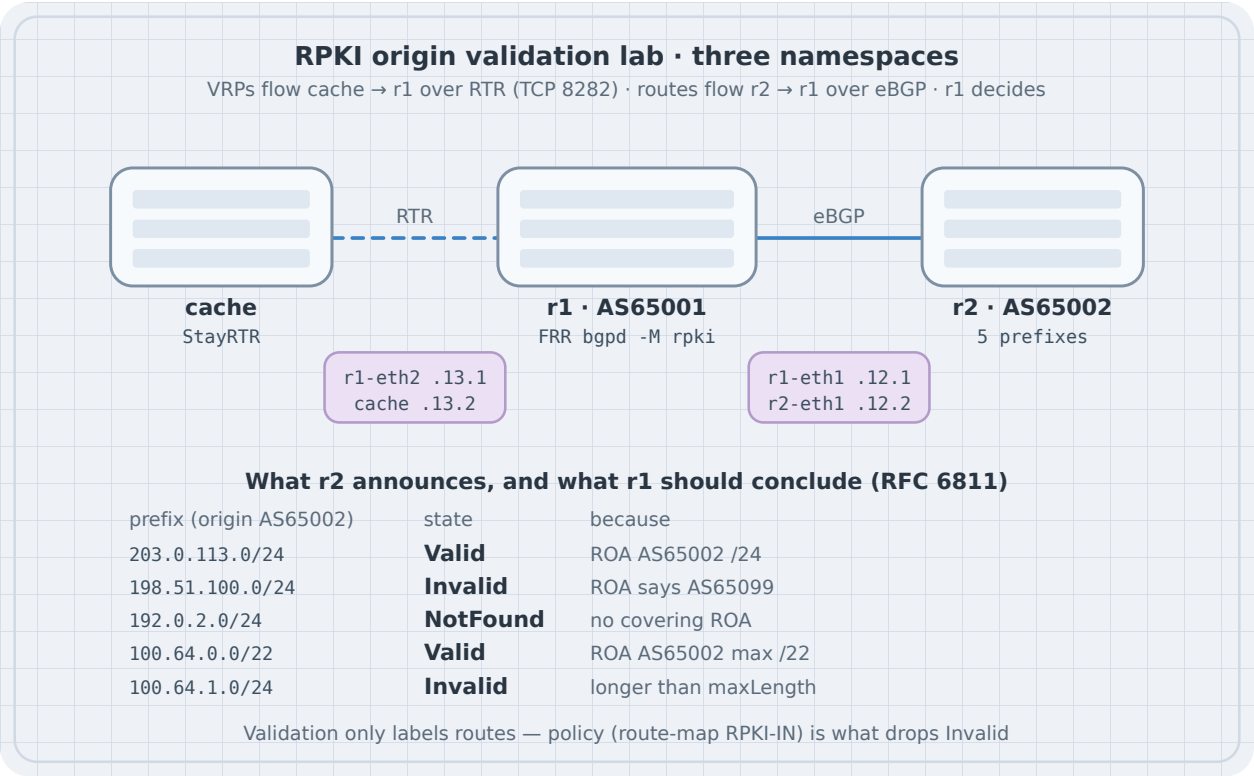


Figure 1: RPKI ROV lab

Who validates. A **relying-party validator** (Routinator, rpki-client, FORT, OctoRPKI…) fetches every repository, checks every signature and expiry, and emits a flat list of **Validated ROA Payloads (VRPs)**: `(prefix, maxLength, ASN)` tuples. Your routers never touch certificates.

How the router learns them. The validator (or a small cache in front of it, such as StayRTR) serves VRPs to routers over the **RPKI-to-Router protocol** (RTR, RFC 8210 for version 1), normally TCP port 8282 or 323. It is a serial-numbered, incremental feed: routers get a full table once, then diffs.

The algorithm (RFC 6811). For a route with prefix R and origin AS O (the last AS in the path):

1. Collect the **covering** VRPs: those whose prefix contains R.
2. None → **NotFound**. Nobody made a claim about this space.
3. Any covering VRP with ASN $= O$ **and** length(R) $\leq$ maxLength → **Valid**.
4. Covering VRPs exist but none match → **Invalid**. Either the wrong AS, or a longer prefix than the holder authorised.

What the router does with the label. Nothing, until you write policy. RFC 7115 describes operational practice; the common baseline today is *drop Invalid, accept NotFound (much of the table still has no ROAs), accept Valid.*

Term	Meaning
ROA	Signed object: prefix, maxLength, origin ASN
VRP	One validated `(prefix, maxLength, ASN)` tuple
RP / validator	Software that turns the RPKI into VRPs
RTR	Protocol that carries VRPs from cache to router
Covering VRP	A VRP whose prefix contains the route's prefix
ROV	Labelling routes by RFC 6811; policy acts on the label

The algorithm fits in a page of Python. Feeding it the lab's VRPs first makes the router output later look familiar:

```
#!/usr/bin/env python3
# rov.py - RFC 6811 route origin validation, nothing else
import ipaddress, json, sys

def load_vrps(path):
    with open(path) as f:
        roas = json.load(f)["roas"]
    return [(ipaddress.ip_network(r["prefix"]), r["maxLength"],
             int(str(r["asn"]).removeprefix("AS"))) for r in roas]

def validate(route, origin, vrps):
    route = ipaddress.ip_network(route)
    covering = [v for v in vrps
                if v[0].version == route.version and route.subnet_of(v[0])]
    if not covering:
        return "NotFound", []
    for pfx, maxlen, asn in covering:
        if asn == origin and route.prefixlen <= maxlen:
            return "Valid", [(str(pfx), maxlen, asn)]
    return "Invalid", [(str(p), m, a) for p, m, a in covering]

if __name__ == "__main__":
    vrps = load_vrps(sys.argv[1])
    for line in sys.stdin:
        route, origin = line.split()
        state, why = validate(route, int(origin), vrps)
        print(f"{route:<18} AS{origin:<6} {state:<9} {why}")
```

```
{
  "metadata": { "buildtime": "2026-10-08T00:00:00Z" },
  "roas": [
    { "prefix": "203.0.113.0/24", "maxLength": 24, "asn": "AS65002" },
    { "prefix": "198.51.100.0/24", "maxLength": 24, "asn": "AS65099" },
    { "prefix": "100.64.0.0/22",  "maxLength": 22, "asn": "AS65002" }
```

```
  ]
}
```

Save the JSON as `vrps.json` (StayRTR reads the same file later) and run:

```
printf '%s\n' "203.0.113.0/24 65002" "198.51.100.0/24 65002" "192.0.2.0/24 65002" \
  "100.64.0.0/22 65002" "100.64.1.0/24 65002" "100.64.1.0/24 65099" \
  | python3 rov.py vrps.json
```

```
203.0.113.0/24     AS65002  Valid     [('203.0.113.0/24', 24, 65002)]
198.51.100.0/24    AS65002  Invalid   [('198.51.100.0/24', 24, 65099)]
192.0.2.0/24       AS65002  NotFound  []
100.64.0.0/22      AS65002  Valid     [('100.64.0.0/22', 22, 65002)]
100.64.1.0/24      AS65002  Invalid   [('100.64.0.0/22', 22, 65002)]
100.64.1.0/24      AS65099  Invalid   [('100.64.0.0/22', 22, 65002)]
```

Read the last two lines carefully: the /24 inside the /22 is Invalid *whoever* originates it, because the holder only authorised the /22. That is the property that makes sub-prefix hijacks fail — and the property a loose `maxLength` gives away (Drill 3).

Addressing plan (RPKI lab)

Node	Interface	Address	Role
r1	r1-eth1	10.0.12.1/24	AS 65001, validating router
r1	r1-eth2	10.0.13.1/24	Management link to the cache
r2	r2-eth1	10.0.12.2/24	AS 65002, announces five prefixes
cache	cache-eth0	10.0.13.2/24	StayRTR, RTR on TCP 8282

Prefix announced by r2	VRP	Expected state
203.0.113.0/24	203.0.113.0/24-24 AS65002	Valid
198.51.100.0/24	198.51.100.0/24-24 AS65099	Invalid (wrong origin — the “hijack”)
192.0.2.0/24	none	NotFound
100.64.0.0/22	100.64.0.0/22-22 AS65002	Valid
100.64.1.0/24	(covered by the /22)	Invalid (too specific)

Lab build (Linux namespaces + FRR + StayRTR)

No container images and no Internet RPKI: three network namespaces, two veth pairs, one FRR instance per router namespace (FRR's pathspace option `-N`, exactly like the BFD chapter's lab), and **StayRTR** serving the `vrps.json` file above as an RTR cache. A real deployment points the same router configuration at a validator such as Routinator or rpki-client; only the cache address changes.

It was run on **Debian 13 with the distro packages FRR 10.3 (`frr`, `frr-rpki-rtrlib`) and StayRTR 0.6.2**, as root. Outputs below are copied from those runs; timestamps and PIDs will differ on yours.

```
sudo apt-get install -y frr frr-rpki-rtrlib stayrtr
sudo systemctl disable --now frr      # the lab starts its own instances
```

`frr-rpki-rtrlib` ships the bgpd module `bgpd_rpki.so`; bgpd only loads it when started with `-M rpki`. Without the package bgpd refuses to start with that flag.

```
#!/usr/bin/env bash
# lab.sh - r1 (validating FRR) peers with r2 (announcing FRR); "cache" runs StayRTR
# usage: sudo ./lab.sh up | down
set -euo pipefail
FRR=/usr/lib/frr
DAEMONS="mgmtd zebra bgpd staticd"
HERE=$(cd "$(dirname "$0")" && pwd)

up() {
  for ns in r1 r2 cache; do ip netns add "$ns"; ip -n "$ns" link set lo up; done

  # eBGP link r1 <-> r2
  ip link add r1-eth1 netns r1 type veth peer name r2-eth1 netns r2
  ip -n r1 addr add 10.0.12.1/24 dev r1-eth1
  ip -n r2 addr add 10.0.12.2/24 dev r2-eth1
  ip -n r1 link set r1-eth1 up; ip -n r2 link set r2-eth1 up

  # management link r1 <-> RPKI cache (RTR runs over plain TCP here)
  ip link add r1-eth2 netns r1 type veth peer name cache-eth0 netns cache
  ip -n r1 addr add 10.0.13.1/24 dev r1-eth2
  ip -n cache addr add 10.0.13.2/24 dev cache-eth0
  ip -n r1 link set r1-eth2 up; ip -n cache link set cache-eth0 up

  # the relying-party cache: StayRTR serving VRPs from a local JSON file
  ip netns exec cache stayrtr -bind 10.0.13.2:8282 -cache "$HERE/vrps.json" \
    -checktime=false -refresh 5 -metrics.addr "" >"$HERE/stayrtr.log" 2>&1 &
  echo $! > "$HERE/stayrtr.pid"

  for ns in r1 r2; do
```

```
    install -d -o frr -g frr "/var/run/frr/$ns" "/etc/frr/$ns"
    : > "/etc/frr/$ns/vtysh.conf"
    for d in $DAEMONS; do
      extra=""; [ "$d" = bgpd ] && [ "$ns" = r1 ] && extra="-M rpki"
      ip netns exec "$ns" "$FRR/$d" -d -N "$ns" -F traditional $extra \
        -u frr -g frr --limit-fds 1024 >/dev/null 2>&1
    done
    vtysh -N "$ns" -f "$HERE/$ns.conf" >/dev/null
  done
}

down() {
  for ns in r1 r2; do
    for d in $DAEMONS; do
      pidf="/var/run/frr/$ns/$d.pid"
      [ -f "$pidf" ] && kill "$(cat "$pidf")" 2>/dev/null || true
    done
  done
  [ -f "$HERE/stayrtr.pid" ] && kill "$(cat "$HERE/stayrtr.pid")" 2>/dev/null || true
  for ns in r1 r2 cache; do ip netns del "$ns" 2>/dev/null || true; done
}

"$@"
```

StayRTR flags used: `-cache` accepts a URL *or* a local file; `-checktime=false` stops it rejecting our hand-written file for having no fresh validator timestamp; `-refresh 5` re-reads the file every five seconds so the drills move quickly (the default is 600 s).

r2 announces all five prefixes; the blackhole statics only exist so that `network` statements have something in the RIB to match:

```
! r2.conf - AS 65002, announces four prefixes with four different RPKI outcomes
hostname r2
ip route 203.0.113.0/24 blackhole
ip route 198.51.100.0/24 blackhole
ip route 192.0.2.0/24 blackhole
ip route 100.64.0.0/22 blackhole
ip route 100.64.1.0/24 blackhole
router bgp 65002
 bgp router-id 10.0.12.2
 no bgp ebgp-requires-policy
 neighbor 10.0.12.1 remote-as 65001
 address-family ipv4 unicast
  network 203.0.113.0/24
  network 198.51.100.0/24
  network 192.0.2.0/24
  network 100.64.0.0/22
```

```
  network 100.64.1.0/24
 exit-address-family
```

r1 connects to the cache and validates — but has **no RPKI policy yet**:

```
! r1.conf - AS 65001, validates origins against the RPKI cache (no policy yet)
hostname r1
rpki
 rpki cache tcp 10.0.13.2 8282 preference 1
exit
router bgp 65001
 bgp router-id 10.0.12.1
 no bgp ebgp-requires-policy
 neighbor 10.0.12.2 remote-as 65002
```

`preference` orders several caches: the router uses the lowest-numbered group it can reach. Production routers should have at least two caches on different hosts.

Bring it up and check the RTR session:

```
chmod +x lab.sh
sudo ./lab.sh up
sleep 12
sudo vtysh -N r1 -c 'show rpki cache-connection'
sudo vtysh -N r1 -c 'show rpki prefix-table'
```

```
Connected to group 1
rpki tcp cache 10.0.13.2 8282 pref 1 (connected)
RPKI/RTR prefix table
Prefix                                  Prefix Length  Origin-AS
100.64.0.0                                 22 -  22   65002
203.0.113.0                                24 -  24   65002
198.51.100.0                               24 -  24   65099
Number of IPv4 Prefixes: 3
Number of IPv6 Prefixes: 0
```

`22 - 22` is "prefix length 22, maxLength 22". You can ask the cache directly what it serves, from r1's namespace, with StayRTR's companion tool:

```
sudo ip netns exec r1 rtrdump -connect 10.0.13.2:8282 -file /dev/stdout
```

```
{"metadata":{"vrps":3,"bgpsec_pubkeys":0,"sessionid":16344,"serial":0},"roas":[{"prefix":"19
```

Drill 1 — validation is a label, not a filter

Predict

r1 knows that 198.51.100.0/24 belongs to AS65099 and that nothing more specific than /22 should exist inside 100.64.0.0/22. Which of r2's five routes will r1 install?

Observe

```
sudo vtysh -N r1 -c 'show bgp ipv4 unicast'
sudo ip netns exec r1 ip route show proto bgp
```

```
RPKI validation codes: V valid, I invalid, N Not found

     Network          Next Hop            Metric LocPrf Weight Path
V*>  100.64.0.0/22    10.0.12.2                0             0 65002 i
I*>  100.64.1.0/24    10.0.12.2                0             0 65002 i
N*>  192.0.2.0/24     10.0.12.2                0             0 65002 i
I*>  198.51.100.0/24  10.0.12.2                0             0 65002 i
V*>  203.0.113.0/24   10.0.12.2                0             0 65002 i

Displayed 5 routes and 5 total paths
100.64.0.0/22 nhid 10 via 10.0.12.2 dev r1-eth1 metric 20
100.64.1.0/24 nhid 10 via 10.0.12.2 dev r1-eth1 metric 20
192.0.2.0/24 nhid 10 via 10.0.12.2 dev r1-eth1 metric 20
198.51.100.0/24 nhid 10 via 10.0.12.2 dev r1-eth1 metric 20
203.0.113.0/24 nhid 10 via 10.0.12.2 dev r1-eth1 metric 20
```

The per-route view names the state explicitly:

```
sudo vtysh -N r1 -c 'show bgp ipv4 unicast 100.64.1.0/24'
sudo vtysh -N r1 -c 'show bgp ipv4 unicast rpki invalid'
```

```
BGP routing table entry for 100.64.1.0/24, version 2
Paths: (1 available, best #1, table default)
  Advertised to non peer-group peers:
  10.0.12.2
  65002
    10.0.12.2 from 10.0.12.2 (10.0.12.2)
      Origin IGP, metric 0, valid, external, best (First path received), rpki validation-sta
...
     Network          Next Hop            Metric LocPrf Weight Path
I*>  100.64.1.0/24    10.0.12.2                0             0 65002 i
I*>  198.51.100.0/24  10.0.12.2                0             0 65002 i
```

```
Displayed 2 routes and 5 total paths
```

Explain

All five are best (`*>`) and all five are in the kernel FIB, Invalids included. FRR's documentation says so plainly: enabling RPKI does not change best-path selection. Note the two meanings of "valid" in one line — `valid, external, best` is BGP's next-hop/attribute validity, `rpki validation-state: invalid` is origin validation. Do not grep for the wrong one.

Drill 2 — drop Invalid, de-prefer NotFound

```
! r1-policy.conf - drop Invalid, de-prefer NotFound, keep Valid
route-map RPKI-IN deny 10
 match rpki invalid
exit
route-map RPKI-IN permit 20
 match rpki notfound
 set local-preference 50
exit
route-map RPKI-IN permit 30
exit
router bgp 65001
 address-family ipv4 unicast
  neighbor 10.0.12.2 soft-reconfiguration inbound
  neighbor 10.0.12.2 route-map RPKI-IN in
 exit-address-family
```

`soft-reconfiguration inbound` keeps an unmodified Adj-RIB-In copy of what the peer sent, including routes your policy rejected. FRR's RPKI documentation requires it for cache updates to be applied to routes; Drill 4 shows why you want it anyway.

```
sudo vtysh -N r1 -f r1-policy.conf
sudo vtysh -N r1 -c 'clear bgp ipv4 unicast 10.0.12.2 soft in'
sudo vtysh -N r1 -c 'show bgp ipv4 unicast'
sudo vtysh -N r1 -c 'show bgp ipv4 unicast neighbors 10.0.12.2 received-routes'
sudo ip netns exec r1 ip route show proto bgp
```

```
     Network          Next Hop            Metric LocPrf Weight Path
V*>  100.64.0.0/22    10.0.12.2                0             0 65002 i
N*>  192.0.2.0/24     10.0.12.2                0     50      0 65002 i
V*>  203.0.113.0/24   10.0.12.2                0             0 65002 i

Displayed 3 routes and 3 total paths
```

```
    Network          Next Hop            Metric LocPrf Weight Path
 *> 100.64.0.0/22    10.0.12.2                0             0 65002 i
 *> 100.64.1.0/24    10.0.12.2                0             0 65002 i
 *> 192.0.2.0/24     10.0.12.2                0             0 65002 i
 *> 198.51.100.0/24  10.0.12.2                0             0 65002 i
 *> 203.0.113.0/24   10.0.12.2                0             0 65002 i

Total number of prefixes 5 (2 filtered)
100.64.0.0/22 nhid 10 via 10.0.12.2 dev r1-eth1 metric 20
192.0.2.0/24 nhid 10 via 10.0.12.2 dev r1-eth1 metric 20
203.0.113.0/24 nhid 10 via 10.0.12.2 dev r1-eth1 metric 20
```

The hijacked 198.51.100.0/24 is gone. The too-specific 100.64.1.0/24 is gone, but traffic to 100.64.1.x still flows via the legitimate /22 — dropping an Invalid more-specific does not blackhole the space, it just stops the more-specific from winning longest-match. `received-routes` proves the peer still sends all five and the filter is doing the work.

Why `set local-preference 50` on NotFound rather than dropping it: a large share of the Internet table still has no ROAs. Dropping NotFound would cut you off from it. De-preferring means a Valid path wins whenever one exists.

Drill 3 — the maxLength trap

Predict

The holder of 100.64.0.0/22 decides to "save time" and publishes its ROA with `maxLength 24`, so it can announce /24s later without touching RPKI. It still announces only the /22. What happens to r2's 100.64.1.0/24?

Edit `vrps.json`, changing only the /22 line to `"maxLength": 24`. StayRTR notices within five seconds and bumps its serial:

```
time="2026-10-08T09:03:09+05:30" level=info msg="New update (3 uniques, 3 total prefixes, 0
time="2026-10-08T09:03:09+05:30" level=info msg="Update added, new serial 1"
```

```
sudo vtysh -N r1 -c 'show rpki prefix-table'
sudo vtysh -N r1 -c 'clear bgp ipv4 unicast 10.0.12.2 soft in'
sudo vtysh -N r1 -c 'show bgp ipv4 unicast'
```

```
Prefix                                  Prefix Length  Origin-AS
100.64.0.0                                 22 -  24   65002
...
     Network          Next Hop            Metric LocPrf Weight Path
V*>  100.64.0.0/22    10.0.12.2                0             0 65002 i
V*>  100.64.1.0/24    10.0.12.2                0             0 65002 i
N*>  192.0.2.0/24     10.0.12.2                0     50      0 65002 i
```

```
V*>  203.0.113.0/24   10.0.12.2                0             0 65002 i
```

Explain

The /24 is now **Valid**. In the lab r2 really is AS65002, but on the Internet the attacker does not need to be: a **forged-origin** hijack announces `100.64.1.0/24` with an AS path ending in `65002`. ROV checks only the last AS, so the forged route validates, and because it is more specific than the real /22 it attracts all the traffic for that quarter of the block. With `maxLength 22`, the same forgery is Invalid and dropped by every ROV network.

RFC 9319 (BCP 185) turns this into the rule: **do not use maxLength unless every prefix it authorises is actually announced**; publish one ROA per announced prefix instead ("minimal ROAs"). Put the setting back to 22 before the next drill.

Drill 4 — state changes under accepted routes

Predict

You tighten the ROA back to `maxLength 22`. The cache updates; r1's prefix-table updates. Is 100.64.1.0/24 withdrawn from r1's FIB?

Observe

About ten seconds after the edit, and again twenty seconds later:

```
sudo vtysh -N r1 -c 'show bgp ipv4 unicast'
sudo ip netns exec r1 ip route show 100.64.1.0/24
```

```
     Network          Next Hop            Metric LocPrf Weight Path
V*>  100.64.0.0/22    10.0.12.2                0             0 65002 i
I*>  100.64.1.0/24    10.0.12.2                0             0 65002 i
N*>  192.0.2.0/24     10.0.12.2                0     50      0 65002 i
V*>  203.0.113.0/24   10.0.12.2                0             0 65002 i

Displayed 4 routes and 4 total paths
100.64.1.0/24 nhid 10 via 10.0.12.2 dev r1-eth1 proto bgp metric 20
```

The label flipped to `I` — and the route stayed best and installed. On this FRR 10.3 build, the new validation state was recorded but the inbound route-map was not re-run for an already-accepted route. Re-running inbound policy fixes it:

```
sudo vtysh -N r1 -c 'clear bgp ipv4 unicast 10.0.12.2 soft in'
sudo vtysh -N r1 -c 'show bgp ipv4 unicast'
sudo ip netns exec r1 ip route show 100.64.1.0/24
```

```
     Network          Next Hop            Metric LocPrf Weight Path
V*>  100.64.0.0/22    10.0.12.2                0             0 65002 i
N*>  192.0.2.0/24     10.0.12.2                0     50      0 65002 i
V*>  203.0.113.0/24   10.0.12.2                0             0 65002 i

Displayed 3 routes and 3 total paths
```

(no FIB line — the /24 is gone). The same happened in the other direction in Drill 3: the /24 rejected under `maxLength 22` did not reappear when the ROA was loosened until the soft clear.

Explain

FRR's RPKI guide says cache updates are applied and path selection updated, provided soft reconfiguration is enabled. Whether a particular release re-applies *policy* (not just the label) on every VRP change is exactly the kind of thing to **measure on your version** rather than assume. Operational answers that work everywhere: keep `soft-reconfiguration inbound` on RPKI-filtered sessions, alert on `show bgp ipv4 unicast rpki invalid` being non-empty, and schedule a periodic `clear bgp * soft in` (cheap with an Adj-RIB-In) until you have proved your release does it for you.

Drill 5 — the cache goes away (predict, then try)

Stop the cache and look at r1:

```
sudo kill "$(cat stayrtr.pid)"
sleep 8
sudo vtysh -N r1 -c 'show rpki cache-connection'
sudo vtysh -N r1 -c 'show rpki prefix-table' | tail -2
```

```
Connected to group 1
rpki tcp cache 10.0.13.2 8282 pref 1
Number of IPv4 Prefixes: 3
Number of IPv6 Prefixes: 0
```

The `(connected)` marker is gone but the VRPs remain: RTR data lives until the **expire interval** (StayRTR advertises 7200 s by default; FRR has `rpki expire_interval`). Routing continues unchanged. After expiry, every route becomes NotFound — Invalids included — and your "drop Invalid" policy silently stops dropping anything. ROV **fails open** by design: a dead validator must not take the network down, but it does take your protection away. Hence two caches, on different hosts, monitored. Newer FRR releases add `neighbor PEER rpki strict` to hold a session down until a cache is connected; FRR 10.3 rejects that command (`% Unknown command`), so check your version.

Checking real-world origins

The same three states exist on the live Internet, and RIPEstat exposes a validator over HTTPS. A real ROA, then a deliberately wrong origin:

```
curl -s 'https://stat.ripe.net/data/rpki-validation/data.json?resource=AS13335&prefix=1.1.1.
  | python3 -c "import json,sys;d=json.load(sys.stdin)['data'];print(json.dumps({k:d[k] for
curl -s 'https://stat.ripe.net/data/rpki-validation/data.json?resource=AS64500&prefix=1.1.1.
  | python3 -c "import json,sys;d=json.load(sys.stdin)['data'];print(d['resource'],d['prefix
```

```
{
 "resource": "13335",
 "prefix": "1.1.1.0/24",
 "status": "valid",
 "validating_roas": [
  {
   "origin": "13335",
   "prefix": "1.1.1.0/24",
   "validity": "valid",
   "max_length": 24
  }
 ]
}
64500 1.1.1.0/24 invalid_asn
```

Use it to check your own prefixes before you publish a ROA, and after: the holder can make its own routes Invalid with a mistake, and every ROV network will then drop them.

Deploying it for real

Step	What	Why
1	Publish ROAs for your own space (RIR portal, hosted RPKI) — one per announced prefix, `maxLength` = prefix length	Others can protect you; avoids Drill 3
2	Run two validators/caches (e.g., Routinator and rpki-client) on separate hosts	Fail-open is fine only if it is rare (Drill 5)
3	Point every border router at both (`rpki cache … preference 1/2`)	Cache failover
4	Start with **monitor-only**: label, do not filter; review `rpki invalid`	Find your customers' broken ROAs first

Step	What	Why
5	Enable `deny invalid` on transit and peer sessions, then on customers	The RFC 7115 path
6	Re-run inbound policy on VRP changes, or prove your release does (Drill 4)	No stale accepts
7	Alert on cache disconnect, VRP count drops, and serial not advancing	Validator outages are silent

ROV protects against wrong *origins*. It does not stop route **leaks** (valid origin, wrong path) or forged-origin hijacks of space with loose ROAs; path validation (ASPA, BGPsec) is the next layer and is outside this chapter.

Verification checklist

Check	Command	Intent
Cache connected	`show rpki cache-connection`	`(connected)` on at least one group
VRPs loaded	`show rpki prefix-table`	Non-zero, plausible count
State per route	`show bgp ipv4 unicast <prefix>`	`rpki validation-state:` line
Invalids accepted	`show bgp ipv4 unicast rpki invalid`	Should be empty with policy on
Policy attached	`show running-config` → `route-map RPKI-IN in`	Labels mean nothing without it
Filter doing work	`… neighbors <ip> received-routes`	`(N filtered)` matches expectations
Cache content	`rtrdump -connect <cache>:8282`	What routers actually receive

Common mistakes

Mistake	Symptom
RPKI enabled, no route-map	Invalids best and installed (Drill 1)
Dropping NotFound	Large parts of the Internet unreachable
`maxLength` wider than what you announce	Forged-origin sub-prefix hijack validates (Drill 3)
No soft-reconfiguration / no re-evaluation	Stale Invalids stay installed after VRP changes (Drill 4)
One cache	Validator outage → silent fail-open (Drill 5)

Mistake	Symptom
ROA for the aggregate only, then announcing a /24 for traffic engineering	Your own /24 is Invalid and dropped by others
Grepping for "valid" in `show bgp`	Confuses BGP path validity with RPKI state

Summary

- A ROA authorises **one origin AS for a prefix up to a maxLength**; validators flatten the RPKI into VRPs; routers receive VRPs over RTR
- RFC 6811: no covering VRP → NotFound; matching origin within maxLength → Valid; otherwise → Invalid
- Validation labels routes; **policy** drops them — without a route-map all five lab routes were installed
- Loose `maxLength` turned the 100.64.1.0/24 sub-prefix from Invalid into **Valid** (RFC 9319: don't)
- On FRR 10.3, VRP changes updated labels but did not re-run inbound policy until `clear … soft in`
- With the cache gone, VRPs persist until expiry, then everything becomes NotFound: ROV fails open

Sources

- RFC 6480 — An Infrastructure to Support Secure Internet Routing: https://www.rfc-editor.org/rfc/rfc6480
- RFC 6811 — BGP Prefix Origin Validation: https://www.rfc-editor.org/rfc/rfc6811
- RFC 7115 — Origin Validation Operation Based on the RPKI: https://www.rfc-editor.org/rfc/rfc7115
- RFC 8210 — The RPKI to Router Protocol, Version 1: https://www.rfc-editor.org/rfc/rfc8210
- RFC 9319 (BCP 185) — The Use of maxLength in the RPKI: https://www.rfc-editor.org/rfc/rfc9319
- RFC 9582 — A Profile for Route Origin Authorizations (ROAs): https://www.rfc-editor.org/rfc/rfc9582

- FRRouting user guide — RPKI: https://docs.frrouting.org/en/latest/bgp.html#prefix-origin-validation-using-rpki (source: https://github.com/FRRouting/frr/blob/master/doc/user/rpki.rst)

- StayRTR: https://github.com/bgp/stayrtr

- RIPEstat RPKI Validation data call: https://stat.ripe.net/docs/data-api/api-endpoints/rpki-validation

Segment Routing, SRv6, and TI-LFA

Segment Routing, SRv6, and TI-LFA

IGP shortest paths are easy to explain and slow to heal. Classic Loop-Free Alternates (LFA) and Remote LFA cover many failures but not all topologies. **Segment Routing (SR)** (RFC 8402) lets the ingress (or a repair point) encode an explicit path as a list of **segments**. On IPv6, that list lives in the **SR Header** and is programmed with **SRv6 network programming** (RFC 8986). **TI-LFA** (RFC 9855) uses those segments so a **Point of Local Repair (PLR)** can pre-install a loop-free backup that follows the **expected post-convergence** path — topology-independent coverage in two-connected IGP areas, without Targeted LDP tunnels. This chapter stays vendor-neutral and RFC-first.

Learning goals

By the end of this chapter you can:

- State the SR architecture: Prefix-SID / Adjacency-SID, SR-MPLS vs SRv6
- Explain SRv6 SID lists, active segment, and head-end behaviors at a PLR
- Define TI-LFA, P-space / Q-space, and Repair List (RL)
- Predict what the PLR pushes when the active segment is a node SID vs an adjacency SID
- Relate TI-LFA to BFD-scale detection: FRR is local and precomputed; IGP still converges

Thesis

TI-LFA is not a new IGP. It is a **local repair algorithm** that *expresses* post-failure shortest paths as SR segment lists the PLR can push the moment a link (or protected node/SRLG) dies — then withdraws that repair when the IGP SPF at the PLR catches up.

Mental model

```
Before failure (IGP SPF):     S  → F  → …  → D
                                \                ↑
After local repair (TI-LFA):      RL → P   Q
```

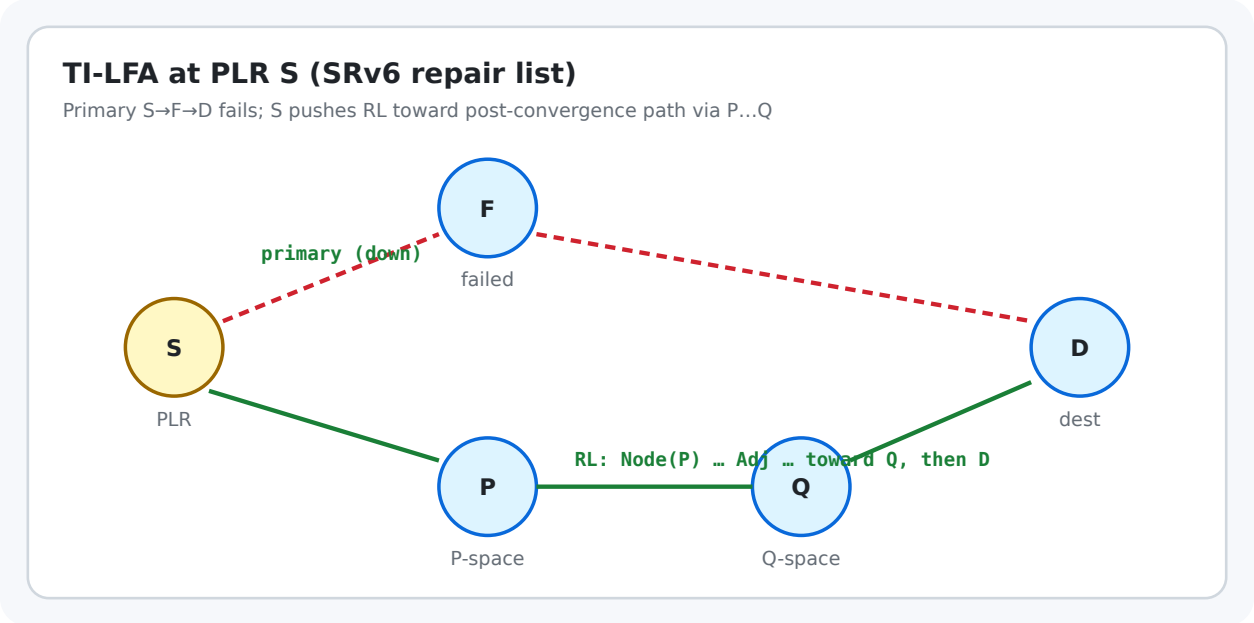


Figure 1: TI-LFA repair at the PLR

Term (RFC 9855 / 8402)	Meaning
PLR	Router that detects local failure and activates a precomputed backup
RL (Repair List)	Segment list encoding a loop-free path avoiding the protected resource
P-space	Nodes reachable from the PLR *without* traversing the failed resource
Q-space	Nodes that can reach the destination *without* traversing the failed resource
Post-convergence path	SPF path the PLR would use after the IGP absorbs the failure

SR data planes

Plane	How a segment is carried	Primary RFCs
SR-MPLS	MPLS label stack	RFC 8402, RFC 8660
SRv6	IPv6 destination + optional SRH (RFC 8754); behaviors in RFC 8986	RFC 8402, 8754, 8986

SRv6 **Adjacency SIDs** are often globally routed, so RLs can be shorter than SR-MPLS (less need to precede an Adj-SID with a Prefix-SID) — RFC 9855 §7.2.

Worked examples

Case 1: Architecture in one page (RFC 8402)

A **segment** is an instruction. Common kinds:

- **Prefix-SID (node segment)** — "go to this node via the IGP algorithm (usually SPF)"
- **Adjacency-SID** — "take this specific link / adjacency"

An **SR Policy** is an ordered list of segments. On SRv6, the **active** segment is the IPv6 Destination Address; `Segments Left` in the SRH walks the list (RFC 8402 §3.1.3 summary).

Operators instantiate SRv6 SIDs explicitly — ordinary interface addresses are **not** SIDs by default.

Case 2: TI-LFA repair flavours (RFC 9855 §5)

Situation along post-convergence path	RL shape
Direct neighbor in P Q	Empty RL; just change next hop (post-convergence LFA)
Remote PQ node	Single node segment to that PQ
Adjacent P then Q	Node-SID(P) then Adj-SID(P→Q)
Distant P and Q	Node-SID(P) + explicit path EP(P,Q) (Adj-SID list or optimized mix)

Protection modes: **link**, **node**, or **SRLG** failure assumptions when computing the backup.

Case 3: What the PLR does to the packet (RFC 9855 §6–7)

Active segment is a node SID. Keep it; **add** the RL so the node SID becomes the first segment *after* the repair list.

Active segment is an adjacency SID S–F (protected). Apply NEXT on that Adj-SID, then PUSH an RL that restores traffic toward F (or toward a Q whose path to the following node SID avoids the failure) — details in §6.2.1–6.2.2.

SRv6 transit PLR. Add the SID list using RFC 8986 **head-end** behaviors (e.g. `H.Encaps` / SRH push as implemented). **Endpoint PLR:** run endpoint processing first, then protect as transit.

USD flavor (RFC 8986) is often cited so the last repair endpoint can **decapsulate** the outer header the PLR added and forward the inner packet — useful for TI-LFA encapsulation on P routers.

Case 4: Numbers from RFC 9855 Appendix B (SR-MPLS measurements)

On real SP/enterprise topologies measured in the RFC:

- Link / local SRLG protection: **1 SID** repair path covers **>99%** of prefixes in the data set
- Node protection: **2 SIDs** yields about **99%** coverage

Exact hardware MSD limits still matter; implementations may optimize RL length.

The trap

- Expecting TI-LFA to protect **inter-domain** exits or non-IGP destinations — RFC 9855 limits scope to link-state IGP areas.
- Forgetting **micro-loops** during distributed convergence: TI-LFA is complementary to loop-free convergence methods (RFC 5715, RFC 8333, etc.), not a full substitute.
- Building RLs from **protected** Adj-SIDs and then suffering **double FRR** if a repair hop fails (RFC 9855 §9); some implementations prefer unprotected Adj-SIDs in RLs.
- Assuming SR Policies with custom constraints stay on-policy during local repair — TI-LFA does not know head-end TE constraints; use policy-level protection or Flexible Algorithm-aware TI-LFA (RFC 9855 §8) when constraints matter.
- Confusing **detection** (BFD, carrier) with **repair encoding** (TI-LFA): you need both for fast restoration.

The boring rule

Run a link-state IGP with SR enabled; precompute TI-LFA RLs toward post-convergence paths; detect failures quickly; keep RL depth inside MSD; let SPF release the repair after convergence.

Try this

1. Read RFC 8402 §§1–3 (architecture) and RFC 8986 §5 (head-end behaviors) skim.
2. Walk RFC 9855 §5 and Figure 1: compute P-space / Q-space intersection with the post-convergence path on paper.

3. On a lab with FRR or another open IGP+SR stack (when you already have it), enable SR and TI-LFA and compare `show` repair SIDs before/after shutting a link — no new packages required for reading RFCs alone.
4. Related in this book: BFD for *detection* timers; this chapter for *where packets go* during the repair window.

Sources

- RFC 8402 — Segment Routing Architecture. https://www.rfc-editor.org/rfc/rfc8402
- RFC 8986 — SRv6 Network Programming. https://www.rfc-editor.org/rfc/rfc8986
- RFC 8754 — IPv6 Segment Routing Header (SRH). https://www.rfc-editor.org/rfc/rfc8754
- RFC 9855 — Topology Independent Fast Reroute Using Segment Routing (TI-LFA). https://www.rfc-editor.org/rfc/rfc9855
- RFC 8660 — SR-MPLS (for the MPLS twin of the same RL algorithms)

BGP Route Reflectors: Escaping the iBGP Full Mesh

BGP Route Reflectors: Escaping the iBGP Full Mesh

iBGP has one rule that shapes every large network: a router does **not** pass a route learned from one iBGP neighbour on to another iBGP neighbour (RFC 4271 §9.2). That rule stops loops inside an AS, where AS_PATH can't, because AS_PATH doesn't change inside the AS. The price is a **full mesh**: every BGP router in the AS needs a session with every other, $n(n-1)/2$ sessions in all. Route reflection (RFC 4456) relaxes the rule on chosen routers, the **route reflectors**, and adds two attributes, ORIGINATOR_ID and CLUSTER_LIST, so the relaxed rule can't loop. It also changes what the clients can see: a reflector passes on only **its own best path**, picked from **its own** place in the IGP. This chapter counts the sessions, runs the reflection rules in a small model where every path is visible, and then builds the same network as a netlab lab on FRR.

Learning goals

- Explain why iBGP needs a full mesh, and count the sessions it costs
- State the RFC 4456 reflection rules: client, non-client and eBGP routes
- Read ORIGINATOR_ID and CLUSTER_LIST on a reflected route, and say which loop each one stops
- Place ORIGINATOR_ID and CLUSTER_LIST length in the BGP decision process
- Predict **path hiding**: when a client's exit choice differs from a full mesh, and fix it with add-path (RFC 7911) or optimal route reflection (RFC 9107)
- Choose between one shared cluster ID and one per reflector, knowing what each costs
- Build a two-reflector lab with netlab, and verify it with `netlab validate`

Lab platform: Python 3.13.5 for the session count and the reflection model, run unprivileged on this box. netlab **26.10** in a Python venv for `netlab create`, which compiles the topology and renders the FRR configuration without starting anything. Starting the lab (`netlab up`, containerlab, FRR **10.7.1** containers) needs root and a container runtime, so that part was **not run here**: it is written from the netlab and FRR documentation and marked where it applies.

Thesis

A route reflector is a router that is allowed to break the iBGP split-horizon rule for its **clients**. That turns $n(n-1)/2$ sessions into about two per client. What you give up is visibility: a full mesh shows every router every exit; a reflector shows each client one exit, the one that is best from the reflector's seat in the IGP. When reflectors sit where their clients are (RFC 4456 §11), nothing changes. When they sit somewhere else, typically a pair of control-plane-only reflectors in one data centre, distant clients start using the reflector's nearest exit instead of their own. That is how a

network gets “hot-potato routing that isn’t”. Know where your reflectors sit, and send more than one path when they can’t sit everywhere.

Mental model

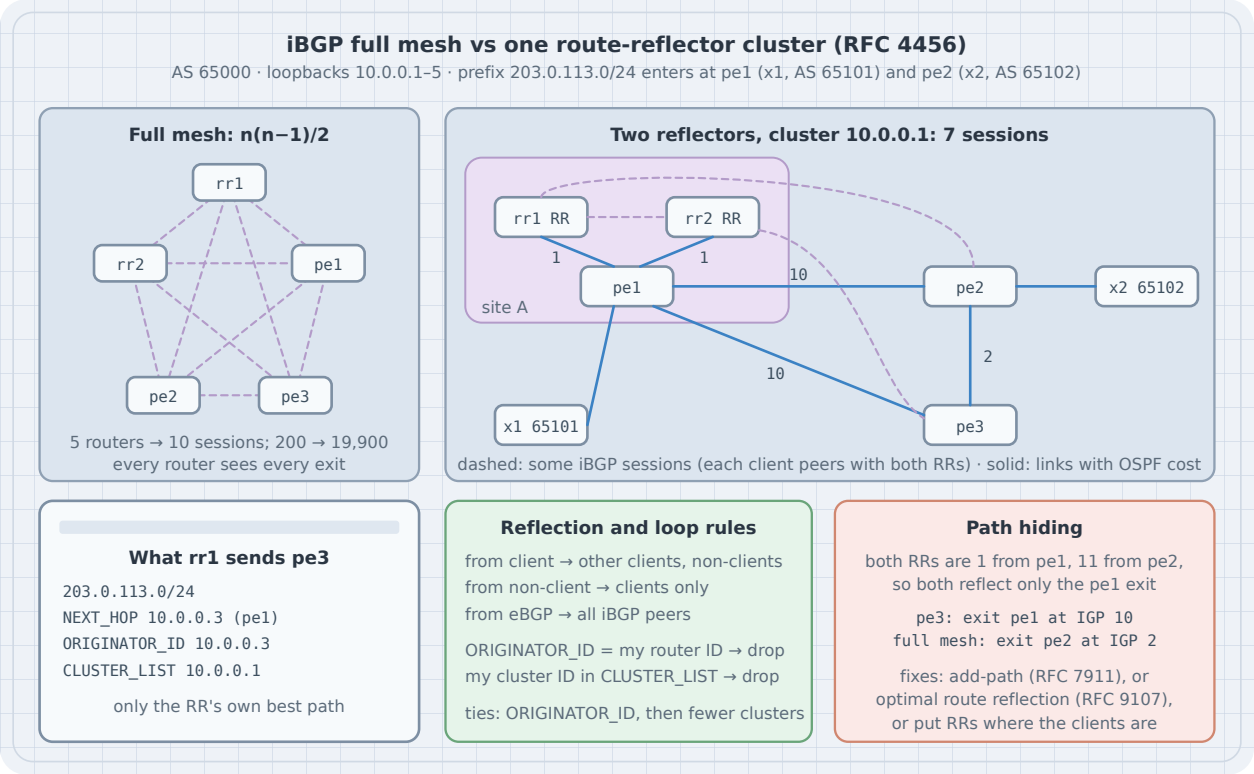


Figure 1: iBGP full mesh vs one route-reflector cluster: session counts, a reflected route’s attributes, the reflection and loop rules, and path hiding

```
          full mesh                                  route reflection (one cluster)
every router   every router                  client   RR (both RRs)      RR   RR (non-clients)
n(n-1)/2 sessions                              k sessions per client, k = number of RRs

a reflected route carries:
   ORIGINATOR_ID  router ID of the client that injected it into the AS   (set once, never
   CLUSTER_LIST   cluster IDs of every reflector it passed, newest first (prepended per re
```

Term	RFC 4456	Meaning
Route reflector (RR)	§5	iBGP speaker allowed to re-advertise iBGP-learned routes to its clients

Term	RFC 4456	Meaning
Client	§5	iBGP peer the RR is configured to reflect to (`route-reflector-client` in FRR). The client itself needs no special config
Non-client	§5	Any other iBGP peer of the RR, typically the other RRs. Non-clients must still be fully meshed with each other
Cluster	§5, §7	An RR and its clients. Several RRs can share one **CLUSTER_ID**; by default an RR uses its router ID
ORIGINATOR_ID	§8, type code 9	Optional non-transitive, 4 bytes: the router ID of the route's originator in this AS. Created by the first RR, never overwritten
CLUSTER_LIST	§8, type code 10	Optional non-transitive list of CLUSTER_IDs. Every RR prepends its own when it reflects

Concepts

The reflection rules (RFC 4456 §6)

After an RR selects its best path, it sends it on according to where it came from:

Best path learned from	Sent to
A **non-client** iBGP peer	All clients
A **client**	All non-clients and all clients ("Hence the Client peers are not required to be fully meshed"). In practice not back to the client it came from
An **eBGP** peer	All iBGP peers, as ordinary BGP (RFC 4271)

Nothing else about the route changes: §10 says an RR **SHOULD NOT** modify NEXT_HOP, AS_PATH, LOCAL_PREF or MED when it reflects. In particular the NEXT_HOP stays the exit router's address, so every client must be able to reach every exit through the IGP. FRR follows this: `next-hop-self` on an RR's client sessions doesn't touch reflected routes unless you add `force`.

Two loop checks

- **ORIGINATOR_ID:** "A router that recognizes the ORIGINATOR_ID attribute SHOULD ignore a route received with its BGP Identifier as the ORIGINATOR_ID." This catches a route coming back to the router that injected it.
- **CLUSTER_LIST:** "If the local CLUSTER_ID is found in the CLUSTER_LIST, the advertisement received SHOULD be ignored." This catches a route coming back to a cluster it has already passed through, including a route from the other RR of the **same** cluster.

Route selection (RFC 4456 §9)

Two changes to the RFC 4271 §9.1.2.2 tie-breaks: in step f), the lowest BGP Identifier, a reflected route is compared by its **ORIGINATOR_ID** instead of the router ID of the peer that sent it; and a new step between f) and g) prefers the **shorter CLUSTER_LIST**. Both come late. Everything before them, including step e), the lowest IGP cost to the NEXT_HOP, is computed by each router from **its own** position. On a reflector, that position is the reflector's, and that's where path hiding comes from.

One cluster ID or two?

	Both RRs share one CLUSTER_ID (netlab's default)	Each RR its own CLUSTER_ID
What an RR does with the other RR's reflection of a client route	Drops it (its own ID is in CLUSTER_LIST)	Keeps it as an extra path
RR memory	One copy per client path	About two copies per client path
If a client loses its session to one RR	That RR no longer has the client's routes. Other clients still get them from the second RR	Each RR still learns the client's routes via the other RR
Typical use	Clients always peer with both RRs	Clients that may peer with only one RR, or RRs in different places

Both are correct. Pick one on purpose; Lab 2 shows the difference in Adj-RIB-In.

Lab 1: what the full mesh costs

```
# sessions.py - iBGP sessions: full mesh vs one cluster with k route reflectors
for n in (5, 10, 50, 200, 1000):
    mesh = n * (n - 1) // 2
    row = [f"{n:5} routers  full mesh {mesh:7}"]
    for k in (2, 4):
        rr = k * (n - k) + k * (k - 1) // 2         # every client to every RR, plus an RR m
        row.append(f"{k} RRs {rr:5} (busiest client {k}, each RR {n - 1})")
    print("  ".join(row))
```

```
python3 sessions.py
```

```
    5 routers  full mesh      10  2 RRs     7 (busiest client 2, each RR 4)  4 RRs    10 (bu
   10 routers  full mesh      45  2 RRs    17 (busiest client 2, each RR 9)  4 RRs    30 (bu
   50 routers  full mesh    1225  2 RRs    97 (busiest client 2, each RR 49)  4 RRs   190 (b
  200 routers  full mesh   19900  2 RRs   397 (busiest client 2, each RR 199)  4 RRs   790 (
 1000 routers  full mesh  499500  2 RRs  1997 (busiest client 2, each RR 999)  4 RRs  3990 (
```

The full mesh grows with the square of the number of routers; with two RRs it grows linearly. The busiest **client** keeps one session per RR however large the AS gets. The **RRs** carry the load: each has a session with every other BGP router, so the RRs, not the PEs, are where you size CPU, memory and update-processing capacity. The full mesh hurts operationally before it hurts in numbers: adding one router means touching the config of every other one.

Lab 2: reflection rules in a model you can see all of

The network from the diagram, in plain Python: five routers in AS 65000, OSPF costs on five links, and one prefix, `203.0.113.0/24`, learned over eBGP at `pe1` and at `pe2`. The model computes IGP distances with Dijkstra, applies the RFC 4271 decision steps from "prefer eBGP" on (LOCAL_PREF, AS_PATH, ORIGIN and MED are equal here) with the RFC 4456 §9 changes, and re-advertises until nothing changes. It runs four designs: a full mesh, two RRs sharing a cluster ID, two RRs with their own cluster IDs, and RRs that choose per client (RFC 9107).

```
# rr_model.py - iBGP full mesh vs route reflection (RFC 4456) for one prefix, by brute force
# usage: python3 rr_model.py [mesh] [rr] [rr-two-clusters] [rr-orr]
import heapq, itertools, sys

# IGP (OSPF costs): both RRs sit next to pe1 in site A; pe3 is in site B, next to pe2
LINKS = {("rr1", "pe1"): 1, ("rr2", "pe1"): 1, ("pe1", "pe2"): 10, ("pe1", "pe3"): 10, ("pe2
RID = {"rr1": "10.0.0.1", "rr2": "10.0.0.2", "pe1": "10.0.0.3", "pe2": "10.0.0.4", "pe3": "1
EXITS = {"pe1": "x1", "pe2": "x2"}          # both learn 203.0.113.0/24 over eBGP (AS65101, A
ROUTERS = list(RID)
```

```
def spf(src):                                  # Dijkstra: IGP distance from src to every route
    adj = {r: [] for r in ROUTERS}
    for (a, b), c in LINKS.items():
        adj[a].append((b, c)); adj[b].append((a, c))
    dist, todo = {src: 0}, [(0, src)]
    while todo:
        d, n = heapq.heappop(todo)
        for m, c in adj[n]:
            if d + c < dist.get(m, 1e9):
                dist[m] = d + c; heapq.heappush(todo, (d + c, m))
    return dist
IGP = {r: spf(r) for r in ROUTERS}

def best(router, paths, position=None):
    """RFC 4271 9.1.2.2 from step d) on (LOCAL_PREF, AS_PATH, ORIGIN and MED are equal here)
    d) eBGP over iBGP, e) lowest IGP cost to NEXT_HOP, f) lowest BGP Identifier, where RFC 4
    section 9 uses ORIGINATOR_ID if present, then the RFC 4456 step: shorter CLUSTER_LIST.
    position != router models RFC 9107 optimal route reflection (IGP cost from the client)."
    pos = position or router
    def rank(p):
        ident = p["originator"] or RID[p["sender"]]
        return (p["ebgp"] is False, IGP[pos][p["nh"]], tuple(map(int, ident.split("."))),
                len(p["cluster_list"]))
    return min(paths, key=rank) if paths else None

def run(mode):
    reflectors = set() if mode == "mesh" else {"rr1", "rr2"}
    cluster_id = {"rr1": "10.0.0.1", "rr2": "10.0.0.2" if mode == "rr-two-clusters" else "10
    if reflectors:   # clients peer with every RR; the RRs peer with each other (non-clients
        sessions = {(rr, c) for rr in sorted(reflectors) for c in ROUTERS if c not in reflec
        sessions |= set(itertools.combinations(sorted(reflectors), 2))
    else:
        sessions = set(itertools.combinations(ROUTERS, 2))
    peers = {r: sorted({b for a, b in sessions if a == r} | {a for a, b in sessions if b ==
             for r in ROUTERS}
    rib_in = {r: {} for r in ROUTERS}        # rib_in[router][neighbour] = path learned from
    for pe, x in EXITS.items():               # next-hop-self on the PEs: NH = exit PE loopb
        rib_in[pe][x] = {"nh": pe, "ebgp": True, "originator": None, "cluster_list": [], "se
    loop_drops = set()                        # (receiver, sender) pairs rejected by a loop
    for _ in range(20):                       # re-advertise until nothing changes
        changed = False
        for r, peer in itertools.product(ROUTERS, ROUTERS):
            if peer not in peers[r]:
                continue
            pos = r if (mode == "rr-orr" and peer in reflectors) else None
            b = best(peer, list(rib_in[peer].values()), pos)
            learned_from = [n for n, p in rib_in[peer].items() if p is b][0] if b else None
```

```
            send = b is not None and learned_from != r
            if send and not b["ebgp"]:                      # an iBGP-learned best path
                if peer not in reflectors:
                    send = False                            # RFC 4271: no iBGP -> iBGP re-ad
                else:                                       # RFC 4456 section 6
                    send = learned_from not in reflectors or r not in reflectors
            new = None
            if send:
                new = dict(b, ebgp=False, sender=peer)
                if peer in reflectors and not b["ebgp"]:   # reflected: set/keep ORIGINATOR_
                    new["originator"] = b["originator"] or RID[learned_from]   # prepend clu
                    new["cluster_list"] = [cluster_id[peer]] + b["cluster_list"]
                if new["originator"] == RID[r] or (r in reflectors and cluster_id[r] in new[
                    loop_drops.add((r, peer)); new = None  # RFC 4456 section 8 loop checks
            if rib_in[r].get(peer) != new:
                if new is None: rib_in[r].pop(peer, None)
                else: rib_in[r][peer] = new
                changed = True
        if not changed:
            break
    print(f"== {mode}: {len(sessions)} iBGP sessions, rejected by loop checks: {sorted(loop_
    for r in ROUTERS:
        b = best(r, list(rib_in[r].values()))
        via = [n for n, p in rib_in[r].items() if p is b][0]
        seen = ", ".join(f"{n}:{p['nh']}" + (f"[{' '.join(p['cluster_list'])}]" if p["cluste
                         for n, p in sorted(rib_in[r].items()))
        print(f"  {r}  exit {b['nh']}  IGP {IGP[r][b['nh']]:2}  via {via:3}  | Adj-RIB-In {s

for m in sys.argv[1:] or ["mesh", "rr", "rr-two-clusters", "rr-orr"]:
    run(m)
```

```
python3 rr_model.py
```

```
== mesh: 10 iBGP sessions, rejected by loop checks: 0
  rr1  exit pe1  IGP  1  via pe1  | Adj-RIB-In pe1:pe1, pe2:pe2
  rr2  exit pe1  IGP  1  via pe1  | Adj-RIB-In pe1:pe1, pe2:pe2
  pe1  exit pe1  IGP  0  via x1   | Adj-RIB-In pe2:pe2, x1:pe1
  pe2  exit pe2  IGP  0  via x2   | Adj-RIB-In pe1:pe1, x2:pe2
  pe3  exit pe2  IGP  2  via pe2  | Adj-RIB-In pe1:pe1, pe2:pe2
== rr: 7 iBGP sessions, rejected by loop checks: [('rr1', 'rr2'), ('rr2', 'rr1')]
  rr1  exit pe1  IGP  1  via pe1  | Adj-RIB-In pe1:pe1, pe2:pe2
  rr2  exit pe1  IGP  1  via pe1  | Adj-RIB-In pe1:pe1, pe2:pe2
  pe1  exit pe1  IGP  0  via x1   | Adj-RIB-In x1:pe1
  pe2  exit pe2  IGP  0  via x2   | Adj-RIB-In rr1:pe1[10.0.0.1], rr2:pe1[10.0.0.1], x2:pe2
  pe3  exit pe1  IGP 10  via rr1  | Adj-RIB-In rr1:pe1[10.0.0.1], rr2:pe1[10.0.0.1]
== rr-two-clusters: 7 iBGP sessions, rejected by loop checks: 0
```

```
  rr1  exit pe1  IGP  1  via pe1  | Adj-RIB-In pe1:pe1, pe2:pe2, rr2:pe1[10.0.0.2]
  rr2  exit pe1  IGP  1  via pe1  | Adj-RIB-In pe1:pe1, pe2:pe2, rr1:pe1[10.0.0.1]
  pe1  exit pe1  IGP  0  via x1   | Adj-RIB-In x1:pe1
  pe2  exit pe2  IGP  0  via x2   | Adj-RIB-In rr1:pe1[10.0.0.1], rr2:pe1[10.0.0.2], x2:pe2
  pe3  exit pe1  IGP 10  via rr1  | Adj-RIB-In rr1:pe1[10.0.0.1], rr2:pe1[10.0.0.2]
== rr-orr: 7 iBGP sessions, rejected by loop checks: [('rr1', 'rr2'), ('rr2', 'rr1')]
  rr1  exit pe1  IGP  1  via pe1  | Adj-RIB-In pe1:pe1, pe2:pe2
  rr2  exit pe1  IGP  1  via pe1  | Adj-RIB-In pe1:pe1, pe2:pe2
  pe1  exit pe1  IGP  0  via x1   | Adj-RIB-In x1:pe1
  pe2  exit pe2  IGP  0  via x2   | Adj-RIB-In x2:pe2
  pe3  exit pe2  IGP  2  via rr1  | Adj-RIB-In rr1:pe2[10.0.0.1], rr2:pe2[10.0.0.1]
```

Read it design by design:

- **mesh:** 10 sessions. `pe3` has both exits in its Adj-RIB-In and picks `pe2` at IGP cost 2. This is the reference answer.
- **rr:** 7 sessions, and `pe3` now exits via **`pe1` at IGP cost 10**. Both reflectors are 1 away from `pe1` and 11 away from `pe2`, so both pick `pe1` and reflect only that path. `pe3` never learns the `pe2` exit: that is **path hiding**. Traffic from site B to `203.0.113.0/24` now crosses the core to site A and back. Every reflected copy carries `CLUSTER_LIST 10.0.0.1`, and the RRs reject each other's reflections, `('rr1', 'rr2'), ('rr2', 'rr1')`, because their own cluster ID is already in the list.
- **rr-two-clusters:** same session count, no loop rejections, and each RR now holds a third path, the other RR's reflection (`rr2:pe1[10.0.0.2]` in `rr1`'s Adj-RIB-In). Path hiding is unchanged: more copies of the same best path don't add a different exit.
- **rr-orr:** the reflectors compute the best path from each **client's** IGP position, which is what RFC 9107 optimal route reflection does. `pe3` gets the `pe2` exit and is back to IGP cost 2 with 7 sessions. Note what `pe2` lost: the best path for `pe2` is its own, which isn't sent back to it, so `pe2` now has no backup path through `pe1`. Add-path (RFC 7911) is the other fix: the RR advertises more than one path, so the client gets both exits and decides itself.

The model makes two simplifications, both noted in its code: it doesn't send a route back to the neighbour it was learned from (as real implementations don't), and it doesn't model the final tie-break on peer address, which is why `via rr1` wins over an identical path from `rr2`.

Lab 3: the same network in netlab (FRR)

netlab compiles a high-level topology into containerlab files and per-device configuration. The BGP module builds the iBGP design for you: nodes listed under `rr` become reflectors, every other member of the AS becomes a client of all of them, and the reflectors are meshed with each other (`bgp.rr_mesh`, default true). All reflectors in an AS get the **same** cluster ID by default, the lowest router ID among them (`bgp.rr_cluster_id` overrides it).

```
# topology.yml - two route reflectors next to pe1, a distant pe3, two exits (netlab 26.10, F
---
provider: clab
```

```
defaults.device: frr
module: [ ospf, bgp ]

bgp.as_list:
  65000:
    members: [ rr1, rr2, pe1, pe2, pe3 ]
    rr: [ rr1, rr2 ]              # same cluster ID by default: the lowest RR router ID
  65101:
    members: [ x1 ]
  65102:
    members: [ x2 ]

nodes:                            # order sets the loopbacks: rr1 10.0.0.1 … x2 10.0.0.7
  rr1:
  rr2:
  pe1:
  pe2:
  pe3:
  x1:
    module: [ bgp ]             # external routers run BGP only
    bgp.originate: [ 203.0.113.0/24 ]
  x2:
    module: [ bgp ]
    bgp.originate: [ 203.0.113.0/24 ]

links:
- rr1:
  pe1:
  ospf.cost: 1
- rr2:
  pe1:
  ospf.cost: 1
- pe1:
  pe2:
  ospf.cost: 10
- pe1:
  pe3:
  ospf.cost: 10
- pe2:
  pe3:
  ospf.cost: 2
- pe1-x1
- pe2-x2

validate:
  rr_sessions:
    description: pe3 has Established IPv4 sessions with both route reflectors
```

```
    wait: 60
    nodes: [ pe3 ]
    devices: [ frr ]
    show: bgp summary json
    valid: >-
      {% for n in bgp.neighbors if n.type == 'ibgp' %}
      ipv4Unicast.peers["{{ n.ipv4 }}"].state == "Established" and
      {% endfor %} True
  reflected:
    description: pe3 has 203.0.113.0/24 only as reflected copies of pe1's path
    wait: 30
    nodes: [ pe3 ]
    devices: [ frr ]
    show: bgp ipv4 unicast 203.0.113.0/24 json
    valid: >-
      len(paths) == 2 and
      all(p.originatorId == "{{ nodes.pe1.loopback.ipv4.split('/')[0] }}" for p in paths)
```

A custom configuration template for the add-path fix, deployed later with `netlab config`. netlab picks `frr.j2` from the `addpath/` directory for FRR nodes:

```
{# addpath/frr.j2 - on a route reflector: send every path to iBGP neighbours (RFC 7911) #}
router bgp {{ bgp.as }}
 address-family ipv4 unicast
{% for n in bgp.neighbors if n.type == 'ibgp' and n.ipv4 is defined %}
  neighbor {{ n.ipv4 }} addpath-tx-all-paths
{% endfor %}
 exit-address-family
```

Compile it (run here)

`netlab create` checks the topology, plans addresses, and renders every device's configuration. It needs no root and starts nothing:

```
python3 -m venv netlab-venv && netlab-venv/bin/pip install networklab==26.10
mkdir lab && cp -r topology.yml addpath lab/ && cd lab
../netlab-venv/bin/netlab create
```

```
[CONFIG]  rr1: initial,ospf,bgp,daemons,hosts
[CONFIG]  rr2: initial,ospf,bgp,daemons,hosts (shared)
[CONFIG]  pe1: initial,ospf,bgp,daemons,hosts (shared)
[CONFIG]  pe2: initial,ospf,bgp,daemons,hosts (shared)
[CONFIG]  pe3: initial,ospf,bgp,daemons,hosts (shared)
[CONFIG]  x1: initial,routing,bgp,daemons,hosts (shared)
[CONFIG]  x2: initial,routing,bgp,daemons,hosts (shared)
```

```
Created provider configuration file: clab.yml
Created pickled transformed topology data into netlab.snapshot.pickle
Created group_vars for all
Created group_vars for modules
Created group_vars for frr
Created host_vars for rr1, rr2, pe1, pe2, pe3, x1, x2
Created minimized Ansible inventory hosts.yml
Created Ansible configuration file: ansible.cfg
```

The rendered FRR configuration for `rr1`, filtered to the lines that make it a reflector:

```
grep -E 'router bgp|router-id|cluster-id|neighbor 10\.0\.0\.[0-9]+ (remote-as|route-reflecto
```

```
router bgp 65000
  bgp router-id 10.0.0.1
  bgp cluster-id 10.0.0.1
  neighbor 10.0.0.2 remote-as 65000
  neighbor 10.0.0.3 remote-as 65000
  neighbor 10.0.0.4 remote-as 65000
  neighbor 10.0.0.5 remote-as 65000
  neighbor 10.0.0.3 route-reflector-client
  neighbor 10.0.0.4 route-reflector-client
  neighbor 10.0.0.5 route-reflector-client
```

`rr2`'s file has `bgp router-id 10.0.0.2` but the same `bgp cluster-id 10.0.0.1`. `pe3`'s has only two neighbours, `10.0.0.1` and `10.0.0.2`, and no reflector lines at all: being a client needs no client configuration. The generated `clab.yml` uses the `quay.io/frrouting/frr:10.7.1` image. A first draft of this topology failed `netlab create` with "Validation test rr_sessions does not include any valid device": tests whose `show`/`valid` are plain strings need `devices: [ frr ]`.

Start it and verify (not run here)

The next steps need root (containerlab creates network namespaces and veth pairs) and a container runtime, so they were not run on this box. They are written from the netlab and FRR documentation; the expected results come from the model in Lab 2 and the rendered configuration above.

```
netlab up                   # containerlab deploy + push the rendered configs (needs root)
netlab validate             # runs the two tests in topology.yml
netlab connect pe3 vtysh -c "show bgp ipv4 unicast 203.0.113.0/24"
```

What to look for:

1. `netlab validate` should pass both tests: `pe3` has two Established sessions, and two paths for `203.0.113.0/24`, both with `originatorId` `10.0.0.3` (`pe1`).

2. In the `show bgp` output on `pe3`, each path should show `Originator: 10.0.0.3, Cluster list: 10.0.0.1` (FRR prints this line for reflected routes), and the next hop should be `10.0.0.3`, not the reflector's address.
3. `show bgp ipv4 unicast 203.0.113.0/24` on `rr1` should list both the `pe1` and the `pe2` paths, with `pe1` best.
4. Apply the fix and check again:

```
netlab config addpath -l rr1,rr2    # renders addpath/frr.j2 per node and pushes it
netlab connect pe3 vtysh -c "show bgp ipv4 unicast 203.0.113.0/24"
netlab validate                     # 'reflected' should now FAIL: pe3 sees pe2's path too
```

With `addpath-tx-all-paths` on the reflectors, `pe3` should receive both exits from each RR and pick the `pe2` path (originator `10.0.0.4`) on IGP cost, as the full mesh did. Failing the `reflected` test is the expected result: it was written to prove path hiding. `netlab down --cleanup` removes the lab.

Case: placing reflectors

Situation	Design
Small AS (a handful of routers), stable	Full mesh is fine. Each new router is one more session on every other router
PoPs with a core, RRs in the forwarding path at the PoP/core boundary	One cluster per PoP, the PoP's routers as clients. RFC 4456 §11's conditions keep results equal to a full mesh
Control-plane-only RRs in one or two locations (common for Internet and VPN routes)	Expect path hiding for distant clients. Turn on add-path towards clients, or optimal route reflection (RFC 9107) where your implementations support it, and check memory on the RRs
Clients that may lose one RR session (single-homed sites)	Separate cluster IDs per RR, so each RR still learns those clients' routes via the other
EVPN / VPN overlays with RRs	Same rules, per address family. Next hops stay the PE loopbacks, so loopback reachability is the first thing to check

The trap

Trap: "the RR is just a hub, it doesn't change routing." It changes which exits the clients can see. In Lab 2 the only difference between `mesh` and `rr` is the session design, and `pe3`'s exit cost went from 2 to 10. Nothing is broken, no alarm fires, and traceroutes from site B quietly cross the core.

Trap: "add more RRs for more paths." Two reflectors at the same place pick the same best path. Lab 2's `rr-two-clusters` adds copies of the same path, not a different exit.

Trap: "set next-hop-self on the RR." RFC 4456 §10 says a reflector should not change NEXT_HOP. With `next-hop-self force` on an out-of-path reflector, every client sends the traffic **to the reflector**, which then needs a forwarding path and capacity it was never sized for.

Trap: "clients don't need to be configured, so they can't be misconfigured." Clients still need IGP reachability to every exit's loopback (the NEXT_HOP), and sessions to every RR. A client that peers with only one RR, in a cluster where both RRs share an ID, has a single point of failure that the "two RRs" diagram hides.

Trap: "ORIGINATOR_ID and CLUSTER_LIST are just diagnostics." They are in the decision process (§9). Filtering them, or a box that strips them, removes loop protection and changes tie-breaks.

The boring rule

Use a full mesh while it is small enough to configure without thinking; after that, use two route reflectors per cluster, with every client peered to both. Leave NEXT_HOP alone on reflected routes. Decide whether the RRs share a cluster ID and write the reason down. If the reflectors don't sit where the clients' exits are, assume path hiding until `show bgp` on a distant client proves otherwise, and fix it with add-path or optimal route reflection, not with more reflectors in the same place.

Try this

1. In `rr_model.py`, move `rr2` next to `pe3` (replace `("rr2", "pe1"): 1` with `("rr2", "pe3"): 1`). Does `rr` mode still hide the `pe2` exit from `pe3`? Which of RFC 4456 §11's conditions did you just satisfy?
2. Add a sixth router `pe4` far from both exits. How many sessions do `mesh` and `rr` use now, and which exit does `pe4` pick in each?
3. In `sessions.py`, add a column for a two-level hierarchy: 4 top-level RRs, 20 second-level RRs as their clients, and 50 PEs per second-level RR pair. Count the sessions.
4. In the netlab topology, set `bgp.rr_cluster_id` on `rr2` to `10.0.0.2`, run `netlab create`, and confirm the rendered `cluster-id` changes. If you can run the lab, compare `show bgp ipv4 unicast 203.0.113.0/24` on `rr1` before and after.
5. Replace `addpath-tx-all-paths` with `addpath-tx-bestpath-per-AS` in `addpath/frr.j2`. Why does it give `pe3` the same two exits in this topology, and with fewer paths in general?

Next

- Policy and Filters
- BFD: Fast Failure Detection

Verification

- **Run here:** `sessions.py` and `rr_model.py` with Python **3.13.5** on Debian 13 (Linux 6.12.94+), unprivileged. Output blocks are pasted from those runs; the model is deterministic.
- **Run here:** `netlab create` with netlab **26.10** (`networklab` from PyPI, installed in a Python venv under the working directory; no system packages). It compiled the topology, rendered the FRR configuration shown above, and wrote `clab.yml` with `quay.io/frrouting/frr:10.7.1`. The `addpath/frr.j2` template was also rendered by `netlab create` with `config: [ addpath ]` on both RRs; it produced `neighbor 10.0.0.2 … 10.0.0.5 addpath-tx-all-paths` under `address-family ipv4 unicast` for `rr1`.
- **Not run:** `netlab up`, `netlab validate`, `netlab connect`, `netlab config` and `netlab down`. They need root and a container runtime (containerlab), which this run does not use. The expected results are stated as expectations, not as output. The FRR JSON keys used in the validation test (`paths`, `originatorId`) and the `Originator: …, Cluster list: …` text line were checked in FRR's `bgpd/bgp_route.c`.
- Current versions on 2026-10-19: netlab 26.10, FRR 10.7.1 (latest release), containerlab 0.81.1.
- No `sudo`, no system packages, no containers started.

Sources

- RFC 4456 — BGP Route Reflection: An Alternative to Full Mesh Internal BGP (IBGP): §5 terminology and concepts, §6 operation, §7 redundant RRs, §8 ORIGINATOR_ID and CLUSTER_LIST, §9 route selection, §10 attributes not to modify, §11 deployment
- RFC 4271 — A Border Gateway Protocol 4 (BGP-4): §9.1.2.2 breaking ties, §9.2 update-send process (iBGP split horizon)
- RFC 7911 — Advertisement of Multiple Paths in BGP
- RFC 9107 — BGP Optimal Route Reflection (BGP ORR)
- netlab — BGP configuration module (`bgp.as_list`, `rr`, `bgp.rr_cluster_id`, `bgp.rr_mesh`, `bgp.originate`), OSPF module (`ospf.cost`), Lab validation tests, `netlab config`
- containerlab — documentation (the provider netlab drives with `provider: clab`)
- FRR — BGP user guide (route reflection, `next-hop-self force`, `addpath-tx-all-paths`, `addpath-tx-bestpath-per-AS`); `bgpd/bgp_route.c` for the JSON and text output of reflected paths: https://github.com/FRRouting/frr/blob/master/bgpd/bgp_route.c

IPv4 Routes over IPv6 Next Hops: BGP Extended Next Hop (RFC 8950)

IPv4 Routes over IPv6 Next Hops: BGP Extended Next Hop (RFC 8950)

A BGP route says two things: *where* (the prefix) and *through whom* (the next hop). For years the address family of the prefix fixed the family of the next hop: IPv4 prefixes got IPv4 next hops. RFC 8950 (which replaced RFC 5549 in 2020) breaks that link. An IPv4 prefix can carry an **IPv6 next hop**, so a link, or a whole core, can run with IPv6 addresses only and still forward IPv4. This is what makes "BGP unnumbered" fabrics work. Two routers peer over IPv6 link-local addresses and swap IPv4 routes, and nobody hands out a /31 per link. This chapter encodes the two wire pieces by hand: the **Extended Next Hop Encoding capability** and an **MP_REACH_NLRI** with a 32-octet next hop. It then builds the forwarding result in a Linux network namespace, an IPv4 route `via inet6`, and shows that the kernel finds the next hop's MAC address with IPv6 neighbour discovery, not ARP. Last comes the FRR configuration for a two-router containerlab lab.

Learning goals

- Say why a next hop's address family doesn't have to match the prefix's
- Encode and decode capability code 5 and its `<NLRI AFI, NLRI SAFI, Nexthop AFI>` triples
- Read an MP_REACH_NLRI and use the **next-hop length** to decide whether the next hop is IPv4, IPv6 or VPN-IPv6
- Know what changed from RFC 5549: VPN next hops are 24 or 48 octets with a zero RD, and IPv4 multicast VPN (1/129) is allowed
- Install and inspect an IPv4 route with an IPv6 gateway on Linux, and see neighbour discovery resolve it
- Configure FRR for extended next hop, on both interface (link-local) peers and global-address peers

Thesis

A next hop is not a destination. It is a **neighbour to hand the frame to**. A router forwarding an IPv4 packet needs only a layer-2 address for that neighbour, and IPv6 neighbour discovery gives one as well as ARP does. RFC 8950 is the BGP half of that idea: a way to *say* "IPv6 next hop" for IPv4 routes, and a capability that makes sure the peer understands it before you say it.

Mental model

Keep three layers apart:

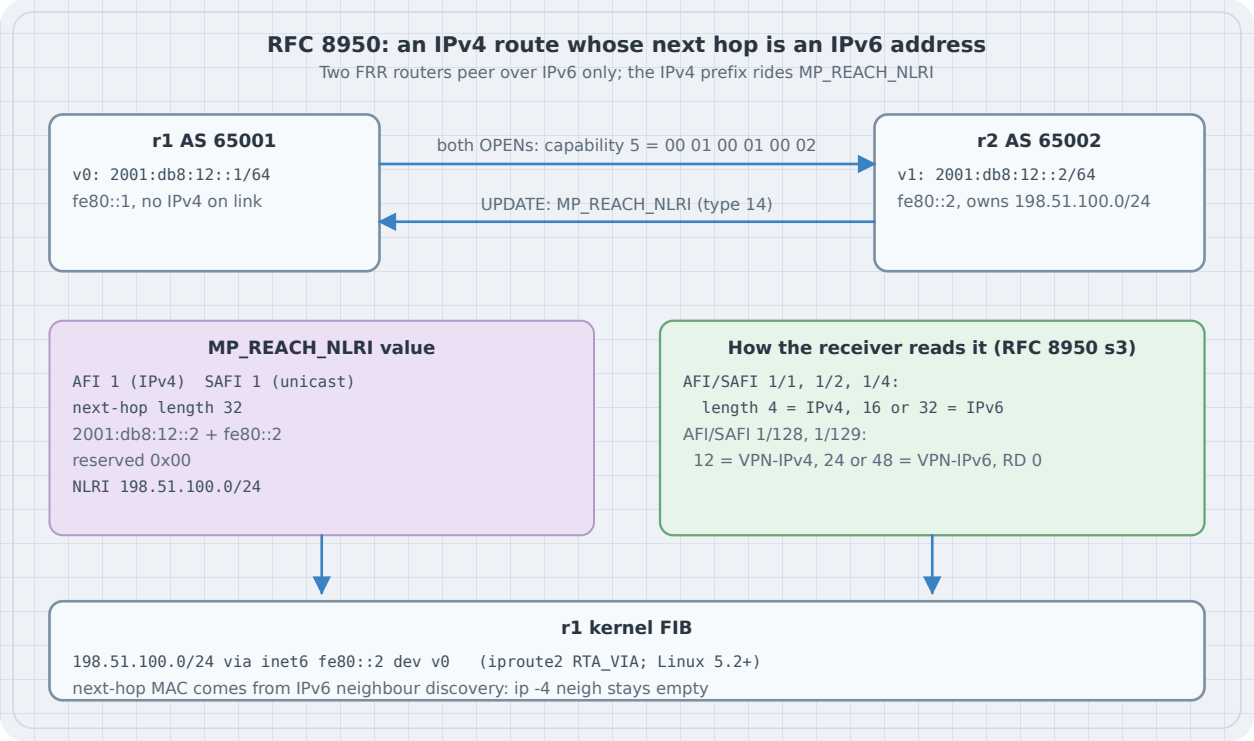


Figure 1: RFC 8950 on one link: the capability in OPEN, the MP_REACH_NLRI layout, how the receiver decides the next-hop family, and the IPv4 route the kernel installs

1. **Negotiation.** In OPEN, each speaker lists the NLRI families for which it accepts an IPv6 next hop. No capability, no IPv6 next hop: RFC 8950 §4 says a speaker MUST NOT send one until the peer has advertised support for that AFI/SAFI.
2. **Encoding.** The IPv4 route moves out of the classic IPv4-only UPDATE fields into the multiprotocol attribute MP_REACH_NLRI (RFC 4760). That attribute has its own next-hop field with an explicit length.
3. **Forwarding.** The RIB entry `198.51.100.0/24 via fe80::2` needs a data plane that can resolve an IPv6 gateway for an IPv4 route. On Linux, that is the `RTA_VIA` route attribute.

Concepts

Why the classic NEXT_HOP can't do it

The original BGP-4 UPDATE (RFC 4271) carries IPv4 prefixes in its NLRI field and the next hop in the NEXT_HOP path attribute, type 3. That attribute is exactly four octets: an IPv4 address. There is no room for 16 octets. Multiprotocol BGP (RFC 4760) added MP_REACH_NLRI, type 14. It carries `AFI, SAFI, length of next hop, next hop, reserved octet, NLRI`, so any family can say any next hop as long as the receiver knows how to read the length. RFC 8950 uses MP_REACH_NLRI with AFI 1 (IPv4) and says which lengths are legal.

The length rule (RFC 8950 §3)

The receiver decides the next hop's family from the **Length of Next Hop Address** field:

NLRI AFI/SAFI	Length	Next hop is
1/1 unicast, 1/2 multicast, 1/4 labeled	4	IPv4 (the old case)
1/1, 1/2, 1/4	16	IPv6 global
1/1, 1/2, 1/4	32	IPv6 global, then IPv6 link-local
1/128 VPN, 1/129 VPN multicast	12	VPN-IPv4 (8-octet RD + IPv4)
1/128, 1/129	24	VPN-IPv6, RD set to zero
1/128, 1/129	48	VPN-IPv6 global + VPN-IPv6 link-local, RDs zero

The 32-octet form follows RFC 2545 §3, the same rule IPv6 BGP uses: global address first, link-local second. A router that shares the link uses the link-local address to forward.

What changed from RFC 5549

RFC 8950 §2 lists two changes. First, for VPN-IPv4 (1/128), RFC 5549 had said the next hop was a plain 16- or 32-octet IPv6 address. Deployed code actually sent a **VPN-IPv6** address, 24 or 48 octets with a zero route distinguisher, matching how VPNv4 over IPv4 and VPNv6 over IPv6 already worked (erratum 5253). RFC 8950 writes down what the implementations did. Second, IPv4 multicast VPN (1/129) gets the same treatment. Plain IPv4 unicast didn't change, so a 5549-era unicast speaker and an 8950 speaker interoperate.

The capability (RFC 8950 §4)

Capability code **5**, *Extended Next Hop Encoding*, rides in the OPEN message's Capabilities optional parameter (RFC 5492). Its value is a list of 6-octet triples:

```
+--------------------+---------------------+-----------------------+
| NLRI AFI (2 octets)| NLRI SAFI (2 octets)| Nexthop AFI (2 octets)|  ...repeated
+--------------------+---------------------+-----------------------+
```

RFC 8950 allows only NLRI AFI 1, NLRI SAFI 1, 2, 4, 128 or 129, and Nexthop AFI 2. The capability doesn't *enable* an address family. That is still the job of the Multiprotocol Extensions capability (code 1). It only says how a family that is already enabled may encode its next hop. It isn't meant for families that already allow both next-hop types. IPv6 prefixes with IPv4 next hops, for instance, have their own older mechanism (RFC 4798, RFC 4659).

Route reflectors and mixed peers (RFC 8950 §5)

A route reflector passes the next hop on unchanged, and RFC 8950 says its encoding MUST NOT change. If one client can't take IPv6 next hops (it didn't send capability 5 for that family), the reflector **can't send it that route at all**. It doesn't translate. In practice, every client of a reflector that carries RFC 8950 routes needs the capability.

Lab 1: the bytes, by hand (run here)

`enh.py` builds the capability and an MP_REACH_NLRI for two IPv4 prefixes with a global + link-local IPv6 next hop. It then decodes them the way a receiver would, using only the length to pick the next-hop family. It is plain Python 3, standard library only.

```
"""Encode and decode the RFC 8950 pieces of a BGP session by hand."""
import ipaddress, struct

AFI = {1: "IPv4", 2: "IPv6"}
SAFI = {1: "unicast", 2: "multicast", 4: "labeled", 128: "VPN", 129: "VPN multicast"}

def ext_nh_capability(triples):
    """Capability code 5: a list of <NLRI AFI, NLRI SAFI, Nexthop AFI>, 2 octets each."""
    value = b"".join(struct.pack("!HHH", a, s, n) for a, s, n in triples)
    return struct.pack("!BB", 5, len(value)) + value

def decode_capability(blob):
    code, length = blob[0], blob[1]
    assert code == 5, code
    out = []
    for i in range(2, 2 + length, 6):
        a, s, n = struct.unpack("!HHH", blob[i:i + 6])
        out.append(f"NLRI {AFI[a]}/{SAFI[s]} may use an {AFI[n]} next hop")
    return out

def mp_reach_ipv4_unicast(next_hops, prefixes):
    """MP_REACH_NLRI (type 14) for AFI 1 / SAFI 1 with one or two IPv6 next hops."""
    nh = b"".join(ipaddress.IPv6Address(h).packed for h in next_hops)
    nlri = b""
    for p in prefixes:
        net = ipaddress.IPv4Network(p)
        nbytes = (net.prefixlen + 7) // 8
        nlri += bytes([net.prefixlen]) + net.network_address.packed[:nbytes]
    value = struct.pack("!HBB", 1, 1, len(nh)) + nh + b"\x00" + nlri
    # flags 0x80 = optional, non-transitive; type code 14
    return struct.pack("!BBB", 0x80, 14, len(value)) + value

def decode_mp_reach(attr):
    flags, typ, length = attr[0], attr[1], attr[2]
    v = attr[3:3 + length]
    afi, safi, nhlen = struct.unpack("!HBB", v[:4])
    nh = v[4:4 + nhlen]
    pos = 4 + nhlen + 1          # skip the reserved octet
    prefixes = []
    while pos < len(v):
        plen = v[pos]; nbytes = (plen + 7) // 8
```

```
        raw = v[pos + 1:pos + 1 + nbytes] + b"\x00" * (4 - nbytes)
        prefixes.append(f"{ipaddress.IPv4Address(raw)}/{plen}")
        pos += 1 + nbytes
    # RFC 8950 section 3: the length alone says which protocol the next hop is
    kinds = {4: "IPv4", 16: "IPv6 global", 32: "IPv6 global + link-local",
             12: "VPN-IPv4", 24: "VPN-IPv6 (RD 0)", 48: "VPN-IPv6 global + link-local"}
    hops = []
    if nhlen in (16, 32):
        hops = [str(ipaddress.IPv6Address(nh[i:i + 16])) for i in range(0, nhlen, 16)]
    return {"afi/safi": f"{afi}/{safi}", "nh length": nhlen,
            "nh kind": kinds.get(nhlen, "invalid for this AFI/SAFI"),
            "next hops": hops, "nlri": prefixes}

if __name__ == "__main__":
    cap = ext_nh_capability([(1, 1, 2), (1, 128, 2)])
    print("capability bytes:", cap.hex(" "))
    for line in decode_capability(cap):
        print("  ", line)

    attr = mp_reach_ipv4_unicast(["2001:db8:12::2", "fe80::2"],
                                 ["198.51.100.0/24", "203.0.113.128/25"])
    print("MP_REACH_NLRI bytes:", attr.hex(" "))
    for k, val in decode_mp_reach(attr).items():
        print(f"   {k:10} {val}")

    single = mp_reach_ipv4_unicast(["2001:db8:12::2"], ["10.0.0.0/8"])
    d = decode_mp_reach(single)
    print("global-only next hop:", d["nh length"], "octets ->", d["nh kind"], d["next hops"]
```

```
$ python3 enh.py
capability bytes: 05 0c 00 01 00 01 00 02 00 01 00 80 00 02
   NLRI IPv4/unicast may use an IPv6 next hop
   NLRI IPv4/VPN may use an IPv6 next hop
MP_REACH_NLRI bytes: 80 0e 2e 00 01 01 20 20 01 0d b8 00 12 00 00 00 00 00 00 00 00 00 02 fe
   afi/safi   1/1
   nh length  32
   nh kind    IPv6 global + link-local
   next hops  ['2001:db8:12::2', 'fe80::2']
   nlri       ['198.51.100.0/24', '203.0.113.128/25']
global-only next hop: 16 octets -> IPv6 global ['2001:db8:12::2']
```

Read the capability bytes left to right: `05` is the code, `0c` the length (12 octets, two triples), then `00 01 00 01 00 02` (IPv4 unicast may use an IPv6 next hop) and `00 01 00 80 00 02` (VPN-IPv4, SAFI 128 = `0x80`). In the attribute, `80 0e 2e` is flags (optional, non-transitive), type 14, and a value length of 46. Then come AFI `00 01` and SAFI `01`, and **`20`, the next-hop length: 32**. After it sit the two 16-octet addresses, the reserved `00`, and the NLRI: `18 c6 33 64` is /24 198.51.100 and `19 cb 00 71 80` is /25 203.0.113.128. Nothing in the attribute says "IPv6" outright. The length

of 32 is the whole signal.

Lab 2: an IPv4 route with an IPv6 gateway (run here, no sudo)

This is what bgpd hands the kernel once it accepts such a route. `unshare -rn` gives an unprivileged user a private network namespace in which they're root, so the lab touches nothing on the host. A veth pair stands in for the link, and **v0 gets no IPv4 address at all**. (A `dummy` interface would be simpler, but loading its kernel module needs real root, so a veth pair it is.) `nh.py` sends one UDP datagram to 198.51.100.7 and listens on the far end, v1, for whatever v0 sends to find the next hop.

```
#!/bin/sh
# Run as: unshare -rn sh lab.sh   (a private network namespace, no sudo)
ip link add v0 type veth peer name v1
ip link set lo up; ip link set v0 up; ip link set v1 up
ip addr add 192.0.2.1/32 dev lo            # a source address for our own packets
ip -6 addr add 2001:db8:12::1/64 dev v0 nodad
ip route add 198.51.100.0/24 via inet6 2001:db8:12::2 dev v0
echo "--- ipv4 routes";  ip -4 route show
echo "--- route lookup"; ip -4 route get 198.51.100.7
echo "--- ipv4 addresses on v0"; ip -4 addr show dev v0
sleep 1
python3 nh.py                              # send one UDP datagram, watch v1
echo "--- ipv6 neighbours on v0"; ip -6 neigh show dev v0
echo "--- ipv4 neighbours on v0"; ip -4 neigh show dev v0
```

```
"""Send one IPv4 datagram to 198.51.100.7 and report how v0 resolves the next hop."""
import socket, struct

cap = socket.socket(socket.AF_PACKET, socket.SOCK_RAW, socket.htons(0x0003))  # all frames
cap.bind(("v1", 0))          # the far end of the veth pair sees what v0 transmits
cap.settimeout(3)

socket.socket(socket.AF_INET, socket.SOCK_DGRAM).sendto(b"hello", ("198.51.100.7", 9))

while True:
    try:
        f = cap.recv(2048)
    except socket.timeout:
        print("nothing seen"); break
    src, ethertype = f[6:12], struct.unpack("!H", f[12:14])[0]
    if ethertype == 0x0806:
        print(f"{src.hex(':')} sent an ARP request"); break
    # IPv6 (0x86dd), next header 58 = ICMPv6, ICMPv6 type 135 = neighbour solicitation
    if ethertype == 0x86dd and len(f) >= 78 and f[20] == 58 and f[54] == 135:
```

```
        target = socket.inet_ntop(socket.AF_INET6, f[62:78])
        if not target.startswith("fe80"):          # skip the link-local DAD noise
            print(f"{src.hex(':')} -> {f[:6].hex(':')} ICMPv6 neighbour solicitation"
                  f" for {target}")
            break
```

```
$ unshare -rn sh lab.sh
--- ipv4 routes
198.51.100.0/24 via inet6 2001:db8:12::2 dev v0
--- route lookup
198.51.100.7 via inet6 2001:db8:12::2 dev v0 src 192.0.2.1 uid 0
    cache
--- ipv4 addresses on v0
ce:c8:07:ce:15:7e -> 33:33:ff:00:00:02 ICMPv6 neighbour solicitation for 2001:db8:12::2
--- ipv6 neighbours on v0
2001:db8:12::2 INCOMPLETE
--- ipv4 neighbours on v0
```

Three things to notice:

- `ip route` takes `via inet6 <addr>` on an IPv4 route. iproute2 sends this as the `RTA_VIA` attribute, and Linux has accepted IPv6 gateways for IPv4 routes since kernel 5.2. This run used kernel 6.12 with iproute2 6.15.
- v0 has **no IPv4 address**, and the lookup still works. The source address, 192.0.2.1, comes from the loopback, the way a router uses its loopback /32 in an unnumbered fabric.
- To send the IPv4 datagram, the kernel sent an **ICMPv6 neighbour solicitation** to the solicited-node multicast group of 2001:db8:12::2 (`33:33:ff:00:00:02`). The IPv4 neighbour table stays empty, because no ARP is involved. (`INCOMPLETE` is expected: nobody owns ::2 on this veth.)

In a real fabric, bgpd installs the link-local form, `via inet6 fe80::2 dev eth1`. A link-local gateway is only meaningful together with the interface, which is why the route always names the device.

Lab 3: two FRR routers in containerlab (from the docs, not run here)

Running containers needs a container runtime and root, which this book's build box doesn't have. The topology below follows the containerlab topology-file format, and the FRR commands come from the FRR BGP documentation. Run it yourself and compare against the checks below.

`rfc8950.clab.yml`:

```
name: rfc8950
topology:
  nodes:
    r1:
```

```
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - r1/daemons:/etc/frr/daemons
        - r1/frr.conf:/etc/frr/frr.conf
    r2:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - r2/daemons:/etc/frr/daemons
        - r2/frr.conf:/etc/frr/frr.conf
  links:
    - endpoints: ["r1:eth1", "r2:eth1"]
```

`r1/daemons` sets `bgpd=yes` (zebra always runs). `r1/frr.conf`, an **interface peer** with no addresses on eth1 at all:

```
frr defaults datacenter
!
interface lo
 ip address 192.0.2.1/32
!
router bgp 65001
 bgp router-id 192.0.2.1
 no bgp ebgp-requires-policy
 neighbor eth1 interface remote-as external
 !
 address-family ipv4 unicast
  network 192.0.2.1/32
 exit-address-family
!
```

`r2` is the same with AS 65002, router ID and loopback 198.51.100.1, and `network 198.51.100.0/24` backed by a static `ip route 198.51.100.0/24 blackhole`.

What FRR does for you here, per its documentation:

- `neighbor eth1 interface` peers over the IPv6 **link-local** address it learns from the other side's router advertisements. bgpd asks zebra to send RAs on that interface (the `bgp ipv6-auto-ra` behaviour, on by default).
- On a link-local peering, FRR turns **`capability extended-nexthop` on automatically**. For a peer reached by a *global* IPv6 address you set it yourself: `neighbor 2001:db8:12::2 capability extended-nexthop`. That's what lets bgpd install IPv4 routes with IPv6 next hops when the interfaces have no IPv4.

Checks to run after `containerlab deploy -t rfc8950.clab.yml`:

```
docker exec clab-rfc8950-r1 vtysh -c "show bgp neighbors eth1"   # look for the Extended nex
docker exec clab-rfc8950-r1 vtysh -c "show ip route bgp"         # 198.51.100.0/24 via fe80:
docker exec clab-rfc8950-r1 ip -4 route show proto bgp           # ... via inet6 fe80::... d
docker exec clab-rfc8950-r1 ip -4 neigh show dev eth1            # empty: no ARP on this lin
```

If the neighbour output shows the capability **advertised but not received**, the peer won't accept IPv6 next hops for IPv4, and per RFC 8950 §4 your router must not send them. On a global-address peer, the usual cause is a missing `capability extended-nexthop` on the far side.

Case: an unnumbered leaf-spine

A leaf-spine fabric with 64 leaves and 4 spines has 256 links. Numbered with IPv4 /31s, that's 256 /31s to allocate, document and keep consistent in two configs per link. With RFC 8950 and interface peering, each switch needs one IPv4 /32 on its loopback (for sourcing and management) and nothing on its links: IPv6 link-local addresses come up on their own. The price is a hard dependency on IPv6 being enabled on every fabric port. You also have to watch three things:

- **Traceroute** replies come from the loopback (or whatever the vendor picks), because the link has no IPv4 address to answer from.
- **Out-of-band tools** that assume an IPv4 next hop in BGP (old collectors, some BMP pipelines) may print the IPv6 next hop oddly, or drop the route.
- **Every reflector client** needs the capability (RFC 8950 §5), so one old box can quietly miss routes.

The trap

"The session is up, so the routes will be fine." Capability 5 is negotiated per family, and only if *both* sides advertise it. A global-address IPv6 session with IPv4 unicast activated, but no `capability extended-nexthop` on one end, comes up fine and exchanges IPv6 routes. The IPv4 routes then fail to install, or are never sent. Check the capability line in the neighbour output, not just `Established`.

Second trap: the 32-octet next hop is global *then* link-local. A hand-built decoder, or a filter that reads only the first 16 octets, gets the global address. That's fine for a multihop peer, but on a shared link the router actually forwards to the link-local one.

The boring rule

Peer over interfaces where you can, so the capability is on automatically. Turn it on explicitly for global-address IPv6 peers that carry IPv4. Then verify three things: capability received, the IPv4 route installed `via inet6`, and the IPv4 neighbour table empty on the link.

Try this

1. Change `mp_reach_ipv4_unicast` in `enh.py` to send a 24-octet next hop for AFI/SAFI 1/1. What does `decode_mp_reach` call it? What should a real receiver do with it? (RFC 8950 §3 and RFC 7606 on malformed attributes.)
2. Add a third triple, `(2, 1, 1)`: IPv6 unicast with an IPv4 next hop. Why does RFC 8950 §4 rule it out?
3. In Lab 2, add `2001:db8:12::2/64` on v1 and re-run. What changes in `ip -6 neigh`, and why does `nh.py` now see a different frame?
4. Re-run Lab 2 with `fe80::2` as the gateway. Why does `ip route add 198.51.100.0/24 via inet6 fe80::2` fail without `dev v0`?

Next

Use this with route reflectors (RFC 8950 §5): a reflector won't translate next-hop encodings for a client that lacks the capability.

Verification

- `enh.py` was run with Python 3 (standard library only). The capability and MP_REACH_NLRI bytes and decodes shown are its real output.
- Lab 2 (`lab.sh` + `nh.py`) was run unprivileged with `unshare -rn` on Linux 6.12 with iproute2 6.15. The routes, the lookup, the neighbour-solicitation capture and the neighbour tables are genuine output. MAC addresses are random per run.
- Lab 3 (containerlab + FRR) was **not run**: it needs a container runtime and root. It is written from the containerlab and FRR documentation, and the expected output is described, not pasted.

Sources

- RFC 8950, *Advertising IPv4 Network Layer Reachability Information (NLRI) with an IPv6 Next Hop*: https://www.rfc-editor.org/rfc/rfc8950. §2 changes from RFC 5549, §3 next-hop lengths, §4 capability, §5 operations.
- RFC 4760, *Multiprotocol Extensions for BGP-4* (MP_REACH_NLRI): https://www.rfc-editor.org/rfc/rfc4760
- RFC 5492, *Capabilities Advertisement with BGP-4*: https://www.rfc-editor.org/rfc/rfc5492
- RFC 2545, *Use of BGP-4 Multiprotocol Extensions for IPv6 Inter-Domain Routing* (global + link-local next hop): https://www.rfc-editor.org/rfc/rfc2545
- RFC 4271, *A Border Gateway Protocol 4* (NEXT_HOP attribute): https://www.rfc-editor.org/rfc/rfc4271
- IANA, BGP Capability Codes (code 5): https://www.iana.org/assignments/capability-codes/
- FRR documentation, BGP (`capability extended-nexthop`, interface peers, `bgp ipv6-auto-ra`): https://docs.frrouting.org/en/latest/bgp.html

- containerlab documentation, topology definition: https://containerlab.dev/manual/topo-def-file/
- iproute2 `ip-route(8)` (`via [ FAMILY ] ADDRESS`): https://man7.org/linux/man-pages/man8/ip-route.8.html

Ops and Automation

Observability for Networks

Observability for Networks

If you cannot see it, you cannot operate it. This chapter turns lab verification habits into a minimal **observability stack**: interface signals, routing state, captures, logs, and a path toward metrics/telemetry—without requiring paid NMS.

Learning goals

By the end of this chapter you can:

- Build a layered observability checklist for any lab or small network
- Use Linux and FRR show/counters for incident triage
- Capture and filter packets productively
- Sketch metrics that matter (link, session, error)
- Write a short runbook entry from an observed failure

Observability layers

Layer	Questions	Tools
L1/L2 link	Up? errors? flaps?	`ip -s link`, ethtool, MAC/FDB
L3 reachability	Path? loss?	ping, mtr, traceroute
Control plane	Neighbors/sessions?	vtysh shows, logs
Data plane path	Which hop?	route get, capture
Services	App timeouts?	ss, curl, synthetic checks
Continuity	When did it start?	logs, metrics, journals

Baseline: what "healthy" looks like

Before incidents, save baselines:

```
mkdir -p baseline
docker exec clab-ospf-tri-r1 vtysh -c 'show ip ospf neighbor' > baseline/r1-ospf-nbr.txt
```

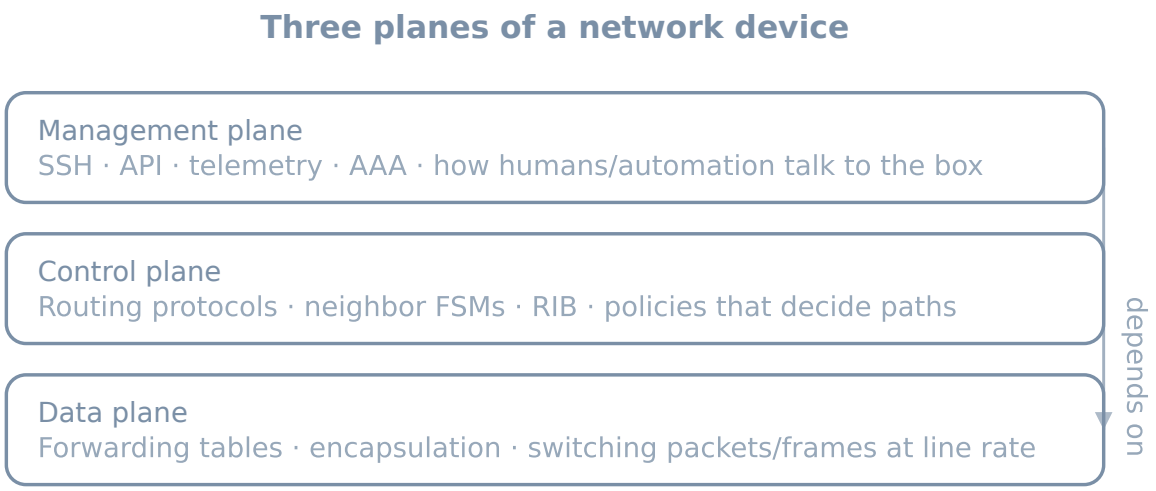


Figure 1: Planes

```
docker exec clab-ospf-tri-r1 vtysh -c 'show ip route' > baseline/r1-route.txt
docker exec clab-ospf-tri-r1 ip -s link > baseline/r1-link.txt
```

Diff during incidents:

```
diff -u baseline/r1-ospf-nbr.txt <(docker exec clab-ospf-tri-r1 vtysh -c 'show ip ospf neigh
```

Interface and error signals

```
ip -s -s link show eth1
# Look at RX/TX errors, drops, carrier changes
```

Drill: bounce a link; confirm counters and oper state track reality.

```
ip monitor link
```

Control-plane signals

OSPF

```
vtysh -c 'show ip ospf neighbor'
vtysh -c 'show ip ospf interface'
vtysh -c 'show log'   # if logging configured
```

BGP

```
vtysh -c 'show bgp summary'
vtysh -c 'show bgp neighbors 10.0.0.2'
vtysh -c 'show bgp neighbors 10.0.0.2 routes'
```

Watch for: session flaps, prefix count drops to zero, unexpected AS paths.

Path quality

```
ping -c 50 -i 0.2 192.168.2.10
mtr -rwzc 100 192.168.2.10
traceroute -n 192.168.2.10
```

Interpret:

Pattern	Suspect
Loss from first hop	Local segment
Loss starting mid-path	Transit / congestion / ACL
High latency one hop jump	That hop or link serialization
Jittery wireless-like	Lab CPU contention sometimes!

On shared lab hosts, **CPU starvation** fakes network loss—check `docker stats`.

Packet capture practice

```
tcpdump -ni eth1 -w /tmp/inc.pcap not port 22
# reproduce
tcpdump -r /tmp/inc.pcap -nn
tshark -r /tmp/inc.pcap -Y 'icmp || bgp || arp' -V | less
```

Capture points

Question	Where to capture
Did host send?	Host interface
Did router rewrite L2?	Router egress
Did peer receive?	Peer ingress
Policy drop?	Both sides + counters

Logging

FRR

Enable meaningful logging to file or syslog inside the node (image-dependent):

```
log file /var/log/frr/frr.log
log timestamp precision 3
```

```
docker exec clab-ospf-tri-r1 tail -f /var/log/frr/frr.log
```

Kernel

```
dmesg -T | tail
# martians, neighbor table full, etc.
```

Metrics sketch (Prometheus-shaped thinking)

Even if you do not run Prometheus yet, define signals:

Metric idea	Type	Labels
`link_up`	gauge	node, iface
`interface_errors_total`	counter	node, iface, dir
`bgp_session_up`	gauge	node, peer
`bgp_prefixes_received`	gauge	node, peer
`ospf_neighbors_full`	gauge	node
`ping_loss_ratio`	gauge	src, dst

Synthetic ping exporter + node exporter covers a surprising amount for home/lab NetOps.

Minimal blackbox idea

```
#!/usr/bin/env bash
# probe.sh - exit 1 on failure for CI/cron
ping -c 3 -W 1 192.168.2.10 >/dev/null
```

Cron or systemd timer + alert on fail is valid ops.

Telemetry awareness (2026)

Modern NOS (e.g. SR Linux community) emphasize streaming telemetry (gNMI). FRR/Linux labs may stick to show + exporters. Skill transfer:

- **Subscribe** to interface + protocol state
- Prefer push metrics over SSH scraping when scale grows
- Still keep packet capture for truth

Incident timeline template

```
## Incident
Start (UTC):
Detect signal:
Impact:
## Timeline
- T0:
- T+2m:
## Hypotheses
1.
2.
## Checks performed
## Root cause
## Fix
## Follow-ups
- monitor:
- prevent:
```

Lab drill: observe a BGP flap

1. Baseline `show bgp summary`
2. `neighbor shutdown` or link down
3. Record timestamps of state change and route withdrawal
4. Restore; measure time to Established and ping recovery
5. Write incident note with detection idea (“alert if bgp_session_up==0 for 60s”)

Lab drill: silent MTU blackhole

From the MTU chapter: large ping fails, small works.

Observability angle: which metric would catch this? (not link_down). Need synthetic large-packet probe or TCP service check.

Runbook snippet library

Keep short runbooks next to labs:

```
## Runbook: OSPF neighbors down
1. Check link state ip link
2. Check addressing same subnet
3. show ip ospf interface MTU
4. capture hello packets
5. check area / auth / passive
```

Privacy and safety

- Captures may contain payloads—store carefully
- Do not tcpdump production spans without authorization
- Redact journals before sharing

Summary

- Observability is layered: link → path → control → packet → service
- Baselines make incidents measurable
- Counters + mtr + captures beat vibes
- Define metrics before you buy platforms
- Runbooks convert personal heroics into repeatable ops

Next: **automation / lab-as-code**—pushing configs, smoke deploys, and CI-shaped checks.

Automation & Lab-as-Code

Automation & Lab-as-Code

A lab you cannot rebuild is a rumor. This chapter automates the Containerlab lifecycle, config push patterns, and smoke verification so your networking skill compounds in git.

Learning goals

By the end of this chapter you can:

- Wrap deploy/destroy/verify in repeatable scripts
- Organize inventories for multi-node labs
- Push or render configs with light Ansible or shell
- Sketch CI that smoke-tests a topology
- Treat secrets and image pins correctly

Principles

1. **Topology and config in git**
2. **Idempotent as practical** — re-run without fear
3. **Verify automatically** — exit codes matter
4. **No click-ops source of truth**
5. **Small steps** — shell first, Ansible when it pays off

Directory layout (automation-ready)

```
labs/ospf-tri/
  topology.clab.yml
  addressing.md
  README.md
```

```
  Makefile
  verify.sh
  up.sh
  down.sh
  config/
    r1/frr.conf
    r1/daemons
    r2/...
  ansible/
    inventory.yml
    playbooks/deploy-config.yml
```

Makefile pattern

```
TOPO := topology.clab.yml

.PHONY: up down verify smoke

up:
    sudo containerlab deploy -t $(TOPO)

down:
    sudo containerlab destroy -t $(TOPO) --cleanup

verify:
    ./verify.sh

smoke: up verify
    @echo smoke ok
```

```
make smoke
make down
```

up.sh / down.sh

```
#!/usr/bin/env bash
# up.sh
set -euo pipefail
cd "$(dirname "$0")"
sudo containerlab deploy -t topology.clab.yml
./verify.sh
```

```
#!/usr/bin/env bash
# down.sh
set -euo pipefail
cd "$(dirname "$0")"
sudo containerlab destroy -t topology.clab.yml --cleanup
```

Config strategies

Strategy	Pros	Cons
Bind-mount `/etc/frr`	Simple, git-native	Image load quirks
`containerlab` startup-config	Kind-supported	Kind-specific
Ansible after deploy	Flexible templating	Extra tool
Render templates → bind	Pure git artifacts	Build step

Prefer bind + pure files for FRR until pain appears.

Safe interactive explore → promote

```
docker exec -it clab-ospf-tri-r1 vtysh
# make change, show run
# copy running config back into config/r1/frr.conf
# redeploy clean to prove
```

Light Ansible (optional)

`ansible/inventory.yml`:

```
all:
  children:
    routers:
      hosts:
        r1:
          ansible_host: clab-ospf-tri-r1
          ansible_connection: docker
        r2:
          ansible_host: clab-ospf-tri-r2
          ansible_connection: docker
        r3:
          ansible_host: clab-ospf-tri-r3
          ansible_connection: docker
```

Playbook sketch:

```
- name: Push FRR config files
  hosts: routers
  gather_facts: false
  tasks:
    - name: Copy frr.conf
      copy:
        src: "{{ playbook_dir }}/../../config/{{ inventory_hostname }}/frr.conf"
        dest: /etc/frr/frr.conf
    - name: Reload FRR
      shell: vtysh -c 'write' || true
      # better: proper frr reload unit - image dependent
```

```
ansible-playbook -i ansible/inventory.yml ansible/playbooks/deploy-config.yml
```

Python with scrapli/netmiko is equally valid; pick one tool family and go deep later.

Templating addressing: DIY vs. Netlab

The DIY Approach (Jinja2 + custom scripts)

```
# addressing.yml
links:
  r1_r2: { r1: 10.0.12.1/24, r2: 10.0.12.2/24 }
lans:
  h1: { gw: 192.168.1.1/24, host: 192.168.1.10/24 }
```

You can render `frr.conf.j2` → `config/r1/frr.conf` with a custom Python/Jinja script or Ansible template module. **Generated files can be committed** for diff visibility or generated in CI.

The Netlab Declarative Approach

Writing custom Python Jinja compilers and manual IPAM spreadsheets becomes difficult to maintain when topologies exceed 3–4 nodes. Netlab provides a turnkey, declarative engine designed specifically for network labs:

```
# topology.yml
---
provider: clab
defaults.device: frr
module: [ ospf, bfd ]

nodes:
```

```
  r1:
  r2:
  r3:

links:
  - r1:
    r2:
  - r2:
    r3:
  - r3:
    r1:

validate:
  ospf_full:
    description: "Verify all OSPF adjacencies reach Full"
    nodes: [ r1, r2, r3 ]
    plugin: ospf_neighbor(count=2, state='Full')
```

Running `netlab up` calculates IPAM, renders Jinja2 templates for each device, stands up the Containerlab environment, and configures the routers. Running `netlab validate` executes the validation suite automatically.

Verification: verify.sh vs. netlab validate

Custom Shell Script (`verify.sh`)

```
#!/usr/bin/env bash
set -euo pipefail
fail() { echo "FAIL: $*" >&2; exit 1; }

need_full() {
  local node=$1
  docker exec "$node" vtysh -c 'show ip ospf neighbor' | grep -q Full \
    || fail "$node missing Full neighbor"
}

need_full clab-ospf-tri-r1
need_full clab-ospf-tri-r2
need_full clab-ospf-tri-r3

docker exec clab-ospf-tri-h1 ping -c 2 -W 1 192.168.2.10 || fail "h1->h2"
docker exec clab-ospf-tri-h2 ping -c 2 -W 1 192.168.1.10 || fail "h2->h1"

echo "OK $(date -Is)"
```

Netlab Automated Validation

With Netlab, assertions are declared directly in YAML and executed with exit codes suitable for CI:

```
# Returns exit code 0 on pass, non-zero on failure
netlab validate
```

CI smoke (GitHub Actions sketch)

Conceptual job (runner must support Docker + Containerlab—often self-hosted):

```
# conceptual - adapt to your runners
jobs:
  lab-smoke:
    runs-on: self-hosted-linux
    steps:
      - uses: actions/checkout@v4
      - name: deploy+verify
        working-directory: books/Networking/labs/ospf-tri
        run: |
          sudo containerlab deploy -t topology.clab.yml
          ./verify.sh
      - name: cleanup
        if: always()
        working-directory: books/Networking/labs/ospf-tri
        run: sudo containerlab destroy -t topology.clab.yml --cleanup
```

This book's monorepo CI renders Quarto; **lab smoke is optional local/self-hosted**. Do not assume GitHub-hosted runners allow nested Containerlab without setup.

Image pins in automation

```
# .env or Makefile
FRR_IMAGE=quay.io/frrouting/frr:10.2.1
ALPINE_IMAGE=alpine:3.20
```

Topology can be generated or documented to match. Record digests for serious CI.

Idempotency drills

1. `make smoke` twice without `down` — should not corrupt

2. `make down && make smoke` — clean path
3. Break config, `make smoke` fails exit 0
4. Fix config, smoke passes

Secret hygiene

- No production passwords in lab repos
- Lab default creds documented as lab-only
- Use `.gitignore` for private env files

Graph and inventory export

```
sudo containerlab inspect -t topology.clab.yml --format json > inspect.json
sudo containerlab graph -t topology.clab.yml || true
```

Feed inspect outputs into your own tools later.

Predict → observe → fix

Predict: A peer clones the lab folder, runs `make smoke`, gets OK without chatting you.

Observe: Remove tribal knowledge; only README + scripts.

Fix: Missing apk packages, wrong container names, unpinned images.

Harden: Add `make smoke` badge or CI on self-hosted when ready.

Anti-patterns

Anti-pattern	Replace with
Only screenshot proof	verify.sh
Snowflake node edits	bind + git
`latest` tags	pins
Huge mono playbook day one	Makefile smoke
CI without cleanup	always destroy

Summary

- Lab-as-code = topology + config + verify in git
- Start with Makefile/scripts; add Ansible when templating hurts
- Exit codes turn labs into testable systems
- CI is optional but shaping; cleanup always
- Promote working state from nodes back into the repo

Next: **capstone outline**—projects that prove the whole journey.

Capstone Outline

Capstone Outline

This chapter is a **map of finishing projects**, not a single mandatory mega-lab. Pick at least one capstone and run it to the book's definition of done: design, code, break, prove, operate.

Learning goals

By the end of this chapter you can:

- Choose a capstone that matches your goals
- Scope addressing, topology, and success metrics
- Apply models + verification + automation end-to-end
- Produce a portfolio-quality lab folder
- Identify natural next topics beyond this spine

Definition of done (all capstones)

☐ Written design (1–2 pages): goals, non-goals, addressing, failure modes

☐ `topology.clab.yml` + configs in git

☐ `addressing.md` + diagram

☐ `verify.sh` green on clean deploy

☐ 3 failure drills with incident notes

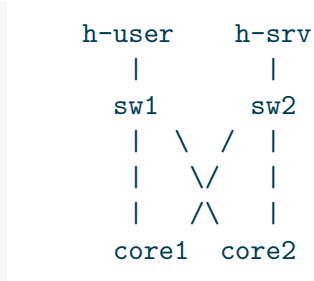
☐ One automation wrapper (`make smoke` or equivalent)

☐ Short retrospective: what model changed in your head

Capstone C1 — Campus-style dual core (open images)

Intent

Two access switches (or Linux bridges), dual core routers, dual-homed access, VLANs for users/servers, OSPF underlay between cores, static or OSPF toward access.



Skills exercised

- VLANs/trunks
- L2/L3 boundary
- OSPF or static redundancy
- Failure: core link down, trunk VLAN missing

Success metrics

- Hosts in VLAN 10 communicate across access
- Inter-VLAN via cores
- One core or one uplink loss recovers within your documented budget

Suggested stack

FRR cores + Alpine hosts + Linux bridge “switches” or SR Linux if you want NOS practice.

Capstone C2 — Dual-homed site to provider edge

Intent

Customer edge (CE) with two eBGP uplinks to two "ISP" PE routers; prefer primary; failover to backup; filter what you advertise/accept.

```
        PE1 (AS65010)
       /
CE (AS65001)
       \
        PE2 (AS65020)
```

Skills exercised

- eBGP sessions
- Prefix filters / route-maps
- Local-pref / prepend
- Session loss drill

Success metrics

- Only owned prefixes exported
- ISP rejects hijack test prefix
- Traffic shifts on primary PE failure

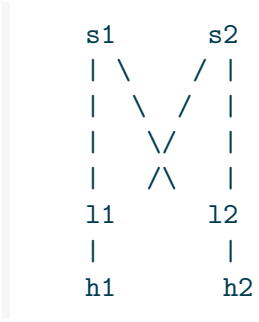
verify extras

Negative test: unauthorized prefix must not appear on PE.

Capstone C3 — Mini fabric snack

Intent

2 spine + 2 leaf (or 2×2) IP fabric with OSPF or eBGP underlay; endpoints on leaves; break a leaf uplink.



Skills exercised

- Underlay design
- ECMP behavior
- Scale of configs (templating pays off)
- Observability under failure

Stretch (optional)

Add a simple tunnel/overlay between leaves **after** underlay is solid—never before.

Capstone C4 — Incident week

Intent

Take any working lab (C1–C3 or static triangle). Each day for five days inject a **hidden** fault (or swap with a peer) and practice detect → diagnose → fix → prevent.

Day	Example inject
1	Wrong VLAN on one access port
2	Missing return route
3	MTU clamp + ICMP filter
4	OSPF cost surprise / link flap
5	BGP filter too strict

Skills exercised

- Observability under time pressure
- Incident notes
- Avoiding random change storms

Success metrics

- Each day: root cause written with evidence
- verify.sh restored green
- One new automated check added by end of week

How to pick

Goal	Pick
Campus / enterprise ops	C1
Edge / multihoming	C2
DC-ish fabric thinking	C3
Diagnostic mastery	C4 (optionally + another)

Time box: **1–2 weeks evenings** per capstone at 5–8h/week pace.

Project template (copy)

```
capstones/c2-dual-home/
  DESIGN.md
  README.md
  topology.clab.yml
  addressing.md
  verify.sh
  Makefile
  config/
  journals/
    drill-1.md
    drill-2.md
    drill-3.md
```

```
  diagrams/
  RETRO.md
```

DESIGN.md skeleton

```
# Design: Dual-homed CE

## Goals
## Non-goals
## Topology
## Addressing
## Control plane
## Policy
## Failure modes
## Observability
## Test plan
```

RETRO.md prompts

- Which plane failed most often in your drills?
- What did you overcomplicate?
- Which verify checks caught real bugs?
- What would you template next?

Integration with the book spine

Book part	Capstone use
Models	Language in DESIGN.md
Lab craft	Containerlab hygiene
L2	C1 trunks/VLANs
L3	All (addressing, static edges)
Interior routing	C1 OSPF, C2 BGP, C3 underlay
Ops automation	Makefile, verify, journals

Beyond this book (next horizons)

When capstones feel comfortable:

- Multi-area OSPF / IS-IS
- EVPN/VXLAN on free images where feasible
- QoS classification labs
- Stronger automation (CI self-hosted, config generate)
- Streaming telemetry on SR Linux
- Traffic engineering and more advanced BGP

Stay open-tools-first unless your job provides licensed platforms.

Final checkpoint: "expert" as defined in the syllabus

You can:

- Design multi-site L2/L3 with clear underlay roles
- Implement in Containerlab with free images
- Diagnose control vs data failures with tables + captures
- Apply routing policy without creating loops
- Automate lab lifecycle and basic validation
- Operate with observability and recovery habits

If any bullet is weak, return to that part's drills—not to random new features.

Summary

- Capstones prove integration, not trivia
- C1 campus, C2 dual-home BGP, C3 fabric snack, C4 incident week
- Same definition of done for all: design, code, verify, break, document
- Automation and negative tests separate portfolio work from toy demos
- The journey map continues; the habits stay

Return to the syllabus for the full long-range map (overlays, QoS, multi-area depth). Your next commit should be a capstone folder, not another unread PDF.

Edge and Services

First-Hop Redundancy

First-Hop Redundancy

Hosts rarely run a routing protocol. They send traffic to a **default gateway**. If that gateway dies, the LAN dies with it—unless you give the LAN a **virtual first hop** that can move between routers. This chapter builds the model and a free-stack lab: Linux + FRR + Containerlab (mid-2026 open path).

Learning goals

By the end of this chapter you can:

- Explain why a single gateway IP is a single point of failure
- Contrast VRRP-class first-hop redundancy with floating statics and ECMP
- Deploy a dual-router VRRP pair on FRR for a shared virtual IP (VIP)
- Predict host behavior when the master fails and when it recovers
- Verify with `ip neigh`, pings, and FRR VRRP state—not hope

Concepts

The problem

```
h1 ---- [LAN] ---- gw 10.0.0.1  (one box)
           |
        Internet-ish
```

Host config: `default via 10.0.0.1`. When that box reboots, ARP still points at a MAC that is gone until the host relearns—or traffic blackholes forever if nothing answers.

First-hop redundancy idea

Two (or more) routers share:

Object	Role
Virtual IP (VIP)	Address hosts use as gateway

Object	Role
Virtual MAC (often)	Stable L2 identity for the VIP
Master / backup roles	Who owns the VIP right now
Hello / advertise	"I am alive and I claim priority"

Hosts keep one gateway IP. The **active forwarder** changes under them.

Protocol family (vendor-agnostic)

Name	Notes
VRRP	Open standard idea; FRR supports VRRP
HSRP / GLBP	Proprietary dialects of the same problem class
CARP	OpenBSD-family cousin

This book uses **VRRP on FRR** because it is free, documented, and works in Containerlab without licenses. Optional: Nokia SR Linux community images have their own FHRP configuration surface—same model, different CLI.

Priority, preempt, timers

- **Priority:** higher wins master election (within rules).
- **Preempt:** when a higher-priority router returns, does it take the VIP back?
- **Advertisement interval:** how fast backups notice silence.
- **Skew / master-down interval:** how long before backup promotes.

What FHRP is *not*

Not this	Because
Full routing protocol	Hosts do not learn remote prefixes via VRRP
Load balancing of arbitrary flows (unless designed)	Classic VRRP is active/standby for a VIP
Multihoming policy	That is BGP/statics + dual default; separate chapter lab

FHRP protects the **LAN's first hop**. Dual-homing the site edge is complementary (see dual-home lab later in this part).

ARP and the VIP

When master changes, hosts may still have the old MAC in the neighbor cache briefly. Good implementations use a **shared virtual MAC** or gratuitous ARP so clients rebind quickly. In labs, clear neighbor cache deliberately when debugging:

```
# Linux host
ip neigh flush dev eth1
# or
ip neigh del 10.0.0.254 dev eth1
```

Addressing plan

Node	Role	Interface	Address
r1	VRRP priority high	eth1 (LAN)	10.10.10.2/24
r2	VRRP priority low	eth1 (LAN)	10.10.10.3/24
VIP	gateway for hosts	—	10.10.10.254
h1	client	eth1	10.10.10.10/24 via VIP
r1 eth2	uplink	eth2	172.16.1.1/30
r2 eth2	uplink	eth2	172.16.2.1/30
pe	“upstream”	eth1/eth2	172.16.1.2, 172.16.2.2

Upstream `pe` can be a simple FRR or Linux box that advertises a sink for drills (`203.0.113.1` loopback).

Topology YAML

```
name: vrrp-edge

topology:
  nodes:
    r1:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/r1:/etc/frr
    r2:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/r2:/etc/frr
    pe:
```

```
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/pe:/etc/frr
    h1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 10.10.10.10/24 dev eth1
        - ip link set eth1 up
        - ip route add default via 10.10.10.254

  links:
    - endpoints: ["h1:eth1", "r1:eth1"]
    - endpoints: ["r1:eth1", "r2:eth1"]   # same LAN: use bridge or multipoint
    - endpoints: ["r1:eth2", "pe:eth1"]
    - endpoints: ["r2:eth2", "pe:eth2"]
```

LAN wiring note: two point-to-point links do not make a shared segment. Prefer a Linux bridge node as "LAN switch," or Containerlab multipoint / bridge patterns from your lab-craft notes:

```
    lan:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 bridge-utils
        - ip link add name br0 type bridge
        - ip link set br0 up
  # links: h1, r1, r2 all into lan with bridge enslavement in exec
```

Simpler teaching pattern: **one host + two routers on a bridge** so ARP and VRRP multicast behave like a campus LAN.

FRR daemons

```
zebra=yes
vrrpd=yes
# staticd or bgpd if you route beyond the VIP
staticd=yes
```

Check mid-2026 FRR image docs for exact daemon names (`vrrpd` may be integrated differently by version—`vtysh show vrrp` is the operator check).

FRR VRRP sketch (r1)

```
frr version 10.2.1
frr defaults traditional
hostname r1
!
interface eth1
 ip address 10.10.10.2/24
!
interface eth2
 ip address 172.16.1.1/30
!
interface eth1
 vrrp 1 ip 10.10.10.254
 vrrp 1 priority 200
 vrrp 1 advertisement-interval 1000
!
ip route 0.0.0.0/0 172.16.1.2
!
line vty
```

r2: real IP `10.10.10.3/24`, priority `100`, same VIP and VRID `1`, default via its own uplink.

Syntax varies slightly by FRR minor version—always validate with:

```
docker exec clab-vrrp-edge-r1 vtysh -c 'show vrrp'
docker exec clab-vrrp-edge-r1 vtysh -c 'show interface eth1'
```

Linux alternative (keepalived)

Some operators run keepalived on Linux instead of NOS VRRP. Same objects: VIP, priority, advert. In pure Alpine labs you may:

```
# illustrative - package availability varies
# apk add keepalived
# configure vrrp_instance with virtual_ipaddress
```

For this book's default path, **FRR VRRP** keeps one control-plane stack with your OSPF/BGP labs.

Deploy and baseline

```
sudo containerlab deploy -t vrrp-edge.clab.yml

docker exec clab-vrrp-edge-r1 vtysh -c 'show vrrp'
docker exec clab-vrrp-edge-r2 vtysh -c 'show vrrp'
docker exec clab-vrrp-edge-h1 ip route
docker exec clab-vrrp-edge-h1 ip neigh
docker exec clab-vrrp-edge-h1 ping -c 3 10.10.10.254
docker exec clab-vrrp-edge-h1 ping -c 3 172.16.1.2
```

Predict (baseline)

- r1 is **Master**, r2 is **Backup** (priority 200 > 100)
- h1 ARP for VIP resolves to master's virtual/real MAC pattern for your stack
- Ping VIP succeeds; traffic to upstream uses r1's uplink

Observe

Record: VRRP state, VIP owner interface counters, `ip neigh` on h1, traceroute if upstream is reachable.

Drill 1 — Master failure

```
# Simulate master death (stop data plane or whole node)
docker exec clab-vrrp-edge-r1 ip link set eth1 down
# or: docker stop clab-vrrp-edge-r1

sleep 3
docker exec clab-vrrp-edge-r2 vtysh -c 'show vrrp'
docker exec clab-vrrp-edge-h1 ip neigh show 10.10.10.254
docker exec clab-vrrp-edge-h1 ping -c 10 10.10.10.254
docker exec clab-vrrp-edge-h1 ping -c 5 203.0.113.1   # if pe has this
```

Predict → observe → fix

Predict	Observe	Fix if wrong
r2 becomes Master after down-timer	`show vrrp`	timers, VRID mismatch, not same LAN

Predict	Observe	Fix if wrong
Short ping loss then recovery	count drops	ARP stuck—flush neigh; check VIP MAC
Upstream still works via r2	traceroute	r2 default / pe return routes

Restore:

```
docker exec clab-vrrp-edge-r1 ip link set eth1 up
# or docker start + wait FRR ready
sleep 5
docker exec clab-vrrp-edge-r1 vtysh -c 'show vrrp'
docker exec clab-vrrp-edge-r2 vtysh -c 'show vrrp'
```

Preempt on: r1 reclaims Master. **Preempt off:** r2 may stay Master—document which you configured.

Drill 2 — Split brain / LAN partition (concept)

If both routers believe they are alone (broken LAN bridge), **two masters** can both answer the VIP → intermittent ARP thrash.

Predict

Duplicate VIP answers; flapping neighbor cache; odd packet loss.

Observe

```
docker exec clab-vrrp-edge-r1 vtysh -c 'show vrrp'
docker exec clab-vrrp-edge-r2 vtysh -c 'show vrrp'
# tcpdump for VRRP (IP protocol 112) on the LAN
docker exec clab-vrrp-edge-h1 sh -c 'apk add --no-cache tcpdump; tcpdump -ni eth1 proto 112
```

Fix / harden

- Ensure single L2 domain for the VRID
- BFD or link tracking to uplinks (advanced) so a “master” without upstream demotes
- Monitoring on “two masters seen” is an ops smell

Drill 3 — Priority change live

```
docker exec -it clab-vrrp-edge-r2 vtysh
configure terminal
interface eth1
 vrrp 1 priority 250
end
write
```

Predict

If preempt allowed, r2 becomes Master; h1 may re-ARP; path to upstream shifts to r2's uplink.

Observe

```
docker exec clab-vrrp-edge-r1 vtysh -c 'show vrrp'
docker exec clab-vrrp-edge-r2 vtysh -c 'show vrrp'
docker exec clab-vrrp-edge-h1 traceroute -n 172.16.2.2
```

Revert priorities in git-mounted configs after the experiment.

Return path and asymmetric edges

VIP failover only moves the **outbound first hop**. If upstream routing always returns via r1 while r2 is master, you get asymmetric paths or blackholes. Pair FHRP with:

- Shared upstream design, or
- Dynamic routing so return traffic follows the active edge, or
- State-aware NAT only on the active path (harder—prefer avoid state at edge in labs first)

Verification checklist

Check	Intent
`show vrrp`	One Master per VRID on the LAN
Host default route	Points at VIP only
`ping VIP`	L2/L3 ownership of first hop
Kill master	Backup promotes; host recovers
Capture proto 112	Hellos present on LAN
Upstream ping during fail	Data plane + return path

verify.sh sketch

```
#!/usr/bin/env bash
set -euo pipefail
fail() { echo "FAIL: $*" >&2; exit 1; }

docker exec clab-vrrp-edge-h1 ping -c 2 -W 1 10.10.10.254 || fail "VIP"
# Exactly one master is harder in shell-parse show vrrp or accept soft check:
docker exec clab-vrrp-edge-r1 vtysh -c 'show vrrp' | grep -qi master \
  || docker exec clab-vrrp-edge-r2 vtysh -c 'show vrrp' | grep -qi master \
  || fail "no master"
echo OK
```

Common mistakes

Mistake	Symptom
Routers not same L2 domain	Never form master/backup correctly
Different VRID or VIP	Two independent groups
Host points at real IP not VIP	No redundancy
No return routes via backup	Failover pings die mid-path
Priority tie without tie-break awareness	Unstable election
Forgetting to enable vrrp daemon	Empty `show vrrp`

Optional SR Linux note (community)

SR Linux community Containerlab kinds expose interface and IRP/FHRP-related configuration in a different tree (YANG/CLI). **Do not memorize CLI here**—map objects: VIP, group id, priority, interface, preempt. Same predict→observe drills.

Summary

- Hosts need a stable **first hop**; VRRP-class protocols move a VIP between routers
- FRR + Containerlab is enough to learn Master/Backup, preempt, and failure timers
- Always verify **host ARP**, **VRRP state**, and **end-to-end path** after failover
- FHRP is not multihoming policy—combine later with dual-home BGP/statics
- Keep configs in binds; promote live priority experiments back to git

Next: **DHCP relay and NAT**—how edge services attach to this LAN model without becoming mysterious middleboxes.

DHCP Relay and NAT

DHCP Relay and NAT

Edges do more than forward packets. They **hand out addresses** (or relay DHCP) and often **translate** private space toward a provider. This chapter keeps both topics vendor-agnostic and lab-first: Linux services + FRR routing in Containerlab (mid-2026 free path).

Learning goals

By the end of this chapter you can:

- Explain DHCP discover/offer/request/ack across a relay
- Configure a simple DHCP server and a relay path in a lab
- Contrast source NAT (masquerade), static NAT, and "why not NAT everything forever"
- Implement Linux `nftables`/`iptables` MASQUERADE on an edge
- Predict breakage when relays, helpers, or return translations fail

Concepts — DHCP

Why not static everything?

Labs love static host IPs. Real access LANs use DHCP for:

Benefit	Trade-off
Central policy (lease, DNS, gateway)	Dependency on server/relay
Fast readdressing	Lease timers and "sticky" clients
Audit of who got what	Need logging and hygiene

Four-way dance (v4)

```
Client          Relay (optional)         Server
  |-- Discover ------(unicast to server)-->|
  |<-- Offer -------------------------------|
  |-- Request ------------------------------>|
```

```
   |<-- Ack --------------------------------|
```

On a flat LAN, broadcast Discover reaches the server. Across subnets, a **relay agent** (ip helper / DHCP relay) listens on the client VLAN and unicasts to the server, inserting **giaddr** (gateway IP of the client interface) so the server knows which pool to use.

Critical fields

Field / idea	Why it matters
Client MAC / Client-ID	Lease binding
giaddr	Selects pool / subnet
Options: router, DNS, domain	Host becomes usable
Lease time	How long address is "owned"

Relay failure modes

- Wrong giaddr → wrong pool or no offer
- ACL blocking UDP 67/68
- Server unreachable from relay
- Multiple relays without design → duplicate offers chaos

Concepts — NAT family

Kind	Meaning
SNAT / MASQUERADE	Rewrite source as packets leave (common "PAT")
DNAT	Rewrite destination (port forward / publish service)
1:1 static NAT	Stable mapping for a host
NAT64 / CLAT	v6/v4 transition tools (awareness only here)

NAT is **state + rewrite**. It is not a security model by itself. Stateful firewalls may sit with NAT; keep roles clear.

When NAT is appropriate in this book

- Lab edge simulating consumer/ISP CPE
- Hiding RFC1918 behind a single lab "public"

- Temporary access during dual-stack migration

Prefer **real global or ULA design** for multi-site fabrics later—NAT at every hop becomes undebugable.

Addressing plan (combined edge lab)

```
  h-client (DHCP)          dhcp-srv
       |                      |
      lan ---- edge-rtr ---- pe (public-ish)
                  |
              NAT outside
```

Net	Use
10.20.30.0/24	Client LAN (DHCP pool)
10.20.30.1	Edge LAN IP + VRRP VIP optional
172.31.0.0/30	Edge–server link for DHCP server placement
198.51.100.0/30	Edge–PE “public” lab space (TEST-NET)
Pool	10.20.30.100–200, GW 10.20.30.1, DNS 10.20.30.1 or public lab DNS

Topology YAML

```
name: dhcp-nat-edge

topology:
  nodes:
    edge:
      kind: linux
      image: alpine:3.20
      # or FRR image if you co-locate routing; Alpine for pure Linux NAT/relay
      exec:
        - apk add --no-cache iproute2 iptables iputils dhcrelay dnsmasq
    pe:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 198.51.100.2/30 dev eth1
        - ip link set eth1 up
        - ip addr add 203.0.113.1/32 dev lo
        - ip link set lo up
    dhcpsrv:
```

```
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 dnsmasq iputils
        - ip addr add 172.31.0.2/30 dev eth1
        - ip link set eth1 up
    client:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 dhclient iputils
    lan:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2
        - ip link add br0 type bridge && ip link set br0 up

  links:
    - endpoints: ["client:eth1", "lan:eth1"]
    - endpoints: ["edge:eth1", "lan:eth2"]
    - endpoints: ["edge:eth2", "dhcpsrv:eth1"]
    - endpoints: ["edge:eth3", "pe:eth1"]
```

Bridge enslavement (run after deploy or in richer `exec` scripts):

```
# on lan node - enslaves data ports into br0
for i in eth1 eth2; do ip link set $i master br0; ip link set $i up; done
```

Edge addressing and forward

```
# edge LAN toward clients
docker exec clab-dhcp-nat-edge-edge ip addr add 10.20.30.1/24 dev eth1
docker exec clab-dhcp-nat-edge-edge ip link set eth1 up
# toward DHCP server
docker exec clab-dhcp-nat-edge-edge ip addr add 172.31.0.1/30 dev eth2
docker exec clab-dhcp-nat-edge-edge ip link set eth2 up
# toward PE
docker exec clab-dhcp-nat-edge-edge ip addr add 198.51.100.1/30 dev eth3
docker exec clab-dhcp-nat-edge-edge ip link set eth3 up
docker exec clab-dhcp-nat-edge-edge sysctl -w net.ipv4.ip_forward=1
docker exec clab-dhcp-nat-edge-edge ip route add default via 198.51.100.2
```

Bake these into startup scripts for real lab-as-code.

DHCP server (dnsmasq) on dhcpsrv

```
docker exec clab-dhcp-nat-edge-dhcpsrv sh -c 'cat >/etc/dnsmasq.conf <<EOF
interface=eth1
dhcp-range=10.20.30.100,10.20.30.200,12h
dhcp-option=option:router,10.20.30.1
dhcp-option=option:dns-server,10.20.30.1
log-dhcp
EOF'
docker exec clab-dhcp-nat-edge-dhcpsrv dnsmasq
```

Server is **not** on 10.20.30.0/24—the relay's giaddr must make the pool selection work. For dnsmasq/simple labs, operators often put the server **on** the client subnet first (no relay). Teach both:

1. **Same subnet:** client broadcast reaches server—no relay.

2. **Off-subnet:** relay required; server must understand the client subnet (shared-network / giaddr-aware config).

Minimal same-subnet lab (sanity first)

Place dnsmasq on edge's LAN IP or a server on the bridge:

```
dhcp-range=10.20.30.100,10.20.30.200,12h
dhcp-option=option:router,10.20.30.1
```

Relay path (concept + command)

```
# edge: relay client-facing interface toward server
docker exec clab-dhcp-nat-edge-edge dhcrelay -i eth1 172.31.0.2
```

Predict (DHCP)

- Client sends Discover; server logs Offer for 10.20.30.x

- Client installs default via 10.20.30.1

- Lease appears in server logs

Observe

```
docker exec clab-dhcp-nat-edge-client ip link set eth1 up
docker exec clab-dhcp-nat-edge-client dhclient eth1 -v
docker exec clab-dhcp-nat-edge-client ip addr show eth1
docker exec clab-dhcp-nat-edge-client ip route
docker exec clab-dhcp-nat-edge-dhcpsrv cat /var/lib/misc/dnsmasq.leases 2>/dev/null || true
```

Fix

Symptom	Checks
No offer	bridge, interface up, firewall, wrong interface in dnsmasq
Offer but wrong GW	dhcp-option router
Relay silent	UDP 67/68, giaddr, server route

NAT — MASQUERADE on edge

Clients use private 10.20.30.0/24; PE only knows 198.51.100.0/30.

```
docker exec clab-dhcp-nat-edge-edge iptables -t nat -A POSTROUTING \
  -o eth3 -j MASQUERADE
# nftables equivalent pattern:
# nft add table ip nat
# nft add chain ip nat postrouting { type nat hook postrouting priority 100 \; }
# nft add rule ip nat postrouting oifname "eth3" masquerade
```

PE needs a way to answer (connected route to 198.51.100.1 is enough for return of MASQUERADE traffic).

Predict

- `client → 203.0.113.1` works
- On pe, sources appear as 198.51.100.1, not 10.20.30.x
- Without MASQUERADE, pe has no return path to 10.20.30.0/24

Observe

```
docker exec clab-dhcp-nat-edge-client ping -c 3 203.0.113.1
docker exec clab-dhcp-nat-edge-pe sh -c 'apk add --no-cache tcpdump; tcpdump -ni eth1 -c 5 i
docker exec clab-dhcp-nat-edge-edge iptables -t nat -L -n -v
```

Fix

- `ip_forward=1`
- Correct `-o` egress interface
- Default route on client and edge
- Conntrack full / flush during experiments: `conntrack -F` if tool present

Drill — break DHCP then NAT

1. Stop dnsmasq → new client gets nothing; journal "dependency."
2. Remove MASQUERADE → pings to PE fail; add static route on PE for 10.20.30.0/24 via edge **as a teaching alternate** (routing vs NAT).
3. DNAT publish (optional): map 198.51.100.1:8080 → client:80 and curl from pe.

```
docker exec clab-dhcp-nat-edge-edge iptables -t nat -A PREROUTING \
  -i eth3 -p tcp --dport 8080 -j DNAT --to-destination 10.20.30.100:80
```

Only works if a client actually holds that lease/IP and listens.

FRR on the edge (optional co-location)

Many designs run FRR for BGP/OSPF and Linux for NAT/DHCP. Pattern:

- FRR owns routing (`vtysh`)
- Linux netfilter owns NAT
- DHCP server may be centralized; FRR/host runs relay

Do not assume FRR "is" the NAT engine—check which process owns rewrite.

Verification checklist

Check	Command / intent
Lease	client has /24 + default
Options	DNS/GW match design
Relay	server log shows giaddr path
NAT	capture shows public source
Forward	sysctl ip_forward
Rules	iptables/nft list nat

Common mistakes

Mistake	Symptom
DHCP server wrong subnet logic	No lease or wrong mask
Client static leftover IP	"DHCP broken" false positive
NAT without hairpin plan	Local services odd
Forgetting return routing when NAT removed	One-way pings
Blocking bootp/dhcp in filter	Silent failure

Summary

- DHCP centralizes host addressing; **relay** extends it across L3
- **giaddr** and pool design matter as much as the daemon
- NAT rewrites and tracks state—use deliberately, verify with captures
- Linux edge + Containerlab is enough to practice both without licenses
- Prefer lab scripts that start dnsmasq, relay, and NAT rules idempotently

Next: **name, time, and log hygiene**—the quiet services that make edge failures diagnosable.

Name, Time, and Log Hygiene

Name, Time, and Log Hygiene

Broken DNS, skewed clocks, and silent logs turn simple outages into multi-hour mysteries. This chapter treats **name resolution, time sync, and logging** as first-class edge services—still on free Linux tooling in Containerlab (mid-2026).

Learning goals

By the end of this chapter you can:

- Design a minimal lab DNS path (stub resolver → recursive or authoritative)
- Explain why certificate and log correlation depend on clock sync
- Run chrony/ntp-style time sync in a lab and detect skew
- Centralize or at least structure logs so incidents are replayable
- Apply a hygiene checklist to any edge or fabric lab

Why this chapter exists

Failure	Symptom people mis-blame
DNS	"The network is down" (TCP works, names fail)
Time skew	TLS fails, Kerberos fails, log order nonsense
No logs	Every fix is guesswork; no postmortem

NetOps competence includes **service dependencies of the control and management planes**.

Concepts — naming

Layers of DNS use

```
Application
    → stub resolver (/etc/resolv.conf)
        → recursive resolver (lab or ISP)
            → authoritative servers
```

In labs you often run:

Role	Example tool
Authoritative for `lab.example`	dnsmasq, unbound, knot (as available)
Recursive	unbound, dnsmasq
Stub on nodes	`nameserver` in resolv.conf

Records you actually need

Type	Lab use
A/AAAA	Host and VIP names
PTR	Reverse for traceroute readability (optional)
SRV	Only when practicing real apps
CNAME	Alias hygiene—avoid chains

Split horizon awareness

Internal names may not match external. Document which view a resolver serves. Mis-split horizon looks like “works on my jump host.”

Concepts — time

- **UTC everywhere** in configs and logs when possible
- **NTP/chrony/ptp** family: clients step or slew toward sources
- Auth and TLS validate **notBefore/notAfter** against local clock
- Distributed tracing and log merge need comparable timestamps

Skew of minutes is enough to break modern auth. Skew of seconds confuses incident timelines.

Concepts — logs

Plane	What to capture
Device/OS	link up/down, daemon start, OOM
Routing	neighbor flaps, policy denies (where logged)
Security	filter drops if logged
Lab harness	deploy/destroy, verify.sh results

Structured fields beat walls of free text: `timestamp host facility message`.

Addressing plan (hygiene lab)

```
  client ---- edge ---- dns-ntp
                 \
                  pe
```

Node	IP	Role
edge eth1	10.40.0.1/24	client GW
client	10.40.0.10/24	stub resolver → edge or dns
dns-ntp	10.40.0.53/24	dnsmasq + chrony server
edge eth2	198.51.100.1/30	optional upstream

Topology YAML

```yaml
name: hygiene-edge

topology:
  nodes:
    edge:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iptables iputils bind-tools chrony
    dnsntp:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 dnsmasq chrony iputils bind-tools
    client:
      kind: linux
```

```
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils bind-tools
    pe:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils

  links:
    - endpoints: ["client:eth1", "edge:eth1"]
    - endpoints: ["dnsntp:eth1", "edge:eth2"]
    - endpoints: ["edge:eth3", "pe:eth1"]
```

Use a small bridge if you want client and dnsntp on one LAN with edge as L3 gateway only—either design is fine if documented.

Bring-up addressing (example)

```
# edge: two LANs + pe
docker exec clab-hygiene-edge-edge sh -c '
  ip addr add 10.40.0.1/24 dev eth1; ip link set eth1 up
  ip addr add 10.40.1.1/24 dev eth2; ip link set eth2 up
  ip addr add 198.51.100.1/30 dev eth3; ip link set eth3 up
  sysctl -w net.ipv4.ip_forward=1
'
docker exec clab-hygiene-edge-dnsntp sh -c '
  ip addr add 10.40.1.53/24 dev eth1; ip link set eth1 up
  ip route add default via 10.40.1.1
'
docker exec clab-hygiene-edge-client sh -c '
  ip addr add 10.40.0.10/24 dev eth1; ip link set eth1 up
  ip route add default via 10.40.0.1
  echo nameserver 10.40.1.53 > /etc/resolv.conf
'
# edge route to dns LAN is connected; client needs path to 10.40.1.53
docker exec clab-hygiene-edge-edge ip route add 10.40.1.0/24 dev eth2 2>/dev/null || true
```

Ensure client can reach 10.40.1.53 (forwarding + reverse path).

DNS lab — dnsmasq authoritative snack

```
docker exec clab-hygiene-edge-dnsntp sh -c 'cat >/etc/dnsmasq.conf <<EOF
interface=eth1
no-resolv
address=/vip.lab.example/10.40.0.254
address=/edge.lab.example/10.40.0.1
address=/client.lab.example/10.40.0.10
log-queries
EOF'
docker exec clab-hygiene-edge-dnsntp dnsmasq
```

Predict

```
docker exec clab-hygiene-edge-client nslookup vip.lab.example 10.40.1.53
# or
docker exec clab-hygiene-edge-client dig +short vip.lab.example @10.40.1.53
```

Returns `10.40.0.254`.

Observe / fix

Symptom	Fix
timeout	routing, firewall, dnsmasq not listening
NXDOMAIN	typo in address= or wrong server
uses wrong resolver	resolv.conf

Drill — DNS outage looks like network outage

```
docker exec clab-hygiene-edge-dnsntp pkill dnsmasq || true
docker exec clab-hygiene-edge-client ping -c 2 vip.lab.example   # may fail resolve
docker exec clab-hygiene-edge-client ping -c 2 10.40.0.1         # IP still works
```

Predict → observe → fix

- Name fails, IP works → **name plane**, not forwarding
- Restart dnsmasq; retest dig
- Harden: secondary resolver IP in resolv.conf (even if both lab-local)

```
nameserver 10.40.1.53
nameserver 10.40.1.54
```

Time sync — chrony sketch

On dnsntp as a simple lab time source (isolated labs often use local orphan mode or host chrony):

```
docker exec clab-hygiene-edge-dnsntp sh -c 'cat >/etc/chrony/chrony.conf <<EOF
local stratum 8
allow 10.40.0.0/16
makestep 1.0 3
EOF'
docker exec clab-hygiene-edge-dnsntp chronyd
```

Client/edge:

```
docker exec clab-hygiene-edge-client sh -c '
  apk add --no-cache chrony
  echo "server 10.40.1.53 iburst" > /etc/chrony/chrony.conf
  chronyd
'
docker exec clab-hygiene-edge-client chronyc tracking
docker exec clab-hygiene-edge-client chronyc sources -v
```

Package paths vary (`/etc/chrony.conf` vs `/etc/chrony/chrony.conf`)—adjust to the image.

Skew drill

```
# induce skew on client (lab only)
docker exec clab-hygiene-edge-client date -s "2020-01-01 00:00:00"
docker exec clab-hygiene-edge-client chronyc -a makestep
docker exec clab-hygiene-edge-client date
docker exec clab-hygiene-edge-dnsntp date
```

Predict

After makestep/slew, client approaches server time; `chronyc tracking` shows low offset.

Observe

Record offset before/after. Correlate with a fake "TLS" check: if you generate certs with short validity in lab tools, wrong dates fail verification.

Logging hygiene

Local journal pattern

```
# example: log dnsmasq and link events to a file on edge
docker exec clab-hygiene-edge-edge sh -c '
  mkdir -p /var/log/lab
  echo "$(date -Is) lab start" >> /var/log/lab/edge.log
'
```

syslog-style forward (concept)

```
device/app → rsyslog/syslog-ng → central lab collector
```

In Containerlab, mounting a host directory for `/var/log/lab` keeps evidence after destroy:

```
    edge:
      kind: linux
      image: alpine:3.20
      binds:
        - ./logs/edge:/var/log/lab
```

What to log for network incidents

Event	Why
Link down/up	Correlates with neighbor loss
Daemon restart	Control plane blip
DHCP ACK / VRRP transition	Edge role changes
Filter deny (sampled)	Policy vs outage
verify.sh pass/fail	Automation truth

FRR logging (when routers are FRR)

```
log file /var/log/frr/frr.log
log timestamp precision 3
```

```
docker exec clab-...-r1 vtysh -c 'show log'
# or tail the bound log file
```

Bind `/var/log/frr` into the lab folder for postmortems.

Hygiene checklist (use every serious lab)

- ☐ Addressing plan names every important VIP and service IP
- ☐ DNS A records for VIP, routers, servers (even if only lab zone)
- ☐ Clients have 1 working resolver; document fallback
- ☐ Clocks within seconds of a known source (or explicitly isolated)
- ☐ Logs land on durable storage (bind mount / central)
- ☐ Timezone policy stated (UTC recommended)
- ☐ Incident notes include timestamps from the **same** clock domain

Predict → observe → fix mini-scenarios

Scenario A — "Git clone fails" in a lab VM

- Predict: DNS or proxy
- Observe: `ping 1.1.1.1` vs `ping github.com`
- Fix: resolv.conf / reachability to recursive resolver

Scenario B — "Certificate expired" on brand-new lab CA

- Predict: node year is wrong
- Observe: `date -u` on client and server
- Fix: chrony; disable fake date experiments

Scenario C — "It failed an hour ago" with no trace

- Predict: no logs retained
- Observe: empty `/var/log/lab`, container destroyed
- Fix: binds + verify.sh archival

Optional: name the lab graph

Containerlab graph exports help humans; DNS helps apps. Keep **hostname = inventory name = DNS label** when you can:

```
clab-hygiene-edge-edge     edge.lab.example
```

Common mistakes

Mistake	Symptom
resolv.conf points at host DNS unreachable from namespace	all names fail
No forward path to DNS subnet	intermittent based on which node
Logging only to container overlay	evidence vanishes on destroy
Mixing local timezones in notes	impossible timelines
NTP blocked by ACL you added "for hardening"	chronic skew

Summary

- **Name, time, logs** are edge/management dependencies, not optional polish
- Practice DNS outage vs forwarding outage until the difference is reflex
- Sync clocks before debugging TLS or distributed logs
- Bind-mount lab logs; structure enough fields to postmortem
- Add this hygiene checklist to dual-home and fabric labs later

Next: **edge dual-home lab**—tie FHRP, routing preference, and these services into one site edge workout.

Edge Dual-Home Lab

Edge Dual-Home Lab

This chapter is a **full edge workout**: one customer site, two uplinks to two providers, first-hop redundancy on the LAN, filtered eBGP, and failure drills. Stack: Containerlab + FRR + Alpine (mid-2026 free path). Optional SR Linux as a PE substitute for CLI diversity.

Learning goals

By the end of this chapter you can:

- Design dual-homed CE addressing and AS layout
- Prefer one uplink with local-preference; backup with AS-path prepend
- Combine VRRP VIP for hosts with dual CE or single CE dual-uplink patterns
- Prove failover with ping bursts, BGP tables, and traceroute
- Ship a lab folder that meets the book's definition of done

Design choices (pick one primary)

Pattern A — Single CE, two eBGP uplinks (simpler)

```
      pe1 (AS65010)
     /
ce (AS65001) ---- LAN ---- hosts (GW = ce LAN IP or VIP if HA pair)
     \
      pe2 (AS65020)
```

Pattern B — Dual CE + VRRP (more realistic campus edge)

```
      pe1
     /
ce1 === VRRP VIP === LAN --- hosts
     \   /
      ce2
```

```
        \
         pe2
```

This chapter implements **Pattern B** as the main lab and notes how to simplify to A.

Addressing plan

Object	Value
Site LAN	192.168.50.0/24
VIP gateway	192.168.50.254
ce1 LAN	192.168.50.2/24
ce2 LAN	192.168.50.3/24
host	192.168.50.10/24 via VIP
ce1–pe1	172.16.10.0/30 (ce1 .1, pe1 .2)
ce2–pe2	172.16.20.0/30 (ce2 .1, pe2 .2)
Site prefix	192.168.50.0/24 (advertise)
pe1 lo	1.1.1.1/32 (AS65010)
pe2 lo	2.2.2.2/32 (AS65020)
Hijack test	203.0.113.0/24 must **not** be accepted from CE

Topology YAML

```yaml
name: dualhome

topology:
  nodes:
    ce1:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/ce1:/etc/frr
    ce2:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/ce2:/etc/frr
    pe1:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/pe1:/etc/frr
    pe2:
      kind: linux
```

```
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/pe2:/etc/frr
    h1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 192.168.50.10/24 dev eth1
        - ip link set eth1 up
        - ip route add default via 192.168.50.254
    lan:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2
        - ip link add br0 type bridge
        - ip link set br0 up

  links:
    - endpoints: ["h1:eth1", "lan:eth1"]
    - endpoints: ["ce1:eth1", "lan:eth2"]
    - endpoints: ["ce2:eth1", "lan:eth3"]
    - endpoints: ["ce1:eth2", "pe1:eth1"]
    - endpoints: ["ce2:eth2", "pe2:eth1"]
```

Enslave LAN ports to `br0` in a deploy hook:

```
for n in eth1 eth2 eth3; do
  docker exec clab-dualhome-lan ip link set $n master br0
  docker exec clab-dualhome-lan ip link set $n up
done
```

FRR daemons

```
zebra=yes
bgpd=yes
staticd=yes
vrrpd=yes
```

CE1 config sketch

```
frr version 10.2.1
frr defaults traditional
hostname ce1
!
interface lo
 ip address 10.0.0.1/32
!
interface eth1
 ip address 192.168.50.2/24
 vrrp 1 ip 192.168.50.254
 vrrp 1 priority 200
!
interface eth2
 ip address 172.16.10.1/30
!
router bgp 65001
 bgp router-id 10.0.0.1
 no bgp ebgp-requires-policy
 neighbor 172.16.10.2 remote-as 65010
 !
 address-family ipv4 unicast
  network 192.168.50.0/24
  neighbor 172.16.10.2 route-map FROM-PE in
  neighbor 172.16.10.2 route-map TO-PE out
 exit-address-family
!
ip prefix-list SITE seq 5 permit 192.168.50.0/24
ip prefix-list SITE seq 10 deny any
!
ip prefix-list DEFAULT-OK seq 5 permit 0.0.0.0/0
ip prefix-list DEFAULT-OK seq 10 deny any
!
route-map TO-PE permit 10
 match ip address prefix-list SITE
route-map TO-PE deny 20
!
route-map FROM-PE permit 10
 match ip address prefix-list DEFAULT-OK
 set local-preference 200
route-map FROM-PE deny 20
!
line vty
```

`no bgp ebgp-requires-policy` is a lab convenience on some FRR versions—prefer explicit policies always in "production-shaped" labs.

CE2 config differences

- LAN IP `.3`, VRRP priority `100`
- Uplink `172.16.20.1/30` to pe2
- **Outbound prepend** so pe2 is less preferred by others if both advertise:

```
route-map TO-PE permit 10
 match ip address prefix-list SITE
 set as-path prepend 65001 65001
```

- **Inbound** local-pref `100` (lower than ce1's 200) if both learn default:

```
route-map FROM-PE permit 10
 match ip address prefix-list DEFAULT-OK
 set local-preference 100
```

For dual-CE, each CE may only have one uplink—**VRRP** chooses which CE hosts use; each CE's BGP prefers its own default. Advanced: iBGP between ce1/ce2—optional stretch goal.

PE1 / PE2 sketch

```
hostname pe1
!
interface lo
 ip address 1.1.1.1/32
!
interface eth1
 ip address 172.16.10.2/30
!
router bgp 65010
 bgp router-id 1.1.1.1
 neighbor 172.16.10.1 remote-as 65001
 !
 address-family ipv4 unicast
  network 0.0.0.0/0
  network 1.1.1.1/32
  neighbor 172.16.10.1 route-map FROM-CE in
  neighbor 172.16.10.1 route-map TO-CE out
 exit-address-family
!
ip prefix-list CE-ALLOW seq 5 permit 192.168.50.0/24
ip prefix-list CE-ALLOW seq 10 deny any
!
```

```
route-map FROM-CE permit 10
 match ip address prefix-list CE-ALLOW
route-map FROM-CE deny 20
!
ip prefix-list ORIGINATE seq 5 permit 0.0.0.0/0
ip prefix-list ORIGINATE seq 10 permit 1.1.1.1/32
route-map TO-CE permit 10
 match ip address prefix-list ORIGINATE
!
line vty
```

pe2 mirrors with AS 65020, `2.2.2.2`, `172.16.20.2`.

Hijack drill: temporarily add `network 203.0.113.0/24` on CE—PE must **not** install it.

Deploy and baseline verify

```
sudo containerlab deploy -t dualhome.clab.yml
# bridge hook as above

docker exec clab-dualhome-ce1 vtysh -c 'show vrrp'
docker exec clab-dualhome-ce2 vtysh -c 'show vrrp'
docker exec clab-dualhome-ce1 vtysh -c 'show bgp summary'
docker exec clab-dualhome-ce1 vtysh -c 'show ip route'
docker exec clab-dualhome-pe1 vtysh -c 'show bgp ipv4 uni'
docker exec clab-dualhome-h1 ping -c 3 192.168.50.254
docker exec clab-dualhome-h1 ping -c 3 1.1.1.1
docker exec clab-dualhome-h1 traceroute -n 1.1.1.1
```

Predict (baseline)

- ce1 VRRP Master
- h1 reaches VIP and pe1 loopback via ce1
- pe1 has 192.168.50.0/24 only from CE
- pe2 may also have site prefix if ce2 session up

Drill 1 — Primary PE session loss

```
docker exec clab-dualhome-ce1 ip link set eth2 down
sleep 2
docker exec clab-dualhome-ce1 vtysh -c 'show bgp summary'
docker exec clab-dualhome-h1 ping -c 20 2.2.2.2
docker exec clab-dualhome-h1 traceroute -n 2.2.2.2
```

Predict → observe → fix

Predict	If false
Host still uses ce1 VIP (VRRP unchanged)	—
ce1 loses default → traffic blackholes unless ce2 takes VIP or ce1 has alternate	Add static floating / iBGP / kill VRRP master

Important teaching moment: if only ce1 is master and its uplink dies, **VRRP does not care** unless you track the uplink (BFD/link tracking) or lower priority. Options:

1. Shut ce1 LAN to force VRRP failover (crude).
2. Configure track/priority decrement when eth2 down (if supported).
3. Run dual default via both CEs with host-side multipath (rare).

Document which design you chose. Professional edges **tie FHRP priority to uplink health**.

Drill 2 — Force VRRP failover

```
docker exec clab-dualhome-ce1 ip link set eth1 down
sleep 3
docker exec clab-dualhome-ce2 vtysh -c 'show vrrp'
docker exec clab-dualhome-h1 ip neigh flush dev eth1
docker exec clab-dualhome-h1 ping -c 10 2.2.2.2
docker exec clab-dualhome-h1 traceroute -n 2.2.2.2
```

Predict

ce2 Master; path via pe2; short disruption.

Drill 3 — Policy rejection

```
# on ce1 temporarily
docker exec -it clab-dualhome-ce1 vtysh
# network 203.0.113.0/24 under bgp + dummy interface address if needed
```

```
docker exec clab-dualhome-pe1 vtysh -c 'show bgp ipv4 uni 203.0.113.0/24'
```

Predict

Empty / not best—filter works. If accepted, fix FROM-CE prefix-list.

Drill 4 — Name/time smoke (hygiene)

Optional: add dnsmasq A record `vip.site.lab → 192.168.50.254` and dig from h1; confirm `date -u` roughly matches host.

verify.sh

```
#!/usr/bin/env bash
set -euo pipefail
fail() { echo "FAIL: $*" >&2; exit 1; }

docker exec clab-dualhome-h1 ping -c 2 -W 1 192.168.50.254 || fail "vip"
docker exec clab-dualhome-pe1 vtysh -c 'show bgp ipv4 uni' | grep -q '192.168.50.0' \
  || fail "pe1 missing site"
# hijack should be absent
if docker exec clab-dualhome-pe1 vtysh -c 'show bgp ipv4 uni' | grep -q '203.0.113.0'; then
  fail "hijack accepted"
fi
echo OK
```

Definition of done (this lab)

- ☐ Topology + configs in git
- ☐ Addressing plan + diagram
- ☐ VRRP master/backup documented

☐ eBGP policies on CE and PE

☐ Three drills with notes (PE loss, VRRP fail, hijack)

☐ `verify.sh` green on clean deploy

☐ Written note: how uplink loss interacts with VRRP

Optional SR Linux PE

Replace pe1 with community SR Linux kind in Containerlab; keep CE on FRR. Same BGP objects: neighbor, import prefix allow-list, export default. Compare `show bgp` dialects in your journal—models stay identical.

Common mistakes

Mistake	Symptom
No bridge on LAN	VRRP never forms
Host uses .2 not VIP	No HA
PE accepts any prefix	Hijack succeeds
Uplink down but VRRP stays on blind CE	Blackhole
Asymmetric return without pe routes	One-way

Summary

- Dual-home labs combine **multihoming policy** and **first-hop HA**—design their interaction
- FRR eBGP + prefix-lists + VRRP are enough for a portfolio-quality edge
- Always test **policy negatives** (hijack) and **uplink vs VIP** failures separately
- Prefer link tracking or clear runbooks when master loses upstream
- This lab maps to capstone C2 in the ops part

Next part: **policy, QoS vocabulary, and hardening**—filters beyond BGP, control-plane protection, and checklists.

DNS Authoritative and Recursive

DNS Authoritative and Recursive

Name resolution fails more often than forwarding. This chapter treats **DNS roles**—stub, recursive, authoritative—as lab-first architecture: who answers from zone data, who walks the tree, what gets cached, and how operators verify SOA/NS/glue and DNSSEC with `dig`. Tooling stays free Linux containers in Containerlab (mid-2026 path): CoreDNS for authoritative lab zones, Unbound for recursion/validation, `bind-tools` / `dig` for truth.

Learning goals

By the end of this chapter you can:

- Draw the stub → recursive → authoritative path and predict failure symptoms
- Distinguish authoritative answers (AA) from referrals and cached recursed answers
- Explain SOA, NS, glue, and why in-bailiwick names need A/AAAA glue
- Run a two-node lab: CoreDNS authoritative + Unbound recursive
- Verify with `dig` (+trace, +norecurse, DNSSEC flags) and a short checklist
- State what DNSSEC validates for an operator (and what it does not)

Why roles matter

Role	Answers from	Typical lab tool
Stub resolver	Asks one or two configured servers	`/etc/resolv.conf`
Recursive resolver	Cache, else iterative queries toward authority	Unbound
Authoritative server	Local zone data (primary/secondary)	CoreDNS, BIND, Knot, PowerDNS

Mixing roles on one daemon is possible (classic BIND "do everything") but **lab clarity** comes from splitting: one box owns `lab.example`, another recurses for clients.

```
Application
  → stub (/etc/resolv.conf)
      → recursive (Unbound)
          → root / TLD / child authoritative (or lab fake root)
```

On the public Internet the recursive walks from the **root**. In a closed lab you either:

- Forward Unbound to a reachable upstream recursive, or
- Serve a private root / stub zones that point at your authoritative.

This chapter uses a **private zone** plus Unbound **stub-zone** / forward to the CoreDNS auth IP—no need for a full root mirror.

Concepts — recursive vs authoritative

Authoritative

Configured for a set of zones. Answers from local data **without** querying others for those names. Flags **AA** (Authoritative Answer) when the response is from that zone's data.

Primary vs secondary: secondaries still answer authoritatively once they hold a zone copy (IXFR/AXFR). Delegation in the *parent* does not by itself make a server authoritative—you must configure the zone.

Recursive

Receives queries from stubs (RD=1). Satisfies from **cache** when possible; otherwise starts at a configured root or forwarder and follows **referrals** until an authoritative answer is obtained, then returns it to the client (usually with AA clear on the final client-facing message, AD set if DNSSEC validation succeeded—see below).

Stub

Library or OS resolver: almost no policy. Points at recursive IPs. Failures here look like "DNS is down" even when the path to 8.8.8.8 is fine but `resolv.conf` points at a black hole inside the namespace.

Record	Operator meaning

Concepts — SOA, NS, glue

Record	Operator meaning
SOA	Apex start-of-authority: primary name, contact, serial, refresh/retry/expire/minimum (negative TTL flavor)
NS	Nameserver hostnames for the zone (apex) or delegation (parent)
Glue	A/AAAA for NS hostnames that lie **inside** the child zone (in-bailiwick)—parent must supply them in additional data or resolution cannot start

```
Parent (.example) referral for lab.example:
  NS ns1.lab.example.
  Additional: ns1.lab.example. A 10.50.0.53   ← glue

Child apex (authoritative):
  lab.example. SOA ...
  lab.example. NS ns1.lab.example.
  ns1.lab.example. A 10.50.0.53
```

Parent NS/glue and child apex NS should agree. Mismatches are a classic "works from some resolvers" outage (parent-centric vs child-centric resolver behavior differs; treat consistency as hygiene).

Concepts — caching

Recursors cache positive answers and negative answers (NXDOMAIN / NODATA) within TTL / SOA minimum policies. Operator effects:

- Lower TTL → faster change pickup, more query load on auth
- Raise TTL carefully on stable records
- Flush cache after lab zone edits when testing (`unbound-control flush` or restart)

Authoritative servers may also cache *nothing* for their own zones—they serve configured data. Do not expect editing a zone file on auth to update a recursive client until TTL expires or cache is flushed.

Concepts — DNSSEC at operator level

DNSSEC adds authenticity and integrity (not confidentiality). Chain of trust:

```
Trust anchor (root DS/DNSKEY in validator)
  → DS at parent points to child DNSKEY
    → RRSIGs over RRSets in the child
```

What to verify as an operator:

Check	Tooling idea
Validator has a current trust anchor	Unbound `auto-trust-anchor-file` / system anchors
`AD` bit on answers you care about	`dig +dnssec example.com` from the recursive
Broken chain symptoms	`SERVFAIL` from validating resolver; dig from non-validating still "works"
Delegation consistency	Parent **DS** matches child **DNSKEY**; NS/glue still required

Lab note: signing a private `lab.example` needs a trust anchor you install in Unbound, or you practice DNSSEC against public zones from a recursive that can reach the Internet. Do not confuse **AD** (Authenticated Data from a validator) with **AA** (Authoritative Answer from a zone server).

Addressing plan (DNS roles lab)

```
client ---- (L2) ---- recur ---- (L2) ---- auth
               10.50.0.0/24
```

Node	IP	Role
auth	10.50.0.53/24	CoreDNS authoritative for `lab.example`
recur	10.50.0.54/24	Unbound recursive; stub-zone → auth
client	10.50.0.10/24	Stub → 10.50.0.54

Topology YAML

```yaml
name: dns-roles

topology:
  nodes:
    auth:
      kind: linux
```

```
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils bind-tools
        # Install coredns binary via your preferred lab method
        # (apk package, static binary bind-mount, or distro package).
    recur:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils bind-tools unbound
    client:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils bind-tools
  links:
    - endpoints: ["auth:eth1", "recur:eth1"]
    - endpoints: ["recur:eth2", "client:eth1"]
```

Adjust images if you standardize on a CoreDNS-bearing image. The verification culture below is what matters more than the exact package name.

Authoritative zone (CoreDNS sketch)

Zone file `lab.example.db`:

```
$ORIGIN lab.example.
$TTL 60
@   IN SOA ns1.lab.example. hostmaster.lab.example. (
        2026100301 ; serial
        300        ; refresh
        60         ; retry
        86400      ; expire
        60         ; minimum
)
@       IN NS  ns1.lab.example.
ns1     IN A   10.50.0.53
www     IN A   10.50.0.80
api     IN A   10.50.0.81
```

Corefile:

```
lab.example:53 {
    file /etc/coredns/lab.example.db
    log
```

```
    errors
}
```

Bring-up sketch:

```
docker exec clab-dns-roles-auth ip addr add 10.50.0.53/24 dev eth1
docker exec clab-dns-roles-auth ip link set eth1 up
# copy zone + Corefile, then:
docker exec clab-dns-roles-auth coredns -conf /etc/coredns/Corefile
```

Direct auth checks (no recursion)

```
dig @10.50.0.53 www.lab.example A +norecurse
dig @10.50.0.53 lab.example SOA +norecurse
dig @10.50.0.53 lab.example NS +norecurse
```

Expect **AA** on answers for names in zone. Querying a name outside the zone should fail closed (REFUSED/NXDOMAIN per config)—auth should **not** recurse unless you deliberately enable it.

Recursive (Unbound) sketch

`/etc/unbound/unbound.conf` fragment:

```
server:
  interface: 0.0.0.0
  access-control: 10.50.0.0/24 allow
  do-ip6: no
  # For public DNSSEC labs, keep trust anchors enabled.
  # auto-trust-anchor-file: "/usr/share/dnssec-root/root.key"

stub-zone:
  name: "lab.example"
  stub-addr: 10.50.0.53

forward-zone:
  name: "."
  # Optional: only if auth/private root is not enough and egress exists
  # forward-addr: 1.1.1.1
```

```
docker exec clab-dns-roles-recur ip addr add 10.50.0.54/24 dev eth1
docker exec clab-dns-roles-recur ip link set eth1 up
docker exec clab-dns-roles-recur unbound -d -c /etc/unbound/unbound.conf &
```

Client stub:

```
docker exec clab-dns-roles-client sh -c 'cat >/etc/resolv.conf <<EOF
nameserver 10.50.0.54
search lab.example
EOF'
```

dig verification workflows

1. Happy path via recursive

```
docker exec clab-dns-roles-client dig www.lab.example A +noall +answer +comments
```

Predict: answer `10.50.0.80`, from recursive (AA typically unset on the client-facing message).

2. Prove auth is the source of truth

```
dig @10.50.0.53 www.lab.example A +norecurse
dig @10.50.0.54 www.lab.example A
```

Change the zone A record / bump SOA serial / reload CoreDNS, then:

```
# may still see old data until TTL expires
dig @10.50.0.54 www.lab.example A
# flush and retest (command name varies by package)
docker exec clab-dns-roles-recur unbound-control flush www.lab.example
```

3. Trace mindset (public names)

When the recursive has real root access:

```
dig example.com +trace
```

Shows iterative referrals. In a closed lab, `+trace` may fail at root—use stub-zone labs instead of forcing a fake public trace.

4. Glue / NS consistency

```
dig @10.50.0.53 lab.example NS +norecurse
dig @10.50.0.53 ns1.lab.example A +norecurse
```

If you later add a parent zone, inspect **additional** section glue on the referral.

5. DNSSEC operator glance (public)

```
dig @10.50.0.54 cloudflare.com A +dnssec
# Look for ad flag when validation works; SERVFAIL when chain breaks
```

Compare with a non-validating query path if you temporarily disable validation—difference isolates DNSSEC vs reachability.

Predict → observe → fix

Scenario A — client NXDOMAIN for `www.lab.example`

- Predict: wrong stub IP, auth down, or typo in zone
- Observe: `dig @10.50.0.54`, then `dig @10.50.0.53 +norecurse`
- Fix: restore auth listening; fix zone; fix resolv.conf

Scenario B — auth works, recursive SERVFAIL

- Predict: stub-zone missing, auth unreachable from recur, or DNSSEC failure
- Observe: `dig @10.50.0.53` OK; `dig @10.50.0.54` SERVFAIL; check Unbound log
- Fix: routing between recur auth; stub-addr; trust anchor only if validating private zones incorrectly

Scenario C — "update not visible"

- Predict: recursive cache / TTL
- Observe: auth shows new A; recur shows old
- Fix: wait TTL, lower lab TTL, or flush cache

Scenario D — AA vs AD confusion

- Predict: operator expects AD on auth dig
- Observe: `dig @auth` has AA, not AD
- Fix: AD comes from a **validating recursive**, not from AA

Verification checklist

Check	Intent
`dig @auth SOA`	AA + expected serial
`dig @auth NS` + glue A	Apex NS resolve
`dig @recur name`	Client path works
Cache flush drill	TTL behavior understood
resolv.conf	Stub points at recur, not auth (unless intentional)
Logs	CoreDNS/Unbound query logs on failure
DNSSEC (optional)	AD on public name; document trust anchors

Common mistakes

Mistake	Symptom
Pointing stubs at authoritative only	Public names fail; private names OK
Enabling recursion on Internet-exposed auth	Abuse / cache poisoning risk
Missing glue for in-bailiwick NS	Delegation black hole
Forgetting SOA serial bump on secondaries	Stale secondaries (when you add transfers later)
Assuming AA means DNSSEC-valid	Conflates authority with authenticity
Lab resolv.conf → Docker host DNS unreachable	Intermittent or total failure inside netns

Summary

- **Authoritative** servers answer from zone data (AA); **recursives** cache and walk or stub toward authority
- Stubs are configuration—wrong `nameserver` lines create false "network down" stories
- SOA/NS/glue consistency is operational hygiene; dig `+norecurse` against auth is your ground truth
- DNSSEC: verify AD via a validating recursive and keep DS/DNSKEY aligned—do not confuse with AA
- Split CoreDNS (auth) and Unbound (recur) in Containerlab until the roles are muscle memory

Next edge labs can reuse this DNS path whenever VIP names and dual-home failovers need human-readable verification.

QUIC Version 2 and Compatible Version Negotiation

QUIC Version 2 and Compatible Version Negotiation

QUIC version 1 (RFC 9000) won the Internet. That success is also a risk: middleboxes that only ever see `0x00000001` in the long-header Version field, or that hard-code the version-1 Initial salt, will quietly break the next version. **QUIC version 2** (RFC 9369) is deliberately almost identical to version 1 — same streams, same TLS, same congestion control — with a handful of wire-image changes whose only job is to break that ossification and to exercise the negotiation machinery. **Compatible version negotiation** (RFC 9368) is how endpoints switch between v1 and v2 (or any pair of compatible versions) without paying an extra round trip. This chapter teaches the Version field, Compatible VN versus Incompatible VN, when v2 actually matters on the wire, and how to observe it in a modern browser stack.

Learning goals

By the end of this chapter you can:

- State what QUICv2 changes (and what it deliberately does *not* change)
- Decode a long-header Version field: `0x00000001` vs `0x6b3343cf`
- Explain Compatible VN vs Incompatible VN, and why v1 v2 is Compatible
- Predict which packets keep the Original Version and which must use the Negotiated Version
- Name the `version_information` transport parameter (0x11) and why v2 endpoints MUST use it
- Observe negotiated version in Firefox (Nightly 154+, all channels from 155) and reason about server support

Concepts — why a "version 2" that does nothing new?

RFC 9369 is explicit: QUICv2 exists to mitigate ossification and to exercise version negotiation. Application capabilities are unchanged. ALPN tokens defined for QUICv1 (`h3`, `doq`, …) also work over v2. Extensions that work with v1 work with v2 unless they say otherwise.

What *does* change:

Knob	QUICv1 (RFC 9000 / 9001)	QUICv2 (RFC 9369)
Long-header Version field	`0x00000001`	`0x6b3343cf`
Long-header Type bits	Initial `0b00`, 0-RTT `0b01`, Handshake `0b10`, Retry `0b11`	Initial `0b01`, 0-RTT `0b10`, Handshake `0b11`, Retry `0b00`
Initial salt	v1 salt	`0x0dede3def700a6db819381be6e269dcbf9bd2ed9`
HKDF labels	`quic key` / `quic iv` / `quic hp` / `quic ku`	`quicv2 key` / `quicv2 iv` / `quicv2 hp` / `quicv2 ku`
Retry Integrity Tag keying	v1 secret	v2 secret / key / nonce (RFC 9369 §3.3.3)

The version number is *not* the integer 2. It is the first four bytes of `sha256("QUICv2 version number")`, chosen randomly on purpose so middleboxes cannot guess "the next number". A provisional draft codepoint `0x709a50c4` also sits in the IANA registry for historical draft traffic.

```
Long header (RFC 8999 invariants):
  Header Form (1) = 1
  Fixed Bit    (1) = 1
  Long Packet Type (2)   ← different encoding in v2
  Type-Specific (4)
  Version (32)           ← 0x00000001 or 0x6b3343cf
  DCID Len / DCID / SCID Len / SCID / …
```

Concepts — Compatible VN vs Incompatible VN

RFC 9000 gave servers a Version Negotiation packet that says "I do not speak the version you chose", but it never defined a *safe* client reaction. RFC 9368 fills that hole with two mechanisms:

Incompatible VN. Server cannot parse the client's first flight. It sends a Version Negotiation packet listing Offered Versions. Client starts a *new* connection attempt with a mutually supported version. Cost: one extra RTT. Works between any versions.

Compatible VN. Server *can* parse the first flight, extracts the client's Version Information, and may switch to another version that (1) it supports and (2) it knows is compatible with the client's Chosen Version. It converts the first flight and replies as if that converted flight had arrived. Cost: zero extra RTT, but only when the versions share a convertible first-flight format.

RFC 9369 states that v2 is compatible with v1 and v1 is compatible with v2. Endpoints that speak both SHOULD use Compatible VN.

```
Compatible VN (happy path, client offers v1+v2, prefers v1 as Chosen):

  Client                                          Server
  Chosen=v1, Available=(v1,v2)  ---------------->
                               (server prefers v2, converts first flight)
                               <---------------- CRYPTO in v2 long headers
  (client sees Version 0x6b3343cf   Chosen) → Negotiated = v2
  subsequent Initial / Handshake / 1-RTT use v2
```

Version Information (transport parameter 0x11)

```
Version Information {
  Chosen Version (32),
  Available Versions (32) ...,
}
```

- **Client Available Versions:** every version this first flight is compatible with, preference order descending. Chosen Version MUST appear in the list.
- **Server Available Versions:** the deployment's Fully Deployed Versions (may be empty during rollouts; ordering carries no meaning).
- QUICv2 endpoints **MUST** send, process, and validate this parameter (downgrade defence). QUICv1 endpoints that only ever speak v1 may omit it; RFC 9368 §8 special-cases that.

Packet-version rules after a Compatible switch (v1 v2)

Packet class	Rule
Retry	Always Original Version; client must not switch version in the next Initial that carries the Retry token
Server Initial before client TP processed	May use Original Version
All CRYPTO frames from the server	Negotiated Version
Handshake and 1-RTT	MUST be Negotiated Version; other versions dropped
Client 0-RTT	MUST stay on Original Version even after seeing Negotiated Version

Packet class	Rule
Session tickets / NEW_TOKEN	Version-scoped; do not resume v1 tickets on a v2 connection (and vice versa). After Compatible VN, issued tokens belong to the *Negotiated* version

The client learns the Negotiated Version by watching the first long-header Version field that differs from the Original Version. If the server answers with CRYPTO still in the Original Version, negotiation landed on Original == Negotiated.

When v2 matters vs when it does not

Situation	Prefer
Fighting middleboxes that ossify on v1 Initial crypto / Version bytes	Offer v2; some clients start with v2 when they know the server supports it (accepts Incompatible VN cost on miss)
Maximising reach to old clients / Alt-Svc `h3` without version signalling	Keep speaking v1; RFC 9369 says HTTP servers SHOULD support multiple versions
Measuring deployment health	Log negotiated version per handshake (Firefox Glean: `networking.http_3_version`)
Application features (streams, datagrams, HTTP/3)	Irrelevant — v2 adds none

v2 does **not** deprecate v1. Treating "we enabled v2" as "we can drop v1" breaks clients that only offer v1, and Alt-Svc still does not distinguish versions.

Worked observation — Firefox and Compatible VN

As of mid-2026, Mozilla's neqo stack can offer QUICv2. Bug 2040450 enabled Compatible VN on **Firefox Nightly 154** via the pref `network.http.http3.version_negotiation.enabled` (Nightly-only at first). Bug 2059947 later set that pref to `true` on **all channels** (Firefox 155). When the pref is true, the client offers `Version::all()` (v2 preferred, then v1); when false, v1 only. The *server* still chooses:

```
Client offers: [v2, v1]     Server speaks only v1  → connection uses v1
Client offers: [v2, v1]     Server prefers v2        → Compatible VN → v2
Client offers: [v1]         Server prefers v2        → stays v1 (client never offered v2)
```

Try this (browser)

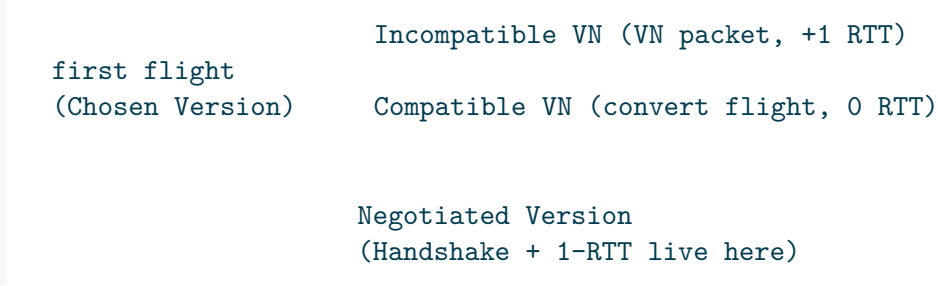
1. Firefox 155+ (or Nightly 154+ with the pref true).
2. Visit an HTTP/3 origin you control that advertises QUICv2 (or a public origin known to prefer v2).
3. `about:networking#httpconn` (or your TLS/QUIC debugger) — confirm HTTP/3 and note the negotiated QUIC version if exposed.
4. On Nightly builds that still gate the pref, toggle `network.http.http3.version_negotiation.enabled` and compare.

Server-side (conceptual checklist, library-specific):

- Advertise both version codepoints in Compatible VN.
- Emit `version_information` on every v2 (and dual-stack) handshake.
- Log Original vs Negotiated Version.
- Keep issuing version-scoped session tickets.
- Roll versions through Acceptable → Offered → Fully Deployed with 1 MSL between steps (RFC 9368 §5) so downgrade detection does not false-positive during fleet upgrades.

Libraries commonly cited for dual-version support include **quiche**, **quic-go**, and browser stacks built on **neqo** / NSS-adjacent HTTP/3 plumbing. Always check the *current* release notes for your pin — Compatible VN and v2 landed at different times in different stacks.

Mental model — one diagram



The trap

- **Assuming the Version field integer "2".** The wire value is `0x6b3343cf`. Fingerprinting scripts that look for `version == 2` will miss every real v2 packet.

- **Enabling v2 only on the client.** Without servers that select v2 from Available Versions, you negotiate v1 forever and learn nothing.
- **Reusing tickets across versions.** A v1 session ticket on a v2 attempt must be rejected (full handshake, no 0-RTT).
- **Skipping `version_information` on a v2 endpoint.** Mandatory; without it you lose downgrade protection and violate RFC 9369.
- **Expecting application-visible differences.** If your HTTP/3 metrics are identical on v1 and v2, that is success — the change is at the transport wire image.

The boring rule

Offer QUICv1 and QUICv2 together. Use Compatible VN so the upgrade is free when both sides agree. Keep v1 around for reachability. Authenticate negotiation with `version_information`. Measure which version you actually land on.

Sources

- RFC 9369 — QUIC Version 2: https://www.rfc-editor.org/rfc/rfc9369.html
- RFC 9368 — Compatible Version Negotiation for QUIC: https://www.rfc-editor.org/rfc/rfc9368.html
- RFC 9000 — QUIC: https://www.rfc-editor.org/rfc/rfc9000.html
- RFC 8999 — Version-Independent Properties of QUIC: https://www.rfc-editor.org/rfc/rfc8999.html
- Firefox Nightly 154 notes (Bug 2040450): https://www.firefox.com/en-US/firefox/154.0a1/releasenotes/
- Bug 2040450 — Enable QUIC version negotiation: https://bugzilla.mozilla.org/show_bug.cgi?id=2040450
- Bug 2059947 — Enable QUIC v2 VN on all channels (Firefox 155): https://bugzilla.mozilla.org/show_bug.cgi?id=2059947

Happy Eyeballs: Racing IPv6 and IPv4 (RFC 8305 and the v3 Draft)

Happy Eyeballs: Racing IPv6 and IPv4 (RFC 8305 and the v3 Draft)

A dual-stack name resolves to both AAAA and A records. If the client tries them one at a time, a single broken AAAA record (a typo in the zone, a blackholed prefix, a dead load balancer) costs every user the full TCP SYN timeout before IPv4 is even tried. **Happy Eyeballs** fixes this on the client: start with the preferred address, and if it has not answered within a short delay, start the next one *in parallel*. The first handshake to finish wins. Version 2 is RFC 8305 (2017). **Version 3** (`draft-ietf-happy-happyeyeballs-v3-04`, July 2026) folds in SVCB/HTTPS records, so the race now also chooses *protocols* (QUIC vs TCP, ECH or not), not just address families. This chapter builds the failure in a two-namespace lab, measures the race, and explains what v3 changes.

Learning goals

- Reproduce a broken AAAA record and measure its cost with and without Happy Eyeballs
- Read the race in a packet capture: one SYN, a 200–250 ms gap, a second SYN
- Tell a *silent* failure (drop) from a *loud* one (RST), and explain why only the silent one needs racing
- Name the v3 timers (Resolution Delay, Connection Attempt Delay, and its bounds) and say what SVCB adds
- Explain why Happy Eyeballs **hides** IPv6 outages, and add a per-family health check

Lab platform: Linux 6.12, iproute2, nftables, curl 8.14.1, Python 3.13 (`asyncio`), tcpdump. All commands need root. Every output below is pasted from a run.

Concepts in one page

```
client (cli)  2001:db8::20 / 192.0.2.20          server (srv)  one veth
                                                  2001:db8::10   healthy AAAA
         "DNS" (hosts)                            2001:db8::bad  SYN silently DROPPED
         www.lab AAAA 2001:db8::bad               2001:db8::dead SYN answered with RST
         www.lab A    192.0.2.10                  192.0.2.10     healthy A
                                                  http.server :8080 on [::] (dual-stack)

HEv2/v3 connection race:   t=0     SYN → preferred (IPv6)
                           t=Δ     no answer yet → SYN → next family (IPv4)   Δ   250 ms (re
                           first SYN-ACK wins, the others are cancelled
```

Build the lab

```
#!/usr/bin/env bash
# lab.sh - a dual-stack server whose second IPv6 address silently eats SYNs
set -euo pipefail
ip netns add srv; ip netns add cli
ip link add s0 netns srv type veth peer name c0 netns cli
ip -n srv addr add 192.0.2.10/24 dev s0
ip -n srv addr add 2001:db8::10/64 dev s0 nodad      # healthy AAAA
ip -n srv addr add 2001:db8::bad/64 dev s0 nodad     # broken AAAA (blackholed below)
ip -n srv addr add 2001:db8::dead/64 dev s0 nodad    # refused AAAA (RST below)
ip -n cli addr add 192.0.2.20/24 dev c0
ip -n cli addr add 2001:db8::20/64 dev c0 nodad
for n in srv cli; do ip -n $n link set lo up; done
ip -n srv link set s0 up; ip -n cli link set c0 up
ip netns exec srv /usr/sbin/nft -f - <<'NFT'
table inet lab {
  chain input {
    type filter hook input priority 0; policy accept;
    ip6 daddr 2001:db8::bad tcp dport 8080 drop
    ip6 daddr 2001:db8::dead tcp dport 8080 reject with tcp reset
  }
}
NFT
ip netns exec srv python3 -m http.server 8080 --bind :: >/dev/null 2>&1 &
sleep 1
```

```
# ./lab.sh
# mkdir -p /etc/netns/cli
# printf '2001:db8::bad www.lab\n192.0.2.10 www.lab\n' > /etc/netns/cli/hosts
```

`ip netns exec cli` bind-mounts `/etc/netns/cli/hosts` over `/etc/hosts`, so inside the client namespace `www.lab` resolves to the broken AAAA first and the healthy A second. That is the same order RFC 6724 address selection gives for a real dual-stack name.

Drill 1: three kinds of AAAA, one client that races

curl implements Happy Eyeballs with a default 200 ms head start for the first family. `--resolve` lets us swap the AAAA without touching the hosts file:

```
# C() { ip netns exec cli curl -s -o /dev/null -w '%{remote_ip} connect=%{time_connect}s\n'
# for a in 2001:db8::10 2001:db8::bad 2001:db8::dead; do echo "== AAAA $a"; $C --resolve "ww
== AAAA 2001:db8::10
2001:db8::10 connect=0.000232s
== AAAA 2001:db8::bad
```

```
192.0.2.10 connect=0.200484s
== AAAA 2001:db8::dead
192.0.2.10 connect=0.000122s
```

A healthy AAAA wins at once. A **refused** AAAA (RST) fails immediately, so IPv4 starts without waiting. A **silent** AAAA costs exactly the head start, about 200 ms, and then IPv4 wins. That cost is the price of preferring IPv6, and it is bounded.

The head start is a tunable, and it is the whole cost:

```
# $C --happy-eyeballs-timeout-ms 1000 --resolve 'www.lab:8080:[2001:db8::bad],192.0.2.10' ht
192.0.2.10 connect=1.001154s
```

Drill 2: the same name, a client that does not race

```
# seq.py - what a client without Happy Eyeballs does: try each address in order
import socket, time
addrs = [("2001:db8::bad", 8080), ("192.0.2.10", 8080)]
t0 = time.monotonic()
for host, port in addrs:
    fam = socket.AF_INET6 if ":" in host else socket.AF_INET
    s = socket.socket(fam, socket.SOCK_STREAM); s.settimeout(5)
    try:
        s.connect((host, port)); print(f"connected {host} after {time.monotonic()-t0:.3f}s")
    except OSError as e:
        print(f"{host}: {e!r} after {time.monotonic()-t0:.3f}s")
    finally:
        s.close()
```

```
# ip netns exec cli python3 seq.py
2001:db8::bad: TimeoutError('timed out') after 5.005s
connected 192.0.2.10 after 5.006s
```

The sequential client pays its *whole* connect timeout. Here that is 5 s because the script sets it. A client with no timeout pays the kernel's SYN retry budget (`net.ipv4.tcp_syn_retries`, about two minutes by default). This is the outage users report as "the site is slow, but only for some people".

`asyncio` ships both behaviours. Only one of them is on by default:

```
# he.py - asyncio with and without Happy Eyeballs against the broken name
import asyncio, socket, sys, time

async def attempt(delay):
    t0 = time.monotonic()
    try:
```

```
        r, w = await asyncio.wait_for(
            asyncio.open_connection("www.lab", 8080, happy_eyeballs_delay=delay), 3)
        peer = w.get_extra_info("peername")[0]
        w.close(); await w.wait_closed()
        return f"connected to {peer} in {time.monotonic()-t0:.3f}s"
    except TimeoutError:
        return f"gave up after {time.monotonic()-t0:.3f}s"

async def main():
    for _, _, _, _, sa in socket.getaddrinfo("www.lab", 8080, type=socket.SOCK_STREAM):
        print("resolver order:", sa[0])
    print("happy_eyeballs_delay=None :", await attempt(None))
    print("happy_eyeballs_delay=0.25 :", await attempt(0.25))

asyncio.run(main())
```

```
# ip netns exec cli python3 he.py
resolver order: 2001:db8::bad
resolver order: 192.0.2.10
happy_eyeballs_delay=None : gave up after 3.004s
happy_eyeballs_delay=0.25 : connected to 192.0.2.10 in 0.252s
```

`happy_eyeballs_delay=None` (the default for `open_connection` and `loop.create_connection`) is sequential. Passing `0.25`, the RFC 8305 recommended value, turns the race on. If you write connection code on top of a socket API, check which of the two you are getting.

Drill 3: read the race on the wire

```
# ip netns exec cli tcpdump -i c0 -n -tt 'tcp[tcpflags] & tcp-syn != 0 or (ip6 and ip6[53] &
# ip netns exec cli curl -s -o /dev/null http://www.lab:8080/
+0.000s IP6 2001:db8::20.43750 > 2001:db8::bad.8080: Flags [S], seq 329770494, win 64800, …
+0.201s IP 192.0.2.20.58954 > 192.0.2.10.8080: Flags [S], seq 1595341819, win 64240, …
+0.201s IP 192.0.2.10.8080 > 192.0.2.20.58954: Flags [S.], seq 694257496, ack 1595341820, …
```

(Timestamps are rebased to the first SYN.) One IPv6 SYN goes out and gets no reply. 201 ms later the IPv4 SYN goes out and is answered in microseconds. When you suspect Happy Eyeballs is masking something, this three-line pattern is what you look for in a client-side capture.

What version 3 changes

HEv2 raced *addresses*. The v3 draft races *endpoints*: an address **plus** the protocol and security properties learned from SVCB/HTTPS records (RFC 9460). The draft's main mechanics are:

Stage	v3 behaviour	Default
Queries	Send HTTPS/SVCB, then AAAA, then A, back to back	—
Resolution Delay	After A arrives first, wait briefly for AAAA and SVCB	50 ms
Grouping	Group by ALPN/ECH, then by SVCB priority, then sort by RFC 6724 and RTT history, interleaving families	Preferred Address Family Count 1
Connection Attempt Delay	Gap between attempts, ideally the first SYN/Initial retransmit time	250 ms (min 100 ms, never < 10 ms; max 2 s)
Success	TCP-only: handshake done. TCP+TLS: wait for TLS. QUIC: its one handshake	—
IPv6-only	Synthesise NAT64 addresses from PREF64 (RFC 8781) for IPv4 literals and A answers	Last-resort synthesis delay 2 s

Two practical consequences for operators. First, an HTTPS record with `alpn="h3"` and `ipv6hint` lets a v3 client start a QUIC attempt *before* the AAAA answer arrives. A wrong hint is therefore a new way to publish a broken address. Second, with encrypted or DNSSEC-validated DNS and ECH support, clients **must wait** for the SVCB answer before sending a ClientHello. A slow HTTPS-record path then becomes connection latency, not just a missed optimisation.

The trap

Happy Eyeballs makes a broken IPv6 path *look healthy*. Drill 1 is a 100% IPv6 outage that shows up as a 200 ms slowdown. RFC 8305 and the v3 draft both warn that the race hides operational problems. Monitoring that connects "to the name" will stay green. Probe each family on its own:

```
# ip netns exec cli curl -s -o /dev/null --connect-timeout 3 -6 http://www.lab:8080/; echo "
exit=28
# ip netns exec cli curl -s -o /dev/null --connect-timeout 3 -4 http://www.lab:8080/; echo "
exit=0
```

Exit 28 is curl's timeout. Alert on `-6` and `-4` separately.

The boring rule

Clients race: use a library that implements Happy Eyeballs, or turn it on (`happy_eyeballs_delay=0.25`). Servers publish only addresses they can serve, *including* SVCB `ipv6hint`/`ipv4hint`. Monitoring never trusts the race and probes IPv6 and IPv4 separately. A silent drop is the failure to design for, because an RST is already fast.

Try this

1. Change the nftables rule for `2001:db8::bad` from `drop` to `reject with icmpv6 type admin-prohibited`. Predict the curl `connect=` time first, then run it.
2. Put two broken AAAA records before the A record in `--resolve`. Does curl try the second IPv6 address before falling back to IPv4? Compare with the v3 "interleave families" rule.
3. Set `net.ipv4.tcp_syn_retries=1` in `cli` and rerun `seq.py` with `settimeout(None)`. How long does the sequential client wait now?
4. Write the HTTPS record you would publish for `www.lab` with `alpn="h3,h2"` and both hints. Which v3 group does each address land in?

Sources

- RFC 8305, *Happy Eyeballs Version 2: Better Connectivity Using Concurrency*: https://www.rfc-editor.org/rfc/rfc8305
- draft-ietf-happy-happyeyeballs-v3-04 (July 2026): https://datatracker.ietf.org/doc/draft-ietf-happy-happyeyeballs-v3/
- RFC 9460, SVCB and HTTPS resource records: https://www.rfc-editor.org/rfc/rfc9460
- RFC 6724, Default Address Selection for IPv6: https://www.rfc-editor.org/rfc/rfc6724
- curl `--happy-eyeballs-timeout-ms`: https://curl.se/docs/manpage.html
- Python `asyncio loop.create_connection(happy_eyeballs_delay=…)`: https://docs.python.org/3/library/asyncio-eventloop.html

QUIC Load Balancing with nftables: Routing on Connection IDs

QUIC Load Balancing with nftables: Routing on Connection IDs

A TCP load balancer can lean on the kernel. Pick a backend on the SYN, DNAT it, and conntrack keeps every later segment of that 4-tuple on the same backend. QUIC breaks both assumptions behind that recipe. A QUIC connection is named by its **connection ID** (CID), not by its 4-tuple. A client can change its address and port mid-connection (migration, or a NAT rebinding it never notices). Two connections can also share one 4-tuple. A load balancer that routes on the 4-tuple sends a migrated connection to a server that has never heard of it. This chapter builds a QUIC load balancer from nothing but nftables rules: no proxy, no kernel module, no eBPF. It routes **handshakes** by client address and **established connections** by a server ID that each backend writes into every CID it issues. Then it drains a backend, deregisters it, and shows the one return-path rule that makes the whole thing look fine in a lab and fail at the edge.

Learning goals

- Read the QUIC wire image a load balancer is allowed to rely on (RFC 8999) and find the CID with `@th` raw payload matches
- Explain why the ruleset needs **conntrack zones**, not just DNAT, and show what breaks without them
- Encode a server ID in the CID the way QUIC-LB lays it out, and route on it
- Drain and deregister a backend with nothing but map element edits, and know what the client sees afterwards
- Explain why every backend must share the **stateless reset** key
- Put the return-path zone rule in the right hook, and prove it with an edge filter

Lab platform: Linux 6.12, nftables 1.1.3, iproute2, Python 3.13, Go 1.27.2 with quic-go v0.63.0, tcpdump 4.99.5. All commands need root. Every output below is the captured output of one run of the listings shown.

Mental model

```
clients 198.51.100.10-17          load balancer (qlb)                    backends (qbe)
              l0     VIP 198.51.100.1:443     l1     be1 10.0.0.11:8080  server ID 0x01
                                                                    be2 10.0.0.12:8081  serv
                                                                    be3 10.0.0.13:8082  serv
                                                                    be4 10.0.0.14:8083  (joi
```

```
prerouting, priority raw (-300): decide the backend, put it in the packet mark, then in a c
  long header (first byte & 0x80)       → @flows[client IP]  or  random slot → @slots → bac
  short header, DCID[0] & 0xe0 == 0     → @sids[DCID[1]]  (server ID)       → backend ID
  short header, unknown server ID       → treated like a new flow
  from a backend (iif l1)               → @replies[saddr . sport]           → backend ID
  ct zone = backend ID
prerouting, conntrack (-200): look up / create the entry *inside that zone*
prerouting, priority dstnat (-100): new entry → dnat to @backends[zone]
postrouting, srcnat (100): replies get the VIP back as their source
```

Three facts carry the design:

1. **What a middlebox may parse.** RFC 8999 (QUIC invariants) fixes, for every QUIC version, the high bit of the first byte (1 = long header, 0 = short header), the long header's version and CID length fields, and the fact that a short header's Destination CID starts right after the first byte. A short header does *not* carry the CID length. RFC 9000 §5.1 says a load balancer that routes on CIDs has to agree on a fixed length or an encoding with the servers. Nothing else in the packet is readable: the rest is encrypted.
2. **Who chooses CIDs.** The client's first Initial carries a random DCID of at least 8 bytes (RFC 9000 §7.2). The server answers with a CID *it* chose, and from then on the client sends to the server's CIDs. So handshake packets cannot be routed by CID. Everything after the handshake can be, if the server encodes "it's me" into its CIDs.
3. **Why zones.** NAT is decided once, on the first packet of a conntrack entry. Every later packet with the same 4-tuple follows that entry, whatever its CID says. Putting each backend in its own conntrack zone gives the same 4-tuple one entry *per backend*, so the CID decides again on every packet. A reply has to be looked up in the same zone, which is why the return path needs its own rule.

In nftables, `@th,OFFSET,LEN` reads raw bits relative to the transport header. A UDP header is 8 bytes (64 bits), so:

Match	Bits	What it is in a short-header packet
`@th,64,8`	UDP payload byte 0	QUIC first byte; `& 0x80` is the header form
`@th,72,8`	payload byte 1	DCID byte 0 (QUIC-LB "first octet": 3 config rotation bits + 5 bits)
`@th,80,8`	payload byte 2	DCID byte 1: the server ID

The CID layout

QUIC-LB (`draft-ietf-quic-load-balancers-21`, the QUIC working group's draft for routable CIDs) reserves the first CID octet. Its top three bits are a **config rotation** codepoint, and `0b111` means "unroutable" (§3). The server ID follows, then a nonce of at least 4 bytes (§5.2). This lab uses the draft's *unencrypted* layout with an 8-byte CID: `[000 + 5 random bits][server ID][6`

`random bytes]`. The load balancer reads one byte at a fixed offset. That is exactly what nftables is good at.

Build the lab

```
#!/usr/bin/env bash
# lab.sh - clients, one load balancer, and the backends, in three namespaces
set -euo pipefail
for n in qcli qlb qbe; do ip netns add $n; ip -n $n link set lo up; done
ip link add c0 netns qcli type veth peer name l0 netns qlb       # client side
ip link add l1 netns qlb  type veth peer name b0 netns qbe       # backend side
for a in 10 11 12 13 14 15 16 17; do ip -n qcli addr add 198.51.100.$a/24 dev c0; done
ip -n qlb addr add 198.51.100.1/24 dev l0                        # the VIP
ip -n qlb addr add 10.0.0.1/24 dev l1
for a in 11 12 13 14; do ip -n qbe addr add 10.0.0.$a/24 dev b0; done   # be1..be4
ip -n qcli link set c0 up; ip -n qlb link set l0 up; ip -n qlb link set l1 up; ip -n qbe lin
ip -n qcli route add default via 198.51.100.1
ip -n qbe route add default via 10.0.0.1
ip netns exec qlb sysctl -qw net.ipv4.ip_forward=1
```

The load balancer's whole configuration is one file. Six maps hold every per-backend fact, so adding, draining and removing a backend are element edits, never rule edits:

```
# lb-quic.nft - route QUIC by connection ID with nftables alone
table ip lb-quic {
    map slots {                    # new connections: random slot -> backend ID
        typeof meta mark : meta mark
        elements = { 0 : 1, 1 : 2, 2 : 3 }
    }

    map sids {                     # server ID byte in the DCID -> backend ID
        typeof @th,80,8 : meta mark
        elements = { 0x01 : 1, 0x02 : 2, 0x03 : 3 }
    }

    map flows {                    # client IP -> backend ID while handshakes run
        typeof ip saddr : meta mark
        size 65536
        flags dynamic,timeout
        timeout 5s
    }

    map backends {                 # conntrack zone -> backend address
        typeof ct zone : ip daddr . udp dport
        elements = { 1 : 10.0.0.11 . 8080, 2 : 10.0.0.12 . 8081, 3 : 10.0.0.13 . 8082 }
```

```
    }

    map replies {                   # backend address -> backend ID (return path)
        typeof ip saddr . udp sport : meta mark
        elements = { 10.0.0.11 . 8080 : 1, 10.0.0.12 . 8081 : 2, 10.0.0.13 . 8082 : 3 }
    }

    map zone-of {                   # backend ID -> conntrack zone
        typeof meta mark : ct zone
        elements = { 1 : 1, 2 : 2, 3 : 3, 4 : 4, 5 : 5, 6 : 6 }
    }

    chain raw {
        type filter hook prerouting priority raw; policy accept;
        iifname "l1" meta mark set ip saddr . udp sport map @replies
        iifname "l0" udp dport 443 @th,64,8 & 0x80 == 0x00 jump short-header
        iifname "l0" udp dport 443 @th,64,8 & 0x80 == 0x80 jump new-flow
        ct zone set meta mark map @zone-of
    }

    chain short-header {            # DCID = [config rotation 000|5 bits][server ID][nonce]
        @th,72,8 & 0xe0 != 0x00 goto new-flow
        meta mark set @th,80,8 map @sids return
        goto new-flow
    }

    chain new-flow {                # long header, or a CID no registered backend owns
        meta mark set ip saddr map @flows return
        meta mark set numgen random mod 3
        meta mark set meta mark map @slots update @flows { ip saddr : meta mark }
    }

    chain to-backend {
        type nat hook prerouting priority dstnat; policy accept;
        meta l4proto udp dnat ip to ct zone map @backends
    }
}
```

Two details are deliberate:

- `short-header` starts with `goto`, not `jump`, for unroutable config rotation bits. A `jump` would return into `short-header` and keep evaluating its rules.
- The packet mark carries the **backend ID**, and `zone-of` turns it into a zone at the end of `raw`. Handshake pins in `@flows` store the backend ID, not the random slot, so a pinned handshake keeps its backend while the slots are remapped.

Two probe scripts stand in for clients and servers before real QUIC arrives. `qsend.py` sends one QUIC-shaped datagram: a long-header Initial (version 1, padded to 1200 bytes like a real client

Initial) or a short header with a chosen DCID. `backend.py` answers with what it saw. They exercise the steering, not QUIC itself.

```
# qsend.py - send one QUIC-shaped datagram to the VIP and print the reply
#   python3 qsend.py long|short DCID_HEX [SRC_IP] [SRC_PORT]
import socket, sys
kind, dcid = sys.argv[1], bytes.fromhex(sys.argv[2])
src = (sys.argv[3] if len(sys.argv) > 3 else "198.51.100.10",
       int(sys.argv[4]) if len(sys.argv) > 4 else 0)
if kind == "long":   # Initial, version 1, DCID, empty SCID
    pkt = bytes([0xc0]) + (1).to_bytes(4, "big") + bytes([len(dcid)]) + dcid + b"\x00"
else:                # 1-RTT: fixed bit set, then the DCID the server chose
    pkt = bytes([0x40]) + dcid
pkt += b"\x00" * (1200 - len(pkt))           # QUIC pads client Initials to 1200
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
s.bind(src); s.settimeout(1)
s.sendto(pkt, ("198.51.100.1", 443))
try:
    data, peer = s.recvfrom(2048)
    print(f"{s.getsockname()[1]} <- {peer[0]}:{peer[1]}  {data.decode()}")
except TimeoutError:
    print(f"{s.getsockname()[1]} <- (no reply)")
```

```
# backend.py - a UDP "QUIC server" that only reports what it received
import socket, sys
name, addr, port = sys.argv[1], sys.argv[2], int(sys.argv[3])
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
s.bind((addr, port))
while True:
    data, peer = s.recvfrom(2048)
    if data[0] & 0x80:                       # long header (RFC 8999 §5.1)
        dcid = data[6:6 + data[5]]
        kind = "long "
    else:                                    # short header (RFC 8999 §5.2)
        dcid = data[1:9]
        kind = "short"
    s.sendto(f"{name} got {kind} dcid={dcid.hex()} from {peer[0]}:{peer[1]}".encode(), peer)
```

Real QUIC uses one Go program as backend and client. A backend generates every CID with its server ID in byte 1 and shares one stateless reset key with the others. The client can migrate to a new local port, or idle and then send again:

```
// main.go - a quic-go backend that puts its server ID in every CID, and a client
//
//  qlb server ID ADDR:PORT      e.g. qlb server 1 10.0.0.11:8080
//  qlb client [-src IP] [-migrate] [-pause 7s [-pad 200]] VIP:PORT
package main
```

```
import (
    "context"
    "crypto/ecdsa"
    "crypto/elliptic"
    "crypto/rand"
    "crypto/tls"
    "crypto/x509"
    "crypto/x509/pkix"
    "flag"
    "fmt"
    "io"
    "log"
    "math/big"
    "net"
    "os"
    "strconv"
    "strings"
    "time"

    "github.com/quic-go/quic-go"
)

// QUIC-LB unencrypted layout (draft-ietf-quic-load-balancers-21 §5.2):
// [config rotation 000 + 5 random bits][server ID][6-byte nonce]
type lbCIDs struct{ id byte }

func (g lbCIDs) ConnectionIDLen() int { return 8 }
func (g lbCIDs) GenerateConnectionID() (quic.ConnectionID, error) {
    b := make([]byte, 8)
    if _, err := rand.Read(b); err != nil {
        return quic.ConnectionID{}, err
    }
    b[0] &= 0x1f // config rotation codepoint 0
    b[1] = g.id  // server ID
    return quic.ConnectionIDFromBytes(b), nil
}

// Every backend shares this key, so any of them can reset any connection.
var resetKey = quic.StatelessResetKey{0x6c, 0x61, 0x62}

func server(id byte, addr string, ownKey bool) {
    udp, err := net.ListenUDP("udp4", mustAddr(addr))
    check(err)
    key := resetKey
    if ownKey { // the mistake: every backend invents its own reset key
        rand.Read(key[:])
```

```
    }
    tr := &quic.Transport{Conn: udp, ConnectionIDGenerator: lbCIDs{id}, StatelessResetKey: &
    ln, err := tr.Listen(selfSigned(), &quic.Config{})
    check(err)
    for {
        conn, err := ln.Accept(context.Background())
        check(err)
        go func() {
            for {
                str, err := conn.AcceptStream(context.Background())
                if err != nil {
                    return
                }
                msg, _ := io.ReadAll(str)
                fmt.Fprintf(str, "be%d: %s (client %s)", id, msg, conn.RemoteAddr())
                str.Close()
            }
        }()
    }
}

func ask(conn *quic.Conn, msg string) string {
    str, err := conn.OpenStreamSync(context.Background())
    check(err)
    str.Write([]byte(msg))
    str.Close()
    reply, err := io.ReadAll(str)
    if err != nil {
        return "error: " + err.Error()
    }
    return string(reply)
}

func client(vip, src string, migrate bool, pause time.Duration, pad int) {
    ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
    defer cancel()
    tr := &quic.Transport{Conn: listen(src)}
    conn, err := tr.Dial(ctx, mustAddr(vip), &tls.Config{InsecureSkipVerify: true, NextProto
        &quic.Config{MaxIdleTimeout: 10 * time.Second})
    check(err)
    fmt.Printf("port %d: %s\n", port(tr), ask(conn, "hello"))
    if migrate { // new UDP socket = new 4-tuple, same connection
        tr2 := &quic.Transport{Conn: listen(src)}
        path, err := conn.AddPath(tr2)
        check(err)
        check(path.Probe(ctx))
        check(path.Switch())
```

```
        fmt.Printf("port %d: %s\n", port(tr2), ask(conn, "after migration"))
    }
    if pause > 0 { // meanwhile the backend that owns this connection goes away
        time.Sleep(pause)
        t0 := time.Now()
        reply := ask(conn, "still there?"+strings.Repeat(".", pad))
        fmt.Printf("port %d after %v: %s (%.1fs)\n", port(tr), pause, reply, time.Since(t0).
    }
    conn.CloseWithError(0, "")
}

func main() {
    migrate := flag.Bool("migrate", false, "switch to a new local port mid-connection")
    pause := flag.Duration("pause", 0, "wait, then use the connection again")
    ownKey := flag.Bool("own-key", false, "server: use a private stateless reset key")
    src := flag.String("src", "", "client: source IP")
    pad := flag.Int("pad", 0, "client: bytes of padding on the request sent after -pause")
    if len(os.Args) < 2 {
        log.Fatal("usage: qlb server ID ADDR:PORT | qlb client [-src IP] [-migrate] [-pause
    }
    flag.CommandLine.Parse(os.Args[2:]) // flags follow the subcommand
    a := append([]string{os.Args[1]}, flag.Args()...)
    switch {
    case len(a) == 3 && a[0] == "server":
        id, err := strconv.Atoi(a[1])
        check(err)
        server(byte(id), a[2], *ownKey)
    case len(a) == 2 && a[0] == "client":
        client(a[1], *src, *migrate, *pause, *pad)
    default:
        log.Fatal("usage: qlb server ID ADDR:PORT | qlb client [-src IP] [-migrate] [-pause
    }
}

func port(t *quic.Transport) int { return t.Conn.LocalAddr().(*net.UDPAddr).Port }

func listen(src string) *net.UDPConn {
    c, err := net.ListenUDP("udp4", &net.UDPAddr{IP: net.ParseIP(src)})
    check(err)
    return c
}

func mustAddr(s string) *net.UDPAddr {
    a, err := net.ResolveUDPAddr("udp4", s)
    check(err)
    return a
}
```

```
func selfSigned() *tls.Config { // lab only: throwaway key, client skips verification
    key, _ := ecdsa.GenerateKey(elliptic.P256(), rand.Reader)
    tmpl := &x509.Certificate{SerialNumber: big.NewInt(1), Subject: pkix.Name{CommonName: "l
        NotBefore: time.Now().Add(-time.Hour), NotAfter: time.Now().Add(24 * time.Hour)}
    der, err := x509.CreateCertificate(rand.Reader, tmpl, tmpl, &key.PublicKey, key)
    check(err)
    return &tls.Config{Certificates: []tls.Certificate{{Certificate: [][]byte{der}, PrivateK
}

func check(err error) {
    if err != nil {
        fmt.Fprintln(os.Stderr, "error:", err)
        os.Exit(1)
    }
}
```

```
# build (quic-go v0.63.0 needs Go 1.26 or newer; GOTOOLCHAIN fetches it)
cd qgo                                  # main.go lives here
export GOTOOLCHAIN=go1.27.2
go mod init qlb
go get github.com/quic-go/quic-go@v0.63.0
go mod tidy
go build -o ../qlb .
```

```
# uname -r
6.12.94+
# nft --version
nftables v1.1.3 (Commodore Bullmoose #4)
# go version -m qlb | grep -E '^qlb|quic-go'
qlb: go1.27.2
    dep github.com/quic-go/quic-go  v0.63.0 h1:LIFGHI4PFUhhw2dDD1ARHdCff143ffMHwZtbnbuJ78A=
# python3 --version
Python 3.13.5
```

Why not paste the published ruleset?

The design here follows Marten Seemann's April 2026 post (see Sources). He installs his rules through the Go `google/nftables` netlink library and warns that the `nft` CLI cannot display them. The CLI cannot load his listing either. Saved as `lb-udp-seemann.nft` and checked with `nft -c`:

```
# ./lab.sh
# ip netns exec qlb nft -c -f lb-udp-seemann.nft
lb-udp-seemann.nft:30:78-80: Error: syntax error, unexpected map, expecting '}'
        ct zone set meta mark map @backend-zones add @flows { ip saddr : meta mark map @back
```

```
                                                                                ^^^
lb-udp-seemann.nft:28:15-33: Error: datatype mismatch: expected integer, expression has type
        ct zone set ip saddr map @flows return
        ~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^
lb-udp-seemann.nft:35:33-36: Error: transport protocol mapping is only valid after transport
        ct zone 3 dnat to 192.168.0.3:8082
                    ~~~~                ^^^^
lb-udp-seemann.nft:36:33-36: Error: transport protocol mapping is only valid after transport
        ct zone 2 dnat to 192.168.0.2:8081
                    ~~~~                ^^^^
lb-udp-seemann.nft:37:33-36: Error: transport protocol mapping is only valid after transport
        ct zone 1 dnat to 192.168.0.1:8080
                    ~~~~                ^^^^
# ip netns exec qlb nft -f lb-quic.nft
```

Published listing (installed over netlink)	This chapter (loads with `nft -f`)	Why
maps whose value type is `mark` feeding `ct zone set`	`typeof … : ct zone` for zone-valued maps	CLI type check: "expected integer, expression has type packet mark"
`add @flows { ip saddr : meta mark map @backend-zones }`	backend ID travels in the mark, `@zone-of` maps it to a zone	The CLI grammar has no map lookup inside a set element. In this lab, writing `ct zone` itself as the element value loaded fine and inserted nothing, so the mark carries it
`ct zone 3 dnat to 192.168.0.3:8082`, one rule per backend	`meta l4proto udp dnat ip to ct zone map @backends`	A port in `dnat to` needs a transport-protocol match. A map turns N rules into one
per-backend rules in the short-header and output chains	element maps `@sids`, `@backends`, `@replies`	Register, drain and deregister become element edits applied atomically by `nft -f`
server ID in CID byte 0	config rotation in byte 0, server ID in byte 1	QUIC-LB reserves the first octet
return path set in an `output` hook chain	return path set in `raw` prerouting, keyed on the backend address	The `output` hook only sees packets the LB host generates. Backends on their own hosts reply *through* the LB (see The trap)

Drill 1: handshakes stick to a client address

```
# for i in 1 2 3; do ip netns exec qbe python3 backend.py be$i 10.0.0.1$i 808$((i-1)) & done
# S() { ip netns exec qcli python3 qsend.py "$@"; }
# for i in 1 2 3; do S long $(openssl rand -hex 8); done
55715 <- 198.51.100.1:443  be2 got long  dcid=4248938fa0c5ae57 from 198.51.100.10:55715
34938 <- 198.51.100.1:443  be2 got long  dcid=a9807a6bd375cecc from 198.51.100.10:34938
53655 <- 198.51.100.1:443  be2 got long  dcid=7d4a699acc7a4368 from 198.51.100.10:53655
# for i in 1 2 3; do S long $(openssl rand -hex 8) 198.51.100.11; done
60861 <- 198.51.100.1:443  be3 got long  dcid=19b6d724a3379218 from 198.51.100.11:60861
45629 <- 198.51.100.1:443  be3 got long  dcid=4b1e25e33ff2ddd5 from 198.51.100.11:45629
58044 <- 198.51.100.1:443  be3 got long  dcid=323209b7e60d3201 from 198.51.100.11:58044
# ip netns exec qlb nft list map ip lb-quic flows
table ip lb-quic {
    map flows {
        typeof ip saddr : meta mark
        size 65536
        flags dynamic,timeout
        timeout 5s
        elements = { 198.51.100.10 expires 4s872ms : 0x00000002,
                     198.51.100.11 expires 4s940ms : 0x00000003 }
    }
}
```

Every long-header packet from `198.51.100.10` reached be2 and every one from `.11` reached be1, each with a fresh random DCID. The first packet from each address took the random slot, and the `update` stored the result in `@flows`. Later packets found the pin and returned early. A real handshake sends several long-header packets (Initial, Handshake) before the client switches to the server's CID, and all of them have to reach the server that holds the handshake state. The 5-second timeout is the handshake budget. It also means two clients behind one NAT address that start handshakes within 5 s land on the same backend. Every reply came from `198.51.100.1:443`: conntrack undid the DNAT on the way out.

Drill 2: established packets follow the server ID

```
# for sid in 01 02 03; do S short 07${sid}a1b2c3d4e5f6; done
45811 <- 198.51.100.1:443  be1 got short dcid=0701a1b2c3d4e5f6 from 198.51.100.10:45811
44651 <- 198.51.100.1:443  be2 got short dcid=0702a1b2c3d4e5f6 from 198.51.100.10:44651
38289 <- 198.51.100.1:443  be3 got short dcid=0703a1b2c3d4e5f6 from 198.51.100.10:38289
# S short 0704a1b2c3d4e5f6      # server ID 4: nobody registered it
34962 <- 198.51.100.1:443  be2 got short dcid=0704a1b2c3d4e5f6 from 198.51.100.10:34962
# S short e702a1b2c3d4e5f6      # config rotation 0b111: unroutable by definition
60745 <- 198.51.100.1:443  be2 got short dcid=e702a1b2c3d4e5f6 from 198.51.100.10:60745
```

The server ID byte picked the backend, regardless of source port. Server ID `0x04` has no `@sids` entry, so the `sids` lookup failed and `goto new-flow` treated the packet like a handshake. It found the pin `198.51.100.10 → be2` from Drill 1, still inside its 5 s. Config rotation `0b111` (`0xe7`) never reached the `@sids` lookup and took the same fallback. QUIC-LB defines `0b111` as unroutable (§3.2), so the load balancer falls back instead of trusting byte 1.

Drill 3: two connections, one 4-tuple, and why zones exist

```
# S short 0701000000000001 198.51.100.10 40000
40000 <- 198.51.100.1:443  be1 got short dcid=0701000000000001 from 198.51.100.10:40000
# S short 0703000000000002 198.51.100.10 40000
40000 <- 198.51.100.1:443  be3 got short dcid=0703000000000002 from 198.51.100.10:40000
# ip netns exec qlb grep 'sport=40000 ' /proc/net/nf_conntrack
ipv4     2 udp      17 29 src=198.51.100.10 dst=198.51.100.1 sport=40000 dport=443 src=10.0.
ipv4     2 udp      17 29 src=198.51.100.10 dst=198.51.100.1 sport=40000 dport=443 src=10.0.
```

Same source port, same VIP, two CIDs, two backends. Conntrack holds two entries with identical original tuples, kept apart only by `zone=1` and `zone=3`. Now the same steering without zones:

```
# nozone.nft - the same CID steering, minus conntrack zones
table ip nozone {
    chain raw {
        type filter hook prerouting priority raw; policy accept;
        iifname "lo" udp dport 443 @th,80,8 0x01 meta mark set 1
        iifname "lo" udp dport 443 @th,80,8 0x03 meta mark set 3
    }
    chain to-backend {
        type nat hook prerouting priority dstnat; policy accept;
        meta l4proto udp meta mark 1 dnat to 10.0.0.11:8080
        meta l4proto udp meta mark 3 dnat to 10.0.0.13:8082
    }
}
```

```
# ip netns exec qlb nft delete table ip lb-quic
# ip netns exec qlb nft -f nozone.nft
# S short 0701000000000001 198.51.100.10 40001
40001 <- 198.51.100.1:443  be1 got short dcid=0701000000000001 from 198.51.100.10:40001
# S short 0703000000000002 198.51.100.10 40001
40001 <- 198.51.100.1:443  be1 got short dcid=0703000000000002 from 198.51.100.10:40001
# ip netns exec qlb nft delete table ip nozone
# ip netns exec qlb nft -f lb-quic.nft
```

The second packet carried be3's server ID and still went to be1. The mark said 3, but the 4-tuple already had a conntrack entry, and an existing entry is never handed to the NAT chain again. With zones, the CID chooses on every packet. Without them, the first packet chooses for the whole 4-tuple.

Drill 4: real QUIC, including migration

The probes go away and real quic-go servers take their ports:

```
# pkill -f backend.py
# for i in 1 2 3; do ip netns exec qbe ./qlb server $i 10.0.0.1$i:808$((i-1)) & done; sleep
# Q() { ip netns exec qcli ./qlb client "$@" 198.51.100.1:443; }
# for a in 10 11 12 13; do Q -src 198.51.100.$a; done
port 39756: be2: hello (client 198.51.100.10:39756)
port 43470: be1: hello (client 198.51.100.11:43470)
port 52985: be2: hello (client 198.51.100.12:52985)
port 35223: be2: hello (client 198.51.100.13:35223)
# Q -src 198.51.100.14 -migrate
port 33196: be3: hello (client 198.51.100.14:33196)
port 48439: be3: after migration (client 198.51.100.14:48439)
```

Each handshake was steered by `@flows`. After it, every 1-RTT packet carried a server-chosen CID with that server's ID in byte 1. The `-migrate` run opened a second UDP socket, so the connection got a new 4-tuple. It probed the path (`PATH_CHALLENGE`/`PATH_RESPONSE`, RFC 9000 §9), switched, and sent again. The new 4-tuple had no conntrack entry. `@sids` sent it to the same backend anyway, and be3 saw the client's new port.

Compare the TCP recipe applied to UDP, choosing a backend per 4-tuple:

```
# naive.nft - the TCP recipe applied to UDP: pick a backend per 4-tuple
table ip naive {
    map backends {
        type mark : ipv4_addr . inet_service
        flags constant
        elements = { 0 : 10.0.0.11 . 8080, 1 : 10.0.0.12 . 8081, 2 : 10.0.0.13 . 8082 }
    }
    chain to-backend {
        type nat hook prerouting priority dstnat; policy accept;
        udp dport 443 dnat ip to numgen random mod 3 map @backends
    }
}
```

```
# ip netns exec qlb nft delete table ip lb-quic
# ip netns exec qlb nft -f naive.nft
# for a in 10 11 12 13 14 15; do Q -src 198.51.100.$a -migrate; done
port 36487: be2: hello (client 198.51.100.10:36487)
error: context deadline exceeded
port 58578: be3: hello (client 198.51.100.11:58578)
port 44403: be3: after migration (client 198.51.100.11:44403)
port 60557: be3: hello (client 198.51.100.12:60557)
error: context deadline exceeded
port 57019: be2: hello (client 198.51.100.13:57019)
```

```
error: context deadline exceeded
port 45910: be2: hello (client 198.51.100.14:45910)
port 41195: be2: after migration (client 198.51.100.14:41195)
port 35883: be2: hello (client 198.51.100.15:35883)
error: context deadline exceeded
# ip netns exec qlb nft delete table ip naive
# ip netns exec qlb nft -f lb-quic.nft
```

Four of six migrations failed. The new path was a new 4-tuple, so `numgen` picked a backend at random. Two times out of six it happened to be the same server. The other four times the `PATH_CHALLENGE` landed on a backend with no state for that connection, and the probe ran out of its 5 s budget. On the Internet you don't choose when this happens: a NAT rebinding changes the client's port without asking.

Drill 5: drain be3, add be4, deregister be3, reset its clients

```
# drain-be3.nft - be4 joins; slot 2 now means be4, so be3 gets no new connections
delete element ip lb-quic slots { 2 }
add element ip lb-quic slots { 2 : 4 }
add element ip lb-quic sids { 0x04 : 4 }
add element ip lb-quic backends { 4 : 10.0.0.14 . 8083 }
add element ip lb-quic replies { 10.0.0.14 . 8083 : 4 }
```

```
# deregister-be3.nft - be3 is gone: its CIDs are no longer routable
delete element ip lb-quic sids { 0x03 }
delete element ip lb-quic backends { 3 }
delete element ip lb-quic replies { 10.0.0.13 . 8082 }
```

```
# sleep 6      # let earlier handshake pins in @flows expire
# ip netns exec qlb nft add element ip lb-quic flows '{ 198.51.100.10 : 3 }'   # pin the nex
# Q -src 198.51.100.10 -pause 7s -pad 200 & qa=$!; sleep 1
port 46079: be3: hello (client 198.51.100.10:46079)
# ip netns exec qbe ./qlb server 4 10.0.0.14:8083 & sleep 1
# ip netns exec qlb nft -f drain-be3.nft
# for a in 11 12 13 14 15 16; do Q -src 198.51.100.$a; done
port 57472: be4: hello (client 198.51.100.11:57472)
port 37147: be2: hello (client 198.51.100.12:37147)
port 36258: be2: hello (client 198.51.100.13:36258)
port 58225: be2: hello (client 198.51.100.14:58225)
port 34076: be2: hello (client 198.51.100.15:34076)
port 49176: be1: hello (client 198.51.100.16:49176)
# ip netns exec qlb nft -f deregister-be3.nft; pkill -f 'qlb server 3'
# wait $qa
port 46079 after 7s: error: received a stateless reset (0.0s)
```

The client on `198.51.100.10` was pinned to be3 before the drain (the first `add element` exists only to make the drill deterministic). `drain-be3.nft` swapped slot 2 from be3 to be4 in one transaction. The six handshakes that followed went to be1, be2 and be4, never be3, while be3 kept serving its connection. In this ruleset a client address already pinned to be3 in `@flows` would still reach it for up to 5 s, so give a drain at least the pin timeout before deregistering. Deregistering removed be3's server ID, address and return path. The pinned client's next packet then carried an unknown server ID, fell through to `new-flow`, and reached a live backend. That backend has no state for the CID, so it answered with a **stateless reset** (RFC 9000 §10.3). The client checked the reset token and closed at once: `received a stateless reset (0.0s)`.

The reset token is derived from the CID and a key. quic-go computes it as HMAC-SHA256 keyed with `Transport.StatelessResetKey`, and picks a random key if none is set (`stateless_reset.go`). Another backend can only produce the right token if it has the same key. Here is the same drill with every backend inventing its own key:

```
# pkill -f 'qlb server'; sleep 1
# for i in 1 2 3 4; do ip netns exec qbe ./qlb server -own-key $i 10.0.0.1$i:808$((i-1)) & d
# ip netns exec qlb nft delete table ip lb-quic; ip netns exec qlb nft -f lb-quic.nft
# ip netns exec qlb nft add element ip lb-quic flows '{ 198.51.100.10 : 3 }'
# Q -src 198.51.100.10 -pause 7s -pad 200 & qa=$!; sleep 1
port 54876: be3: hello (client 198.51.100.10:54876)
# ip netns exec qlb nft -f drain-be3.nft
# ip netns exec qlb nft -f deregister-be3.nft; pkill -f 'qlb server -own-key 3'
# wait $qa
port 54876 after 7s: error: timeout: no recent network activity (10.0s)
```

The reset still arrived, but its token didn't verify, so the client ignored it. The client retransmitted into the void until its 10 s idle timeout. A real client would hang for the negotiated idle timeout (quic-go's default is 30 s). `-pad 200` is in both drills because quic-go v0.63.0 does not answer packets of 42 bytes or less with a reset (`MinStatelessResetSize` in `transport.go`). RFC 9000 §10.3 permits answering those with a reset one byte shorter. An unpadded 41-byte request makes the outcome depend on packet sizes rather than on the key.

The trap

The return path looks optional because the lab can pass without it. Flush `@replies`, the equivalent of keeping the zone-restoring rule in an `output` chain while the backends live on other hosts:

```
# edge.nft - BCP 38 at the upstream: nothing leaves l0 with a source we don't own
table inet edge {
    chain egress {                  # after srcnat, so it sees the translated source
        type filter hook postrouting priority srcnat + 10; policy accept;
        oifname "l0" ip saddr != 198.51.100.0/24 counter drop
    }
}
```

```
# pkill -f 'qlb server'; sleep 1
# for i in 1 2 3; do ip netns exec qbe ./qlb server $i 10.0.0.1$i:808$((i-1)) & done; sleep
# ip netns exec qlb nft delete table ip lb-quic; ip netns exec qlb nft -f lb-quic.nft
# ip netns exec qlb nft flush map ip lb-quic replies      # "the output chain handles replie
# ip netns exec qcli tcpdump -tqlni c0 -c 1 'udp and src net 10.0.0.0/24' 2>/dev/null & td=$
# Q -src 198.51.100.17; wait $td
port 50391: be2: hello (client 198.51.100.17:50391)
IP 10.0.0.12.8081 > 198.51.100.17.50391: UDP, length 1280
# ip netns exec qlb grep 'src=198.51.100.17 ' /proc/net/nf_conntrack | sort
ipv4     2 udp      17 29 src=10.0.0.12 dst=198.51.100.17 sport=8081 dport=50391 [UNREPLIED]
ipv4     2 udp      17 29 src=198.51.100.17 dst=198.51.100.1 sport=50391 dport=443 [UNREPLIE
# ip netns exec qlb nft -f edge.nft
# Q -src 198.51.100.16
error: context deadline exceeded
# ip netns exec qlb nft add element ip lb-quic replies '{ 10.0.0.11 . 8080 : 1, 10.0.0.12 .
# Q -src 198.51.100.16
port 42633: be2: hello (client 198.51.100.16:42633)
# ip netns exec qlb nft list chain inet edge egress
table inet edge {
    chain egress {
        type filter hook postrouting priority srcnat + 10; policy accept;
        oifname "l0" ip saddr != 198.51.100.0/24 counter packets 16 bytes 12520 drop
    }
}
```

Read it in order:

1. The handshake **succeeded**, and that is the trap. The backend's reply came in on `l1` with no mark, so it was looked up in zone 0. It didn't match the zone-2 entry, so conntrack created a *new* zone-0 entry instead, `[UNREPLIED]` in both directions. Nothing undid the DNAT. The client received `10.0.0.12.8081 > 198.51.100.17`, a packet from the backend's private address. quic-go demultiplexes by CID on an unconnected socket, so it accepted the packet.
2. On a real network, that packet leaves the load balancer with a source address the site doesn't own. Egress filtering (BCP 38 / RFC 2827), uRPF upstream, or the client's own NAT, which only admits replies from `198.51.100.1:443`, drops it. `edge.nft` models that. Its filter sits after `srcnat` in postrouting because the reply's source is only translated back to the VIP at that hook. A filter in `forward` would also drop correct replies. With the edge filter on, the same setup times out.
3. With `@replies` restored, replies carry the request's zone, find their entry, and leave as `198.51.100.1:443`. The edge filter passes them. Its counter records the leaked replies it dropped.

Rule: the zone for replies must be set before conntrack's lookup (prerouting priority −200) on the interface the replies actually *arrive* on. Backends on the LB host reply through `output`. Backends anywhere else reply through `prerouting`. Test it with an egress filter, not with a client that accepts anything.

Notes and pitfalls

- **A plaintext server ID is visible to everyone.** Any observer can tell which backend owns a connection, and can link a migrating client's old and new paths by that byte. That is the linkability RFC 9000 §5.1 and §9.5 try to prevent. QUIC-LB's answer is to encrypt the server ID with AES, which nftables cannot do. Use this design where that leak is acceptable, such as internal fleets or few clients per server, or move CID decoding to software that can decrypt (a userspace or eBPF load balancer).
- **The packet mark is now taken.** Policy routing (`ip rule fwmark`) or other firewall code that uses marks will collide with it. Mask a field of the mark if you need to share.
- **IPv4 only.** `table ip` and `ip saddr` keys. An `inet` or `ip6` version needs IPv6-keyed `@flows`/`@replies` and IPv6 `@backends`.
- **Server IDs are one byte.** That allows 255 backends. A 2-byte ID is `@th,80,16` and a wider `@sids` key.
- **The flows timeout is a trade-off.** Too short and slow handshakes split across backends. Too long and a draining backend keeps receiving new handshakes from pinned addresses.

The boring rule

Route handshakes by client address and everything else by a server ID the backends write into their CIDs. Separate backends with conntrack zones, and restore the zone on the path replies actually take. Keep every backend fact in a map and change it with atomic `nft -f` element edits: drain by remapping slots, wait out the pin timeout, then deregister. Give all backends one stateless reset key. Prove the return path with an egress filter.

Try this

1. Change `lbCIDs` to use a 2-byte server ID (`b[1:3]`), switch the match to `@th,80,16`, and register a backend with ID `0x0102`.
2. Run `nft monitor` in `qlb` during Drill 5 and watch the element changes land as one transaction.
3. Put a NAT between the client and the VIP (a fourth namespace with `masquerade`). Delete its conntrack entry mid-connection (`conntrack -D`) to force a rebinding, and confirm the connection survives under `lb-quic.nft` but not under `naive.nft`.
4. Set `@flows`' timeout to `1s` and add 400 ms of delay to `l1` with `tc qdisc add dev l1 root netem delay 400ms`. Find the point where handshakes start to fail.
5. Rewrite the table as `table inet` with dual-stack `@flows` and `@replies`, then repeat Drill 4 over IPv6.

Sources

- RFC 8999 — Version-Independent Properties of QUIC: https://www.rfc-editor.org/rfc/rfc8999.html

- RFC 9000 — QUIC (§5.1 connection IDs, §7.2 negotiating CIDs, §9 migration, §9.5 privacy, §10.3 stateless reset): https://www.rfc-editor.org/rfc/rfc9000.html
- QUIC-LB: Generating Routable QUIC Connection IDs, draft-ietf-quic-load-balancers-21: https://datatracker.ietf.org/doc/draft-ietf-quic-load-balancers/
- nftables wiki — Netfilter hooks and priorities: https://wiki.nftables.org/wiki-nftables/index.php/Netfilter_hooks
- nftables wiki — Configuring chains (priority keywords): https://wiki.nftables.org/wiki-nftables/index.php/Configuring_chains
- nftables wiki — Setting packet connection tracking metainformation (`ct zone`): https://wiki.nftables.org/wiki-nftables/index.php/Setting_packet_connection_tracking_metainformation
- nft(8) manual (raw payload `@th`, maps, `numgen`, dynamic sets): https://www.netfilter.org/projects/nftables/manpage.html
- quic-go v0.63.0 `Transport` (`ConnectionIDGenerator`, `StatelessResetKey`, reset size floor): https://github.com/quic-go/quic-go/blob/v0.63.0/transport.go
- quic-go v0.63.0 reset token derivation: https://github.com/quic-go/quic-go/blob/v0.63.0/stateless_reset.go
- RFC 2827 (BCP 38) — Network Ingress Filtering: https://www.rfc-editor.org/rfc/rfc2827.html
- Marten Seemann, "Using nftables for QUIC load balancing" (2026-04-08), the design this chapter adapts: https://seemann.io/posts/2026-04-08---nftables-quic-load-balancing/

HTTP/3 Stream Cancellation: STOP_SENDING, RESET_STREAM, and 499

HTTP/3 Stream Cancellation: STOP_SENDING, RESET_STREAM, and 499

A user closes a tab, a mobile client races a new search, an SDK hits its read timeout. The HTTP request is over for the client. Whether the origin finds out — and how fast — depends on the transport. **HTTP/1.1** has no per-request abort: the only signal is closing the TCP connection, which also kills keep-alive and any pipelined work on that socket. **HTTP/2** can abort one stream with `RST_STREAM` and error code `CANCEL` (0x08) while the TLS connection stays up. **HTTP/3** splits that into two QUIC frames on independent stream halves: `STOP_SENDING` (stop the peer sending) and `RESET_STREAM` (I am done sending), usually carrying `H3_REQUEST_CANCELLED` (0x010c). This chapter is the mapping, the proxy duty (propagate immediately, log 499, do not wait for origin 200), and a localhost HTTP/3 lab that captures the frames.

Learning goals

- Name the abort signal on HTTP/1.1, HTTP/2, and HTTP/3, and say what happens to sibling requests
- Read a QUIC bidirectional stream as two state machines, and say when `STOP_SENDING` vs `RESET_STREAM` is the right frame (RFC 9000 §3, §19.4, §19.5)
- Pick `H3_REQUEST_CANCELLED` (0x010c) vs `H3_NO_ERROR` (0x0100) vs `H3_REQUEST_REJECTED` (0x010b) from RFC 9114 §4.1.1 and §8.1
- Write the proxy rule: on downstream cancel, stop the origin stream *now* and record a 499-style "client closed request"
- Design an origin so a cancelled GET stops work, and a cancelled POST does not double-apply on retry
- Run a localhost HTTP/3 server and client, cancel mid-response, and point at the qlog frames

Lab platform: Python 3.13.5, aioquic 1.7.0 from PyPI, OpenSSL 3.5.7, Linux 6.12. Bind `127.0.0.1:4433`. No root. Every listing output below is pasted from one run.

This chapter does not reteach QUIC version negotiation or CID-based load balancing.

Mental model

A cancel is "this request is no longer of interest." The transport has to carry that to the peer without taking down every other request that happens to share a connection.

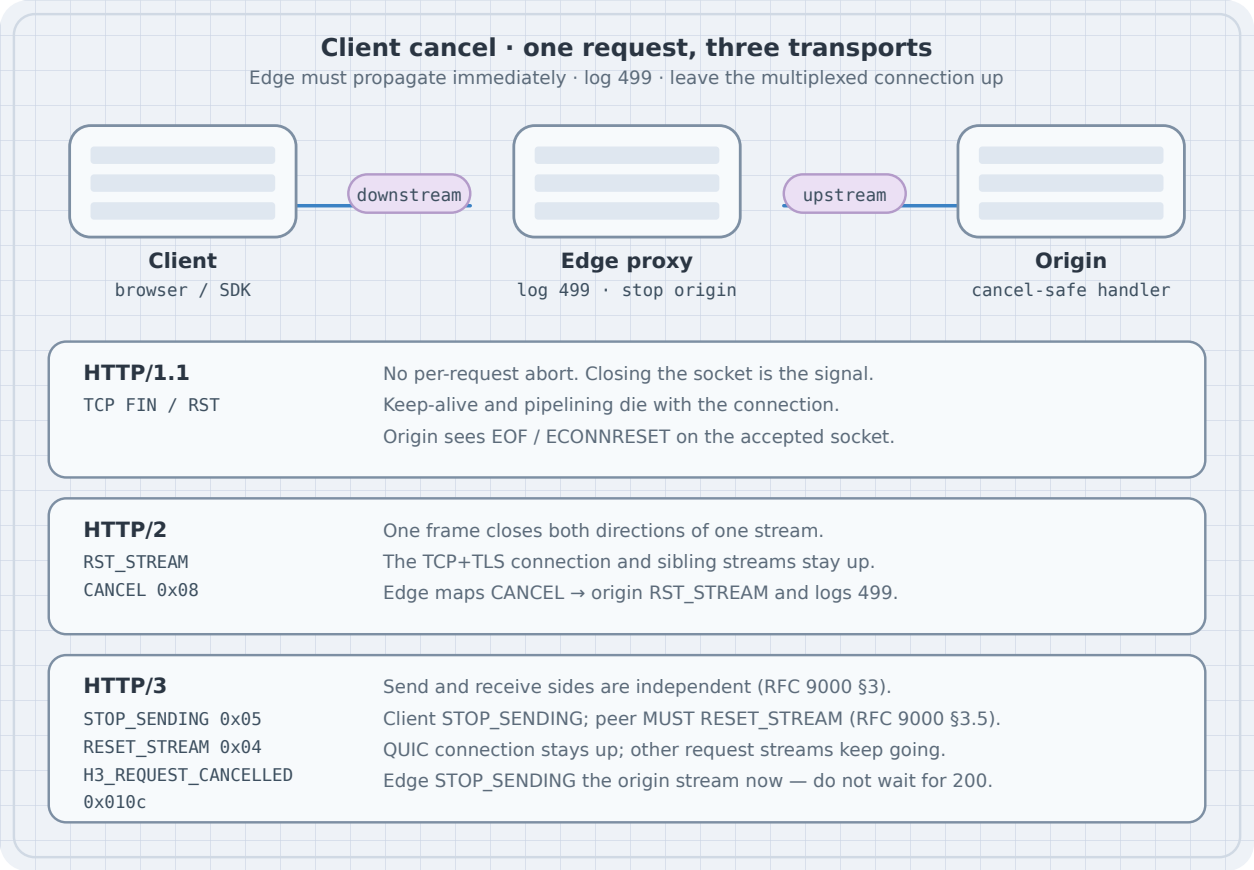


Figure 1: Client, edge proxy, and origin, with the abort signal on HTTP/1.1, HTTP/2, and HTTP/3

	HTTP/1.1 (RFC 9112)	HTTP/2 (RFC 9113)	HTTP/3 (RFC 9114 on RFC 9000)
Abort signal	Close the TCP connection (FIN or RST)	`RST_STREAM` with `CANCEL` (0x08)	`STOP_SENDING` and/or `RESET_STREAM` with `H3_REQUEST_CANCELLED` (0x010c)
Scope	The whole connection	One stream; both directions	One stream; send and receive halves independently
Sibling requests	Die with the socket	Keep going	Keep going
What the origin must notice	`read()` returns 0 / `ECONNRESET`	Stream reset	`StopSendingReceived` / `StreamReset` (library events), then stop the handler

The edge is not a spectator. RFC 9114 Appendix A.4.1 says an intermediary *may* pass client cancellations indicated by `H3_REQUEST_CANCELLED`. In production that "may" is the difference between origin CPU spent on a tab that already closed and a 499 in the log. nginx invented status **499** ("client closed request") for exactly this: the downstream went away before a response was sent. It is a log code, not an IETF status the client ever sees.

Concepts — two halves of a QUIC stream

HTTP/3 puts each request/response on one client-initiated **bidirectional** QUIC stream (RFC 9114 §2, §6.1). QUIC treats that stream as two independent parts (RFC 9000 §3): a sending part and a receiving part. Closing one does not close the other.

```
Client-initiated bidir stream (id 0, 4, 8, …)

  client send    STREAM / STREAM+FIN / RESET_STREAM     server recv
  client recv    STREAM / STREAM+FIN / RESET_STREAM    server send
                 STOP_SENDING travels the opposite way
                 of the data it wants to stop
```

Application operations (RFC 9000 §2.4):

- **Reset sending** → `RESET_STREAM` (type 0x04). "I will not send more on this stream."
- **Abort reading** → `STOP_SENDING` (type 0x05). "Stop sending to me on this stream."

Frame layouts (RFC 9000 §19.4, §19.5):

```
RESET_STREAM Frame {
  Type (i) = 0x04,
  Stream ID (i),
  Application Protocol Error Code (i),
```

```
  Final Size (i),
}

STOP_SENDING Frame {
  Type (i) = 0x05,
  Stream ID (i),
  Application Protocol Error Code (i),
}
```

The coupling that operators forget is RFC 9000 §3.5: an endpoint that receives `STOP_SENDING` **MUST** send `RESET_STREAM` if its sending part is in "Ready" or "Send". It SHOULD copy the error code. STREAM frames that still arrive after `STOP_SENDING` still count toward flow control even if the application discards them.

So a client that has already finished its GET (HEADERS + FIN) and then loses interest in the response does this:

1. `STOP_SENDING` with `H3_REQUEST_CANCELLED` — abort reading.
2. `RESET_STREAM` only if the send side is still open (a POST body still in flight). If the GET is already acknowledged, `RESET_STREAM` is a no-op.
3. The server MUST `RESET_STREAM` the response. The client then sees a stream reset with the same application error code.

That pair is the HTTP/3 equivalent of HTTP/2's single `RST_STREAM`. HTTP/2 streams are one state machine (RFC 9113 §5.1): `RST_STREAM` moves the stream to "closed" in both directions at once.

Concepts — which HTTP/3 error code

RFC 9114 §4.1.1: implementations SHOULD cancel by resetting any sending parts that are still open and aborting reading on receiving parts. The codes (RFC 9114 §8.1):

Code	Value	When
`H3_NO_ERROR`	0x0100	Close with no error. A server that has sent a complete response and does not need the rest of the request SHOULD abort reading with this code (RFC 9114 §4.1). Also used for graceful connection close.
`H3_REQUEST_REJECTED`	0x010b	Server refused the request **without** application processing. The client may retry as if the request never happened. Servers MUST NOT use this after passing data to a higher layer.

Code	Value	When
`H3_REQUEST_CANCELLED`	0x010c	The request or its response is cancelled. Clients SHOULD use this. A server that abandons a *partially processed* response SHOULD use this, not `H3_REQUEST_REJECTED`.

RFC 9114 Appendix A.4 maps HTTP/2 `CANCEL` (0x08) to `H3_REQUEST_CANCELLED`. Treat them as the same operator event: "this request is no longer needed."

Do not send `H3_NO_ERROR` when the user cancelled. The stream is torn down either way, but `0x0100` tells the peer "nothing went wrong", so metrics and origin cancellation hooks that key off `0x010c` will miss it.

Case — what a proxy must do

Downstream cancel is not "the origin can finish and we will drop the body." The body is the expensive part.

```
Client                        Edge                          Origin

    GET /report             GET /report
                                                          (query running)
    STOP_SENDING 0x010c
                                  log 499
                                  STOP_SENDING 0x010c
                                                          abort handler
        RESET_STREAM 0x010c        RESET_STREAM 0x010c
```

RFC 9114 Appendix A.4.1: it is beneficial to pass client cancellations indicated by `H3_REQUEST_CANCELLED`. The operational translation:

1. On downstream `STOP_SENDING` / `RESET_STREAM` / HTTP/2 `RST_STREAM CANCEL` / HTTP/1.1 connection close, **stop the origin stream immediately**. Do not wait for origin headers.
2. Log **499** (nginx `NGX_HTTP_CLIENT_CLOSED_REQUEST`; Cloudflare uses the same number in analytics and logs). The client never receives a 499 status line — they already left.
3. Exclude 499 from server-error rates. It is a client abort, not a 5xx. A jump in 499s after you start honouring HTTP/3 cancels is reporting catching up, not a new outage.
4. Leave `proxy_ignore_client_abort` (and equivalents) off for reads. Turn it on only for work that *must* finish without a client, and only with an idempotency story.

Cloudflare's 9 October 2026 changelog records the HTTP/3-shaped version of this bug: the edge did not always stop an HTTP/3 request when the client cancelled the stream, so some requests continued to the origin and were logged with the upstream status instead of 499. The fix stops

those requests sooner and records 499. Operators then see more 499s on HTTP/3; the cancellations were already happening.

HTTP/1.1 through the same proxy is the same 499 with a worse blast radius: the only way the client can abort is to close TCP, so the proxy also loses connection reuse.

Case — cancel-safe origins

The transport can only deliver the signal. The handler has to listen.

- **Context.** In Go, `Request.Context()` is cancelled when the peer goes away. In Python asyncio, cancel the task that is writing the response (the lab below). In any language, the HTTP server must surface stream reset as cancellation, not as "write failed later."
- **Reads (GET, HEAD).** Abort the query, the upstream HTTP call, the file read. Retrying a partial GET is safe (RFC 9110: GET is idempotent).
- **Writes (POST, PUT, PATCH).** A cancel after the origin began processing is *not* "the write never happened." The client may retry. Use an idempotency key (client-supplied or derived) and make the handler safe to run twice. Do not use `H3_REQUEST_REJECTED` after you have touched storage — RFC 9114 §4.1.1 forbids it, because that code tells the client the request is retryable as if it never ran.
- **Do not finish work for a gone client** on the default path. The exception is a small class of must-complete actions (some payments, some audit writes), and those need their own queue, not a streaming HTTP response the client already abandoned.

A 200 that nobody reads still cost the database. A 499 that you treat as 5xx pages people at 3 a.m. for a user tapping Back.

Lab — cancel mid-response on localhost

Self-signed certificate for `localhost` and `127.0.0.1`, then aioquic 1.7.0 as both HTTP/3 server and client. The server streams 40 × 256-byte chunks with 50 ms between them (~2 s if nobody cancels). The client cancels after 512 bytes, then issues `GET /fast` on a new stream of the **same** connection to prove the abort was per-stream.

```
$ python3 -m venv /workspace/run-2026-10-17-am/venv-net-http3
$ /workspace/run-2026-10-17-am/venv-net-http3/bin/pip install aioquic
...
Successfully installed aioquic-1.7.0 attrs-26.1.0 certifi-2026.7.22 cffi-2.1.1 cryptography-
$ openssl version
OpenSSL 3.5.7 9 Jun 2026 (Library: OpenSSL 3.5.7 9 Jun 2026)
$ openssl req -x509 -newkey ec -pkeyopt ec_paramgen_curve:prime256v1 -nodes \
    -days 1 -subj /CN=localhost \
    -addext subjectAltName=DNS:localhost,IP:127.0.0.1 \
    -keyout key.pem -out cert.pem
$ ls -l cert.pem key.pem
-rw-r--r-- 1 box box 615 Oct 17 08:47 cert.pem
```

```
-rw------- 1 box box 241 Oct 17 08:47 key.pem
```

```
# cancel_lab.py - HTTP/3 client cancel mid-response (aioquic, localhost)
import asyncio
import json
import ssl
import sys
import time
from typing import Optional

from aioquic.asyncio import connect, serve
from aioquic.asyncio.protocol import QuicConnectionProtocol
from aioquic.h3.connection import H3_ALPN, ErrorCode, H3Connection
from aioquic.h3.events import DataReceived, H3Event, HeadersReceived
from aioquic.quic.configuration import QuicConfiguration
from aioquic.quic.events import ProtocolNegotiated, QuicEvent, StopSendingReceived, StreamRe
from aioquic.quic.logger import QuicLogger

T0 = time.monotonic()
SERVER_CHUNKS_SENT = 0
SERVER_SEND_ABORTED = False
SERVER_SAW_STOP_SENDING: Optional[tuple[int, int]] = None
SERVER_SAW_RESET: Optional[tuple[int, int]] = None
CLIENT_BYTES = 0
CLIENT_SAW_RESET: Optional[tuple[int, int]] = None
CHUNK = b"x" * 256
N_CHUNKS = 40
CHUNK_PAUSE = 0.05


def stamp(role: str, msg: str) -> None:
    print(f"{time.monotonic() - T0:7.3f}s  {role:6}  {msg}", flush=True)


def dump_cancel_frames(label: str, logger: QuicLogger) -> None:
    kinds = {"reset_stream", "stop_sending"}
    print(f"\n# qlog {label}: STOP_SENDING / RESET_STREAM frames")
    n = 0
    for trace in logger.to_dict()["traces"]:
        vp = trace["vantage_point"]["type"]
        for ev in trace["events"]:
            for fr in ev.get("data", {}).get("frames") or []:
                if fr.get("frame_type") in kinds:
                    n += 1
                    extra = (
                        f"  final_size={fr.get('final_size')}"
```

```
                        if "final_size" in fr
                        else ""
                    )
                    print(
                        f"  {vp:6}  {ev['name']:28}  {fr['frame_type']}"
                        f"  stream={fr.get('stream_id')}"
                        f"  error_code={fr.get('error_code')}{extra}"
                    )
    if n == 0:
        print("  (none)")


class H3ServerProtocol(QuicConnectionProtocol):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self._http: Optional[H3Connection] = None
        self._tasks: dict[int, asyncio.Task] = {}

    def quic_event_received(self, event: QuicEvent) -> None:
        global SERVER_SAW_STOP_SENDING, SERVER_SAW_RESET
        if isinstance(event, ProtocolNegotiated):
            stamp("server", f"ALPN={event.alpn_protocol}")
            if event.alpn_protocol in H3_ALPN:
                self._http = H3Connection(self._quic)
        if isinstance(event, StopSendingReceived):
            SERVER_SAW_STOP_SENDING = (event.stream_id, event.error_code)
            stamp(
                "server",
                f"StopSendingReceived stream={event.stream_id} "
                f"error_code=0x{event.error_code:x} ({event.error_code})",
            )
            task = self._tasks.pop(event.stream_id, None)
            if task is not None:
                task.cancel()
        if isinstance(event, StreamReset):
            SERVER_SAW_RESET = (event.stream_id, event.error_code)
            stamp(
                "server",
                f"StreamReset stream={event.stream_id} "
                f"error_code=0x{event.error_code:x} ({event.error_code})",
            )
        if self._http is not None:
            for http_event in self._http.handle_event(event):
                self._http_event(http_event)

    def _http_event(self, event: H3Event) -> None:
        if isinstance(event, HeadersReceived):
```

```
            headers = {k.decode(): v.decode() for k, v in event.headers}
            path = headers.get(":path", "?")
            stamp(
                "server",
                f"request {headers.get(':method')} {path} stream={event.stream_id}",
            )
            if path == "/fast":
                self._http.send_headers(
                    event.stream_id,
                    [(b":status", b"200"), (b"content-type", b"text/plain")],
                )
                self._http.send_data(event.stream_id, b"ok\n", end_stream=True)
                self.transmit()
                stamp("server", f"/fast complete stream={event.stream_id}")
            elif path == "/slow":
                self._http.send_headers(
                    event.stream_id,
                    [
                        (b":status", b"200"),
                        (b"content-type", b"application/octet-stream"),
                        (b"x-lab", b"slow-body"),
                    ],
                )
                self.transmit()
                self._tasks[event.stream_id] = asyncio.create_task(
                    self._slow_body(event.stream_id)
                )

    async def _slow_body(self, stream_id: int) -> None:
        global SERVER_CHUNKS_SENT, SERVER_SEND_ABORTED
        try:
            for i in range(N_CHUNKS):
                self._http.send_data(
                    stream_id, CHUNK, end_stream=(i == N_CHUNKS - 1)
                )
                self.transmit()
                SERVER_CHUNKS_SENT += 1
                if i == 0:
                    stamp(
                        "server",
                        f"started slow body stream={stream_id} chunk=0/{N_CHUNKS}",
                    )
                await asyncio.sleep(CHUNK_PAUSE)
            stamp(
                "server",
                f"slow body FIN stream={stream_id} chunks={SERVER_CHUNKS_SENT}",
            )
```

```
        except asyncio.CancelledError:
            SERVER_SEND_ABORTED = True
            stamp(
                "server",
                f"origin send cancelled stream={stream_id} "
                f"chunks_sent={SERVER_CHUNKS_SENT}/{N_CHUNKS}",
            )
        except RuntimeError as exc:
            SERVER_SEND_ABORTED = True
            stamp(
                "server",
                f"origin send RuntimeError: {exc} chunks_sent={SERVER_CHUNKS_SENT}",
            )


class H3ClientProtocol(QuicConnectionProtocol):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self._http: Optional[H3Connection] = None
        self._slow_id: Optional[int] = None
        self._fast_id: Optional[int] = None
        self._cancelled = False
        self.done = asyncio.get_running_loop().create_future()

    def start_slow(self) -> None:
        self._slow_id = self._quic.get_next_available_stream_id()
        self._http.send_headers(
            self._slow_id,
            [
                (b":method", b"GET"),
                (b":scheme", b"https"),
                (b":authority", b"localhost"),
                (b":path", b"/slow"),
                (b"user-agent", b"cancel-lab/1.0"),
            ],
            end_stream=True,
        )
        self.transmit()
        stamp("client", f"GET /slow stream={self._slow_id}")

    def start_fast(self) -> None:
        self._fast_id = self._quic.get_next_available_stream_id()
        self._http.send_headers(
            self._fast_id,
            [
                (b":method", b"GET"),
                (b":scheme", b"https"),
```

```
                (b":authority", b"localhost"),
                (b":path", b"/fast"),
                (b"user-agent", b"cancel-lab/1.0"),
            ],
            end_stream=True,
        )
        self.transmit()
        stamp("client", f"GET /fast stream={self._fast_id}")

    def cancel_slow(self) -> None:
        if self._cancelled or self._slow_id is None:
            return
        self._cancelled = True
        code = ErrorCode.H3_REQUEST_CANCELLED
        self._quic.stop_stream(self._slow_id, code)
        self._quic.reset_stream(self._slow_id, code)
        self.transmit()
        stamp(
            "client",
            f"cancel stream={self._slow_id} "
            f"STOP_SENDING+RESET_STREAM error_code=0x{int(code):x} "
            f"({int(code)} H3_REQUEST_CANCELLED) bytes_so_far={CLIENT_BYTES}",
        )

    def quic_event_received(self, event: QuicEvent) -> None:
        global CLIENT_SAW_RESET
        if isinstance(event, ProtocolNegotiated):
            stamp("client", f"ALPN={event.alpn_protocol}")
            if event.alpn_protocol in H3_ALPN:
                self._http = H3Connection(self._quic)
                self.start_slow()
        if isinstance(event, StreamReset):
            CLIENT_SAW_RESET = (event.stream_id, event.error_code)
            stamp(
                "client",
                f"StreamReset stream={event.stream_id} "
                f"error_code=0x{event.error_code:x} ({event.error_code})",
            )
        if self._http is not None:
            for http_event in self._http.handle_event(event):
                self._http_event(http_event)

    def _http_event(self, event: H3Event) -> None:
        global CLIENT_BYTES
        if isinstance(event, HeadersReceived):
            status = dict(event.headers).get(b":status", b"?").decode()
            stamp(
```

```
                "client",
                f"headers :status={status} stream={event.stream_id} "
                f"ended={event.stream_ended}",
            )
        elif isinstance(event, DataReceived):
            CLIENT_BYTES += len(event.data)
            if (
                event.stream_id == self._slow_id
                and not self._cancelled
                and CLIENT_BYTES >= 512
            ):
                stamp("client", f"received {CLIENT_BYTES} bytes of /slow; cancelling")
                self.cancel_slow()
                asyncio.get_running_loop().call_later(0.15, self.start_fast)
            if event.stream_id == self._fast_id:
                stamp(
                    "client",
                    f"/fast data={event.data!r} ended={event.stream_ended}",
                )
                if event.stream_ended and not self.done.done():
                    self.done.set_result(True)


async def main() -> None:
    import aioquic

    print(f"python {sys.version.split()[0]}")
    print(f"aioquic {aioquic.__version__}")
    print(
        f"H3_REQUEST_CANCELLED = {int(ErrorCode.H3_REQUEST_CANCELLED)} "
        f"(0x{int(ErrorCode.H3_REQUEST_CANCELLED):x})"
    )
    print(
        f"H3_NO_ERROR          = {int(ErrorCode.H3_NO_ERROR)} "
        f"(0x{int(ErrorCode.H3_NO_ERROR):x})"
    )
    print()

    server_logger = QuicLogger()
    client_logger = QuicLogger()
    server_conf = QuicConfiguration(
        alpn_protocols=H3_ALPN, is_client=False, quic_logger=server_logger
    )
    server_conf.load_cert_chain("cert.pem", "key.pem")
    server = await serve(
        "127.0.0.1", 4433, configuration=server_conf, create_protocol=H3ServerProtocol
    )
```

```
    stamp("lab", "HTTP/3 server listening on 127.0.0.1:4433")

    client_conf = QuicConfiguration(
        alpn_protocols=H3_ALPN,
        is_client=True,
        server_name="localhost",
        quic_logger=client_logger,
    )
    client_conf.verify_mode = ssl.CERT_NONE
    try:
        async with connect(
            "127.0.0.1",
            4433,
            configuration=client_conf,
            create_protocol=H3ClientProtocol,
        ) as client:
            await asyncio.wait_for(client.done, timeout=5)
            await asyncio.sleep(0.2)
            client.close()
            await client.wait_closed()
    finally:
        server.close()

    print("\n# summary")
    print(f"server chunks sent     : {SERVER_CHUNKS_SENT}/{N_CHUNKS}")
    print(f"server send aborted    : {SERVER_SEND_ABORTED}")
    print(f"server StopSending     : {SERVER_SAW_STOP_SENDING}")
    print(f"server StreamReset     : {SERVER_SAW_RESET}")
    print(f"client bytes (end)     : {CLIENT_BYTES}")
    print(f"client StreamReset     : {CLIENT_SAW_RESET}")
    dump_cancel_frames("client", client_logger)
    dump_cancel_frames("server", server_logger)


if __name__ == "__main__":
    asyncio.run(main())
```

The listing above is the lab that ran, with one print-label tweak (`client bytes (end)`) so the summary is not misread as "bytes at cancel." aioquic 1.7.0 reports `H3_REQUEST_CANCELLED = 268`, which is 0x010c.

```
$ python cancel_lab.py
python 3.13.5
aioquic 1.7.0
H3_REQUEST_CANCELLED = 268 (0x10c)
H3_NO_ERROR          = 256 (0x100)
```

```
  0.003s  lab     HTTP/3 server listening on 127.0.0.1:4433
  0.016s  server  ALPN=h3
  0.017s  client  ALPN=h3
  0.018s  client  GET /slow stream=0
  0.019s  server  request GET /slow stream=0
  0.019s  server  started slow body stream=0 chunk=0/40
  0.020s  client  headers :status=200 stream=0 ended=False
  0.071s  client  received 512 bytes of /slow; cancelling
  0.071s  client  cancel stream=0 STOP_SENDING+RESET_STREAM error_code=0x10c (268 H3_REQUEST
  0.071s  server  StopSendingReceived stream=0 error_code=0x10c (268)
  0.071s  server  origin send cancelled stream=0 chunks_sent=2/40
  0.071s  client  StreamReset stream=0 error_code=0x10c (268)
  0.222s  client  GET /fast stream=4
  0.222s  server  request GET /fast stream=4
  0.223s  server  /fast complete stream=4
  0.223s  client  headers :status=200 stream=4 ended=False
  0.223s  client  /fast data=b'ok\n' ended=True

# summary
server chunks sent     : 2/40
server send aborted    : True
server StopSending     : (0, 268)
server StreamReset     : None
client bytes (end)     : 515
client StreamReset     : (0, 268)

# qlog client: STOP_SENDING / RESET_STREAM frames
  client  transport:packet_sent         stop_sending  stream=0  error_code=268
  client  transport:packet_received     reset_stream  stream=0  error_code=268  final_size=5

# qlog server: STOP_SENDING / RESET_STREAM frames
  server  transport:packet_received     stop_sending  stream=0  error_code=268
  server  transport:packet_sent         reset_stream  stream=0  error_code=268  final_size=5
```

Read it in order:

1. **ALPN `h3`.** Both sides negotiated HTTP/3. Stream 0 is the first client-initiated bidirectional stream.
2. **Cancel is `STOP_SENDING`, not a connection close.** qlog shows the client *sending* `stop_sending` with `error_code=268` (0x010c) and *not* sending `reset_stream`. The GET already had FIN; aioquic's `reset_stream()` is a no-op once the send side is finished (RFC 9000 §3.1: no `RESET_STREAM` from a terminal send state). RFC 9114 §4.1.1 still says to reset sending parts that are *still open* — here they were not.
3. **The server MUST answer `STOP_SENDING` with `RESET_STREAM`.** aioquic copies the error code (RFC 9000 §3.5). The client event is `StreamReset stream=0 error_code=0x10c`. qlog `final_size=555` is the STREAM offset the server had committed: two 256-byte DATA chunks plus the HEADERS framing (~43 bytes).
4. **The origin stopped.** 2 of 40 chunks. Without the `task.cancel()` on `StopSendingReceived`,

the handler would keep calling `send_data` until aioquic raised `RuntimeError: Cannot send data after the stream was reset`. That is the "proxy that does not propagate" failure, on one box.

5. **The connection survived.** `GET /fast` on stream 4 returned `200` and `ok\n` 150 ms later. That is the whole point of multiplexed abort. HTTP/1.1 cannot do this: the cancel would have been a TCP close, and `/fast` would have needed a new connection.
6. **515 vs 512.** The cancel fired at 512 body bytes. The summary's 515 is bytes counted through the end of the run — three extra bytes of STREAM data already in flight. RFC 9000 §3.5: STREAM frames received after `STOP_SENDING` still count toward flow control even when discarded.

nginx was not run here. Its 499 is a log-only code for "downstream closed while we still owed a response," documented by nginx and by Cloudflare's Error 499 page. The lab's `StopSendingReceived` is the HTTP/3 event that a proxy should turn into that log line *and* an upstream cancel.

The trap

- **Closing the QUIC connection to cancel one request.** That is the HTTP/1.1 move. It dumps every other stream, burns the handshake, and wrecks 0-RTT. Cancel the stream.
- **Waiting for origin 200 after downstream cancel.** The user is gone. Every extra millisecond is origin work you will throw away, then log as 200 and lie to your SLO.
- **Putting 499 in the 5xx burn rate.** 499 means the client left. After you start honouring HTTP/3 `STOP_SENDING`, 499 volume on HTTP/3 will rise. That is the metric becoming honest.
- **Sending `H3_REQUEST_REJECTED` after you processed the request.** RFC 9114 §4.1.1: that code means "safe to retry as if never sent." Use `H3_REQUEST_CANCELLED` once a handler has run.
- **Sending `H3_NO_ERROR` for a user abort.** 0x0100 is "no error" (complete response, leftover request body; graceful GOAWAY). User cancel is 0x010c.
- **Ignoring `STOP_SENDING` in the application.** The transport will `RESET_STREAM` for you. The database query will not, unless you cancel the task.

The boring rule

Cancel is a stream event, not a connection event. On HTTP/3 the client sends `STOP_SENDING` (and `RESET_STREAM` if it is still sending) with `H3_REQUEST_CANCELLED`. The peer MUST `RESET_STREAM`. The proxy copies that to the origin immediately and logs 499, excluded from server errors. The origin aborts the handler and treats writes as retryable only when they are actually idempotent. Sibling streams keep running.

Try this

1. Change the client to cancel with `ErrorCode.H3_NO_ERROR` (0x0100). Predict the qlog `error_code` on both `stop_sending` and the server's `reset_stream`, then run it. The origin

still stops; your cancel metrics that filter on 0x010c go silent.
2. Comment out `task.cancel()` on the server. How many extra chunks are sent, and do you hit `RuntimeError: Cannot send data after the stream was reset`?
3. Turn `/slow` into a POST whose body is sent in 20 chunks without FIN. Cancel after the first response byte. Confirm a client-originated `reset_stream` now appears in qlog alongside `stop_sending`.
4. Start two `/slow` requests in parallel, cancel only stream 0, and confirm stream 4 still delivers 40 chunks. That is the HTTP/1.1 counterfactual.

Sources

- RFC 9000 — QUIC: A UDP-Based Multiplexed and Secure Transport (§2.4 operations, §3 stream states, §3.5 solicited state transitions, §19.4 `RESET_STREAM`, §19.5 `STOP_SENDING`): https://www.rfc-editor.org/rfc/rfc9000.html
- RFC 9114 — HTTP/3 (§4.1 leftover request body / `H3_NO_ERROR`, §4.1.1 request cancellation and rejection, §8.1 error codes, Appendix A.4 mapping from HTTP/2, Appendix A.4.1 intermediaries): https://www.rfc-editor.org/rfc/rfc9114.html
- RFC 9113 — HTTP/2 (§5.1 stream states, §5.4.2 stream errors, §6.4 `RST_STREAM`, §7 `CANCEL` 0x08): https://www.rfc-editor.org/rfc/rfc9113.html
- RFC 9112 — HTTP/1.1 (§9 connection management; abort is connection close): https://www.rfc-editor.org/rfc/rfc9112.html
- RFC 9110 — HTTP Semantics (idempotent methods; GET/PUT/DELETE retry): https://www.rfc-editor.org/rfc/rfc9110.html
- IANA HTTP/3 error codes (`H3_NO_ERROR` 0x0100, `H3_REQUEST_REJECTED` 0x010b, `H3_REQUEST_CANCELLED` 0x010c): https://www.iana.org/assignments/http3-parameters
- aioquic 1.7.0 (`H3Connection`, `ErrorCode`, `stop_stream` / `reset_stream`, qlog): https://aioquic.readthedocs.io/en/latest/ and https://pypi.org/project/aioquic/1.7.0/
- aioquic `QuicConnection.stop_stream` / `reset_stream`: https://aioquic.readthedocs.io/en/latest/quic.html
- Cloudflare changelog, 9 October 2026 — Improved HTTP/3 client cancellation reporting: https://developers.cloudflare.com/changelog/post/2026-10-09-http3-499-reporting-improvement/
- Cloudflare Error 499 (nginx `499 Client Closed Request` in logs and analytics): https://developers.cloudflare.com/support/troubleshooting/http-status-codes/4xx-client-error/error-499/
- nginx source — `NGX_HTTP_CLIENT_CLOSED_REQUEST` 499 in `ngx_http_request.h`: https://hg.nginx.org/nginx/file/tip/src/http/ngx_http_request.h

Inside the TLS 1.3 Key Schedule: HKDF, Three Secrets, and Every Key in a Handshake

Inside the TLS 1.3 Key Schedule: HKDF, Three Secrets, and Every Key in a Handshake

Every TLS 1.3 connection turns one shared secret into a dozen keys, IVs and MACs: separate keys for the handshake and for application data, separate keys per direction, a Finished MAC per side, an exporter secret, and a ticket PSK for resumption. All of them come from one small, fully specified computation, the **key schedule**: two HKDF primitives, two wrappers and about thirty lines of Python. This chapter writes it from scratch with the standard library and checks it, value by value, against the RFC 8448 example handshake. Then it uses the same functions to decrypt RFC 8448's records and a live OpenSSL 3.5 connection that negotiated the post-quantum hybrid group. It finishes with the one-line bugs that break it and where each one shows up.

Learning goals

- Write HKDF-Extract and HKDF-Expand (RFC 5869), HKDF-Expand-Label and Derive-Secret (RFC 9846 §7.1), and say what each input is for
- Draw the three-stage Extract ladder (Early, Handshake, Main) and say which transcript each derived secret covers
- Derive record keys and IVs (§7.3), build a record nonce (§5.3), and compute a Finished MAC (§4.5.3)
- Verify an implementation against RFC 8448's trace, and read a live connection with an RFC 9850 key log
- Recognise the classic mistakes from the first value that goes wrong
- Say what a hybrid X25519MLKEM768 key exchange (RFC 10024) changes in the schedule, and what the proposed XOF schedule draft would change

Lab platform: Python 3.13.5 and its standard library (`hashlib`, `hmac`) on Linux 6.12. Labs 3 and 4 also use **cryptography 50.0.2** from PyPI in a virtual environment (`python3 -m venv venv && venv/bin/pip install cryptography==50.0.2`) for AES-GCM and a throwaway certificate, and the system **OpenSSL 3.5.7** through Python's `ssl` module. No root. Apart from fetching the RFC 8448 text once, nothing touches the network: the live handshake runs in memory. Every output below is pasted from a run.

This chapter doesn't reteach the TLS handshake message flow or certificates. It is about the keys.

Mental model

Think of the schedule as two moves, repeated:

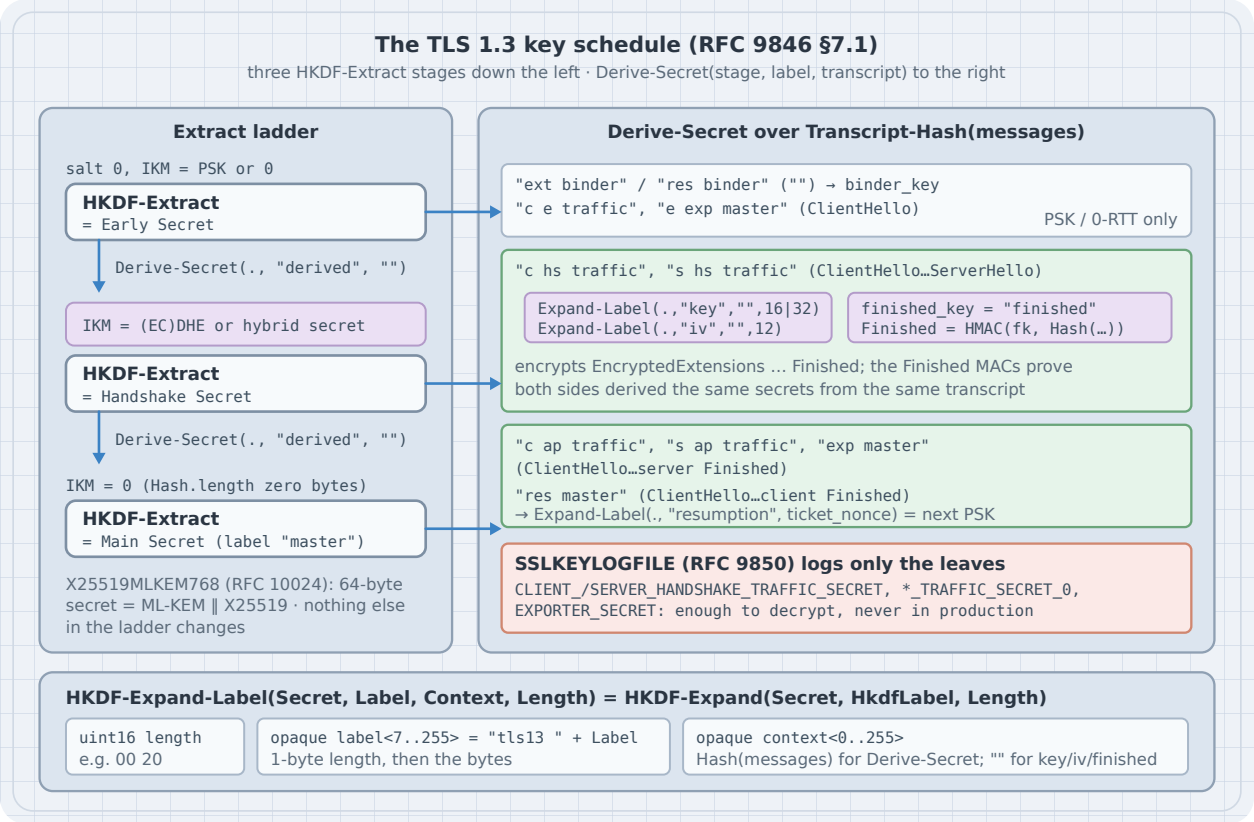


Figure 1: The TLS 1.3 key schedule: a ladder of three HKDF-Extract stages, the secrets each stage derives over which transcript, the leaf keys, and HkdfLabel

- **Extract** (down the left) mixes a new input secret into the running state: first the PSK (or zeros), then the key-exchange secret, then zeros. Each stage's output is pure entropy with no context.
- **Derive** (to the right) binds that entropy to a *purpose* (the label) and to *what was said so far* (a hash of the handshake messages). A traffic secret is only usable by someone who saw the same transcript.

Between stages, `Derive-Secret(., "derived", "")` turns the old stage into the salt for the next. Nothing is skipped: without a PSK, the Early Secret is still `HKDF-Extract(0, 0)`, and "0" means `Hash.length` zero bytes (RFC 9846 §7.1).

Concepts

HKDF in two functions

HKDF (RFC 5869) is HMAC used twice:

- `HKDF-Extract(salt, IKM) = HMAC(salt, IKM)` concentrates possibly non-uniform input keying material into a pseudorandom key (PRK). An absent salt means `HashLen` zero bytes.
- `HKDF-Expand(PRK, info, L)` stretches the PRK to L bytes: `T(1) = HMAC(PRK, info ‖ 0x01)`, `T(i) = HMAC(PRK, T(i-1) ‖ info ‖ i)`, concatenated and truncated.

The hash is the cipher suite's: SHA-256 for `TLS_AES_128_GCM_SHA256` and `TLS_CHACHA20_POLY1305_SHA256`, SHA-384 for `TLS_AES_256_GCM_SHA384`.

The two TLS wrappers

TLS never calls HKDF-Expand with free-form `info`. It always passes a serialised `HkdfLabel`:

```
HKDF-Expand-Label(Secret, Label, Context, Length) = HKDF-Expand(Secret, HkdfLabel, Length)

struct {
    uint16 length = Length;
    opaque label<7..255> = "tls13 " + Label;      // 1-byte length prefix, then the bytes
    opaque context<0..255> = Context;             // 1-byte length prefix, then the bytes
} HkdfLabel;

Derive-Secret(Secret, Label, Messages) =
    HKDF-Expand-Label(Secret, Label, Transcript-Hash(Messages), Hash.length)
```

Three properties matter in practice:

- **Each output is bound to its length and its purpose.** "key" with length 16 and "key" with length 32 are different values.

- **The transcript enters as a hash.** `Transcript-Hash(M1, …, Mn) = Hash(M1 ‖ … ‖ Mn)` over the Handshake structures (type, 3-byte length, body), without record headers (§4.1). `Derive-Secret(., "derived", "")` therefore hashes the *empty string*: its context is `e3b0c442…`, SHA-256 of nothing, not zero bytes.
- **Labels are short on purpose.** The "tls13" prefix replaced "TLS 1.3," in draft-20 (2017), to "fit within one compression block". RFC 9846 renamed the master secret to "main secret" but kept the wire label `"master"` for compatibility.

Leaf keys, nonces and Finished

What	Computation	Section
Record key, IV	`Expand-Label(traffic_secret, "key", "", key_len)` and `(…, "iv", "", 12)`	RFC 9846 §7.3
Record nonce	the 64-bit record sequence number, left-padded to 12 bytes, XOR the IV. The sequence restarts at 0 for each new key	§5.3
Record AAD	the 5-byte record header (`17 03 03` + length)	§5.2
Inner plaintext	content ‖ real content type (1 byte) ‖ zero padding	§5.2
Finished	`HMAC(Expand-Label(base, "finished", "", Hash.length), Transcript-Hash(…up to the message before Finished))`; `base` is the sender's handshake traffic secret	§4.5.3
Next update	`Expand-Label(application_traffic_secret_N, "traffic upd", "", Hash.length)`	§7.2
Resumption PSK	`Expand-Label(resumption_secret, "resumption", ticket_nonce, Hash.length)`	§4.7.1

Lab 1: HKDF from the standard library

```python
# hkdf.py - HKDF (RFC 5869) and the two TLS 1.3 wrappers (RFC 9846 §7.1), standard library o
import hashlib, hmac

H = hashlib.sha256            # default: TLS_AES_128_GCM_SHA256; pass h=hashlib.sha384 for A
HASH_LEN = H().digest_size

def hkdf_extract(salt: bytes, ikm: bytes, h=H) -> bytes:
    return hmac.new(salt or bytes(h().digest_size), ikm, h).digest()   # no salt = HashLen z

def hkdf_expand(prk: bytes, info: bytes, length: int, h=H) -> bytes:
```

```
    out, t, i = b"", b"", 1
    while len(out) < length:
        t = hmac.new(prk, t + info + bytes([i]), h).digest()
        out += t; i += 1
    return out[:length]

def hkdf_label(label: str, context: bytes, length: int) -> bytes:
    """struct HkdfLabel { uint16 length; opaque label<7..255>; opaque context<0..255>; }"""
    full = b"tls13 " + label.encode()
    return length.to_bytes(2, "big") + bytes([len(full)]) + full + bytes([len(context)]) + c

def expand_label(secret: bytes, label: str, context: bytes, length: int, h=H) -> bytes:
    return hkdf_expand(secret, hkdf_label(label, context, length), length, h)

def derive_secret(secret: bytes, label: str, messages: bytes, h=H) -> bytes:
    return expand_label(secret, label, h(messages).digest(), h().digest_size, h)

if __name__ == "__main__":          # RFC 5869 Appendix A.1, test case 1
    ikm  = bytes.fromhex("0b" * 22)
    salt = bytes.fromhex("000102030405060708090a0b0c")
    info = bytes.fromhex("f0f1f2f3f4f5f6f7f8f9")
    prk = hkdf_extract(salt, ikm)
    okm = hkdf_expand(prk, info, 42)
    print("PRK", prk.hex())
    print("OKM", okm.hex())
    assert prk.hex() == "077709362c2e32df0ddc3f0dc47bba6390b6c73bb50f9c3122ec844ad7c2b3e5"
    assert okm.hex() == ("3cb25f25faacd57a90434f64d0362f2a2d2d0a90cf1a5a4c5db02d56ecc4c5bf"
                         "34007208d5b887185865")
    print("RFC 5869 A.1: ok")
```

```
$ python3 hkdf.py
PRK 077709362c2e32df0ddc3f0dc47bba6390b6c73bb50f9c3122ec844ad7c2b3e5
OKM 3cb25f25faacd57a90434f64d0362f2a2d2d0a90cf1a5a4c5db02d56ecc4c5bf34007208d5b887185865
RFC 5869 A.1: ok
```

Nothing TLS-specific is checked yet: these are RFC 5869's own test vectors. The two TLS wrappers are six lines on top.

Lab 2: the whole schedule against RFC 8448

RFC 8448 §3 is a complete example handshake (X25519, `TLS_AES_128_GCM_SHA256`, server-only authentication), with every private key, message and intermediate value printed in hex. A small parser turns its text into Python bytes:

```
# rfc8448.py - read the labelled hex values of one RFC 8448 trace into Python
import re, urllib.request, pathlib

def load(section="3.  Simple 1-RTT Handshake", end="4.  Resumed 0-RTT Handshake"):
    path = pathlib.Path("rfc8448.txt")
    if not path.exists():
        urllib.request.urlretrieve("https://www.rfc-editor.org/rfc/rfc8448.txt", path)
    text = path.read_text()
    text = text[text.index("\n" + section) : text.index("\n" + end)]
    lines = [l for l in text.splitlines()          # drop page footers/headers
             if not re.match(r"(Thomson .*\[Page \d+\]|RFC 8448 .*January 2019|\f)", l)]
    steps, step, item = [], None, None
    for l in lines:
        if m := re.match(r"   \{(client|server)\}  (.*)", l):
            step = {"who": m[1], "what": m[2], "values": {}}; steps.append(step); item = Non
        elif step and not step["values"] and re.match(r"      \S", l) and "octets" not in l
                and not l.strip().endswith(":"):
            step["what"] += " " + l.strip()                # step title wrapped onto 2 line
        elif m := re.match(r"      (\S.*?) \((\d+) octets\):\s+(.*)", l):
            item = m[1]; step["values"][item] = m[3]
        elif item and re.match(r"         [0-9a-f]{2}( |$)", l):
            step["values"][item] += " " + l.strip()
        elif l.strip():
            item = None                                    # blank lines (page breaks) keep
    for s in steps:
        s["values"] = {k: bytes.fromhex(v.replace("(empty)", "")) for k, v in s["values"].it
    return steps

def get(steps, who, what, name):
    """The value `name` from the first step by `who` whose title starts with `what`."""
    for s in steps:
        if s["who"] == who and s["what"].startswith(what) and name in s["values"]:
            return s["values"][name]
    raise KeyError((who, what, name))
```

X25519 itself is short enough to write out from RFC 7748 §5. That way even the shared secret is computed here, not copied:

```
# x25519.py - the X25519 function from RFC 7748 §5 (Montgomery ladder), standard library onl
P = 2**255 - 19
A24 = 121665

def _clamp(k: bytes) -> int:
    k = bytearray(k); k[0] &= 248; k[31] &= 127; k[31] |= 64
    return int.from_bytes(k, "little")

def x25519(scalar: bytes, u: bytes) -> bytes:
```

```
    k = _clamp(scalar)
    x1 = int.from_bytes(u, "little") & ((1 << 255) - 1)
    x2, z2, x3, z3, swap = 1, 0, x1, 1, 0
    for t in reversed(range(255)):
        bit = (k >> t) & 1
        swap ^= bit
        if swap: x2, x3, z2, z3 = x3, x2, z3, z2
        swap = bit
        a, b = (x2 + z2) % P, (x2 - z2) % P
        aa, bb = a * a % P, b * b % P
        e = (aa - bb) % P
        c, d = (x3 + z3) % P, (x3 - z3) % P
        da, cb = d * a % P, c * b % P
        x3, z3 = (da + cb) ** 2 % P, x1 * (da - cb) ** 2 % P
        x2, z2 = aa * bb % P, e * (aa + A24 * e) % P
    if swap: x2, x3, z2, z3 = x3, x2, z3, z2
    return (x2 * pow(z2, P - 2, P) % P).to_bytes(32, "little")

BASE = (9).to_bytes(32, "little")
```

Now the schedule, top to bottom, each value compared with the RFC's:

```
# schedule.py - the whole TLS 1.3 key schedule for the RFC 8448 §3 handshake, checked value
from hkdf import hkdf_extract, derive_secret, expand_label, H, HASH_LEN
from x25519 import x25519, BASE
import hmac, rfc8448

T = rfc8448.load()
def rfc(who, what, name): return rfc8448.get(T, who, what, name)
checked = 0
def check(name, mine, theirs):
    global checked; checked += 1
    print(f"{name:<26} {mine[:8].hex()}…  {'ok' if mine == theirs else 'MISMATCH'}")
    assert mine == theirs, name

# 1. ECDHE: each side combines its private key with the other's public key
c_priv, c_pub = rfc("client", "create an ephemeral", "private key"), rfc("client", "create a
s_priv, s_pub = rfc("server", "create an ephemeral", "private key"), rfc("server", "create a
check("client public = priv·9", x25519(c_priv, BASE), c_pub)
shared_c, shared_s = x25519(c_priv, s_pub), x25519(s_priv, c_pub)
check("ECDHE (client = server)", shared_c, shared_s)
check("ECDHE vs RFC 8448", shared_c, rfc("server", 'extract secret "handshake"', "IKM"))

# 2. the handshake messages, in order (Handshake structs: type, uint24 length, body)
CH  = rfc("client", "construct a ClientHello", "ClientHello")
SH  = rfc("server", "construct a ServerHello", "ServerHello")
EE  = rfc("server", "construct an EncryptedExtensions", "EncryptedExtensions")
```

```
CRT = rfc("server", "construct a Certificate", "Certificate")
CV  = rfc("server", "construct a CertificateVerify", "CertificateVerify")
SF  = rfc("server", "construct a Finished", "Finished")
CF  = rfc("client", "construct a Finished", "Finished")

# 3. the schedule (RFC 8446 §7.1)
zeros = bytes(HASH_LEN)
early   = hkdf_extract(b"", zeros)                          # no PSK: IKM = 32 zero bytes
check("early secret", early, rfc("server", 'extract secret "early"', "secret"))
derived = derive_secret(early, "derived", b"")
check('derived (from early)', derived, rfc("server", 'derive secret for handshake "tls13 der
hs      = hkdf_extract(derived, shared_c)
check("handshake secret", hs, rfc("server", 'extract secret "handshake"', "secret"))
c_hs    = derive_secret(hs, "c hs traffic", CH + SH)
s_hs    = derive_secret(hs, "s hs traffic", CH + SH)
check("client hs traffic", c_hs, rfc("server", 'derive secret "tls13 c hs traffic"', "expand
check("server hs traffic", s_hs, rfc("server", 'derive secret "tls13 s hs traffic"', "expand
derived2 = derive_secret(hs, "derived", b"")
master  = hkdf_extract(derived2, zeros)
check("master secret", master, rfc("server", 'extract secret "master"', "secret"))

# 4. traffic keys (§7.3) for the server's handshake flight
key, iv = expand_label(s_hs, "key", b"", 16), expand_label(s_hs, "iv", b"", 12)
check("server hs key", key, rfc("server", "derive write traffic keys for handshake", "key ex
check("server hs iv", iv,  rfc("server", "derive write traffic keys for handshake", "iv expa

# 5. Finished (§4.4.4): HMAC over the transcript so far, keyed from the sender's hs secret
def finished(base, transcript):
    fk = expand_label(base, "finished", b"", HASH_LEN)
    return hmac.new(fk, H(transcript).digest(), H).digest()
check("server Finished", finished(s_hs, CH + SH + EE + CRT + CV), SF[4:])
check("client Finished", finished(c_hs, CH + SH + EE + CRT + CV + SF), CF[4:])

# 6. application secrets cover the transcript up to the *server* Finished
upto_sf = CH + SH + EE + CRT + CV + SF
c_ap = derive_secret(master, "c ap traffic", upto_sf)
s_ap = derive_secret(master, "s ap traffic", upto_sf)
check("client ap traffic", c_ap, rfc("server", 'derive secret "tls13 c ap traffic"', "expand
check("server ap traffic", s_ap, rfc("server", 'derive secret "tls13 s ap traffic"', "expand
check("exporter master", derive_secret(master, "exp master", upto_sf),
      rfc("server", 'derive secret "tls13 exp master"', "expanded"))
res = derive_secret(master, "res master", upto_sf + CF)  # resumption also covers the client
check("resumption master", res, rfc("client", 'derive secret "tls13 res master"', "expanded"
check("PSK for ticket nonce 0000", expand_label(res, "resumption", b"\x00\x00", HASH_LEN),
      rfc("server", 'generate resumption secret', "expanded"))
print(f"all {checked} values match RFC 8448 §3")
```

```
$ python3 schedule.py
client public = priv·9      99381de560e4bd43…  ok
ECDHE (client = server)     8bd4054fb55b9d63…  ok
ECDHE vs RFC 8448           8bd4054fb55b9d63…  ok
early secret                33ad0a1c607ec03b…  ok
derived (from early)        6f2615a108c702c5…  ok
handshake secret            1dc826e93606aa6f…  ok
client hs traffic           b3eddb126e067f35…  ok
server hs traffic           b67b7d690cc16c4e…  ok
master secret               18df06843d13a08b…  ok
server hs key               3fce516009c21727…  ok
server hs iv                5d313eb2671276ee…  ok
server Finished             9b9b141d906337fb…  ok
client Finished             a8ec436d677634ae…  ok
client ap traffic           9e40646ce79a7f9d…  ok
server ap traffic           a11af9f05531f856…  ok
exporter master             fe22f881176eda18…  ok
resumption master           7df235f2031d2a05…  ok
PSK for ticket nonce 0000   4ecd0eb6ec3b4d87…  ok
all 18 values match RFC 8448 §3
```

Every intermediate in the trace comes out of about thirty lines of HKDF and the message bytes. Two things to notice:

- **Each Finished covers a different transcript.** The server's covers everything up to its CertificateVerify. The client's also covers the server's Finished.
- **The application secrets stop at the server Finished.** That's why a server may send application data before the client's Finished arrives (the second case in §4.5.3). The resumption secret is the only one that needs the client Finished.

Lab 3: open RFC 8448's records with those keys

The trace also prints every encrypted record. Deriving the right key and IV is necessary but not sufficient. The nonce and the AAD must be right too, or AES-GCM refuses.

```
# decrypt.py - open RFC 8448's encrypted records with the keys from schedule.py (needs `cryp
import contextlib, io
with contextlib.redirect_stdout(io.StringIO()):          # reuse schedule.py's secrets, quie
    from schedule import rfc, expand_label, s_hs, c_hs, s_ap, c_ap
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

def open_record(secret, seq, record):
    key, iv = expand_label(secret, "key", b"", 16), expand_label(secret, "iv", b"", 12)
    nonce = bytes(a ^ b for a, b in zip(iv, seq.to_bytes(12, "big")))   # §5.3: iv XOR seq
    header, body = record[:5], record[5:]                               # §5.2: AAD = heade
    inner = AESGCM(key).decrypt(nonce, body, header)
```

```
    content = inner.rstrip(b"\x00")                        # strip zero padding, then the typ
    return header, content[:-1], content[-1]

NAMES = {0x08: "EncryptedExtensions", 0x0b: "Certificate", 0x0f: "CertificateVerify",
         0x14: "Finished", 0x04: "NewSessionTicket"}
def walk(msgs):                          # split a handshake byte stream into its messages
    while msgs:
        n = int.from_bytes(msgs[1:4], "big")
        yield NAMES.get(msgs[0], hex(msgs[0])), n; msgs = msgs[4 + n:]

T = [("server", "send handshake record", 1, s_hs, 0, "server handshake flight"),
     ("client", "send handshake record", 1, c_hs, 0, "client Finished"),
     ("server", "send handshake record", 2, s_ap, 0, "NewSessionTicket"),
     ("client", "send application_data", 0, c_ap, 0, "client app data"),
     ("server", "send application_data", 0, s_ap, 1, "server app data")]
import rfc8448
for who, what, nth, secret, seq, label in T:
    steps = [s for s in rfc8448.load() if s["who"] == who and s["what"].startswith(what)]
    rec, want = steps[nth]["values"]["complete record"], steps[nth]["values"]["payload"]
    header, plain, ctype = open_record(secret, seq, rec)
    kind = {22: "handshake", 23: "application_data", 21: "alert"}[ctype]
    print(f"{label:<24} {header.hex(' ')}  seq={seq}  → inner type {ctype} ({kind}),"
          f" {len(plain)} bytes, {'= RFC payload' if plain == want else 'MISMATCH'}")
    if ctype == 22:
        print("    " + ", ".join(f"{n} ({l})" for n, l in walk(plain)))
    else:
        print("    " + plain.hex(" ", 2)[:59] + " …")
```

```
$ venv/bin/python decrypt.py
server handshake flight  17 03 03 02 a2  seq=0  → inner type 22 (handshake), 657 bytes, = RF
    EncryptedExtensions (36), Certificate (441), CertificateVerify (132), Finished (32)
client Finished          17 03 03 00 35  seq=0  → inner type 22 (handshake), 36 bytes, = RFC
    Finished (32)
NewSessionTicket         17 03 03 00 de  seq=0  → inner type 22 (handshake), 205 bytes, = RF
    NewSessionTicket (201)
client app data          17 03 03 00 43  seq=0  → inner type 23 (application_data), 50 bytes
    0001 0203 0405 0607 0809 0a0b 0c0d 0e0f 1011 1213 1415 1617 …
server app data          17 03 03 00 43  seq=1  → inner type 23 (application_data), 50 bytes
    0001 0203 0405 0607 0809 0a0b 0c0d 0e0f 1011 1213 1415 1617 …
```

- The server's whole encrypted flight (EncryptedExtensions, Certificate, CertificateVerify, Finished: 657 bytes) travels in one record at sequence number 0 under the server handshake key.
- The record header says `17` (application_data) even for handshake data. The real type (22) is the last byte of the decrypted plaintext (§5.2). An observer sees only `17 03 03` and a length.
- The server's NewSessionTicket and its first application data share the **server application key**, at sequence numbers 0 and 1. The client's first application record is sequence 0 under a

different key: each direction and each key has its own counter.

Lab 4: a live OpenSSL 3.5 connection, through its key log

RFC 8448 is from 2019. A connection made today by OpenSSL 3.5 differs in three visible ways: it prefers `TLS_AES_256_GCM_SHA384` (so SHA-384 everywhere), it sends a change_cipher_spec record for middlebox compatibility, and its default key exchange is the hybrid **X25519MLKEM768** (group `0x11EC`, RFC 10024).

You can't see a live connection's ECDHE private key, but you can ask the client to write an **SSLKEYLOGFILE** (RFC 9850). Python's `SSLContext.keylog_filename` does exactly that. The lab runs client and server in one process over `ssl.MemoryBIO` pairs, keeps every byte each side sends, then opens the records with `hkdf.py` and the logged secrets:

```
# live.py - a real OpenSSL TLS 1.3 handshake in memory, opened with hkdf.py and the key log
import ssl, os, tempfile, datetime, hashlib, hmac
from cryptography import x509
from cryptography.x509.oid import NameOID
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import ec
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from hkdf import expand_label

tmp = tempfile.mkdtemp()
key = ec.generate_private_key(ec.SECP256R1())                       # throwaway self-signed ce
name = x509.Name([x509.NameAttribute(NameOID.COMMON_NAME, "lab.test")])
now = datetime.datetime.now(datetime.timezone.utc)
cert = (x509.CertificateBuilder().subject_name(name).issuer_name(name).public_key(key.public
        .serial_number(1).not_valid_before(now).not_valid_after(now + datetime.timedelta(day
        .add_extension(x509.SubjectAlternativeName([x509.DNSName("lab.test")]), False)
        .sign(key, hashes.SHA256()))
open(f"{tmp}/c.pem", "wb").write(cert.public_bytes(serialization.Encoding.PEM))
open(f"{tmp}/k.pem", "wb").write(key.private_bytes(serialization.Encoding.PEM,
     serialization.PrivateFormat.PKCS8, serialization.NoEncryption()))

sctx = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER); sctx.load_cert_chain(f"{tmp}/c.pem", f"{tmp}
cctx = ssl.SSLContext(ssl.PROTOCOL_TLS_CLIENT); cctx.load_verify_locations(f"{tmp}/c.pem")
cctx.keylog_filename = f"{tmp}/keys.log"                            # what SSLKEYLOGFILE write
for c in (sctx, cctx): c.minimum_version = ssl.TLSVersion.TLSv1_3

bios = {s: (ssl.MemoryBIO(), ssl.MemoryBIO()) for s in ("c", "s")}
cli = cctx.wrap_bio(*bios["c"], server_hostname="lab.test")
srv = sctx.wrap_bio(*bios["s"], server_side=True)
wire = {"c": b"", "s": b""}                                           # every byte each side s
def pump():
    for me, peer in (("c", "s"), ("s", "c")):
```

```
        data = bios[me][1].read()
        wire[me] += data; bios[peer][0].write(data)
for _ in range(6):
    for side in (cli, srv):
        try: side.do_handshake()
        except ssl.SSLWantReadError: pass
    pump()
cli.write(b"GET / HTTP/1.1\r\n\r\n"); pump(); srv.read(); srv.write(b"hello from OpenSSL");
print(cli.version(), cli.cipher()[0], "| OpenSSL", ssl.OPENSSL_VERSION.split()[1])

def records(stream):
    while stream:
        n = int.from_bytes(stream[3:5], "big"); yield stream[:5 + n]; stream = stream[5 + n:
ch_rec, *c_rest = list(records(wire["c"]))
sh_rec, *s_rest = list(records(wire["s"]))
CH, SH = ch_rec[5:], sh_rec[5:]
def extensions(hello):                                          # ServerHello body → {ty
    i = 4 + 2 + 32; i += 1 + hello[i]; i += 2 + 1 + 2            # version, random, sess
    out = {}
    while i < len(hello):
        t, n = int.from_bytes(hello[i:i+2], "big"), int.from_bytes(hello[i+2:i+4], "big")
        out[t] = hello[i+4:i+4+n]; i += 4 + n
    return out
ks = extensions(SH)[0x33]                                       # key_share: group, key_
print(f"ServerHello key_share: group 0x{ks[:2].hex()}, {int.from_bytes(ks[2:4], 'big')} byte
      " | server record types:", " ".join(f"{r[0]}" for r in s_rest))

client_random = CH[6:38].hex()
log = {}
for line in open(f"{tmp}/keys.log"):
    if line.startswith("#"): continue
    label, rnd, secret = line.split()
    if rnd == client_random: log[label] = bytes.fromhex(secret)
print("key log labels:", ", ".join(log))

suite = cli.cipher()[0]                                         # TLS_AES_256_GCM_SHA384
h, klen = (hashlib.sha384, 32) if suite.endswith("SHA384") else (hashlib.sha256, 16)
def opener(secret):
    k, iv = expand_label(secret, "key", b"", klen, h), expand_label(secret, "iv", b"", 12, h
    seq = 0
    def open_(rec):
        nonlocal seq
        nonce = bytes(a ^ b for a, b in zip(iv, seq.to_bytes(12, "big"))); seq += 1
        inner = AESGCM(k).decrypt(nonce, rec[5:], rec[:5]).rstrip(b"\x00")
        return inner[-1], inner[:-1]
    return open_
```

```
NAMES = {8: "EncryptedExtensions", 11: "Certificate", 15: "CertificateVerify", 20: "Finished
         4: "NewSessionTicket"}
def messages(stream):
    while stream:
        n = int.from_bytes(stream[1:4], "big"); yield stream[:4 + n]; stream = stream[4 + n:

hs_open, ap_open = opener(log["SERVER_HANDSHAKE_TRAFFIC_SECRET"]), opener(log["SERVER_TRAFFI
flight, after = b"", []
for rec in s_rest:
    if rec[0] != 23: continue                                          # skip change_cipher_spe
    if not any(m[0] == 20 for m in messages(flight)):                   # until the server Fini
        ctype, body = hs_open(rec); flight += body
    else:
        after.append(ap_open(rec))
msgs = list(messages(flight))
print("server handshake flight:", ", ".join(f"{NAMES[m[0]]} ({len(m) - 4})" for m in msgs))

def finished(base, transcript):
    fk = expand_label(base, "finished", b"", h().digest_size, h)
    return hmac.new(fk, h(transcript).digest(), h).digest()
sf = msgs[-1]
mine = finished(log["SERVER_HANDSHAKE_TRAFFIC_SECRET"], CH + SH + b"".join(msgs[:-1]))
print("server Finished recomputed:", "ok" if mine == sf[4:] else "MISMATCH")

c_open = opener(log["CLIENT_HANDSHAKE_TRAFFIC_SECRET"])
c_recs = [r for r in c_rest if r[0] == 23]
_, cf = c_open(c_recs[0])
mine = finished(log["CLIENT_HANDSHAKE_TRAFFIC_SECRET"], CH + SH + flight)
print("client Finished recomputed:", "ok" if mine == cf[4:] else "MISMATCH")

for ctype, body in after:
    if ctype == 22: print("after the handshake, server sent:", NAMES[body[0]], f"({len(body)
    else: print("after the handshake, server sent application_data:", body)
_, req = opener(log["CLIENT_TRAFFIC_SECRET_0"])(c_recs[1])
print("client application_data:", req)
```

```
$ venv/bin/python live.py
TLSv1.3 TLS_AES_256_GCM_SHA384 | OpenSSL 3.5.7
ServerHello key_share: group 0x11ec, 1120 bytes | server record types: 20 23 23 23 23 23 23
key log labels: SERVER_HANDSHAKE_TRAFFIC_SECRET, EXPORTER_SECRET, SERVER_TRAFFIC_SECRET_0, C
server handshake flight: EncryptedExtensions (2), Certificate (311), CertificateVerify (76),
server Finished recomputed: ok
client Finished recomputed: ok
after the handshake, server sent: NewSessionTicket (229)
after the handshake, server sent: NewSessionTicket (229)
after the handshake, server sent application_data: b'hello from OpenSSL'
```

```
client application_data: b'GET / HTTP/1.1\r\n\r\n'
```

The certificate is generated per run, so the Certificate and CertificateVerify sizes change by a byte or two between runs (DER-encoded ECDSA signatures vary in length). Everything else is identical every time.

- **The hybrid key exchange is invisible to the schedule.** The server's key_share is 1120 bytes (a 1088-byte ML-KEM-768 ciphertext plus a 32-byte X25519 share). The shared secret that enters the second Extract is 64 bytes (ML-KEM's 32 ‖ X25519's 32, RFC 10024 §4.3) instead of 32. HKDF-Extract takes any length of IKM, so not one line of `hkdf.py` changed.
- **SHA-384 changes only `h` and the lengths.** Finished is 48 bytes; the AES-256 key is 32.
- **The key log holds only the leaves.** It logs the handshake and application traffic secrets and the exporter secret, never the Early, Handshake or Main secrets. That's enough to decrypt everything and to check both Finished MACs, as above, but not to derive the resumption secret. RFC 9850 §1.1 is blunt: the file "completely undermines the core guarantees that TLS provides", and the mechanism "MUST NOT be used in a production system".
- OpenSSL sends **two** NewSessionTickets by default, and each record (EncryptedExtensions, Certificate, …) separately. RFC 8448's server packed its whole flight into one.

Lab 5: one-line bugs and where they surface

The schedule is unforgiving and the failure is always the same: the handshake fails, often only as a `bad_record_mac` or `decrypt_error` alert. The useful skill is knowing which value to check first.

```
# bugs.py - five one-line mistakes and the first RFC 8448 value each one breaks
import contextlib, io, hmac
with contextlib.redirect_stdout(io.StringIO()):
    import schedule as S
from hkdf import hkdf_extract, hkdf_expand, expand_label, derive_secret, H, HASH_LEN

def run(bug=None):
    zeros = bytes(HASH_LEN)
    early = hkdf_extract(b"", zeros)
    if bug == "derived = Expand-Label(…, context \"\")":
        derived = expand_label(early, "derived", b"", HASH_LEN)      # forgot Hash("")
    else:
        derived = derive_secret(early, "derived", b"")
    salt = early if bug == "skip the \"derived\" step" else derived
    hs = hkdf_extract(salt, S.shared_c)
    if bug == "label prefix \"TLS 1.3, \" (draft-19)":
        full = b"TLS 1.3, c hs traffic"
        info = (32).to_bytes(2, "big") + bytes([len(full)]) + full + bytes([32]) + H(S.CH +
        c_hs = hkdf_expand(hs, info, 32)
    else:
        c_hs = derive_secret(hs, "c hs traffic", S.CH + S.SH)
    s_hs = derive_secret(hs, "s hs traffic", S.CH + S.SH)
```

```
    tr = S.CH + S.SH + S.EE + S.CRT + S.CV
    fk = expand_label(s_hs, "finished", b"", HASH_LEN)
    if bug == "Finished over the raw transcript":
        sf = hmac.new(fk, tr, H).digest()
    else:
        sf = hmac.new(fk, H(tr).digest(), H).digest()
    master = hkdf_extract(derive_secret(hs, "derived", b""), zeros)
    upto = tr + S.SF + (S.CF if bug == "app secrets include the client Finished" else b"")
    c_ap = derive_secret(master, "c ap traffic", upto)
    return [("handshake secret", hs, S.hs), ("client hs traffic", c_hs, S.c_hs),
            ("server Finished", sf, S.SF[4:]), ("client ap traffic", c_ap, S.c_ap)]

for bug in [None, "skip the \"derived\" step", "derived = Expand-Label(…, context \"\")",
            "label prefix \"TLS 1.3, \" (draft-19)", "Finished over the raw transcript",
            "app secrets include the client Finished"]:
    first = next((name for name, mine, want in run(bug) if mine != want), None)
    print(f"{bug or 'no bug':<42} first wrong value: {first or '- (all four match)'}")
```

```
$ python3 bugs.py
no bug                                      first wrong value: - (all four match)
skip the "derived" step                     first wrong value: handshake secret
derived = Expand-Label(…, context "")       first wrong value: handshake secret
label prefix "TLS 1.3, " (draft-19)         first wrong value: client hs traffic
Finished over the raw transcript            first wrong value: server Finished
app secrets include the client Finished     first wrong value: client ap traffic
```

Read this as a debugging table:

- **Handshake secret wrong:** your Extract ladder or "derived" step is off. Typically the "derived" context is zero bytes instead of `Hash("")`, or the step is skipped.
- **Handshake secret right, traffic secret wrong:** the label encoding (prefix, length bytes) or the transcript is off.
- **Traffic secrets right, Finished wrong:** the MAC input. It must be the transcript *hash*, keyed with `finished_key`, and the transcript must exclude the Finished being computed.
- **Handshake right, application wrong:** the transcript boundary. It ends at the server Finished, not the client's.

Case: what the proposed XOF key schedule would change

`draft-sullivan-tls-xof-schedule-00` (Nick Sullivan, July 2026) was presented to the TLS working group the same month. It is an individual draft, not adopted, and not part of TLS. It defines an extension that negotiates a different KDF *beside* the cipher suite. With it, the transcript hash, every derivation, and the Finished and binder MACs run on one Keccak-based extendable-output function: TurboSHAKE256 (12 rounds, RFC 9861) or SHAKE256 (FIPS 202). No SHA-2, HMAC or HKDF is left in the schedule.

The motivation is not that SHA-2 is weak. ML-KEM already ships the Keccak permutation, so a pure post-quantum stack could drop SHA-2 entirely. And a keyed sponge needs one permutation call per output, where HMAC runs two nested hashes. Without the extension the connection runs exactly the schedule in this chapter, and the record layer, cipher suites and state machine are untouched. Even if it's adopted, the ladder-and-leaves shape above is what such a schedule would mirror. The draft's own §7.2 maps its derivations to TLS 1.3's "shape by shape".

The trap

- **Treating the label as a free-form string.** `"key"` is really `00 10 09 "tls13 key" 00`: the length, a length-prefixed label with its prefix, and an empty context. A missing prefix or a two-byte label length produces valid-looking, wrong keys.
- **Zero bytes where the spec says `Hash("")`.** `Derive-Secret(., "derived", "")` hashes the empty transcript. Zeros appear only as Extract inputs ("0" in the diagram).
- **Transcripts with record headers, or with the wrong end.** Hash the Handshake messages (4-byte header + body) only. Stop before the message you are computing.
- **Reusing a nonce.** The sequence number is per key and per direction. If you forget to reset it when the keys change, or share one counter across directions, AES-GCM fails on the first record, if you are lucky.
- **Leaving SSLKEYLOGFILE on.** Any process that inherits the environment variable and uses a library that honours it writes every connection's traffic secrets to disk.

The boring rule

Don't implement the key schedule in production code; use a maintained TLS library. When you have to (a protocol on top of TLS, an exporter, QUIC, a test harness, a decoder), build it from the two wrappers, test it against RFC 8448 value by value before anything touches the network, and debug it by finding the first value that differs: secret, then traffic secret, then key/IV, then Finished. Use key logs on test systems only.

Try this

1. Change `schedule.py` to `TLS_AES_256_GCM_SHA384` thinking. Which values in RFC 8448 §3 would you no longer be able to check, and why can't you just "pass `h=sha384`"?
2. QUIC uses the same `HKDF-Expand-Label` with its own labels (RFC 9001 §5.2). With `initial_secret = HKDF-Extract(0x38762cf7f55934b34d179ae6a4c80cadccbb7f0a, 0x8394c8f03e515708)`, check that `Expand-Label(initial_secret, "client in", "", 32)` starts with `c00cf151` and that its `"quic key"` is `1f369613dd76d5467730efcbe3b1a22d` (RFC 9001 Appendix A.1).
3. Implement a key update: derive `application_traffic_secret_1` with the label `"traffic upd"` (§7.2) and say which counter resets.
4. RFC 8448 §4 is a resumed 0-RTT handshake. Using the resumption PSK from Lab 2, derive the binder key (`"res binder"`) and the client early traffic secret, and check them against §4.

5. In `live.py`, add `cctx.set_ecdh_curve("X25519")` after the contexts are made. Confirm that the key_share becomes group `0x001d` with 32 bytes, and that nothing else in the output changes.

Next

- QUIC v2 and version negotiation: QUIC's own HKDF labels and salts, built on the same Expand-Label.
- HTTP/3 stream cancellation: what runs over those application keys.

Verification

- **Run here:**
  - `hkdf.py` with Python 3.13.5's standard library: RFC 5869 A.1 matched.
  - `schedule.py`: all 18 compared values matched RFC 8448 §3, including an X25519 shared secret computed in pure Python.
  - `decrypt.py` and `bugs.py` (cryptography 50.0.2 in a venv for AES-GCM): every RFC 8448 §3 record decrypts to the RFC's payload.
  - `live.py`: OpenSSL 3.5.7 negotiated TLS 1.3 with `TLS_AES_256_GCM_SHA384` and group `0x11ec`; both Finished MACs were recomputed from the logged secrets, and all records decrypted. It was run six times; only the certificate-dependent sizes changed.
  - The QUIC values in Try this 2 were checked with `hkdf.py` against RFC 9001 Appendix A.1.
- **Checked against primary texts:**
  - RFC 9846, which obsoleted RFC 8446 in July 2026: section numbers §4.1, §4.5.3, §4.7.1, §5.2, §5.3, §7.1–§7.3, and the "main"/"master" note.
  - RFC 10024: group value 4588 (0x11EC), share sizes, secret order.
  - RFC 9850: labels and the applicability statement.
  - draft-ietf-tls-tls13-19 and -20: the `"TLS 1.3, "` → `"tls13 "` label change.
  - draft-sullivan-tls-xof-schedule-00: abstract, §1, §1.1.
- No root, no packages outside a user venv, no network beyond downloading the RFC text.

Sources

- RFC 9846 — The Transport Layer Security (TLS) Protocol Version 1.3 (obsoletes RFC 8446): §4.1 transcript hash, §4.5.3 Finished, §4.7.1 NewSessionTicket and the resumption PSK, §5.2–§5.3 record protection and nonce, §7.1 key schedule, §7.2 key update, §7.3 traffic keys
- RFC 8448 — Example Handshake Traces for TLS 1.3: §3 simple 1-RTT handshake, §4 resumed 0-RTT handshake
- RFC 5869 — HMAC-based Extract-and-Expand Key Derivation Function (HKDF): §2, Appendix A.1
- RFC 7748 — Elliptic Curves for Security: §5 the X25519 function

- RFC 10024 — Post-Quantum Traditional (PQ/T) Hybrid Key Agreement Mechanisms for TLS 1.3: §4.1–§4.3, §7.1
- RFC 9850 — The SSLKEYLOGFILE Format for TLS: §1.1, §2.1
- RFC 9001 — Using TLS to Secure QUIC: §5.2, Appendix A.1
- draft-ietf-tls-tls13-20 — change log, "Shorten labels for HKDF-Expand-Label"
- draft-sullivan-tls-xof-schedule-00 — TLS 1.3 key schedule on an extendable-output function (individual draft, work in progress)
- Python — `ssl.SSLContext.keylog_filename`, `ssl.MemoryBIO`
- pyca/cryptography — AEAD: `AESGCM`

The 2026 Root KSK Rollover: Trust Anchors, RFC 5011, and What Breaks When Keys Are Hard-Coded

The 2026 Root KSK Rollover: Trust Anchors, RFC 5011, and What Breaks When Keys Are Hard-Coded

Every DNSSEC validator on the Internet starts from one fact it cannot look up: the public key that signs the root zone's DNSKEY set, the root **Key Signing Key** (KSK). On **11 October 2026 at 14:00 UTC** that key changed for the second time ever. **KSK-2017** (key tag 20326) stopped signing and **KSK-2024** (key tag 38696) took over. A validator that has learned the new key keeps working, unnoticed. One that still trusts only KSK-2017 now marks *every signed answer* as bogus, and depending on its configuration fails every lookup. This chapter shows the chain from trust anchor to root DNSKEY, checks today's root keys live against IANA's published anchors, reproduces the "old anchor only" failure, and explains how RFC 5011 is meant to make this automatic and when it cannot.

Learning goals

- Explain what a trust anchor is, and why the KSK, not the ZSK, is the one that matters
- Read the root DNSKEY RRset: flags 257 vs 256, key tags, which key signed it
- Check a resolver's anchors against `root-anchors.xml` by computing the DS digest yourself
- Describe RFC 5011's add hold-down and revoke bit, and the dates IANA used for this rollover
- Find hard-coded anchors (old images, appliances, smart contracts, embedded resolvers) before they fail

Mental model

```
trust anchor (20326, 38696)            root zone
     matches DS digest / key
                          DNSKEY 257 20326 (KSK-2017, not signing)
                                        DNSKEY 257 38696 (KSK-2024)    RRSIG
                                        DNSKEY 256  8763 (ZSK)
                                             signs SOA, NS, DS com., DS org. ...

2025-01-11 publish 38696   2025-02-10 trusted (5011)   2026-10-11 ROLL   2027-01-11 revoke 203
```

Three rules:

1. **Only the KSK is anchored.** The ZSK changes every quarter without anyone noticing, because the KSK signs the DNSKEY set that contains it.

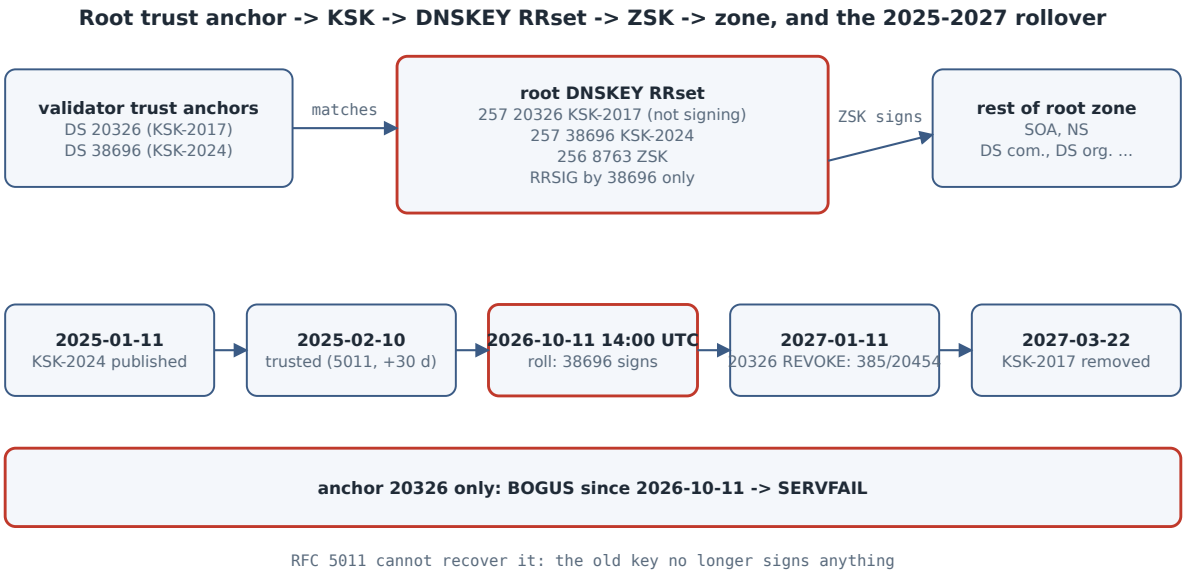


Figure 1: Trust anchor in the validator verifies the KSK, which signs the root DNSKEY RRset containing the ZSK, which signs the rest of the root zone including the DS records for TLDs. Timeline of the 2024 to 2027 rollover.

2. **An anchor is just a key, or the hash of one.** `root-anchors.xml` publishes the SHA-256 DS digest of each KSK. Any validator can recompute it from the DNSKEY it receives and compare.
3. **RFC 5011 only works while an old trusted key still signs.** A new KSK is accepted automatically only if a key you already trust signs the DNSKEY set that introduces it. Since 11 October 2026 KSK-2017 signs nothing, so a validator that missed the 2025–2026 window can no longer catch up on its own.

Concepts

Key tags and flags

A DNSKEY's **flags** field is 256 for a zone-signing key and 257 when the Secure Entry Point (SEP) bit is set, which by convention marks a KSK. The **key tag** is a 16-bit checksum over the whole DNSKEY RDATA (RFC 4034 Appendix B), so it is a quick label, not an identity: two keys can collide, and flipping a flag changes the tag. RRSIG records name the signing key by tag.

Trust anchors in the wild

Validator	Where the root anchor lives	RFC 5011
Unbound	`auto-trust-anchor-file` (for example `/var/lib/unbound/root.key`), bootstrapped by `unbound-anchor` from `root-anchors.xml`	yes, writes state back to that file
BIND 9	built-in `trust-anchors { . initial-key ...; }` / `initial-ds`, state in `managed-keys.bind`	yes for `initial-*`; `static-key` / `static-ds` are never updated
Knot Resolver	`trust_anchors` module, keyfile with RFC 5011 state	yes
systemd-resolved	anchor compiled into the binary; overrides in `dnssec-trust-anchors.d` `.positive` files	updated by upgrading systemd
Public resolvers (1.1.1.1, 8.8.8.8, 9.9.9.9)	operator-managed	operator's job
Hand-written validators, appliances, smart contracts	wherever the developer put a constant	no

The last row is where the 2026 rollover hurt: the inspiration for this chapter is a report that an on-chain DNSSEC oracle anchored only KSK-2017, so DNS-based claims verified against it stopped working after 11 October.

RFC 5011 in one paragraph

A validator doing RFC 5011 re-fetches the root DNSKEY set periodically. When a new SEP key appears in a set validly signed by a key it already trusts, it starts an **add hold-down** timer (30 days). If the key is still there when the timer expires, it becomes a trust anchor. To remove a key, the zone publishes it with the **REVOKE** bit (0x0080) set, signed by the revoked key itself; the validator then drops it immediately. IANA's dates follow this: KSK-2024 was published on 11 January 2025 and "should begin to be trusted by resolvers that follow RFC 5011" on 10 February 2025, 30 days later.

Lab 1: the root keys right now, against IANA's anchors

IANA's trust anchor file (abridged to the fields used here; fetched with `curl -sS https://data.iana.org/root-a`

```
<KeyDigest id="Kjqmt7v" validFrom="2010-07-15T00:00:00+00:00" validUntil="2019-01-11T00:00:0
    <KeyTag>19036</KeyTag>
    <Digest>49AAC11D7B6F6446702E54A1607371607A1A41855200FD2CE1CDDE32F24E8FB5</Digest>
<KeyDigest id="Klajeyz" validFrom="2017-02-02T00:00:00+00:00">
```

```
    <KeyTag>20326</KeyTag>
    <Digest>E06D44B80B8F1D39A95C0B0D7C65D08458E880409BBC683457104237C7F8EC8D</Digest>
<KeyDigest id="Kmyv6jo" validFrom="2024-07-18T00:00:00+00:00">
    <KeyTag>38696</KeyTag>
    <Digest>683D2D0ACB8C9B712A1948B27F741219298D0A450D612C483AF444A4C0FB2B16</Digest>
```

KSK-2010 (19036) has a `validUntil`; the other two do not. The box this was written on cannot send DNS queries straight to the root servers (port 53 to the Internet is filtered), so `rootkeys.py` asks for the root DNSKEY set over DNS-over-HTTPS (RFC 8484) with dnspython 2.9.1, then recomputes each KSK's DS digest:

```
import urllib.request
import dns.message, dns.query, dns.rdatatype, dns.dnssec, dns.name, datetime
q = dns.message.make_query('.', 'DNSKEY', want_dnssec=True, payload=4096)
req = urllib.request.Request('https://cloudflare-dns.com/dns-query', data=q.to_wire(),
                             headers={'content-type': 'application/dns-message', 'accept': '
r = dns.message.from_wire(urllib.request.urlopen(req, timeout=10).read())   # RFC 8484 DoH P
keys = [rr for rr in r.answer if rr.rdtype == dns.rdatatype.DNSKEY][0]
sigs = [rr for rr in r.answer if rr.rdtype == dns.rdatatype.RRSIG][0]
print(f"DNSKEY RRset for '.', TTL {keys.ttl}, via DNS-over-HTTPS (cloudflare-dns.com)")
for k in sorted(keys, key=lambda k: -k.flags):
    role = {257: 'KSK (SEP)', 256: 'ZSK'}.get(k.flags & 0xFFFF & ~0x80, str(k.flags))
    rev = ' REVOKED' if k.flags & 0x80 else ''
    print(f"  flags={k.flags:<3} alg={k.algorithm.value} tag={dns.dnssec.key_id(k):<5} {role
for s in sigs:
    exp = datetime.datetime.fromtimestamp(s.expiration, datetime.UTC)
    print(f"RRSIG over DNSKEY: signed by key tag {s.key_tag}, expires {exp:%Y-%m-%d %H:%M} U
for tag, digest in ((20326, 'E06D44B80B8F1D39A95C0B0D7C65D08458E880409BBC683457104237C7F8EC8
                    (38696, '683D2D0ACB8C9B712A1948B27F741219298D0A450D612C483AF444A4C0FB2B1
    for k in keys:
        if dns.dnssec.key_id(k) == tag:
            ds = dns.dnssec.make_ds(dns.name.root, k, 'SHA256')
            print(f"tag {tag}: DS digest matches IANA root-anchors.xml: {ds.digest.hex().upp
```

```
$ python rootkeys.py
DNSKEY RRset for '.', TTL 2562, via DNS-over-HTTPS (cloudflare-dns.com)
  flags=257 alg=8 tag=20326 KSK (SEP)
  flags=257 alg=8 tag=38696 KSK (SEP)
  flags=256 alg=8 tag=8763  ZSK
RRSIG over DNSKEY: signed by key tag 38696, expires 2026-11-11 00:00 UTC
tag 20326: DS digest matches IANA root-anchors.xml: True
tag 38696: DS digest matches IANA root-anchors.xml: True
```

Both KSKs are still *published*, but the only RRSIG over the DNSKEY set comes from 38696. The TTL is what was left in Cloudflare's cache; the root zone itself publishes DNSKEY with a 172800-second (two-day) TTL. Both digests match IANA's file byte for byte, which is exactly the check `unbound-anchor` does (plus verifying the S/MIME signature in `root-anchors.p7s`).

Lab 2: what each trust-anchor configuration sees today

`anchors.py` validates the live DNSKEY RRset three times with different anchor stores, then computes what KSK-2017's tag will be once it is revoked:

```
import urllib.request, datetime
import dns.message, dns.dnssec, dns.name, dns.rdatatype, dns.rrset, dns.rdtypes.ANY.DNSKEY

def doh(qname, qtype):
    q = dns.message.make_query(qname, qtype, want_dnssec=True, payload=4096)
    req = urllib.request.Request('https://cloudflare-dns.com/dns-query', data=q.to_wire(),
                                 headers={'content-type': 'application/dns-message',
                                          'accept': 'application/dns-message'})
    return dns.message.from_wire(urllib.request.urlopen(req, timeout=10).read())

r = doh('.', 'DNSKEY')
keys = next(rr for rr in r.answer if rr.rdtype == dns.rdatatype.DNSKEY)
sig = next(rr for rr in r.answer if rr.rdtype == dns.rdatatype.RRSIG)

# A validator's trust anchor store: the DS digests from IANA's root-anchors.xml
ANCHORS = {20326: 'E06D44B80B8F1D39A95C0B0D7C65D08458E880409BBC683457104237C7F8EC8D',  # KSK
           38696: '683D2D0ACB8C9B712A1948B27F741219298D0A450D612C483AF444A4C0FB2B16'}  # KSK

def validate(anchor_tags):
    trusted = [k for k in keys if dns.dnssec.key_id(k) in anchor_tags and
               dns.dnssec.make_ds(dns.name.root, k, 'SHA256').digest.hex().upper()
               == ANCHORS[dns.dnssec.key_id(k)]]
    anchor = dns.rrset.from_rdata_list(dns.name.root, keys.ttl, trusted)
    try:
        dns.dnssec.validate(keys, sig, {dns.name.root: anchor})
        return 'SECURE (DNSKEY RRset verifies)'
    except dns.dnssec.ValidationFailure as e:
        return f'BOGUS ({e})'

now = datetime.datetime.now(datetime.UTC)
print(f"checked {now:%Y-%m-%d %H:%M} UTC; RRSIG(DNSKEY) key tags: {[s.key_tag for s in sig]}
print("anchor = KSK-2017 only (hard-coded before 2025):", validate({20326}))
print("anchor = KSK-2024 only:                         ", validate({38696}))
print("anchor = both (RFC 5011 resolver today):        ", validate({20326, 38696}))

# What KSK-2017 will look like once revoked (RFC 5011: REVOKE bit = 0x0080)
old = next(k for k in keys if dns.dnssec.key_id(k) == 20326)
revoked = dns.rdtypes.ANY.DNSKEY.DNSKEY(old.rdclass, old.rdtype, old.flags | 0x0080,
                                        old.protocol, old.algorithm, old.key)
print(f"KSK-2017 flags {old.flags} tag {dns.dnssec.key_id(old)} -> with REVOKE bit: flags {r
```

```
$ python anchors.py
checked 2026-10-31 03:33 UTC; RRSIG(DNSKEY) key tags: [38696]
anchor = KSK-2017 only (hard-coded before 2025): BOGUS (no RRSIGs validated)
anchor = KSK-2024 only:                          SECURE (DNSKEY RRset verifies)
anchor = both (RFC 5011 resolver today):         SECURE (DNSKEY RRset verifies)
KSK-2017 flags 257 tag 20326 -> with REVOKE bit: flags 385 tag 20454
```

- **KSK-2017 only** is the configuration of any validator that stopped updating anchors before 2025, or that copied a `root.key` into an image years ago. The DNSKEY set has no signature by 20326, so nothing validates: **BOGUS**. Everything below the root inherits that. Even an unsigned zone cannot be proven insecure without a validated root, so a validating resolver in this state answers `SERVFAIL` for essentially every name.
- **Both keys** is what a healthy RFC 5011 resolver holds today, and it validates.
- In January 2027 KSK-2017 will appear with flags 385 and therefore a **different key tag, 20454**, signed by itself. Alerting that greps for "20326" will not see it.

Lab 3 (operator checklist, from vendor docs; not run here)

No resolver daemons are installed on the box, so these commands come from the vendors' documentation and were not run:

```
# Unbound: the anchor file must list 38696 as VALID
grep -E '20326|38696' /var/lib/unbound/root.key
unbound-anchor -v -a /var/lib/unbound/root.key      # re-bootstrap from root-anchors.xml if

# BIND 9: managed (RFC 5011) keys and their state
rndc managed-keys status

# Any validator: a signed name should come back with the AD flag, a deliberately broken one
dig +dnssec www.isc.org @127.0.0.1 | grep -E 'flags:|status:'
dig dnssec-failed.org @127.0.0.1 | grep status:
```

The trap

- **"It worked on 10 October" does not mean the anchor was updated.** Until the roll, a resolver holding only KSK-2017 validated perfectly. The failure arrived all at once on the roll date.
- **Read-only anchor files.** RFC 5011 needs to write state. Containers that mount `root.key` read-only, or rebuild it from the image on every start, never keep what they learn. They are effectively hard-coded.
- **Images built from old packages.** A golden image or a container base layer from 2023 ships an anchor file without 38696. If the resolver starts after 11 October 2026 it cannot learn the new key (rule 3), so it must be re-bootstrapped from `root-anchors.xml` or upgraded.

- **Clocks.** Validation checks RRSIG inception and expiration. A device that boots with a 1970 clock fails DNSSEC whatever its anchors, which is the classic "NTP needs DNS, DNS needs time" loop.
- **Monitoring by key tag.** Tags change on revocation (20326 → 20454). Compare digests or full keys.

The boring rule

Let the resolver manage the root anchor with RFC 5011 or its package manager, and make sure its anchor file is writable and persistent. Never paste a root key into code or config. If you must pin one (an appliance, a contract, an offline validator), pin every key in `root-anchors.xml` that has no `validUntil`, ship a way to update it, and put the IANA rollover dates in your calendar.

Try this

1. Run `anchors.py` again after 11 January 2027. How many RRSIGs over the DNSKEY set do you expect, and with which tags?
2. Change `validate()` to also accept the KSK-2010 digest from `root-anchors.xml`. Does anything change, and why not?
3. Modify `rootkeys.py` to fetch `com. DNSKEY` and `com. DS` (the DS from the root zone), and verify that one of com's KSKs hashes to the DS digest. That is the next link of the chain.
4. Look for hard-coded anchors in a codebase you know: search for `20326`, `E06D44B8`, or `AwEAAaz/tAm8`.

Next

- DNS authoritative and recursive: stub, recursive and authoritative roles, and validating with Unbound in the lab.

Verification

- **Run here** on 31 October 2026 (03:33 UTC), from a box without direct DNS access, using DNS-over-HTTPS to cloudflare-dns.com:
  - `rootkeys.py` and `anchors.py` were run with dnspython 2.9.1 and Python 3.13.5 in a user venv. Output is genuine and unedited.
  - Both DS digests matched IANA's `root-anchors.xml`, fetched the same morning. Validation succeeded with KSK-2024 and failed with KSK-2017 alone. The revoked-tag value is computed, not observed.
- **Not run:** the Unbound, BIND and `dig` commands in Lab 3 (no resolver or `dig` on the box). They come from the vendor docs listed below.

- Dates come from IANA's *DNSSEC Trust Anchors and Rollovers* page, ICANN's rollover status page ("On 11 October 2026 at 14:00 UTC, KSK-2024 began signing the DNS root zone as scheduled") and IANA's news item of 15 October 2026.

Sources

- IANA — DNSSEC Trust Anchors and Rollovers, and the trust anchor file root-anchors.xml
- IANA — Root Zone KSK Rollover Completed as Planned (2026-10-15)
- ICANN — Root Zone KSK Rollover
- RFC 5011 — Automated Updates of DNS Security (DNSSEC) Trust Anchors: §2.1 revocation, §2.2 and §2.4.1 add hold-down, §3 the REVOKE bit
- RFC 4034 — Resource Records for the DNS Security Extensions: §2.1.1 flags, §5 DS, Appendix B key tag
- RFC 9718 — DNSSEC Trust Anchor Publication for the Root Zone
- RFC 8484 — DNS Queries over HTTPS (DoH)
- Unbound — unbound-anchor(8); BIND 9 — trust-anchors; systemd — dnssec-trust-anchors.d(5)
- dnspython — DNSSEC functions
- Inspiration: @taylor_ on the ENS DNSSEC oracle and KSK-2017

TLS 1.3 0-RTT: Replaying Early Data, and What Actually Stops It

TLS 1.3 0-RTT: Replaying Early Data, and What Actually Stops It

Learning goals

- Explain why 0-RTT early data can be replayed, even though it is encrypted and authenticated.
- Record a real 0-RTT request and replay it byte for byte, and watch a server without anti-replay deliver it to the application **twice**.
- Watch single-use tickets (OpenSSL's default) stop that replay, and say why that protection is per server process, not per fleet.
- Decide which requests may ride in early data, and use HTTP's `Early-Data: 1` header and the `425 Too Early` status for the rest.

Version note: all of the labs ran on one Linux box with **OpenSSL 3.5.7** (`openssl s_server` / `s_client`) and Python 3.13 (stdlib only), over 127.0.0.1. No packages were installed, and every output block is pasted from those runs. The HTTP-level advice (RFC 8470) is not exercised by a real HTTP server here, and it is marked as such.

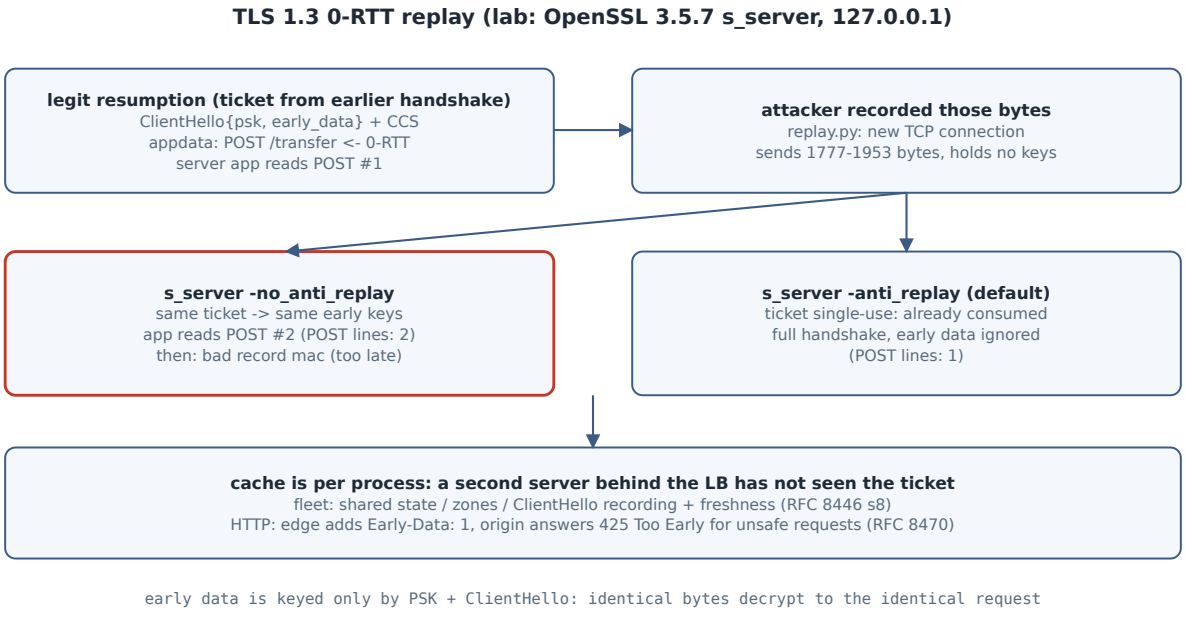


Figure 1: A recorded 0-RTT request replayed against OpenSSL s_server: delivered twice without anti-replay, once with single-use tickets.

Mental model

A resumed TLS 1.3 handshake can carry application data in the client's **first flight**. That data is encrypted with keys derived from the resumption PSK, the session ticket, and the ClientHello. It covers nothing the server contributed: no server random, no fresh key share.

```
client's first flight                                        keys depend on

ClientHello {pre_shared_key: ticket, early_data}
[ChangeCipherSpec]                                             PSK + ClientHello
application_data: "POST /transfer?…"   ← 0-RTT                (client_early_traffic_secret)
                        ··· server flight ···
EndOfEarlyData, Finished                ← needs server's ServerHello, so a recording can't fa
```

So the same bytes, sent again, decrypt to the same request. RFC 8446 §8 and its successor RFC 9846 §8 say it plainly: TLS gives no protection against replay of 0-RTT data. Defences have to come from the server: single-use tickets (§8.1), ClientHello recording (§8.2), freshness checks (§8.3), or the application refusing unsafe early requests.

Lab: record one 0-RTT request, replay it

Three small pieces:

- `proxy.py` forwards one TCP connection and records every byte the client sends.
- `replay.py` opens a **new** connection and sends those bytes again, without any keys.
- `lab.sh` runs the scenario against `openssl s_server -early_data`.

```
"""proxy.py LISTEN UPSTREAM OUT - forward one TCP connection, record client->server bytes to
import socket, sys, threading
lport, uport, out = int(sys.argv[1]), int(sys.argv[2]), sys.argv[3]
ls = socket.socket(); ls.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1); ls.bind(("12
c, _ = ls.accept(); u = socket.create_connection(("127.0.0.1", uport)); rec = bytearray()
def pump(a, b, keep):
    while (d := a.recv(65536)):
        if keep: rec.extend(d)
        b.sendall(d)
    b.shutdown(socket.SHUT_WR)
t = threading.Thread(target=pump, args=(u, c, False)); t.start(); pump(c, u, True); t.join()
open(out, "wb").write(rec)
```

```
"""replay.py PORT FILE - resend every byte the client sent (recorded by proxy.py) as a brand
import socket, sys
data = open(sys.argv[2], "rb").read()
recs, i = [], 0
while i < len(data):
```

```
    ln = int.from_bytes(data[i+3:i+5]); recs.append({20: "CCS", 21: "alert", 22: "handshake"
s = socket.create_connection(("127.0.0.1", int(sys.argv[1]))); s.sendall(data)
s.settimeout(2); got = b""
try:
    while (d := s.recv(65536)): got += d
except (socket.timeout, ConnectionResetError): pass
print(f"replayed {len(data)} bytes = records {' '.join(recs)}; server answered {len(got)} by
```

```
#!/bin/sh
# lab.sh MODE - MODE is -anti_replay (OpenSSL default) or -no_anti_replay
MODE=$1; LOG=server$MODE.log; rm -f sess.pem cap.bin
openssl s_server -accept 4433 -cert cert.pem -key key.pem -tls1_3 -early_data -max_early_dat
SRV=$!; sleep 0.5
echo "1. full handshake, keep the session ticket"
sleep 1 | openssl s_client -connect 127.0.0.1:4433 -tls1_3 -sess_out sess.pem -quiet >/dev/n
echo "2. resume through the recording proxy, sending req.txt as 0-RTT early data"
python3 proxy.py 4434 4433 cap.bin & PX=$!; sleep 0.3
sleep 1 | openssl s_client -connect 127.0.0.1:4434 -tls1_3 -sess_in sess.pem -early_data req
wait $PX
echo "3. an attacker replays every byte the client sent, on a new connection"
python3 replay.py 4433 cap.bin | sed 's/^/   /'
sleep 0.5; kill $SRV
echo "4. what the server application received:"
grep -c 'POST /transfer' $LOG | sed 's/^/   POST lines in server output: /'
```

`req.txt` holds the "dangerous" request, and `cert.pem` is a throwaway P-256 self-signed certificate:

```
openssl req -x509 -newkey ec -pkeyopt ec_paramgen_curve:P-256 -nodes \
  -keyout key.pem -out cert.pem -days 1 -subj /CN=bank.test
printf 'POST /transfer?to=mallory&amount=100 HTTP/1.1\r\nHost: bank.test\r\n\r\n' > req.txt
for m in -anti_replay -no_anti_replay; do echo "== s_server $m"; ./lab.sh $m; done
```

```
== s_server -anti_replay
1. full handshake, keep the session ticket
2. resume through the recording proxy, sending req.txt as 0-RTT early data
   client: Early data was accepted
   client: Reused, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384
3. an attacker replays every byte the client sent, on a new connection
   replayed 1777 bytes = records handshake CCS appdata appdata appdata appdata; server answe
4. what the server application received:
   POST lines in server output: 1
== s_server -no_anti_replay
1. full handshake, keep the session ticket
2. resume through the recording proxy, sending req.txt as 0-RTT early data
   client: Early data was accepted
   client: Reused, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384
```

```
3. an attacker replays every byte the client sent, on a new connection
   replayed 1953 bytes = records handshake CCS appdata appdata appdata appdata; server answe
4. what the server application received:
   POST lines in server output: 2
```

What the server application printed in the `-no_anti_replay` run (`server-no_anti_replay.log`):

```
POST /transfer?to=mallory&amount=100 HTTP/1.1
Host: bank.test

POST /transfer?to=mallory&amount=100 HTTP/1.1
Host: bank.test

40970F6DE07F0000:error:0A000119:SSL routines:tls_get_more_records:decryption failed or bad r
40970F6DE07F0000:error:0A000139:SSL routines::record layer failure:../ssl/record/rec_layer_s
```

Reading it:

- **Step 2** is a normal 0-RTT resumption: `Reused, TLSv1.3` and `Early data was accepted`.
- **Step 3 without anti-replay:** `replay.py` holds no keys and no ticket secret. It resends opaque records: one handshake record, the compatibility CCS, and four application_data records. The server derived the same early keys and handed the POST to the application **a second time**. Only after that did the handshake fail (`decryption failed or bad record mac`). The recorded client Finished is encrypted under handshake keys that depend on the *new* ServerHello, so it is garbage on this connection. The failure comes too late: the early data had already been read.
- **Step 3 with anti-replay:** the POST appears **once**. The OpenSSL documentation says that when a server allows early data, it "automatically enables replay protection". Tickets become single use, and "the second and subsequent attempts will fall back to a full handshake (and any early data that was submitted will be ignored)". That is RFC 8446 §8.1, single-use tickets.

Where the protection ends

OpenSSL's single-use check lives in **that process's internal session cache**. The same documentation warns that applications using an external session cache must keep it in sync. Run four TLS terminators behind one load balancer, or restart one, and a replayed ClientHello sent to a server that has not seen the ticket is accepted again. RFC 8446 §8 frames it this way: replay protection only holds across the set of servers that accept a ticket. That set can be one shared database, or "zones" whose tickets are not accepted by other zones. ClientHello recording works the same way: a window bounded by freshness checks (§8.2, §8.3). Many deployments therefore accept 0-RTT only for requests that are safe to replay.

Two consequences for applications:

- **Replay-safe only.** GET-style requests with no side effects are fine in early data. A transfer, a login or a "send email" is not.

- **No IP-based trust.** A replayed request comes from whoever replayed it, from any address, and the bytes are identical, so the source address of a 0-RTT request proves nothing. Rate limits or allow-lists keyed on client IP must not authorise early data.

HTTP's answer: `Early-Data: 1` and `425 Too Early` (not run here)

RFC 8470 defines the HTTP side:

- A server or proxy that is not sure a request is replay-safe can answer **`425 Too Early`**. The client then retries after the handshake (§5.2, §4).
- A TLS-terminating proxy that forwards a request received in early data adds **`Early-Data: 1`** (§5.1), so the origin can make the decision even though it never saw TLS.

```
client  0-RTT POST  edge proxy  "POST … Early-Data: 1"   origin
                                      425 Too Early          (not replay-safe)
client    425   edge; client retries the POST after the handshake completes (1-RTT)
```

The boring rule

- Leave OpenSSL's anti-replay on (the default). Treat it as per process, not as a fleet guarantee.
- Allow 0-RTT only for idempotent requests. Have the edge mark early requests with `Early-Data: 1`, and have origins answer `425` for anything with side effects.
- Never let the client address, or anything else replayable, authorise an early request.

Try this

1. Re-run the lab with `-num_tickets 2` on the server and use each ticket once. Then replay one capture against a **second** `s_server` started with a different port and the same certificate. Is the early data accepted? (A second process does not share the first one's ticket key, so check whether it even decrypts the ticket.)
2. Add `-max_early_data 0` on the server and watch `Early data was rejected` on the client.
3. Open `cap.bin` with a TLS-aware dissector and find the `early_data` and `pre_shared_key` extensions in the ClientHello.

Sources

- RFC 8446, TLS 1.3: §4.2.10 Early Data Indication, §8 0-RTT and Anti-Replay (§8.1–8.3), Appendix E.5 Replay Attacks on 0-RTT: https://www.rfc-editor.org/rfc/rfc8446
- RFC 9846, TLS 1.3 (obsoletes RFC 8446): §8 0-RTT and Anti-Replay, Appendix F.5 Replay Attacks on 0-RTT: https://www.rfc-editor.org/rfc/rfc9846
- RFC 8470, Using Early Data in HTTP (`Early-Data`, `425 Too Early`): https://www.rfc-editor.org/rfc/rfc8470

- OpenSSL 3.5, SSL_read_early_data(3), “Replay protection”: https://docs.openssl.org/3.5/man3/SSL_read_early_data/
- OpenSSL 3.5, openssl-s_server(1) (`-early_data`, `-anti_replay`, `-no_anti_replay`): https://docs.openssl.org/3.5/man1/openssl-s_server/

Encrypting IP Addresses in Logs with IPCrypt

Encrypting IP Addresses in Logs with IPCrypt

Learning goals

- Explain why hashing or truncating client IPs is a weak way to share network logs, and what keyed, invertible IP encryption gives you.
- Choose between IPCrypt's three families: **deterministic**, **prefix-preserving** (`ipcrypt-pfx`) and **non-deterministic** (`ipcrypt-nd` / `ipcrypt-ndx`). Each one keeps different analytics possible and leaks different things.
- Implement three of the modes in about 70 lines of Python and check them against the specification's test vectors.
- Spot the operational traps: equal pfx key halves, IPv4 turning into IPv6, and blobs that are no longer IP addresses.

Version note: this follows **draft-denis-ipcrypt-14** (Informational, IETF Internet-Draft, Barber & Denis, expires February 2027). The labs ran on 31 Oct 2026 with Python 3.13 and the `cryptography` 43 package already on the box. No installs were needed. All addresses come from the documentation ranges (RFC 5737, RFC 3849), and the keys are demo values only. The output blocks are pasted from the runs.

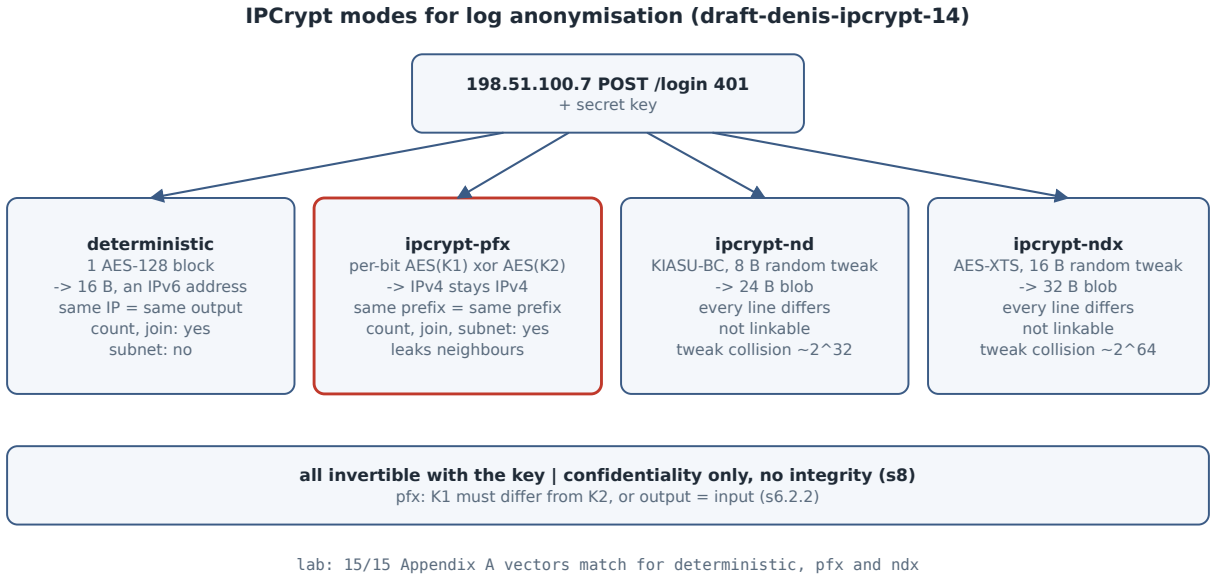


Figure 1: The four IPCrypt modes, what each output looks like and which log questions still work.

Why not just hash or truncate?

The common shortcuts each fail in a different way:

- **Unkeyed hashes** of IPv4 addresses are not private. There are only 2^32 inputs, so anyone can hash them all and reverse the mapping.
- **Truncation** (zeroing the last octet) destroys the data you need for abuse handling, and still identifies small networks.
- **Ad hoc encryption** with a library's default mode gives outputs that are not addresses, so they break every downstream tool.

The draft's motivation (§1.1) is to provide a common, specified set of constructions with test vectors, so that two implementations of the same mode, with the same key, produce the same output.

The modes in one table

Mode	Construction (draft §)	Output	Same IP, same output?	Keeps subnet structure?
`ipcrypt-deterministic`	AES-128 block, 16-byte key (§5.1)	16 bytes, an IPv6 address	yes	no
`ipcrypt-pfx`	per-bit PRF = AES(K1) AES(K2), 32-byte key (§6.2)	native size: IPv4 stays IPv4	yes	**yes**
`ipcrypt-nd`	KIASU-BC, 8-byte random tweak (§7.4.1)	24 bytes	no	no
`ipcrypt-ndx`	single-block AES-XTS, 16-byte random tweak (§7.4.2)	32 bytes	no	no

All four modes are invertible with the key. The draft states plainly that they provide **confidentiality, not integrity** (§8): someone who can edit the log can still swap or forge ciphertexts.

Every mode except `pfx` first maps an address to 16 bytes, turning IPv4 into `::ffff:a.b.c.d` (§3.1.2). That is why `192.0.2.1` and `::ffff:192.0.2.1` encrypt identically.

Lab 1: implement it and check the test vectors

`ipcrypt.py` covers deterministic, pfx and ndx. `ipcrypt-nd` is left out because KIASU-BC (AES with a tweak mixed into every round key) is not in standard crypto libraries. The pfx loop follows the draft's §9.6 pseudocode. The PRF input at each step is `zeros || 1 || prefix-so-far`, where the prefix is always the **plaintext** prefix. That is what makes equal prefixes encrypt to equal prefixes.

```
"""ipcrypt-deterministic, ipcrypt-pfx and ipcrypt-ndx, following
draft-denis-ipcrypt-14 sections 3, 5.1, 6.2, 7.4.2 and 9.  AES from `cryptography`."""
import ipaddress, os
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes

def aes(key, block, dec=False):
    c = Cipher(algorithms.AES(key), modes.ECB())
    op = c.decryptor() if dec else c.encryptor()
    return op.update(block) + op.finalize()

def to16(ip):                       # section 3.1: IPv4 -> ::ffff:a.b.c.d
    a = ipaddress.ip_address(ip)
    if a.version == 4:
        return bytes(10) + b"\xff\xff" + a.packed
    return a.packed

def from16(b):                      # section 3.2
    if b[:12] == bytes(10) + b"\xff\xff":
        return str(ipaddress.IPv4Address(b[12:]))
    return str(ipaddress.IPv6Address(b))

def xor(a, b): return bytes(x ^ y for x, y in zip(a, b))

# --- 5.1 ipcrypt-deterministic: one AES-128 block --------------------------
def det_encrypt(ip, key): return from16(aes(key, to16(ip)))
def det_decrypt(ip, key): return from16(aes(key, to16(ip), dec=True))

# --- 6.2 / 9.6 ipcrypt-pfx: one PRF bit per address bit --------------------
def _get(d, pos): return (d[15 - pos // 8] >> (pos % 8)) & 1
def _set(d, pos, v): d[15 - pos // 8] |= (v & 1) << (pos % 8)
def _shl(d):
    r, carry = bytearray(16), 0
    for i in range(15, -1, -1):
        r[i] = ((d[i] << 1) | carry) & 0xFF
        carry = (d[i] >> 7) & 1
    return r

def _pfx(ip, key, decrypt):
    assert len(key) == 32
    k1, k2 = key[:16], key[16:]
    src = to16(ip)
    out = bytearray(16)
    pad = bytearray(16)
    if src[:12] == bytes(10) + b"\xff\xff":
        start = 96; out[10] = out[11] = 0xFF
        pad[3] = 0x01; pad[14] = pad[15] = 0xFF
    else:
```

```
        start = 0; pad[15] = 0x01
    for n in range(start, 128):
        e = xor(aes(k1, bytes(pad)), aes(k2, bytes(pad)))
        pos = 127 - n
        bit = _get(src, pos) ^ (e[15] & 1)
        _set(out, pos, bit)
        plain_bit = bit if decrypt else _get(src, pos)
        pad = _shl(pad); _set(pad, 0, plain_bit)   # prefix is always the PLAINTEXT prefix
    return from16(bytes(out))

def pfx_encrypt(ip, key): return _pfx(ip, key, False)
def pfx_decrypt(ip, key): return _pfx(ip, key, True)

# --- 7.4.2 / 9.8 ipcrypt-ndx: single-block AES-XTS (XEX), random 16-byte tweak
def ndx_encrypt(ip, key, tweak=None):
    tweak = tweak or os.urandom(16)
    et = aes(key[16:], tweak)
    return tweak + xor(aes(key[:16], xor(to16(ip), et)), et)

def ndx_decrypt(blob, key):
    tweak, c = blob[:16], blob[16:]
    et = aes(key[16:], tweak)
    return from16(xor(aes(key[:16], xor(c, et), dec=True), et))
```

`vectors.py` checks every encryption against the draft's Appendix A and decrypts it back:

```
# Check ipcrypt.py against draft-denis-ipcrypt-14 Appendix A (A.1, A.2, A.4).
from ipcrypt import *
H = bytes.fromhex
det = [("0123456789abcdeffedcba9876543210", "0.0.0.0", "bde9:6789:d353:824c:d7c6:f58a:6bd2:2
       ("1032547698badcfeefcdab8967452301", "255.255.255.255", "aed2:92f6:ea23:58c3:48fd:8b8
       ("2b7e151628aed2a6abf7158809cf4f3c", "192.0.2.1", "1dbd:c1b9:fff1:7586:7d0b:67b4:e76e
K1 = "0123456789abcdeffedcba98765432101032547698badcfeefcdab8967452301"
K2 = "2b7e151628aed2a6abf7158809cf4f3ca9f5ba40db214c3798f2e1c23456789a"
pfx = [(K1, "0.0.0.0", "151.82.155.134"), (K1, "255.255.255.255", "94.185.169.89"),
       (K1, "192.0.2.1", "100.115.72.131"), (K1, "2001:db8::1", "c180:5dd4:2587:3524:30ab:fa
       (K2, "10.0.0.47", "19.214.210.244"), (K2, "10.0.0.129", "19.214.210.80"),
       (K2, "172.16.5.193", "210.78.229.136"), (K2, "172.16.97.42", "210.78.179.241"),
       (K2, "2001:db8::a5c9:4e2f:bb91:5a7d", "7cec:702c:1243:f70:1956:125:b9bd:1aba"),
       (K2, "2001:db8:3a5c:0:e7d1:4b9f:2c8a:f673", "7cec:702c:3503:bef:e616:96bd:be33:a9b9")
ndx = [("0123456789abcdeffedcba98765432101032547698badcfeefcdab8967452301", "0.0.0.0",
        "21bd1834bc088cd2b4ecbe30b70898d7", "21bd1834bc088cd2b4ecbe30b70898d782db0d4125fdace
       ("2b7e151628aed2a6abf7158809cf4f3c3c4fcf098815f7aba6d2ae2816157e2b", "2001:db8::1",
        "21bd1834bc088cd2b4ecbe30b70898d7", "21bd1834bc088cd2b4ecbe30b70898d76089c7e05ae30c2
ok = 0; total = 0
for k, ip, want in det:
```

```
    got = det_encrypt(ip, H(k)); back = det_decrypt(got, H(k)); total += 1
    ok += got == want and back == ip
    print(f"det  {ip:<36} -> {got:<40} {'OK' if got == want and back == ip else 'FAIL'}")
for k, ip, want in pfx:
    got = pfx_encrypt(ip, H(k)); back = pfx_decrypt(got, H(k)); total += 1
    ok += got == want and back == ip
    print(f"pfx  {ip:<36} -> {got:<40} {'OK' if got == want and back == ip else 'FAIL'}")
for k, ip, tw, want in ndx:
    got = ndx_encrypt(ip, H(k), H(tw)).hex(); back = ndx_decrypt(H(got), H(k)); total += 1
    ok += got == want and back == ip
    print(f"ndx  {ip:<36} -> {got[32:]:<40} {'OK' if got == want and back == ip else 'FAIL'}
print(f"{ok}/{total} Appendix A vectors match (encrypt and decrypt round-trip)")
```

```
$ python3 vectors.py
det  0.0.0.0                                -> bde9:6789:d353:824c:d7c6:f58a:6bd2:26eb  OK
det  255.255.255.255                        -> aed2:92f6:ea23:58c3:48fd:8b8:74e8:45d8   OK
det  192.0.2.1                              -> 1dbd:c1b9:fff1:7586:7d0b:67b4:e76e:4777  OK
pfx  0.0.0.0                                -> 151.82.155.134                           OK
pfx  255.255.255.255                        -> 94.185.169.89                            OK
pfx  192.0.2.1                              -> 100.115.72.131                           OK
pfx  2001:db8::1                            -> c180:5dd4:2587:3524:30ab:fa65:6ab6:f88   OK
pfx  10.0.0.47                              -> 19.214.210.244                           OK
pfx  10.0.0.129                             -> 19.214.210.80                            OK
pfx  172.16.5.193                           -> 210.78.229.136                           OK
pfx  172.16.97.42                           -> 210.78.179.241                           OK
pfx  2001:db8::a5c9:4e2f:bb91:5a7d          -> 7cec:702c:1243:f70:1956:125:b9bd:1aba    OK
pfx  2001:db8:3a5c:0:e7d1:4b9f:2c8a:f673    -> 7cec:702c:3503:bef:e616:96bd:be33:a9b9   OK
ndx  0.0.0.0                                -> 82db0d4125fdace61db35b8339f20ee5         OK
ndx  2001:db8::1                            -> 6089c7e05ae30c2d10ca149870a263e4         OK
15/15 Appendix A vectors match (encrypt and decrypt round-trip)
```

All 15 published vectors match, including the prefix-preserving ones. For example, `10.0.0.47` and `10.0.0.129` both encrypt into `19.214.210.0/24`.

Lab 2: anonymise a log, then ask analyst questions

A tiny access log:

```
198.51.100.7 GET /login 200
198.51.100.7 POST /login 401
198.51.100.7 POST /login 401
198.51.100.42 POST /login 401
203.0.113.9 GET / 200
2001:db8:1::5 GET /api 200
2001:db8:1::5 GET /api 200
```

```
2001:db8:2::77 GET /api 503
```

`logs.py` encrypts it in each mode, then asks the questions an analyst would ask: how many distinct clients, who failed login most, and how traffic splits per subnet.

```
# Anonymise access.log three ways, then ask the questions an analyst asks.
import ipaddress, os, collections
from ipcrypt import *
k16 = bytes.fromhex("000102030405060708090a0b0c0d0e0f")              # demo keys only
k32 = bytes.fromhex("000102030405060708090a0b0c0d0e0f" "f0e1d2c3b4a5968778695a4b3c2d1e0f")
rows = [l.split(" ", 1) for l in open("access.log").read().splitlines()]
modes_ = {"deterministic": lambda ip: det_encrypt(ip, k16),
          "pfx":           lambda ip: pfx_encrypt(ip, k32),
          "ndx":           lambda ip: ndx_encrypt(ip, k32).hex()}
for name, f in modes_.items():
    out = [(f(ip), rest) for ip, rest in rows]
    print(f"== {name}")
    for ip, rest in out: print(f"   {ip:<66} {rest}")
    ips = [ip for ip, _ in out]
    print(f"   distinct clients seen: {len(set(ips))} (true: {len(set(i for i, _ in rows))})
    fails = collections.Counter(ip for ip, r in out if r.endswith("401"))
    print(f"   top failed-login source: {fails.most_common(1)[0]}")
    if name == "pfx":
        nets = collections.Counter(str(ipaddress.ip_network(ip + ("/24" if "." in ip else "/
                                   for ip in ips)
        print(f"   requests per encrypted /24 or /48: {dict(nets)}")
print("== pitfalls")
same = k16 + k16
print(f"   pfx with K1 == K2:            198.51.100.7 -> {pfx_encrypt('198.51.100.7', same)}
print(f"   deterministic IPv4 output is an IPv6 address: 198.51.100.7 -> {det_encrypt('198.5
print(f"   ::ffff:198.51.100.7 and 198.51.100.7 encrypt alike: "
      f"{det_encrypt('::ffff:198.51.100.7', k16) == det_encrypt('198.51.100.7', k16)}")
blob = ndx_encrypt("198.51.100.7", k32)
print(f"   ndx decrypts with the key: {ndx_decrypt(blob, k32)}  (32-byte blob, not an IP)")
```

```
$ python3 logs.py
== deterministic
   6cf6:bc3b:adc5:c807:7270:735:8db6:e26e                          GET /login 200
   6cf6:bc3b:adc5:c807:7270:735:8db6:e26e                          POST /login 401
   6cf6:bc3b:adc5:c807:7270:735:8db6:e26e                          POST /login 401
   bd7:ebb4:ef51:d560:d775:ffc0:9fdc:98a1                          POST /login 401
   9e96:b5fa:cc75:1ff5:b970:3a73:7d65:bdc                          GET / 200
   1eca:41ba:da03:6949:d97:4996:c9b4:2d45                          GET /api 200
   1eca:41ba:da03:6949:d97:4996:c9b4:2d45                          GET /api 200
   2dd7:5e83:56db:9d8c:438:164f:a863:1ba9                          GET /api 503
   distinct clients seen: 5 (true: 5)
```

```
   top failed-login source: ('6cf6:bc3b:adc5:c807:7270:735:8db6:e26e', 2)
== pfx
   138.109.141.226                                                    GET /login 200
   138.109.141.226                                                    POST /login 401
   138.109.141.226                                                    POST /login 401
   138.109.141.217                                                    POST /login 401
   130.129.173.184                                                    GET / 200
   58c4:2b5:1d26:30c6:9e1:f8b:da5b:4d8b                               GET /api 200
   58c4:2b5:1d26:30c6:9e1:f8b:da5b:4d8b                               GET /api 200
   58c4:2b5:1d24:2949:9b9c:dcc9:77e0:4639                             GET /api 503
   distinct clients seen: 5 (true: 5)
   top failed-login source: ('138.109.141.226', 2)
   requests per encrypted /24 or /48: {'138.109.141.0/24': 4, '130.129.173.0/24': 1, '58c4:2
== ndx
   9aa70866a42bf765f8b3301d0c5560aa5333b450f7dc10f61382a832c6c0451d   GET /login 200
   8f4e760fb216b782a7589e395730dbf098e5e6497705a51a817225643e46b607   POST /login 401
   3128a30be457eee93d19d47b9d8030263a71d85a2607f2951c0b2d16761afc03   POST /login 401
   33c8caec07707f1613ff523449732512bf2bdaa16896728597ca60c0504fb9de   POST /login 401
   79baa7f5b899ad7728bec6c41d220a51aa6bed86b0151eb28bbef603189b645d   GET / 200
   504ffca0be16d2841be91d61ca1483a7962c7ca76c531fc34f02faa689e5bce0   GET /api 200
   e5082e0614cb69f1a481568759435b20345a213d6b16055de98ecaf3f3b80053   GET /api 200
   3e529042bac91aa14af5151e5ac02842c56622440e726931c7b84c7a80da7624   GET /api 503
   distinct clients seen: 8 (true: 5)
   top failed-login source: ('8f4e760fb216b782a7589e395730dbf098e5e6497705a51a817225643e46b6
== pitfalls
   pfx with K1 == K2:            198.51.100.7 -> 198.51.100.7
   deterministic IPv4 output is an IPv6 address: 198.51.100.7 -> 6cf6:bc3b:adc5:c807:7270:73
   ::ffff:198.51.100.7 and 198.51.100.7 encrypt alike: True
   ndx decrypts with the key: 198.51.100.7  (32-byte blob, not an IP)
```

The `ndx` rows will differ on every run, because the tweak comes from `os.urandom`. Everything else is deterministic for these keys.

Reading it:

- **Deterministic:** counting and joins still work: 5 distinct clients, and the brute-force source is visible with 2 failures. But `198.51.100.7` and `198.51.100.42` are now unrelated values, so you cannot tell they sit in the same /24. Every IPv4 client also becomes an IPv6-looking address, which surprises tools that bucket by family.
- **pfx:** everything deterministic gives, plus structure. The two `198.51.100.x` clients share `138.109.141.0/24`. `2001:db8:1::/48` and `2001:db8:2::/48` become `…:1d26::/48` and `…:1d24::/48`, which share 46 bits exactly as the originals do. That is useful for "which subnet is attacking us". It is also a leak: anyone with outside knowledge of one host can learn about its neighbours. §6.2.3 says it outright: prefix-preserving encryption "intentionally reveals network structure", and organisations needing complete obfuscation should use a non-prefix-preserving mode. §7.4.3 lists network analytics, subnet monitoring and DDoS mitigation as its use cases.
- **ndx:** 8 rows give 8 distinct values. Without the key you cannot even tell that one client failed

login twice. Use it when the log reader should not be able to link anything. The output is a 32-byte blob, not an IP, so the schema has to change.

Pitfalls the lab shows

- **pfx with equal key halves is the identity function.** The PRF is AES(K1) AES(K2), so K1 = K2 cancels out. The draft says the halves MUST NOT be identical (§6.2.2), and the lab shows `198.51.100.7 -> 198.51.100.7`. Generate the 32-byte key randomly, and never by repeating a 16-byte key.
- **Format change in deterministic mode.** IPv4 goes in and IPv6 comes out. Size and parse columns accordingly.
- **Keys are the whole security.** Anyone holding the key decrypts everything; that is by design (§8). Store the key like any other secret, and rotate it on a schedule. Rotation also breaks correlation across periods, which may be exactly what a retention policy wants.
- **Tweak collisions in the nd modes** are bounded by about 2^32 encryptions per key for nd and 2^64 for ndx (§7.4.1, §7.4.2). A collision reveals only a repetition, not the address.

The boring rule

- Logs shared outside the team that only need "same client?" get `ipcrypt-deterministic`.
- Abuse and DDoS work that needs subnets gets `ipcrypt-pfx`, accepting the neighbour leak, with a random 32-byte key.
- Anything that must not be linkable gets `ipcrypt-ndx`, with a schema that stores 32 bytes.
- Keep the key in your secret store, rotate it, and never treat ciphertext as tamper-proof.

Try this

1. Encrypt every address in `10.0.0.0/24` with `pfx_encrypt` and confirm that all 256 outputs fall into one /24. Then try `10.0.0.0/23`.
2. Change one bit of a `det_encrypt` output and decrypt it. What comes back? (There is no integrity, so you get a different, valid-looking address.)
3. Add `ipcrypt-nd` by implementing KIASU-BC from the draft's §9.9 and check it against Appendix A.3.

Sources

- F. Denis, draft-denis-ipcrypt-14, *Methods for IP Address Encryption and Obfuscation*: §3 conversion, §5.1 deterministic, §6.2 ipcrypt-pfx, §7.4 nd/ndx, §8 security considerations, §9.6 and §9.8 pseudocode, Appendix A test vectors: https://datatracker.ietf.org/doc/draft-denis-ipcrypt/
- IPCrypt project site (implementations and examples): https://ipcrypt-std.github.io/
- RFC 4291 §2.5.5.2 (IPv4-mapped IPv6 addresses): https://www.rfc-editor.org/rfc/rfc4291

- RFC 5737 and RFC 3849 (documentation address ranges): https://www.rfc-editor.org/rfc/rfc5737, https://www.rfc-editor.org/rfc/rfc3849
- `cryptography` AES/ECB primitive docs: https://cryptography.io/en/latest/hazmat/primitives/symmetric-encryption/

Policy, QoS, and Hardening

ACL Thinking

ACL Thinking

Access control lists are not a pile of lines—they are a **decision policy** evaluated in order. This chapter builds vendor-agnostic ACL decision patterns and implements them with Linux nftables/iptables and FRR filters in Containerlab (mid-2026 free tools).

Learning goals

By the end of this chapter you can:

- Write allow/deny policies with explicit default stance
- Place filters at the right plane (data plane vs route policy vs management)
- Implement Linux netfilter rules that match a written policy
- Apply FRR prefix-lists/route-maps without confusing them with packet ACLs
- Predict asymmetric ACL failures and fix with evidence

Concepts

Three different "filters" people mix up

Kind	Acts on	Example
Packet ACL / firewall	Data-plane packets	nftables drop tcp/23
Route policy	Routes in RIB/advertisements	prefix-list on BGP
Control-plane protection	Traffic **to** the CPU/RE	CoPP / policers (next chapter)

Using a BGP prefix-list will **not** stop a host scan. Dropping packets will **not** stop a bad route install (unless you couple systems).

Policy statement template

Before typing rules, write:

```
Stance: default deny (or default allow-pick deliberately)
Allow:
  - established/related (if stateful)
  - DNS to resolver R from clients
  - HTTPS to internet from clients
  - BGP from peer P on interface I
Deny:
  - everything else (log sample)
```

First-match vs best-match

Most packet ACLs are **first match wins**. Order is part of the policy. Route filters also often first-match in lists. Never "add a deny at the top" without reading the whole list.

Stateful vs stateless

	Stateless	Stateful
Idea	Each packet alone	Track flows/conntrack
Return traffic	Must allow reverse explicitly	Often allow established
Lab tools	Simple iptables rules	iptables/nft conntrack

Direction and attachment

```
inbound on client LAN   inbound on uplink
```

Always specify **interface + direction + zone**. Diagram the edge:

```
[clients] --in--> | edge | --out--> [uplink]
                  |      |
                  +-- mgmt
```

Written policy for the lab

Edge role: small site router (Linux).

1. Default **drop** forwarded traffic not explicitly allowed.

2. Allow clients (`10.60.0.0/24`) to ping and HTTP to lab server `10.60.1.10`.

3. Allow clients DNS to `10.60.0.53`.
4. Allow SSH **only** from mgmt host `10.60.0.9` to edge.
5. Deny client access to edge SSH from other addresses.
6. Log dropped new TCP (rate-limited mentally—don't flood disks).

Topology YAML

```yaml
name: acl-think

topology:
  nodes:
    edge:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iptables iputils curl
    client:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils curl
    mgmt:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils openssh-client
    srv:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils busybox-extras
        # tiny http: nc -l -p 80 ...

  links:
    - endpoints: ["client:eth1", "edge:eth1"]
    - endpoints: ["mgmt:eth1", "edge:eth1"]  # same LAN via bridge preferred
    - endpoints: ["edge:eth2", "srv:eth1"]
```

Prefer a bridge node so client + mgmt share `10.60.0.0/24`:

```yaml
    lan:
      kind: linux
```

```
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2
        - ip link add br0 type bridge && ip link set br0 up
  links:
    - endpoints: ["client:eth1", "lan:eth1"]
    - endpoints: ["mgmt:eth1", "lan:eth2"]
    - endpoints: ["edge:eth1", "lan:eth3"]
    - endpoints: ["edge:eth2", "srv:eth1"]
```

Addressing

Node	IP
edge eth1	10.60.0.1/24
edge eth2	10.60.1.1/24
client	10.60.0.10/24 via .1
mgmt	10.60.0.9/24 via .1
srv	10.60.1.10/24 via .1
DNS (optional)	10.60.0.53 on edge lo or dnsmasq

```
docker exec clab-acl-think-edge sh -c '
  sysctl -w net.ipv4.ip_forward=1
  ip addr add 10.60.0.1/24 dev eth1; ip link set eth1 up
  ip addr add 10.60.1.1/24 dev eth2; ip link set eth2 up
'
docker exec clab-acl-think-client sh -c '
  ip addr add 10.60.0.10/24 dev eth1; ip link set eth1 up
  ip route add default via 10.60.0.1
'
docker exec clab-acl-think-mgmt sh -c '
  ip addr add 10.60.0.9/24 dev eth1; ip link set eth1 up
  ip route add default via 10.60.0.1
'
docker exec clab-acl-think-srv sh -c '
  ip addr add 10.60.1.10/24 dev eth1; ip link set eth1 up
  ip route add default via 10.60.1.1
'
```

Implement with iptables (readable teaching form)

Flush carefully in lab only:

```
docker exec clab-acl-think-edge sh -c '
iptables -F
iptables -X
iptables -t nat -F
iptables -P INPUT DROP
iptables -P FORWARD DROP
iptables -P OUTPUT ACCEPT

# loopback + established
iptables -A INPUT -i lo -j ACCEPT
iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
iptables -A FORWARD -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT

# SSH to edge only from mgmt
iptables -A INPUT -p tcp -s 10.60.0.9 --dport 22 -j ACCEPT

# allow ping to edge from LAN (optional)
iptables -A INPUT -p icmp -s 10.60.0.0/24 -j ACCEPT

# forward: client/mgmt to server http + ping
iptables -A FORWARD -s 10.60.0.0/24 -d 10.60.1.10 -p tcp --dport 80 -j ACCEPT
iptables -A FORWARD -s 10.60.0.0/24 -d 10.60.1.10 -p icmp -j ACCEPT

# explicit deny counters
iptables -A FORWARD -j LOG --log-prefix "FWD-DROP: " --log-level 4
iptables -A INPUT -j LOG --log-prefix "IN-DROP: " --log-level 4
'
```

If `conntrack` module unavailable in minimal Alpine, approximate with broad return allows **only for learning**, then move to an image with conntrack.

nftables sketch (same policy)

```
docker exec clab-acl-think-edge sh -c 'apk add --no-cache nftables; nft -f - <<EOF
flush ruleset
table inet filter {
  chain input {
    type filter hook input priority 0; policy drop;
    iif lo accept
    ct state established,related accept
    tcp dport 22 ip saddr 10.60.0.9 accept
    ip protocol icmp ip saddr 10.60.0.0/24 accept
  }
  chain forward {
    type filter hook forward priority 0; policy drop;
    ct state established,related accept
```

```
    ip saddr 10.60.0.0/24 ip daddr 10.60.1.10 tcp dport 80 accept
    ip saddr 10.60.0.0/24 ip daddr 10.60.1.10 ip protocol icmp accept
  }
  chain output {
    type filter hook output priority 0; policy accept;
  }
}
EOF'
```

Predict → observe → fix drills

Drill 1 — Allowed path

```
docker exec clab-acl-think-srv sh -c 'echo OK | nc -l -p 80 &'
docker exec clab-acl-think-client ping -c 2 10.60.1.10
docker exec clab-acl-think-client wget -qO- http://10.60.1.10 || \
  docker exec clab-acl-think-client nc -w2 10.60.1.10 80 </dev/null
```

Predict: success. **If fail:** forwarding, routes, rule order, server listen.

Drill 2 — Denied path

```
docker exec clab-acl-think-client sh -c 'nc -w2 10.60.1.10 22 || true'
docker exec clab-acl-think-edge iptables -L FORWARD -n -v
```

Predict: fail; counters/logs increment.

Drill 3 — SSH source restriction

```
# from mgmt should work if dropbear/sshd runs on edge
# from client should fail
docker exec clab-acl-think-client sh -c 'nc -w2 10.60.0.1 22 || echo blocked'
```

Drill 4 — Asymmetric paths

Allow only client→server tcp/80 but forget established return:

Predict: SYN works, data stalls if truly stateless wrong.
Observe: conntrack ESTABLISHED rule presence.
Fix: stateful allow or explicit reverse rule.

Drill 5 — Wrong plane

Install FRR on edge and **only** add a prefix-list denying `10.60.1.0/24`. Client ping to server still works if connected routes exist.

Predict: route policy packet filter.
Observe: `ip route` still forwards; iptables counters unchanged for that myth.
Fix: teach the table of three filter kinds again.

FRR: route policy is still "ACL thinking"

```
ip prefix-list BOGON seq 5 deny 0.0.0.0/8 le 32
ip prefix-list BOGON seq 10 deny 10.0.0.0/8 le 32
ip prefix-list BOGON seq 100 permit any
!
route-map IMPORT deny 10
 match ip address prefix-list BOGON
route-map IMPORT permit 20
```

Same discipline: **written intent → ordered rules → negative tests**.

Object-group idea (vendor-agnostic)

Group hosts and services even if the engine lacks object-groups:

```
CLIENTS = 10.60.0.0/24
SERVERS = 10.60.1.10/32
WEB     = tcp/80
```

Expand into concrete rules; keep the group doc next to the ruleset in git.

Change control for ACLs

1. Write policy diff in markdown.
2. Add rules in lab.
3. Positive + negative tests.
4. Only then promote.
5. Never "temporary allow any" without a ticket/expiry note.

Verification checklist

Check	Intent
Policy doc matches counters	Rules do what you think
Default action explicit	No surprise open
Negative tests	Deny paths proven
Management path	You cannot lock yourself out without console plan
Logs sampled	Drops are visible

Common mistakes

Mistake	Symptom
Allow after broad deny	Never hits
Filter only one direction	One-way connectivity
Open SSH to world in lab configs committed	Habit risk
Logging every drop at full rate	Disk / CPU melt
Confusing RIB filter with packet filter	False confidence

Summary

- ACLs are **ordered policies** with an explicit default stance
- Separate **packet**, **route**, and **control-plane** filtering in your head
- Linux netfilter in Containerlab is enough to practice real data-plane policy
- Always pair positive allows with **negative** tests
- Write the policy in prose before the rule engine dialect

Next: **control-plane protection**—stop the CPU from drowning while the data plane still forwards.

Control-Plane Protection

Control-Plane Protection

The data plane can forward millions of packets while a handful of exception packets **kill the control plane**. Control-plane protection (CoPP / CPPr / RE protect—names vary) rate-limits and classifies traffic **to the device CPU**. This chapter builds the model with Linux and FRR-facing labs (Containerlab, mid-2026 open stack).

Learning goals

By the end of this chapter you can:

- Distinguish transit traffic from traffic destined to the router
- List common control-plane protocols worth protecting
- Apply Linux rate limits / policers toward local services
- Design a CoPP policy matrix (class → police → action)
- Predict symptoms of over-tight vs over-loose protection

Concepts

Two paths through a router

```
Transit (data plane):  in → FIB lookup → out
Receive (to CPU):      in → "this is for me" → process
```

Examples to CPU	Examples transit
BGP, OSPF, SSH, SNMP, NTP	Customer flows through the box
ARP to router MAC	Switched L2 through (L2 device)
ICMP echo to router IP	ICMP through to remote host
TTL expiry messages	Normal forwarded packets

Why unprotected CPUs melt

- Routing protocol floods / session storms
- Management scans (SSH/HTTPS)
- ICMP floods to the router IP
- ARP storms
- Fragment / exception paths

Symptoms: neighbor flaps, slow CLI, missed hellos → **self-inflicted outage** even if bandwidth remains.

Policy matrix (start simple)

Class	Match idea	Police	Notes
Critical routing	OSPF/BGP from known peers	generous	Never starve hellos blindly
Management	SSH/HTTPS from mgmt net	moderate	Jump host only
Monitoring	SNMP/telemetry agents	moderate	
Normal ICMP to RE	echo to router	low	
Default to RE	everything else	very low / drop	

Vendors implement class-maps + policers. Linux: nftables meter/limit, tc, or firewall rate modules.

Hardening adjacent controls

CoPP is **not** a substitute for:

- Interface ACLs that drop garbage early
- Management plane VRF / out-of-band
- Routing policy
- Authentication on protocols

Defense in depth: each layer different failure mode.

Lab topology

```
  attacker ----|          |---- peer (OSPF/BGP)
  mgmt --------|  router  |
  client ------|          |---- transit target
```

```yaml
name: copp-lab

topology:
  nodes:
    rtr:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/rtr:/etc/frr
      # also need iptables in image or side package
    peer:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/peer:/etc/frr
    mgmt:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils openssh-client
    attacker:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
    client:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
    target:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils

  links:
    - endpoints: ["rtr:eth1", "peer:eth1"]
    - endpoints: ["rtr:eth2", "mgmt:eth1"]
    - endpoints: ["rtr:eth3", "attacker:eth1"]
```

```
    - endpoints: ["rtr:eth4", "client:eth1"]
    - endpoints: ["rtr:eth5", "target:eth1"]
```

Addressing

Link	Subnet
rtr–peer	10.0.0.0/30 (.1 rtr, .2 peer)
rtr–mgmt	10.255.0.0/30 (.1 rtr)
rtr–attacker	10.66.0.0/30
rtr–client	10.10.0.0/24 (.1 rtr, .10 client)
rtr–target	10.20.0.0/24 (.1 rtr, .10 target)
rtr lo	192.0.2.1/32

Enable forward for transit client→target; BGP or static for peer experiments.

Baseline routing (static snack)

```
docker exec clab-copp-lab-rtr sysctl -w net.ipv4.ip_forward=1
# addresses omitted-set on each node per table
docker exec clab-copp-lab-client ip route add default via 10.10.0.1
docker exec clab-copp-lab-target ip route add default via 10.20.0.1
```

Optional OSPF between rtr and peer on eth1 (reuse interior-routing habits).

Linux receive-path protection (teaching CoPP)

Protect **SSH to the router** and **ICMP to the router**, without blocking transit.

```
# On rtr (needs iptables in container-install if FRR image lacks it)
docker exec clab-copp-lab-rtr sh -c 'command -v iptables || (apt-get update && apt-get insta
  2>/dev/null || true
```

FRR images vary; if package install is painful, run CoPP rules on an Alpine "router" with `ip_forward` and FRR only when needed. Pattern matters more than package manager.

```
docker exec clab-copp-lab-rtr sh -c '
# INPUT chain = traffic to local stack (control/management plane analogue)
iptables -N COPP 2>/dev/null || iptables -F COPP
iptables -F COPP

# BGP from peer only (TCP 179)
```

```
iptables -A COPP -p tcp -s 10.0.0.2 --dport 179 -j ACCEPT
iptables -A COPP -p tcp -s 10.0.0.2 --sport 179 -j ACCEPT

# OSPF (IP proto 89) from peer subnet
iptables -A COPP -p 89 -s 10.0.0.0/30 -j ACCEPT

# SSH from mgmt only, rate-limit new connections
iptables -A COPP -p tcp -s 10.255.0.0/30 --dport 22 -m conntrack --ctstate NEW -m limit --li
iptables -A COPP -p tcp -s 10.255.0.0/30 --dport 22 -m conntrack --ctstate ESTABLISHED -j AC

# ICMP to self: low rate
iptables -A COPP -p icmp --icmp-type echo-request -m limit --limit 5/sec -j ACCEPT

# established to self
iptables -A COPP -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT

# drop other to-self
iptables -A COPP -j DROP

iptables -I INPUT 1 -j COPP
'
```

Critical: do **not** put transit in INPUT. FORWARD remains separate:

```
docker exec clab-copp-lab-rtr sh -c '
iptables -P FORWARD ACCEPT
# or tighter ACL from previous chapter
'
```

Predict → observe → fix

Drill 1 — Transit still works under "attack" to RE

```
# flood ping to router IP from attacker
docker exec clab-copp-lab-attacker sh -c 'ping -f -c 1000 10.66.0.1 || ping -c 200 10.66.0.1
# simultaneous transit
docker exec clab-copp-lab-client ping -c 20 10.20.0.10
```

Predict: transit mostly fine; many attacker pings dropped by limit; CLI may still respond if limits work.

Observe: `iptables -L COPP -n -v` counters.

Fix if transit dies: rules mistakenly in FORWARD; CPU still overloaded—tighten attacker path earlier (interface ACL).

Drill 2 — SSH from non-mgmt

```
docker exec clab-copp-lab-attacker sh -c 'nc -w2 10.66.0.1 22 || echo blocked'
docker exec clab-copp-lab-mgmt sh -c 'nc -w2 10.255.0.1 22 || true'
```

Predict: attacker blocked; mgmt allowed (if sshd up).

Drill 3 — Over-tight BGP protection

Intentionally rate-limit TCP 179 too aggressively:

```
iptables -I COPP 1 -p tcp --dport 179 -m limit --limit 1/hour -j ACCEPT
iptables -I COPP 2 -p tcp --dport 179 -j DROP
```

Predict: BGP session flaps or won't establish.
Observe: `vtysh -c 'show bgp summary'`.
Fix: remove bad rules; document that **routing classes need headroom**.

Drill 4 — Peer-only OSPF

Send spoofed or neighbor attempt from attacker interface (concept): OSPF from non-peer should not be accepted.

Observe: no adjacency from wrong interface; INPUT drops proto 89 from wrong sources.

FRR-side levers (awareness)

Lever	Role
`ttl-security` / GTSM	BGP hop check
MD5/TCP-AO (as available)	Session auth
Interface ACLs + passive-interface	Reduce attack surface
Max prefix	Limit RIB blowups
BFD timers sanity	Avoid micro-flap storms

These complement packet CoPP; they do not replace it.

Building a CoPP document

```
Device role: lab edge
Classes:
  - ROUTING_PEERS: list IPs/protos, police 1Mbps, prio high
```

```
  - MGMT: 10.255.0.0/30 tcp/22, 100kbps
  - ICMP_RE: 64kbps
  - DEFAULT_RE: 32kbps drop excess
Exceptions: console always
Change window: test in lab before prod
Rollback: saved ruleset file in git
```

Store as `copp-policy.md` beside topology.

IPv6 notes

Same model: traffic to link-local and global router addresses hits receive path. ND/RA are control-plane-adjacent—filter carefully or you break neighbor discovery.

Optional SR Linux / NOS CoPP

Commercial and community NOS expose CoPP/CPM filter policies in YANG. Map your matrix classes to their filter entries; validate with protocol sessions + flood tests. Free path still proves the **idea** on Linux.

Verification checklist

Check	Intent
Transit ping during RE flood	Data vs control separation
Known peer protocols up	Not over-policed
Unknown src management down	Surface reduced
Counters move on floods	Policy hit
Rollback file exists	Safety

Common mistakes

Mistake	Symptom
Policing FORWARD as “CoPP”	Kills customers, not RE
Starving BGP/OSPF	Random reconvergence
No mgmt exception	Locked out
Only cloud dashboards, no on-box	Outage of mgmt path blind
Copy-paste prod CoPP into lab scale	Lab protocols die

Summary

- Control-plane protection polices **traffic to the CPU**, not transit
- Build a **class matrix** before touching syntax
- Linux INPUT rate limits teach the model; NOS CoPP is the same idea
- Always retest **routing adjacencies** after tightening
- Combine with mgmt ACLs, auth, and max-prefix for defense in depth

Next: **QoS vocabulary**—how queues and marks express intent without vendor exam laundry lists.

QoS Vocabulary

QoS Vocabulary

Quality of Service is a pile of vendor features if you start from menus. Start from **vocabulary and intent**: classify → mark → queue → schedule → (maybe) shape/police. This chapter stays conceptual with small Linux `tc` experiments in Containerlab—no paid QoS licenses (mid-2026).

Learning goals

By the end of this chapter you can:

- Explain classification, marking, queuing, scheduling, policing, shaping
- Map DSCP/PHB ideas without memorizing every codepoint table
- Build a minimal Linux htb/fq_codel lab and measure fairness under load
- Predict what happens when you police vs shape
- Know what *not* to promise with lab QoS

Concepts — the pipeline

```
packets in

Classify  (match: 5-tuple, interface, DSCP, VLAN, app)

Mark      (set DSCP / internal class / MPLS TC / VLAN PCP)

Police    (drop/remark excess - hard limit)

Queue     (buffers per class)

Schedule  (which queue sends next: PQ, WRR, WFQ-like…)
```

```
Shape     (smooth egress to rate - buffer then send)


wire
```

Not every hop does every stage. Trust boundaries matter: **remark at edges**, honor marks in the core only if you trust them.

Vocabulary table

Term	Meaning
Classification	Deciding which class a packet belongs to
Marking	Writing a field others can see (e.g. DSCP)
PHB	Per-hop behavior—what a class *should* experience
Policing	Enforce rate; excess drop or remark (little buffering)
Shaping	Enforce rate by delaying (buffering) excess
Queue depth	How much can wait; latency vs drop trade-off
Congestion avoidance	ECN, RED/WRED-like early drop/mark
Scheduling	Priority vs sharing bandwidth among classes
LLQ / priority queue	Low-latency class that can starve others if unbounded
Fair queueing	Prevent one flow from dominating (fq / fq_codel ideas)
Trust boundary	Where you accept or rewrite marks

DSCP / PHB (operator view)

Intent class	Typical idea	Notes
Network control	Highest care for protocols	Protect, don't just "EF everything"
Expedited (EF-like)	Low latency, low loss	Strict admission; small volume
Assured / AF-like	Business data tiers	Drop precedence inside class
Default / BE	Best effort	Most traffic
Scavenger	Below BE	Backups, bulk

Exact codepoints are standards-listed; **do not** treat exam bingo as design. Document *your* map:

```
EF-like   → voice lab class (tiny)
AF31-like → interactive
CS6-like  → routing (if marked)
BE        → default
```

Policing vs shaping

	Police	Shape
Excess	Drop/remark	Delay in buffer
Latency	Can be lower for in-profile	Adds delay when congested
Use	Enforce contract, ingress	Match egress to circuit rate

Predict: shaping a 100 mbit stream to 10 mbit increases latency under load; policing 10 mbit drops packets and breaks TCP more brutally.

Linux lab topology

```
name: qos-vocab

topology:
  nodes:
    r1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils iperf3
    h1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils iperf3
    h2:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils iperf3

  links:
    - endpoints: ["h1:eth1", "r1:eth1"]
    - endpoints: ["r1:eth2", "h2:eth1"]
```

Addressing

Node	IP
h1	10.80.1.10/24 via 10.80.1.1
r1 eth1	10.80.1.1/24

Node	IP
r1 eth2	10.80.2.1/24
h2	10.80.2.10/24 via 10.80.2.1

```
docker exec clab-qos-vocab-r1 sysctl -w net.ipv4.ip_forward=1
# apply addresses on all nodes...
```

Experiment 1 — bottleneck without QoS

```
# limit r1 eth2 with a simple tbf shape to create congestion
docker exec clab-qos-vocab-r1 tc qdisc replace dev eth2 root tbf rate 5mbit burst 32kbit lat

docker exec -d clab-qos-vocab-h2 iperf3 -s
docker exec clab-qos-vocab-h1 iperf3 -c 10.80.2.10 -t 20 -P 4
```

Predict

Aggregate ~5 Mbit; latency rises; flows share roughly (TCP dynamics).

Observe

```
docker exec clab-qos-vocab-h1 iperf3 -c 10.80.2.10 -t 10 -J | head
docker exec clab-qos-vocab-r1 tc -s qdisc show dev eth2
```

Experiment 2 — two classes with HTB

Goal: bulk class 3 mbit ceiling; interactive class 2 mbit guaranteed-ish; total 5 mbit.

```
docker exec clab-qos-vocab-r1 sh -c '
tc qdisc del dev eth2 root 2>/dev/null || true
tc qdisc add dev eth2 root handle 1: htb default 20
tc class add dev eth2 parent 1: classid 1:1 htb rate 5mbit ceil 5mbit
tc class add dev eth2 parent 1:1 classid 1:10 htb rate 2mbit ceil 5mbit prio 1
tc class add dev eth2 parent 1:1 classid 1:20 htb rate 3mbit ceil 5mbit prio 2
tc qdisc add dev eth2 parent 1:10 handle 10: fq_codel
tc qdisc add dev eth2 parent 1:20 handle 20: fq_codel
# classify by destination port: "interactive" = 5202, bulk = 5201
tc filter add dev eth2 protocol ip parent 1:0 prio 1 u32 match ip dport 5202 0xffff flowid 1
tc filter add dev eth2 protocol ip parent 1:0 prio 2 u32 match ip dport 5201 0xffff flowid 1
'
```

Run two iperf servers/ports on h2:

```
docker exec -d clab-qos-vocab-h2 iperf3 -s -p 5201
docker exec -d clab-qos-vocab-h2 iperf3 -s -p 5202
docker exec -d clab-qos-vocab-h1 iperf3 -c 10.80.2.10 -p 5201 -t 30 -P 2
docker exec clab-qos-vocab-h1 iperf3 -c 10.80.2.10 -p 5202 -t 20
```

Predict

Interactive class gets better latency / share under contention than pure bulk; total still capped ~5 mbit.

Observe

```
docker exec clab-qos-vocab-r1 tc -s class show dev eth2
docker exec clab-qos-vocab-r1 tc -s qdisc show dev eth2
```

Fix

Wrong filter → all traffic in default class. Check `flowid` and dport match (u32 host byte care).

Experiment 3 — police vs shape feel

```
# police ingress-ish on r1 eth1 (example)
docker exec clab-qos-vocab-r1 sh -c '
tc qdisc replace dev eth1 handle ffff: ingress
tc filter add dev eth1 parent ffff: protocol ip u32 match u32 0 0 \
  police rate 2mbit burst 20k drop flowid :1
'
```

Compare TCP goodput and retransmissions vs tbf shape on egress.

Predict: police drops → TCP sawtooth pain; shape delays → higher RTT, smoother rate.

Marking sketch (DSCP)

```
# set DSCP on packets from h1 (example iptables)
docker exec clab-qos-vocab-h1 sh -c '
apk add --no-cache iptables
iptables -t mangle -A OUTPUT -p tcp --dport 5202 -j DSCP --set-dscp 0x2e
'
```

```
# on r1 classify on DSCP instead of dport (tc flower/u32 match)
```

Predict

If r1 trusts and classifies EF-like mark into priority class, marked flows win under load.

Observe

```
docker exec clab-qos-vocab-r1 sh -c 'apk add --no-cache tcpdump; tcpdump -ni eth1 -v -c 5 tc
```

What lab QoS cannot prove

- Hardware queue behavior of a specific ASIC
- Carrier hierarchical QoS at scale
- Exact voice MOS scores
- Multi-vendor PHB interoperability without careful design

It **can** prove: classification logic, bottlenecks, unfairness, and the difference between police/shape.

FRR / NOS note

FRR is not your main QoS engine. Real NOS (including SR Linux community) expose class-maps and queues in their models. Translate:

```
class VOICE → match DSCP EF → queue LLQ-like → police admit
class BULK  → match default → shaped remainder
```

Same vocabulary.

Design mini-checklist

- ☐ Trust boundary defined
- ☐ Classes 4–5 for first design
- ☐ Admission for low-latency class

- ☐ Capacity math: sum guarantees link
- ☐ Congestion metrics (drop, latency, ECN) collected
- ☐ Failure mode: QoS mis-class worse than no QoS?

Predict → observe → fix template

Step	Example
Predict	Bulk should not starve interactive under 5 mbit cap
Observe	iperf + `tc -s` + ping RTT during load
Fix	filters, class rates, default class, ECN/fq_codel

Common mistakes

Mistake	Symptom
Mark without classifying later	Marks are cosmetics
Guarantees sum > link	Unexpected drops
Priority unbounded	Starvation of BE
QoS to "fix" bandwidth shortage	Still need capacity
Policing management/routing	Self-outage (see CoPP)

Summary

- QoS is a **pipeline of intents**, not a single CLI feature
- Classify → mark → queue → schedule → police/shape
- Linux `tc` + iperf3 in Containerlab is enough to feel congestion policy
- Document trust boundaries and class maps in git
- Keep routing/management classes out of scavenger police

Next: **hardening checklists**—turn models into repeatable gates for labs and edges.

Hardening Checklists

Hardening Checklists

Hardening is not a one-time "secure the router" checkbox. It is a **repeatable set of gates** you run after every design change. This chapter consolidates edge, control-plane, ACL, and ops hygiene into checklists you can attach to Containerlab labs (mid-2026 open stack: FRR, Linux, optional SR Linux notes).

Learning goals

By the end of this chapter you can:

- Run role-based hardening checklists (edge, P, lab host)
- Turn checklist items into automated `verify-hard.sh` probes
- Balance security with operability (break-glass paths)
- Record exceptions with expiry and owner
- Apply the same gates to dual-home and fabric labs

How to use these lists

1. **Pick the role** of the device or lab.
2. Mark each item: Pass / Fail / N/A / Exception.
3. Failures become backlog; exceptions need owner + date.
4. Automate the boring Pass/Fail checks.
5. Re-run after every topology or policy change.

```
Design → Implement → Functional verify → Hardening verify → Document
```

Checklist 0 — Lab hygiene (every topology)

#	Gate	Pass looks like
0.1	Topology in git	`.clab.yml` + configs committed
0.2	Addressing plan	No overlapping accidental spaces
0.3	Secrets	No production passwords in repo
0.4	Destroy works	`containerlab destroy -c` clean
0.5	Logs bound	Evidence survives destroy
0.6	Images free	Main path needs no paid license
0.7	Resource note	RAM/CPU expected documented

Checklist 1 — Management plane

#	Gate	Pass looks like
1.1	SSH/API not on all interfaces blindly	Listen limited or filtered
1.2	Mgmt sources limited	ACL/CoPP allow jump net only
1.3	Auth	Keys or strong lab passwords; no default vendor admin left
1.4	Banner / MOTD	Optional but marks non-prod
1.5	Out-of-band story	Console or dual path to recover
1.6	Telemetry endpoints known	Who scrapes what
1.7	DNS for mgmt names	Optional but reduces fat-finger

Linux probes

```
# who listens?
docker exec clab-X-r1 ss -lntp
# INPUT policy not wide open without reason
docker exec clab-X-r1 iptables -L INPUT -n -v | head
```

Checklist 2 — Control plane & routing

#	Gate	Pass looks like
2.1	Passive interfaces on access LANs	No accidental OSPF on user ports
2.2	BGP peer ACLs / TTL security as applicable	Only intended peers
2.3	Max-prefix on eBGP	Limit configured + action known
2.4	Prefix filters in and out	Negative hijack test fails closed

#	Gate	Pass looks like
2.5	Router-IDs unique	Documented
2.6	Auth on IGP/BGP where in scope	Or explicit exception
2.7	CoPP/receive ACL present on edges	Matrix documented
2.8	Loop prevention	No rediscovery of classic redistrib loops

FRR probes

```
docker exec clab-X-r1 vtysh -c 'show ip ospf interface'
docker exec clab-X-r1 vtysh -c 'show bgp summary'
docker exec clab-X-r1 vtysh -c 'show run' | grep -E 'prefix-list|route-map|max-prefix|passiv
```

Checklist 3 — Data plane filters

#	Gate	Pass looks like
3.1	Written policy exists	Prose before rules
3.2	Default stance explicit	Deny or allow documented
3.3	Positive tests	Needed paths work
3.4	Negative tests	Forbidden paths fail
3.5	Anti-spoof (uRPF-like idea)	As scope allows
3.6	Directed broadcast / dangerous services off	No chargen era nonsense
3.7	IPv6 filters parity	Not “v6 open because forgot”

Checklist 4 — Edge services

#	Gate	Pass looks like
4.1	DHCP pools correct	giaddr/relay tested
4.2	NAT scoped	Only intended interfaces
4.3	VRRP/anycast VIP documented	Single master observed
4.4	DNS dependency known	Outage drill done once
4.5	Time sync	chrony/ntp status OK
4.6	Logging on role changes	VRRP/BGP flap visible

Checklist 5 — Secrets, users, supply chain

#	Gate	Pass looks like
5.1	Image sources pinned	Digests/tags known
5.2	No shared root SSH everywhere	Or accepted lab-only risk noted
5.3	CI does not print secrets	Logs clean
5.4	Third-party scripts reviewed	Especially curl

Checklist 6 — Resilience & abuse

#	Gate	Pass looks like
6.1	Link failure drill	Recover within budget
6.2	Peer failure drill	Documented
6.3	Flood to RE drill	Transit survives
6.4	Config rollback	Known good in git
6.5	Capacity headroom	Lab not at 100% RAM always

Role profiles (quick)

Lab host (Containerlab hypervisor)

- Disk for images; unused labs destroyed
- Access limited to operators
- sysctl baseline documented
- Docker/podman not exposed to world

FRR edge CE

- Checklists 1–4 heavily
- Dual-home policies + VRRP tracking story
- Export only owned space

FRR P / fabric leaf

- Minimal services listening
- Underlay filters; no customer ACLs confusion
- ECMP + BFD timer sanity
- CoPP light but present

Linux bridge “switch”

- No IP on data plane unless needed
- STP/loop story if L2 loops possible
- Do not run random services on bridge NS

Exception register template

ID	Control	Reason	Owner	Expiry	Review
E-001	BGP without TTL security	Lab only single hop	you	2026-12-01	

Exceptions without expiry become permanent holes.

Automating gates — `verify-hard.sh`

```
#!/usr/bin/env bash
set -euo pipefail
LAB=${LAB:-clab-dualhome}
fail() { echo "FAIL: $*" >&2; exit 1; }
pass() { echo "PASS: $*"; }

# 2.4 negative: hijack absent on pe1
if docker exec ${LAB}-pe1 vtysh -c 'show bgp ipv4 uni' 2>/dev/null | grep -q '203.0.113.0';
  fail "hijack prefix present"
else
  pass "no hijack prefix"
fi

# 1.2 example: ssh port not open to all - soft check listening interfaces
```

```
docker exec ${LAB}-ce1 ss -lntp | grep -q ssh && pass "sshd present" || pass "no sshd (ok if

# 4.3 VIP answers
docker exec ${LAB}-h1 ping -c1 -W1 192.168.50.254 >/dev/null && pass "VIP" || fail "VIP"

# 0.1 topology file exists
test -f dualhome.clab.yml && pass "topology file" || fail "missing topology"

echo "HARDENING SMOKE OK"
```

Wire into Makefile:

```
smoke: ; ./verify.sh
hard:  ; ./verify-hard.sh
```

Predict → observe → fix (meta-drill)

Break one control on purpose:

```
# remove FROM-CE filter on pe1 in lab
```

Predict | `verify-hard.sh` fails hijack test when you advertise junk |
Observe | script FAIL line |
Fix | restore route-map; re-run |

Never disable a gate silently—disable via exception register.

Mapping to prior chapters

Chapter theme	Checklist
ACL thinking	3.x
CoPP	2.7, 6.3
QoS	don't police control into death; document classes
Dual-home edge	2.x + 4.x
Name/time/log	4.4–4.6, 0.5

Optional SR Linux

Export checklist to NOS-specific show commands in an appendix note. Same gates: peer authenticity, prefix limits, mgmt ACL, logging. Free community image is for dialect practice, not a second security model.

Definition of done — hardened lab

- ☐ Role checklist completed with Pass/Fail
- ☐ Exceptions registered
- ☐ `verify.sh` + `verify-hard.sh` green
- ☐ One intentional fail demonstrated (gate works)
- ☐ Rollback path known

Common mistakes

Mistake	Symptom
Checklist once, never again	Drift
Automating only happy path	False green
Lockout without console	Unrecoverable lab (or prod!)
Copying bank-grade CoPP into tiny lab	Protocols die
Paper checklists not in repo	Lost tribal knowledge

Summary

- Hardening is **repeatable gates**, not a mood
- Use role-based checklists + exception register
- Automate negative tests (hijack, flood survival, mgmt limits)
- Re-run after every material change
- Operability (break-glass) is part of security

Next part: **overlays and tunnels**—why we encapsulate, and how free stacks practice VXLAN/EVPN-class ideas.

SYN Floods and Listen Queues: SYN_RECV, SYN Cookies, and Accept Overflow

SYN Floods and Listen Queues: SYN_RECV, SYN Cookies, and Accept Overflow

A TCP server commits memory before it knows the client is real. The first SYN makes the kernel allocate a request socket, send a SYN-ACK, and wait. A **SYN flood** sends SYNs from spoofed addresses that will never send the final ACK, so those half-open entries pile up until the listener refuses real clients. Linux has two queues per listener, two different limits, and a stateless escape hatch, **SYN cookies**, that is on by default. Most "we tuned it and nothing changed" incidents come from mixing up which queue overflowed. This chapter separates them, measures each one with real counters, and runs an actual spoofed flood, without root, inside a throwaway network namespace.

Learning goals

- Draw the SYN queue (request sockets, `SYN_RECV`) and the accept queue (`ESTABLISHED`, waiting for `accept()`) and say which limit applies to each
- Explain why `net.ipv4.tcp_max_syn_backlog` does **not** size the SYN queue on a modern kernel, and what it actually does
- Read `ss -lnt Recv-Q/Send-Q` on a listener and the `TcpExt` counters (`ListenOverflows`, `ListenDrops`, `TCPReqQFullDrop`, `TCPReqQFullDoCookies`, `Syncookies*`)
- Say what a SYN cookie encodes, and what a cookie-built connection loses (MSS precision, and every option when timestamps are off)
- Tell a flood from an application that accepts too slowly, and fix the right one
- Reproduce a spoofed SYN flood safely with an unprivileged user + network namespace

Lab platform: Linux 6.12, iproute2 6.15 (`ss`, `ip`, `nstat`), Python 3.13.5, run as an unprivileged user. Each lab runs in its own `unshare -rn` namespace: root there, with its own sysctls, raw sockets, and veth, and no effect on the host. Every listing output below is pasted from those runs. Kernel behaviour is quoted from `net/ipv4/tcp_input.c`, `tcp_minisocks.c`, and `syncookies.c` in Linux 6.12.

This chapter covers the host's own defences. Upstream scrubbing, SYN proxies in front of the server (`nftables synproxy`, load balancers), and XDP drop paths are out of scope here.

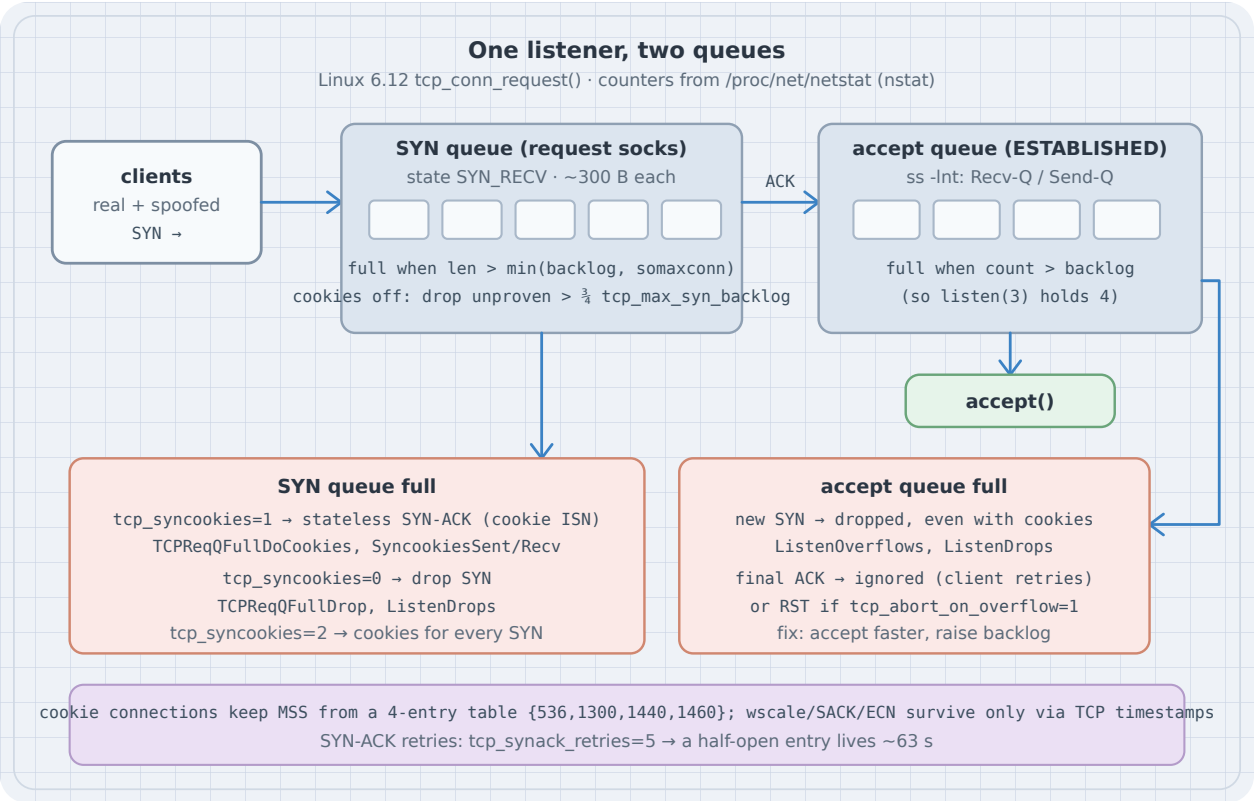


Figure 1: One listener, two queues

Mental model

```
              SYN                          final ACK                 accept()
client          [ SYN queue ]              [ accept queue ]      application
                     request socks, SYN_RECV          full sockets, ESTABLISHED
                     limit: len > min(backlog,         limit: count > min(backlog,
                            somaxconn)                        somaxconn)
                     overflow: cookie or drop         overflow: drop SYN; ignore ACK
                                                                 (RST if abort_on_overflow)
```

Both queues hang off the same `listen()` call. In Linux 6.12, `tcp_conn_request()` checks them in this order for every incoming SYN:

1. `tcp_syncookies == 2`, or the **SYN queue** is full (`inet_csk_reqsk_queue_len(sk) > sk_max_ack_backlog`)? Then `tcp_syn_flood_action()`: with cookies enabled, answer with a cookie (`TCPReqQFullDoCookies`); otherwise drop (`TCPReqQFullDrop`).
2. The **accept queue** is full (`sk_ack_backlog > sk_max_ack_backlog`)? Drop the SYN (`ListenOverflows`). Cookies do not help here.
3. Cookies off and the SYN queue more than three quarters of `tcp_max_syn_backlog`? Drop SYNs from peers the kernel has not seen before (`tcp_peer_is_proven()` is false). `ListenDrops` only.

4. Otherwise allocate a request socket (300 bytes), send the SYN-ACK, and retransmit it up to `tcp_synack_retries` times (5 → about 63 s before giving up).

`sk_max_ack_backlog` is the `backlog` you passed to `listen()`, silently capped at `net.core.somaxconn` (default 4096 since Linux 5.4). The `>` comparisons mean `listen(N)` holds **N + 1** entries.

Knob	Default here	What it really does on 6.12
`listen(fd, backlog)`	app-chosen	Sets **both** queue limits (`sk_max_ack_backlog`)
`net.core.somaxconn`	4096	Caps `backlog` without telling the application
`net.ipv4.tcp_syncookies`	1	0 = never, 1 = when the SYN queue is full, 2 = always
`net.ipv4.tcp_max_syn_backlog`	1024 (128, scales with RAM)	Only with cookies **off**: keep the last quarter for proven peers
`net.ipv4.tcp_synack_retries`	5	How long a half-open entry occupies a slot
`net.ipv4.tcp_abort_on_overflow`	0	Final ACK into a full accept queue: 0 = ignore it, 1 = send RST

All of these are per network namespace, which is what makes the labs below safe.

Concepts — what a SYN cookie is

With a cookie, the server keeps **no state**. It encodes the essentials in the 32-bit initial sequence number of its SYN-ACK: a keyed hash of the 4-tuple and a coarse timestamp, plus a 2-bit index into a table of four MSS values (`msstab = {536, 1300, 1440, 1460}` in `net/ipv4/syncookies.c`). When an ACK arrives with no request socket behind it, the kernel checks `ack - 1` against the hash (valid for two 60-second periods), rebuilds the connection, and counts `SyncookiesRecv` (or `SyncookiesFailed`).

The price is the TCP options. The ISN has no room for window scale, SACK, or ECN. Linux hides them in the low 6 bits of the TCP timestamp value if the client sent timestamps (`TS_OPT_WSCALE_MASK`, `TS_OPT_SACK`, `TS_OPT_ECN`). If timestamps are off, a cookie connection has no window scaling (64 KiB window), no SACK, and an MSS rounded down to the table. The kernel's own sysctl documentation is blunt: cookies are a fallback and "seriously violate TCP protocol"; if you see SYN-flood warnings from **legitimate** load, fix the backlog, not the cookies.

Lab — setup: a private network namespace with a void

`unshare -rn` gives an unprivileged user a new user namespace (mapped to root) and a new network namespace. The helper below adds a veth so the server has a real address (`10.9.0.1`) and routes

TEST-NET-2 (`198.51.100.0/24`, RFC 5737) to a fake next hop. SYN-ACKs to "spoofed" clients leave `v0` and are never answered, exactly like a real spoofed flood.

```
#!/bin/sh
# netns_lab.sh - run CMD inside a throwaway network namespace:
#   lo up, veth v0 = 10.9.0.1/24, and 198.51.100.0/24 (TEST-NET-2) routed into a void
# usage: unshare -rn ./netns_lab.sh CMD [ARGS...]
set -e
ip link set lo up
ip link add v0 type veth peer name v1
ip link set v0 up && ip link set v1 up
ip addr add 10.9.0.1/24 dev v0
ip route add 198.51.100.0/24 via 10.9.0.2 dev v0
# a fake next hop: SYN-ACKs to the "spoofed" clients leave v0 and are never answered
ip neigh replace 10.9.0.2 lladdr 02:00:00:00:00:02 dev v0 nud permanent
exec "$@"
```

```
#!/bin/sh
# listen_defaults.sh - the knobs that size and protect a TCP listener (this netns)
for k in net/ipv4/tcp_syncookies net/ipv4/tcp_max_syn_backlog \
         net/ipv4/tcp_synack_retries net/ipv4/tcp_abort_on_overflow net/core/somaxconn; do
  printf '%-32s %s\n' "$k" "$(cat /proc/sys/$k)"
done
nstat -az TcpExtListenOverflows TcpExtListenDrops TcpExtTCPReqQFullDrop \
          TcpExtTCPReqQFullDoCookies TcpExtSyncookiesSent TcpExtSyncookiesRecv
```

```
chmod +x netns_lab.sh listen_defaults.sh
./listen_defaults.sh                              # host namespace, read-only
unshare -rn ./netns_lab.sh ip route               # the lab namespace
```

```
net/ipv4/tcp_syncookies          1
net/ipv4/tcp_max_syn_backlog     1024
net/ipv4/tcp_synack_retries      5
net/ipv4/tcp_abort_on_overflow   0
net/core/somaxconn               4096
#kernel
TcpExtSyncookiesSent            0                  0.0
TcpExtSyncookiesRecv            0                  0.0
TcpExtListenOverflows           0                  0.0
TcpExtListenDrops               0                  0.0
TcpExtTCPReqQFullDoCookies      0                  0.0
TcpExtTCPReqQFullDrop           0                  0.0
```

```
10.9.0.0/24 dev v0 proto kernel scope link src 10.9.0.1
198.51.100.0/24 via 10.9.0.2 dev v0
```

The host counters are all zero, and they stay zero after every lab below: each `unshare -rn` namespace has its own `/proc/net/netstat` and its own sysctls.

`tcpext.py` is the shared counter reader (the same numbers `nstat` prints):

```
# tcpext.py - read TcpExt counters from /proc/net/netstat (what `nstat` reports)
KEYS = ("ListenOverflows", "ListenDrops", "TCPReqQFullDrop",
        "TCPReqQFullDoCookies", "SyncookiesSent", "SyncookiesRecv", "SyncookiesFailed")

def tcpext():
    lines = open("/proc/net/netstat").read().splitlines()
    for names, values in zip(lines[::2], lines[1::2]):
        if names.startswith("TcpExt:"):
            return dict(zip(names.split()[1:], map(int, values.split()[1:])))

def delta(before, after):
    return {k: after[k] - before[k] for k in KEYS if after[k] != before[k]}
```

Lab — accept queue: a server that never calls `accept()`

```
# accept_queue_lab.py - a listener that never calls accept(): what happens to client #5?
# usage: unshare -rn ./netns_lab.sh python3 accept_queue_lab.py BACKLOG CLIENTS ABORT_ON_OVE
import socket, subprocess, sys, time
from tcpext import tcpext, delta

backlog, clients, abort = map(int, sys.argv[1:4])
with open("/proc/sys/net/ipv4/tcp_abort_on_overflow", "w") as f:
    f.write(str(abort))

lsock = socket.socket()
lsock.bind(("10.9.0.1", 8080))
lsock.listen(backlog)                          # never accept()

before = tcpext()
socks = []
for i in range(clients):
    c = socket.socket()
    c.setblocking(False)
    c.connect_ex(("10.9.0.1", 8080))           # non-blocking: SYN goes out now
    socks.append(c)
    time.sleep(0.05)
time.sleep(1.5)                                # let one SYN retransmit (RTO 1 s) happen

print(f"listen({backlog}), {clients} clients, tcp_abort_on_overflow={abort}")
print(subprocess.run(["ss", "-ltn", "sport = :8080"], capture_output=True, text=True).stdout
```

```
states = subprocess.run(["ss", "-Htn", "dport = :8080"], capture_output=True, text=True).std
count = {}
for line in states.splitlines():
    count[line.split()[0]] = count.get(line.split()[0], 0) + 1
print("client-side states:", count)
srv = subprocess.run(["ss", "-Htn", "state", "syn-recv", "sport = :8080"],
                     capture_output=True, text=True).stdout
print("server SYN-RECV:", len(srv.splitlines()))
print("TcpExt deltas:", delta(before, tcpext()))
```

```
unshare -rn ./netns_lab.sh python3 accept_queue_lab.py 3 8 0
unshare -rn ./netns_lab.sh python3 accept_queue_lab.py 3 8 1
```

```
listen(3), 8 clients, tcp_abort_on_overflow=0
State  Recv-Q Send-Q Local Address:Port Peer Address:Port
LISTEN 4      3           10.9.0.1:8080      0.0.0.0:*
client-side states: {'SYN-SENT': 4, 'ESTAB': 4}
server SYN-RECV: 0
TcpExt deltas: {'ListenOverflows': 8, 'ListenDrops': 8}

listen(3), 8 clients, tcp_abort_on_overflow=1
State  Recv-Q Send-Q Local Address:Port Peer Address:Port
LISTEN 4      3           10.9.0.1:8080      0.0.0.0:*
client-side states: {'ESTAB': 4, 'SYN-SENT': 4}
server SYN-RECV: 0
TcpExt deltas: {'ListenOverflows': 8, 'ListenDrops': 8}
```

How to read it:

- On a **listening** socket, `ss` reuses the columns: `Recv-Q` is the current accept-queue length (4), `Send-Q` is its limit (3). `Recv-Q` = `Send-Q` + 1 is the "full" signature, because the check is `>`.
- Four clients completed the handshake and sit `ESTAB` in the queue. The application has not seen them, but the clients think they are connected and may already be sending.
- The other four never got a SYN-ACK. Their SYNs hit step 2 (accept queue full) and were dropped. Each client retransmitted once after the 1 s initial RTO, so $4 \times 2 = \mathbf{8}$ `ListenOverflows`. They stay in `SYN-SENT` and keep retrying with backoff; if the application wakes up and calls `accept()`, they get in.
- `tcp_abort_on_overflow=1` changed nothing. It acts only in `tcp_check_req()` when a **final ACK** arrives for a request whose accept queue filled in the meantime (`listen_overflow:` → `embryonic_reset`). Here the SYNs were refused before any handshake. In production it turns a short burst into connection resets for clients that would have recovered, which is why the kernel docs say to enable it only if the daemon really cannot keep up.
- No `SYN_RECV` entries and no cookie counters: this is not a flood. It is an application that does not accept fast enough, and the fix is in the application or its `listen()` backlog.

Lab — `somaxconn` silently caps the backlog

```
# somaxconn_lab.py - listen() backlog is silently capped by net.core.somaxconn
# usage: unshare -rn ./netns_lab.sh python3 somaxconn_lab.py
import socket, subprocess

def show(somaxconn, backlog):
    open("/proc/sys/net/core/somaxconn", "w").write(str(somaxconn))
    s = socket.socket()
    s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    s.bind(("10.9.0.1", 8080))
    s.listen(backlog)
    line = subprocess.run(["ss", "-Hltn", "sport = :8080"], capture_output=True, text=True).
    print(f"somaxconn={somaxconn:<5} listen({backlog:<6}) -> ss Send-Q={line[2]}")
    s.close()

show(4096, 128)
show(4096, 65535)
show(128, 1024)
```

```
unshare -rn ./netns_lab.sh python3 somaxconn_lab.py
```

```
somaxconn=4096  listen(128   ) -> ss Send-Q=128
somaxconn=4096  listen(65535 ) -> ss Send-Q=4096
somaxconn=128   listen(1024  ) -> ss Send-Q=128
```

`listen()` never fails because the backlog is too large. `__sys_listen_socket()` in `net/socket.c` clamps it to `somaxconn`, and only `ss -lnt` `Send-Q` shows what the kernel actually used. Many servers have their own setting (nginx `listen … backlog=`, default 511 on Linux; HAProxy `backlog`; Go's `net` package passes `somaxconn` itself; Java's `ServerSocket` defaults to 50). Raising `somaxconn` without raising the application value does nothing, and the reverse is capped.

Lab — a spoofed SYN flood, three ways

The flood builds IPv4 + TCP SYN headers by hand on a raw socket (allowed in the namespace, where the user is root). Sources cycle through `198.51.100.1-254`. After the flood, one **real** client in the same namespace tries to connect with a 3 s timeout.

```
# synflood_lab.py - spoofed SYN flood against a listener, inside a private netns.
# usage: unshare -rn ./netns_lab.sh python3 synflood_lab.py COOKIES MAX_SYN_BACKLOG BACKLOG
import os, random, socket, struct, sys, time
from tcpext import tcpext, delta

cookies, max_syn_backlog, backlog, syns = map(int, sys.argv[1:5])
```

```
SERVER, PORT = "10.9.0.1", 8080

def sysctl(name, value):
    with open("/proc/sys/" + name.replace(".", "/"), "w") as f:
        f.write(str(value))

sysctl("net.ipv4.tcp_syncookies", cookies)
sysctl("net.ipv4.tcp_max_syn_backlog", max_syn_backlog)

lsock = socket.socket()
lsock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
lsock.bind((SERVER, PORT))
lsock.listen(backlog)

def csum(data):
    if len(data) % 2:
        data += b"\0"
    s = sum(struct.unpack(f"!{len(data)//2}H", data))
    s = (s >> 16) + (s & 0xFFFF)
    return ~(s + (s >> 16)) & 0xFFFF

def syn(src, sport):
    dst = socket.inet_aton(SERVER)
    seq = random.getrandbits(32)
    tcp = struct.pack("!HHIIBBHHH", sport, PORT, seq, 0, 5 << 4, 0x02, 64240, 0, 0)
    pseudo = socket.inet_aton(src) + dst + struct.pack("!BBH", 0, 6, len(tcp))
    tcp = tcp[:16] + struct.pack("!H", csum(pseudo + tcp)) + tcp[18:]
    ip = struct.pack("!BBHHHBBH4s4s", 0x45, 0, 20 + len(tcp), 0, 0, 64, 6, 0,
                     socket.inet_aton(src), dst)
    return ip + tcp

def syn_recv_count():
    n = 0
    with open("/proc/net/tcp") as f:
        next(f)
        for line in f:
            local, state = line.split()[1], line.split()[3]
            if state == "03" and local.endswith(f":{PORT:04X}"):   # 03 = SYN_RECV
                n += 1
    return n

before = tcpext()
raw = socket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPROTO_RAW)
for i in range(syns):
    src = f"198.51.100.{1 + i % 254}"
    raw.sendto(syn(src, 1024 + i), (SERVER, 0))
time.sleep(0.5)
```

```
print(f"tcp_syncookies={cookies} tcp_max_syn_backlog={max_syn_backlog} "
      f"listen({backlog}) spoofed SYNs={syns}")
print(f"  SYN_RECV entries on :{PORT}: {syn_recv_count()}")

c = socket.socket()
c.settimeout(3)
t0 = time.monotonic()
try:
    c.connect((SERVER, PORT))
    conn, _ = lsock.accept()
    print(f"  real client: connected in {1000*(time.monotonic()-t0):.1f} ms, accepted")
except OSError as e:
    print(f"  real client: FAILED after {time.monotonic()-t0:.1f} s ({e})")
print(f"  TcpExt deltas: {delta(before, tcpext())}")
```

```
unshare -rn ./netns_lab.sh python3 synflood_lab.py 0  128 4096 1000   # cookies off, small m
unshare -rn ./netns_lab.sh python3 synflood_lab.py 0 4096   64 1000   # cookies off, small l
unshare -rn ./netns_lab.sh python3 synflood_lab.py 1  128   64 1000   # default: cookies whe
unshare -rn ./netns_lab.sh python3 synflood_lab.py 2  128   64   10   # cookies always
```

```
tcp_syncookies=0 tcp_max_syn_backlog=128 listen(4096) spoofed SYNs=1000
  SYN_RECV entries on :8080: 97
  real client: FAILED after 3.0 s (timed out)
  TcpExt deltas: {'ListenDrops': 906}
tcp_syncookies=0 tcp_max_syn_backlog=4096 listen(64) spoofed SYNs=1000
  SYN_RECV entries on :8080: 65
  real client: FAILED after 3.0 s (timed out)
  TcpExt deltas: {'ListenDrops': 938, 'TCPReqQFullDrop': 938}
tcp_syncookies=1 tcp_max_syn_backlog=128 listen(64) spoofed SYNs=1000
  SYN_RECV entries on :8080: 65
  real client: connected in 0.2 ms, accepted
  TcpExt deltas: {'TCPReqQFullDoCookies': 936, 'SyncookiesSent': 936, 'SyncookiesRecv': 1}
tcp_syncookies=2 tcp_max_syn_backlog=128 listen(64) spoofed SYNs=10
  SYN_RECV entries on :8080: 0
  real client: connected in 0.1 ms, accepted
  TcpExt deltas: {'TCPReqQFullDoCookies': 11, 'SyncookiesSent': 11, 'SyncookiesRecv': 1}
```

Every number is accounted for:

1. **Cookies off, `listen(4096)`, `tcp_max_syn_backlog=128`.** The SYN queue is nowhere near the listen limit, but it stops at **97** entries: step 3 drops unproven peers once fewer than `128 >> 2 = 32` slots of `tcp_max_syn_backlog` remain (`128 - 97 < 32`). 1000 − 97 = 903 spoofed SYNs dropped, plus the real client's SYN and its two retransmits (at 1 s and 3 s) = **906** `ListenDrops`. This path does not count `TCPReqQFullDrop` because the queue is not "full", it is being rationed. The real client is a new, unproven peer, so it loses too.
2. **Cookies off, `listen(64)`, `tcp_max_syn_backlog=4096`.** Now the listen backlog is the limit: **65** entries (`> 64`), then step 1 drops: 935 + 3 client attempts = **938**, counted as both

`TCPReqQFullDrop` and `ListenDrops`. A large `tcp_max_syn_backlog` did not buy a single extra slot.

3. **Default cookies (1).** The same 65 entries fill, then every further SYN gets a cookie: 935 spoofed + 1 real = **936** `SyncookiesSent`. The real client's ACK validates (`SyncookiesRecv` 1) and it connects in 0.2 ms, with a queue full of junk.
4. **Cookies always (2).** No request sockets at all (`SYN_RECV` 0). Every SYN, including the real one, is answered statelessly. This is a test mode, and it costs every client its options (next lab).

The kernel also logged the event once per listener (ratelimited, `tcp_syn_flood_action()`), on the host's `dmesg`, which is shared by all namespaces:

```
TCP: request_sock_TCP: Possible SYN flooding on port 10.9.0.1:8080. Dropping request.
TCP: request_sock_TCP: Possible SYN flooding on port 10.9.0.1:8080. Sending cookies.
```

Case 1 logs nothing: step 3 drops are reported only through `pr_drop_req()` at debug level. A host can be shedding real clients with no warning in the log, which is one more reason to read the counters.

Lab — what a cookie connection keeps

```
# cookie_options_lab.py - what a SYN-cookie connection keeps of the client's TCP options
# usage: unshare -rn ./netns_lab.sh python3 cookie_options_lab.py
import socket, struct

TS, SACK, WSCALE = 1, 2, 4                  # TCPI_OPT_* bits in tcp_info.tcpi_options

def sysctl(name, value):
    with open("/proc/sys/" + name.replace(".", "/"), "w") as f:
        f.write(str(value))

def options(sock):
    raw = sock.getsockopt(socket.IPPROTO_TCP, socket.TCP_INFO, 104)
    opts, wscale = raw[5], raw[6]
    snd_mss = struct.unpack_from("I", raw, 16)[0]
    names = [n for bit, n in ((TS, "timestamps"), (SACK, "sack"), (WSCALE, "wscale")) if opt
    if opts & WSCALE:
        names[-1] += f"={wscale & 0xF}/{wscale >> 4}"
    return f"{' '.join(names) or '(no options)'}  snd_mss={snd_mss}"

def handshake(cookies, timestamps):
    sysctl("net.ipv4.tcp_syncookies", cookies)
    sysctl("net.ipv4.tcp_timestamps", timestamps)
    lsock = socket.socket()
    lsock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    lsock.bind(("10.9.0.1", 8080))
```

```
    lsock.listen(16)
    c = socket.socket()
    c.connect(("10.9.0.1", 8080))
    conn, _ = lsock.accept()
    print(f"syncookies={cookies} timestamps={timestamps}: server sees {options(conn)}")
    for s in (c, conn, lsock):
        s.close()

handshake(0, 1)   # normal handshake
handshake(2, 1)   # always cookies, timestamps on: options ride in the TS value
handshake(2, 0)   # always cookies, no timestamps: only an MSS bucket survives
```

```
unshare -rn ./netns_lab.sh python3 cookie_options_lab.py
```

```
syncookies=0 timestamps=1: server sees timestamps sack wscale=7/7  snd_mss=32768
syncookies=2 timestamps=1: server sees timestamps sack wscale=7/7  snd_mss=1448
syncookies=2 timestamps=0: server sees (no options)  snd_mss=1460
```

- A normal handshake negotiates timestamps, SACK, and window scale 7 in both directions. The MSS is large because the path is local (veth address over loopback, MTU 65536).
- With a cookie and timestamps, window scale and SACK survive (they rode in the timestamp's low bits). The MSS drops to **1448**: 1460 from `msstab` minus 12 bytes of timestamp option. A loopback-class path now uses Ethernet-sized segments.
- With a cookie and **no timestamps**, nothing survives: no window scaling (receive window capped at 64 KiB, which caps throughput at 64 KiB per RTT), no SACK (loss recovery without selective acknowledgements, one hole per RTT), MSS 1460. Some middleboxes and hosts disable TCP timestamps for privacy or legacy reasons; their users pay this price on every cookie connection.

That is why cookies are the default *fallback* (`1`), not the default *mode* (`2`).

Case — reading a production listener

The same counters tell three different stories. Take two samples a few seconds apart (`nstat` with no `-a` prints the delta since its last run):

You see	It is	Do
`ListenOverflows` rising, `Recv-Q` = `Send-Q` + 1 on `ss -lnt`, cookie counters flat	Accept queue full: the application is slow (GC pause, blocked event loop, too few workers) or the backlog is tiny	Fix the accept loop, raise the app's `listen` backlog **and** `somaxconn`
`TCPReqQFullDoCookies` / `SyncookiesSent` rising, many `SYN_RECV` from scattered sources, `SyncookiesRecv` small	SYN flood (or a burst far above the backlog), cookies are absorbing it	Nothing urgent on the host; raise the backlog if real clients also trigger it; push filtering upstream

You see	It is	Do
`SyncookiesRecv` `SyncookiesSent`, few distinct sources	Legitimate load above the SYN-queue limit, served by cookies	Raise the listen backlog; this is the case the sysctl docs warn about
`TCPReqQFullDrop` or `ListenDrops` rising with cookies off	Real clients are being refused	Turn cookies back on (`tcp_syncookies=1`)

```
ss -lnt '( sport = :443 )'                        # Recv-Q / Send-Q per listener
ss -Htn state syn-recv '( sport = :443 )' | wc -l  # half-open count
nstat TcpExtListenOverflows TcpExtListenDrops TcpExtTCPReqQFullDoCookies TcpExtSyncookiesSen
```

(Commands for a production host; their output here would be the zero counters shown in the setup.)

The trap

Trap: "raise `tcp_max_syn_backlog` to survive a SYN flood." On current kernels the SYN queue limit is the listen backlog (capped by `somaxconn`). `tcp_max_syn_backlog` matters only when cookies are **off**, and then it is a rationing threshold, not a size. Lab case 2 set it to 4096 and still got 65 slots. The `ip-sysctl` text still calls it a "maximal number of remembered connection requests"; the code in `tcp_conn_request()` is the authority.

Trap: "SYN cookies protect against a full accept queue." They do not. Step 2 drops SYNs when the accept queue is full, cookies or not. A slow `accept()` loop looks like a SYN flood to clients (timeouts in `SYN-SENT`) but shows `ListenOverflows`, not cookie counters.

Trap: "`listen(65535)` gives me a 64 K backlog." `somaxconn` clamps it without an error. Check `Send-Q` on the listener.

Trap: "turn on `tcp_abort_on_overflow` so clients fail fast." It only resets handshakes whose final ACK arrives into a full accept queue, and it turns a recoverable burst into hard `ECONNRESET`s. Fix the accept rate instead.

Trap: "`tcp_syncookies=2` is safer." It disables request sockets entirely, so every client gets cookie-quality options (MSS table, no options without timestamps). Leave it at `1`.

The boring rule

Size the listen backlog in the application and in `somaxconn` together, leave `tcp_syncookies=1`, and when clients time out, read `ss -lnt` and the `TcpExt` counters before touching a sysctl: `ListenOverflows` means accept faster, `SyncookiesSent` means you are being flooded and the host is coping.

Try this

1. Re-run the accept-queue lab, then add an `accept()` loop after 2 s. How long do the `SYN-SENT` clients take to get in, and why (SYN retransmit backoff 1, 2, 4 s …)?
2. In the flood lab, set `tcp_synack_retries=1` with cookies off. Re-run the real client after 5 s instead of immediately. What changed, and what does that tell you about how long a flood must be sustained?
3. Add IPv6 to `netns_lab.sh` (`2001:db8::/32` routed into the void) and repeat case 3. Which counters are shared between IPv4 and IPv6?
4. With `tcp_timestamps=0` on the **client** side only, make a cookie connection and send 10 MB over the veth. Compare throughput with a normal connection.
5. Put an `nft synproxy` rule in front of the lab listener (the namespace lets you load rules there) and compare which counters move during the flood.

Verification

- Linux **6.12.94+** (Debian 13), iproute2 **6.15.0**, Python **3.13.5**, unprivileged user `uid=1000`; each lab ran in its own `unshare -rn` user + network namespace. All outputs above are pasted from those runs.
- The flood was real (1000 hand-built SYN packets on a raw socket per case) but never left the namespace: spoofed sources are TEST-NET-2, SYN-ACKs went to a static fake neighbour on a veth with no peer stack. Host `TcpExt` counters stayed 0.
- Counter arithmetic was checked against `tcp_conn_request()`, `tcp_syn_flood_action()`, `inet_csk_reqsk_queue_is_full()`, `sk_acceptq_is_full()` (Linux 6.12) and the SYN retransmit timing of the real client (initial RTO 1 s).
- **Not shown:** the `tcp_abort_on_overflow` RST path (it needs a final ACK to race a filling accept queue, which local handshakes do not do); it is described from `tcp_check_req()` in `tcp_minisocks.c`. Production `ss`/`nstat` commands in the Case section were not run against a production host.
- No packages installed, no `sudo`, no host sysctl changed.

Sources

- Linux 6.12 `net/ipv4/tcp_input.c` — `tcp_conn_request()`, `tcp_syn_flood_action()`: https://git.kernel.org/pub/scm/linux/kernel/git/stable/linux.git/tree/net/ipv4/tcp_input.c?h=linux-6.12.y
- Linux 6.12 `net/ipv4/syncookies.c` — `msstab`, timestamp option bits, cookie validity: https://git.kernel.org/pub/scm/linux/kernel/git/stable/linux.git/tree/net/ipv4/syncookies.c?h=linux-6.12.y
- Linux 6.12 `net/ipv4/tcp_minisocks.c` — `tcp_check_req()` `listen_overflow` / `tcp_abort_on_overflow`
- Linux 6.12 `include/net/inet_connection_sock.h`, `include/net/sock.h` — queue-full checks; `net/socket.c` — `somaxconn` clamp
- Kernel docs — IP sysctl: `tcp_syncookies`, `tcp_max_syn_backlog`, `tcp_synack_retries`, `tcp_abort_on_overflow`

- man7 — `tcp(7)`, `listen(2)`, `ss(8)`, `user_namespaces(7)`
- RFC 4987 — TCP SYN Flooding Attacks and Common Mitigations
- RFC 9293 — Transmission Control Protocol (three-way handshake, §3.5)
- RFC 5737 — IPv4 Address Blocks Reserved for Documentation

FQ-CoDel and CAKE: How Linux Fights Bufferbloat, and How to Check It Is Actually On

FQ-CoDel and CAKE: How Linux Fights Bufferbloat, and How to Check It Is Actually On

A link that is full does not have to be slow. When a bulk upload makes a video call stutter, the culprit is almost never bandwidth. It is a **queue** in front of the bottleneck that fills with the upload's packets until each call packet waits hundreds of milliseconds behind them. That is **bufferbloat**. Linux has two well-known answers. **FQ-CoDel** (RFC 8290) gives each flow its own small queue and keeps each queue short with the **CoDel** AQM (RFC 8289). **CAKE** adds a shaper so that *you* own the bottleneck queue rather than the modem. This chapter builds the algorithms from the RFC pseudocode in a small simulator, measures FIFO against CoDel and FQ-CoDel, and ends with the configuration and checks that matter on a real router.

Learning goals

- Explain why a full FIFO causes delay without causing loss, and why loss-based TCP keeps filling it
- Describe CoDel's control law: sojourn time, `target`, `interval`, and drop spacing `interval/sqrt(count)`
- Describe FQ-CoDel's DRR scheduler with new and old flow lists, and why sparse flows skip the queue
- Know when FQ-CoDel is enough and when you need CAKE's shaper (or HTB + FQ-CoDel)
- Verify which qdisc is really running, and whether the queue you are managing is the one that fills

Mental model

```
              one queue                          per-flow queues + DRR
FIFO       [b b b b b b b b b v b b b]      FQ-CoDel   new: [v]
           v waits behind everything                    old: [b b] [b b] [b b] [b b]    link
CoDel      [b b b v b]   drop at head when     each queue has its own CoDel,
           sojourn > 5 ms for 100 ms                    a sparse flow is served from "new"
```

Three rules:

1. **Delay is queue length divided by link rate.** A thousand 1514-byte packets at 10 Mbit/s is 1.2 seconds. Memory is cheap, so buffers are deep.
2. **Loss-based TCP grows until something drops.** A deep tail-drop buffer gives no signal, so every bulk flow fills it. AQM sends the signal (drop or ECN mark) early, based on *time* spent queued rather than bytes.

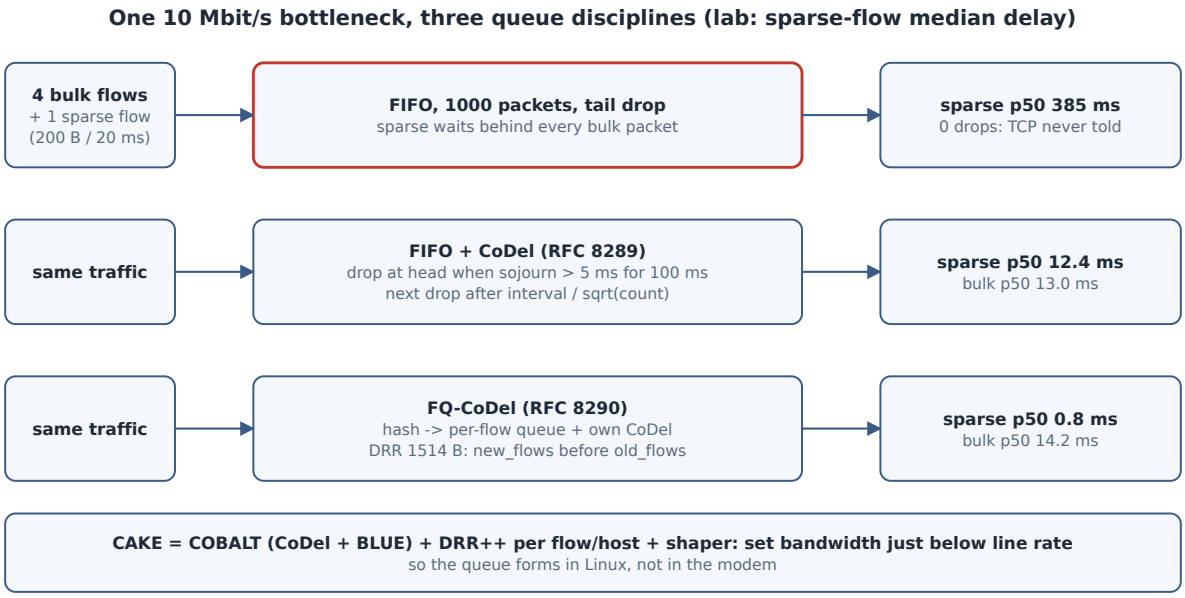


Figure 1: FIFO, CoDel and FQ-CoDel in front of a 10 Mbit/s bottleneck, with the sparse-flow latency each one produced in the lab.

3. **You can only manage a queue you own.** The qdisc must sit on the bottleneck. On a home router the bottleneck is usually the modem's buffer, so you shape slightly below line rate (CAKE `bandwidth`, or HTB) to move the queue into Linux.

Concepts

CoDel in four lines (RFC 8289)

- Stamp every packet at enqueue. At dequeue compute its **sojourn time**.
- If sojourn has been above **target** (5 ms) continuously for at least **interval** (100 ms), enter the dropping state and drop the head packet.
- While dropping, drop the next packet after `interval / sqrt(count)`, so the drop rate rises until TCP backs off.
- Leave the dropping state as soon as sojourn falls below target. If you re-enter soon, resume near the previous `count` rather than starting over.

Dropping at the **head** matters: the sender learns about congestion one queue-length sooner than it would with tail drop.

FQ-CoDel (RFC 8290)

- Packets are hashed on the 5-tuple into one of **1024** queues (`flows`). Each queue has its own CoDel state.

- A deficit round robin scheduler with a **1514-byte quantum** serves two lists. A queue that becomes active joins **new_flows** and is served first. When it uses up its quantum, it moves to the tail of **old_flows**.
- A flow that never builds a queue (DNS, game traffic, voice, TCP ACKs) is served from new_flows almost every time it has a packet, which is why its latency collapses.
- The defaults in `tc-fq_codel(8)` are `limit 10240` packets, `memory_limit` 32 MB, `target 5ms`, `interval 100ms`, and `ecn` **on**, which unlike plain `codel` marks ECN-capable packets instead of dropping them.

CAKE (tc-cake(8))

CAKE is "a shaping-capable queue discipline which uses both AQM and FQ". Its parts:

- **COBALT**, which is CoDel combined with BLUE for unresponsive flows
- a deficit-mode shaper
- DRR++ flow isolation with 8-way set-associative hashing
- diffserv tins (`diffserv3` by default)
- `triple-isolate` per-host fairness by default, with `nat` to see through masquerading
- overhead keywords (`docsis`, `ethernet`, `pppoe-*`, `atm`) that make the shaper count bytes the way the link does

Who is on by default

- The kernel's built-in `net.core.default_qdisc` is `pfifo_fast`. Virtual devices such as `lo` and `veth` ignore it and default to `noqueue`.
- systemd's `sysctl.d/50-default.conf` sets `net.core.default_qdisc = fq_codel`, so most systemd distributions get FQ-CoDel on new interfaces.
- Multi-queue NICs get `mq` with a per-queue child of that default.
- Interfaces created before the sysctl applied, and kernels without the module, keep whatever they had.

Lab 1: what this host actually runs

```
$ cat /proc/sys/net/core/default_qdisc
pfifo_fast
$ ip link show enp0s4 | head -1
2: enp0s4: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP mode DEFAULT
```

The box has `pfifo_fast`, not `fq_codel`. `tc` is not installed, so `kernel.py` asks the kernel directly over netlink (pyroute2 0.9.7 in a user venv) inside an unprivileged network namespace (`unshare -rn`). It creates a veth pair and tries to attach each qdisc:

```
from pyroute2 import IPRoute
ip = IPRoute()
```

```
ip.link('add', ifname='a', kind='veth', peer='b')
i = ip.link_lookup(ifname='a')[0]
for kind in ('pfifo', 'fq_codel', 'cake'):
    try:
        ip.tc('add', kind, i, '1:'); print(f"{kind:9} available"); ip.tc('del', kind, i, '1:
    except Exception as e:
        print(f"{kind:9} {type(e).__name__}: {e}")
```

```
$ unshare -rn python kernel.py
pfifo     available
fq_codel  NetlinkError: (2, 'No such file or directory')
cake      NetlinkError: (2, 'No such file or directory')
```

`ENOENT` from `RTM_NEWQDISC` means the kernel does not know that qdisc, either because `sch_fq_codel` / `sch_cake` is not built or because the module cannot be loaded. This kernel (6.12) is a minimal build with neither. On a real router the same check is `tc qdisc add dev X root fq_codel` failing with "No such file or directory". **Check this before you tune parameters that are not running.**

Lab 2: the algorithms, simulated

Because the kernel here has no FQ-CoDel, `aqm.py` implements both from the RFCs. CoDel follows the RFC 8289 §5 pseudocode. FQ-CoDel follows RFC 8290 §4: DRR over new/old lists with a CoDel per queue. The bottleneck is 10 Mbit/s with a 1000-packet buffer. Four Reno-like bulk flows run with a 20 ms base RTT, alongside one sparse flow sending a 200-byte packet every 20 ms. Delay is measured from enqueue to the end of transmission:

```
"""Bottleneck simulator: FIFO vs CoDel (RFC 8289) vs FQ-CoDel (RFC 8290).
10 Mbit/s link, 1514-byte bulk packets, 4 Reno-like bulk flows (base RTT 20 ms),
plus one sparse flow: a 200-byte packet every 20 ms (a VoIP-ish probe)."""
import heapq, math, statistics, sys
from collections import deque

RATE = 10e6 / 8            # bytes per second
BASE_RTT = 0.020
TARGET, INTERVAL = 0.005, 0.100   # RFC 8289 defaults
LIMIT = 1000               # packets, a "bufferbloated" tail-drop buffer
SIM = float(sys.argv[1]) if len(sys.argv) > 1 else 30.0

class CoDel:                        # RFC 8289 section 5 pseudocode, one instance per queue
    def __init__(s): s.first_above = 0.0; s.drop_next = 0.0; s.count = 0; s.lastcount = 0; s
    def _ok_to_drop(s, sojourn, now, qlen_bytes):
        if sojourn < TARGET or qlen_bytes <= 1514:
            s.first_above = 0.0; return False
        if s.first_above == 0.0:
```

```
            s.first_above = now + INTERVAL; return False
        return now >= s.first_above
    def dequeue(s, q, now, drop):
        """q: deque of (enq_time, size, flow, seq); returns a packet or None; drop(p) record
        if not q: s.dropping = False; return None
        p = q.popleft(); ok = s._ok_to_drop(now - p[0], now, sum(x[1] for x in q) + p[1])
        if s.dropping:
            if not ok: s.dropping = False
            while s.dropping and now >= s.drop_next:
                drop(p); s.count += 1
                if not q: s.dropping = False; return None
                p = q.popleft()
                if not s._ok_to_drop(now - p[0], now, sum(x[1] for x in q) + p[1]): s.droppi
                else: s.drop_next += INTERVAL / math.sqrt(s.count)
        elif ok:
            drop(p)
            if not q: return None
            p = q.popleft(); s.dropping = True
            delta = s.count - s.lastcount
            s.count = delta if (delta > 1 and now - s.drop_next < 16 * INTERVAL) else 1
            s.drop_next = now + INTERVAL / math.sqrt(s.count); s.lastcount = s.count
        return p

class Fifo:
    def __init__(s, aqm): s.q = deque(); s.codel = CoDel() if aqm else None
    def enqueue(s, p, drop):
        if len(s.q) >= LIMIT: drop(p)
        else: s.q.append(p)
    def dequeue(s, now, drop):
        if s.codel: return s.codel.dequeue(s.q, now, drop)
        return s.q.popleft() if s.q else None
    def __len__(s): return len(s.q)

class FqCodel:                      # RFC 8290 section 4: DRR over new/old flow lists, CoDel
    QUANTUM = 1514
    def __init__(s): s.qs = {}; s.new = deque(); s.old = deque(); s.n = 0
    def enqueue(s, p, drop):
        f = p[2]
        if f not in s.qs: s.qs[f] = [deque(), CoDel(), 0, None]   # queue, codel, deficit, l
        st = s.qs[f]; st[0].append(p); s.n += 1
        if st[3] is None: st[2] = s.QUANTUM; st[3] = 'new'; s.new.append(f)
        if s.n > LIMIT * 10:        # drop from the fattest queue (section 4.1)
            fat = max(s.qs, key=lambda k: sum(x[1] for x in s.qs[k][0])); drop(s.qs[fat][0].
    def dequeue(s, now, drop):
        while s.new or s.old:
            lst = s.new if s.new else s.old
            f = lst[0]; st = s.qs[f]
```

```
            if st[2] <= 0:
                st[2] += s.QUANTUM; lst.popleft(); s.old.append(f); st[3] = 'old'; continue
            before = len(st[0])
            p = st[1].dequeue(st[0], now, drop); s.n -= before - len(st[0])
            if p is None:
                lst.popleft()
                if lst is s.new: s.old.append(f); st[3] = 'old'
                else: st[3] = None
                continue
            st[2] -= p[1]; return p
        return None
    def __len__(s): return s.n

def run(name, qdisc):
    ev, now, seq = [], 0.0, 0
    cwnd = {f: 2.0 for f in range(4)}; inflight = {f: 0 for f in range(4)}; last_cut = {f: -
    delays = {f: [] for f in range(5)}; delivered = {f: 0 for f in range(5)}; drops = [0]
    busy = [False]
    def push(t, kind, data=None):
        nonlocal seq; seq += 1; heapq.heappush(ev, (t, seq, kind, data))
    def drop(p):
        drops[0] += 1
        if p[2] < 4: push(now + BASE_RTT, 'loss', p)     # sender notices about one RTT late
    def send(f, size):
        qdisc.enqueue((now, size, f, seq), drop)
        if f < 4: inflight[f] += 1
        if not busy[0]: push(now, 'tx')
        busy[0] = True
    def pump(f):
        while inflight[f] < int(cwnd[f]): send(f, 1514)
    for f in range(4): push(0.01 * f, 'start', f)
    push(0.5, 'sparse')
    while ev:
        now, _, kind, d = heapq.heappop(ev)
        if now > SIM: break
        if kind == 'start': pump(d)
        elif kind == 'sparse': send(4, 200); push(now + 0.020, 'sparse')
        elif kind == 'tx':
            p = qdisc.dequeue(now, drop)
            if p is None: busy[0] = False; continue
            done = now + p[1] / RATE
            if now > 5.0: delays[p[2]].append(done - p[0]); delivered[p[2]] += p[1]
            if p[2] < 4: push(done + BASE_RTT, 'ack', p)
            push(done, 'tx')
        elif kind == 'ack':
            f = d[2]; inflight[f] -= 1; cwnd[f] += 1 / cwnd[f]; pump(f)
        elif kind == 'loss':
```

```
            f = d[2]; inflight[f] -= 1
            if now - last_cut[f] > BASE_RTT: cwnd[f] = max(2.0, cwnd[f] / 2); last_cut[f] =
            pump(f)
    ms = lambda xs, q: 1000 * statistics.quantiles(xs, n=100)[q - 1]
    bulk = [x for f in range(4) for x in delays[f]]
    tput = sum(delivered[f] for f in range(4)) * 8 / (SIM - 5.0) / 1e6
    share = [round(delivered[f] / sum(delivered[g] for g in range(4)), 2) for f in range(4)]
    print(f"{name:9} sparse p50 {ms(delays[4],50):6.1f} ms  p99 {ms(delays[4],99):6.1f} ms |
          f"bulk p50 {ms(bulk,50):6.1f} ms | bulk {tput:4.2f} Mbit/s shares {share} | drops

print(f"10 Mbit/s bottleneck, 4 bulk flows + 1 sparse flow, {SIM:.0f} s simulated, stats aft
run('fifo', Fifo(False)); run('codel', Fifo(True)); run('fq_codel', FqCodel())
```

```
$ python3 aqm.py
10 Mbit/s bottleneck, 4 bulk flows + 1 sparse flow, 30 s simulated, stats after 5 s
fifo      sparse p50  385.3 ms  p99  511.7 ms | bulk p50  385.3 ms | bulk 9.92 Mbit/s shares
codel     sparse p50   12.4 ms  p99   21.3 ms | bulk p50   13.0 ms | bulk 9.92 Mbit/s shares
fq_codel  sparse p50    0.8 ms  p99    1.4 ms | bulk p50   14.2 ms | bulk 9.92 Mbit/s shares
$ python3 aqm.py 120
10 Mbit/s bottleneck, 4 bulk flows + 1 sparse flow, 120 s simulated, stats after 5 s
fifo      sparse p50  755.6 ms  p99 1048.2 ms | bulk p50  755.2 ms | bulk 9.92 Mbit/s shares
codel     sparse p50   12.7 ms  p99   21.8 ms | bulk p50   13.0 ms | bulk 9.92 Mbit/s shares
fq_codel  sparse p50    0.8 ms  p99    1.4 ms | bulk p50   14.2 ms | bulk 9.92 Mbit/s shares
```

What the numbers say:

- **FIFO never drops, and gets worse with time.** With 0 drops TCP gets no signal, so the queue keeps growing. The sparse flow's median delay rises from 385 ms after 30 s to 756 ms after 120 s, for the same 9.92 Mbit/s of throughput.
- **CoDel holds the queue near target.** About 13 ms (5 ms target, plus the time it takes the four flows to react, plus serialisation) at the same throughput, by dropping roughly one bulk packet in 30.
- **FQ-CoDel isolates.** The bulk flows see the same ~14 ms as under CoDel, but the sparse flow sees **0.8 ms**, because it is almost always on the new-flows list. That is the effect you notice on a call.

This is a model, not a kernel. There is no ECN, no pacing, no GSO, no hash collisions, and a simplified Reno. Treat the ratios as the lesson, not the milliseconds.

Lab 3: configuring it (from the man pages; not run here)

The box has no `tc` and no FQ-CoDel/CAKE in its kernel, so these come from `tc-fq_codel(8)`, `tc-cake(8)` and `systemd.network(5)` and were not run:

```
# What is running, with drop/mark/backlog counters
tc -s qdisc show dev eth0

# Hosts and servers where the NIC is the bottleneck: FQ-CoDel at root
tc qdisc replace dev eth0 root fq_codel

# Router WAN on a 100/20 Mbit/s cable line: shape upload just below the line rate
tc qdisc replace dev wan0 root cake bandwidth 19mbit docsis nat
```

```
# systemd-networkd equivalent (/etc/systemd/network/10-wan.network)
[Match]
Name=wan0

[CAKE]
Parent=root
Bandwidth=19M
NAT=yes
OverheadBytes=18
MPUBytes=64
```

For download, the queue forms at the ISP side, so shaping ingress requires redirecting it to an `ifb` device and attaching CAKE there with the `ingress` keyword.

The trap

- **Managing a queue that never fills.** FQ-CoDel on the WAN NIC of a router with a 1 Gbit/s port in front of a 20 Mbit/s modem does nothing: the queue builds in the modem. Shape below line rate, or the AQM has nothing to manage.
- **Shaping exactly at line rate.** Shape slightly below the measured rate, so the queue moves into Linux and not into the modem. The man pages leave the margin to you, and setting CAKE's overhead keyword correctly matters as much as the number.
- **Believing the sysctl.** `default_qdisc` applies only to qdiscs created after it is set, and only where the kernel has the module. Lab 1 shows a 6.12 kernel without it. Always read `tc -s qdisc`.
- **Tuning `target` for a LAN-only path.** `interval` should be on the order of the worst-case RTT through the bottleneck (`tc-fq_codel(8)`). Lowering it for a datacentre path makes Internet flows back off too hard.
- **Benchmarking with one flow.** Fairness and isolation show up only with several bulk flows plus a latency probe, as in Lab 2. A single `iperf3` stream mostly measures the shaper.

The boring rule

Make sure FQ-CoDel (or `fq` with paced TCP on servers) is actually attached wherever Linux is the bottleneck. On routers in front of slower links, run CAKE (or HTB + FQ-CoDel) with `bandwidth`

just under the real rate and the right overhead keyword, in both directions. Then measure latency **under load**, with several flows, before and after.

Try this

1. In `aqm.py`, raise `LIMIT` to 5000 and run FIFO for 300 s. When do the first drops appear, and what is the sparse delay then?
2. Add an unresponsive bulk flow, a constant 6 Mbit/s that ignores loss. What happens to the Reno flows' shares under CoDel versus FQ-CoDel? This is the case BLUE in CAKE's COBALT is for.
3. Send the sparse flow at 2 Mbit/s instead of 80 kbit/s, so it is no longer sparse. Does it stay on the new-flows list?
4. Change `INTERVAL` to 10 ms with the 20 ms RTT. Watch throughput and drops, and explain them with rule 2.
5. On a Linux machine you control, run `tc -s qdisc` while a large upload is running, and read `dropped`, `ecn_mark` and `new_flows_len`.

Next

- QoS vocabulary: classes, shaping vs policing, and where FQ-CoDel sits as a leaf under HTB.
- SYN floods and listen queues: another queue that fills silently.

Verification

- **Run here** (31 October 2026, Python 3.13.5):
  - `aqm.py` uses only the standard library. Both runs are pasted unedited, and the results are deterministic.
  - `kernel.py` ran under `unshare -rn` with pyroute2 0.9.7 in a user venv, on the box's 6.12 kernel. `default_qdisc` and `ip link` were read live.
- **Not run:** everything in Lab 3. The box has no `tc` binary and its kernel has no `sch_fq_codel`/`sch_cake`. Commands and option names were checked against `tc-fq_codel(8)`, `tc-cake(8)` and `systemd.network(5)`.
- The `fq_codel` defaults (`limit` 10240, `flows` 1024, `quantum` 1514, `target` 5ms, `interval` 100ms, `memory_limit` 32 MB, ECN on) are quoted from `tc-fq_codel(8)`. The systemd default comes from `sysctl.d/50-default.conf` in systemd's repository.

Sources

- RFC 8289 — Controlled Delay Active Queue Management: §3 building blocks, §4 overview, §5 annotated pseudocode

- RFC 8290 — The Flow Queue CoDel Packet Scheduler and Active Queue Management Algorithm: §4.1 enqueue, §4.2 dequeue, §5 implementation considerations
- RFC 7567 — IETF Recommendations Regarding Active Queue Management
- tc-fq_codel(8), tc-cake(8)
- systemd.network(5): `[FairQueueingControlledDelay]` and `[CAKE]` sections
- systemd sysctl.d/50-default.conf
- Linux kernel documentation — net.core.default_qdisc
- Inspiration: @FBorsik on FQ_CoDel / CAKE at Linux Plumbers and WLPC 2026

Overlays and Tunnels

Why Encapsulate

Why Encapsulate

Encapsulation is not a fashion—it is how networks **reuse an underlay** while giving tenants or sites a different topology, address space, or policy domain. This chapter builds durable models before GRE/IPsec/VXLAN/EVPN syntax (Containerlab + FRR + Linux, mid-2026 free path).

Learning goals

By the end of this chapter you can:

- Separate underlay from overlay responsibilities
- List reasons to tunnel or encapsulate (and reasons not to)
- Trace a packet through outer/inner headers
- Predict MTU, ECMP entropy, and troubleshooting failure modes
- Map encapsulation choices to common design problems

The core picture

```
overlay topology (what apps believe)

site A    .......    site B

     encapsulated path

   underlay IP fabric/WAN
```

Plane	Job
Underlay	Deliver outer packets between tunnel endpoints (IP reachability, IGP/BGP, ECMP)
Overlay	Present virtual links, L2 segments, or VRFs to tenants/sites

Plane	Job
Control (overlay)	Who is where (EVPN-class, controller, static maps)
Data (overlay)	Encapsulation format (VXLAN, GRE, Geneve, IPsec…)

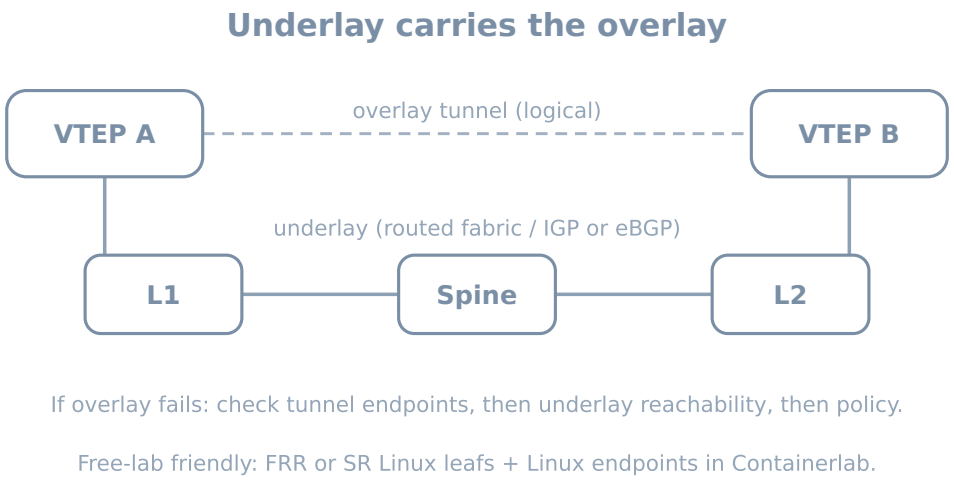


Figure 1: Underlay vs overlay

Why people encapsulate

Driver	What encapsulation buys
Overlap addresses	Two tenants both use 10.0.0.0/8
L2 adjacency over L3	VM mobility, some clusters, legacy apps
Policy domains	Separate routing tables / VRFs
Traffic engineering	Steer logical topology physical
Multi-site stretch	Same segment in two DCs (careful!)
Security boundary	IPsec provides crypto + integrity
Abstraction	Apps ignore underlay renumbering

Why *not* to encapsulate

Temptation	Cost
Tunnel everything by default	MTU tax, state, debug opacity

Temptation	Cost
Stretch L2 globally	Failure domains explode
Overlay without underlay skill	“EVPN down” actually IGP down
Double NAT + tunnel + hairpin	Undebuggable piles

Rule: prefer plain IP routing until a requirement forces an overlay.

Encapsulation as Russian dolls

Example mental stack (VXLAN-ish):

```
[ Eth ][ IP underlay ][ UDP ][ VXLAN ][ Eth inner ][ IP inner ][ TCP ][ payload ]
```

Header	Owned by
Outer IP	Underlay endpoints (VTEPs / tunnel routers)
Outer L4	Often UDP for ECMP entropy (VXLAN)
Overlay header	VNI / keys / flags
Inner frame	Tenant

GRE classic:

```
[ Eth ][ IP ][ GRE ][ IP inner ][ ...]
```

IPsec tunnel mode adds crypto headers and often transports inner IP.

Identifiers you must name

Identifier	Role
Tunnel endpoint IP	Underlay address of VTEP/router
VNI / VPN id / GRE key	Separates segments
VRF	Routing table isolation
Route target / RD (EVPN/MPLS-ish)	Control-plane membership
Mapping	End-host IP/MAC → endpoint

Without a clear mapping system (static, flood-and-learn, EVPN), overlays become amateur radio.

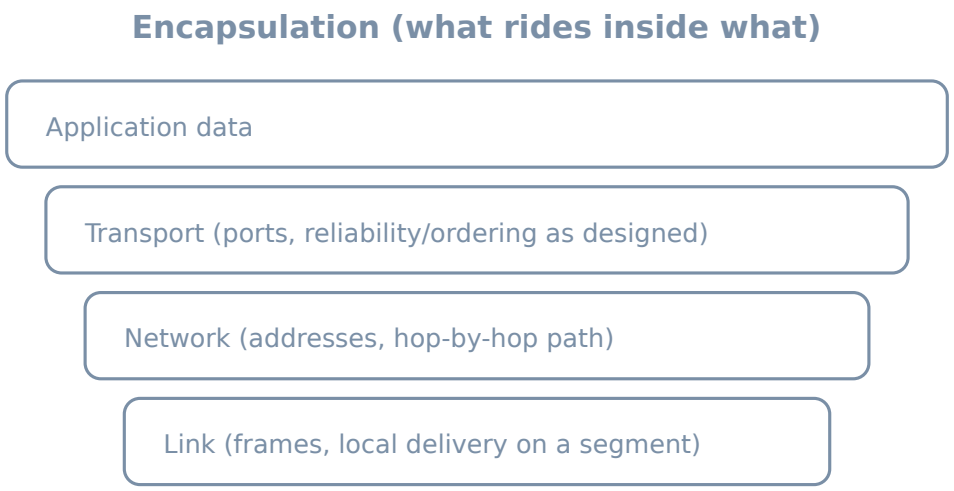


Figure 2: Encapsulation

Failure domains

```
Underlay down → all overlays using it down
One VTEP down → locals on that leaf isolated from fabric overlay
VNI mis-map → blackhole or cross-tenant leak (severe)
MTU blackhole → big packets die, small pings live
```

Always ask: **is this underlay, overlay data, or overlay control?**

MTU tax

Every header steals bytes. If underlay MTU is 1500 and you add ~50–100 bytes, inner needs lower MTU or underlay needs jumbo.

Symptom	Likely
Ping works, TCP hangs	PMTUD broken + encapsulation
Only large transfers fail	MTU
DF set drops	Need clamp or larger underlay

Lab habit: set explicit MTU on overlay interfaces and test large pings.

```
ping -M do -s 1472 10.0.0.1   # classic Ethernet payload probe
# inside overlay, lower size until success; compute header tax
```

ECMP and entropy

Underlays load-balance on outer five-tuple. If outer UDP/TCP ports are fixed, many inner flows hash to **one** path → polarization.

Design	Entropy
VXLAN UDP source port from inner flow hash	Good
GRE without entropy helpers	Often poor
IPsec without cleverness	Can polarize

When debugging "uneven spines," check **outer** headers, not only inner apps.

Control plane vs data plane (again)

	Data plane only	With control plane
Static GRE	Manual who-to-whom	—
Flood-and-learn VXLAN	Dynamic MAC via flood	Limited scale
EVPN-class	Advertises reachability	Mapping distributed

This book's later chapters: site tunnels (mostly static/service), VXLAN data plane, EVPN intro where free images allow.

Mini lab — see encapsulation with tcpdump

```
name: encap-why

topology:
  nodes:
    a:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils tcpdump
    b:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils tcpdump
    under:
      kind: linux
      image: alpine:3.20
```

```
      exec:
        - apk add --no-cache iproute2 iputils tcpdump
        - sysctl -w net.ipv4.ip_forward=1

  links:
    - endpoints: ["a:eth1", "under:eth1"]
    - endpoints: ["b:eth1", "under:eth2"]
```

GRE between a and b across under

```
# addresses: a 10.0.0.1/24, under eth1 10.0.0.254; b 10.0.1.1/24, under eth2 10.0.1.254
# underlay route a b via under

docker exec clab-encap-why-a sh -c '
  ip tunnel add gre1 mode gre remote 10.0.1.1 local 10.0.0.1 ttl 64
  ip link set gre1 up
  ip addr add 172.16.0.1/30 dev gre1
'
docker exec clab-encap-why-b sh -c '
  ip tunnel add gre1 mode gre remote 10.0.0.1 local 10.0.1.1 ttl 64
  ip link set gre1 up
  ip addr add 172.16.0.2/30 dev gre1
'
```

Predict → observe → fix

```
docker exec clab-encap-why-a ping -c 2 172.16.0.2
docker exec clab-encap-why-under tcpdump -ni eth1 -c 5 proto gre
# or: tcpdump -ni eth1 proto 47
```

Predict: underlay capture shows outer IP a→b with GRE; inner ICMP not as plain on underlay without decap.
Observe: header stack.
Fix: if no gre, check tunnel remote/local and underlay ping 10.0.1.1 from a.

Design questions checklist

Before choosing an overlay:

1. What problem is pure IP routing failing to solve?

2. L2 stretch or L3 VPN?

3. Who allocates VNIs / VRFs?
4. What is the underlay routing design?
5. MTU strategy?
6. How do we debug (mirror, drop counters, inner/outer captures)?
7. Failure domain acceptable?

Mapping to the journey map

```
Models (planes, encap)
  → underlay skill (L2/L3/IGP/BGP)
  → edge services
  → overlays (this part)
  → fabrics (scale the underlay + overlay)
```

Skipping underlay competence makes overlay chapters mystical.

Common mistakes

Mistake	Symptom
Troubleshooting only inner	Miss underlay loss
Ignoring MTU	Mystery TCP issues
One giant L2 VNI	Broadcast storms / failure blast
No inventory of VNIs	Collisions and leaks
Encrypt + fragment chaos	Performance cliff

Summary

- Encapsulation reuses underlays to build different logical networks
- Always name **underlay, overlay data, overlay control**, and **identifiers**
- MTU and ECMP entropy are not footnotes
- Prefer routing until requirements force tunnels
- A tiny GRE lab + tcpdump makes the model real

Next: **site tunnels**—practical GRE/IPsec-style site-to-site patterns on free Linux/FRR labs.

Site Tunnels

Site Tunnels

Site-to-site tunnels connect islands of private addressing across a routed underlay (lab "Internet" or WAN). This chapter implements **GRE and policy-based / route-based IPsec-style** patterns with Linux in Containerlab, optionally fronted by FRR for routing (mid-2026 free tools).

Learning goals

By the end of this chapter you can:

- Build a GRE tunnel and route site prefixes across it
- Explain route-based vs policy-based VPN approaches
- Run a simple WireGuard or LibreSwan/strongSwan-style lab path (tool availability permitting)
- Fail underlay links and predict overlay behavior
- Clamp MTU and verify with large pings and tcpdump

When site tunnels are the right tool

Use	Not the first tool for
Two branches over untrusted WAN	DC fabric east-west (use fabric overlay)
Quick lab multi-site	Massive multi-tenant scale (EVPN fabrics)
Crypto requirement	Cleartext GRE alone on hostile nets

Topology — two sites + “Internet”

```
h1--siteA--\                /--siteB--h2
            +---- inet ----+
```

```yaml
name: site-tun

topology:
  nodes:
    sitea:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils tcpdump iptables wireguard-tools || true
    siteb:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils tcpdump iptables wireguard-tools || true
    inet:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils tcpdump
        - sysctl -w net.ipv4.ip_forward=1
    h1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
    h2:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils

  links:
    - endpoints: ["h1:eth1", "sitea:eth1"]
    - endpoints: ["h2:eth1", "siteb:eth1"]
    - endpoints: ["sitea:eth2", "inet:eth1"]
    - endpoints: ["siteb:eth2", "inet:eth2"]
```

Addressing plan

Object	Address
Site A LAN	192.168.10.0/24 (sitea .1, h1 .10)
Site B LAN	192.168.20.0/24 (siteb .1, h2 .10)
sitea WAN	198.51.100.1/30
inet eth1	198.51.100.2/30
siteb WAN	203.0.113.1/30
inet eth2	203.0.113.2/30
GRE outer	WAN IPs
GRE inner	10.255.255.0/30 (.1 a, .2 b)

```
# site A
docker exec clab-site-tun-sitea sh -c '
  sysctl -w net.ipv4.ip_forward=1
  ip addr add 192.168.10.1/24 dev eth1; ip link set eth1 up
  ip addr add 198.51.100.1/30 dev eth2; ip link set eth2 up
  ip route add 203.0.113.0/30 via 198.51.100.2
'
docker exec clab-site-tun-h1 sh -c '
  ip addr add 192.168.10.10/24 dev eth1; ip link set eth1 up
  ip route add default via 192.168.10.1
'
# site B + h2 analogous with 192.168.20.0/24 and 203.0.113.1
docker exec clab-site-tun-inet sh -c '
  ip addr add 198.51.100.2/30 dev eth1; ip link set eth1 up
  ip addr add 203.0.113.2/30 dev eth2; ip link set eth2 up
'
```

Lab 1 — GRE + static routes

```
docker exec clab-site-tun-sitea sh -c '
  ip tunnel add gre1 mode gre remote 203.0.113.1 local 198.51.100.1 ttl 64
  ip link set gre1 up mtu 1400
  ip addr add 10.255.255.1/30 dev gre1
  ip route add 192.168.20.0/24 dev gre1
'
docker exec clab-site-tun-siteb sh -c '
  ip tunnel add gre1 mode gre remote 198.51.100.1 local 203.0.113.1 ttl 64
  ip link set gre1 up mtu 1400
  ip addr add 10.255.255.2/30 dev gre1
  ip route add 192.168.10.0/24 dev gre1
'
```

Predict

- Underlay: sitea pings 203.0.113.1
- Overlay: h1 pings h2
- tcpdump on inet shows GRE (proto 47), not plain inner ICMP as clear tenant IP without decap context

Observe

```
docker exec clab-site-tun-sitea ping -c 2 203.0.113.1
docker exec clab-site-tun-h1 ping -c 3 192.168.20.10
docker exec clab-site-tun-inet tcpdump -ni eth1 -c 8 proto 47
docker exec clab-site-tun-sitea ip route get 192.168.20.10
```

Fix

Symptom	Check
No underlay	inet addrs, routes
Underlay OK overlay fail	gre remote/local swap, routes to LAN via gre
One-way	missing return route

Lab 2 — MTU blackhole drill

```
docker exec clab-site-tun-h1 ping -c 2 -M do -s 1472 192.168.20.10
docker exec clab-site-tun-h1 ping -c 2 -M do -s 1372 192.168.20.10
```

Predict

Large DF pings fail; smaller succeed if gre mtu 1400.
Fix: lower inner interface MTU, TCP MSS clamp, or raise underlay MTU.

```
docker exec clab-site-tun-sitea iptables -t mangle -A FORWARD -p tcp --tcp-flags SYN,RST SYN
```

Lab 3 — Underlay failure

```
docker exec clab-site-tun-inet ip link set eth2 down
docker exec clab-site-tun-h1 ping -c 5 192.168.20.10
docker exec clab-site-tun-inet ip link set eth2 up
```

Predict

Overlay dies with underlay; no magic. Multi-homed underlay would need dual tunnels + routing.

Route-based vs policy-based VPN

Model	Idea	Lab analogue
Route-based	Tunnel is an interface; RIB steers prefixes into it	GRE, WireGuard `wg0`, VTI
Policy-based	SPD selects traffic by ACL selectors for crypto	classic "interesting traffic" IPsec

Route-based composes better with dynamic routing (run OSPF/BGP over tunnel—carefully).

Lab 4 — WireGuard route-based (if packages available)

```
# Generate keys on both sides (persist in lab folder for real work)
docker exec clab-site-tun-sitea sh -c '
  apk add --no-cache wireguard-tools
  wg genkey | tee /tmp/a.key | wg pubkey > /tmp/a.pub
'
# exchange pubs out of band in lab notes
```

```
docker exec clab-site-tun-sitea sh -c '
  ip link add wg0 type wireguard
  ip addr add 10.255.255.1/30 dev wg0
  wg set wg0 private-key /tmp/a.key listen-port 51820 \
    peer B_PUBLIC_KEY endpoint 203.0.113.1:51820 allowed-ips 192.168.20.0/24,10.255.255.2/32
  ip link set wg0 up
  ip route add 192.168.20.0/24 dev wg0
'
```

Mirror on siteb with `allowed-ips 192.168.10.0/24`. Open UDP 51820 on path (inet forwards by default).

Predict → observe

```
docker exec clab-site-tun-h1 ping -c 3 192.168.20.10
docker exec clab-site-tun-inet tcpdump -ni eth1 -c 5 udp port 51820
docker exec clab-site-tun-sitea wg show
```

If `wireguard-tools` or kernel module missing in container, treat as **optional**—GRE still teaches site tunnels. Host-level WireGuard between namespaces is an alternate craft path from lab-craft chapters.

FRR over GRE (dynamic optional)

Enable `zebra` + `ospfd` or `bgpd` on FRR images replacing Alpine sites:

```
interface gre1
 ip ospf area 0
router ospf
 ospf router-id 1.1.1.1
```

Predict: LAN prefixes appear via OSPF over tunnel.
Risk: if underlay also runs OSPF without careful design, you can create recursive routing (tunnel endpoint learned *via* tunnel). **Keep underlay and overlay routing domains separate.**

Policy-based IPsec awareness

```
Selectors: src 192.168.10.0/24 dst 192.168.20.0/24
Peer: 203.0.113.1
```

Tools: strongSwan, LibreSwan—configs are verbose; for this book, understand selectors + SA state:

```
# illustrative
ip xfrm state
ip xfrm policy
```

Debug with state/policy tables + outer ESP captures (proto 50).

Security notes (vendor-agnostic)

- GRE alone is **not** confidential
- Authenticate/encrypt on untrusted underlays
- Filter what enters the tunnel (no free transit for world)
- Manage keys as secrets (not committed plaintext in public repos)

verify.sh sketch

```
#!/usr/bin/env bash
set -euo pipefail
docker exec clab-site-tun-h1 ping -c2 -W1 192.168.20.10
docker exec clab-site-tun-h2 ping -c2 -W1 192.168.10.10
echo OK
```

Common mistakes

Mistake	Symptom
Tunnel up, no LAN routes	Only gre peer pings
Recursive routing	Flapping tunnel endpoint
MTU ignored	App failures
NAT on underlay without keepalive	Idle VPN dies
Asymmetric interesting traffic	Policy VPN one-way

Summary

- Site tunnels extend private sites across a routed underlay
- GRE is the clearest teaching tool; WireGuard is a modern route-based crypto option
- Separate underlay reachability from overlay routing
- Always test **MTU** and **underlay failure**
- Prefer route-based models when combining with dynamic routing

Next: **VXLAN data plane**—L2 segments over L3 underlays with VNI encapsulation.

WireGuard: Cryptokey Routing and the Pre-Shared Key

WireGuard: Cryptokey Routing and the Pre-Shared Key

The site-tunnel chapter used WireGuard as one route-based VPN option among several. This chapter opens it up. WireGuard has two ideas you must understand before you run it in production. **Cryptokey routing** means that a peer's `AllowedIPs` list is both its routing table and its source-address filter. The optional **pre-shared key (PSK)** is a 32-byte symmetric secret mixed into every handshake. In 2026 the PSK is getting attention as a cheap **post-quantum hedge**: VPN providers now generate it by default for router and native-client configs. Both ideas are easy to misconfigure, and both fail *silently*. This chapter makes them fail on purpose in a two-namespace lab and reads the evidence.

Learning goals

By the end of this chapter you can:

- Build a WireGuard tunnel between two Linux network namespaces with `wg` and `ip`, without config files
- Explain why `AllowedIPs` drops a correctly encrypted packet and where that drop is counted
- Recognise WireGuard's "no peer for this destination" behaviour
- Add a PSK, explain exactly what it does and does not protect against, and rotate it live
- Diagnose a PSK mismatch from a packet capture (both handshake messages are on the wire, yet the tunnel never comes up)

Lab platform: Debian 13, Linux 6.12, `wireguard-tools` v1.0.20210914 (the current Debian package), `iproute2`, `tcpdump` 4.99.5. All commands need root. Every output below is pasted from a run.

Concepts in one page



Direction	What `AllowedIPs` does
Outbound (a packet leaves via `wg0`)	Longest-prefix match of the *destination* against every peer's AllowedIPs picks the peer, and so the key and endpoint. No match means the packet is not sent, and the sender gets an error.
Inbound (a packet arrives and decrypts)	The decrypted inner *source* must fall inside the AllowedIPs of the peer whose key decrypted it. Otherwise it is dropped.

The kernel route only gets a packet *to* `wg0`. AllowedIPs decides *which peer*, if any.

The handshake is Noise **`IKpsk2`** (Curve25519, ChaCha20-Poly1305, BLAKE2s). There are two messages. The **initiation** (type 1, 148 bytes of UDP payload) carries the initiator's encrypted static key and a timestamp. The **response** (type 2, 92 bytes) carries the responder's ephemeral key and an empty AEAD box. The PSK is hashed into the chaining key while the **response** is being built. Without a PSK the protocol uses 32 zero bytes in its place.

What the PSK buys: if Curve25519 is broken later (the "harvest now, decrypt later" scenario with a large quantum computer), traffic recorded today stays confidential, *provided the PSK itself was never exposed*. What it does not buy: it is not a post-quantum key exchange, it does not replace the public keys, and a PSK that was sent over a classical channel (for example a TLS session protected by ECDHE) is only as strong as that channel. Protocols such as Rosenpass run a post-quantum key exchange alongside WireGuard and feed the result into the PSK slot, rotating it. That is the direction to take when this matters.

Lab build

```
#!/usr/bin/env bash
# lab-up.sh - two namespaces, a veth "internet", and a WireGuard tunnel between them
set -euo pipefail
ip netns add siteA; ip netns add siteB
ip link add vA netns siteA type veth peer name vB netns siteB
ip -n siteA addr add 198.51.100.1/24 dev vA; ip -n siteA link set vA up; ip -n siteA link se
ip -n siteB addr add 198.51.100.2/24 dev vB; ip -n siteB link set vB up; ip -n siteB link se

umask 077
for s in A B; do wg genkey | tee $s.key | wg pubkey > $s.pub; done
wg genpsk > ab.psk

# create wg0 inside each namespace: a WireGuard interface sends its UDP from the namespace i
ip -n siteA link add wg0 type wireguard
ip -n siteB link add wg0 type wireguard
ip netns exec siteA wg set wg0 private-key A.key listen-port 51820 \
   peer "$(cat B.pub)" preshared-key ab.psk endpoint 198.51.100.2:51820 allowed-ips 10.0.0.2
ip netns exec siteB wg set wg0 private-key B.key listen-port 51820 \
   peer "$(cat A.pub)" preshared-key ab.psk endpoint 198.51.100.1:51820 allowed-ips 10.0.0.1
ip -n siteA addr add 10.0.0.1/24 dev wg0; ip -n siteA link set wg0 up
```

```
ip -n siteB addr add 10.0.0.2/24 dev wg0; ip -n siteB link set wg0 up
# siteB also owns a LAN prefix that siteA routes into the tunnel
ip -n siteB addr add 192.0.2.1/24 dev lo
ip -n siteA route add 192.0.2.0/24 dev wg0
echo "lab up"
```

```
#!/usr/bin/env bash
# lab-down.sh - remove everything lab-up.sh created
ip netns del siteA 2>/dev/null; ip netns del siteB 2>/dev/null
rm -f A.key A.pub B.key B.pub ab.psk wrong.psk
echo "lab down"
```

`wg genpsk` produces 32 random bytes in base64, the same format as a private key. The PSK is configured **per peer** and must be identical on both ends. `umask 077` keeps the key files private. In production these values live in `/etc/wireguard/wg0.conf` (mode 600) or in your configuration management tool's secret store, never in a world-readable file.

```
$ sudo ./lab-up.sh
lab up
$ sudo ip netns exec siteA ping -c 2 10.0.0.2 | tail -2
2 packets transmitted, 2 received, 0% packet loss, time 1015ms
rtt min/avg/max/mdev = 0.547/0.590/0.633/0.043 ms
$ sudo ip netns exec siteA wg show
interface: wg0
  public key: SfKQSp7pB2JRf6CnHF048gHOIul0sUi0k9lCH/V9P2I=
  private key: (hidden)
  listening port: 51820

peer: SVXjcMNZVP6LGPEc+CYMNGGQ0/4KHY05bo34Oc5MuVg=
  preshared key: (hidden)
  endpoint: 198.51.100.2:51820
  allowed ips: 10.0.0.2/32, 192.0.2.0/24
  latest handshake: 1 second ago
  transfer: 348 B received, 404 B sent
$ sudo ip -n siteA link show wg0
3: wg0: <POINTOPOINT,NOARP,UP,LOWER_UP> mtu 1420 qdisc noqueue state UNKNOWN mode DEFAULT gr
```

The handshake completed on the first ping (`latest handshake: 1 second ago`), and `wg show` reports the PSK as `(hidden)`. Its presence is visible and its value is not. The interface MTU is **1420** by default. That leaves room for the outer IPv6 header (40), UDP (8), and WireGuard's 32 bytes of data-message overhead (a 16-byte header plus a 16-byte Poly1305 tag) inside a 1500-byte path. Over IPv4 there are 20 bytes to spare.

Drill 1: cryptokey routing drops a perfectly encrypted packet

```
#!/usr/bin/env bash
# drill-cryptokey.sh - AllowedIPs is a routing table outbound and a source filter inbound
set -euo pipefail
ip -n siteA addr add 203.0.113.5/32 dev lo           # an address siteB never allowed for th
echo "== inside the allowed source (10.0.0.1 -> 10.0.0.2)"
ip netns exec siteA ping -c 2 -W 1 10.0.0.2 | tail -2
echo "== outside it (203.0.113.5 -> 10.0.0.2)"
ip netns exec siteA ping -c 3 -W 1 -I 203.0.113.5 10.0.0.2 | tail -2 || true
echo "== siteB wg0 counters (decrypted fine, then dropped: src not in peer's allowed-ips)"
ip -n siteB -s link show wg0 | sed -n '3,4p'
echo "== a route into wg0 that no peer's allowed-ips covers"
ip -n siteA route add 198.18.0.0/15 dev wg0
ip netns exec siteA ping -c 1 -W 1 198.18.0.1 2>&1 | head -2 || true
```

```
$ sudo ./drill-cryptokey.sh
== inside the allowed source (10.0.0.1 -> 10.0.0.2)
2 packets transmitted, 2 received, 0% packet loss, time 1028ms
rtt min/avg/max/mdev = 0.138/0.334/0.531/0.196 ms
== outside it (203.0.113.5 -> 10.0.0.2)
3 packets transmitted, 0 received, 100% packet loss, time 2040ms

== siteB wg0 counters (decrypted fine, then dropped: src not in peer's allowed-ips)
    RX:  bytes packets errors dropped  missed   mcast
           660       5      3       0       0       0
== a route into wg0 that no peer's allowed-ips covers
PING 198.18.0.1 (198.18.0.1) 56(84) bytes of data.
From 10.0.0.1 icmp_seq=1 Destination Host Unreachable
```

Read it in order:

1. From the allowed source, the ping works.
2. From `203.0.113.5`, siteA's kernel still routes the packet into `wg0`, because the destination `10.0.0.2` matches peer B's AllowedIPs. WireGuard encrypts it and siteB **decrypts it successfully**. Then siteB checks the inner source against peer A's AllowedIPs (`10.0.0.1/32`), finds no match, and drops it. The only trace is the **RX `errors`** counter on siteB's `wg0` (3 packets, one per ping). Nothing is logged, and no ICMP message goes back.
3. A destination that the kernel routes into `wg0` but that no peer allows (`198.18.0.0/15`) is never sent. WireGuard rejects it at transmit, and the sender sees `Destination Host Unreachable`.

That makes AllowedIPs a security boundary as well as a convenience. A peer that holds a valid key still cannot inject traffic from addresses it was not given. The flip side is that every new subnet behind a peer means editing AllowedIPs on **both** sides: the outbound list on one end, the inbound filter on the other.

Drill 2: a PSK mismatch, seen from the wire

```
#!/usr/bin/env bash
# drill-psk-mismatch.sh - same keypairs, different PSK on each side: the handshake never com
set -euo pipefail
umask 077; wg genpsk > wrong.psk
# re-adding the peer throws away its current session, forcing a new handshake
for ns in siteA siteB; do
  peer=$(ip netns exec $ns wg show wg0 peers)
  ep=$(ip netns exec $ns wg show wg0 endpoints | cut -f2)
  aips=$(ip netns exec $ns wg show wg0 allowed-ips | cut -f2 | tr ' ' ',')
  psk=ab.psk; [ $ns = siteB ] && psk=wrong.psk
  ip netns exec $ns wg set wg0 peer "$peer" remove
  ip netns exec $ns wg set wg0 peer "$peer" preshared-key $psk endpoint "$ep" allowed-ips "$
done
ip netns exec siteB timeout 8 tcpdump -lnni vB -c 4 udp port 51820 2>/dev/null &
sleep 1
ip netns exec siteA ping -c 3 -W 1 10.0.0.2 | tail -2 || true
wait
echo "siteA latest-handshake (0 = never):"; ip netns exec siteA wg show wg0 latest-handshake
echo "siteB latest-handshake (0 = never):"; ip netns exec siteB wg show wg0 latest-handshake
```

```
$ sudo ./drill-psk-mismatch.sh
03:40:05.224460 IP 198.51.100.1.51820 > 198.51.100.2.51820: UDP, length 148
03:40:05.224668 IP 198.51.100.2.51820 > 198.51.100.1.51820: UDP, length 92
3 packets transmitted, 0 received, 100% packet loss, time 2049ms

03:40:10.317113 IP 198.51.100.1.51820 > 198.51.100.2.51820: UDP, length 148
03:40:10.317315 IP 198.51.100.2.51820 > 198.51.100.1.51820: UDP, length 92
siteA latest-handshake (0 = never):
SVXjcMNZVP6LGPEc+CYMNGGQO/4KHYO5bo34Oc5MuVg=    0
siteB latest-handshake (0 = never):
SfKQSp7pB2JRf6CnHFO48gHOIul0sUiOk9lCH/V9P2I=    0
```

This is the most confusing failure in WireGuard, and the capture explains it:

- The **148-byte initiation** goes out, and siteB **accepts** it. The initiation is authenticated with the static keys and does not depend on the PSK, so siteB has no reason to be silent.
- siteB answers with a **92-byte response**. It mixes *its* PSK into the chaining key while building that response, so the empty AEAD box inside is sealed under a key that siteA cannot derive.
- siteA fails to open the response and discards it without logging. About five seconds later (`REKEY_TIMEOUT` plus jitter) it tries again. Note the timestamps.
- Neither side records a handshake (`0` means never).

So "packets in both directions on UDP 51820, tunnel down" points to a **PSK mismatch** (or a corrupted PSK). A wrong *public* key looks different: the responder cannot authenticate the initiation, so it sends nothing back, and you see only the 148-byte packets.

Drill 3: fix and rotate the PSK live

```
#!/usr/bin/env bash
# drill-psk-fix.sh - put the right PSK back on siteB (live, no restart) and watch the tunnel
set -euo pipefail
peer=$(ip netns exec siteB wg show wg0 peers)
ip netns exec siteB wg set wg0 peer "$peer" preshared-key ab.psk
ip netns exec siteA ping -c 8 -W 1 10.0.0.2 | tail -2
now=$(date +%s)
for ns in siteA siteB; do
  echo "$ns latest handshake: $(( now - $(ip netns exec $ns wg show wg0 latest-handshakes |
done
```

```
$ sudo ./drill-psk-fix.sh
8 packets transmitted, 8 received, 0% packet loss, time 7114ms
rtt min/avg/max/mdev = 0.268/1859.476/5006.915/1836.351 ms, pipe 5
siteA latest handshake: 2 s ago
siteB latest handshake: 2 s ago
$ sudo ./lab-down.sh
lab down
```

`wg set ... preshared-key` changes the PSK on a running interface. Existing sessions keep their keys until the next handshake, and new handshakes use the new PSK. Here the first ping waited for siteA's next retry (the 5-second maximum RTT), and after that the tunnel ran normally. To rotate a PSK in production, change both ends inside one rekey window (sessions rekey about every two minutes) so that at most one handshake attempt fails. Or let a key-exchange daemon do it for you.

Deploying it for real

- **Generate a separate PSK per peer pair** with `wg genpsk`. One PSK shared across a whole fleet becomes a single secret whose leak removes the hedge for everyone.
- **Distribute it like a private key**, over a channel you would trust in 2040: out of band, through a secret store, or from a post-quantum key exchange. A PSK pasted into a TLS-protected web form is protected only by that TLS session.
- **Store it with the private key**: `PresharedKey = ...` under `[Peer]` in `wg0.conf` (mode 600), or `PresharedKeyFile=` in a systemd-networkd `.netdev`.
- **Monitor the handshake age.** `wg show wg0 latest-handshakes` returns epoch seconds. Alert when the value is `0` or older than about three minutes for a peer that should be up.
- **Watch RX errors on `wg0`.** A rising count usually means AllowedIPs is filtering traffic someone expects to pass.

Verification checklist

- ☐ `wg show` lists `preshared key: (hidden)` on **both** ends for every peer that should have one
- ☐ `latest-handshakes` is recent (non-zero and under about 3 minutes) on both ends
- ☐ Capture shows both 148-byte initiations and 92-byte responses, and data packets follow
- ☐ Each remote prefix appears in the sender's AllowedIPs *and* in the receiver's AllowedIPs for the return path
- ☐ `ip -s link show wg0` RX errors are flat during normal traffic
- ☐ Large pings at your path MTU (`ping -M do -s 1392` over IPv4 with MTU 1420) succeed

Common mistakes

Mistake	Symptom	Fix
PSK set on one side only, or different on each side	Initiations and responses on the wire, no handshake	Same `wg genpsk` value on both peers
Subnet added to the route but not to AllowedIPs	`Destination Host Unreachable` from the local `wg0`	Add the prefix to the peer's AllowedIPs
Subnet added on the sender, not on the receiver	Silent loss, RX errors on the far `wg0`	Mirror AllowedIPs for the return direction
Overlapping AllowedIPs across peers	Traffic goes to the "wrong" peer (longest prefix wins, and a later `wg set` moves the prefix)	One owner per prefix; review `wg show allowed-ips`
Calling the PSK "post-quantum encryption"	False confidence	It is a symmetric hedge for recorded traffic. Distribute it safely or use a PQ key exchange
One fleet-wide PSK	One leak removes the hedge everywhere	One PSK per peer pair

Summary

WireGuard's configuration is small because AllowedIPs does two jobs: it chooses the peer on the way out and filters the source on the way in, and both outcomes are silent unless you read `wg show` and the interface counters. The PSK adds a symmetric secret to the Noise `IKpsk2` handshake. That protects recorded traffic against a future break of Curve25519, as long as the PSK itself stays secret. A PSK mismatch has a distinctive signature on the wire: both handshake messages appear and no session forms, because the responder cannot detect the mismatch and the initiator cannot open its reply. Rotate PSKs per peer pair, distribute them like private keys, and monitor handshake age.

Sources

- WireGuard, "Protocol & Cryptography": https://www.wireguard.com/protocol/. Noise `IKpsk2`, message layouts (148 and 92 bytes), where the PSK is mixed in, timers (`REKEY_TIMEOUT`).
- J. A. Donenfeld, "WireGuard: Next Generation Kernel Network Tunnel", NDSS 2017 (whitepaper): https://www.wireguard.com/papers/wireguard.pdf. Cryptokey routing, PSK as a post-quantum hedge.
- `wg(8)` manual page (`genpsk`, `set ... preshared-key`, `show ... latest-handshakes`): https://git.zx2c4.com/wireguard-tools/about/src/man/wg.8.
- `systemd.netdev(5)`, `[WireGuardPeer] PresharedKeyFile=`: https://www.freedesktop.org/software/systemd/man/latest/systemd.netdev.html.
- Rosenpass (post-quantum key exchange that supplies WireGuard's PSK): https://rosenpass.eu/.

OpenVPN Data Channel Offload (DCO)

OpenVPN Data Channel Offload (DCO)

Thesis

Classic OpenVPN runs the **control plane and the data plane in userspace**. Every encrypted packet crosses the kernel userspace boundary twice. That copy-and-context-switch tax is why OpenVPN used to lose throughput races against kernel-native VPNs. **Data Channel Offload (DCO)** moves encryption, decryption, and encapsulation into a Linux kernel module (`ovpn` since Linux **6.16**; earlier out-of-tree `ovpn-dco` / `ovpn-dco-v2`). Userspace OpenVPN keeps TLS, authentication, and option negotiation; the kernel keeps the hot path. The security model does not change — only *where* the ciphertext is processed. On modern hardware, published UDP benchmarks regularly show roughly **2×** legacy OpenVPN throughput, and independent tests put DCO in the same ballpark as other high-performance VPN designs.

Mental model

```
Before DCO (userspace data path)          With DCO (kernel data path)

NIC    kernel    openvpn(userspace)     NIC    kernel ovpn module
                encrypt / decrypt                        encrypt / decrypt
                encapsulate                              encapsulate

       kernel    NIC                                 NIC (no userspace hop)

Control plane (TLS, auth, PUSH) stays in userspace either way.
```

Piece	Role
Control plane	TLS handshake, certificates, `--push` / NCP, peer lifecycle — always userspace
Data plane	AEAD encrypt/decrypt, tunnel encapsulation, peer forwarding — kernel with DCO
`ovpn` module	In-tree since Linux **6.16**; needs **OpenVPN 2.7+**
`ovpn-dco` / backports	Out-of-tree path for kernels < 6.16 or OpenVPN **2.6.x**
`--disable-dco`	Force the legacy userspace data path (benchmarks, unsupported options)

Cipher and topology constraints (by design). DCO is **L3 / `dev tun` only**. Supported data-channel ciphers are **AES-GCM-128/192/256** and **ChaCha20-Poly1305**. Compression framing is out; servers use **`--topology subnet`**. Older CBC-only peers force a non-DCO fallback. Routing to clients uses the **host routing table** (a client VPN IP as nexthop) rather than OpenVPN's internal iroute machinery — see the official `README.dco.md`.

Version pairing that matters in 2026:

Kernel / userspace	Module name	Notes
Linux 6.16 + OpenVPN 2.7	`ovpn` (in-tree)	Preferred; no DKMS on stock distro kernels
Older kernel or OpenVPN 2.6	`ovpn-dco` via ovpn-backports / distro DKMS	Still fine; package names differ by distro
Windows	`ovpn-dco-win`	Ships with official Windows builds since 2.6; multi-core workers improved further in 2026

Worked examples

Case 1: Confirm the kernel side before you touch configs

On a host that already runs Linux 6.16+ with OpenVPN 2.7+, you do not install a third-party module. Check what you have:

```
# dco-probe.sh - inventory for OpenVPN DCO (read-only; no packages installed here)
uname -r
modinfo ovpn 2>/dev/null | head -5 || echo "ovpn module metadata not present"
lsmod | grep -E '^ovpn' || echo "ovpn not loaded yet (normal until openvpn starts)"
openvpn --version 2>/dev/null | head -3 || echo "openvpn binary not on PATH"
```

Expected shape on a DCO-ready box (versions will differ):

```
$ uname -r
6.16.3-1-amd64
$ modinfo ovpn | head -3
filename:       /lib/modules/6.16.3-1-amd64/kernel/drivers/net/ovpn/ovpn.ko
description:    OpenVPN data channel offload
license:        GPL
```

If `modinfo ovpn` is missing on an older kernel, use the ovpn-backports project or your distro's `ovpn-dkms` / `kmod-ovpn` package — **not** covered as an install lab in this chapter (no package installs in the book environment).

Case 2: Let OpenVPN auto-enable DCO (and force it off for A/B)

OpenVPN **auto-detects** DCO when the module is present and the config is compatible. You only need an explicit flag to *disable* it.

```
# Minimal server sketch - AEAD + subnet topology so DCO can engage
# openvpn-server-dco.conf
port 1194
proto udp
dev tun
topology subnet
server 10.8.0.0 255.255.255.0
dh none
tls-groups X25519:secp256r1
data-ciphers AES-256-GCM:AES-128-GCM:CHACHA20-POLY1305
persist-key
keepalive 10 60
verb 3
# certificates omitted - use your existing PKI
```

```
# Run with DCO (default when available):
#   openvpn --config openvpn-server-dco.conf
# Force userspace data path for a throughput comparison:
#   openvpn --config openvpn-server-dco.conf --disable-dco
```

Log lines to look for:

Message	Meaning
DCO / `ovpn` enabled (wording varies by version)	Kernel data path is live
`Note: Kernel support for ovpn-dco missing, disabling data channel offload.`	Module absent or userspace too old for in-tree `ovpn`
Warning that an option disabled DCO	Config used a feature DCO cannot implement (compression, unsupported cipher, …)

Antonio Quartulli's workstation numbers (Ryzen 9 9950X, 25 Gbps NIC, iperf3 UDP, published via the OpenVPN DCO guide, Feb 2026): legacy userspace **3.61 Gbps**; DCO **7.15 Gbps** — about **2×** on the same box. Third-party reports (IPFire, consumer VPN vendors, router vendors) show similar order-of-magnitude gains; treat any single percentage as hardware-specific.

Case 3: One-sided DCO still helps

You do **not** need DCO on both ends for a win. A DCO server alone cuts server CPU and raises aggregate capacity; both ends DCO maximizes the path. Clients that speak OpenVPN 2.6+/Connect 3.4+ with AEAD are enough for the control plane to negotiate a DCO-capable data channel when the server supports it.

```
laptop (no DCO)  TLS   gateway (DCO / ovpn)

        ciphertext still         encrypt/decrypt + forward
        encrypted tunnel     stays in kernel on the gateway
```

Case 4: Routing mental shift under DCO

With DCO, "iroutes" become ordinary host routes whose nexthop is the client's VPN address. Client-to-client is no longer a special userspace hairpin: packets hit the tunnel interface and the **system routing table** decides the next peer.

```
# Example: send 192.0.2.0/24 to the client that owns 10.8.0.5
# ip route add 192.0.2.0/24 via 10.8.0.5
```

If you still need a blackhole when that client is offline, add an explicit companion route that only exists while the peer is connected — DCO will not invent OpenVPN's old internal routing table for you.

The trap

1. **CBC or compression "just for compatibility".** A single `--data-ciphers` entry that forces CBC, or leftover `--compress`, silently (or with a warning) drops you back to userspace. You keep the old config working and wonder why the benchmark looks unchanged.
2. **OpenVPN 2.6 + in-tree `ovpn`.** The Linux 6.16 `ovpn` module speaks the **2.7** netlink API. On 2.6 you still want the older `ovpn-dco` out-of-tree module (or upgrade userspace). Mixing them yields "kernel support missing" even though *a* DCO module is loaded.
3. **Assuming DCO changes cryptography.** It does not. TLS, certificates, and cipher *selection* stay in userspace; DCO only relocates AEAD packet processing. Threat modeling stays the same; capacity planning does not.
4. **Comparing protocols by blog headlines alone.** Marketing posts crown a winner every quarter. Measure *your* NIC, CPU crypto offload, and MTU. DCO removes OpenVPN's historical userspace tax; it does not erase the need for sound AEAD choice, MSS clamping, and CPU affinity on multi-gigabit paths.

The boring rule

Ship **OpenVPN 2.7** on **Linux 6.16** with **AES-GCM or ChaCha20-Poly1305**, `--topology subnet`, and no compression. Let DCO auto-enable. Keep `--disable-dco` in your lab toolkit for A/B tests. On older kernels, use the matching out-of-tree module — do not assume the in-tree `ovpn` name. Verify with `modinfo` / logs before you trust a speed claim.

Try this

1. On a 6.16+ lab host, run `modinfo ovpn` and `openvpn --version`. Confirm the 2.7 + `ovpn` pairing.
2. Bring up a two-namespace UDP tunnel (reuse your existing PKI). Capture a short iperf3 run with DCO on, then the same run with `--disable-dco`. Record throughput and `top` CPU for `openvpn`.
3. Intentionally add a DCO-incompatible option (e.g. force a CBC cipher). Confirm the log shows DCO disabled, then remove it.
4. Add a host route `via` a client VPN IP and traceroute from the server VRF/namespace to prove DCO is using the system FIB.

Sources

- OpenVPN DCO: The Definitive Guide (OpenVPN blog; updated 2026-10-08)
- OpenVPN `README.dco.md` (upstream limitations and enablement)
- Community wiki: DataChannelOffload
- ovpn-backports (out-of-tree `ovpn` for older kernels)

WireGuard Multi-Tunnel: AllowedIPs, Policy Routing, Kill Switch

WireGuard Multi-Tunnel: AllowedIPs, Policy Routing, Kill Switch

The cryptokey-routing chapter showed one tunnel and one peer. Real laptops and edge routers often run **several** WireGuard interfaces at once: a full-tunnel VPN to work, a split-tunnel to a lab, maybe a site-to-site mesh peer. The hard part is not key exchange. It is **who owns** `0.0.0.0/0`, how **policy routing** and **fwmarks** keep ciphertext off the wrong table, and how a **kill switch** (which is *not* a WireGuard feature) drops plaintext when the tunnel is down.

This chapter makes those three ideas concrete. Lab notes use Debian 13 ideas and `wireguard-tools` **v1.0.20210914** as found on this host; privileged `ip netns` drills need root and are shown as scripts you run on a disposable machine. No packages are installed in the scout sandbox.

```
wg version
ip -V | head -1
```

```
wireguard-tools v1.0.20210914 - https://git.zx2c4.com/wireguard-tools/
ip utility, iproute2-6.15.0, libbpf 1.5.0
```

Learning goals

By the end of this chapter you can:

- Explain why two peers both listing `AllowedIPs = 0.0.0.0/0` cannot share one routing story without policy routing
- Set `fwmark` on a WireGuard interface and steer plaintext with `ip rule` / extra tables (the `wg-quick` pattern)
- Sketch a kill switch as **filter** rules that drop non-marked traffic, not as a WireGuard knob
- Know when network namespaces (wireguard.com/netns) beat classic route hacks

Concepts in one page

```
App traffic (no mark)    ip rule: lookup table 2468    default dev wg0
wg0 UDP ciphertext        socket marked fwmark 1234    main table / ISP

Second tunnel wg1 with its own mark + table
   rules choose table by mark, uid, or fwmark from cgroup/VRF
```

Piece	Job
`AllowedIPs`	Cryptokey routing: outbound peer selection **and** inbound source filter
Kernel route to `wg0`	Only gets the packet *onto* the interface
`wg set … fwmark`	Marks WireGuard's **own** UDP so policy routing can exempt it from the tunnel default
`ip rule` + custom table	Sends unmarked (or otherwise classified) traffic into the tunnel table
Kill switch	Firewall (`iptables`/`nftables`) drop of traffic that is neither local nor marked for a live tunnel
Netns physical/init split	Alternative full-tunnel design from the WireGuard site — ciphertext socket stays in the "physical" namespace

Worked examples

Case 1: Two tunnels, one default — the collision

```
# Bad idea on one host without policy routing:
# /etc/wireguard/wg0.conf  (work VPN)
[Peer]
AllowedIPs = 0.0.0.0/0, ::/0

# /etc/wireguard/wg1.conf  (personal VPN)
[Peer]
AllowedIPs = 0.0.0.0/0, ::/0
```

Both interfaces claim the entire Internet in cryptokey routing **and** (via `wg-quick`) typically install default routes. Whichever `wg-quick up` ran last wins the default route; the other peer's `AllowedIPs` still claim destinations they will never see. Symptoms: "VPN is up" in `wg show` but traffic leaks to the wrong peer or blackholes.

Boring fix: at most **one** peer on a host with `0.0.0.0/0`. Other tunnels get **split** prefixes (`10.0.0.0/8`, `192.0.2.0/24`, …). If you truly need two full tunnels, separate them with **namespaces**, **VRFs**, or **per-uid / per-mark** policy routing — not two defaults in `main`.

Case 2: `fwmark` + policy routing (single full tunnel)

This is the improved rule-based pattern from Routing & Network Namespaces — the same idea `wg-quick` uses:

```
#!/usr/bin/env bash
# wg-policy-up.sh - docs-faithful sketch (needs root; not run in scout sandbox)
set -euo pipefail
# Assume wg0 already has keys/peers; we only add routing policy.
wg set wg0 fwmark 1234
ip route add default dev wg0 table 2468
ip rule add not fwmark 1234 table 2468
ip rule add table main suppress_prefixlength 0
```

Step	Meaning
`fwmark 1234` on `wg0`	Ciphertext UDP from WireGuard carries mark 1234
`default dev wg0 table 2468`	Tunnel table sends everything into `wg0`
`not fwmark 1234 table 2468`	Plaintext (unmarked) uses the tunnel table
`suppress_prefixlength 0`	Still allow non-default routes from `main` (LAN) for unmarked packets

Without the mark exemption, tunnel ciphertext would be routed back into `wg0` (routing loop). With it, only application traffic takes the tunnel default.

```
# Tear-down sketch
ip rule del not fwmark 1234 table 2468 2>/dev/null || true
ip rule del table main suppress_prefixlength 0 2>/dev/null || true
ip route flush table 2468 2>/dev/null || true
wg set wg0 fwmark 0
```

Verification on this scout host: commands were **not** executed with root. On a lab VM, `wg show wg0` should list `fwmark: 0x4d2`, and `ip rule` should show the two policy lines.

Case 3: Multi-tunnel with distinct marks

```
  wg0 (work)   fwmark 0x10   table 100   rule: from all fwmark 0x10 lookup 100
  wg1 (lab)    fwmark 0x20   table 200   rule: from all fwmark 0x20 lookup 200

  Mark traffic deliberately:
    - systemd: IPAccounting / socket marking via nft / cgroup classifiers
    - "work browser" runs under a unit that sets mark 0x10
    - default unmarked traffic: either ISP (no kill switch) or drop (kill switch)
```

```
# Sketch - two tunnel tables (root; not run here)
wg set wg0 fwmark 16
wg set wg1 fwmark 32
ip route add default dev wg0 table 100
ip route add default dev wg1 table 200
ip rule add fwmark 16 table 100
```

```
ip rule add fwmark 32 table 200
# Exempt each interface's own ciphertext from its tunnel table using
# "not fwmark <that-iface-mark>" rules, same idea as Case 2.
```

`AllowedIPs` on each peer should match the **prefixes you actually send** into that interface. A work VPN peer might get `0.0.0.0/0` *inside table 100 only*; a lab peer gets `10.50.0.0/16` only.

Case 4: Kill switch = firewall, not WireGuard

WireGuard has no `KillSwitch=true`. Operators implement "if the tunnel is down, do not leak" with packet filters:

```
# nftables sketch - drop forwarded/output traffic not marked for a tunnel
# (illustrative; tighten to your iface list and local subnets before use)
nft add table inet killswitch
nft add chain inet killswitch output '{ type filter hook output priority 0; policy drop; }'
nft add rule  inet killswitch output meta mark 16 accept   # work tunnel mark
nft add rule  inet killswitch output meta mark 32 accept   # lab tunnel mark
nft add rule  inet killswitch output oifname "lo" accept
nft add rule  inet killswitch output ip daddr 192.168.0.0/16 accept  # example LAN
# cipher path: accept UDP to known endpoints OR accept mark that wg sets on socket
```

When `wg0` goes down, marks never get applied by your classifier → output drops → no ISP leak. Mis-write the LAN exceptions and you lock yourself out of SSH; test on console.

Case 5: Namespace alternative for "all traffic through one tunnel"

From the official netns guide: move `eth0` into a `physical` namespace, birth `wg0` there, move `wg0` to init, default-route init through `wg0`. Ciphertext UDP stays in `physical`; ordinary processes never see `eth0`. That design **avoids** half of the `main`-table fights and is often cleaner than multiple full tunnels in one namespace. Multiple *simultaneous* full tunnels still want marks, VRFs, or separate netns per tunnel.

The trap

- Two `AllowedIPs = 0.0.0.0/0` peers without separate policy domains.
- Forgetting fwmark exemption → mysterious loss of handshake (ciphertext sucked into the tunnel route).
- Calling `wg-quick` "the kill switch" — `SaveConfig` / `Table=` help routing; leak prevention is firewall policy.
- Skipping endpoint / LAN exceptions and bricking remote admin access.
- Treating X marketing ("multi-tunnel + kill switch") as WireGuard core features — they are **operator composition** on top of cryptokey routing.

The boring rule

One default owner per routing domain; mark ciphertext; steer plaintext with `ip rule`; implement kill switches in `nftables`/`iptables`; prefer netns when the whole host should only speak through one tunnel.

Try this

1. On a disposable VM, bring up one `wg-quick` full tunnel and `wg show` / `ip rule` / `ip route show table all` to see marks and tables.
2. Add a second split-tunnel interface with a narrow `AllowedIPs` and ping across both.
3. Implement a minimal nft output drop policy and prove that stopping `wg-quick` blocks IPv4 to `1.1.1.1` while LAN still works.
4. Read wireguard.com/netns end-to-end and sketch when you would choose netns over fwmark.

Sources

- WireGuard · Routing & Network Namespaces
- `wg(8)`, `wg-quick(8)`, `ip-rule(8)`, `ip-route(8)`
- Prior chapter in this book: cryptokey routing and PSK (`02b-wireguard-cryptokey-routing-and-psk.qmd`)
- Inspiration: CumulusVPN on X (AllowedIPs fights, policy routing, fwmarks, kill switch as firewall)

WireGuard and MTU: Where the 1420 Comes From, Padding, and Silent Fragmentation

WireGuard and MTU: Where the 1420 Comes From, Padding, and Silent Fragmentation

Learning goals

- Derive WireGuard's default MTU of 1420 from the kernel source, and know why it is 80 bytes and not 60.
- Measure the real outer packet size for a given inner packet, including WireGuard's **16-byte padding**, on a live kernel tunnel.
- See what Linux WireGuard does when the underlay shrinks below what `wg0` assumes. It sends outer IPv4 with **DF=0**, so the packets are fragmented instead of dropped, and you need to know when that rescue stops working.
- Size `wg0` for a smaller underlay and prove the result with a DF probe.

Version note: everything below ran on Linux **6.12** (the in-tree `wireguard` module) inside an unprivileged user and network namespace (`unshare -rn`): no sudo, no `wg` tool and no `ping` on the box. Keys were set over netlink with pyroute2 0.9.7 (in a /workspace venv), packets were captured with an `AF_PACKET` socket in Python, and the underlay is a veth pair between two network namespaces. Kernel citations are to `drivers/net/wireguard/` at tag v6.12. The output is pasted from the run.

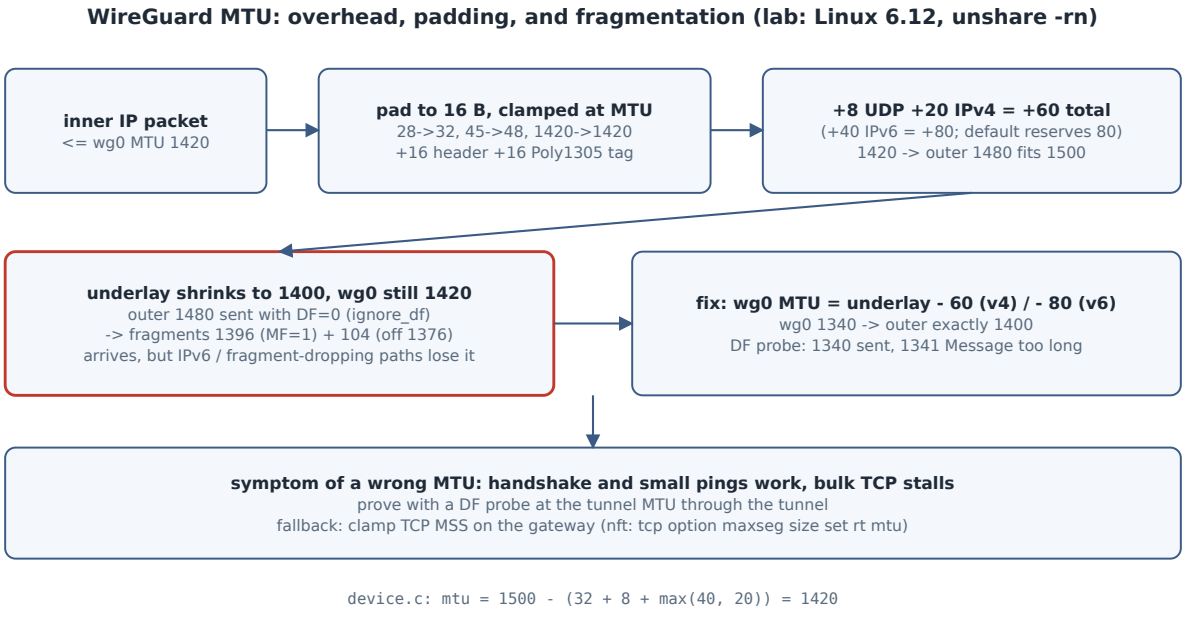


Figure 1: WireGuard MTU arithmetic and what happens when the underlay shrinks.

Where 1420 comes from

From `drivers/net/wireguard/device.c` (v6.12):

```
const int overhead = MESSAGE_MINIMUM_LENGTH + sizeof(struct udphdr) +
                     max(sizeof(struct ipv6hdr), sizeof(struct iphdr));
...
dev->mtu = ETH_DATA_LEN - overhead;
```

`MESSAGE_MINIMUM_LENGTH` is the data-message framing: a 16-byte header (type, receiver index and counter) plus the 16-byte Poly1305 tag, 32 bytes in all. Add the 8-byte UDP header and the **larger** of the two IP headers (40 for IPv6) and you get 80, so $1500 - 80 = \mathbf{1420}$. Over an IPv4 underlay the real overhead is 60, which leaves 20 bytes of slack. Over IPv6 it is exactly 80.

The plaintext is also padded. `send.c`'s `calculate_skb_padding()` aligns it to `MESSAGE_PADDING_MULTIPLE` (16) but clamps it to the interface MTU, so a full-size packet is never pushed over the MTU. The WireGuard paper describes the padding in §5.4.6.

Lab: a real tunnel between two namespaces

```
#!/bin/sh
# Run as: unshare -rn ./lab.sh   (unprivileged: user + net namespace, no sudo)
set -e
PY=/workspace/venv-qos/bin/python
unshare -n sleep 60 >/dev/null 2>&1 & B=$!; sleep 0.2
inB() { nsenter -t $B -n --preserve-credentials "$@"; }
ip link set lo up; inB ip link set lo up
ip link add vA type veth peer name vB; ip link set vB netns /proc/$B/ns/net
ip addr add 192.0.2.1/24 dev vA; ip link set vA up
inB ip addr add 192.0.2.2/24 dev vB; inB ip link set vB up
ip link add wg0 type wireguard; inB ip link add wg0 type wireguard
$PY wgset.py wg0 a.key b.pub 51820 192.0.2.2 51820 10.9.0.2/32
inB $PY wgset.py wg0 b.key a.pub 51820 192.0.2.1 51820 10.9.0.1/32
ip addr add 10.9.0.1/24 dev wg0; ip link set wg0 up
inB ip addr add 10.9.0.2/24 dev wg0; inB ip link set wg0 up
echo "== links"; ip -o link show vA | awk '{print "   "$2, $4, $5}'; ip -o link show wg0 | a
inB python3 cap.py vB 8 & C=$!; sleep 0.3
step() { echo "-- $1"; shift; python3 send.py 10.9.0.2 "$@"; sleep 0.3; }
step "first packet triggers the handshake" 0
step "inner sizes vs outer IPv4 packet size (underlay MTU 1500)" 0 1 15 16 17 1392
step "DF probe through wg0 (MTU 1420)" 1393
ip link set vA mtu 1400; inB ip link set vB mtu 1400
step "underlay shrinks to 1400, wg0 left at 1420: inner 1420-byte packet" 1392
ip link set wg0 mtu 1340
step "fix: wg0 MTU 1340 (= 1400 - 60)" 1312 1313
```

```
wait $C; kill $B
```

Helpers: `wgset.py` sets keys and the peer over netlink, `send.py` sends UDP datagrams with DF set, and `cap.py` prints every IPv4 frame arriving on the far end of the underlay.

```
# wgset.py IFNAME PRIVFILE PEERPUBFILE LISTENPORT PEER_ENDPOINT_IP PEER_PORT PEER_ALLOWED
import sys
from pyroute2 import WireGuard
ifn, priv, peer, lport, eip, eport, allowed = sys.argv[1:]
WireGuard().set(ifn, private_key=open(priv).read().strip(), listen_port=int(lport),
    peer={"public_key": open(peer).read().strip(), "endpoint_addr": eip,
          "endpoint_port": int(eport), "allowed_ips": [allowed]})
```

```
# send.py DST SIZE...: send one UDP datagram per SIZE with DF set (IP_PMTUDISC_DO)
import socket, sys
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
s.setsockopt(socket.IPPROTO_IP, 10, 2)  # IP_MTU_DISCOVER, IP_PMTUDISC_DO (linux/in.h)
for n in map(int, sys.argv[2:]):
    try:
        s.sendto(b"x" * n, (sys.argv[1], 9)); print(f"   payload {n:5d} (IP {n+28:5d}): sent
    except OSError as e:
        print(f"   payload {n:5d} (IP {n+28:5d}): {e.strerror}")
```

```
# cap.py IFACE SECONDS: print every IPv4 frame received on IFACE (AF_PACKET, no tcpdump)
import socket, struct, sys, time
s = socket.socket(socket.AF_PACKET, socket.SOCK_RAW, socket.htons(0x0800))
s.bind((sys.argv[1], 0)); s.settimeout(0.2)
end = time.time() + float(sys.argv[2])
KIND = {1: "handshake-init", 2: "handshake-resp", 3: "cookie", 4: "data"}
while time.time() < end:
    try: f, addr = s.recvfrom(65535)
    except socket.timeout: continue
    if addr[2] == socket.PACKET_OUTGOING: continue
    ip = f[14:]; tot, = struct.unpack("!H", ip[2:4]); fl, = struct.unpack("!H", ip[6:8])
    df, mf, off = fl >> 14 & 1, fl >> 13 & 1, (fl & 0x1fff) * 8
    what = ""
    if ip[9] == 17 and off == 0:
        udp = ip[20:]; what = KIND.get(udp[8], "?")
        if what == "data" and not mf: what += f" (encrypted inner packet {tot - 20 - 8 - 32}
        elif what == "data": what += " (first fragment)"
    elif off: what = "(later fragment)"
    print(f"      rx IP {tot:5d}  DF={df} MF={mf} off={off:<5d} {what}", flush=True)
```

The keys `a.key`/`b.key` and `a.pub`/`b.pub` are fresh X25519 keys, generated with Python `cryptography` and base64-encoded, which is the same format as `wg genkey`/`wg pubkey`.

```
$ unshare -rn ./lab.sh
== links
   vA@if2: mtu 1500
   wg0: mtu 1420
-- first packet triggers the handshake
   payload     0 (IP    28): sent
      rx IP   176  DF=0 MF=0 off=0     handshake-init
      rx IP    92  DF=0 MF=0 off=0     data (encrypted inner packet 32 B incl. padding)
-- inner sizes vs outer IPv4 packet size (underlay MTU 1500)
   payload     0 (IP    28): sent
   payload     1 (IP    29): sent
   payload    15 (IP    43): sent
   payload    16 (IP    44): sent
   payload    17 (IP    45): sent
   payload  1392 (IP  1420): sent
      rx IP    92  DF=0 MF=0 off=0     data (encrypted inner packet 32 B incl. padding)
      rx IP    92  DF=0 MF=0 off=0     data (encrypted inner packet 32 B incl. padding)
      rx IP   108  DF=0 MF=0 off=0     data (encrypted inner packet 48 B incl. padding)
      rx IP   108  DF=0 MF=0 off=0     data (encrypted inner packet 48 B incl. padding)
      rx IP   108  DF=0 MF=0 off=0     data (encrypted inner packet 48 B incl. padding)
      rx IP  1480  DF=0 MF=0 off=0     data (encrypted inner packet 1420 B incl. padding)
-- DF probe through wg0 (MTU 1420)
   payload  1393 (IP  1421): Message too long
-- underlay shrinks to 1400, wg0 left at 1420: inner 1420-byte packet
   payload  1392 (IP  1420): sent
      rx IP  1396  DF=0 MF=1 off=0     data (first fragment)
      rx IP   104  DF=0 MF=0 off=1376  (later fragment)
-- fix: wg0 MTU 1340 (= 1400 - 60)
   payload  1312 (IP  1340): sent
   payload  1313 (IP  1341): Message too long
      rx IP  1400  DF=0 MF=0 off=0     data (encrypted inner packet 1340 B incl. padding)
```

The capture only shows packets arriving at B, so the handshake response (B → A) is not listed.

Reading the numbers

Inner IP packet	Padded plaintext	Outer IPv4 packet	Overhead
28	32	92	64
29	32	92	63
43 / 44 / 45	48	108	65 / 64 / 63
1420	1420 (clamped, not 1424)	1480	60

- **Fixed overhead is 60 bytes on IPv4**, plus up to 15 bytes of padding. The padding means small packets cost slightly more. At full size it costs nothing, because padding is clamped to the MTU: 1420 is not a multiple of 16, and yet the outer packet is exactly 1480.

- **The DF probe** (`IP_PMTUDISC_DO`, the socket equivalent of `ping -M do`) shows the inner limit: 1392 bytes of UDP payload (1420 IP) is sent, and 1393 gets `EMSGSIZE` locally.
- **When the underlay shrinks to 1400**, `wg0` still believes in 1420, and the encrypted 1480-byte packet leaves as **two IPv4 fragments**, 1396 + 104, with `DF=0`. It arrives. That is no accident: `socket.c` sets `skb->ignore_df = 1` and passes `df = 0` to `udp_tunnel_xmit_skb()`. A Linux WireGuard sender lets an IPv4 underlay fragment rather than drop.
- **After the fix** (`wg0` MTU 1340 = 1400 − 60), a 1340-byte inner packet becomes exactly one 1400-byte outer packet with no fragments, and anything larger fails fast at the sender.

When fragmentation stops saving you (not run here)

The fragment rescue in the lab worked because the box doing the fragmenting was the sender, and IPv4 allows any router to fragment a DF=0 packet. It stops working when:

- **The underlay is IPv6.** IPv6 routers never fragment (RFC 8200 §5), so an oversized outer packet in the middle of the path is dropped. The sender learns about it only from ICMPv6 Packet Too Big, if that arrives.
- **Something on the path drops fragments.** Firewalls, some cloud load balancers and NAT devices do. RFC 8900 catalogues why IP fragmentation is fragile.
- **The fragments survive but cost you.** Every full-size packet becomes two, and losing either one loses both. Throughput drops and loss amplifies even though "it works".

In all three cases the symptom is the classic one: handshakes and small pings work, and bulk TCP stalls.

The boring rule

- **Default 1420** is right when the underlay MTU is 1500, over either IP family.
- **Smaller underlay** (PPPoE 1492, another tunnel, a cloud VPC with odd MTUs): set `wg0` MTU = underlay MTU − 80. Use − 60 only if the underlay is IPv4-only and will stay that way.
- **Prove it with a DF probe** at the tunnel MTU through the tunnel, not with a default-size ping.
- **TCP clamping:** if you cannot control every host's MTU, clamp TCP MSS on the tunnel gateway (nftables `tcp option maxseg size set rt mtu`).

Try this

1. Move the underlay to IPv6 (`2001:db8::1/64` on the veths, endpoints in IPv6) and repeat the shrink step. Does the outer packet still go out as fragments from the sender? Do the sizes change by 20 bytes?
2. Set `wg0` MTU to 1424 (a multiple of 16) and send a 1424-byte inner packet. What does the clamp do now?
3. Measure throughput with and without fragmentation using a 10-second UDP blast from `send.py` and the rx counters in `/proc/net/dev`.

Sources

- Linux v6.12 `drivers/net/wireguard/device.c` (`overhead`, `dev->mtu`), `send.c` (`calculate_skb_padding`), `socket.c` (`ignore_df`, `df = 0`), `messages.h` (`MESSAGE_PADDING_MULTIPLE`, `MESSAGE_MINIMUM_LENGTH`): https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/tree/drivers/net/wireguard?h=v6.12
- J. A. Donenfeld, *WireGuard: Next Generation Kernel Network Tunnel*, §5.4.6 (transport data messages and padding): https://www.wireguard.com/papers/wireguard.pdf
- RFC 8200 §5 (IPv6 packet size; no fragmentation by routers): https://www.rfc-editor.org/rfc/rfc8200
- RFC 8900, *IP Fragmentation Considered Fragile*: https://www.rfc-editor.org/rfc/rfc8900
- ip(7), `IP_MTU_DISCOVER` / `IP_PMTUDISC_DO`: https://man7.org/linux/man-pages/man7/ip.7.html
- nft(8), TCP option `maxseg` and `rt mtu`: https://www.netfilter.org/projects/nftables/manpage.html

VXLAN Data Plane

VXLAN Data Plane

VXLAN encapsulates Ethernet frames in UDP/IP so you can stretch **L2 segments over an L3 underlay**. This chapter focuses on the **data plane**: VNI, VTEP, flood behavior, and a Linux VXLAN lab in Containerlab (mid-2026). Control planes (EVPN) come next.

Learning goals

By the end of this chapter you can:

- Define VTEP, VNI, and underlay vs overlay roles for VXLAN
- Configure Linux `ip link add type vxlan` with static flood lists or bridge
- Capture VXLAN UDP (port 4789) and relate outer/inner headers
- Predict MAC learning and flooding behavior without EVPN
- Handle MTU for VXLAN overhead

Concepts

Roles

Term	Meaning
VTEP	VXLAN Tunnel End Point—encap/decap device
VNI	VXLAN Network Identifier—segment ID (~24-bit space)
Underlay	IP network between VTEP loopbacks/addresses
Overlay	Ethernet segments identified by VNI

```
VM/host -- L2 -- [ VTEP ]==== underlay IP ====[ VTEP ] -- L2 -- VM/host
                   VNI 100                                VNI 100
```

Header (mental)

```
Outer Eth | Outer IP | UDP dport 4789 | VXLAN (VNI) | Inner Eth | Inner payload
```

UDP source port often encodes entropy for ECMP.

Data-plane only learning

Without EVPN (or a controller):

1. Unknown unicast / broadcast / multicast may **flood** to all VTEPs in the flood list (head-end replication) or underlay multicast group.
2. Source MAC learning binds MAC → remote VTEP.
3. Scale limits and wasteful flood—why EVPN exists.

Multicast vs head-end replication

Mode	Idea
Underlay multicast	BUM to group; needs multicast routing
Head-end replication (HER)	Ingress VTEP unicasts BUM to each peer VTEP

Labs often use **HER / static remote VTEP list** for simplicity.

When VXLAN is appropriate

- DC fabric tenant segments
- Lab simulation of multi-leaf L2
- Separating many logical L2s without many VLANs on underlay

Not always needed for simple L3-only campuses.

Topology — two VTEPs + underlay + hosts

```
name: vxlan-dp

topology:
  nodes:
```

```
  leaf1:
    kind: linux
    image: alpine:3.20
    exec:
      - apk add --no-cache iproute2 bridge iputils tcpdump
  leaf2:
    kind: linux
    image: alpine:3.20
    exec:
      - apk add --no-cache iproute2 bridge iputils tcpdump
  spine:
    kind: linux
    image: alpine:3.20
    exec:
      - apk add --no-cache iproute2 iputils tcpdump
      - sysctl -w net.ipv4.ip_forward=1
  h1:
    kind: linux
    image: alpine:3.20
    exec:
      - apk add --no-cache iproute2 iputils
  h2:
    kind: linux
    image: alpine:3.20
    exec:
      - apk add --no-cache iproute2 iputils

links:
  - endpoints: ["h1:eth1", "leaf1:eth1"]
  - endpoints: ["h2:eth1", "leaf2:eth1"]
  - endpoints: ["leaf1:eth2", "spine:eth1"]
  - endpoints: ["leaf2:eth2", "spine:eth2"]
```

Addressing

Node	Underlay	Access
leaf1 lo	10.0.0.1/32	—
leaf2 lo	10.0.0.2/32	—
leaf1 eth2	10.1.1.1/30	—
spine eth1	10.1.1.2/30	—
leaf2 eth2	10.1.2.1/30	—
spine eth2	10.1.2.2/30	—
h1	—	10.10.10.10/24
h2	—	10.10.10.20/24
VNI	100	same L2 segment

Node	Underlay	Access

Underlay routes: leaf1 reaches 10.0.0.2 via spine; advertise loopbacks with statics:

```
docker exec clab-vxlan-dp-leaf1 sh -c '
  ip addr add 10.0.0.1/32 dev lo
  ip addr add 10.1.1.1/30 dev eth2; ip link set eth2 up
  ip route add 10.0.0.2/32 via 10.1.1.2
  ip route add 10.1.2.0/30 via 10.1.1.2
'
docker exec clab-vxlan-dp-spine sh -c '
  ip addr add 10.1.1.2/30 dev eth1; ip link set eth1 up
  ip addr add 10.1.2.2/30 dev eth2; ip link set eth2 up
  ip route add 10.0.0.1/32 via 10.1.1.1
  ip route add 10.0.0.2/32 via 10.1.2.1
'
# leaf2 similar with 10.0.0.2 and 10.1.2.1
```

VXLAN + bridge on leaf1

```
docker exec clab-vxlan-dp-leaf1 sh -c '
  ip link add br100 type bridge
  ip link set br100 up
  ip link add vxlan100 type vxlan id 100 \
    local 10.0.0.1 dev eth2 dstport 4789
  # static remote VTEP (HER)
  bridge fdb append 00:00:00:00:00:00 dev vxlan100 dst 10.0.0.2
  ip link set vxlan100 up
  ip link set eth1 up
  ip link set eth1 master br100
  ip link set vxlan100 master br100
'
```

leaf2:

```
docker exec clab-vxlan-dp-leaf2 sh -c '
  ip link add br100 type bridge
  ip link set br100 up
  ip link add vxlan100 type vxlan id 100 \
    local 10.0.0.2 dev eth2 dstport 4789
  bridge fdb append 00:00:00:00:00:00 dev vxlan100 dst 10.0.0.1
  ip link set vxlan100 up
  ip link set eth1 up
  ip link set eth1 master br100
```

```
  ip link set vxlan100 master br100
'
```

Hosts (same subnet, no default needed for local L2 test):

```
docker exec clab-vxlan-dp-h1 sh -c '
  ip addr add 10.10.10.10/24 dev eth1; ip link set eth1 up
'
docker exec clab-vxlan-dp-h2 sh -c '
  ip addr add 10.10.10.20/24 dev eth1; ip link set eth1 up
'
```

Syntax note: `bridge fdb` / `ip link add vxlan` flags vary slightly by iproute2 version; consult `man ip-link` on your image. Some use `vxlan ... remote 10.0.0.2` for point-to-point single peer.

Deploy checks

```
docker exec clab-vxlan-dp-leaf1 ping -c 2 10.0.0.2
docker exec clab-vxlan-dp-h1 ping -c 3 10.10.10.20
docker exec clab-vxlan-dp-leaf1 bridge fdb show
docker exec clab-vxlan-dp-spine tcpdump -ni eth1 -c 10 udp port 4789
```

Predict

- Underlay loopback ping works
- Host ping works over VNI 100
- Spine sees UDP/4789, not bare inner ARP as local LAN

Observe

```
docker exec clab-vxlan-dp-spine tcpdump -ni eth1 -vv -c 5 udp port 4789
docker exec clab-vxlan-dp-leaf1 ip -d link show vxlan100
```

Fix

Failure	Checks
Underlay down	static routes, spine forward
No FDB flood entry	remote VTEP missing
VNI mismatch	id 100 both sides

Failure	Checks
Host not in bridge	master br100

Drill — MTU

VXLAN overhead commonly ~50 bytes class. Set underlay MTU 1550 or overlay-aware 1450:

```
docker exec clab-vxlan-dp-leaf1 ip link set vxlan100 mtu 1450
docker exec clab-vxlan-dp-leaf2 ip link set vxlan100 mtu 1450
docker exec clab-vxlan-dp-h1 ping -c 2 -M do -s 1472 10.10.10.20
docker exec clab-vxlan-dp-h1 ping -c 2 -M do -s 1400 10.10.10.20
```

Predict

Too-large DF fails; tuned sizes pass.

Drill — wrong VNI isolation

Create VNI 200 on leaf2 only for a third host experiment—or change leaf2 to `id 200` temporarily:

Predict

h1 cannot reach h2 (different segments).
Fix

Align VNI; document allocation.

Drill — MAC move / learning

```
docker exec clab-vxlan-dp-leaf1 bridge fdb show br100
docker exec clab-vxlan-dp-h1 ping -c 1 10.10.10.20
docker exec clab-vxlan-dp-leaf1 bridge fdb show br100
```

Predict

After traffic, remote MAC points at vxlan device / dst VTEP.

Flood scope risk

Static `00:00:00:00:00:00` FDB means BUM replicates to listed VTEPs. More leaves → more replication. This is why control planes advertise **who needs which VNI**.

FRR / SR Linux awareness

NOS VTEPs bind VNI to VLAN/IRB and underlay loopbacks. Data-plane counters and `show vxlan` equivalents matter operationally. Linux lab teaches headers; production uses integrated L2/L3.

Verification checklist

Check	Intent
Underlay VTEP reachability	Outer IP works
UDP 4789 on path	Encap happening
Same VNI	Segment match
FDB remote	Mapping exists
Host ARP/ping	Inner L2
MTU tests	Size tax handled

Common mistakes

Mistake	Symptom
VTEP IP not loopback stable	Flaps when link IP changes
No underlay route to remote VTEP	Silent blackhole
Bridging mis-order	Loops or no pass
Expecting EVPN routes without config	Only static works
One huge flood list forever	BUM explosion

Summary

- VXLAN is Ethernet-in-UDP/IP with **VNI** segment IDs
- VTEPs encap/decap; underlay only sees outer IP/UDP
- Static HER labs teach data plane before EVPN
- Verify with underlay ping, host ping, and spine captures

- Plan MTU and flood scope deliberately

Next: **EVPN control-plane intro**—distributing MAC/IP and VNI membership without pure flood-and-learn.

EVPN Control-Plane Intro

EVPN Control-Plane Intro

EVPN (Ethernet VPN) is a **BGP-based control plane** for overlay reachability—most often paired with VXLAN data planes in modern fabrics. This chapter is an **introduction to models and lab reality on free stacks** (FRR EVPN/VXLAN where available, optional SR Linux community). Syntax shifts; objects do not.

Learning goals

By the end of this chapter you can:

- Explain what EVPN advertises versus what VXLAN carries
- Name core route types at a practical level (2, 3, 5—awareness)
- Relate RD/RT-style membership ideas to "who joins which VNI"
- Sketch an FRR EVPN + VXLAN lab and know how to verify BGP EVPN tables
- Predict failure when underlay OK but EVPN session down

Why EVPN exists

Data-plane-only VXLAN:

- Floods BUM to all peers
- Learns MAC reactively
- Scales poorly; slow on moves

EVPN:

- Distributes MAC/IP bindings and VTEP locations via BGP
- Reduces unknown flooding (in good designs)
- Supports L2 and L3 (IRB) services in one framework

```
BGP EVPN  (control)     VXLAN/GRE/...  (data)

          same fabric intent
```

Objects (vendor-agnostic)

Object	Role
VTEP IP	Underlay endpoint
VNI	L2 segment id in data plane
MAC-VRF / EVI	Service instance
RD	Disambiguates routes
RT (route target)	Import/export membership
Type-2 route	MAC (/IP) advertisement
Type-3 route	Inclusive multicast / flood tree membership ideas
Type-5 route	IP prefix (L3 VPN style overlay)
IRB / L3 VNI	Inter-subnet routing in fabric

You do not need full RFC memorization to operate: **session up → EVPN routes present → data plane VNI up → hosts pass**.

Control vs data verification split

Layer	Healthy signs
Underlay	VTEP loopbacks ping; ECMP present
BGP EVPN	Neighbors Established; AFI/SAFI EVPN negotiated
EVPN RIB	Type-2 for host MACs; correct next-hop VTEP
Data plane	VXLAN encap; host ping; FDB consistent

Minimal design story — 2 leaf

```
h1--leaf1----spine----leaf2--h2
      \      underlay     /
       BGP EVPN (often via spine route-reflector or eBGP unnumbered fabric)
```

Common free lab patterns:

1. **iBGP EVPN** to a route reflector (spine or dedicated).
2. **eBGP EVPN** unnumbered on fabric links (advanced; FRR supports variants).

Start with **iBGP + RR** mentally even if your first lab is two leaves peering directly.

FRR capability note (mid-2026)

FRR continues to expand EVPN/VXLAN integration. Treat the following as **structural config**—verify against your image's `vtysh` help and docs:

```
# daemons: zebra, bgpd, staticd (and platform vxlan integration)
router bgp 65000
 bgp router-id 10.0.0.1
 neighbor 10.0.0.3 remote-as 65000
 neighbor 10.0.0.3 update-source lo
 !
 address-family l2vpn evpn
  neighbor 10.0.0.3 activate
  advertise-all-vni
 exit-address-family
!
# VNI / bridge mapping is platform-specific:
# Linux FRR may integrate with kernel bridge/vxlan via zebra
```

Always confirm with:

```
vtysh -c 'show bgp l2vpn evpn summary'
vtysh -c 'show bgp l2vpn evpn'
vtysh -c 'show evpn vni'
vtysh -c 'show evpn mac vni all'
```

If a command is missing, your build differs—fall back to kernel VXLAN static (previous chapter) and read FRR release notes.

Topology sketch (EVPN lab)

```
name: evpn-intro

topology:
  nodes:
    leaf1:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/leaf1:/etc/frr
    leaf2:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/leaf2:/etc/frr
```

```
    rr:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/rr:/etc/frr
    h1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
    h2:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils

  links:
    - endpoints: ["leaf1:eth1", "rr:eth1"]
    - endpoints: ["leaf2:eth1", "rr:eth2"]
    - endpoints: ["h1:eth1", "leaf1:eth2"]
    - endpoints: ["h2:eth1", "leaf2:eth2"]
```

Underlay: give leaf1/leaf2/rr loopbacks `10.0.0.1/32`, `.2`, `.3` and point-to-point links with static or OSPF so loopbacks mesh. EVPN BGP between leaves and rr using loopbacks.

RR config idea

```
router bgp 65000
 bgp router-id 10.0.0.3
 neighbor 10.0.0.1 remote-as 65000
 neighbor 10.0.0.1 update-source lo
 neighbor 10.0.0.2 remote-as 65000
 neighbor 10.0.0.2 update-source lo
 address-family l2vpn evpn
  neighbor 10.0.0.1 activate
  neighbor 10.0.0.1 route-reflector-client
  neighbor 10.0.0.2 activate
  neighbor 10.0.0.2 route-reflector-client
 exit-address-family
```

Predict → observe → fix drills

Drill 1 — Session before service

```
docker exec clab-evpn-intro-leaf1 vtysh -c 'show bgp l2vpn evpn summary'
```

Predict: Established to RR.
If Idle: underlay loopback ping, update-source, firewall, daemons.

Drill 2 — Host MAC appears as EVPN route

```
docker exec clab-evpn-intro-h1 ping -c 2 10.10.10.20
docker exec clab-evpn-intro-leaf1 vtysh -c 'show bgp l2vpn evpn'
docker exec clab-evpn-intro-leaf2 vtysh -c 'show evpn mac vni all'
```

Predict: Type-2-like entries; next-hop remote VTEP.
Fix: VNI advertise config, bridge binding, host traffic to trigger learning if required.

Drill 3 — Kill EVPN, keep underlay

```
docker exec clab-evpn-intro-leaf1 vtysh -c 'conf t' -c 'router bgp 65000' -c 'neighbor 10.0.
docker exec clab-evpn-intro-h1 ping -c 5 10.10.10.20
```

Predict: underlay pings between leaves still work; overlay host traffic fails or relies on stale/f flood only.
Observe: BGP down, EVPN table ages out.
Fix: `no neighbor … shutdown`.

Drill 4 — RT mismatch (concept)

If export/import RTs do not match, routes exist on one leaf but are not imported on the other.

Predict: `show bgp l2vpn evpn` has routes locally originated but remote missing or not installed.
Fix: align VRF/VNI RT policy.

L2 VNI vs L3 VNI (awareness)

Service	Host experience	EVPN idea
L2 VNI	Same subnet two leaves	MAC routes (type-2)
L3 VNI / IRB	Different subnets routed in fabric	Type-5 prefixes + symmetric/asymmetric IRB models

Do not implement every IRB mode on day one. Get **same-subnet across two leaves** solid first.

Optional SR Linux community path

Nokia SR Linux is a common Containerlab free/community kind for EVPN-VXLAN labs. Workflow:

1. Underlay eBGP or OSPF
2. Overlay BGP EVPN
3. `mac-vrf` / `vxlan-interface` style objects

Map to the same verification split. Use SR Linux documentation for exact CLI; journal the object mapping:

```
VNI   vxlan-interface
MAC-VRF   bridge table
bgp-evpn   advertisement
```

Security / isolation checklist

- VNI allocation authority
- RT import discipline (no accidental cross-tenant)
- Underlay ACL optional for UDP/4789 between VTEPs only
- Control-plane protection on BGP

What this intro deliberately skips

- Full multihoming Ethernet segments (ESI) deep dive
- Complex IRB interop matrices
- Multicast optimized BUM underlays
- DCI stretch best practices

Those are fabric/scale topics once the skeleton works.

Common mistakes

Mistake	Symptom
EVPN before underlay	Endless BGP down
Wrong AFI/SAFI	Session up but no EVPN
Data plane VNI advertised	Control OK, ping fail
Treating EVPN as L2 switch only	Miss L3 service models later
No RR scaling plan	Full mesh pain

Summary

- EVPN is BGP control for overlay membership and MAC/IP reachability
- VXLAN (or similar) remains the data plane
- Verify **underlay → BGP EVPN → EVPN RIB → host data** in order
- FRR and SR Linux are free-ish paths to practice; check your image's feature set
- Master two-leaf same-subnet before IRB gymnastics

Next: **overlay lab**—a single integrated workout combining underlay, VXLAN, and (if available) EVPN verification.

Overlay Lab

Overlay Lab

This chapter is a **portfolio lab**: build a small multi-site / multi-leaf overlay on free tools, break it on purpose, and prove each plane. Choose track A (GRE sites), track B (VXLAN static), or track C (EVPN if your FRR/SR Linux build supports it). Mid-2026 defaults: Containerlab + FRR + Alpine.

Learning goals

By the end of this chapter you can:

- Deliver one complete overlay lab folder that rebuilds from git
- Show captures proving encapsulation
- Run a failure matrix (underlay, tunnel endpoint, VNI/policy)
- Document MTU and security stance
- Meet the book's definition of done for overlay work

Pick a track

Track	Focus	Difficulty
A — Dual-site GRE/WG	Site tunnels + routing	
B — VXLAN HER two-leaf	L2 overlay data plane	
C — EVPN two-leaf + RR	Control + data	

Do **A then B** if new to overlays. Attempt C only after B host pings work.

Definition of done (all tracks)

- ☐ `README.md` with diagram + addressing
- ☐ `*.clab.yml` + configs/scripts in git

- ☐ Clean `containerlab deploy` on cold start
- ☐ `verify.sh` green
- ☐ Capture artifact (pcap or logged tcpdump) showing outer header
- ☐ 3 failure drills with notes
- ☐ MTU test recorded
- ☐ Retrospective: what was underlay vs overlay

Track A — Dual-site GRE (reference build)

Reuse topology from **Site Tunnels**; package it cleanly.

```
labs/overlay-a/
  site-tun.clab.yml
  addressing.md
  scripts/bringup.sh
  verify.sh
  drills.md
  captures/.gitkeep
```

Success metrics

- h1 h2 continuous ping
- inet tcpdump shows GRE
- Underlay link down stops overlay; restore recovers
- DF large ping behavior documented

verify.sh

```
#!/usr/bin/env bash
set -euo pipefail
docker exec clab-site-tun-h1 ping -c2 -W1 192.168.20.10
docker exec clab-site-tun-h2 ping -c2 -W1 192.168.10.10
docker exec clab-site-tun-sitea ip link show gre1 | grep -q UP
echo OK-A
```

Failure matrix A

#	Action	Expect
A1	`inet` eth2 down	overlay down
A2	delete gre on sitea	blackhole
A3	remove LAN route	gre peer lives, hosts die
A4	lower mtu wrongly	large packet fail

Track B — VXLAN two-leaf (reference build)

Package **VXLAN data plane** chapter topology as `labs/overlay-b/`.

Success metrics

- Underlay VTEP lo ping
- h1 h2 same subnet across leaves
- Spine capture UDP/4789
- Wrong VNI experiment isolates
- FDB shows remote MAC after traffic

bringup excerpt

```
#!/usr/bin/env bash
set -euo pipefail
# called after containerlab deploy
sudo containerlab deploy -t vxlan-dp.clab.yml
# apply addresses + vxlan/bridge (idempotent scripts per node)
./scripts/addr.sh
./scripts/vxlan.sh
./verify.sh
```

verify.sh B

```
#!/usr/bin/env bash
set -euo pipefail
fail() { echo FAIL:$* >&2; exit 1; }
docker exec clab-vxlan-dp-leaf1 ping -c1 -W1 10.0.0.2 || fail underlay
```

```
docker exec clab-vxlan-dp-h1 ping -c2 -W1 10.10.10.20 || fail overlay
echo OK-B
```

Failure matrix B

#	Action	Expect
B1	spine forward off	underlay fail → overlay fail
B2	remove FDB flood entry	BUM/unknown fail
B3	VNI mismatch	isolation
B4	host unbridge eth1	local access fail

Capture helper

```
docker exec clab-vxlan-dp-spine sh -c \
  'tcpdump -ni eth1 -c 20 udp port 4789 -w /tmp/vx.pcap'
# docker cp out to captures/vx.pcap
```

Track C — EVPN intro (stretch)

In raw Containerlab, configuring BGP EVPN with VXLAN encapsulation on FRR requires intricate route distinguishers (RD), route targets (RT), VTEP interfaces, and EVPN address families.

Option 1: Netlab EVPN-VXLAN Topology (Recommended)

With Netlab (`provider: clab`), EVPN and VXLAN are first-class modules:

```
# topology.yml (Track C)
---
provider: clab
defaults.device: frr
module: [ ospf, bgp, vlan, vxlan, evpn ]

vlans:
  tenant_a:
    vni: 100100
    links: [ h1-l1, h2-l2 ]

nodes:
  spine: { role: spine }
  l1: { role: leaf }
  l2: { role: leaf }
```

```
  h1: { device: linux }
  h2: { device: linux }

links:
  - l1: spine
  - l2: spine
  - h1: l1
  - h2: l2

validate:
  evpn_sessions:
    description: "Verify BGP EVPN control plane is Established"
    nodes: [ l1, l2 ]
    plugin: bgp_neighbor(family='evpn', state='Established')
  tenant_ping:
    description: "Verify tenant host reachability across the EVPN overlay"
    nodes: [ h1 ]
    exec:
      ping_h2: ping -c 3 172.16.0.2
```

Deploy and validate with:

```
netlab up
netlab validate
```

Option 2: Manual FRR Configuration

If authoring the manual FRR configurations, package the EVPN chapter with explicit **feature probe**:

```
docker exec clab-evpn-intro-leaf1 vtysh -c 'show bgp l2vpn evpn summary' \
  || { echo "EVPN not available-document fallback to Track B"; exit 0; }
```

Success metrics

- BGP EVPN Established
- EVPN MAC routes visible (`show bgp l2vpn evpn`)
- Host ping works across tenant VNI
- Neighbor shutdown kills overlay learning path
- Underlay remains pingable

Failure matrix C

#	Action	Expect
C1	RR down	leaves lose routes (no alternate)
C2	advertise-all-vni removed	control stops updates
C3	underlay lo route removed	BGP over loopback dies
C4	RT mismatch experiment	import empty

Integrated "mini fabric snack" (optional merge)

If ambitious, combine track B underlay style with dual spines from the fabrics part later. **Do not** expand scope until verify is green on two leaves.

```
    s1
   /  \
 l1    l2
 |      |
 h1     h2
```

Underlay ECMP + VXLAN HER to both directions is enough complexity for one week.

Observability expectations

Tool	Use
`ping` / `traceroute`	Inner vs outer endpoints
`tcpdump` / `tshark`	Prove encap
`bridge fdb` / `wg show` / `ip xfrm`	State tables
`vtysh show bgp l2vpn evpn`	Control plane
`ip route get`	Which path outer takes

Hardening lite (overlay)

- ☐ Underlay allows only needed outer ports between VTEPs
- ☐ No accidental route leaking inner prefixes to "inet"
- ☐ Keys for WireGuard/IPsec not in public git
- ☐ VNI inventory file (`vnis.md`)

Lab report template

```
# Overlay lab report
Track: B
Date: 2026-...
Underlay: static loopbacks via spine
Overlay: VXLAN VNI 100 HER
Evidence:
- captures/vx.pcap
- verify.sh OK
Drills: B1-B4 notes...
What surprised me:
Next: EVPN or dual-spine
```

Predict → observe → fix (capstone habit)

For every drill row, write three lines in `drills.md`. If observe predict, you learned something—**that is the point**.

Time-box guidance

Block	Time
Deploy + address	30–60 min
Overlay up	1–2 h
Captures + verify	30 min
Failure matrix	1–2 h
Writeup	30 min

If blocked >1 h on EVPN syntax, drop to Track B and file an issue note—do not thrash.

Common mistakes

Mistake	Symptom
Mixing all tracks half-done	Nothing verifies
No cold-start test	"Works on my dirty ns"
Capturing on wrong node	Never see UDP 4789
Skipping underlay prove	Overlay debug theater
Giant jumbo scope	No finish

Summary

- One finished overlay lab beats three partial ones
- Tracks A→B→C escalate control-plane complexity
- Always prove **encap with packets** and **failures with a matrix**
- Keep VNI/tunnel inventory and MTU notes in git
- Next part scales underlays into **fabrics and multi-area design**

When this lab is green, you are ready for Clos-style underlays and multi-implementation leaves.

Head Office and Branch Office

Head Office and Branch Office

Two sites, one LAN. Each site is a small office in Containerlab. A Tailscale client on each gateway joins an in-lab tailnet, and VXLAN rides that tailnet so a workstation at the branch ARPs for a host at head office. The lab Internet only carries the encrypted underlay.

Versions this chapter is written against: Headscale **0.29.6** (Tailscale client **1.80** or newer; stable track at the time of writing is **1.104**), Debian **13** (trixie) for the gateways and the coordination node, Alpine for the switch and the workstations, Containerlab `kind: linux`. Deploy needs root, a container runtime, and `/dev/net/tun` on the host.

Learning goals

By the end of this chapter you can:

- Place Headscale, two Tailscale gateways, and two workstations in one Containerlab topology
- Use each gateway's Tailscale address as a VXLAN VTEP
- Put both offices on `10.10.10.0/24` and prove the path with ARP, ping, and two captures
- Size the overlay MTU from Tailscale's 1280-byte tunnel plus VXLAN overhead
- Add a third site with a flood entry, and name the L3 pattern for offices that use different prefixes

Why this shape

Tailscale already gives the two gateways addresses in `100.64.0.0/10` and encrypts what they send. That is enough when each office has its own prefix and a subnet route. This lab gives both offices the **same** prefix, `10.10.10.0/24`, and treats them as one Ethernet segment: head office holds `.10`, the branch holds `.20`, and neither host has a default route.

VXLAN is the segment. VNI **100** is that LAN. The VTEP address is the Tailscale address, and the VTEP device is `tailscale0`. The lab Internet (`203.0.113.0/24`, documentation prefix TEST-NET-3) is only how the gateways reach each other and the coordination server.

```
ho-host 10.10.10.10 -- ho-gw == Tailscale / VXLAN VNI 100 == bo-gw -- bo-host 10.10.10.20
                          |                                      |
                          +---- inet (203.0.113.0/24) -- hs -----+
```

Containerlab can also attach a lab's **management** network to a tailnet (`mgmt.tailscale` in the topology). That feature publishes the lab so you can reach a node from a laptop. It is a different job from the data plane in this chapter.

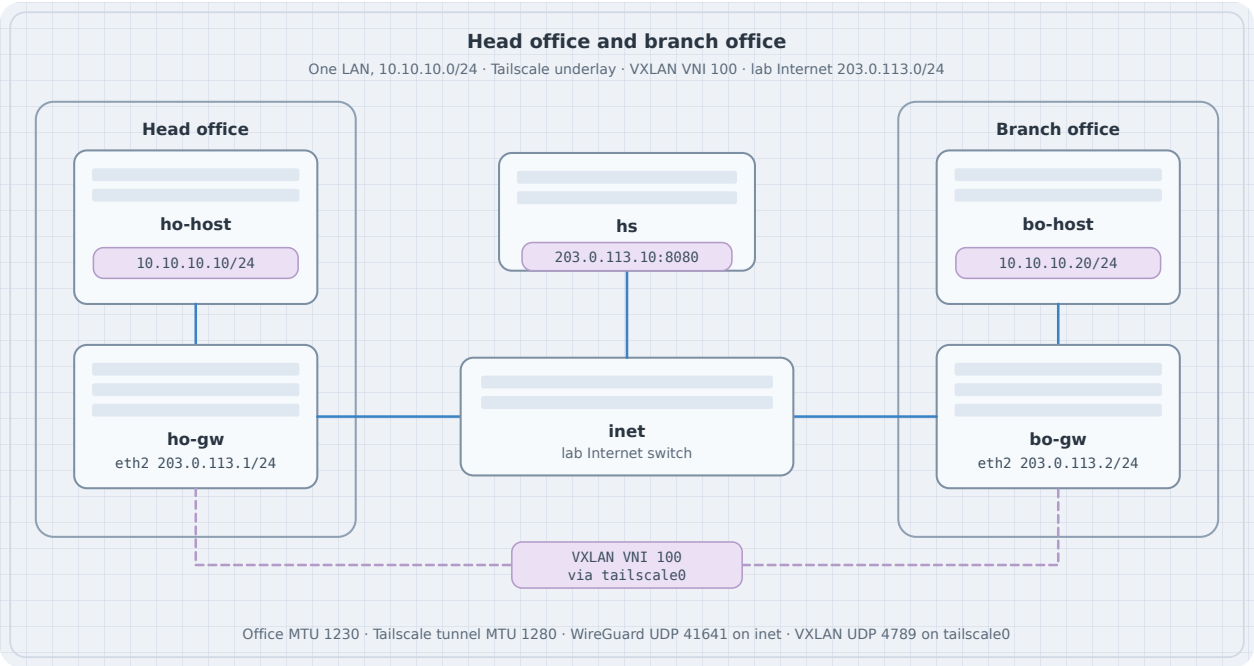


Figure 1: Head office and branch office on one VXLAN segment

Roles

Piece	Job in this lab
Headscale	Coordination server the two clients log in to. One tailnet, SQLite, no policy file (the default is permit).
Tailscale on ho-gw / bo-gw	Kernel `tailscale0`. WireGuard UDP **41641** toward the peer.
VTEP	`vxlan100`, VNI 100, `local` and `remote` set to the two Tailscale IPv4 addresses, underlay device `tailscale0`.
Office LAN	`eth1` and `vxlan100` in bridge `br100`. Workstations sit on `eth1`.
inet	Linux bridge. Shared lab Internet for `203.0.113.0/24`.

Headscale's own container image is a convenience build; the project documents the binary and the `.deb` as the supported installs. This lab runs that binary in a Debian node so the node has a shell, `ip`, and an address you assign yourself.

The coordination URL is plain HTTP on the lab Internet (`http://203.0.113.10:8080`). Headscale accepts that. A tailnet you keep uses HTTPS on a name you control. This topology does not publish port 8080 on the host.

Topology

Save the next two files in an empty directory. The bind path is relative to the topology file.

hs/config.yaml:

```
server_url: http://203.0.113.10:8080
listen_addr: 0.0.0.0:8080
metrics_listen_addr: 127.0.0.1:9090
grpc_listen_addr: 127.0.0.1:50443
grpc_allow_insecure: false

noise:
  private_key_path: /var/lib/headscale/noise_private.key

prefixes:
  v4: 100.64.0.0/10
  v6: fd7a:115c:a1e0::/48
  allocation: sequential

derp:
  server:
    enabled: false
  urls:
    - https://controlplane.tailscale.com/derpmap/default
  paths: []
  auto_update_enabled: false

disable_check_updates: true

database:
  type: sqlite
  sqlite:
    path: /var/lib/headscale/db.sqlite
    write_ahead_log: true

policy:
  mode: file
  path: ""

dns:
  magic_dns: false
  base_domain: offices.lab
  override_local_dns: false
  nameservers:
    global: []
  search_domains: []
  extra_records: []

unix_socket: /var/run/headscale/headscale.sock
unix_socket_permission: "0770"
```

```
logtail:
  enabled: false

log:
  level: info
  format: text
```

ts-vxlan.clab.yml:

```
name: ts-vxlan

topology:
  nodes:
    inet:
      kind: linux
      image: alpine:3.20
      exec:
        - sh -c "apk add --no-cache iproute2 && ip link add br0 type bridge && ip link set b
    hs:
      kind: linux
      image: debian:13-slim
      binds:
        - hs/config.yaml:/etc/headscale/config.yaml:ro
    ho-gw:
      kind: linux
      image: debian:13-slim
    bo-gw:
      kind: linux
      image: debian:13-slim
    ho-host:
      kind: linux
      image: alpine:3.20
      exec:
        - sh -c "apk add --no-cache iproute2 iputils"
    bo-host:
      kind: linux
      image: alpine:3.20
      exec:
        - sh -c "apk add --no-cache iproute2 iputils"

  links:
    - endpoints: ["ho-host:eth1", "ho-gw:eth1"]
    - endpoints: ["bo-host:eth1", "bo-gw:eth1"]
    - endpoints: ["ho-gw:eth2", "inet:eth1"]
    - endpoints: ["bo-gw:eth2", "inet:eth2"]
    - endpoints: ["hs:eth1", "inet:eth3"]
```

Containerlab starts nodes privileged. `tailscaled` uses that to create `tailscale0`. Check the host before deploy:

```
ls -l /dev/net/tun
sudo containerlab deploy -t ts-vxlan.clab.yml
```

Addressing

Node	Interface	Address
ho-host	eth1	10.10.10.10/24, MTU 1230
bo-host	eth1	10.10.10.20/24, MTU 1230
ho-gw	eth2	203.0.113.1/24
bo-gw	eth2	203.0.113.2/24
hs	eth1	203.0.113.10/24
ho-gw, bo-gw	tailscale0	assigned from 100.64.0.0/10
ho-gw, bo-gw	br100, vxlan100	no IP; MTU 1230
VNI	100	both offices

Read the Tailscale addresses after login. Sequential allocation on an empty database usually yields `100.64.0.1` and `100.64.0.2`. The scripts below use whatever `tailscale ip -4` prints.

Install and address the lab Internet

Debian nodes need packages from the network on `eth0`. Run this once after deploy.

```
install_gw() {
  local n="$1"
  docker exec "$n" bash -c '
    apt-get update
    apt-get install -y --no-install-recommends \
      ca-certificates curl iproute2 iptables tcpdump
    install -d -m 0755 /usr/share/keyrings
    curl -fsSL https://pkgs.tailscale.com/stable/debian/trixie.noarmor.gpg \
      -o /usr/share/keyrings/tailscale-archive-keyring.gpg
    curl -fsSL https://pkgs.tailscale.com/stable/debian/trixie.tailscale-keyring.list \
      -o /etc/apt/sources.list.d/tailscale.list
    apt-get update
    apt-get install -y tailscale
  '
}

install_gw clab-ts-vxlan-ho-gw
install_gw clab-ts-vxlan-bo-gw
```

```
docker exec clab-ts-vxlan-hs bash -c '
  apt-get update
  apt-get install -y --no-install-recommends ca-certificates curl iproute2
  arch=$(dpkg --print-architecture)
  curl -fsSL -o /usr/local/bin/headscale \
    "https://github.com/juanfont/headscale/releases/download/v0.29.6/headscale_0.29.6_linux_
  chmod 755 /usr/local/bin/headscale
  mkdir -p /var/lib/headscale /var/run/headscale
'

docker exec clab-ts-vxlan-hs ip addr add 203.0.113.10/24 dev eth1
docker exec clab-ts-vxlan-hs ip link set eth1 up
docker exec clab-ts-vxlan-ho-gw ip addr add 203.0.113.1/24 dev eth2
docker exec clab-ts-vxlan-ho-gw ip link set eth2 up
docker exec clab-ts-vxlan-bo-gw ip addr add 203.0.113.2/24 dev eth2
docker exec clab-ts-vxlan-bo-gw ip link set eth2 up

docker exec clab-ts-vxlan-ho-gw ping -c 2 -W 1 203.0.113.10
docker exec clab-ts-vxlan-ho-gw ping -c 2 -W 1 203.0.113.2
```

`ip route` on a gateway shows `203.0.113.0/24 dev eth2`. That connected route is the lab Internet. `eth0` remains the Containerlab management network, used here only to install packages.

Start Headscale and join both offices

```
docker exec -d clab-ts-vxlan-hs \
  headscale serve
# socket appears after the first migration
for _ in 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15; do
  docker exec clab-ts-vxlan-hs headscale users list && break
  sleep 1
done

docker exec clab-ts-vxlan-hs headscale users create offices
docker exec clab-ts-vxlan-hs headscale users list
```

On a fresh database the user id is **1**. If the list shows another id, pass that id below.

```
KEY=$(docker exec clab-ts-vxlan-hs \
  headscale preauthkeys create --user 1 --reusable --expiration 24h)
KEY=$(printf '%s\n' "$KEY" | grep -oE 'hskey[^[:space:]]+' | head -n1)
printf '%s\n' "$KEY"
```

The key is a credential. It is reusable for 24 hours and it is printed into your shell history. Tear the lab down when you finish, and keep the key out of the topology file.

```
up_site() {
  local n="$1" host="$2"
  docker exec "$n" bash -c "
    mkdir -p /var/lib/tailscale /run/tailscale
    tailscaled --state=/var/lib/tailscale/tailscaled.state \
      >/var/log/tailscaled.log 2>&1 &
    for _ in 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15; do
      [ -S /var/run/tailscale/tailscaled.sock ] && break
      sleep 1
    done
    # Management eth0 is a shortcut between every node. Drop WireGuard
    # there so the underlay you capture is the lab Internet.
    iptables -I OUTPUT -o eth0 -p udp --sport 41641 -j DROP
    iptables -I OUTPUT -o eth0 -p udp --dport 41641 -j DROP
    iptables -I INPUT  -i eth0 -p udp --dport 41641 -j DROP
    sysctl -w net.ipv4.conf.all.rp_filter=0
    sysctl -w net.ipv4.conf.eth2.rp_filter=0
    tailscale up \
      --login-server=http://203.0.113.10:8080 \
      --auth-key=${KEY} \
      --hostname=${host} \
      --accept-dns=false \
      --accept-routes=false
  "
}

up_site clab-ts-vxlan-ho-gw head-office
up_site clab-ts-vxlan-bo-gw branch-office

docker exec clab-ts-vxlan-ho-gw tailscale ip -4
docker exec clab-ts-vxlan-bo-gw tailscale ip -4
docker exec clab-ts-vxlan-ho-gw tailscale status
```

Predict

- Each gateway has one address in `100.64.0.0/10` on `tailscale0`.
- `tailscale status` lists the other office as a peer, path `direct`.
- `tailscale ip -4` is the VTEP address you will pass to VXLAN.

Ping the peer's Tailscale address from head office before you build the overlay. That ping is the underlay.

VXLAN on the tailnet

```
HO=$(docker exec clab-ts-vxlan-ho-gw tailscale ip -4 | head -n1)
BO=$(docker exec clab-ts-vxlan-bo-gw tailscale ip -4 | head -n1)

vxlan_up() {
  local n="$1" local_ip="$2" remote_ip="$3"
  docker exec "$n" bash -c "
    sysctl -w net.ipv4.conf.all.rp_filter=0
    sysctl -w net.ipv4.conf.tailscale0.rp_filter=0
    ip link add br100 type bridge
    ip link set br100 mtu 1230
    ip link set br100 up
    ip link add vxlan100 type vxlan id 100 \
      local ${local_ip} remote ${remote_ip} \
      dev tailscale0 dstport 4789
    ip link set vxlan100 mtu 1230
    ip link set vxlan100 up
    ip link set eth1 up
    ip link set eth1 master br100
    ip link set vxlan100 master br100
  "
}

vxlan_up clab-ts-vxlan-ho-gw "$HO" "$BO"
vxlan_up clab-ts-vxlan-bo-gw "$BO" "$HO"

docker exec clab-ts-vxlan-ho-host sh -c '
  ip link set eth1 mtu 1230
  ip addr add 10.10.10.10/24 dev eth1
  ip link set eth1 up
'
docker exec clab-ts-vxlan-bo-host sh -c '
  ip link set eth1 mtu 1230
  ip addr add 10.10.10.20/24 dev eth1
  ip link set eth1 up
'
```

`tailscale0` stays out of `br100`. It is the underlay port VXLAN encapsulates toward. The bridge holds the office LAN and the VTEP.

MTU, IPv4, from the outside in:

Layer	Size
Tailscale tunnel MTU	1280
Outer IPv4 + UDP + VXLAN + inner Ethernet	$20 + 8 + 8 + 14 = 50$
Office LAN MTU	$1280 - 50 = \mathbf{1230}$

A 1230-byte inner IP packet is 1202 bytes of ICMP payload (`ping -s 1202`) plus 8 bytes of ICMP and 20 bytes of IPv4.

Prove the offices share a LAN

```
docker exec clab-ts-vxlan-ho-gw ping -c 2 -W 2 "$BO"
docker exec clab-ts-vxlan-ho-host ping -c 3 -W 2 10.10.10.20
docker exec clab-ts-vxlan-ho-host ip neigh show dev eth1
docker exec clab-ts-vxlan-ho-gw bridge fdb show br100
```

Captures, in two windows, while the host ping runs:

```
docker exec clab-ts-vxlan-inet tcpdump -ni eth1 -c 8 udp port 41641
docker exec clab-ts-vxlan-ho-gw tcpdump -ni tailscale0 -c 4 udp port 4789
```

Predict

- The workstation ping works with no route on either host.
- `ip neigh` on ho-host shows `10.10.10.20` on `eth1`.
- `bridge fdb` shows the branch MAC behind `vxlan100`.
- `inet` sees UDP/41641. It does not see UDP/4789. The VXLAN header is inside the tunnel.
- `tailscale0` on the gateway sees UDP/4789.

Observe

```
docker exec clab-ts-vxlan-ho-gw ip -d link show vxlan100
docker exec clab-ts-vxlan-ho-gw tailscale status
docker exec clab-ts-vxlan-inet tcpdump -ni eth1 -c 3 udp port 4789
```

The third capture should sit until you interrupt it. Silence on 4789 at `inet` is the result you want during a host ping.

Drill — MTU

```
docker exec clab-ts-vxlan-ho-host ping -c 1 -W 2 -M do -s 1202 10.10.10.20
docker exec clab-ts-vxlan-ho-host ping -c 1 -W 2 -M do -s 1203 10.10.10.20
```

Predict

1202 passes. 1203, with don't-fragment set, fails. The office LAN is 1230 bytes, and the next byte does not fit.

Drill — wrong VNI

```
docker exec clab-ts-vxlan-bo-gw ip link set vxlan100 down
docker exec clab-ts-vxlan-bo-gw ip link del vxlan100
docker exec clab-ts-vxlan-bo-gw bash -c "
  ip link add vxlan100 type vxlan id 200 \
    local ${BO} remote ${HO} dev tailscale0 dstport 4789
  ip link set vxlan100 mtu 1230 up
  ip link set vxlan100 master br100
"
docker exec clab-ts-vxlan-ho-host ping -c 2 -W 2 10.10.10.20
```

Predict

The underlay ping between Tailscale addresses still works. The workstation ping stops. VNI 200 is a different segment.

Put VNI 100 back with the `vxlan_up` function before the next drill.

A third office

Two VTEPs can name each other with `remote`. A third gateway is a flood list instead of a single remote. On each VTEP, for every peer:

```
ip link add vxlan100 type vxlan id 100 local "$LOCAL" dev tailscale0 dstport 4789
bridge fdb append 00:00:00:00:00:00 dev vxlan100 dst "$PEER_A"
bridge fdb append 00:00:00:00:00:00 dev vxlan100 dst "$PEER_B"
```

`00:00:00:00:00:00` is the head-end replication entry: broadcast, unknown unicast, and multicast are copied to each listed Tailscale address. Three offices means two copies of every ARP. That cost is why a control plane exists; this chapter stays on the static list so you can see the copies.

Offices that should stay two networks

Give the sites different prefixes when a host at the branch should route to head office, and ARP should stop at the local gateway. The same two Tailscale nodes can advertise those prefixes:

```
tailscale up \
  --login-server=http://203.0.113.10:8080 \
  --auth-key="$KEY" \
  --hostname=head-office \
  --advertise-routes=10.10.0.0/24 \
  --accept-dns=false
```

Approve the route on Headscale (`headscale nodes list`, then the routes subcommand for that node) and start the peer with `--accept-routes`. Head office might be `10.10.0.0/24` and the branch `10.20.0.0/24`. Each workstation uses its gateway as the default route. There is no bridge and no VNI.

Use that pattern when the prefixes differ. Use VNI 100 when you want one `10.10.10.0/24` and working ARP across the two rooms.

Same exercise on a tailnet you already run

The VXLAN lines do not change if Headscale is replaced by a coordination server you already use. On each gateway:

```
tailscale up --auth-key="$TS_AUTHKEY" --hostname=head-office \
  --accept-dns=false --accept-routes=false
```

Pass `--login-server` only when the client should use a server other than the default. Read `tailscale ip -4` and use those addresses as `local` and `remote`. Keep the preauth key in an environment variable, not in `ts-vxlan.clab.yml`.

The official `tailscale/tailscale` image runs userspace networking unless `TS_USERSPACE=false`. Userspace mode has no kernel `tailscale0`, so VXLAN has nothing to bind to. This lab uses the packaged `tailscaled` in a privileged node, which creates the interface.

Failure table

What you see	What to check
`headscale serve` exits at once	`docker exec clab-ts-vxlan-hs headscale serve` in the foreground. The usual causes are a missing `/var/lib/headscale` or a `dns` block the binary rejects.
`users list` has no socket	The server is still migrating. Wait, then read `/var/log` or the foreground log.
`tailscale up` never gets an address	Ping `203.0.113.10` from the gateway. The login URL has to be that address, the same string as `server_url`.
Peer stuck, no `direct`	`tailscale status` and `iptables -S` on the gateway. UDP/41641 has to be allowed on `eth2` and dropped on `eth0`.
Underlay ping works, host ping does not	`ip -d link show vxlan100` (VNI, local, remote, `dev tailscale0`), both ports in `br100`, host MTU 1230.
Host ping dies after a large packet	MTU 1230 on `vxlan100`, `br100`, and both workstations.
ARP never fills	`rp_filter` on `tailscale0` and `all`. Set both to 0.

What you see	What to check
`inet` shows UDP/4789	The VTEP was bound to `eth2` or to `203.0.113.x`. `local` has to be the Tailscale address and `dev` has to be `tailscale0`.

Tear down

```
sudo containerlab destroy -t ts-vxlan.clab.yml --cleanup
```

Headscale's database and the Tailscale state files live in the container writable layer. Destroy removes them with the nodes. The preauth key dies with the database.

Checklist

Check	What it shows
Ping `203.0.113.10` and the peer `.2`	Lab Internet is up
`tailscale ip -4` on both gateways	VTEP addresses exist
Ping the peer Tailscale address	Underlay, before any VNI
`tailscale0` in `ip -d link show vxlan100`	Encap device is the tailnet
Host ping and `ip neigh`	One LAN, VNI 100
UDP/41641 on `inet`, UDP/4789 on `tailscale0`	Header order
`ping -M do -s 1202` and `-s 1203`	1230-byte office MTU

Summary

- The two gateways log in to Headscale and become each other's underlay.
- VNI 100 bridges `10.10.10.10` and `10.10.10.20` across `tailscale0`.
- The lab Internet sees WireGuard. The Tailscale interface sees VXLAN.
- Office MTU is 1230 because the tunnel is 1280 and VXLAN adds 50 bytes of IPv4 overhead.
- A third site is another flood entry. Different prefixes are subnet routes on the same clients, with no bridge.

Fabrics and Multi-Area

Clos Leaf-Spine

Clos Leaf-Spine

Modern data-center fabrics prefer **scale-out leaf-spine (Clos-inspired)** topologies over deep access-aggregation trees. This chapter builds the underlay model and a free Containerlab fabric snack with FRR + Linux endpoints (mid-2026).

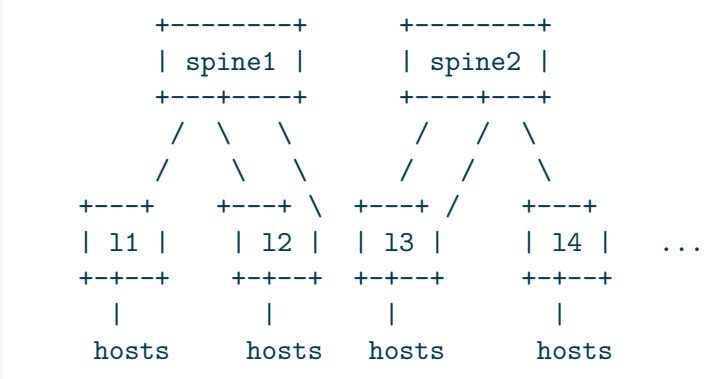
Learning goals

By the end of this chapter you can:

- Explain leaf, spine, and east-west vs north-south roles
- Design a 2-spine × 2-leaf IP underlay with ECMP
- Choose eBGP vs OSPF underlay trade-offs at a model level
- Deploy a Clos-ish lab and verify multipath
- Predict behavior when one spine or one leaf uplink fails

Why Clos-style fabrics

Classic tree pain	Leaf-spine idea
Oversubscription at aggregation	East-west via spines with planned ratio
STP blocking / L2 sprawl	L3 underlay to leaf; overlay for L2 if needed
Hard horizontal scale	Add leaves/spines with more links
Big failure domains	Limit L2; contain blasts



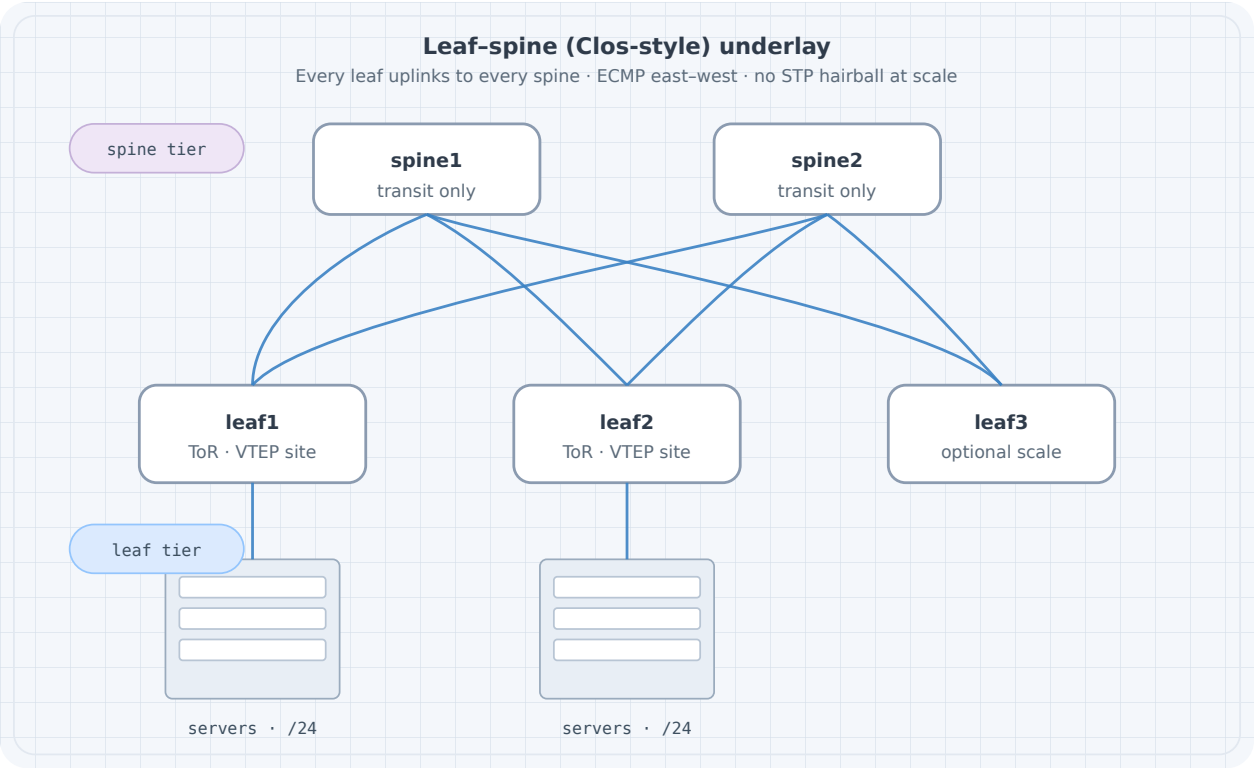


Figure 1: Leaf-spine Clos underlay

Small lab: **2 spines × 2 leaves** (diagram shows a third leaf for scale intuition).

Roles

Role	Function
Leaf	Connects servers; VTEP if overlay; default GW / anycast GW often here
Spine	Transit underlay only (usually no endpoints)
Super-spine / border	Optional higher tier or DC edge (later)

Spines should not become "core with lots of features" on day one—**move packets between leaves**.

Oversubscription (awareness)

```
oversubscription  (sum leaf downlink speeds) / (sum leaf uplink speeds)
```

Lab ignores real bandwidth; production designs document the ratio.

Underlay protocol choices

Underlay	Pros	Cons
eBGP (often unnumbered)	Scale, policy, isolation	More session design
OSPF / IS-IS	Simple adjacency model	Large single domain care
Static	Tiny labs	No scale

This book's first fabric: **OSPF or eBGP on FRR**—pick one and stay consistent. eBGP fabric is popular; OSPF is fine for 2×2 learning.

Addressing plan (2×2)

Node	Loopback	Links
s1	10.0.0.1/32	to l1, l2
s2	10.0.0.2/32	to l1, l2
l1	10.0.0.11/32	to s1, s2; host eth
l2	10.0.0.12/32	to s1, s2; host eth
h1	192.168.1.10/24	via l1
h2	192.168.2.10/24	via l2

Point-to-point `/31` or `/30` on fabric links—example `/30`:

Link	Subnet
l1–s1	10.1.1.0/30
l1–s2	10.1.2.0/30
l2–s1	10.1.3.0/30
l2–s2	10.1.4.0/30

Host subnets via leaf: advertise leaf loopbacks + host subnets into underlay **or** keep hosts only for L3 via leaf and use overlay later.

For pure underlay ECMP lab: ping between leaf loopbacks and between hosts with leaf default routing + OSPF advertised LANs.

Topology YAML

```yaml
name: clos22

topology:
  nodes:
```

```
  s1:
    kind: linux
    image: quay.io/frrouting/frr:10.2.1
    binds:
      - ./config/s1:/etc/frr
  s2:
    kind: linux
    image: quay.io/frrouting/frr:10.2.1
    binds:
      - ./config/s2:/etc/frr
  l1:
    kind: linux
    image: quay.io/frrouting/frr:10.2.1
    binds:
      - ./config/l1:/etc/frr
  l2:
    kind: linux
    image: quay.io/frrouting/frr:10.2.1
    binds:
      - ./config/l2:/etc/frr
  h1:
    kind: linux
    image: alpine:3.20
    exec:
      - apk add --no-cache iproute2 iputils
      - ip addr add 192.168.1.10/24 dev eth1
      - ip link set eth1 up
      - ip route add default via 192.168.1.1
  h2:
    kind: linux
    image: alpine:3.20
    exec:
      - apk add --no-cache iproute2 iputils
      - ip addr add 192.168.2.10/24 dev eth1
      - ip link set eth1 up
      - ip route add default via 192.168.2.1

links:
  - endpoints: ["l1:eth1", "s1:eth1"]
  - endpoints: ["l1:eth2", "s2:eth1"]
  - endpoints: ["l2:eth1", "s1:eth2"]
  - endpoints: ["l2:eth2", "s2:eth2"]
  - endpoints: ["h1:eth1", "l1:eth3"]
  - endpoints: ["h2:eth1", "l2:eth3"]
```

FRR daemons

```
zebra=yes
ospfd=yes
# or bgpd=yes for eBGP underlay
```

OSPF underlay sketch (l1)

```
frr version 10.2.1
hostname l1
!
interface lo
 ip address 10.0.0.11/32
 ip ospf area 0
!
interface eth1
 ip address 10.1.1.1/30
 ip ospf network point-to-point
 ip ospf area 0
!
interface eth2
 ip address 10.1.2.1/30
 ip ospf network point-to-point
 ip ospf area 0
!
interface eth3
 ip address 192.168.1.1/24
 ip ospf area 0
!
router ospf
 ospf router-id 10.0.0.11
 passive-interface eth3
!
line vty
```

Spines: loopbacks + fabric links in area 0; **no host interfaces**. l2 mirrors with its addresses. Point-to-point OSPF avoids DR drama on p2p links.

eBGP underlay sketch (optional alternate)

```
router bgp 65101
 bgp router-id 10.0.0.11
 neighbor 10.1.1.2 remote-as 65100
 neighbor 10.1.2.2 remote-as 65100
 address-family ipv4 unicast
  network 10.0.0.11/32
  network 192.168.1.0/24
  neighbor 10.1.1.2 activate
  neighbor 10.1.2.2 activate
  maximum-paths 4
 exit-address-family
```

Spines use ASN 65100 (or each spine unique ASN—design choice). Leaves unique ASNs. Enable multipath.

Deploy and ECMP verify

```
sudo containerlab deploy -t clos22.clab.yml

docker exec clab-clos22-l1 vtysh -c 'show ip ospf neighbor'
docker exec clab-clos22-l1 vtysh -c 'show ip route 10.0.0.12'
docker exec clab-clos22-l1 vtysh -c 'show ip route 192.168.2.0'
docker exec clab-clos22-h1 ping -c 3 192.168.2.10
docker exec clab-clos22-h1 traceroute -n 192.168.2.10
```

Predict

- Each leaf: 2 OSPF neighbors (both spines)
- Route to remote leaf/host shows **multiple nexthops** (ECMP) if multipath installed
- traceroute may show s1 or s2

Observe multipath

```
docker exec clab-clos22-l1 ip route show 192.168.2.0/24
# or
docker exec clab-clos22-l1 vtysh -c 'show ip route 192.168.2.0/24'
```

FRR/zebra may need `maximum-paths` under OSPF:

```
router ospf
 maximum-paths 4
```

Drill 1 — Spine loss

```
docker exec clab-clos22-s1 ip link set eth1 down
docker exec clab-clos22-s1 ip link set eth2 down
sleep 3
docker exec clab-clos22-l1 vtysh -c 'show ip route 192.168.2.0'
docker exec clab-clos22-h1 ping -c 20 192.168.2.10
```

Predict → observe → fix

Predict | ECMP shrinks to remaining spine; brief loss |
Observe | single nexthop via s2 |
Fix | if blackhole, check l2 uplink to s2 and SPF |

Restore interfaces after.

Drill 2 — One leaf uplink down

```
docker exec clab-clos22-l1 ip link set eth1 down
docker exec clab-clos22-l1 vtysh -c 'show ip ospf neighbor'
docker exec clab-clos22-h1 traceroute -n 192.168.2.10
```

Predict

Still reachable via other spine; neighbor count 1.

Drill 3 — Polarization awareness

Send many parallel iperf flows; without entropy, paths may skew. Underlay traceroute from different sources may stick to one spine. Note in journal; overlays later add UDP entropy.

Pattern	Path

East-west vs north-south

Pattern	Path
East-west h1 h2	leaf → spine → leaf
North-south to Internet	leaf → border/spine path to edge

Do not attach WAN carelessly to all spines without a border design.

Overlay placement (preview)

VTEPs usually on **leaves**. Spines stay underlay-only. That separation keeps failure domains clean—connects to VXLAN/EVPN chapters.

Verification checklist

Check	Intent
All fabric adjacencies	Full mesh leaf-spine links up
Loopback mesh	Any lo to any lo
ECMP present	2 nexthops where topology allows
Host traffic	Data plane
Spine failure	Continuity

Common mistakes

Mistake	Symptom
Hosts on spines	Weird roles, scale pain
L2 across spines	Back to tree problems
Different OSPF areas accidentally	Missing routes
No p2p network type	Extra DR complexity
Expecting overlay without VTEP	Hosts same L2 fantasy

Summary

- Leaf-spine Clos-style fabrics scale east-west with L3 underlays
- Spines transit; leaves attach endpoints (and VTEPs)
- 2×2 FRR lab is enough to feel ECMP and spine loss
- Pick OSPF or eBGP underlay deliberately
- Keep overlays on leaves in later designs

Next: **multi-area OSPF**—when a single area is no longer the right underlay shape (campus/WAN more than fabric).

Multi-Area OSPF

Multi-Area OSPF

Single-area OSPF is the right first lab. Multi-area OSPF exists to **bound flooding, summarize, and structure large domains**. This chapter extends FRR OSPF into a small multi-area topology in Containerlab (mid-2026 free path)—useful for campus/WAN; fabrics often prefer BGP underlays instead.

Learning goals

By the end of this chapter you can:

- Explain area, ABR, ASBR, and LSA-type roles at operator level
- Build a three-router multi-area lab (backbone 0 + spoke area)
- Summarize at an ABR and predict reachability holes
- Contrast stub-ish ideas without drowning in edge-case flags
- Verify with database, route, and failure drills

Concepts

Why areas

Single area	Multi-area
Every router full topology	Topology flooding scoped
SPF on full graph	Smaller SPF domains
Simple	Needs backbone rules

Backbone rule (practical)

- Area **0** is the backbone
- Non-backbone areas attach via **ABRs** that touch area 0
- Virtual links exist for scars—not for greenfield labs

Roles

Role	Meaning
ABR	Router with interfaces in multiple areas; can summarize
ASBR	Injects external routes (redistribution)
Internal	All interfaces one area

LSA awareness (not flash cards)

Type idea	Carries
Router / network	Topology inside area
Summary (ABR)	Inter-area prefixes
External	From outside OSPF

You operate with: `show ip ospf database`, `show ip ospf border-routers`, `show ip route`.

Summarization

ABR can advertise `10.1.0.0/16` instead of many `/24`s. Mis-summary → **blackholes** if holes exist inside the aggregate.

Topology

```
     area 1              area 0              area 2
h1--r1---------------abr-r2---------------r3--h2
    (1.1.1.1)         (2.2.2.2)          (3.3.3.3)
```

Node	Areas	Notes
r1	1 only	internal
r2	0 + 1 + 2	ABR
r3	2 only	internal

Links:

Link	Subnet	Area
r1–r2	10.0.12.0/24	1
r2–r3	10.0.23.0/24	2
r1 LAN	192.168.1.0/24	1
r3 LAN	192.168.3.0/24	2

Link	Subnet	Area
r2 lo	2.2.2.2/32	0

Optional second area-0 link later; one ABR is enough to learn.

Topology YAML

```yaml
name: ospf-ma

topology:
  nodes:
    r1:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/r1:/etc/frr
    r2:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/r2:/etc/frr
    r3:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/r3:/etc/frr
    h1:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 192.168.1.10/24 dev eth1 && ip link set eth1 up
        - ip route add default via 192.168.1.1
    h2:
      kind: linux
      image: alpine:3.20
      exec:
        - apk add --no-cache iproute2 iputils
        - ip addr add 192.168.3.10/24 dev eth1 && ip link set eth1 up
        - ip route add default via 192.168.3.1

  links:
    - endpoints: ["r1:eth1", "r2:eth1"]
    - endpoints: ["r2:eth2", "r3:eth1"]
```

```
    - endpoints: ["h1:eth1", "r1:eth2"]
    - endpoints: ["h2:eth1", "r3:eth2"]
```

FRR configs

r1 (area 1)

```
hostname r1
!
interface lo
 ip address 1.1.1.1/32
 ip ospf area 1
!
interface eth1
 ip address 10.0.12.1/24
 ip ospf area 1
!
interface eth2
 ip address 192.168.1.1/24
 ip ospf area 1
!
router ospf
 ospf router-id 1.1.1.1
 passive-interface eth2
!
line vty
```

r3 (area 2)

Mirror with `3.3.3.3`, `10.0.23.2/24` area 2, LAN `192.168.3.1/24` area 2.

r2 (ABR)

```
hostname r2
!
interface lo
 ip address 2.2.2.2/32
 ip ospf area 0
!
interface eth1
 ip address 10.0.12.2/24
 ip ospf area 1
```

```
!
interface eth2
 ip address 10.0.23.1/24
 ip ospf area 2
!
router ospf
 ospf router-id 2.2.2.2
 area 1 range 192.168.0.0/16
 area 2 range 192.168.0.0/16
!
line vty
```

Careful: broad `192.168.0.0/16` summary is a teaching weapon—it can hide holes. Prefer tight ranges:

```
 area 1 range 192.168.1.0/24
 area 2 range 192.168.3.0/24
```

Or summarize only when you own contiguous blocks.

Deploy and baseline

```
sudo containerlab deploy -t ospf-ma.clab.yml

docker exec clab-ospf-ma-r2 vtysh -c 'show ip ospf neighbor'
docker exec clab-ospf-ma-r2 vtysh -c 'show ip ospf border-routers'
docker exec clab-ospf-ma-r1 vtysh -c 'show ip ospf database'
docker exec clab-ospf-ma-r1 vtysh -c 'show ip route'
docker exec clab-ospf-ma-h1 ping -c 3 192.168.3.10
```

Predict

- r2 has neighbors in area 1 and 2
- r1 sees inter-area path to 192.168.3.0/24 via r2
- Database on r1 does **not** contain full area 2 topology detail (summary only)

Observe

Compare `show ip ospf database` on r1 vs r2. r2 should show more topology.

Drill 1 — Summarization blackhole

On r2:

```
area 1 range 192.168.0.0/16
```

Do **not** put a real 192.168.99.0/24 anywhere, but inject a static on r1 into OSPF… or simpler: summarize too wide then withdraw a component.

Teaching sequence:

1. Advertise 192.168.1.0/24 only.
2. Add `area 1 range 192.168.0.0/16`.
3. On r3, install a static to 192.168.99.1 via null or missing next hop and redistribute—or ping 192.168.99.1 from r3 side expecting blackhole from r1 if summary exists without component.

Simpler blackhole lab:

```
# r2 summarizes 192.168.0.0/16 from area 1 while only .1.0/24 exists
# from r3 ping 192.168.1.10 works; ping 192.168.50.10 fails (good)
# If r3 thinks 192.168.50.0 is under summary toward area1, traffic blackholes at ABR
```

Predict

Summaries without covering specifics attract traffic to ABR then drop.

Fix

Tight ranges; discard routes; or don't summarize.

Drill 2 — ABR failure

```
docker stop clab-ospf-ma-r2
docker exec clab-ospf-ma-h1 ping -c 5 192.168.3.10
```

Predict

Total partition—single ABR is SPOF.
Design lesson

Redundant ABRs for real campus; dual attachment to area 0.

Drill 3 — Area mismatch

Put r1 eth1 in area 0 by mistake:

```
interface eth1
 ip ospf area 0
```

Predict

Adjacency fails or forms wrong; routes missing.
Observe

`show ip ospf neighbor` empty/wrong.
Fix

Restore area 1 both sides.

Stub area ideas (awareness)

Stub/totally stubby/NSSA reduce external LSAs into an area. FRR supports variants—use when external flood is noisy. For this chapter, know **why** (less type-5 in area) more than every flag combo.

```
router ospf
 area 1 stub
```

Retest default injection behavior when you try it.

Multi-area vs fabric BGP

Multi-area OSPF	Clos eBGP underlay
Campus/WAN hierarchy	DC scale-out
ABR summarization	Prefix + policy at leaves
Flooding domains	AS boundary policy

Do not force multi-area OSPF onto a leaf-spine if eBGP underlay is the house style.

Check	Command

Verification checklist

Check	Command
Neighbors per area	`show ip ospf neighbor`
ABR role	`show ip ospf border-routers`
DB scope	`show ip ospf database`
Inter-area routes	`show ip route` IA flags
Data plane	ping h1 h2

Common mistakes

Mistake	Symptom
Discontiguous area 0	Weird inter-area failures
Summary too broad	Blackholes
Expecting full topology remote area	Confusion reading DB
Redistribute without filters	Loops/externals everywhere
Single ABR production	Outage on maintenance

Summary

- Multi-area OSPF bounds flooding and enables ABR summarization
- Area 0 backbone rules still matter in greenfield design
- Always verify **database scope** and **inter-area routes**, not only neighbors
- Summaries can blackhole—tight ranges and drills
- Prefer BGP fabrics for Clos; multi-area OSPF for hierarchical campus/WAN

Next: **fabric with FRR**—package a tighter leaf-spine implementation path and verification kit.

Fabric with FRR

Fabric with FRR

This chapter turns the Clos model into an **FRR-operable fabric kit**: config layout, unnumbered eBGP option, verification scripts, and failure drills. Open stack only—Containerlab + FRR + Alpine (mid-2026). Overlay remains optional on leaves.

Learning goals

By the end of this chapter you can:

- Organize a fabric lab repo for N leaves × M spines
- Implement OSPF or eBGP underlay consistently on FRR
- Use `maximum-paths` and read multipath in RIB/FIB
- Smoke-test a fabric with a single `verify.sh`
- Know where to bolt VXLAN/EVPN on leaves later

Lab repo layout

```
labs/fabric-frr/
  clos22.clab.yml
  addressing.md
  config/
    s1/frr.conf daemons
    s2/...
    l1/...
    l2/...
  scripts/
    render.sh      # optional template expander
    verify.sh
    fail-spine.sh
  captures/
  README.md
```

Templating (jinja/make/python) pays off at 4×4; 2×2 can be hand-written.

Design defaults for this book

Choice	Default
Size	2 spine × 2 leaf
Underlay	OSPF area 0 p2p **or** eBGP
Loopbacks	10.0.0.x/32
Hosts	one per leaf, /24
Overlay	off (underlay first)
Images	`quay.io/frrouting/frr:10.2.1` + `alpine:3.20`

eBGP unnumbered (modern fabric style)

When link-local + IPv6 RA or RFC5549-style patterns available, unnumbered BGP reduces IP inventory. FRR support has matured—verify on your image:

```
interface eth1
 ip address 10.1.1.1/30
!
router bgp 65101
 neighbor fabric peer-group
 neighbor fabric remote-as external
 neighbor fabric capability extended-nexthop
 neighbor eth1 interface peer-group fabric
 !
 address-family ipv4 unicast
  neighbor fabric activate
  network 10.0.0.11/32
 exit-address-family
```

If interface peering is painful in your build, use explicit `/30` neighbors (Clos chapter)—**same topology math**.

Numbered eBGP fabric (reliable teaching)

Node	ASN
s1	65100
s2	65100
l1	65101
l2	65102

Spines same ASN (or not—both designs exist). Leaves unique.

Spine s1 excerpt

```
hostname s1
!
interface lo
 ip address 10.0.0.1/32
!
interface eth1
 ip address 10.1.1.2/30
!
interface eth2
 ip address 10.1.3.2/30
!
router bgp 65100
 bgp router-id 10.0.0.1
 neighbor 10.1.1.1 remote-as 65101
 neighbor 10.1.3.1 remote-as 65102
 !
 address-family ipv4 unicast
  network 10.0.0.1/32
  neighbor 10.1.1.1 activate
  neighbor 10.1.3.1 activate
  maximum-paths 4
 exit-address-family
!
line vty
```

Leaf l1 excerpt

```
hostname l1
!
interface lo
 ip address 10.0.0.11/32
!
interface eth1
 ip address 10.1.1.1/30
!
interface eth2
 ip address 10.1.2.1/30
!
interface eth3
 ip address 192.168.1.1/24
!
router bgp 65101
 bgp router-id 10.0.0.11
```

```
 neighbor 10.1.1.2 remote-as 65100
 neighbor 10.1.2.2 remote-as 65100
 !
 address-family ipv4 unicast
  network 10.0.0.11/32
  network 192.168.1.0/24
  neighbor 10.1.1.2 activate
  neighbor 10.1.2.2 activate
  maximum-paths 4
 exit-address-family
!
line vty
```

`daemons`: `zebra=yes`, `bgpd=yes`.

OSPF alternate (same wiring)

Reuse Clos chapter OSPF configs; add:

```
router ospf
 maximum-paths 4
```

Declarative Fabric with Netlab (Fast & Scalable)

For Clos fabrics, authoring individual router configs and IPAM spreadsheets for every spine and leaf switch quickly becomes error-prone. With Netlab (`provider: clab`), you declare the leaf/spine roles, ASNs, and links; Netlab generates the entire underlay, assigns loopback/transit IPs, enables ECMP multipath, and runs automated health checks:

```
# topology.yml
---
provider: clab
defaults.device: frr
module: [ bgp, bfd ]

nodes:
  s1: { role: spine, bgp.as: 65100 }
  s2: { role: spine, bgp.as: 65100 }
  l1: { role: leaf,  bgp.as: 65101 }
  l2: { role: leaf,  bgp.as: 65102 }
  h1: { device: linux }
  h2: { device: linux }

links:
```

```
  - l1:
    s1:
  - l1:
    s2:
  - l2:
    s1:
  - l2:
    s2:
  - h1:
    l1:
  - h2:
    l2:

validate:
  bgp_established:
    description: "Verify each leaf has 2 active eBGP sessions to spines"
    nodes: [ l1, l2 ]
    plugin: bgp_neighbor(count=2, state='Established')
  end_to_end_hosts:
    description: "Verify host-to-host traffic traverses the fabric"
    nodes: [ h1 ]
    exec:
      ping_h2: ping -c 3 10.0.0.6
```

Deploy and Verify with Netlab

```
# Generate configs, calculate IPAM, and deploy in Containerlab
netlab up

# Run automated validation across all BGP sessions and host pings
netlab validate

# Inject link failure drill: drop link between leaf1 and spine1
netlab tc set l1 eth1 --loss 100%
netlab validate
netlab tc clear l1 eth1
```

Manual Containerlab & FRR Implementation

If deploying without Netlab, you must manage individual configuration files and volume mounts. Below is the manual configuration pattern for reference and deep-dive troubleshooting.

Containerlab topology (`clos22.clab.yml`)

Same as Clos chapter `clos22`—keep one file, swap configs via binds.

Operational show kit

```
# sessions / neighbors
docker exec clab-clos22-l1 vtysh -c 'show bgp summary'
docker exec clab-clos22-l1 vtysh -c 'show ip ospf neighbor'

# multipath
docker exec clab-clos22-l1 vtysh -c 'show ip route 192.168.2.0/24'
docker exec clab-clos22-l1 ip route show 192.168.2.0/24

# loopback mesh
for d in 10.0.0.1 10.0.0.2 10.0.0.11 10.0.0.12; do
  docker exec clab-clos22-l1 ping -c1 -W1 $d || echo fail $d
done

# hosts
docker exec clab-clos22-h1 ping -c2 192.168.2.10
```

verify.sh

```
#!/usr/bin/env bash
set -euo pipefail
fail() { echo "FAIL: $*" >&2; exit 1; }
P=clab-clos22

docker exec $P-l1 ping -c1 -W1 10.0.0.12 || fail "l1->l2 lo"
docker exec $P-l1 ping -c1 -W1 10.0.0.1 || fail "l1->s1 lo"
docker exec $P-h1 ping -c2 -W1 192.168.2.10 || fail "h1->h2"
docker exec $P-h2 ping -c2 -W1 192.168.1.10 || fail "h2->h1"

# multipath soft check: at least one route line with multiple nexthops or two entries
R=$(docker exec $P-l1 vtysh -c 'show ip route 192.168.2.0/24' || true)
echo "$R" | grep -Eq '192.168.2.0|via' || fail "no route to h2 subnet"

echo OK-FABRIC
```

Failure scripts

```
#!/usr/bin/env bash
# fail-spine.sh s1
set -euo pipefail
SP=${1:-s1}
docker exec clab-clos22-$SP ip link set eth1 down
docker exec clab-clos22-$SP ip link set eth2 down
echo "$SP uplinks down; run ping/traceroute; then restore"
```

```
#!/usr/bin/env bash
# restore-spine.sh s1
SP=${1:-s1}
docker exec clab-clos22-$SP ip link set eth1 up
docker exec clab-clos22-$SP ip link set eth2 up
```

Predict → observe → fix

Drill	Predict	Observe
Spine down	Paths via other spine	single nexthop
Leaf uplink down	Still works	1 BGP/OSPF neighbor
Wrong ASN	Session down	`show bgp summary` Active/Idle
Missing network stmt	Host subnet absent	RIB missing

Policy on fabric (light)

Even underlays need hygiene:

```
# do not accept default from random peers in fabric
# do not advertise lab management prefixes
ip prefix-list FABRIC-LO seq 5 permit 10.0.0.0/24 le 32
route-map EXPORT-FABRIC permit 10
 match ip address prefix-list FABRIC-LO
```

Apply out toward spines if you practice policy discipline.

Adding overlay later (hook points)

On each leaf:

1. Keep underlay BGP/OSPF as-is

2. Add VTEP on loopback

3. EVPN AF on BGP or static VXLAN to other leaf loopbacks

4. Spines remain ignorant of VNIs

```
leaf = underlay router + VTEP + (optional) IRB
spine = underlay router only
```

Scale checklist 2×2 → 4×4

Item	Action
IP plan	Spreadsheet / `addressing.md` generator
Configs	Templates
RAM	FRR light; watch host memory
verify	Loop all leaf pairs
Cabling matrix	Automate clab links generation

Observability

- Enable FRR logging to bound files
- Graph: `containerlab graph -t clos22.clab.yml`
- Optional: scrape with your ops chapter tools

Optional SONiC / SR Linux leaf

Replace one leaf image with community NOS; keep underlay protocol same. Document CLI mapping for `show bgp` / `show ip route`. Goal: **multi-implementation confidence** (next chapter deepens this).

Common mistakes

Mistake	Symptom
Spines advertising host defaults weirdly	Traffic trombones
maximum-paths left at 1	No ECMP
Overlay on spine	Operational regret
No cold deploy test	Config drift in running only
Same router-id	BGP/OSPF chaos

Mistake	Symptom

Summary

- FRR is enough to run a real teaching fabric underlay
- Standardize repo layout, ASNs, and verify scripts
- Prove ECMP and spine loss before overlays
- Leaves are the VTEP attachment point later
- Policy-light underlays still need prefix discipline

Next: **multi-implementation lab**—FRR + Linux + optional SR Linux in one topology.

Multi-Implementation Lab

Multi-Implementation Lab

Vendor-agnostic skill means the **same design** survives different CLIs. This capstone-style lab mixes **FRR, Linux endpoints, and optional Nokia SR Linux (community)** in one Containerlab topology (mid-2026 free/community images only).

Learning goals

By the end of this chapter you can:

- Run a leaf-spine underlay with heterogeneous leaf implementations
- Map show-command dialects to the same verification questions
- Keep addressing, ASN, and success metrics identical across nodes
- Document differences without abandoning models
- Produce a multi-NOS lab report

Design principle

```
Same:
  - topology roles (spine/leaf/host)
  - ASNs / areas / loopbacks
  - prefixes advertised
  - failure drills

Different:
  - config language
  - show syntax
  - feature knobs names
```

If behavior diverges, **stop and explain** (bug, knobs, or design mismatch)—do not "fix it" with silent statics on one leaf only.

Recommended topology

```
          s1 (FRR)      s2 (FRR)
          /    \        /    \
         /      \      /      \
   l1 (FRR)      l2 (SR Linux community OR FRR)
      |                 |
     h1                h2
```

If SR Linux image pull is unavailable, run **l2 as FRR** and still complete the dual-leaf discipline—or use VyOS/SONiC community kinds if present in your Containerlab kind list. The **process** matters.

Addressing / ASN (fixed)

Node	Kind	lo	ASN
s1	FRR	10.0.0.1/32	65100
s2	FRR	10.0.0.2/32	65100
l1	FRR	10.0.0.11/32	65101
l2	SR Linux or FRR	10.0.0.12/32	65102
h1	Alpine	192.168.1.10/24	—
h2	Alpine	192.168.2.10/24	—

Fabric links: same `/30` map as Clos chapter.

Topology YAML (FRR + optional srl)

```yaml
name: multi-imp

topology:
  nodes:
    s1:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/s1:/etc/frr
    s2:
      kind: linux
      image: quay.io/frrouting/frr:10.2.1
      binds:
        - ./config/s2:/etc/frr
    l1:
```

```
    kind: linux
    image: quay.io/frrouting/frr:10.2.1
    binds:
      - ./config/l1:/etc/frr
  l2:
    # Option A - all FRR:
    kind: linux
    image: quay.io/frrouting/frr:10.2.1
    binds:
      - ./config/l2:/etc/frr
    # Option B - SR Linux community (uncomment/adjust per containerlab docs):
    # kind: nokia_srlinux
    # image: ghcr.io/nokia/srlinux:latest   # pin a known free tag in real labs
    # binds: ...
  h1:
    kind: linux
    image: alpine:3.20
    exec:
      - apk add --no-cache iproute2 iputils
      - ip addr add 192.168.1.10/24 dev eth1 && ip link set eth1 up
      - ip route add default via 192.168.1.1
  h2:
    kind: linux
    image: alpine:3.20
    exec:
      - apk add --no-cache iproute2 iputils
      - ip addr add 192.168.2.10/24 dev eth1 && ip link set eth1 up
      - ip route add default via 192.168.2.1

links:
  - endpoints: ["l1:eth1", "s1:eth1"]
  - endpoints: ["l1:eth2", "s2:eth1"]
  - endpoints: ["l2:eth1", "s1:eth2"]
  - endpoints: ["l2:eth2", "s2:eth2"]
  - endpoints: ["h1:eth1", "l1:eth3"]
  - endpoints: ["h2:eth1", "l2:eth3"]
```

Pin image tags in real work; `latest` is a lab convenience only. Consult Containerlab kinds for current SR Linux free image references.

FRR side

Reuse fabric-with-FRR spine/leaf configs. Spines must peer to both leaves regardless of leaf NOS.

SR Linux mapping (conceptual)

Intent	FRR-ish	SR Linux-ish
Router id / lo	`interface lo` + BGP router-id	system / network-instance loopback
Interface IP	`ip address`	interface subinterface IPv4
eBGP neighbor	`neighbor A.B.C.D remote-as`	bgp neighbor under network-instance
Advertise lo + LAN	`network` statements	export policy + prefixes
Multipath	`maximum-paths`	ECMP / multipath knobs
Verify BGP	`show bgp summary`	`show network-instance default protocols bgp …` (version-specific)
Verify route	`show ip route`	`show route` / fib show variants

Exact paths change by SR Linux release—**write the mapping table in your lab README** with the commands that worked for you.

Verification questions (dialect-free)

Answer these on **every** leaf and spine:

1. Are fabric links up at L1/L2?
2. Is the underlay protocol adjacent to both spines?
3. Is my loopback in the fabric RIB on the far leaf?
4. Is the remote host subnet reachable with ECMP?
5. Do hosts pass bidirectional ICMP?
6. After killing s1, does traffic continue?

FRR answers

```
docker exec clab-multi-imp-l1 vtysh -c 'show bgp summary'
docker exec clab-multi-imp-l1 vtysh -c 'show ip route 192.168.2.0/24'
```

SR Linux answers (illustrative)

```
# container name pattern depends on clab naming
docker exec clab-multi-imp-l2 sr_cli 'show interface'
docker exec clab-multi-imp-l2 sr_cli 'show network-instance default route-table'
# adjust to your image's CLI entrypoint (sr_cli / srlinux)
```

predict → observe → fix matrix

Drill	Predict	FRR observe	SRL observe
Deploy cold	all sessions up	bgp summary	bgp show
h1→h2 ping	ok	ping	ping from h2 side
s1 down	ok via s2	single path	single path
shutdown l2 LAN network	h2 isolated	route gone on l1	route gone
MTU 1400 fabric	large ping fail	ping -M do	same

verify.sh (heterogeneous)

```
#!/usr/bin/env bash
set -euo pipefail
fail() { echo "FAIL: $*" >&2; exit 1; }
P=clab-multi-imp

docker exec $P-h1 ping -c2 -W1 192.168.2.10 || fail "h1->h2"
docker exec $P-h2 ping -c2 -W1 192.168.1.10 || fail "h2->h1"
docker exec $P-l1 ping -c1 -W1 10.0.0.12 || fail "l1->l2 lo"

# FRR leaf sessions
docker exec $P-l1 vtysh -c 'show bgp summary' | grep -q '65100' \
  || fail "l1 missing spine asn in summary"

echo OK-MULTI
```

Extend with SRL-specific checks when l2 is SR Linux.

Overlay optional add-on

Once underlay multipath works on both implementations:

- Static VXLAN between l1 and l2 loopbacks (Linux vxlan on FRR node may need extra tooling; pure FRR leaf might use other integration)
- Or EVPN only if **both** leaves support it—heterogeneous EVPN is an advanced interop project, not a first lab

Declare non-goals if overlay skipped.

Hardening gates (reuse part 08)

- ☐ Only fabric prefixes advertised
- ☐ Mgmt not exposed on fabric links
- ☐ Image tags pinned
- ☐ destroy/recreate clean

Lab report template

```
# Multi-implementation fabric
Date:
Images: FRR x.y, SR Linux z (or FRR-only fallback)
Underlay: eBGP numbered
Results:
- cold verify.sh: PASS/FAIL
- spine failure: notes
- CLI mapping table: attached
Differences found:
Next improvements:
```

When interop hurts

Symptom	Approach
Session up, no routes	policy / AF / network statements
ECMP one side only	multipath knobs asymmetric
Hosts one-way	RPF, missing return prefix
SRL slow boot	wait; recheck before debug spiral

Time-box interop: 2 hours then fall back to dual FRR with a written "SRL blocked on X" note.

Common mistakes

Mistake	Symptom
Different ASNs than plan	sessions down
Comparing wrong show outputs	false failure
Fixing only FRR side	asymmetric design
Unpinned SRL image	lab broke next month
Skipping host bidirectional test	false green

Summary

- Multi-implementation labs prove **models over CLI**
- Freeze addressing and success metrics; vary only NOS
- Maintain a personal show-command dictionary
- Prefer underlay interop first; EVPN interop later
- FRR-only fallback still completes the fabric curriculum

You now have published depth from edge services through fabrics. Return to ops automation capstones and harden the labs you actually keep.

BGP Security

Why Routing Security Is Harder Than It Looks

Why Routing Security Is Harder Than It Looks

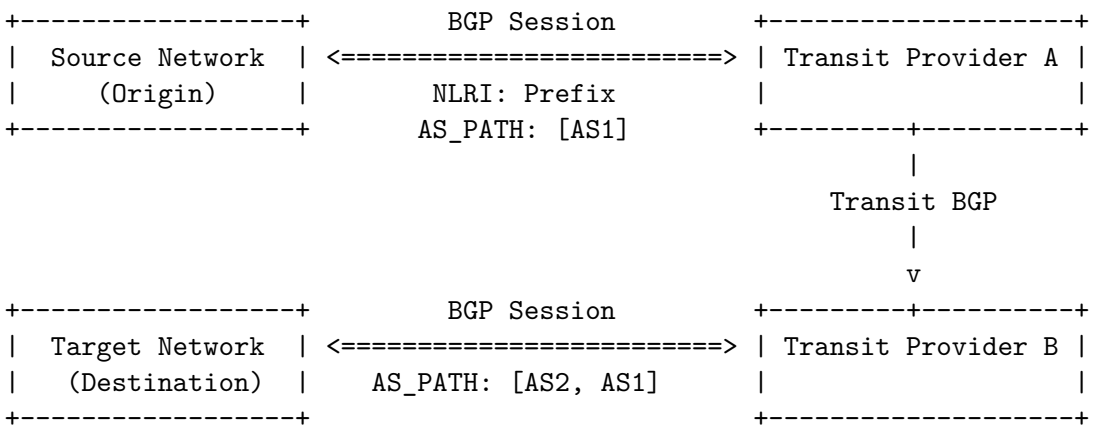
The global Internet is composed of over 75,000 distinct Autonomous Systems (ASes) bound together by a single interdomain routing protocol: the Border Gateway Protocol (BGP-4). Every packet routed across international borders, between cloud providers, financial institutions, and edge consumers depends entirely on the accuracy of reachability assertions exchanged over BGP. Yet, the foundational protocol underpinning this planetary nervous system was architected in 1989 without cryptographic authenticity, origin authorization, or path validation.

At first glance, securing interdomain routing appears to be a straightforward cryptographic problem: sign the IP prefix allocations, sign the routing announcements, and reject any routing update that fails mathematical verification. In reality, routing security is one of the most stubborn, politically fraught, and mechanically difficult distributed systems problems in computing.

The Architectural Paradox

The core difficulty of routing security stems from a fundamental paradox: **the Internet is an federation of sovereign, competing commercial entities operating without a centralized authority, yet global reachability requires universal cooperation.**

No single government, regulatory agency, or corporate entity owns the Internet. Networks peer with one another based on bilateral contracts, informal handshakes, and market incentives. A network operator in Frankfurt has no contractual relationship with, and zero jurisdictional leverage over, a transit network operating in Vladivostok or Montevideo. However, when traffic traverses the global default-free zone (DFZ), that Frankfurt network must trust that updates propagated across five intermediary transit hops have not been spoofed, stripped, leaked, or poisoned.



Because BGP is inherently a distance-vector (path-vector) protocol, routers make local forwarding decisions based on what their neighbors tell them, not on direct observation of the physical topology. If a router receives an advertisement claiming a shorter or more economically attractive path to a destination, its default algorithmic imperative is to accept the update and redirect traffic accordingly.

Why Cryptographic Patches Fall Short

Securing routing cannot be achieved simply by wrapping packets in TLS or signing messages with standard public-key infrastructure (PKI):

1. **Decoupling of the Control Plane and Data Plane**: A BGP routing update is an assertion of reachability in the control plane. Forwarding occurs in the data plane. Validating that an AS is authorized to advertise an IP prefix does *not* guarantee that packets forwarded to that AS will actually reach the destination, nor does it prevent an adversary from dropping, inspecting, or duplicating the traffic once received.
2. **Asymmetric Routing and Incomplete Visibility**: Internet traffic rarely traverses symmetric paths. The outbound path taken by a packet from host A to host B frequently traverses entirely different autonomous systems than the return path from B to A. Consequently, local endpoints lack end-to-end visibility into the hops traversed by their packets.
3. **The Longevity of Legacy Assumptions**: Routers are hardware appliances optimized for wire-speed packet forwarding in silicon (ASICs). Control-plane processors on edge and core routers are historically modest in compute capacity. Cryptographic verification of hundreds of thousands of digital signatures per second during a route burst or flap storm can exhaust control-plane CPU, inducing cascading session failures.
4. **Incremental Deployment Dynamics**: Security mechanisms that require ubiquitous adoption before delivering value fail on the Internet. Any viable defense must provide tangible protection to early adopters even while 80% of the Internet remains legacy and insecure.

Economic Incentives and Asymmetry

Routing policies on the Internet are dictated by business relationships, colloquially formalized by the **Gao-Rexford model**:

Relationship	Export Policy	Economic Incentive
Customer → Provider	Export own routes and customer routes only	Customer pays provider for transit; seeks widest reach
Provider → Customer	Export all known routes (full table or default)	Provider charges customer for traffic volume
Peer ↔ Peer	Export own routes and customer routes only	Settlement-free; neither party pays; transit is forbidden

Routing security mechanisms must coexist with these incentives. For example, route leaks—where an operator accidentally advertises routes learned from one transit provider to another transit

provider—are often not malicious attacks, but simple filter omissions. Yet, because provider-to-provider transit is economically toxic, such leaks suck massive volumes of global traffic into networks that lack the physical capacity to carry it, causing catastrophic outages.

Roadmap of This Deep Dive

This part delivers a rigorous, production-grade examination of global routing architecture, vulnerabilities, cryptographic defenses, and operational implementation:

- **Chapters 1–6**: Foundations of global connectivity, interdomain routing mechanics, BGP protocol internals, path attributes, decision logic, and convergence dynamics.
- **Chapters 7–11**: The threat landscape, historical architectural flaws, origin prefix hijacks, subprefix interception, route leaks, and emerging state-actor vectors.
- **Chapters 12–18**: Public-Key Infrastructure for routing (RPKI), Resource Certificates, ROAs, relying party validators, RTR protocol, and Route Origin Validation (ROV) in operations.
- **Chapters 19–22**: Defense in depth—IRR/RPSL filtering, AS_PATH filters, automated route leak protection, BGPsec reality, and Autonomous System Provider Authorizations (ASPA).
- **Chapters 23–26**: Engineering blueprints, multi-vendor configuration matrices (Cisco IOS XR, Juniper Junos, Nokia SR OS, FRR, BIRD), troubleshooting playbooks, and global telemetry monitoring.
- **Chapters 27–29**: Historical forensics of landmark hijacking incidents, economic governance, MANRS, and the future trajectory of Internet routing security.

The Architecture of Global Connectivity

The Architecture of Global Connectivity

Interdomain routing operates across administrative, organizational, and geopolitical frontiers. To understand how routing breaks and how to secure it, we must first establish the physical, administrative, and economic foundations of global Internet connectivity.

IP Addressing and the Prefix Hierarchy

The Internet's addressing model relies on hierarchical bit-level partitioning. An IP address identifies an interface on a host or network element, but routing at global scale operates entirely on **prefixes**—contiguous blocks of IP addresses represented in Classless Inter-Domain Routing (CIDR) notation.

In IPv4, an address comprises 32 bits (4 bytes); in IPv6, 128 bits (16 bytes). The prefix length indicates the size of the network mask:

$$\text{Total Addresses} = 2^{(32-L)} \quad \text{(IPv4)} \qquad \text{or} \qquad 2^{(128-L)} \quad \text{(IPv6)}$$

Where L is the prefix length. For example: - A `/24` in IPv4 encompasses $2^{(32-24)} = 256$ individual host addresses. - A `/48` in IPv6 provides $2^{(128-48)} = 2^{80}$ individual host addresses, typically delegated to an enterprise or end-site.

The global routing system does not accept arbitrary prefix sizes into the Default-Free Zone (DFZ). By broad operator consensus: - In IPv4, the longest (most specific) prefix universally accepted across transit providers is **`/24`**. Any advertisement more specific than `/24` (e.g., `/25`, `/28`) is dropped by upstream transit filters to protect router memory (TCAM). - In IPv6, the standard filter boundary is **`/48`**. Prefixes longer than `/48` (such as `/56` or `/64`) are filtered in the global DFZ.

Autonomous Systems and Administrative Boundaries

An **Autonomous System (AS)** is a collection of IP routing prefixes under the control of one or more network operators that presents a single, clearly defined routing policy to the Internet (RFC 1930, RFC 4271).

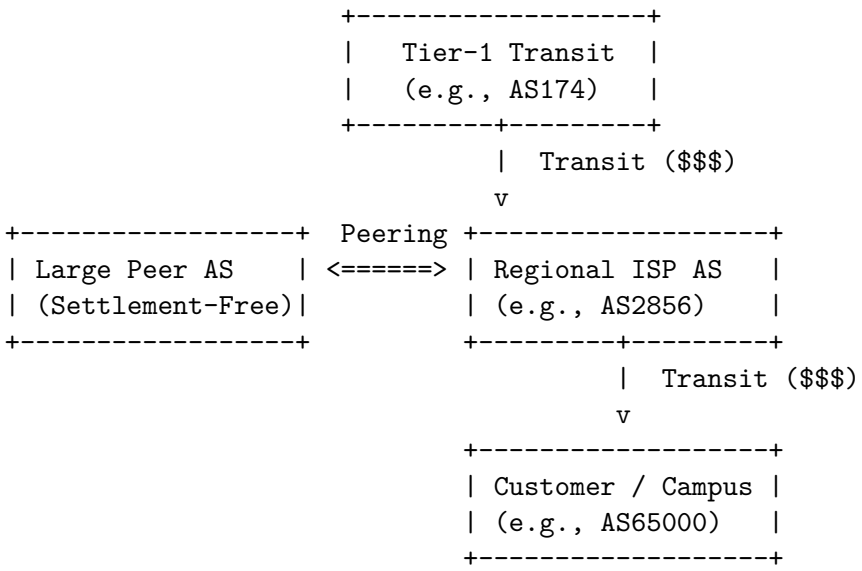
Every AS is identified by a globally unique 32-bit number known as an **Autonomous System Number (ASN)**: - **16-bit ASNs** (legacy): 1 to 65,535. (Private range: 64,512–65,534). - **32-bit ASNs** (RFC 4893 / RFC 6793): 65,536 to 4,294,967,295. (Private range: 4,200,000,000–4,294,967,294).

An ASN serves as the identity token attached to BGP path announcements. Administrative boundaries are absolute: inside an AS, an Interior Gateway Protocol (IGP) like OSPF or IS-IS computes

shortest paths based on link metrics and interface bandwidth. Across AS boundaries, policy rules supersede topology metrics completely.

The Transit-Peering Model and Its Economics

The global routing fabric is organized around two primary relationship types governed by commercial contracts:



1. Transit (Customer-Provider)

In a transit relationship, a customer pays an upstream provider for reachability to the entire Internet. The provider guarantees that packets addressed to any valid global IP destination will be forwarded through its network and uplinks. The provider advertises the customer's prefixes to the entire world and delivers either a default route (`0.0.0.0/0` or `::/0`) or the full global routing table (over 950,000 IPv4 routes and 200,000 IPv6 routes) to the customer.

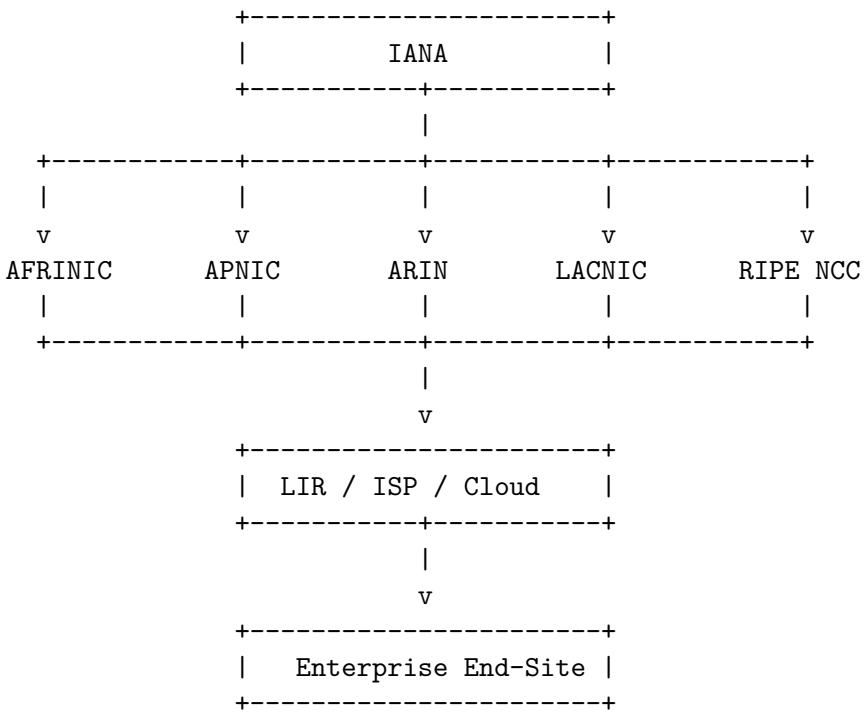
2. Peering (Settlement-Free or Paid)

In a peering relationship, two networks agree to exchange traffic between their respective customers directly, typically at an Internet Exchange Point (IXP) or private interconnect (PNI). Crucially: - Network A only advertises its own prefixes and its downstream customer prefixes to Network B. - Network A does **not** advertise prefixes learned from its transit providers or other peers to Network B. - Neither party pays the other; the exchange is mutually beneficial because it offloads expensive transit traffic.

Delegation: How Addresses and ASNs Become Someone's Responsibility

IP addresses and ASNs are managed globally under the auspices of the Internet Assigned Numbers Authority (IANA), which delegates large allocation blocks to five **Regional Internet Registries (RIRs)**:

RIR	Geographic Region
ARIN	North America and parts of the Caribbean
RIPE NCC	Europe, Middle East, and parts of Central Asia
APNIC	Asia-Pacific
LACNIC	Latin America and the Caribbean
AFRINIC	Africa



The RIRs allocate blocks to **Local Internet Registries (LIRs)**, which include national telcos, cloud providers, and large enterprises. The LIRs in turn assign address space to end-users or deploy it across their own infrastructure.

This delegation chain represents the administrative authority over address space. Historically, this chain was documented only in text-based WHOIS databases without cryptographic integrity, leaving a massive gap between administrative ownership and operational routing assertions.

Scalability Through Hierarchical Aggregation

Without hierarchical aggregation, routers in the core of the Internet would need to hold routing table entries for every individual machine or small subnet on Earth, leading to immediate hardware collapse.

Hierarchical aggregation functions by announcing a covering supernet (e.g., `198.51.100.0/22`) that represents multiple constituent subnets (`198.51.100.0/24`, `198.51.101.0/24`, `198.51.102.0/24`, and `198.51.103.0/24`). Outside observers only need to route towards the aggregate `/22`.

However, hierarchical aggregation creates the very vulnerability that enables **subprefix hijacking**: because longest prefix match (LPM) is an immutable forwarding rule in IP routing, any announcement of a `/24` will always override a covering `/22`, regardless of AS path length or administrative preference.

Interdomain Routing Fundamentals

Interdomain Routing Fundamentals

Interdomain routing is fundamentally distinct from intradomain routing. While an Interior Gateway Protocol (such as OSPF or IS-IS) solves an algorithmic optimization problem—finding the lowest-cost shortest path across a unified topology—interdomain routing solves a **policy enforcement problem** across independent, untrusted autonomous domains.

What Interdomain Routing Must Achieve

An interdomain routing system must fulfill four non-negotiable requirements:

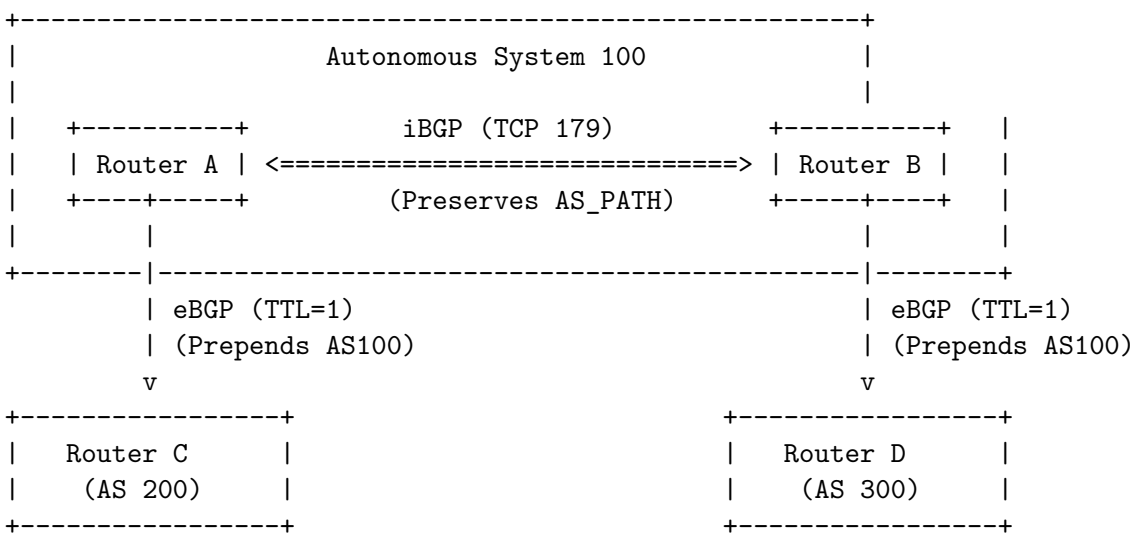
1. **Autonomy of Policy**: Every network operator must retain complete sovereignty over its routing decisions. An AS must be able to choose which paths to prefer, which paths to accept, and which paths to suppress, without revealing its internal topology or commercial agreements to competitors.
2. **Global Scalability**: The control plane must scale to over 100,000 interacting ASes and more than a million prefixes without requiring synchronous global state synchronization or quadratic message complexity.
3. **Loop Freedom**: Because interdomain paths traverse arbitrary meshes of autonomous networks, the protocol must guarantee loop-free forwarding without requiring complete knowledge of the global network graph.
4. **Incremental Convergence**: In a dynamic topology where links bounce, fibers are severed, and routers reboot constantly, updates and withdrawals must propagate incrementally without triggering global routing storms.

Direct Connectivity: iBGP and eBGP Sessions

BGP-4 operates over point-to-point peering sessions using TCP (destination port 179). Sessions are categorized based on the AS membership of the two peers:

Property	External BGP (eBGP)	Internal BGP (iBGP)
ASN Relationship	Peers reside in *different* ASes ($AS_A \neq AS_B$)	Peers reside in the *same* AS ($AS_A = AS_B$)
Physical Proximity	Typically direct point-to-point link or IXP peering fabric	Routers may be multiple IGP hops apart
Default TTL	`TTL = 1` (requires multihop override if not directly connected)	`TTL = 64` or `255` (traverses internal IGP topology)

Property	External BGP (eBGP)	Internal BGP (iBGP)
AS_PATH Modification	Prepends local ASN upon export	Does *not* alter the `AS_PATH` attribute
NEXT_HOP Handling	Rewrites `NEXT_HOP` to local peering interface address	Preserves original eBGP `NEXT_HOP` (unless `next-hop-self`)
Split Horizon Rule	Allows re-advertisement to other eBGP peers	Routes learned via iBGP are *never* advertised to other iBGP peers



Because iBGP enforces a strict split-horizon rule (a route learned from one iBGP peer cannot be passed to another iBGP peer), an AS traditionally required a **full mesh** of iBGP sessions across all internal BGP speakers. At scale, this is resolved using **Route Reflectors** (RFC 4456) or BGP confederations (RFC 5065).

Route Advertisements and Withdrawals in Practice

A BGP session is a continuous stateful exchange of Network Layer Reachability Information (NLRI). Unlike periodic link-state advertisements, BGP is purely event-driven:

Advertisements (UPDATE with NLRI)

When a network originates a prefix or selects a new best path from an upstream neighbor, it crafts an `UPDATE` message containing: - **NLRI**: One or more prefixes (e.g., `192.0.2.0/24`). - **Path Attributes**: Mandatory metadata including `ORIGIN`, `AS_PATH`, and `NEXT_HOP`.

Withdrawals (UPDATE with Withdrawn Routes)

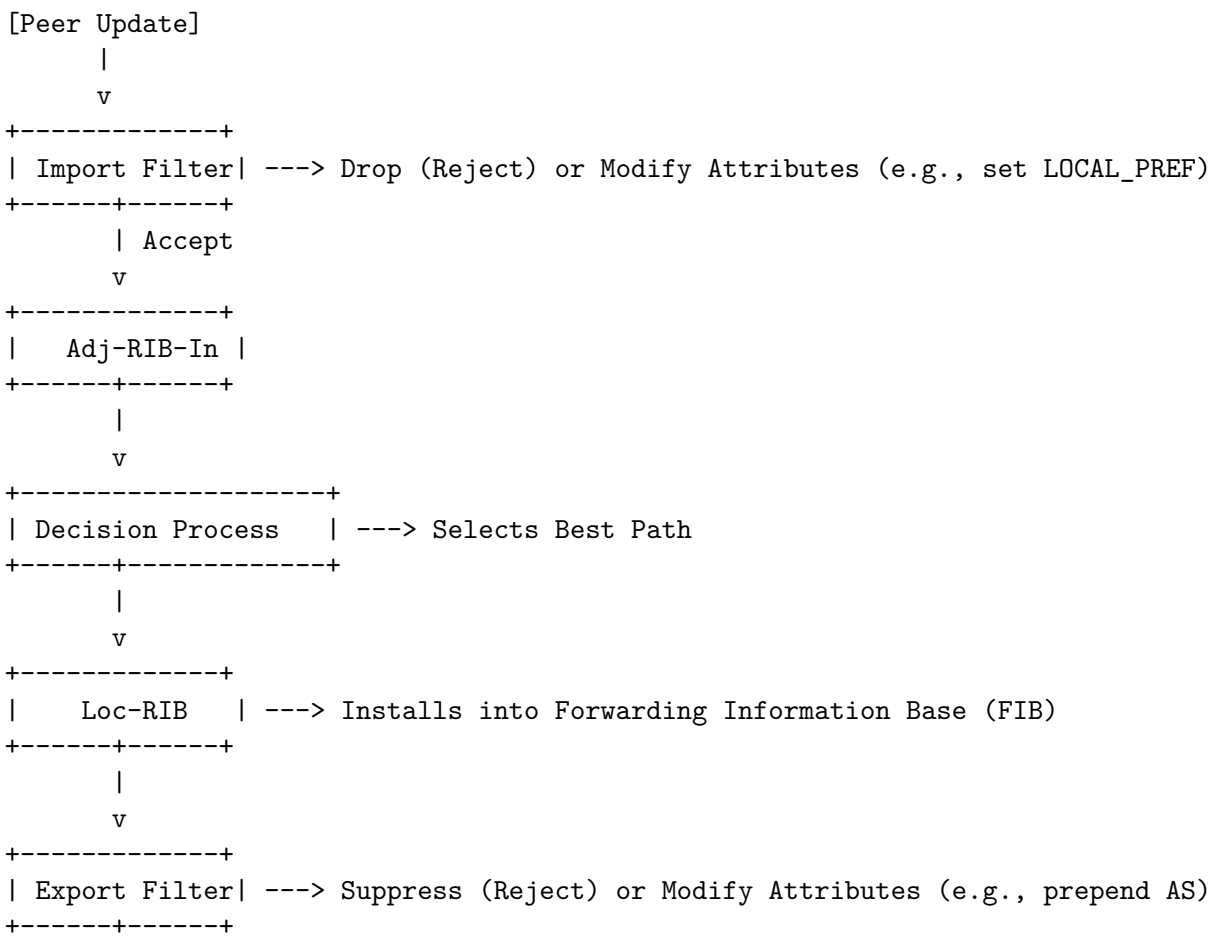
When a prefix becomes unreachable (due to an interface failure, policy change, or withdrawal received from an upstream transit provider), the router immediately issues an `UPDATE` specifying the prefix in the **Withdrawn Routes** field. A withdrawal requires no path attributes; it instructs the peer to purge the prefix from its Routing Information Base (RIB-In) and recompute best paths immediately.

Implicit Withdrawals

If a peer receives an `UPDATE` for a prefix that already exists in its table with different path attributes (e.g., a changed `AS_PATH` or `NEXT_HOP`), the router replaces the existing route in-place. This is known as an implicit withdrawal and avoids the latency of sending explicit teardown messages.

Import and Export Policies: The Operator's Control Point

Routing policy is expressed through programmable filters executed at the boundary of the routing process:



```
       | Accept
       v
+-------------+
| Adj-RIB-Out | ---> Transmit UPDATE to Neighbor
+-------------+
```

1. **Import Policy**: Evaluates inbound `UPDATE` messages before they are processed by the BGP decision engine. Operators can reject unauthorized prefixes, adjust `LOCAL_PREF` to prioritize specific uplinks, or rewrite communities.
2. **Export Policy**: Evaluates best routes from the `Loc-RIB` before they are formatted into `UPDATE` messages and sent to neighbors. Export policies enforce transit boundaries, preventing customer routes from leaking to unauthorized peers.

Propagation Through the Transit Web

When an origin AS announces a prefix, the announcement cascades outward through transit tiers. A Tier-1 transit provider receiving the announcement over eBGP accepts it, prepends its own ASN to the `AS_PATH`, updates the `NEXT_HOP`, and propagates the update across its global backbone using iBGP. From there, edge routers advertise the prefix to downstream Tier-2 providers and peering networks via eBGP.

Within minutes, the prefix populates the routing tables of tens of thousands of routers worldwide. Crucially, unless explicit filtering is configured at each hop, routers along the path accept and propagate the announcement **without any mathematical proof** that the originating AS actually owns the prefix.

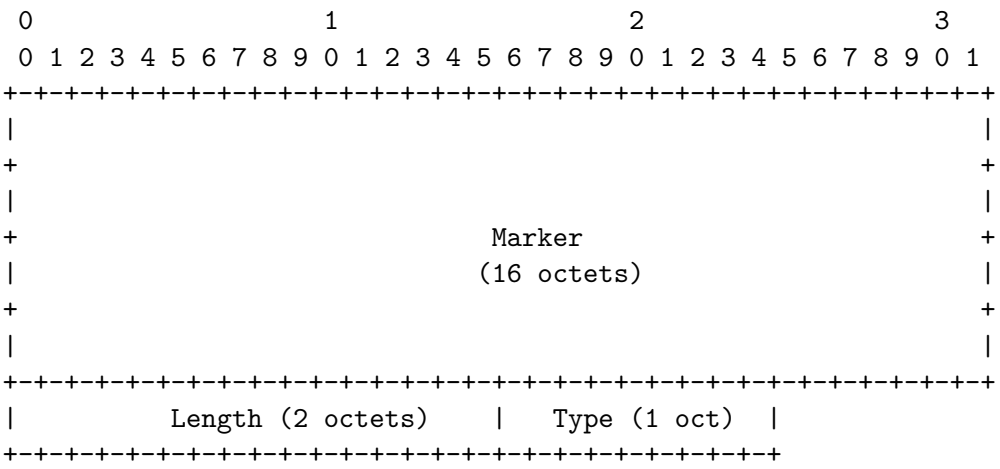
BGP Protocol Architecture

BGP Protocol Architecture

The Border Gateway Protocol version 4 (BGP-4), defined in RFC 4271, is an application-layer protocol running over TCP. Its protocol mechanics dictate how peering sessions are negotiated, maintained, and torn down, and establish the structural framework that modern routing security extensions must operate within.

The BGP Message Types: OPEN, UPDATE, NOTIFICATION, KEEPALIVE

All BGP messages share a standard 19-byte header:



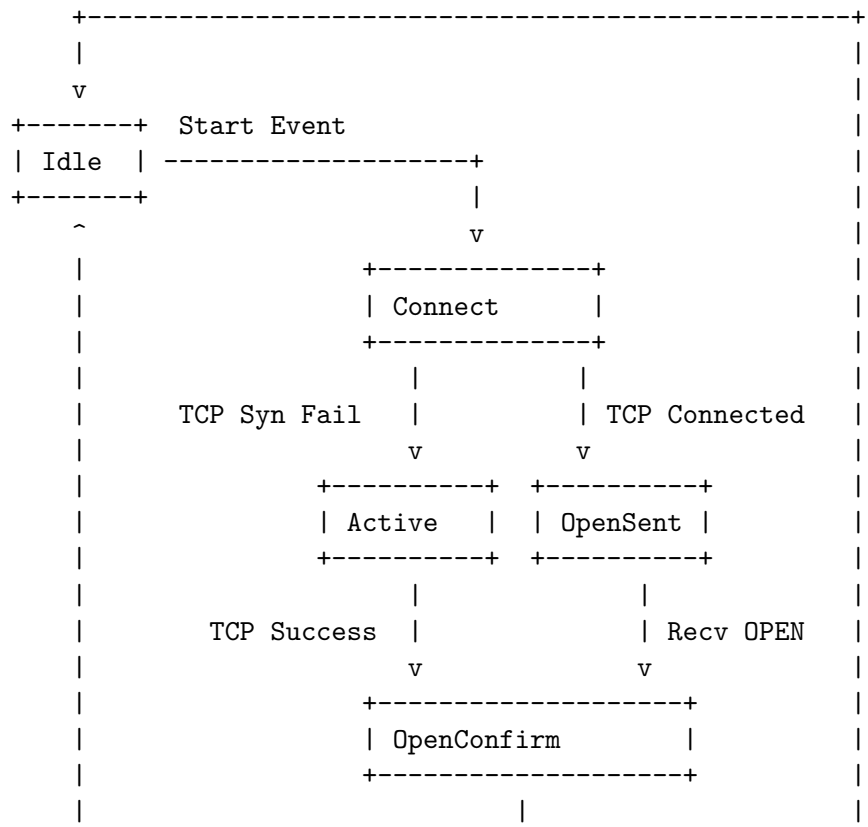
- **Marker** (16 octets): Set to all ones for synchronization and legacy authentication compatibility.
- **Length** (2 octets): Total message length in bytes (minimum 19, standard maximum 4096 bytes; extended to 65,535 bytes under RFC 8654 Extended Message support).
- **Type** (1 octet): Indicates the payload message type:

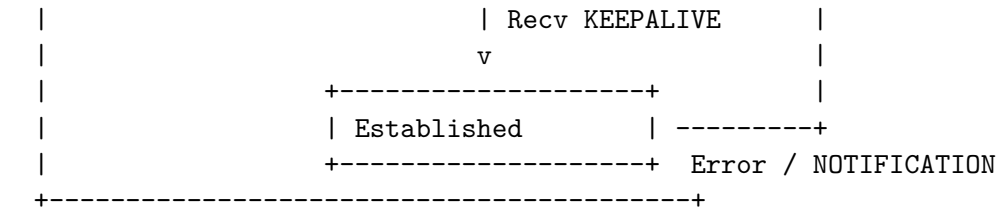
Type	Name	Purpose
1	`OPEN`	Negotiates session parameters (BGP version, ASN, Hold Time, BGP Identifier, Capabilities TLVs).

Type	Name	Purpose
2	`UPDATE`	Advertises reachable routes with path attributes, and withdraws unreachable routes.
3	`NOTIFICATION`	Signals a fatal protocol error or graceful session shutdown; terminates the peering session immediately.
4	`KEEPALIVE`	Periodic heartbeat message (19-byte header only) to verify neighbor liveness before Hold Time expires.
5	`ROUTE-REFRESH`	(RFC 2918) Dynamically requests a peer to re-send its Adj-RIB-Out without resetting the TCP session.

The BGP Finite State Machine: From Idle to Established

The BGP Finite State Machine (FSM) defines the lifecycle of a peering session across six deterministic states:





1. **Idle**: The initial state. The router allocates resources, initializes variables, and waits for a start event (manual or automatic). In response, it initiates a TCP connection and transitions to `Connect`.
2. **Connect**: The router attempts a 3-way TCP handshake with the peer on port 179.
   - If TCP succeeds: it transmits an `OPEN` message and transitions to `OpenSent`.
   - If TCP fails: it starts the `ConnectRetry` timer and transitions to `Active`.
3. **Active**: The router actively listens for incoming TCP connections while attempting to re-establish the connection. If the timer expires without success, it reverts to `Connect`.
4. **OpenSent**: The router has sent its `OPEN` message and awaits an `OPEN` message from the peer. Upon receipt, it validates the peer's ASN, version, and capabilities:
   - If valid: sends a `KEEPALIVE` and moves to `OpenConfirm`.
   - If invalid: sends a `NOTIFICATION` with error code and resets to `Idle`.
5. **OpenConfirm**: Awaiting a `KEEPALIVE` or `NOTIFICATION` from the peer. Receiving a `KEEPALIVE` confirms mutual parameter acceptance, transitioning the session to `Established`.
6. **Established**: The operational state. Peers exchange reachability updates (`UPDATE`), route refresh requests, and periodic heartbeats (`KEEPALIVE`). Any protocol violation or expired hold timer triggers a `NOTIFICATION` and tears the session back to `Idle`.

TCP Port 179 and Transport Considerations

By delegating transport-layer responsibilities to TCP (Port 179), BGP inherited several key characteristics and vulnerabilities:

- **Reliability and Flow Control**: BGP avoids implementing retransmission, fragmentation, windowing, or congestion control at the routing layer. TCP guarantees byte-stream delivery.
- **SYN-Flood and RST Attacks**: Because TCP sessions are vulnerable to blind RST injection and SYN-flood starvation, BGP requires protection mechanisms:
  - **TCP MD5 Signature Option (RFC 2385)**: Embeds an MD5 digest computed over the TCP header, payload, and shared secret in every TCP segment. While cryptographically obsolete (MD5 collisions), it remains widely deployed across legacy hardware.
  - **TCP Authentication Option (TCP-AO, RFC 5925)**: Modern replacement for MD5, utilizing HMAC-SHA-1 or AES-128-CMAC with cryptographic key rollover capabilities.
  - **Generalized TTL Security Mechanism (GTSM, RFC 5082)**: Exploits packet hop counts to block off-path attackers. For directly connected eBGP peers, the outgoing IP TTL is set to 255. The receiver verifies that incoming packets have a TTL of 255 (or 254). An off-path attacker across the Internet cannot craft a packet that arrives with a TTL of 255.

BGP-4 UPDATE Message Structure and TLV Encoding

The `UPDATE` message is the operational workhorse of BGP:

```
+-----------------------------------------------------+
|   Withdrawn Routes Length (2 octets)                |
+-----------------------------------------------------+
|   Withdrawn Routes (variable length)                |
|   [Prefix Length (1 octet), Prefix (variable)]      |
+-----------------------------------------------------+
|   Total Path Attribute Length (2 octets)            |
+-----------------------------------------------------+
|   Path Attributes (variable length)                 |
|   [Flags (1 oct), Type (1 oct), Length (1-2), Value]|
+-----------------------------------------------------+
|   Network Layer Reachability Information (NLRI)     |
|   [Length (1 octet), Prefix (variable)]             |
+-----------------------------------------------------+
```

Each path attribute is encoded in a **Type-Length-Value (TLV)** triple preceded by an Attribute Flags byte: - **Bit 0 (Optional)**: Set to 0 if Well-Known; 1 if Optional. - **Bit 1 (Transitive)**: Set to 1 if Transitive; 0 if Non-Transitive. (All Well-Known attributes are transitive). - **Bit 2 (Partial)**: Set to 1 if an upstream router forwarded an unrecognized optional transitive attribute without understanding it. - **Bit 3 (Extended Length)**: Set to 0 if Attribute Length is 1 octet; set to 1 if Attribute Length is 2 octets.

Capabilities Negotiation and Modern Extensions

During the initial `OPEN` message exchange, BGP speakers negotiate optional capabilities (RFC 5492) using Optional Parameters TLVs:

- **Multiprotocol BGP (MP-BGP, RFC 4760)**: Extends BGP beyond IPv4 unicast by defining Address Family Identifiers (AFI: IPv4=1, IPv6=2) and Subsequent AFIs (SAFI: Unicast=1, Multicast=2, EVPN=70). This decouples protocol transport from payload reachability.
- **4-Byte ASN Support (RFC 6793)**: Allows routers to negotiate 32-bit AS numbers, preserving backwards compatibility with legacy 2-byte speakers through the pseudo-ASN `AS_TRANS` (23456).
- **Graceful Restart (RFC 4724)**: Preserves data-plane forwarding across control-plane restarts, preventing temporary routing flaps during software upgrades.
- **Enhanced Route Refresh (RFC 7313)**: Guarantees synchronization between peers during policy reloads.

BGP Path Attributes and Policy Expression

BGP Path Attributes and Policy Expression

BGP is not merely a routing protocol; it is a distributed policy programming language. Routing policy is expressed, transmitted, and enforced through **Path Attributes** associated with prefixes in `UPDATE` messages. Understanding attribute classification, propagation mechanics, and semantics is essential to auditing route behavior and diagnosing policy vulnerabilities.

Attribute Categories

Every BGP path attribute falls into one of four rigid categories based on two binary flags: *Is it recognized by all implementations?* and *Must it be passed across AS boundaries?*

Classification	Meaning	Must Be Understood?	Propagated across eBGP?	Standard Examples
Well-Known Mandatory	Must be present in every single route advertisement update.	Yes	Yes	`ORIGIN`, `AS_PATH`, `NEXT_HOP`
Well-Known Discretionary	Recognized by all implementations; optional to include in update.	Yes	Yes	`LOCAL_PREF`, `ATOMIC_AGGREGATE`
Optional Transitive	May not be supported by every implementation; if unrecognized, must be preserved and passed to peers.	No	Yes	`COMMUNITY` (RFC 1997), `AGGREGATOR`
Optional Non-Transitive	If unrecognized or across eBGP boundary, must be silently discarded.	No	No	`MULTI_EXIT_DISC` (MED), `ORIGINATOR_ID`, `CLUSTER_LIST`

[!WARNING] The **Optional Transitive** category introduces a notable vector for covert signaling and table bloat: an upstream router will faithfully propagate unrecognized

transitive attributes across the entire global Internet without inspecting or sanitizing their contents.

AS_PATH: Loop Detection and Policy Signal

The `AS_PATH` attribute (Type Code 2) records the sequence of Autonomous Systems that a route advertisement has traversed. It serves two distinct functions:

1. Loop Prevention

When a router receives an `UPDATE` over an eBGP session, it scans the `AS_PATH` for its own ASN. If its ASN is present, the router immediately discards the update. This simple mechanism guarantees loop-free paths in interdomain topologies without running spanning-tree algorithms or Dijkstra SPF across the planet.

Inside an `AS_PATH`, numbers are grouped into segments: - **`AS_SEQUENCE`**: An ordered list of ASNs traversed by the route (most recent hop first). - **`AS_SET`**: An unordered collection of ASNs created during route summarization. (Deprecated by RFC 6472 and RFC 6978 due to security and operational complexity).

2. Traffic Engineering (AS-Path Prepending)

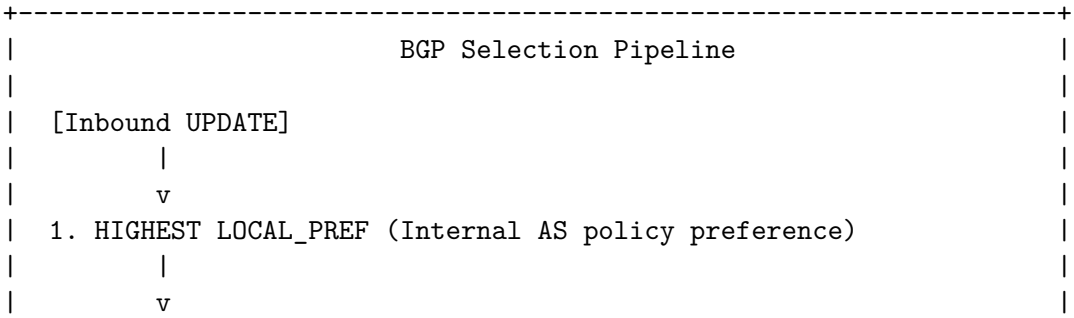
Because the BGP decision process prefers shorter `AS_PATH` lengths, operators intentionally duplicate their ASN multiple times when advertising routes to secondary transit providers:

$$\text{Advertised to Primary ISP:} \quad \text{AS_PATH} = [65001]$$

$$\text{Advertised to Backup ISP:} \quad \text{AS_PATH} = [65001, 65001, 65001]$$

This artificially inflates the path length, discouraging remote networks from selecting the backup link for ingress traffic.

NEXT_HOP, LOCAL_PREF, MED and ORIGIN: Comparative Selection Logic



```
|  2. SHORTEST AS_PATH (Fewest AS hops to destination)                |
|         |                                                           |
|         v                                                           |
|  3. LOWEST ORIGIN (IGP [0] < EGP [1] < Incomplete [2])              |
|         |                                                           |
|         v                                                           |
|  4. LOWEST MED (Metric set by neighboring AS for multi-homing)     |
|         |                                                           |
|         v                                                           |
|  5. eBGP over iBGP                                                  |
|         |                                                           |
|         v                                                           |
|  6. LOWEST IGP METRIC TO NEXT_HOP (Hot-potato routing)              |
+---------------------------------------------------------------------+
```

- `NEXT_HOP` **(Well-Known Mandatory)**: Specifies the IP address of the next-hop router. On eBGP sessions, it is the IP address of the peering neighbor. In iBGP, the eBGP next-hop is preserved across the AS backbone, requiring internal routers to resolve it via their IGP (recursive routing) unless `next-hop-self` is applied at border routers.
- `LOCAL_PREF` **(Well-Known Discretionary)**: Valid only within an AS (disseminated via iBGP; stripped before eBGP export). Default is typically 100. A higher value indicates higher preference. This is the primary mechanism operators use to prefer customer links over transit uplinks.
- `ORIGIN` **(Well-Known Mandatory)**: Defines the origin source of the route:
  - `0 = IGP` (prefix originated via `network` statement or IGP redistribution).
  - `1 = EGP` (historic Exterior Gateway Protocol).
  - `2 = INCOMPLETE` (redistributed from unknown or static sources).
- `MULTI_EXIT_DISC` **(MED, Optional Non-Transitive)**: Conveyed to an adjacent AS across eBGP to indicate preference among multiple parallel ingress links into the originating AS. Lower MED is preferred. By default, routers only compare MED among paths learned from the *same* neighboring AS.

BGP Communities and Extended Communities: Encoding Policy at Scale

BGP Communities (RFC 1997) are 32-bit values attached to routes that convey operational tags and trigger programmatic actions at remote routers. Standard communities are formatted as `ASN:Value` (e.g., `65001:100`).

Well-Known Standard Communities

- `NO_EXPORT (0xFFFFFF01)`: Do not advertise the route outside the local AS (or BGP confederation).
- `NO_ADVERTISE (0xFFFFFF02)`: Do not advertise the route to *any* peer (neither iBGP nor eBGP).

- `BLACKHOLE` **(RFC 7999,** `0xFFFF029A` **or** `ASN:666`**)**: Signals transit providers and peers to discard all traffic destined for the prefix in hardware (Remote Triggered Blackholing).

Extended (RFC 4360) and Large Communities (RFC 8092)

- **Extended Communities (8 octets)**: Provide typed metadata such as Route Targets (RT) and Route Distinguishers (RD) in EVPN and MPLS-VPN deployments.
- **Large Communities (12 octets)**: Formatted as `Global-Admin:Action:Function` (e.g., `65000:1:100`). Designed to support full 32-bit AS numbers alongside flexible operator tagging.

ADD-PATH, GTSM and Modern Security Attributes

- **ADD-PATH (RFC 7911)**: By default, BGP only advertises its single best path to peers. ADD-PATH allows a router to advertise multiple alternative paths for the same prefix, each identified by a unique Path ID. This eliminates route-reflector hiding and accelerates convergence.
- **Generalized TTL Security Mechanism (GTSM, RFC 5082)**: Mitigates CPU denial-of-service by verifying that eBGP packets have traversed exactly one hop (`TTL = 255`).
- **BGP Only to Customer (OTC, RFC 9234)**: An attribute designed to enforce provider-customer hierarchy and automatically block route leaks.

The BGP Decision Process and Route Selection

The BGP Decision Process and Route Selection

When a router receives multiple valid candidate paths for the same prefix from different peers, it executes a deterministic, multi-step tie-breaking pipeline: the **BGP Decision Process**. Understanding the hierarchy of these criteria is critical, as routing attacks and policy leaks frequently exploit specific tie-breaking stages to divert traffic.

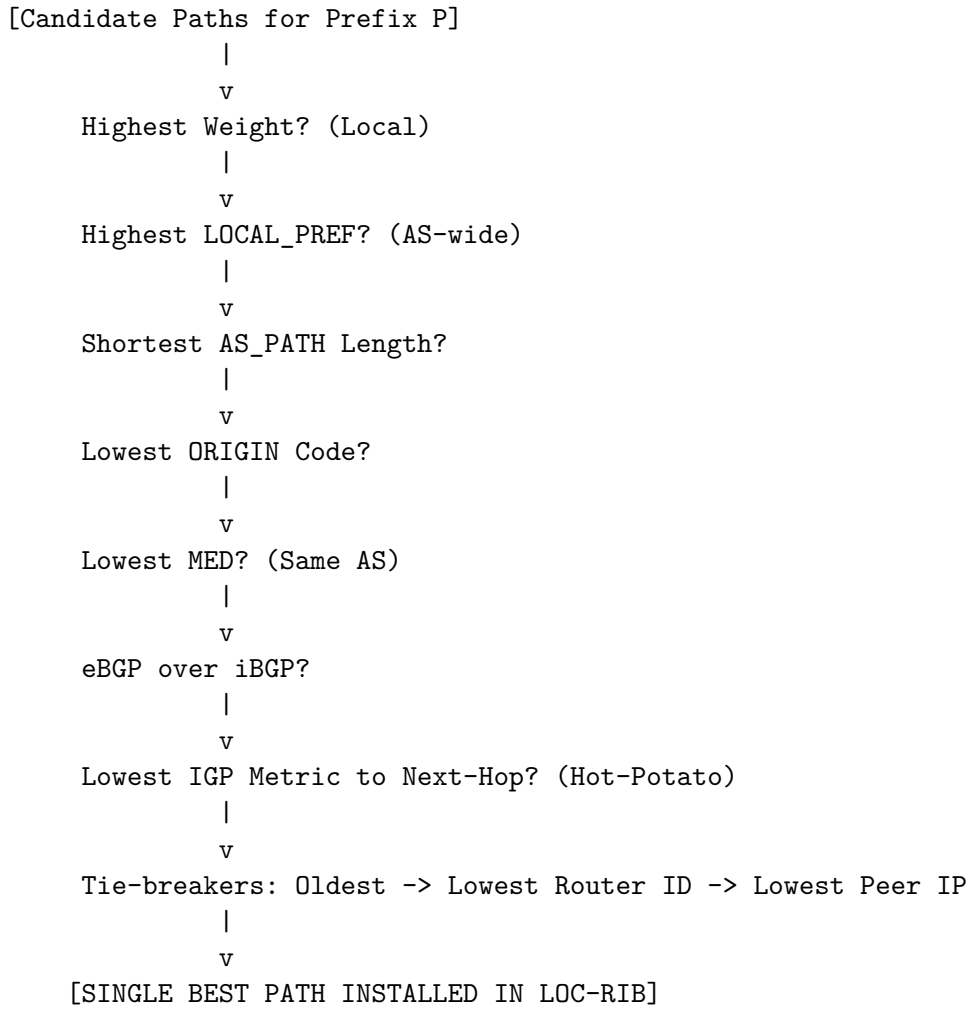
The Step-by-Step BGP Decision Process

Before initiating the comparison steps, the router validates basic eligibility: - The candidate route must be installed in `Adj-RIB-In` with valid syntax. - The `NEXT_HOP` IP address must be resolvable in the routing table (IGP or connected interface). - The `AS_PATH` must not contain the local AS number (unless `allowas-in` is configured).

Once eligible, candidate paths are compared pairwise using the following strict hierarchy:

Step	Attribute / Criterion	Preferred Candidate	Scope & Portability
0	**Weight**	Highest Weight	Local router only (vendor-specific: Cisco, Huawei; not transmitted in BGP)
1	`LOCAL_PREF`	Highest `LOCAL_PREF`	Entire local AS (disseminated over iBGP; stripped over eBGP)
2	**Locally Originated**	Local `network` or `redistribute` over received	Local router origin preference
3	`AS_PATH` **Length**	Shortest sequence of AS hops	Global metric (ignores `AS_SET` size per RFC 5065)
4	`ORIGIN`	IGP (0) < EGP (1) < Incomplete (2)	Global attribute
5	`MED` **(Multi-Exit Disc)**	Lowest MED	Between paths from the *same* neighboring AS (unless `always-compare-med`)
6	**Peer Type**	eBGP path over iBGP path	Boundary preference (prefers external exit over transit)

Step	Attribute / Criterion	Preferred Candidate	Scope & Portability
7	**IGP Metric to `NEXT_HOP`**	Lowest metric in IGP to reach `NEXT_HOP`	Hot-potato routing (exit local AS as rapidly as possible)
8	**Multipath**	Install multiple equal paths (if enabled)	ECMP load-balancing
9	**Oldest Route**	Path received first (eBGP paths only)	Route flap stabilization (prevents unnecessary churn)
10	**Router ID (BGP Identifier)**	Lowest BGP Router ID (IPv4 address)	Deterministic tie-breaker
11	**Cluster List Length**	Shortest `CLUSTER_LIST` length	Route reflector topologies
12	**Neighbor IP Address**	Lowest peer IP address	Final deterministic tie-breaker



Weight, LOCAL_PREF and Administrative Distance

- **Weight**: A 16-bit integer (0 to 65,535) configured locally on a router interface or neighbor. Weight takes precedence over all other BGP attributes. It allows an operator on a single device to force all traffic out of Interface A without affecting any other router in the network.
- **LOCAL_PREF**: The universal AS-wide policy knob. In standard production practice, network operators assign `LOCAL_PREF` values based on business relationships:
  - **Customer Routes**: `LOCAL_PREF = 200` (highest priority; customer pays us).
  - **Settlement-Free Peer Routes**: `LOCAL_PREF = 150` (medium priority; free transport).
  - **Transit Provider Routes**: `LOCAL_PREF = 100` (lowest priority; we pay for bytes).
  - **Backup Links**: `LOCAL_PREF = 50`.

Because `LOCAL_PREF` sits above `AS_PATH` length in the decision pipeline, an operator will always prefer a 10-hop customer path over a 1-hop transit provider path.

- **Administrative Distance / Route Preference**: When BGP selects its best path, that path must still compete against other routing protocols (OSPF, IS-IS, Static, Connected) for installation into the kernel or ASIC FIB:
  - Directly Connected: 0
  - Static Route: 1
  - eBGP: 20 (default on Cisco; Junos uses route preference 170)
  - OSPF: 110
  - iBGP: 200 (default on Cisco; Junos uses route preference 170)

AS_PATH Length, ORIGIN and MED Comparisons

- **AS_PATH Length**: When `LOCAL_PREF` is equal (e.g., comparing two transit providers), the router selects the path with the fewest AS hops. This exposes a primary attack surface: an adversary who announces a stolen prefix with a direct 1-hop `AS_PATH` will defeat a legitimate path that is 3 hops away, winning traffic across the entire Internet.
- **MED and `always-compare-med`**: MED is designed to let an adjacent customer indicate which of multiple links to use. Comparing MED across different ASNs is disabled by default because it can create forwarding loops and nondeterministic oscillation (RFC 3345).

Route Preference and IGP Influence: eBGP vs iBGP vs Static vs IGP

Step 6 (eBGP over iBGP) prioritizes handing traffic directly to an external peer rather than routing it through internal core backbones.

Step 7 (**Hot-Potato Routing**) chooses the exit router whose `NEXT_HOP` IP has the lowest interior metric in OSPF/IS-IS. This shifts the computational burden and bandwidth cost to downstream networks as early as possible. However, if internal IGP metrics flap, the BGP decision engine will recompute its best path, inducing path instability across external sessions.

Multi-Exit Discrimination, Tie-Breakers and Stability

To prevent routing flaps from causing perpetual state changes, Step 9 prefers the **oldest received route** when comparing external eBGP paths. This dampens minor network oscillations. If all policy attributes, lengths, and metrics match, the router breaks the tie deterministically using the lowest peer Router ID and lowest IP socket address.

Convergence, Stability and Propagation Dynamics

Convergence, Stability and Propagation Dynamics

The global routing system is never in a perfectly static state. Across tens of thousands of networks, physical fiber cuts, hardware reboots, BFD timeouts, and maintenance windows trigger a continuous stream of BGP updates and withdrawals. Understanding how these updates ripple through the planetary topology—and how that dynamics can be weaponized—is essential for securing the routing control plane.

Convergence Time and Global Propagation Delays

Convergence is the process by which all routers across the Internet transition from an old state to an agreed new consistent forwarding state following a topology change.

Unlike IGP link-state databases that synchronize in tens of milliseconds, interdomain BGP convergence operates on the order of **seconds to minutes**. The total delay is governed by four factors:

$$T_{\text{convergence}} = \sum (T_{\text{processing}} + T_{\text{MRAI}} + T_{\text{queuing}} + T_{\text{propagation}})$$

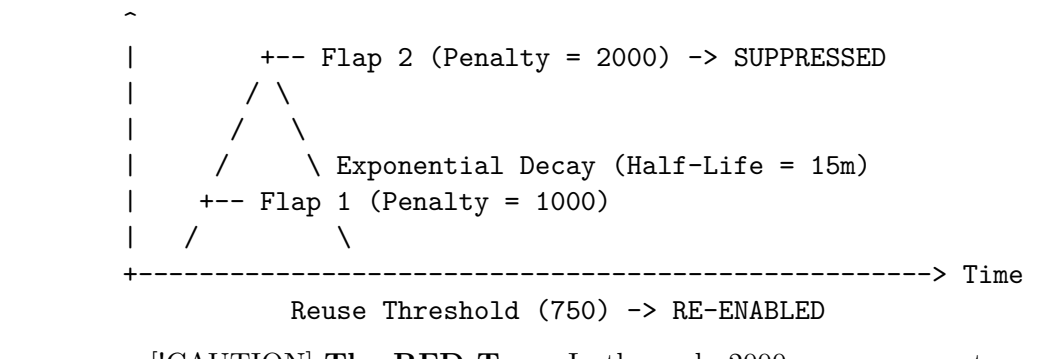
1. **Minimum Route Advertisement Interval (MRAI, RFC 4271)**: To prevent routers from bombarding neighbors with continuous updates during a flap, BGP enforces a rate-limiting timer on eBGP sessions (historically default: 30 seconds; modern implementations allow 5 seconds or 0 seconds). A router cannot send successive updates for the same prefix until the MRAI timer expires.
2. **Path Exploration (BGP Hunting)**: When a preferred path is withdrawn, a router does not immediately know that backup paths traversing the same failing link are also broken. It will iteratively explore each candidate path in order of length, advertising and withdrawing multiple intermediate paths before settling on the true surviving route.
3. **Propagation Delay**: Speed-of-light delays across transoceanic fiber links (e.g., London to Singapore: ~160 ms round-trip).

Route Flapping and Dampening (RFC 2439)

A **route flap** occurs when a prefix is repeatedly advertised and withdrawn in rapid succession. Left unchecked, a single flapping prefix on a flapping DSL line or unstable BGP session would force hundreds of thousands of routers worldwide to continuously recompute their FIBs.

To combat this, **BGP Route Flap Dampening (RFD, RFC 2439)** was introduced: - Every time a route flaps, the router adds a numerical penalty (typically 1000). - If the penalty exceeds a

configured **Suppress Threshold** (e.g., 2000), the route is suppressed: the router stops using it and stops advertising it to peers, even if it is currently advertised as reachable. - While suppressed, the penalty decays exponentially according to a **Half-Life** timer (e.g., 15 minutes). - Once the penalty falls below a **Reuse Threshold** (e.g., 750), the route is unsuppressed and restored to service.



> [!CAUTION] **The RFD Trap**: In the early 2000s, measurement research (Mao et al.) revealed that normal BGP path exploration during a single failover event caused benign prefixes to flap 3–5 times within seconds. Aggressive RFD suppressed legitimate networks for hours. Consequently, RIPE-378 and RFC 7454 advise operators to either disable RFD entirely or set conservative thresholds that only trigger on persistent, sustained oscillations.

Hot Potato Routing and Path Instability

In **Hot-Potato Routing**, an Autonomous System sheds outbound traffic at the closest exit router according to its internal IGP metric.

While computationally efficient, hot-potato routing ties interdomain forwarding decisions directly to intradomain topology. A link flap inside an ISP's internal backbone can shift the lowest IGP metric from Exit Router A in Chicago to Exit Router B in Dallas. Because BGP recomputes the best path, the ISP sends an `UPDATE` with a changed `NEXT_HOP` to external peers, triggering global route churn despite the external destination remaining entirely stable.

Slow Convergence as an Attack Surface

The latency of interdomain convergence creates significant operational and security vulnerabilities:

1. **Transient Forwarding Loops and Blackholes**: During the minutes when Router A has converged to a new path but downstream Router B is still evaluating an old path, packets can bounce between routers in a transient loop or be dropped into a blackhole until TTL expires.
2. **Volumetric Churn Attacks**: An adversary with control of a flapping BGP speaker can intentionally cycle announcements and withdrawals at intervals timed to defeat MRAI filters, forcing upstream route processors to continuously consume CPU cycles in state recalculation.
3. **Attack Opportunity Window**: In an intentional origin hijacking attack, the minutes taken for global looking glasses and monitoring systems to converge and alert operators give attackers an uncontested window to intercept credentials or siphon cryptocurrency.

Measurement Data on Convergence Behavior

Data gathered from global route collection projects (RIPE RIS, RouteViews, CAIDA) indicates: - **Up-Convergence** (a newly announced prefix becoming globally reachable): typically achieves 95% global visibility within **10 to 45 seconds**. - **Down-Convergence** (withdrawing an active prefix): can take **90 to 240 seconds** due to path exploration across Tier-1 networks. - Over 60% of observed global routing updates are duplicate announcements or attribute modifications that do not change the underlying forwarding next-hop.

Historical Design Decisions and Their Security Consequences

Historical Design Decisions and Their Security Consequences

The architectural weaknesses of interdomain routing are neither oversights nor accidental engineering flaws. They are the deliberate, logical consequences of design trade-offs made in the late 1980s and early 1990s under intense operational pressure to save a rapidly collapsing Internet.

EGP, CGE and the Origins of Interdomain Routing Protocols

In the early ARPANET and early NSFNET eras, the network was organized around a strict tree topology with a single centralized core backbone. The protocol used to connect external networks to this core was the **Exterior Gateway Protocol (EGP, RFC 904)**.

EGP had severe limitations: - It was strictly a reachability protocol, incapable of detecting loops in arbitrary mesh topologies. - It required a single hierarchical root; it could not support horizontal peering between commercial networks. - As regional networks proliferated, the centralized architecture became an unsustainable bottleneck.

In 1989, during the "Three Dirt Clods" meeting (Kirk Lougheed of Cisco and Yakov Rekhter of IBM), BGP-1 was famously sketched on two ketchup-stained napkins (RFC 1105). BGP introduced the **path-vector** concept: by including the complete sequence of Autonomous Systems in the route announcement (`AS_PATH`), routers could detect and discard loops in arbitrary, uncoordinated mesh topologies without requiring a centralized backbone.

The Scalability Imperative: Why End-to-End Verification Was Abandoned

By 1993, the Internet faced an existential scaling crisis: the 32-bit IPv4 address space was depleting rapidly under classful allocation (A, B, C), and global routing tables were expanding exponentially. BGP-4 (RFC 1654, later RFC 4271) was developed specifically to introduce **Classless Inter-Domain Routing (CIDR, RFC 1519)**, allowing route aggregation and supernetting.

At the time, typical backbone routers operated with CPU clock speeds measured in tens of megahertz and RAM measured in megabytes. Adding cryptographic signatures to routing updates: - Would have required asymmetric public-key cryptography (RSA or DSA) on every router. - Would have caused packet-processing engines and control processors to grind to a halt. - Would have required a global, authoritative Public Key Infrastructure (PKI) when even the commercial DNS infrastructure was barely nascent.

The engineering consensus was unequivocal: **scalability and survival took absolute precedence over cryptographic verification.**

The Trust Assumptions Embedded in BGP-4's Design

BGP-4 was architected on fundamental trust assumptions reflecting the collegial academic environment of the early Internet:

1. **Physical Session Trust**: It was assumed that if two organizations established a physical cable or leased line and configured a BGP peering session, they knew each other and could be trusted.
2. **Implicit Origin Authority**: If a peer announced an IP prefix P, the receiver assumed the peer had legitimate authority to originate P.
3. **Implicit Path Honesty**: It was assumed that intermediate transit networks would accurately append their own ASN to the `AS_PATH` and would not delete, reorder, or fabricate AS hops.
4. **Benevolent Operation**: Routing anomalies were viewed as rare equipment failures, not malicious asymmetric attacks by state actors or cybercriminals.

Operational Flexibility Over Consistency: A Deliberate Choice

BGP was explicitly designed to allow operators to implement arbitrary, confidential business policies. An AS can modify attributes (`LOCAL_PREF`, MED, communities) arbitrarily on import and export without explaining its reasoning to its neighbors.

This design priority produced the **Gao-Rexford model** of Internet stability: routing converges stably if and only if commercial relationships follow strict hierarchical patterns (prefer customer over peer over provider). However, because policy expression is purely local and unauthenticated: - An operator cannot independently verify whether a path received from a neighbor represents the actual physical or topological path packets will take. - An operator cannot detect if an upstream peer is silently rewriting or stripping policy communities.

Why Security Was Added as an Afterthought

For nearly two decades, the Internet functioned on informal operator cooperation and out-of-band communication (the NANOG mailing list, IRC channels, telephone calls between network operations centers). When an operator misconfigured a router and hijacked traffic, people called each other to fix it.

This social trust model collapsed as the Internet transformed into critical economic infrastructure: - The rise of financial technology, cryptocurrency, and e-commerce created massive monetary incentives to steal traffic. - State-sponsored espionage groups recognized that interdomain routing could be used for covert surveillance without breaking end-to-end cryptography. - The sheer scale of the global network (over 75,000 ASes) rendered informal human relationships obsolete.

Because cryptographic security could not be retrofitted into the core BGP-4 state machine without breaking backwards compatibility across thousands of hardware vendors, security had to be built

out-of-band: through Internet Routing Registries (IRR), the Resource Public Key Infrastructure (RPKI), and Route Origin Validation (ROV).

The Threat Landscape: Accidents and Intentional Attacks

The Threat Landscape: Accidents and Intentional Attacks

Securing an interdomain network requires an adversarial threat model. Disruption in global routing falls along a continuum: from benign clerical typos committed by tired engineers during maintenance windows, to sophisticated, state-sponsored cyber operations designed to covertly intercept encrypted communications.

Accidental Misconfiguration: The Most Common Disruption

Historically, over 90% of global routing anomalies are caused not by malicious attackers, but by accidental human misconfigurations. The most common root causes include: - **Fat-finger errors**: Entering `10.0.0.0/8` or a public IP block instead of a private internal block in an IGP redistribution statement. - **Leaked IGP into BGP**: Redistributing an entire interior routing table (containing hundreds of subnets) into external BGP without an attached route-map or prefix filter. - **Omission of export filters**: Bringing up a new eBGP transit session with an upstream provider while forgetting to apply an outbound route filter, inadvertently advertising full-table routes learned from other upstreams.

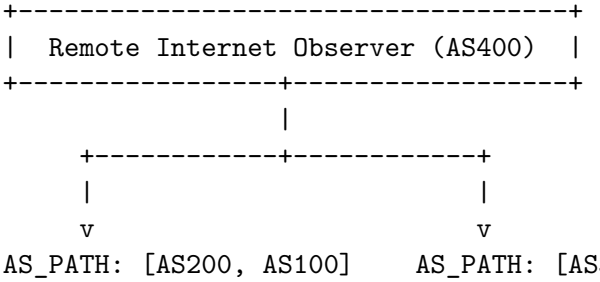
Because BGP treats every received `UPDATE` with equal validity, a single typo in a remote edge router in Melbourne can disrupt connectivity for millions of users in North America within 60 seconds.

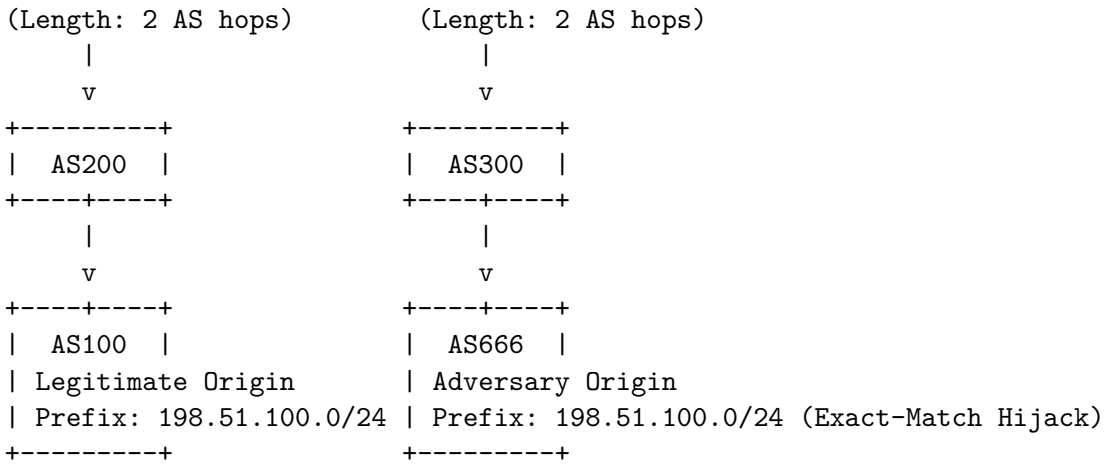
Origin Prefix Hijacking and Impersonation

In an **Origin Prefix Hijack**, an unauthorized Autonomous System (AS_{bad}) announces an IP prefix P that belongs to another network (AS_{victim}), falsely claiming that it is the legitimate origin:

$$\text{Legitimate Announcement:} \quad P \longleftarrow [AS_{\text{transit}}, AS_{\text{victim}}]$$

$$\text{Hijack Announcement:} \quad P \longleftarrow [AS_{\text{bad}}]$$





When an exact-match hijack occurs, the Internet splits: routers topologically closer to AS_{bad} route traffic to the attacker, while routers closer to AS_{victim} continue routing to the legitimate owner.

Subprefix Hijacking and Splitting

A **Subprefix Hijack** exploits the fundamental rule of IP packet forwarding: **Longest Prefix Match (LPM)**. Forwarding hardware in every router always prefers a more specific route over a covering aggregate.

If the legitimate owner announces an aggregate block:

$$\text{Legitimate:} \quad 198.51.100.0/22 \quad (\text{Addresses: } 198.51.100.0 - 198.51.103.255)$$

The attacker announces two constituent subprefixes:

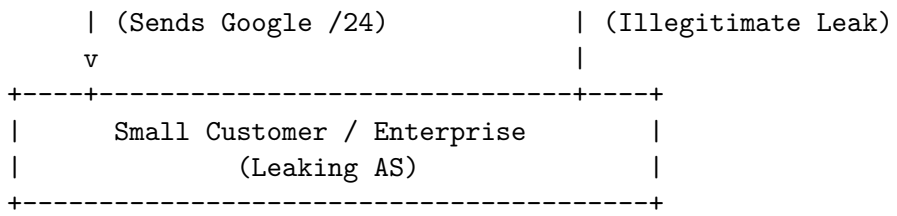
$$\text{Attacker:} \quad 198.51.100.0/24 \quad \text{and} \quad 198.51.101.0/24$$

Because `/24` is a longer mask than `/22`, **every router on Earth will prefer the attacker's route**, regardless of `LOCAL_PREF`, `AS_PATH` length, or topological distance. The hijack is 100% effective globally.

Route Leaks: Transit Rules Violated

Formally defined in RFC 7908, a **Route Leak** occurs when a multihomed Autonomous System receives a route from one transit provider or peer, and re-advertises it to another transit provider or peer in violation of standard commercial transit agreements.





When Customer AS advertises routes learned from Provider A to Provider B, Provider B sees Customer as the lowest-cost path. Provider B diverts massive traffic streams destined for major content networks (Google, AWS, Cloudflare) through Customer's modest 1 Gbps uplink. The customer router's interfaces instantly saturate, inducing massive global packet loss.

Blackholing and Traffic Interception

Depending on how the attacker configures its data plane, a hijack manifests in one of two ways: - **Blackholing (Denial of Service)**: The attacker advertises the prefix but discards all incoming packets (`null0` / drop). Legitimate services become completely unreachable. - **Traffic Interception (Man-in-the-Middle)**: The attacker captures, inspects, or modifies incoming traffic, and then forwards the packets onward to the true destination via a clean, secondary tunnel (e.g., GRE or WireGuard) to avoid detection.

AS_PATH Manipulation and Policy Exploitation

Because BGP lacks cryptographic path integrity, an attacker can modify the `AS_PATH` attribute before transmitting an `UPDATE`: - **AS_PATH Prepending Abuse**: An attacker injects victim ASNs into the path to trigger loop-detection defenses in targeted networks, effectively cutting off their visibility. - **Fake 1-Hop Paths**: An attacker announces a stolen route while forging a direct connection to a Tier-1 provider (`AS_PATH: [Tier1_ASN]`), tricking downstream networks into preferring the forged path over legitimate multi-hop routes.

Control-Plane Exhaustion and Volumetric Attacks

Routers maintain routing tables in expensive Ternary Content-Addressable Memory (TCAM) and process updates using onboard CPUs. Attackers can exploit this through: - **Table Bloat**: Inundating an edge router with hundreds of thousands of disaggregated `/24` prefixes, exhausting router RAM or exceeding hardware TCAM limits, causing line cards to crash. - **Update Churn**: Generating rapid bursts of advertisements and withdrawals designed to drive control-plane CPU utilization to 100%, causing BGP keepalive timeouts and triggering cascading session resets across the network.

Compromised Routers and Network Operators

The threat model must account for insider threats: - **Rogue / Hostile ISPs**: Commercial or state-owned telcos that intentionally weaponize their transit sessions to hijack financial services or suppress dissidents. - **Compromised Edge Devices**: Internet-exposed router management interfaces (SSH, RESTCONF, SNMP) compromised via credential stuffing or zero-day vulnerabilities, allowing adversaries to directly alter BGP configurations.

Origin Hijacking Attacks: Mechanics, Impact and Detection

Origin Hijacking Attacks: Mechanics, Impact and Detection

Origin prefix hijacking is the archetypal interdomain routing attack. In an origin hijack, an attacker illegitimately announces an IP block from its own Autonomous System, corrupting the global mapping between prefixes and authoritative origin networks.

Attack Prerequisites and Execution Steps

Executing an origin prefix hijack requires minimal technical resources:

```
[Attacker with BGP Peering]
             |
             | 1. Configure prefix under local BGP daemon (e.g., FRR/BIRD)
             v
   "network 203.0.113.0/24"
             |
             | 2. Upstream transit provider lacks strict prefix filtering
             v
  [Upstream ISP Border Router]
             |
             | 3. ISP accepts and propagates UPDATE to peers and Tier-1s
             v
 [Global Internet DFZ (Default-Free Zone)]
```

Step 1: Upstream Peering Access

The attacker must operate or control an Autonomous System with an active eBGP peering session to at least one upstream transit provider or IXP.

Step 2: Identification of Unfiltered Upstream

The attacker identifies an upstream provider that fails to enforce strict prefix-list filtering (e.g., relying on loose customer filters or failing to validate against IRR databases).

Step 3: Prefix Injection

The attacker adds the victim's prefix to its BGP configuration (e.g., in FRR: `network 203.0.113.0/24`) and applies an export policy.

Step 4: Upstream Propagation

The upstream provider accepts the announcement, prepends its own ASN, and propagates the `UPDATE` across its transit mesh. Within 30 to 60 seconds, routers globally evaluate the new path.

Complete vs. Partial Hijacking

The blast radius of an origin hijack depends entirely on prefix specificity:

1. Complete Hijacking (Subprefix / More-Specific)

If the victim announces `/22` and the attacker announces `/24`, every BGP speaker in the world that receives the announcement will direct traffic to the attacker due to **Longest Prefix Match (LPM)**. The victim experiences a complete global blackout for that `/24` address space.

2. Partial Hijacking (Exact-Match)

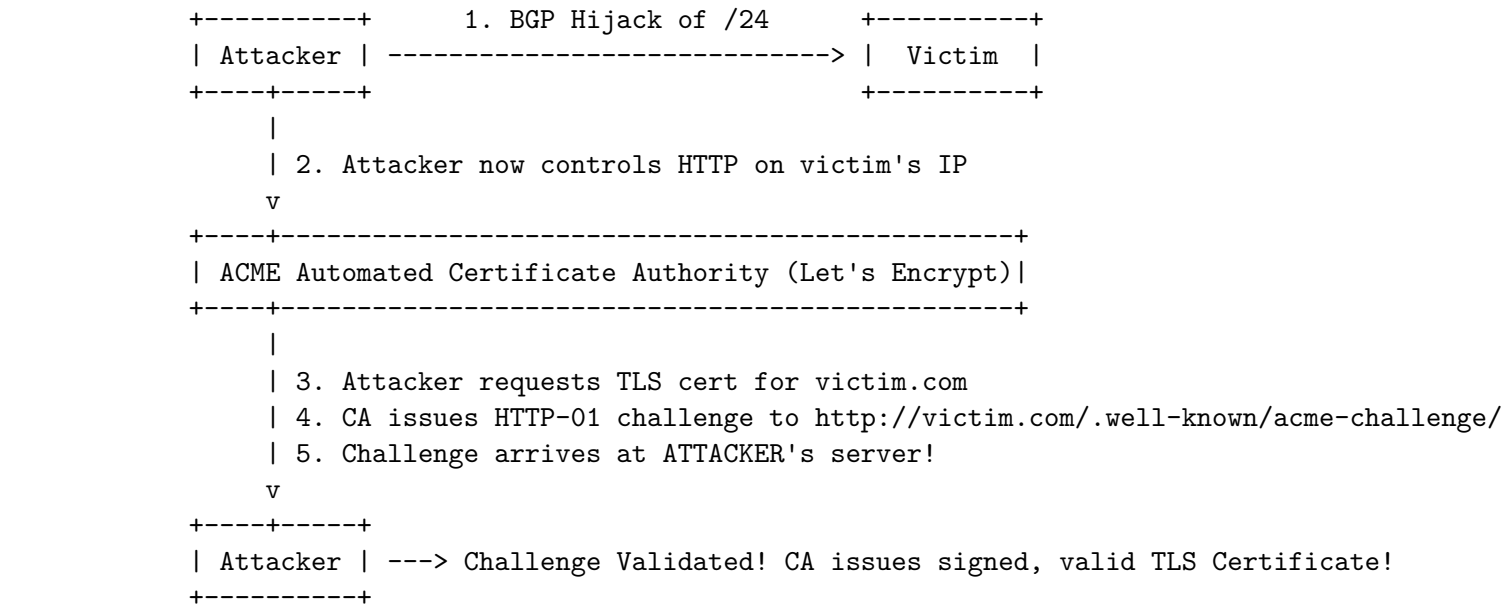
If both the victim and the attacker announce the identical `/24` prefix, the Internet bifurcates based on the BGP decision process (principally `LOCAL_PREF` and `AS_PATH` length): - Networks that have a direct peering or shorter AS path to the attacker route to the attacker. - Networks closer to the victim continue routing to the victim. - Upstream Tier-1 providers with high-degree mesh connectivity become battlegrounds where small differences in propagation time or path length decide the winner.

Confidentiality and Integrity Implications

Once an attacker intercepts traffic, severe security breaches become possible: - **Traffic Sniffing and Decryption**: Unencrypted traffic (DNS, HTTP, unauthenticated database replication) is captured in plaintext. If TLS is used, the attacker can log metadata (IP flows, SNI headers, packet timing) or attempt SSL stripping. - **Data Modification**: The attacker acts as an active proxy, altering DNS responses or inserting malicious payloads into unencrypted data streams.

How Hijacking Enables Phishing and MITM

Origin hijacking is frequently combined with automated Certificate Authority (CA) exploitation:



1. The attacker hijacks the IP prefix hosting the victim's web infrastructure.
2. The attacker requests a new TLS certificate from an ACME-based Certificate Authority (such as Let's Encrypt) for `target-bank.com`.
3. The CA performs an **HTTP-01 domain validation challenge**, sending an HTTP GET request to `target-bank.com/.well-known/acme-challenge/<TOKEN>`.
4. Because the attacker controls the IP in BGP, the CA's probe routes directly to the attacker's server, which responds with the token.
5. The CA issues a cryptographically valid, trusted TLS certificate.
6. The attacker can now terminate TLS connections from legitimate users with green browser padlocks, capturing credentials and session tokens without triggering browser certificate warnings.

Detection via Route Monitors, Looking Glasses and Real-Time Alerts

Detecting an origin hijack requires external vantage points:

- **Public Route Collectors**: Projects like **RouteViews** (University of Oregon) and **RIPE RIS** (Routing Information Service) peer with hundreds of border routers globally, logging all BGP updates and periodic table dumps into public archives.
- **Looking Glasses**: Public web interfaces provided by Tier-1 providers allowing operators to execute `show ip bgp <prefix>` from their internal core routers.
- **Real-Time Streaming Telemetry**:
  - **RIS-Live**: Real-time WebSocket firehose of global BGP updates.
  - **BGPStream**: Open-source framework for processing real-time BGP telemetry.

– **Commercial Alerting**: Tools like Cloudflare Radar, Cisco ThousandEyes, and Kentik monitor customer prefixes and trigger SMS/webhook alerts when an unexpected origin ASN appears in an `AS_PATH`.

The Role of RPKI and ROV in Origin Hijacking Prevention

The definitive cryptographic defense against origin hijacking is **Resource Public Key Infrastructure (RPKI)** combined with **Route Origin Validation (ROV)**.

Under RPKI: 1. The prefix owner signs a cryptographic **Route Origin Authorization (ROA)** stating: *Only AS100 is authorized to originate 198.51.100.0/24 up to maxLength /24.* 2. Remote routers fetch these validated ROAs. 3. When the attacker ($AS666$) announces `198.51.100.0/24`, routers evaluating ROV flag the announcement as **Invalid**. 4. The router automatically drops the Invalid announcement, completely neutralizing the hijack at the border.

Route Leaks and Policy Abuse

Route Leaks and Policy Abuse

While prefix hijacking involves the illegitimate origination of IP address space, **route leaks** represent a distinct, highly disruptive class of interdomain routing anomalies. Route leaks violate commercial routing policy agreements, causing traffic to flow through unauthorized networks without changing the authoritative origin ASN.

What Is a Route Leak and How It Differs From Hijacking

Formally defined in **RFC 7908**, a route leak is: > *"The propagation of routing announcement(s) beyond their intended scope. That is, an announcement from an Autonomous System (AS) of a learned BGP route to another AS in violation of the intended rules of the receiver, the sender, and/or one of the preceding ASes in the path."*

The critical distinction between a hijack and a leak:

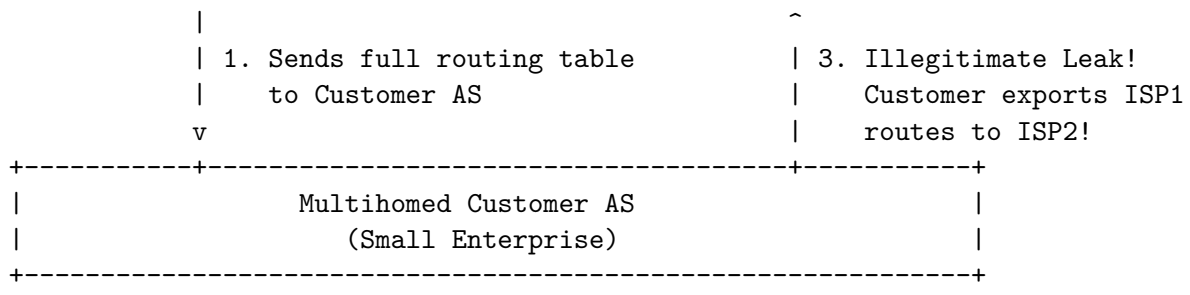
Characteristic	Origin Hijack	Route Leak
Origin ASN	Illegitimate; changed to attacker's ASN	**Legitimate**; original authoritative ASN preserved
`AS_PATH`	Forged or newly originated ($[AS_{\text{bad}}]$)	Contains valid origin, but path contains an illegal valley
Root Cause	Malicious injection or misorigination	Policy violation / export filter omission
RPKI ROV Defense	**Effective** (evaluates prefix vs. origin ASN)	**Ineffective** (origin ASN is valid!)

Because RPKI Route Origin Validation only checks the rightmost (origin) ASN in the `AS_PATH`, **ROV cannot prevent or detect a standard route leak.**

The Customer-Concentrator-Provider Leak Pattern

The most damaging and widespread route leak pattern is the **Customer-Provider Leak** (RFC 7908 Type 1):





1. Multihomed Customer AS purchases transit from ISP1 and ISP2.
2. ISP1 announces a full global routing table (over 900,000 routes) to Customer.
3. Customer accidentally misconfigures its outbound export route-map on its session to ISP2, failing to restrict exports to its own locally originated prefixes.
4. Customer advertises ISP1's routes to ISP2.
5. In ISP2's BGP decision process:
    - Routes learned from Customer have a higher `LOCAL_PREF` (e.g., 200, customer route) than routes learned from peers (150) or other transit providers (100).
6. ISP2 accepts the leaked routes and directs all traffic destined for those prefixes into Customer's network!

Accidental vs. Intentional Policy Violations

- **Accidental Leaks**: Caused by missing outbound prefix-lists, regex typos in BGP community filters, or software crashes during route-map recompilation.
- **Intentional / Adversarial Leaks**: An adversary intentionally leaks routes to:
    - Force competitor traffic onto saturated links (economic sabotage).
    - Act as a transparent Man-in-the-Middle (MitM) snooping node between two major transit backbones.
    - Exploit asymmetric peering billing structures.

Propagation Patterns and Impact on Reachability

When a route leak occurs: 1. **Traffic Funneling**: Upstream providers divert petabits of planetary transit traffic through the leaking AS. 2. **Buffer Exhaustion and Packet Loss**: The leaking network's edge routers, typically provisioned with 1 Gbps or 10 Gbps uplinks, are instantly overwhelmed by 500+ Gbps of incoming traffic. Queues drop 99.9% of packets. 3. **Global Service Collapse**: Remote users attempting to reach major cloud providers, CDNs, or financial hubs experience total timeouts.

Prevention Through Policy, Filtering and Community Tags

Mitigating route leaks requires defense in depth:

1. Strict Outbound Prefix Filtering

Every non-transit customer must configure strict outbound export filters that only allow the exact prefixes originated by the organization:

```
ip prefix-list OWN-PREFIXES permit 198.51.100.0/24
!
route-map EXPORT-TO-TRANSIT permit 10
 match ip address prefix-list OWN-PREFIXES
route-map EXPORT-TO-TRANSIT deny 20
```

2. BGP Communities for Leak Prevention

Operators use communities to classify routes upon ingress and reject them upon egress: - Tag all routes learned from Transit/Peers with `COMMUNITY 65000:TRANSIT`. - Drop any route carrying `65000:TRANSIT` before advertising to any other Transit/Peer.

3. RFC 9234: Autonomous System Provider Authorization (ASPA) and Only to Customer (OTC)

RFC 9234 defines the **Only to Customer (OTC)** BGP attribute: - When a provider or peer advertises a route to a customer or lateral peer, it attaches the `OTC` attribute set to its own ASN. - If a downstream network receives a route with an `OTC` attribute from a customer or lateral peer, it recognizes an illegal "valley" path and automatically rejects the route.

Detection via Monitoring and Analysis

Route leaks are detected by analyzing `AS_PATH` patterns across public collectors: - **Valley Detection**: Identifying sequences in the `AS_PATH` where a route moves from Customer → Provider → Customer → Provider. - **Volume Anomalies**: Sudden spikes in prefix count advertised by small stub ASes (e.g., an AS normally originating 3 prefixes suddenly originates 400,000 prefixes).

Advanced and Emerging Routing Threats

Advanced and Emerging Routing Threats

As basic origin prefix filtering and RPKI Route Origin Validation gain adoption across Tier-1 and Tier-2 networks, sophisticated adversaries and nation-state intelligence agencies have shifted to advanced evasion techniques, targeting routing infrastructure, telemetry systems, and policy manipulation.

Compromised Routers and Rogue Operators

Traditional threat models assume that router operating systems adhere strictly to the BGP state machine and RFC specifications. This assumption collapses when the router itself is compromised: - **Router Implant Malware**: State-sponsored actors have successfully compromised edge and core router platforms (e.g., Cisco IOS XE implants, Juniper Junos backdoors), injecting modified routing binaries directly into system memory. - **Rogue Bulletproof Hosting Operators**: Specialized transit entities operate with the explicit commercial model of providing routing transit to bulletproof hosting networks, phishing cartels, and botnet command-and-control operators. These rogue ASes frequently forge peering agreements, cycle through discarded or stolen AS numbers, and announce unallocated address space (bogon squatting).

Competitive Manipulation: Steering Traffic for Advantage

Routing manipulation is not limited to espionage; it is increasingly weaponized for economic and commercial advantage: - **Asymmetric De-peering and Traffic Dumping**: A large network intentionally manipulates MED or AS_PATH prepending to force a rival peer to carry heavy transit payloads across its internal backbone while taking minimal return traffic. - **Latency Degraded Peering**: Manipulating routing policies to detour traffic destined for a competitor's cloud or gaming platform through high-latency transoceanic paths, degrading user experience and inducing customer churn.

Attacks on Route Collectors and Validation Infrastructure

The global routing monitoring ecosystem relies heavily on public collectors (RouteViews, RIPE RIS, BGPStream): - **Telemetry Poisoning**: An attacker peering with public collectors can intentionally inject false or deceptive routes directly into collector sessions while filtering those same routes from commercial transit links. This blinds detection systems or floods analysts with false positive alerts. - **Denial-of-Service against RPKI Validators**: By flooding RPKI repositories with millions of deeply nested manifests, invalid certificates, or malicious rsync directories, attackers attempt

to trigger algorithmic complexity attacks (denial-of-service) against relying party validators like Routinator or rpki-client.

Long-Term Reconnaissance and Persistent Manipulation

Sophisticated adversaries rarely announce noisy `/24` subprefixes that trigger immediate alarms on Twitter/X or Slack channels: - **Micro-burst Hijacking**: Announcing a target prefix for only 60 to 120 seconds—long enough to complete an automated TLS certificate challenge or siphon authentication tokens—and withdrawing the route before global NOCs can initiate diagnostic traceroutes. - **Low-and-Slow Interception**: Manipulating the `AS_PATH` by appending legitimate ASNs so the route appears to be a normal multi-hop path. The adversary diverts a small percentage of global traffic through an offshore intercept facility, decrypts or logs the data, and forwards it to the true destination without dropping packets.

Threats Specific to Large Networks and Cloud Providers

Hyperscale cloud providers (AWS, Azure, GCP, Cloudflare) operate massive, globally distributed Anycast networks. This architecture exposes distinct threat vectors: - **Anycast Localization Attacks**: Announcing an Anycast prefix from a single rogue geographic location (e.g., Eastern Europe) to capture local user traffic destined for a global DNS resolver (like `1.1.1.1` or `8.8.8.8`). - **Internal Cross-Tenant Leaks**: In multi-tenant cloud environments where thousands of enterprise customers run virtual BGP sessions over Direct Connect or ExpressRoute, configuration bugs in cloud software-defined network (SDN) controllers can allow one customer's private routes to leak into another tenant's virtual private cloud (VPC).

The Routing Registries: IRRs and Their Limitations

The Routing Registries: IRRs and Their Limitations

Long before cryptographic validation through RPKI was deployed, the network operations community created the **Internet Routing Registry (IRR)** system to record routing policies, prefix ownership, and transit relationships. While IRRs remain widely used to generate automated prefix-lists, their fundamental design weaknesses highlight why cryptographic authentication became necessary.

What Routing Registries Are and How They Store Route Objects

An Internet Routing Registry is a distributed database storing text-based policy statements formatted in **Routing Policy Specification Language (RPSL, RFC 2622)**.

The IRR system is not a single database; it is a confederation of more than 30 independent registries worldwide: - **Authoritative RIR Registries**: Run by Regional Internet Registries (RIPE, APNIC, AFRINIC, LACNIC, and ARIN-RR), containing records tied to actual resource allocations. - **Third-Party / Commercial Registries**: Including RADB (Merit), NTT, Level3/Lumen, and ALTDB, established historically to allow multi-region operators to register policy objects without cross-RIR coordination.

Registries mirror each other's databases via daily or hourly bulk dumps, but synchronization is loose, partial, and frequently inconsistent.

Creating Route Objects: Route, Route6, AS-SET and Related Classes

RPSL defines several core object classes:

1. `route` (IPv4) and `route6` (IPv6) Objects

A `route` object binds an IP prefix to an authoritative origin Autonomous System:

```
route:          198.51.100.0/24
descr:          Example Production Web Backbone
origin:         AS65001
mnt-by:         EXAMPLE-MNT
source:         RADB
```

2. `aut-num` Objects

An `aut-num` object specifies the routing policy of an Autonomous System, detailing its import, export, and peering rules:

```
aut-num:        AS65001
as-name:        EXAMPLE-NET
import:         from AS65002 action pref=100; accept ANY;
export:         to AS65002 announce AS-EXAMPLE;
mnt-by:         EXAMPLE-MNT
source:         RADB
```

3. `as-set` Objects

An `as-set` is a hierarchical grouping of Autonomous Systems representing an entire customer cone. Upstream transit providers use `as-set` objects to recursively build prefix-lists for an entire downstream customer hierarchy:

```
as-set:         AS-EXAMPLE-CUSTOMERS
descr:          Downstream customers of AS65001
members:        AS65010, AS65020, AS-SUB-CLIENTS
mnt-by:         EXAMPLE-MNT
source:         RADB
```

Policy-Based Filtering: Using IRR Data to Build Filters

Modern Tier-1 and Tier-2 transit providers use open-source tools such as **`bgpq4`** or **`irrpt`** to query IRR databases and automatically generate vendor-specific prefix filters:

```
# Query RADB and generate a Cisco IOS XR prefix-list for AS-EXAMPLE-CUSTOMERS
bgpq4 -X -l CUSTOMER-IN AS-EXAMPLE-CUSTOMERS
```

Output:

```
prefix-set CUSTOMER-IN
  198.51.100.0/24,
  203.0.113.0/24 le 24
end-set
```

These generated prefix-lists are applied to customer-facing eBGP sessions, preventing customers from advertising arbitrary unauthorized prefixes.

Weaknesses: No Authorization Checking, No Cryptographic Binding

Despite its widespread operational use, the legacy IRR system suffers from critical security flaws:

1. **No Authorization Checking in Third-Party IRRs**: In non-authoritative registries (such as RADB or legacy ALTDB), anyone with a paid account could register a `route` object for *any* IP address on Earth. An attacker could create a fake `route` object claiming that `1.1.1.1/32` originates from `AS666`. If an upstream transit provider built its filters using an unvetted query (`-S radb`), the attacker's route would be automatically permitted through the provider's border filters!
2. **Stale and Ghost Objects**: When organizations renumber, merge, or return IP blocks to an RIR, they rarely delete their old IRR objects. Millions of obsolete `route` objects remain active in mirrored registries decades after the addresses were reassigned.
3. **No Cryptographic Signatures**: IRR objects are plaintext text files. They contain no digital signatures, no public-key certificates, and no cryptographic binding to the resource holder's cryptographic keys.

The Transition from IRR Filtering to RPKI-Based Validation

Because the legacy IRR system could not guarantee origin authenticity or prevent rogue registrations, the IETF developed the **Resource Public Key Infrastructure (RPKI)**.

While RPKI provides cryptographic truth for origin validation, IRRs remain operationally necessary for expanding `as-set` customer cones and generating strict inbound prefix-lists at transit borders. In modern network operations, IRR and RPKI are deployed as complementary controls: **IRR validates the path and customer cone, while RPKI cryptographically verifies the origin.**

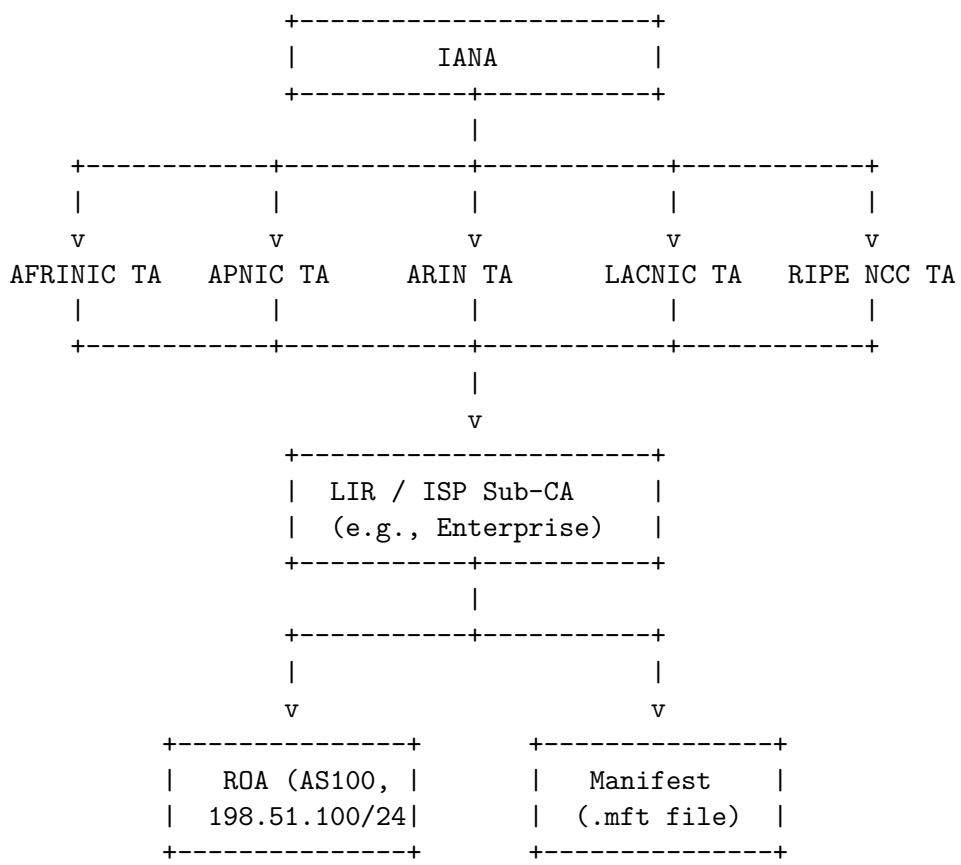
The RPKI Architecture: Resource Certificates and Trust Anchors

The RPKI Architecture: Resource Certificates and Trust Anchors

The **Resource Public Key Infrastructure (RPKI)**, standardized in RFC 6480, is a specialized Public Key Infrastructure designed to cryptographically bind Internet number resources (IPv4 blocks, IPv6 blocks, and Autonomous System Numbers) to public-key certificates held by the organizations to which those resources were formally allocated.

The RPKI Trust Model: Hierarchical Certification from RIRs

Unlike the commercial Web PKI (where hundreds of commercial Certificate Authorities can issue TLS certificates for any domain on Earth), RPKI strictly follows the **hierarchical allocation chain of the Internet**:



The tree begins at IANA and cascades down through the five Regional Internet Registries (RIRs) to Local Internet Registries (ISPs, telcos, cloud providers) and enterprise end-users. A certificate in RPKI cannot authorize resources that were not explicitly delegated down the parent chain.

Trust Anchors and Root Certificates at Each RIR

Every relying party (validator) bootstraps its validation process using **Trust Anchor Locators (TALs)** (RFC 8630). A TAL is a small text file containing: - The URI of the RIR's root certificate (hosted over HTTPS or rsync). - The subject public key identifier (SPKI) and public key of the RIR's self-signed Trust Anchor root certificate.

There are five primary Trust Anchors corresponding to the five RIRs: 1. `arin.tal` 2. `ripe.tal` 3. `apnic.tal` 4. `lacnic.tal` 5. `afrinic.tal`

A validator reads these TALs on startup, downloads the root certificates from the specified publication points, verifies their cryptographic fingerprints, and constructs the root of the validation tree.

Resource Certificates: Structure, Extensions and Resource Holders

RPKI certificates are standard X.509 v3 public key certificates with a critical addition: the **IP Address and AS Number Extensions (RFC 3779)**.

In standard TLS certificates, the certificate subject is a Common Name or Domain (e.g., `CN=example.com`). In an RPKI certificate: - The Subject field contains a non-identifying pseudonym. - The RFC 3779 extensions explicitly list the exact IP address ranges and Autonomous System Numbers the certificate holder is authorized to manage:

```
X509v3 extensions:
  IP Address Delegation:
    IPv4: 198.51.100.0/24
    IPv6: 2001:db8:abcd::/48
  Autonomous System Numbers:
    AS65001
```

A parent CA can only issue a child certificate for a subset of the resources listed in the parent's own certificate. If a rogue administrator at an ISP attempts to issue a certificate for someone else's `/16` block, chain validation fails immediately because the parent certificate does not contain those resources.

CA Certificates and Chain Validation

RPKI distinguishes between two certificate types: - **CA Certificates (Certificate Authority)**: Used to sign child certificates, Certificate Revocation Lists (CRLs), and signed object lists. - **End-Entity (EE) Certificates**: Ephemeral, single-use certificates created solely to sign a specific

payload object—such as a Route Origin Authorization (ROA) or an ASPA object. Once the object is signed, the private key is discarded.

During chain validation, a relying party walks down the cryptographic path from the RIR Trust Anchor to the EE certificate, verifying: 1. That all signatures across the path are mathematically valid (RSA or ECDSA). 2. That each certificate is within its valid start and expiration dates (`notBefore` / `notAfter`). 3. That the resources in the child certificate are strictly a sub-allocation of the parent's RFC 3779 extensions. 4. That no certificate in the chain has been revoked on an active CRL.

Manifests and Signed Object Lists: What They Protect

Every RPKI repository publication point publishes a **Manifest (RFC 6486)**. A manifest is a signed CMS object (`.mft`) that lists the filenames and SHA-256 cryptographic hashes of every valid object currently published in that repository directory.

Manifests protect relying parties against **replay and deletion attacks**: - If an adversary compromises a local repository mirror and deletes a valid ROA, the validator checks the manifest, detects that a listed file is missing, and raises an alarm. - If an attacker attempts to inject a revoked, stale ROA from six months ago, the manifest's hash check fails, preventing the stale authorization from entering the router's decision engine.

Route Origin Authorizations: The Core Authorization Object

Route Origin Authorizations: The Core Authorization Object

In the RPKI ecosystem, the digital object that directly governs BGP route validation is the **Route Origin Authorization (ROA)**, defined in RFC 6482. A ROA is a cryptographically signed statement that specifies which Autonomous System is authorized to originate a given IP prefix into the global BGP routing system.

What a ROA Is and What It Proves

A ROA is an encapsulated Cryptographic Message Syntax (CMS) signed object. It contains a signed payload that binds: 1. An **Autonomous System Number (ASN)**. 2. One or more **IP prefixes** (IPv4 or IPv6). 3. An optional **Maximum Length (`maxLength`)** for each prefix.

A valid ROA proves that the legitimate holder of the IP address space has authorized the specified AS to originate BGP announcements for that address block up to the specified prefix length.

Crucially, **a ROA proves origin authorization only**. It does not prove: - That the originating AS has active physical connectivity to the Internet. - That the AS path between the origin and the receiving router is honest or loop-free. - That the originating AS will actually forward data plane traffic.

Creating a ROA: Prefix, AS Number and Maximum Length

When a network operator creates a ROA through their RIR portal (e.g., ARIN Online, RIPE NCC LIR Portal) or via a delegated Krill CA, the ROA payload structure is formatted as follows:

```
RouteOriginAttestationPayload ::= SEQUENCE {
    asID            ASID,
    ipAddrBlocks    SEQUENCE OF ROAIPAddressFamily
}

ROAIPAddressFamily ::= SEQUENCE {
    addressFamily   OCTET STRING (2 octets),  -- IPv4 or IPv6
    addresses       SEQUENCE OF ROAIPAddress
}

ROAIPAddress ::= SEQUENCE {
    address         IPAddress,                -- e.g., 198.51.100.0/22
```

```
    maxLength       INTEGER OPTIONAL          -- e.g., 24
}
```

Example ROA entry: - **Prefix**: `198.51.100.0/22` - **Origin ASN**: `AS65001` - **Maximum Length (`maxLength`)**: `24`

This ROA authorizes `AS65001` to originate the aggregate `/22`, as well as any constituent `/23` or `/24` subprefixes (e.g., `198.51.100.0/24`, `198.51.101.0/24`). It explicitly forbids `AS65001`—and every other AS on Earth—from originating prefixes longer than `/24` (such as a `/25`).

Max-Length: How It Controls Subprefix Authorization

The `maxLength` attribute is an operational convenience, but it represents the most dangerous trap in RPKI:

ROA Configuration	Authorized Announcements	Unauthorized Announcements (Flagged INVALID)
`198.51.100.0/22` without `maxLength`	Exactly `198.51.100.0/22` only from AS65001	Any `/23`, `/24`, or subprefix from *any* AS (including AS65001)
`198.51.100.0/22` with `maxLength: 24`	`198.51.100.0/22`, and any `/23` or `/24` from AS65001	Any `/25` or longer from AS65001; ANY prefix from any other AS

The `maxLength` Security Vulnerability (RFC 9319)

If an operator announces only `198.51.100.0/22` in BGP, but creates a ROA with `maxLength: 24` "just in case we want to split it later", an attacker can execute a **forged-origin subprefix hijack**: 1. The attacker announces `198.51.100.0/24` with `AS_PATH: [Attacker_AS, 65001]`. 2. The router evaluating ROV checks the origin: the origin is `AS65001`. 3. The router checks prefix length: `/24` is ≤ 24 (`maxLength`). 4. The router flags the attacker's announcement as **VALID**! 5. Longest Prefix Match directs all traffic for that `/24` to the attacker.

> [!IMPORTANT] **RFC 9319 Best Current Practice**: **Do not use `maxLength` unless actively necessary.** Instead, create explicit minimal ROAs matching the exact prefix lengths currently advertised in BGP. If you announce a `/22` and two `/24`s, publish three distinct ROAs with exact lengths.

ROA Lifespan, Expiration and Revocation

ROAs are cryptographic objects with a defined validity period (`notBefore` and `notAfter` dates, typically 12 to 36 months).

- **Automated Re-signing**: Hosted RIR CAs automatically re-sign ROAs prior to expiration. If an operator runs a delegated CA (e.g., Krill) and lets the daemon crash or neglect automated re-signing, the ROA expires. Once expired, the prefix transitions from `Valid` to `NotFound`.
- **Revocation**: If a network changes upstream providers or reassigns an IP block, the old ROA is revoked. The CA signs a Certificate Revocation List (CRL) and publishes a new manifest.

Common Mistakes: Over-Permissive, Under-Covered and Conflicting ROAs

1. **Under-Covered ROAs (Self-Inflicted Outage)**: An operator announces `198.51.100.0/24` in BGP, but registers a ROA for `198.51.100.0/22` without `maxLength`. The router running ROV flags the operator's own `/24` announcement as **INVALID** because the announced prefix length (24) exceeds the authorized length (22). Upstream transit networks drop the route, severing connectivity.
2. **Conflicting Multi-Origin ROAs**: When transitioning between transit providers, an operator must ensure that ROAs for *both* ASNs exist concurrently before shifting BGP traffic, preventing temporary Invalid states during convergence.
3. **Ghost ROAs**: Retaining active ROAs for IP space that was transferred or sold to another organization.

RPKI Validation: Validators, VRPs and Relying Parties

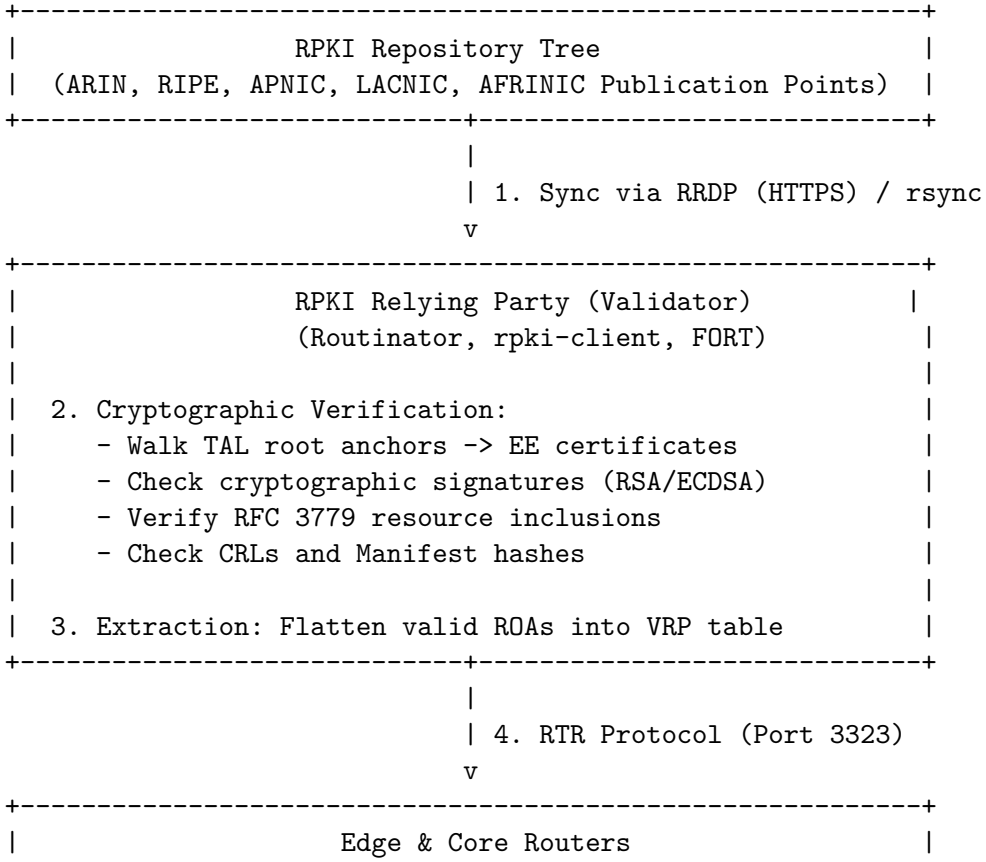
RPKI Validation: Validators, VRPs and Relying Parties

Routers cannot directly walk the planetary RPKI certificate tree. Public-key signature verification, recursive repository crawling, and cryptographic certificate chain parsing require substantial computational power, memory, and disk I/O that router control-plane processors do not possess.

This structural reality necessitates an architectural split: an external server software package—the **RPKI Relying Party (RP) or Validator**—performs the heavy cryptographic heavy lifting and delivers a simple, flattened table to routers.

What an RPKI Validator Does: Download, Verify, Extract

An RPKI Validator performs four primary tasks in a continuous operational loop (typically every 10 to 20 minutes):



```
|                    (Cisco, Juniper, Nokia, FRR)            |
+------------------------------------------------------------+
```

1. **Repository Synchronization**: Synchronizes local cache with all global RPKI publication repositories.
2. **Cryptographic Validation**: Validates the entire certificate chain from the 5 Trust Anchor Locators (TALs) down through intermediate CAs to End-Entity certificates and ROAs.
3. **Revocation and Integrity Checking**: Confirms that certificates have not been revoked on Certificate Revocation Lists (CRLs) and that all files match the hashes recorded on signed manifests.
4. **VRP Extraction**: Extracts valid authorizations into a flattened list of **Validated ROA Payloads (VRPs)** and serves them to routers over the RPKI-to-Router (RTR) protocol.

Validator Architecture and Repository Sync: RRDP vs. rsync

Historically, RPKI repositories synchronized using `rsync`. In production, `rsync` proved disastrously fragile: - Scaling bottlenecks: Thousands of validators executing concurrent `rsync` crawls overwhelmed RIR server CPU and memory. - Partial updates: Network dropouts during `rsync` transfers left directories with missing or inconsistent certificate sets.

The IETF resolved this by introducing **RPKI Repository Delta Protocol (RRDP, RFC 8182)**: - Operates over standard **HTTPS**. - Uses an XML-based **Notification File** (`notification.xml`), **Snapshot Files**, and **Delta Files**. - Enables caching through commercial Content Delivery Networks (CDNs), allowing millions of queries per second without placing load on RIR origin servers. - Validators only fall back to `rsync` if an RRDP publication endpoint fails.

Validated ROA Payloads (VRPs): The Validator's Output

The final output of an RPKI validation cycle is a flat table of **Validated ROA Payloads (VRPs)**. A VRP is a simplified 4-tuple:

```
VRP = (Prefix, PrefixLength, MaxLength, OriginASN)
```

Example VRP table dump:

IP Prefix	Prefix Length	Maximum Length (`maxLength`)	Authorized Origin ASN
`198.51.100.0`	24	24	`AS65001`
`203.0.113.0`	24	24	`AS65002`
`2001:db8::`	32	48	`AS65001`
`192.0.2.0`	24	24	`AS65100`

This compact table is all a router needs to make real-time forwarding decisions.

CRL and SKI/IAI: Revocation and Validation Accuracy

- **Certificate Revocation Lists (CRLs)**: Each CA periodically signs a CRL listing the serial numbers of revoked certificates. Validators must download the CRL before evaluating any certificate issued by that CA.
- **Subject Key Identifier (SKI) & Authority Key Identifier (AKI)**: Ensure exact cryptographic matching between the public key in a parent CA certificate and the issuer signature on a child certificate, preventing key substitution attacks.

Validator Outages, Stale Data and Recovery Behavior

What happens when an RPKI validator server crashes, loses Internet connectivity, or is partitioned from the RIR repositories?

- **Local Cache Persistence**: Validators do not purge their databases when an upstream sync fails. They continue serving the last known valid VRP set to routers until certificates and manifests reach their cryptographic expiration dates.
- **Router Cache Timer**: Routers maintain an RTR expiry timer (default typically 60 to 120 minutes). If all RTR sessions to all local validators drop, the router preserves its VRP table in memory until the timer expires.
- **Redundant Validator Architecture**: Best operational practice dictates deploying at least two independent validator instances running different software implementations (e.g., **Routinator** by NLnet Labs and **rpki-client** by the OpenBSD project) in separate physical data centers to prevent single-vendor software bugs from taking down the routing policy.

RPKI-to-Router (RTR) and Route Origin Validation (ROV)

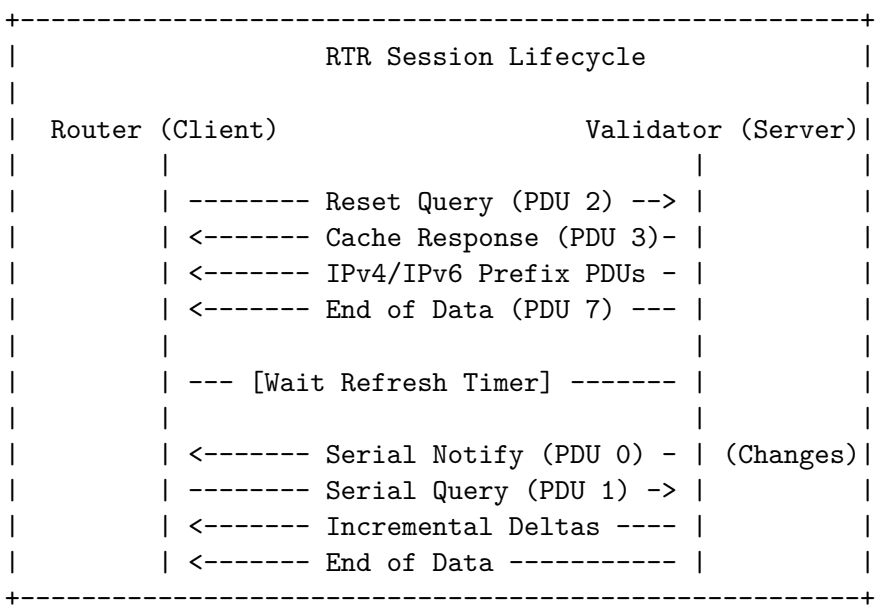
RPKI-to-Router (RTR) and Route Origin Validation (ROV)

Once an RPKI validator has downloaded, cryptographically verified, and extracted the global set of Validated ROA Payloads (VRPs), this data must be delivered into router memory. The protocol that bridges external validators and router control planes is the **RPKI-to-Router (RTR) protocol**, defined in RFC 6810 (RTRv0) and RFC 8210 (RTRv1).

The RPKI-to-Router Protocol: Messages and Mechanics

RTR is a lightweight, binary client-server protocol running over TCP (default port **323** for plaintext; port **3323** when wrapped in TLS or SSH).

The protocol operates using explicit Protocol Data Units (PDUs):



PDU Type	Name	Purpose
0	`Serial Notify`	Sent asynchronously by the validator to inform routers that the VRP cache has been updated to a new Serial Number.
1	`Serial Query`	Sent by the router requesting incremental changes (adds/deletes) since its last known Serial Number.
2	`Reset Query`	Sent by the router upon initial startup or cache corruption, requesting a complete, full dump of all VRPs.
3	`Cache Response`	Acknowledges a query and precedes payload transmission.
4	`IPv4 Prefix`	Transmits an IPv4 VRP: `[Flags (Add/Del), Prefix, Length, MaxLength, OriginASN]`.
6	`IPv6 Prefix`	Transmits an IPv6 VRP with identical fields formatted for 128-bit addresses.
7	`End of Data`	Marks the conclusion of a data transfer and specifies refresh, retry, and expire timers.
8	`Cache Reset`	Sent by the validator if the router's requested serial is too old to compute an incremental delta, forcing a Reset Query.
10	`Error Report`	Signals an encapsulation, syntax, or cryptographic error.

Session Establishment, Keepalive and Data Transfer

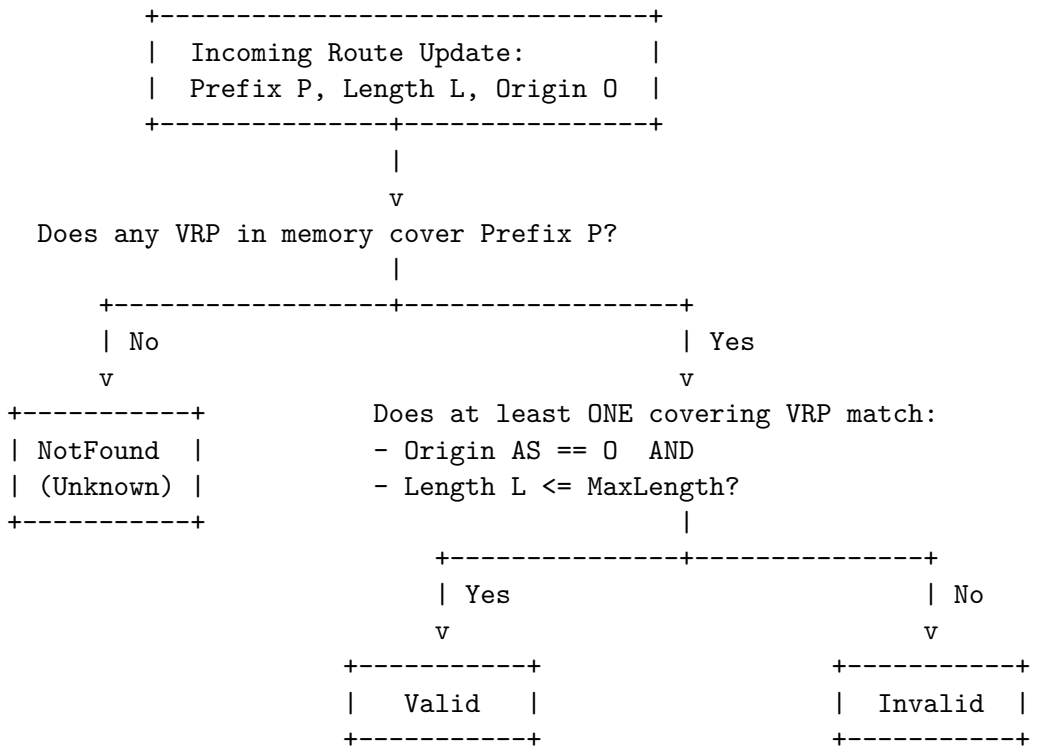
When a router boots: 1. It establishes a TCP session to the configured RTR validator IP on port 323. 2. It sends a `Reset Query` (PDU 2). 3. The validator responds with `Cache Response` (PDU 3), followed by a stream of `IPv4 Prefix` and `IPv6 Prefix` PDUs (over 500,000 records). 4. The

validator terminates the transfer with `End of Data` (PDU 7), containing: - **Refresh Interval**: How often the router should query for updates (default: 3600 seconds). - **Retry Interval**: How long to wait before retrying after a failed query (default: 600 seconds). - **Expire Interval**: How long the router may continue using its cached VRPs if all RTR sessions fail before purging them (default: 7200 seconds).

How VRPs Become Route Decision Criteria: Route Origin Validation (ROV)

With the VRP table resident in router memory, the router executes **Route Origin Validation (ROV)** (RFC 6811) on every incoming BGP route received from peers.

For an incoming route advertisement $R = (\text{Prefix } P, \text{Length } L, \text{Origin AS } O)$: The router queries its local VRP database for any entry $V = (P_V, L_V, M_V, O_V)$ that covers P (where P is a subprefix of or equal to P_V):



1. `Valid`: At least one covering VRP exists where the VRP ASN matches the origin ASN ($O_V == O$) **AND** the route prefix length is less than or equal to the VRP maximum length ($L \leq M_V$).
2. `Invalid`: At least one covering VRP exists, but **neither** the origin ASN matches **nor** the prefix length satisfies the maximum length requirement ($L > M_V$).
3. `NotFound` **(Unknown)**: No covering VRP exists in the database for prefix P.

ROV Integration with the BGP Decision Process

ROV states are integrated into the BGP decision engine via router policy (route-maps):

```
! Standard Production ROV Policy (FRR / Cisco Syntax)
route-map BGP-PEER-IN permit 10
 match rpki invalid
 set local-preference 10      <-- Or direct drop: drop / reject
!
route-map BGP-PEER-IN permit 20
 match rpki valid
 set local-preference 200     <-- Prefer cryptographically validated routes
!
route-map BGP-PEER-IN permit 30
 match rpki notfound
 set local-preference 100     <-- Neutral default preference for unvalidated space
```

In strict production hardening (RFC 7454), **all Invalid routes are immediately dropped upon receipt**:

```
route-map BGP-TRANSIT-IN deny 10
 match rpki invalid
!
route-map BGP-TRANSIT-IN permit 20
```

Handling State Transitions: Valid, Invalid, NotFound and Pending

When a new ROA is published or revoked: - The validator pushes an update over RTR. - The router dynamically re-evaluates all affected routes in its `Adj-RIB-In`. - If a currently installed route transitions from `NotFound` → `Invalid`, the router drops it and re-runs the BGP best-path selection algorithm to pick an alternative path. - During initial session bring-up, routes reside in a temporary `Pending` state for a few seconds until the initial RTR sync concludes, preventing premature route drops.

Operational Consequences of ROV: Invalid, Valid, NotFound and Real-World Effects

Operational Consequences of ROV: Invalid, Valid, NotFound and Real-World Effects

Enabling Route Origin Validation (ROV) changes packet forwarding in the global data plane. While ROV provides definitive protection against origin prefix hijacking, its operational enforcement requires navigating nuanced failure modes, operator errors, and unintended collateral reachability loss.

Rejecting Invalid Routes vs. De-Prioritizing Them

When deploying ROV, an operator must choose between two policy philosophies:

Option A: Hard Dropping (Drop Invalids)

```
route-map INBOUND-POLICY deny 10
 match rpki invalid
```

- **Security Guarantee**: 100% immune to origin hijacks and unauthorized subprefix injections.
- **Operational Risk**: If a legitimate content provider misconfigures its ROA, your network will drop all routes to that service, rendering it completely unreachable for your users.

Option B: Soft De-prioritizing (Deprecate Invalids)

```
route-map INBOUND-POLICY permit 10
 match rpki invalid
 set local-preference 10
```

- **Security Guarantee**: Fails against subprefix hijacks! If an attacker announces a `/24` subprefix, Longest Prefix Match overrides `LOCAL_PREF = 10`. The router still sends traffic to the attacker.
- **Operational Consensus**: The global networking community (MANRS, RFC 7454) firmly rejects Option B. **To achieve routing security, operators must drop Invalid routes entirely.**

The Risk of Rejecting Legitimate Traffic

The primary barrier preventing hesitant network operators from enabling ROV enforcement is the fear of self-inflicted collateral outages.

If a large university, government agency, or SaaS vendor creates a typo in its ROA (or fails to update its ROA when shifting traffic to a new CDN partner), the route is classified as `Invalid`. A Tier-1 ISP that drops Invalids will cut off access to that vendor for all its subscribers, prompting support escalation and angry executive inquiries.

NotFound: The Ambiguous Middle State

Currently, approximately 50% to 55% of global IPv4 prefixes have valid ROAs. The remaining 45% reside in the **`NotFound` (Unknown)** state.

- `NotFound` does not mean a route is legitimate; it merely means the prefix owner has not signed an RPKI ROA.
- Routers cannot drop `NotFound` routes; doing so would break connectivity to nearly half the Internet.
- Consequently, attackers targeting `NotFound` address space can still execute classic origin prefix hijacks uncontested by ROV.

Max-Length Policy Errors and Collateral Damage

As discussed in Chapter 14, `maxLength` mismatches account for over 80% of all benign `Invalid` route states observed globally:

```
[Legitimate Enterprise Allocation: 203.0.113.0/22]
                        |
        ROA published: 203.0.113.0/22 (No maxLength)
                        |
   Enterprise Network Operations adds DDoS Mitigation:
      Announces: 203.0.113.0/24 to Akamai/Cloudflare
                        |
                ROV Evaluation:
       Prefix Length (24) > Authorized Length (22)
                        |
                        v
         ROUTE FLAGGED INVALID AND DROPPED!
```

The enterprise inadvertently blackholes its own DDoS protection service because its network engineers did not coordinate with the team managing its RPKI portal.

Documented Incidents: When ROV Went Wrong

1. The Cloudflare / AWS Route 53 Incident (2018)

When attackers hijacked Amazon Route 53 DNS IP space (`205.251.192.0/24`) to drain MyEtherWallet cryptocurrency wallets, authoritative DNS queries were redirected to Russian servers. Networks that had enabled ROV dropped the hijack because Amazon had published valid ROAs, but networks lacking ROV fell victim.

2. The Orange España RPKI Account Takeover (January 2024)

A rogue actor accessed Orange España's RIPE NCC administrator portal via compromised credentials that lacked Multi-Factor Authentication (MFA). The intruder created fraudulent ROAs for Orange's primary infrastructure prefixes with bogus origin ASNs. Within minutes, global validators downloaded the new ROAs, flagged Orange España's legitimate BGP announcements as **Invalid**, and Tier-1 ISPs dropped the routes, knocking the entire national telco offline for several hours until RIPE administrators manually intervened.

The Scope and Limits of RPKI and ROV

The Scope and Limits of RPKI and ROV

RPKI Route Origin Validation is the cornerstone of modern routing defense, but it is not a silver bullet. Understanding what RPKI can and cannot prove is essential for architects designing resilient network perimeters.

What Origin Validation Proves and What It Does Not

The scope of RPKI ROV is strictly limited to the rightmost element of the BGP announcement:

Security Claim	Can RPKI ROV Prove It?	Explanation
Origin ASN Authorization	**YES**	Proves the rightmost ASN in `AS_PATH` is authorized by the resource holder to originate the prefix.
Prefix Length Authorization	**YES**	Enforces that announced prefix mask does not exceed `maxLength`.
Path Authenticity / Integrity	**NO**	Cannot verify whether intermediate ASNs in `AS_PATH` actually forwarded or received the route.
Data Plane Forwarding	**NO**	A router with a `Valid` route may still drop, blackhole, or snoop packets in the data plane.
Prevention of Route Leaks	**NO**	In a route leak, the origin ASN is legitimate; therefore, the route evaluates as `Valid` under ROV.

Why RPKI Cannot Prevent All Route Leaks

Consider the classic route leak scenario: - Google originates `8.8.8.0/24` with origin `AS15169`. - Google creates a valid ROA: `(8.8.8.0/24, maxLength=24, Origin=15169)`. - Transit Provider A sends the route to Customer B. - Customer B leaks the route to Transit Provider C with `AS_PATH: [Customer_B, Provider_A, 15169]`.

When Transit Provider C receives the route: 1. It inspects the origin ASN: `15169`. 2. It compares `8.8.8.0/24` against the VRP cache. 3. The origin matches `15169`. The prefix length matches `/24`. 4. Result: `Valid`.

Provider C installs the leaked route, directing its traffic into Customer B's saturated connection. **RPKI ROV validated the route as legitimate because the origin was never altered.**

The Dependency on Accurate, Complete ROA Data

RPKI is entirely dependent on human operators keeping their registry data synchronized with physical network changes: - If a network migrates its data center from one transit provider to another but forgets to update its ROA, legitimate traffic will be dropped as `Invalid`. - If an enterprise delegates address management to a regional branch that fails to pay its RIR maintenance fees, the parent RIR can revoke the resource certificate, instantly invalidating the enterprise's global routing.

Cryptographic Soundness vs. Operational Fragility

RPKI introduces an explicit coupling between interdomain routing (traditionally a distributed, loosely coupled control plane) and centralized cryptographic infrastructure: - **Centralized Dependencies**: If an RIR's publication server or TAL repository goes offline for extended periods or suffers DNS failures, validator caches decay. - **Clock Drift**: Because X.509 certificates and manifests rely on UTC timestamps, routers or validator servers with unsynchronized clocks (NTP drift) will reject valid certificates as either expired or not yet valid.

Misconceptions About What RPKI "Solves"

1. *"RPKI encrypts Internet traffic."* — **False**. RPKI validates BGP control-plane assertions; it does not touch or encrypt data-plane payload packets.
2. *"If I enable ROV, nobody can hijack my traffic."* — **False**. An attacker can execute a **forged-origin path hijack** by prepending your ASN to their announcement (`AS_PATH: [Attacker_AS, Victim_AS]`), tricking ROV into evaluating your ASN as the origin.
3. *"RPKI eliminates the need for prefix filtering."* — **False**. Strict customer prefix-lists, IRR filters, and maximum-prefix limits remain essential complementary controls.

Complementary Mechanisms: IRR Filtering, Prefix Filters and AS_PATH Filters

Complementary Mechanisms: IRR Filtering, Prefix Filters and AS_PATH Filters

Because RPKI ROV only authenticates the origin of a route, robust edge security requires **defense in depth**. Network operators must combine ROV with deterministic inbound and outbound prefix filtering, regular-expression `AS_PATH` filtering, and hardware resource limits.

Prefix Filters: What Prefixes Should Be Announced

Every eBGP session must enforce strict boundaries on what prefixes may be received and transmitted:

1. Bogon and Special-Purpose Filtering

Routers must unconditionally discard advertisements for IP space that should never appear on the public Internet (RFC 6890): - IPv4 Bogons: `0.0.0.0/8`, `10.0.0.0/8`, `100.64.0.0/10`, `127.0.0.0/8`, `169.254.0.0/16`, `172.16.0.0/12`, `192.0.2.0/24`, `192.168.0.0/16`, `198.18.0.0/15`, `198.51.100.0/24`, `203.0.113.0/24`, `224.0.0.0/4` (Multicast), `240.0.0.0/4`. - IPv6 Bogons: `::/8`, `0100::/64`, `2001:db8::/32`, `fe80::/10`, `fc00::/7` (ULA), `ff00::/8` (Multicast).

2. Prefix Length Boundaries

- In IPv4: drop any prefix smaller than `/8` or longer than `/24`.
- In IPv6: drop any prefix smaller than `/19` or longer than `/48`.

AS_PATH Filters: Detecting and Rejecting Suspicious Paths

Operators use regular expressions (regex) to inspect `AS_PATH` segments:

```
! Reject private ASNs, reserved ASNs, or path anomalies (Cisco IOS XR / Junos)
as-path-set BOGON-ASNS
  64512-65534,          # 16-bit Private ASNs
  65535,                # Reserved
  4200000000-4294967294 # 32-bit Private ASNs
end-set

! Reject routes containing bogon ASNs in transit
```

```
if as-path enters-with BOGON-ASNS then
  drop
endif

! Customer session enforcement: enforce that the first hop in AS_PATH matches customer's con
if not as-path starts-with ^65001 then
  drop
endif
```

By enforcing that the first ASN in the `AS_PATH` matches the neighbor's ASN, an ISP prevents a customer from spoofing direct peering sessions with other networks.

IRR-Based Filtering: Still Relevant After RPKI

While RPKI validates origins, **IRR filters validate topology and customer cones**: - An ISP queries the customer's `as-set` (e.g., `AS-HURRICANE`) using `bgpq4`. - The tool recursively expands all downstream customer ASNs and their registered `route` objects into an explicit prefix-list. - Even if an attacker uses a valid origin ASN to execute a route leak, the route is rejected at the ISP's border because it is not registered inside the customer's legitimate IRR cone.

Maximum-Prefix Limits: Protection Against Table Poisoning

Every BGP session must be configured with a hardware safety valve: the **Maximum-Prefix Limit**:

```
! FRR / Cisco IOS Syntax
router bgp 65001
 neighbor 192.0.2.2 maximum-prefix 100 80 restart 15
```

- If a customer who normally announces 5 prefixes accidentally leaks 100,000 routes, the moment the count crosses 80 (80%), the router issues a high-severity syslog alarm.
- When the count exceeds 100, the router immediately sends a BGP `NOTIFICATION` (Cease / Maximum Number of Prefixes Reached) and tears down the TCP session.
- This protects the router's control-plane CPU and TCAM from buffer exhaustion.

BGP Communities for Policy Signaling and Leak Prevention

Operators classify every imported route using internal BGP communities:

Community	Operational Meaning	Policy Action
`65000:100`	Learned from Customer	Propagate to Transit, Peers, and Customers

Community	Operational Meaning	Policy Action
`65000:200`	Learned from Settlement-Free Peer	Propagate to Customers only; NEVER to Transit
`65000:300`	Learned from Paid Transit Provider	Propagate to Customers only; NEVER to Peers or Transit

Before exporting routes to any external transit provider or lateral peer, an export route-map checks the community. Any route carrying `65000:200` or `65000:300` is unconditionally suppressed, programmatically guaranteeing that the network cannot become an accidental transit pipe.

Route Leak Detection and Prevention Mechanisms

Route Leak Detection and Prevention Mechanisms

Route leaks remain one of the most volatile disruptions on the global Internet. Because traditional RPKI ROV does not detect route leaks, the industry has developed architectural standards, automated telemetry systems, and industry self-regulation frameworks to systematically detect and prevent them.

Community-Based Leak Prevention and RFC 9234

Operators have long relied on proprietary BGP communities to tag routes. However, proprietary communities do not cross organizational boundaries: upstream ISP A strips or ignores customer B's internal communities.

To standardize leak prevention across administrative domains, the IETF published **RFC 9234: Route Leak Prevention and Detection Using BGP Roles**:

```
+-----------------------------------------------------------+
|               BGP Role Negotiation (RFC 9234)             |
|                                                           |
|  Provider AS                                  Customer AS |
|     Role: Provider <=========================> Role: Customer
|                                                           |
|  - When Provider advertises route to Customer:            |
|    Attaches "Only to Customer" (OTC) Attribute: OTC=Provider_ASN
|                                                           |
|  - When Customer advertises route to another Provider:    |
|    Second Provider inspects OTC attribute:                |
|    "Wait! This route has an OTC set, but I received it    |
|     from a Customer/Peer! This is an illegal leak!"       |
|    --> ROUTE REJECTED AT BORDER!                          |
+-----------------------------------------------------------+
```

1. **Role Negotiation**: When establishing a BGP session, peers negotiate their commercial relationship in the `OPEN` message: `Provider`, `Customer`, `Peer`, or `RS-Client`.
2. **OTC Attribute**: When a route moves down a hierarchy (Provider $\rightarrow$ Customer) or laterally across peers (Peer $\leftrightarrow$ Peer), the router attaches the **Only to Customer (OTC)** path attribute (Type 35), recording its own ASN.

3. **Automatic Leak Rejection**: If an upstream router receives a route from a customer or lateral peer that already contains an OTC attribute, it detects an illegal "valley" path and automatically rejects the route, neutralizing the leak in software without requiring complex manual prefix-lists.

Automated Leak Detection via Route Collectors and Monitors

When route leaks occur, real-time monitoring algorithms detect them through anomalous propagation signatures: - **Valley-Path Algorithms**: Automated telemetry engines inspect the `AS_PATH` streams received by RIPE RIS and RouteViews. By maintaining a global inferred topology map (customer-provider pairs), algorithms flag any path containing a *Customer* $\rightarrow$ *Provider* $\rightarrow$ *Customer* sequence as a probable route leak. - **Traffic Shift Probes**: Automated synthetic measurement networks (e.g., RIPE Atlas, ThousandEyes) detect sudden 200ms latency spikes and buffer-bloat jitter on paths that previously converged in 15ms.

BGPsec Route Leak Prevention Potential

While BGPsec (RFC 8205) was designed to protect path integrity by having each AS cryptographically sign its announcement to the next hop, **base BGPsec does not inherently prevent route leaks**: - In a route leak, the customer legally received the route from Provider A with a valid cryptographic signature. - The customer appends its own valid signature and passes it to Provider B. - The signature chain is mathematically flawless! - To prevent leaks, cryptographic protocols must incorporate role metadata (such as ASPA or signed OTC attributes) into the signature envelope.

MANRS Principles and Their Implementation Status

The **Mutually Agreed Norms for Routing Security (MANRS)** is a global, industry-led initiative operated under the Internet Society (ISOC) that establishes a baseline of routing hygiene:

MANRS Action	Technical Requirement	Operational Focus
Action 1: Filtering	Prevent propagation of incorrect routing information	Strict prefix-lists on customer sessions; RPKI ROV enforcement
Action 2: Anti-Spoofing	Prevent traffic with spoofed source IP addresses	Unicast Reverse Path Forwarding (uRPF, RFC 3704) on edge access links
Action 3: Coordination	Facilitate global operational communication	Accurate, maintained contact info in PeeringDB and RIR WHOIS
Action 4: Global Validation	Publicly document routing policy	Publish valid ROAs in RPKI and maintained `route`/`as-set` objects in IRR

MANRS participation has become a mandatory procurement requirement for Tier-1 telcos, major cloud operators, and financial enterprises.

Coordination and Incident Response When Leaks Occur

When a severe leak bypasses filters and floods a network: 1. **Out-of-Band NOC Communication**: Operators contact the leaking network's 24/7 Network Operations Center (NOC) via phone numbers published in **PeeringDB**. 2. **Session Teardown**: If the leaking customer does not answer, the upstream provider immediately executes `shutdown` on the BGP neighbor session. 3. **Emergency Disaggregation**: The victim network temporarily announces more-specific `/24` subprefixes to reclaim its traffic until the leak converges out of the global DFZ.

BGPsec: Architecture, Security Guarantees and Deployment Reality

BGPsec: Architecture, Security Guarantees and Deployment Reality

For over a decade, the Holy Grail of interdomain routing security was **BGPsec**, standardized by the IETF in RFC 8205 (Protocol Specification) and RFC 8206–8211. BGPsec was conceived to solve the fundamental flaw of path-vector routing: the inability to cryptographically verify the integrity of the `AS_PATH`.

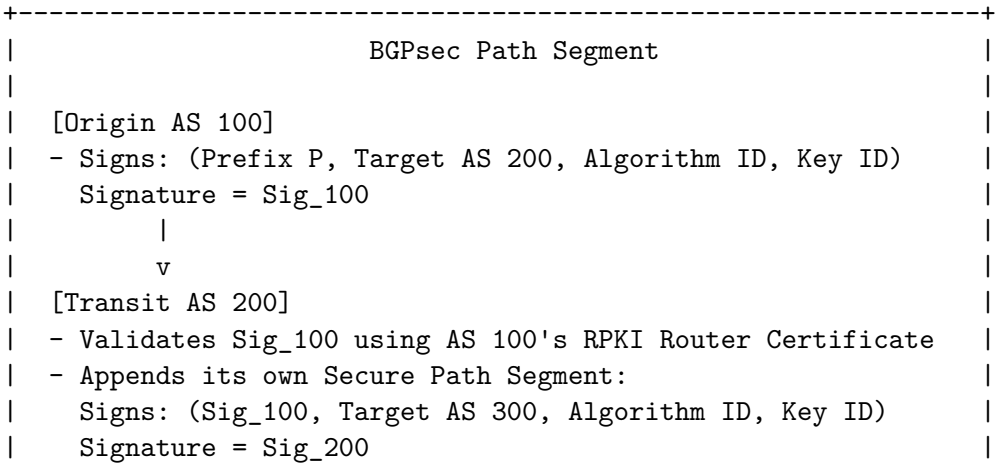
BGPsec's Threat Model: Protecting AS_PATH Integrity

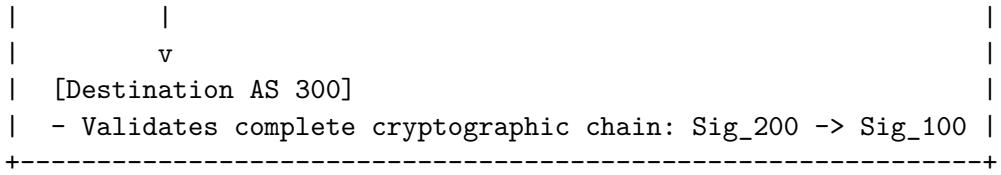
While RPKI ROV authenticates the *origin* of a prefix, it leaves the intermediate hops completely unverified. An adversary can execute a **forged-origin path hijack**: - The attacker operates AS_{666}. - The attacker announces a victim's prefix (AS_{100}) with `AS_PATH: [AS_{666}, AS_{100}]`. - A remote router running ROV inspects the rightmost ASN: it sees AS_{100}. Because AS_{100} is the legitimate owner with a valid ROA, the router marks the announcement as **Valid**, and the attacker succeeds in diverting traffic.

BGPsec was designed to make path forgery mathematically impossible: **every single Autonomous System along the path must cryptographically sign its routing update to the next hop.**

The BGPsec Path Attribute and Signature Chain

BGPsec replaces the traditional `AS_PATH` with a new optional non-transitive path attribute: `BGPsec_Path`:





Each signature binds: 1. The preceding signature block. 2. The local ASN. 3. The ASN of the intended downstream peer (Target AS). 4. A cryptographic timestamp.

An adversary cannot delete an AS hop, rearrange hops, or insert an arbitrary AS into the middle of the path without breaking the cryptographic digital signature chain.

Key Management and Certificate Infrastructure

To support BGPsec: - Every router in an AS must possess a private key paired with an **RPKI Router Certificate (RFC 8209)**. - Router certificates are issued under the AS holder's RPKI CA and published in the global RPKI repository tree. - The cryptographic algorithm mandated is **ECDSA with the P-256 curve and SHA-256** (RFC 8208).

Operational Barriers: Complexity, Interoperability and Performance

Despite being a complete IETF standard since 2017, **BGPsec has seen near-zero production deployment** across the global Internet DFZ. The reasons are structural:

1. The Asymmetric Cryptographic Compute Penalty

Verifying an ECDSA signature requires significant CPU cycles. A backbone router receiving a full routing table during a global convergence storm would need to verify **hundreds of thousands of cryptographic signatures per second**. Router control processors would immediately suffer complete CPU exhaustion, causing sessions to flap and collapse.

2. Message Size and MTU Fragmentation

Standard BGP messages are capped at 4096 bytes. A 5-hop BGPsec signature chain consumes several kilobytes of overhead, requiring universal support for Extended BGP Messages (RFC 8654, 65,535 bytes) and risking IP-layer packet fragmentation across transit links.

3. The All-or-Nothing Deployment Trap

BGPsec requires an unbroken chain of cryptographic signatures. If even a single intermediate transit provider along a path does not support BGPsec, the signature chain is broken, and the route drops back to legacy unauthenticated BGP. There is zero economic incentive for an early adopter to invest in expensive hardware upgrades if their upstreams do not.

Current Deployment Status and Future Prospects

As of 2026, BGPsec remains an academic reference architecture rather than an operational defense. The global networking community has largely pivoted toward lighter, out-of-band path verification mechanisms—most notably **Autonomous System Provider Authorizations (ASPA)**—which provide robust path protection without requiring in-band cryptographic signing on wire-speed routers.

ASPA: Autonomous System Provider Authorizations

ASPA: Autonomous System Provider Authorizations

Recognizing that BGPsec's in-band cryptographic overhead rendered it operationally unviable, the IETF SIDROPS working group designed **Autonomous System Provider Authorization (ASPA)** (draft-ietf-sidrops-aspa-profile and draft-ietf-sidrops-aspa-verification). ASPA provides practical, scalable protection against both **route leaks** and **forged-origin path hijacks** using the existing out-of-band RPKI infrastructure.

What ASPA Is and the Problem It Solves

ASPA shifts the security verification model from *signing every dynamic route announcement* to **publishing verified out-of-band statements of upstream provider relationships**:

- In BGP, a route leak occurs when a customer forwards a route learned from one provider to another provider.
- A forged-origin hijack occurs when an attacker fakes an adjacency to a legitimate origin.
- Both attacks violate the fundamental commercial topology of the Internet: a customer only sends routes to its upstream providers; a provider never routes through a customer to reach the global transit web.

ASPA allows an Autonomous System to formally publish a signed object stating: **"These are the only ASNs that are authorized to act as my upstream transit providers."**

ASPA Structure: Customer, Provider and Peer Relationships

An ASPA is an RPKI signed object cryptographically bound to a Customer ASN:

```
ASPA Object Payload:
  CustomerASID: AS65001
  ProviderASSet: [
    AS174 (Cogent),
    AS2914 (NTT),
    AS3356 (Lumen)
  ]
```

This simple list asserts that `AS65001` only purchases transit from AS174, AS2914, and AS3356. If a route appears on the Internet with `AS_PATH: [AS7018, AS65001, ...]`, but AS7018 is not listed in `AS65001`'s ASPA object, downstream routers know that AS7018 has no legitimate commercial authority to act as `AS65001`'s provider!

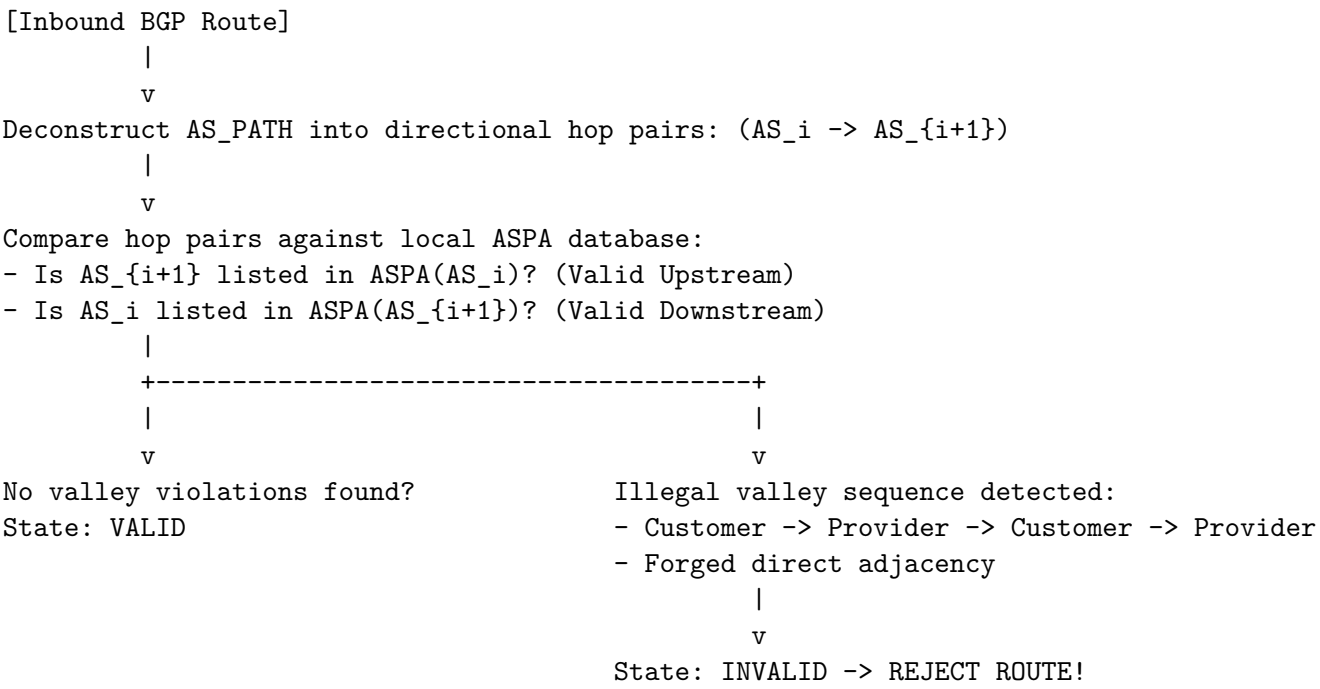
Integration with the RPKI Trust Infrastructure

Like ROAs, ASPA objects: - Are issued under the resource holder's RPKI CA. - Are verified against the RIR Trust Anchor root certificates. - Are downloaded and validated by relying party validators (such as Routinator or rpki-client). - Are delivered to routers over the **RPKI-to-Router (RTR) protocol version 2** (RFC 8210-bis).

Because ASPA objects represent static business contracts rather than dynamic route states, a single ASPA object covers all prefixes originated by that AS. The total global ASPA dataset requires only tens of thousands of records, fitting comfortably inside router RAM.

How ASPA Validation Affects Route Selection

When a router evaluates an incoming `AS_PATH`, it runs the **ASPA Verification Algorithm**:



ASPA assigns one of three verification states to the path: 1. **`Valid`**: The sequence of AS hops complies strictly with the Gao-Rexford model of provider-customer hierarchy. 2. **`Invalid`**: At least one hop pair is demonstrably contradicted by an authoritative ASPA record (e.g., an unlisted AS acting as a provider, or an illegal valley route leak). 3. **`Unknown`**: Insufficient ASPA records exist to determine path validity.

Routers configure inbound policies to unconditionally **drop ASPA Invalid routes**, neutralizing route leaks and forged-origin hijacks before they can enter the `Loc-RIB`.

Current Deployment Status and Standardization Progress

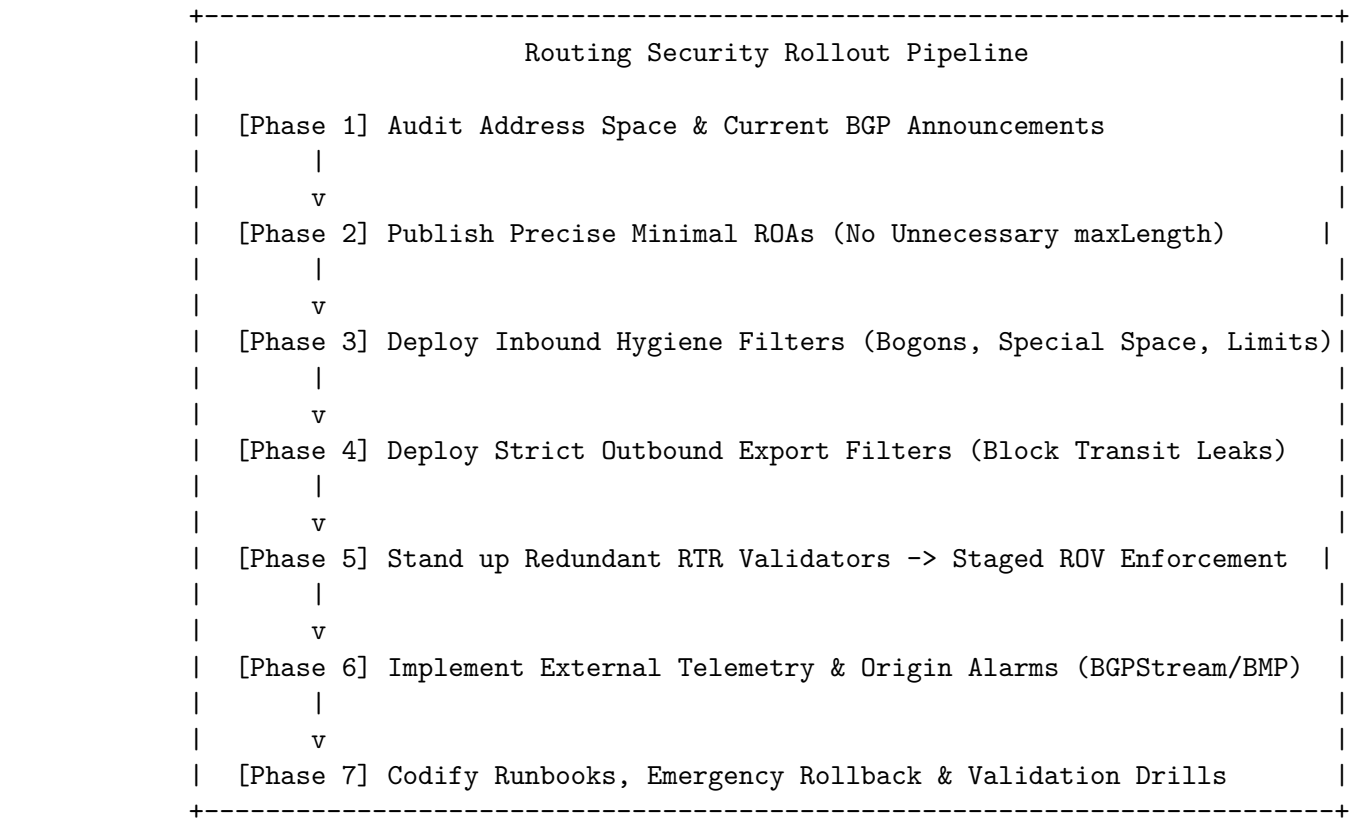
As of 2026, ASPA represents the most actively developed and promising routing security protocol in the IETF: - Supported in open-source validators (**Routinator**, **rpki-client**, **FORT**). - Production implementations running on open routing platforms (**BIRD 2.15+**, **FRR**). - Active commercial trials and interoperability testing across Tier-1 backbones (NTT, Cloudflare, Lumen). - Widespread deployment of ASPA alongside RPKI ROV promises to close the path-manipulation loophole that has plagued BGP for thirty years.

Designing and Implementing Routing Security in Your Network

Designing and Implementing Routing Security in Your Network

Deploying routing security across an active production network requires a structured, multi-phase engineering process. Implementing controls haphazardly—such as enforcing hard ROV drops before auditing address allocations—is a guaranteed recipe for self-inflicted outages.

This chapter outlines an end-to-end, seven-step blueprint for hardening an Autonomous System.



Step 1: Inventory and Audit Your Address Space and Announcements

Before touching a single firewall or router configuration, create an authoritative inventory: - Query public route collectors (e.g., `whois -h whois.radb.net -- "-i origin AS<YOUR_ASN>"`) to list all prefixes currently seen on the global Internet. - Check internal IP Address Management (IPAM) systems to identify all assigned, unassigned, and delegated subnets. - Reconcile differences: identify

any ghost prefixes being announced without business justification, or internal subnets missing from external tables.

Step 2: Verify Ownership and Create or Update ROAs

Log into your organization's RIR portal (ARIN, RIPE, APNIC, LACNIC, AFRINIC): - Verify that your organization is listed as the authoritative Org-ID on all resource certificates. - Generate cryptographically valid ROAs matching your active BGP announcements. - **Enforce the Minimal ROA rule (RFC 9319)**: Set `maxLength` equal to the announced prefix length. If you announce a `/22` and two `/24` subprefixes, create three explicit ROAs rather than a single `/22 max-length 24`.

Step 3: Design and Deploy Import Filters

Apply deterministic inbound hygiene filters on all transit and peering sessions: - Unconditionally deny IPv4/IPv6 bogon ranges (RFC 6890). - Unconditionally deny default routes (`0.0.0.0/0` and `::/0`) from lateral peers and IXPs. - Enforce prefix length sanity: drop IPv4 $<$ /8 or $>$ /24; drop IPv6 $<$ /19 or $>$ /48. - Configure hardware `maximum-prefix` limits on every neighbor session with an 80% warning threshold and auto-restart cooldown.

Step 4: Design and Deploy Export Filters

Prevent accidental route leaks at your outbound border: - Define an explicit prefix-list containing *only* your locally owned and operated address space. - Apply this prefix-list to all outbound route-maps facing transit providers and settlement-free peers. - Tag all inbound routes from providers and peers with internal NO-TRANSIT communities, and ensure export policies discard any route bearing those tags.

Step 5: Enable ROV with Staged Enforcement

Never transition directly from zero validation to hard drops in production. Execute a staged migration:

Stage A: Validation Without Enforcement (Telemetry Mode)

Deploy validators (Routinator / rpki-client) and establish RTR sessions with border routers. Configure the router to evaluate ROV states, but do not alter forwarding:

```
route-map PEER-IN permit 10
 match rpki invalid
 set community 65000:666 additive   <-- Tag only; do not drop
```

Monitor logs and NetFlow data for 30 days to identify legitimate partners whose routes evaluate as `Invalid` due to clerical errors.

Stage B: Hard Drop Enforcement

Once telemetry confirms that no critical business services are flagged as `Invalid`, activate hard drops across all transit and peer borders:

```
route-map PEER-IN deny 10
 match rpki invalid
!
route-map PEER-IN permit 20
```

Step 6: Implement Monitoring, Logging and Alerting

Set up automated alerting infrastructure: - Stream BGP Monitoring Protocol (BMP, RFC 7854) data from border routers to an internal collector. - Subscribe to real-time public telemetry (RIS-Live or BGPStream). - Configure automated alerts that trigger immediately if an unauthorized ASN originates your prefixes or if a validator RTR session drops.

Step 7: Document Policies, Plan Rollback and Test Recovery

Every security configuration must have an emergency recovery procedure: - Document the exact CLI command required to bypass ROV globally if an RIR infrastructure outage corrupts the VRP cache. - Schedule bi-annual fire drills in staging or containerized lab environments (Containerlab) where engineers practice mitigating simulated hijacks, invalid states, and validator synchronization failures.

Configuration Examples and Vendor-Specific Considerations

Configuration Examples and Vendor-Specific Considerations

Routing security concepts are universal, but syntax, execution semantics, and memory structures vary significantly across routing operating systems. This chapter provides production-ready configuration stanzas across major commercial and open-source platforms.

Cisco IOS XR: ROV, Filtering and RTR Configuration

Cisco IOS XR manages RPKI caches using the RPKI-to-Router protocol under the global routing configuration, and applies policy using Routing Policy Language (RPL):

```
! Configure RTR connection to two redundant RPKI validators
router bgp 65001
 rpki server 192.0.2.10
  transport tcp port 3323
  refresh-time 1800
  response-time 60
 !
 rpki server 192.0.2.20
  transport tcp port 3323
  refresh-time 1800
  response-time 60
 !
! Define RPL policy to drop Invalids and prefer Valids
route-policy EBGP-INBOUND
  # Bogon filtering
  if destination in BOGON-PREFIXES then
    drop
  endif
  # Enforce RPKI Route Origin Validation
  if rpki origin state is invalid then
    drop
  elseif rpki origin state is valid then
    set local-preference 200
    pass
  else
    # NotFound / Unknown
    set local-preference 100
```

```
    pass
  endif
end-policy
```

Juniper Junos: ROV, Filtering and RPKI Integration

In Junos, RPKI validation sessions reside under `routing-options validation`, and policy is applied under `policy-statement`:

```
routing-options {
    validation {
        group RPKI-VALIDATORS {
            session 192.0.2.10 {
                port 3323;
                local-address 192.0.2.1;
            }
            session 192.0.2.20 {
                port 3323;
                local-address 192.0.2.1;
            }
        }
    }
}

policy-options {
    policy-statement RPKI-ROV-POLICY {
        term REJECT-INVALID {
            from {
                protocol bgp;
                validation-database invalid;
            }
            then reject;
        }
        term PREFER-VALID {
            from {
                protocol bgp;
                validation-database valid;
            }
            then {
                local-preference 200;
                accept;
            }
        }
        term ACCEPT-UNKNOWN {
            then {
```

```
                local-preference 100;
                accept;
            }
        }
    }
}
```

Nokia SR OS: Routing Security Configuration

In Nokia SR OS (Service Router Operating System), RPKI sessions are established under the global router context, and validation is integrated into routing policy:

```
/configure router
    origin-validation
        rpki-session 192.0.2.10
            port 3323
            admin-state enable
        exit
        rpki-session 192.0.2.20
            port 3323
            admin-state enable
        exit
    exit
    policy-options
        begin
        policy-statement "ROV-POLICY"
            entry 10
                from
                    protocol bgp
                    origin-validation-state invalid
                exit
                action drop
            exit
            entry 20
                from
                    protocol bgp
                    origin-validation-state valid
                exit
                action accept
                    local-preference 200
                exit
            exit
            default-action accept
                local-preference 100
            exit
```

```
        exit
        commit
```

Open Source and Research Platforms: FRR and BIRD

FRRouting (FRR)

FRR integrates RPKI using the `rpki` daemon (linking `librtr`):

```
! /etc/frr/frr.conf
rpki
 rpki cache 192.0.2.10 3323 preference 1
 rpki cache 192.0.2.20 3323 preference 2
 exit
!
route-map TRANSIT-IN deny 10
 match rpki invalid
!
route-map TRANSIT-IN permit 20
 match rpki valid
 set local-preference 200
!
route-map TRANSIT-IN permit 30
 match rpki notfound
 set local-preference 100
!
router bgp 65001
 neighbor 198.51.100.1 route-map TRANSIT-IN in
```

BIRD 2 (Internet Routing Daemon)

BIRD 2 provides native RPKI table evaluation and filter syntax:

```
# bird.conf
protocol rpki RPKI1 {
    rtr 192.0.2.10 port 3323;
    refresh 600;
    retry 300;
    expire 7200;
    roav4 { table r4; };
    roav6 { table r6; };
}

filter bgp_in {
```

```
    # Check ROV state against RPKI table
    if roa_check(r4, net, bgp_path.last) = ROA_INVALID then {
        print "Rejecting RPKI Invalid: ", net, " from AS", bgp_path.last;
        reject;
    }
    if roa_check(r4, net, bgp_path.last) = ROA_VALID then {
        bgp_local_pref = 200;
        accept;
    }
    bgp_local_pref = 100;
    accept;
}
```

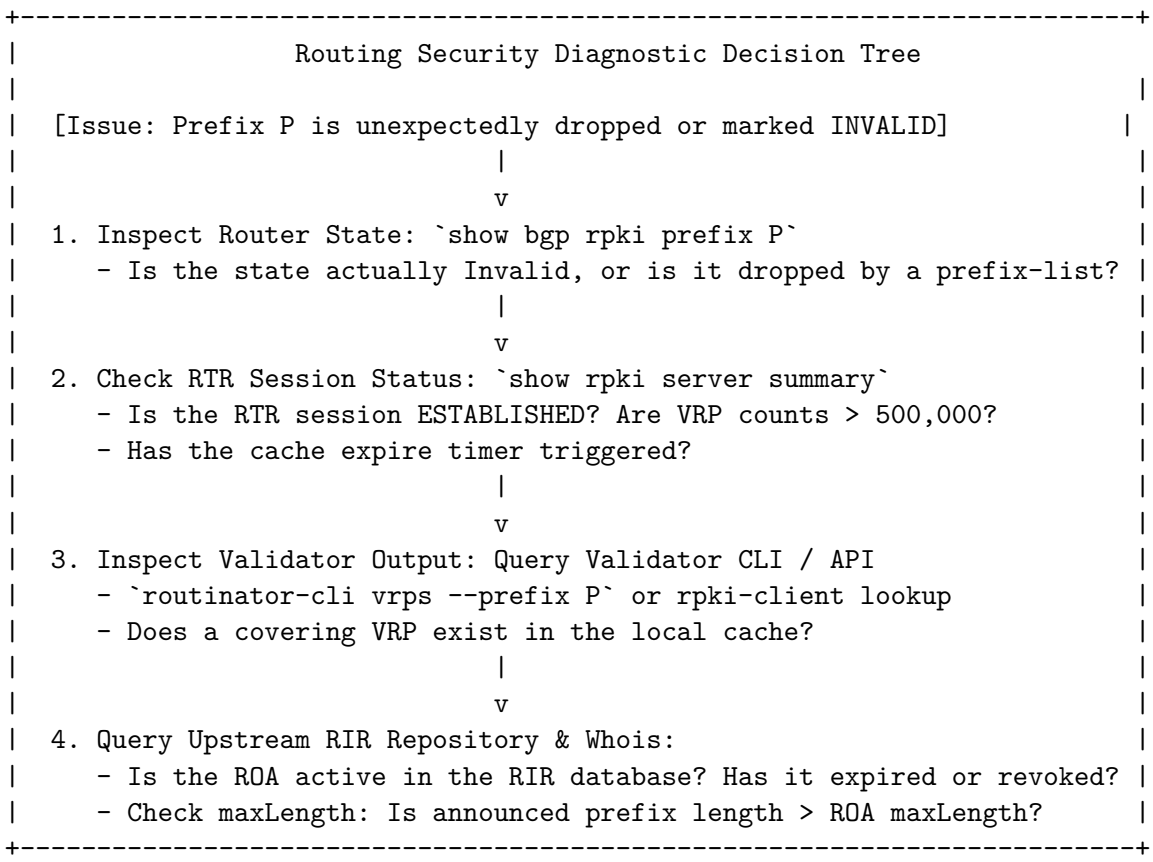
Common Pitfalls Across Platforms

1. **Failure to Commit**: On modular systems (IOS XR and Junos), policy edits require explicit `commit`. Uncommitted RPKI changes remain inactive while misleading administrators.
2. **Implicit Fall-Through Behavior**: In some platforms (Cisco IOS XE), failing to specify an explicit `permit` term after an RPKI `deny` causes an implicit deny-all, blackholing all valid traffic!
3. **Session Flap CPU Spikes**: Configuring RTR refresh timers too aggressively (e.g., 10 seconds) forces continuous re-evaluation of full tables, spiking router CPU. Maintain default timers (1800 to 3600 seconds).

Troubleshooting Routing Security Controls

Troubleshooting Routing Security Controls

When routing security mechanisms malfunction, legitimate traffic is dropped, peering sessions flap, and reachability breaks. Diagnosing these failures requires a disciplined, hypothesis-driven methodology that isolates whether the fault resides in the router's local policy, the RTR transport layer, the validator cache, or the upstream RIR cryptographic hierarchy.



Unexpected Invalid Routes: Systematic Diagnosis

When a route unexpectedly evaluates as `Invalid`: 1. **Identify the exact announced parameters**: Run `show bgp <prefix>` to extract: - Announced Prefix (e.g., `198.51.100.0/24`) - Origin ASN (the rightmost ASN in the `AS_PATH`) 2. **Compare against local VRP entries**: On Cisco: `show bgp rpki prefix 198.51.100.0/24`. On Junos: `show validation database 198.51.100.0/24`. 3. **Determine the exact invalid condition**: - **AS Mismatch**: A covering VRP exists for the prefix, but specifies a different authorized ASN (e.g., authorized AS65001, but

received AS65002). - **Length Mismatch**: The ASN matches, but the announced prefix length exceeds `maxLength` (e.g., announced `/24`, but ROA `maxLength` is `/22`).

RTR Session Failures and Stale Validation Data

If border routers report that all routes have reverted to `NotFound` or that RPKI tables are empty: - **Check TCP Layer Connectivity**: Ensure firewalls permit TCP port 323 / 3323 between the router loopback and the validator IP. - **Inspect Serial Number Desynchronization**: If a validator server reboots and resets its internal database without properly incrementing its serial number, the router may reject updates. Force an explicit reset query: - Cisco: `clear bgp rpki server <IP> reset` - Junos: `clear validation session <IP>` - **Examine Expire Timers**: If the validator fails to respond to keepalives, verify whether the router's `expire-time` has lapsed, purging the cache from memory.

ROA Lookup: Verifying What Validators See

To determine whether an issue is local to the router or originates in the global RPKI tree, query the validator's local API directly from a management host:

```
# Query Routinator's REST API for a specific prefix and origin
curl -s http://192.0.2.10:8323/api/v1/validity/AS65001/198.51.100.0/24 | jq .
```

Expected output:

```
{
  "validated_route": {
    "route": {
      "prefix": "198.51.100.0/24",
      "origin_asn": "AS65001"
    },
    "validity": {
      "state": "valid",
      "reason": null,
      "description": "At least one VRP Matches the Route Prefix"
    }
  }
}
```

If the validator reports `notfound` or `invalid`, query public tools (e.g., RIPE NCC RPKI Validator web dashboard or `rpki-validator.ripe.net`) to see if the global community observes the same state.

Legitimate Routes Rejected by Filters

If a route is dropped despite having a `Valid` RPKI state, the drop is typically caused by a static filter placed before the RPKI evaluation term in the route policy: - **Bogon filter misconfiguration**: The prefix falls within an overly broad bogon regex. - **Prefix-list length restriction**: The inbound prefix-list permits only `/8` to `/22`, rejecting a legitimate `/24`. - **Maximum-prefix trip**: The neighbor's total prefix count exceeded the configured ceiling, causing the router to drop all routes from that session.

Cross-Validator Consistency Checks

Running a single validator instance creates a single point of failure and vulnerability to implementation bugs: - Operate at least two distinct validator engines: e.g., Validator A running **Routinator** (Rust) and Validator B running **rpki-client** (C/OpenBSD). - Run automated cron scripts comparing the total VRP count output by each validator. If the VRP counts diverge by more than 2%, trigger an administrative alert to investigate repository sync failures or parsing discrepancies.

Post-Incident Forensics: Reconstructing What Happened

When a routing security incident concludes: 1. **Pull BGP Table Dumps and BMP Archives**: Capture historical updates from the incident window to reconstruct the exact timestamps when the illegitimate announcement appeared, propagated, and was withdrawn. 2. **Review RIR Audit Logs**: Determine whether the incident was caused by an account takeover, a revoked resource certificate, or an operator typo in the RIR portal. 3. **Conduct Root-Cause Analysis (RCA)**: Document why local and upstream filters did or did not mitigate the issue, and update automated test suites to prevent recurrence.

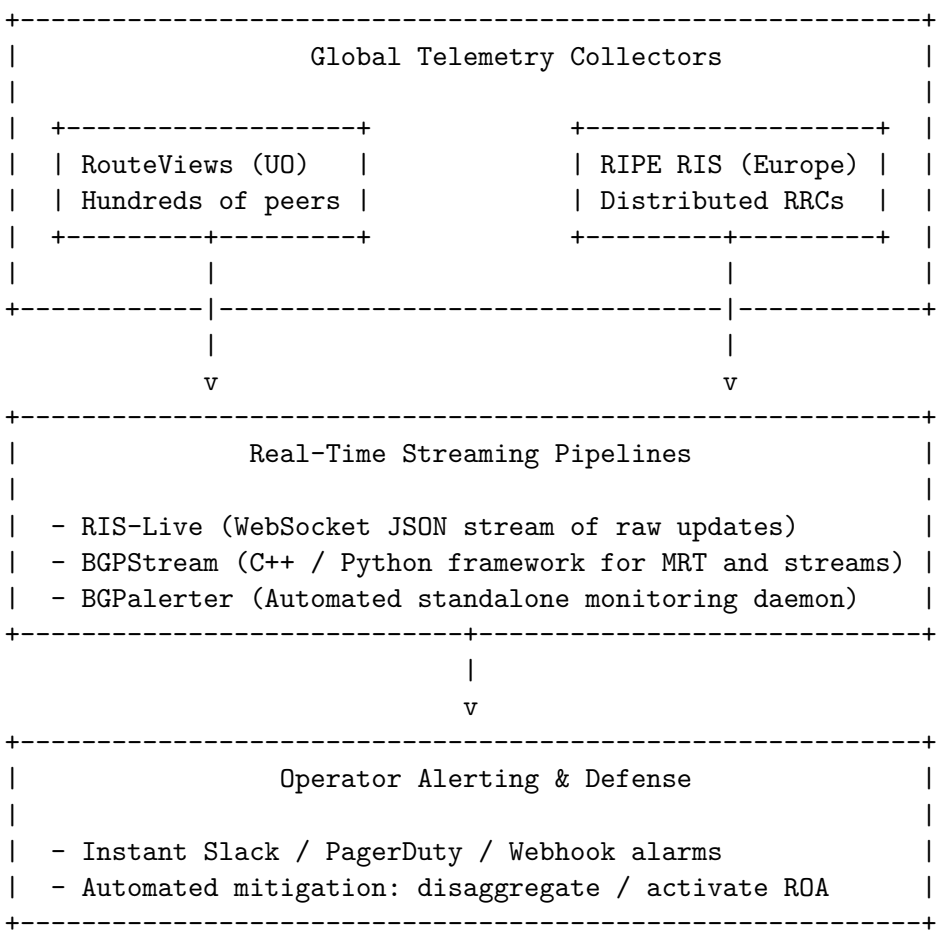
Monitoring, Measurement and Testing

Monitoring, Measurement and Testing

You cannot defend what you cannot see, and you cannot observe global interdomain routing anomalies from the perspective of a single border router. Comprehensive routing defense requires continuous global measurement, real-time streaming telemetry, and isolated testing environments where security controls can be validated safely before live deployment.

Real-Time Route Monitoring: BGPStream, RIS-Live and RouteViews

Modern security operations centers (SOCs) and network operations centers (NOCs) rely on distributed telemetry frameworks to monitor their prefixes from hundreds of external vantage points:



1. RIPE RIS and RIS-Live

The RIPE Routing Information Service (RIS) operates remote route collectors (RRCs) at major Internet Exchange Points globally (London, Amsterdam, Tokyo, New York). **RIS-Live** exposes a real-time WebSocket firehose delivering BGP `UPDATE` messages within milliseconds of their occurrence on global backbones.

2. RouteViews

Operated by the University of Oregon, RouteViews provides multi-decade historical MRT archives and live multihop BGP peering with Tier-1 and Tier-2 service providers, serving as the gold standard for routing research and forensic investigation.

3. BGPStream and BGPalerter

- **BGPStream**: An open-source software framework for parsing, filtering, and analyzing historical MRT dumps and real-time live BGP data streams.
- **BGPalerter**: A self-hosted open-source monitoring application that tracks an organization's prefixes, ASNs, and ROAs, firing instant webhook alerts if a prefix is hijacked, a route leaks, or a ROA nears expiration.

Looking Glasses and Prefix Monitoring Services

When an alert fires, engineers need interactive visibility into remote routing tables: - **Looking Glasses (LG)**: Publicly accessible web portals hosted by Tier-1 operators (e.g., Level3/Lumen, NTT, Hurricane Electric) allowing operators to execute `ping`, `traceroute`, and `show ip bgp <prefix>` from specific points of presence across the planet. - **Commercial SaaS Monitors**: Services such as Cloudflare Radar, Cisco ThousandEyes, and Kentik provide graphical dashboards mapping global propagation paths, prefix visibility percentages, and AS path changes.

Safe Testing: Lab Environments and Controlled Announcements

Deploying new route policies or RPKI validators directly into production is unacceptably risky. Safe verification requires staging in containerized and software-defined testbeds:

1. Containerlab Network Topologies

Using **Containerlab**, engineers spin up multi-vendor containerized topologies (FRR, Nokia SR Linux, Arista cEOS, BIRD) running on a single developer workstation or CI/CD runner. Topologies simulate upstream transit providers, lateral peers, relying party validators (Routinator), and malicious rogue ASes.

```
# Example Containerlab test snippet simulating an upstream validator
name: bgp-security-lab
topology:
  nodes:
    validator:
      kind: linux
      image: routinator/routinator:latest
    border-router:
      kind: linux
      image: frrouting/frr:latest
    attacker-as:
      kind: linux
      image: frrouting/frr:latest
```

2. The PEERING Testbed

For academic and controlled research, the **PEERING testbed** (operated by USC and partner universities) allows vetted researchers to announce experimental prefixes into the real global Internet from multiple worldwide locations to measure live propagation dynamics without disrupting commercial traffic.

Public Incident Detection Systems and Alerts

Automated detection engines scan global feeds to identify attacks: - **CAIDA BGP Mon**: Automatically reconstructs prefix hijack events and tracks global prefix churn. - **MANRS Observatory**: Aggregates routing incidents per Autonomous System, scoring network operators on their compliance with anti-spoofing, filtering, and RPKI deployment standards.

Metrics That Matter: Convergence, Stability and Coverage

To evaluate the operational health of an organization's routing posture, operators track five primary Key Performance Indicators (KPIs): 1. **ROA Coverage**: Percentage of announced IPv4 and IPv6 prefixes covered by valid ROAs (Target: 100%). 2. **ROV Filtering Rate**: Percentage of peering sessions actively enforcing hard drops on Invalid routes (Target: 100% of transit and peer sessions). 3. **Prefix Stability / Flap Rate**: Number of route withdrawal events per hour across peering sessions. 4. **Convergence Latency**: Time required for a planned BGP withdrawal or announcement to achieve 95% global propagation across RouteViews collectors (Target: < 60 seconds). 5. **ASPA Compliance**: Progress in documenting and publishing upstream transit relationships in ASPA objects.

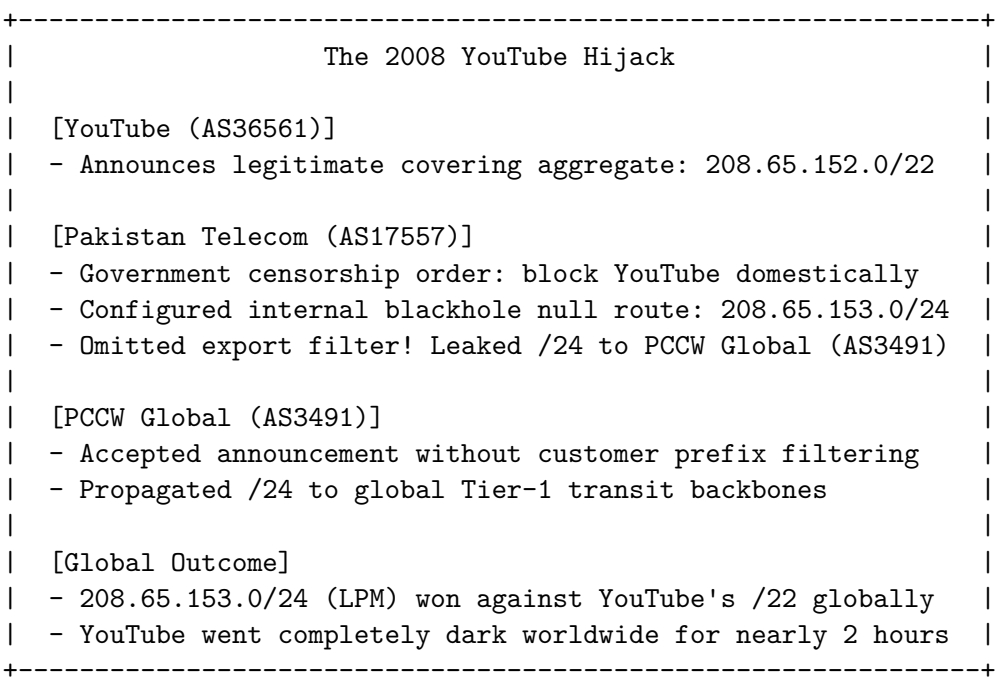
Case Studies: Documented Routing Incidents

Case Studies: Documented Routing Incidents

Analyzing historical routing incidents reveals how protocol vulnerabilities, operator errors, and intentional exploits manifest in real-world networks. These landmark case studies shaped the modern routing security landscape and demonstrate why specific defense layers were developed.

Pakistan Telecom YouTube Hijack (February 24, 2008)

The 2008 YouTube hijack remains the classic textbook demonstration of a **subprefix origin hijack**:



1. **Context**: Following a domestic censorship order by the Pakistan Telecommunication Authority to block YouTube video clips, engineers at Pakistan Telecom (AS17557) configured an internal null route on an edge router for YouTube's IP block: `208.65.153.0/24`.
2. **The Mechanism**: YouTube legitimately originated the covering `/22` aggregate: `208.65.152.0/22`. Pakistan Telecom originated the `/24` subprefix. Crucially, Pakistan Telecom accidentally leaked this `/24` advertisement to its international transit provider, PCCW Global (AS3491).
3. **The Failure**: PCCW Global accepted the announcement without enforcing an inbound customer prefix-list, and propagated the `/24` to its global transit peers.

4. **The Impact**: Because `/24` is a longer prefix than `/22`, Longest Prefix Match directed global Internet traffic for YouTube into Pakistan Telecom's network, instantly saturating its international links and knocking YouTube completely offline globally for nearly two hours.
5. **Mitigation**: YouTube engineers resolved the outage by disaggregating their space, announcing two matching `/24` prefixes to compete, and broadcasting emergency out-of-band requests to PCCW Global to filter Pakistan Telecom's session.

Safe Host / China Telecom Route Leak (June 6, 2019)

On June 6, 2019, a Swiss data center provider caused a massive European and international traffic disruption through a **Customer-Provider Route Leak**:

1. **The Mechanism**: Safe Host (AS21217), a Swiss hosting company, received full Internet routing tables from its upstream providers. Due to a configuration error during maintenance, Safe Host re-announced approximately 70,000 global prefixes to China Telecom (AS4134).
2. **The Failure**: China Telecom accepted the leaked routes and, because they were learned from a customer session, assigned them high preference and propagated them to other international transit networks.
3. **The Impact**: Over 70,000 prefixes—including critical IP blocks belonging to European mobile operators (Swisscom, Bouygues Telecom) and major European cloud services—were detoured through China Telecom's internal network in China. Massive packet loss, severe latency spikes, and complete connection drops lasted for over two hours.
4. **Root Cause**: Complete absence of basic export filtering at Safe Host, combined with the lack of RFC 9234 Only to Customer (OTC) enforcement at China Telecom.

Cloudflare 1.1.1.1 Incident (June 27, 2024)

Even modern, highly protected Anycast DNS resolvers remain vulnerable to misrouting anomalies when upstream transit providers misbehave:

1. **The Incident**: On June 27, 2024, a regional ISP in Asia accidentally originated `1.1.1.0/24`—the primary IP address of Cloudflare's public DNS service.
2. **The Defense in Action**: Cloudflare had published a valid, exact-match ROA: `(1.1.1.0/24, maxLength=24, Origin=AS13335)`.
3. **The Divergence**:
    - Tier-1 and Tier-2 networks that had deployed RPKI Route Origin Validation immediately flagged the regional ISP's announcement as **Invalid** and dropped it, keeping DNS reachability clean for their users.
    - However, networks and ISPs that had *not* deployed ROV accepted the illegitimate announcement, directing user DNS queries into a blackhole.
4. **Key Takeaway**: ROV successfully protected adopters, but highlighted that incomplete global deployment leaves non-adopting networks exposed to disruptions.

3ve BGP Hijacking Campaign (2018)

In 2018, the US Department of Justice unsealed indictments against the operators of **3ve**, a massive international cybercrime syndicate that stole tens of millions of dollars through fraudulent digital advertising:

1. **The Mechanism**: The 3ve syndicate used compromised routing infrastructure and rogue BGP speakers to hijack hundreds of dormant, unannounced IP blocks (bogon and legacy address space) from legitimate corporate holders who were not actively using them.
2. **The Exploitation**: The attackers registered fraudulent `route` objects in permissive third-party IRR databases (such as RADB) to convince transit providers to accept the routes. They then announced the hijacked blocks and used them to host thousands of fake websites and botnet servers, generating billions of fraudulent ad impressions.
3. **The Catalyst for RPKI**: The 3ve campaign served as the wake-up call for the global advertising and financial industries, proving that dormant IP blocks could be weaponized indefinitely unless cryptographically protected by RPKI ROAs.

Comparative Analysis: What Changed, What Remained Vulnerable

Incident	Year	Attack / Fault Type	Would RPKI ROV Prevent It?	Modern Prevention Mechanism
Pakistan Telecom	2008	Subprefix Origin Hijack (`/24` vs `/22`)	**YES** (Dropped as Invalid)	RPKI ROV + Minimal ROA (RFC 9319)
Safe Host	2019	Customer-Provider Route Leak	**NO** (Origin was valid)	RFC 9234 OTC + ASPA
Cloudflare 1.1.1.1	2024	Exact-Match Origin Hijack	**YES** (For networks with ROV)	Universal ROV enforcement
3ve Syndicate	2018	Dormant Prefix Hijacking via Fake IRR	**YES** (If ROA published)	RPKI ROAs + Authoritative IRR cleaning

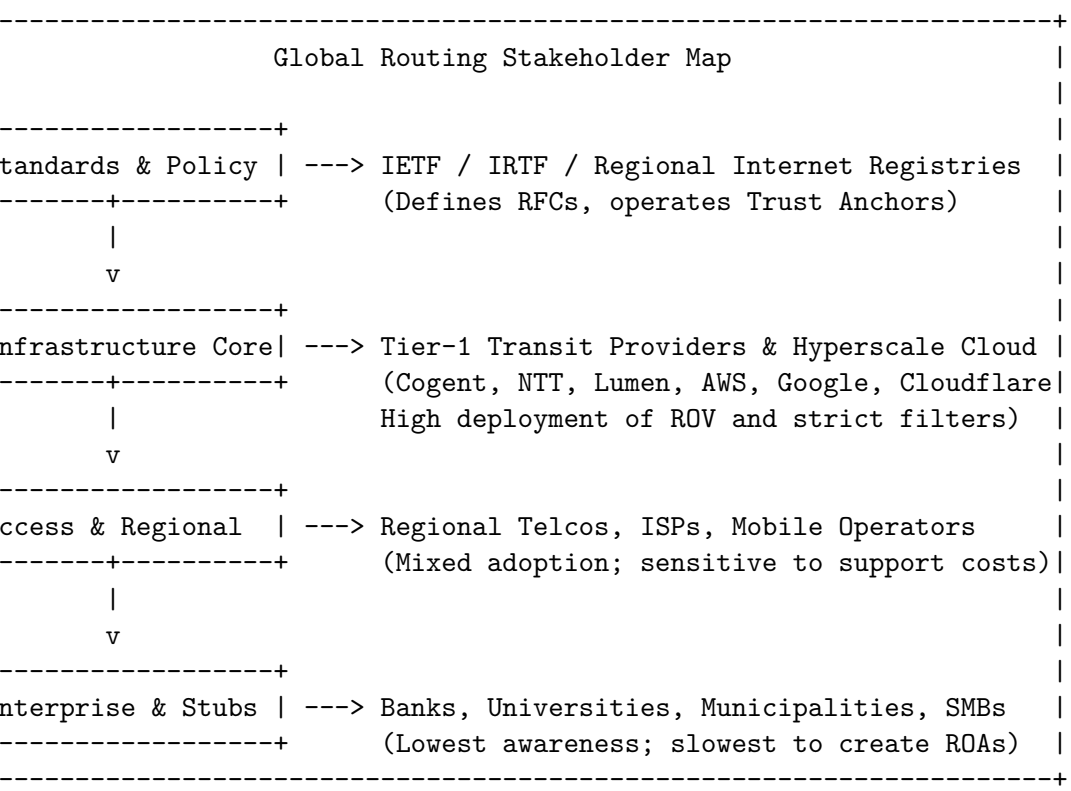
Economics, Governance and the Human Dimension

Economics, Governance and the Human Dimension

Securing global routing is fundamentally an economic, political, and human governance challenge. Technology alone does not secure the Internet; rather, protocols are adopted when economic incentives align, institutional frameworks mandate compliance, and operational liability outweighs the friction of change.

Who Is Responsible for What: A Stakeholder Map

The global routing ecosystem comprises multiple distinct stakeholder groups, each operating under conflicting incentives:



1. **Standards Bodies (IETF, IRTF)**: Engineer protocols (BGPsec, ASPA, RTR). They operate by consensus and have zero authority to force commercial networks to deploy standards.
2. **Regional Internet Registries (RIRs)**: Allocate IP addresses and AS numbers; operate RPKI Trust Anchors and repository infrastructure.

3. **Tier-1 Transit Providers**: Maintain the global backbone. They bear the heaviest computational load and traffic volume during incidents.
4. **Content Providers and Cloud Hyperscalers**: Heavily motivated to adopt security to protect customer trust and latency SLAs.
5. **Stub Networks and Enterprises**: Tens of thousands of small organizations that view routing as a utility, lacking dedicated BGP engineering expertise.

Economic Incentives and Disincentives for Adoption

The persistent delay in achieving universal routing security is rooted in a classic **positive externality problem**: - When Network A signs its ROAs, it protects itself from being hijacked, but it expends engineering labor to do so. - When Network B deploys ROV filtering (dropping Invalids), it expends CPU power, risks support tickets from broken third-party ROAs, and **protects everyone else on the Internet, while receiving no direct commercial revenue for doing so.** - Customers rarely choose an Internet Service Provider based on whether the provider validates RPKI routes; they choose based on price, bandwidth, and geographic availability.

The Role of RIRs, Regional Communities and IETF

The five RIRs (ARIN, RIPE NCC, APNIC, LACNIC, AFRINIC) have transitioned from passive allocation registries to critical operational certificate authorities: - **Hosted vs. Delegated RPKI**: RIRs provide hosted web portals allowing non-technical operators to create ROAs with a few clicks, driving the majority of global RPKI adoption. - **RIR Coordination**: RIRs coordinate via the Number Resource Organization (NRO) to standardize TAL formats, certificate expiration policies, and repository architectures. - **Regional Operator Groups (NANOG, RIPE, APRICOT)**: Serve as the primary forums where network engineers share post-mortem analyses, establish informal social pressure, and develop shared operational norms.

MANRS, IRTF and Industry Self-Regulation

In the absence of a global regulatory body with binding jurisdiction over Internet routing, the industry has turned to self-regulation through **Mutually Agreed Norms for Routing Security (MANRS)**: - **The MANRS Observatory**: Provides public transparency by grading operators on their routing hygiene, making poor security visible to enterprise procurement officers. - **Peer Pressure and Commercial Mandates**: Major financial institutions and cloud providers now require transit providers to be MANRS-compliant as a mandatory condition in Request for Proposal (RFP) vendor contracts.

Why Technically Sound Mechanisms Fail to Deploy

History demonstrates that technical elegance does not guarantee adoption: 1. **The Curse of BGPsec**: A mathematically brilliant protocol (RFC 8205) that failed because it demanded massive

hardware upgrades, high CPU latency, and universal deployment before delivering value. 2. **The Success of RPKI and ASPA**: Pragmatic, out-of-band protocols that deliver immediate security benefits to early adopters without requiring router ASIC replacements. 3. **The Lesson**: On the global Internet, **incremental deployability and backwards compatibility always defeat cryptographic purity.**

The Current State and Future of Internet Routing Security

The Current State and Future of Internet Routing Security

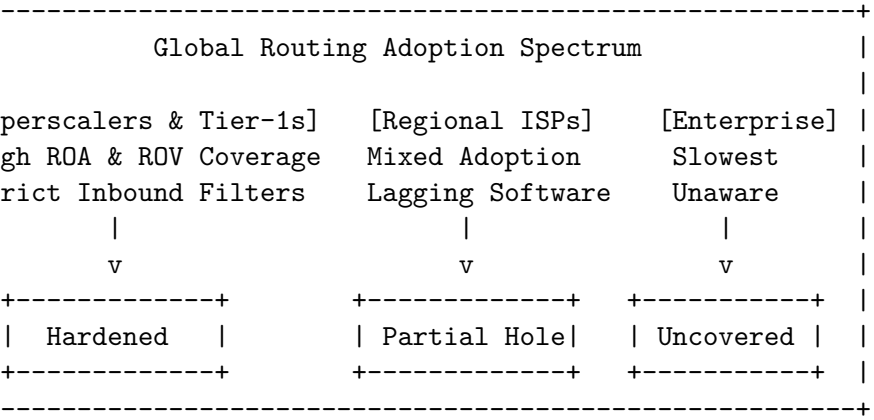
The journey from a completely unauthenticated, trust-based experimental network to a hardened, cryptographically verified planetary communication system is one of the most remarkable transitions in engineering history. As we evaluate the contemporary landscape, we must balance clear-eyed realism about persistent vulnerabilities against the undeniable progress achieved over the past decade.

The Baseline: How Secure Is Global Routing Today?

As of 2026, the baseline of interdomain routing security is fundamentally stronger than at any point in the history of the Internet: - **RPKI ROA Deployment**: Over **55% of global IPv4 prefixes** and more than **60% of IPv6 prefixes** announced in the Default-Free Zone are covered by valid Route Origin Authorizations. - **Route Origin Validation Enforcement**: Major Tier-1 transit backbones (including Cogent, NTT, Lumen, Telia/Arelion, Hurricane Electric) and hyperscale cloud networks (Cloudflare, Google, AWS) actively enforce hard drops on RPKI `Invalid` routes. - **Disappearance of Naive Hijacks**: The era of simple, accidental origin prefix hijacks surviving for hours across the global Internet is over. When a misconfigured router announces an exact-match or more-specific subprefix for a protected block today, the announcement is discarded at the first transit border within seconds.

Deployment Gaps: Who Has Not Adopted and Why?

Despite significant progress, substantial gaps remain across the global network topology:



1. **Enterprise and Stub Network Inertia**: Tens of thousands of small and medium enterprises, regional municipalities, and educational institutions have never signed into their RIR portal to generate ROAs. Their address space remains in the vulnerable `NotFound` state.
2. **Access-Network Economics**: Many consumer broadband and mobile operators in developing markets run legacy router hardware that lacks sufficient memory or software support to run RTR sessions and maintain local VRP tables.
3. **The Unprotected Path**: Because path validation (ASPA or BGPsec) is still in its early operational rollout, adversaries can still forge `AS_PATH` attributes or trigger devastating route leaks that bypass origin validation.

Open Research Problems and Protocol-Level Unknowns

Several critical questions remain active frontiers in academic research and IETF working groups: - **RPKI Repository Centralization**: As reliance on RPKI increases, the five RIR trust anchors represent potential single points of administrative and geopolitical failure. How can the Internet ensure survivability if a major RIR repository is seized, sanctioned, or subjected to sustained volumetric DDoS attacks? - **ASPA Convergence Dynamics**: In a hybrid Internet where only 20% of networks have published ASPA objects, how do ASPA-enabled routers safely interpret ambiguous multi-hop path sequences without causing transient forwarding loops? - **AI/ML in Real-Time Anomaly Mitigation**: Can machine learning models trained on BGPStream telemetry reliably distinguish between legitimate emergency traffic engineering and malicious micro-burst hijacks without generating false-positive disconnects?

Plausible Developments: ASPA, BGPsec Revival, or New Directions?

Over the next decade, the interdomain routing security trajectory will likely unfold across three phases:

1. **Near-Term (ASPA Ubiquity)**: Autonomous System Provider Authorization will follow the adoption path of RPKI. Once major Tier-1 transit providers deploy ASPA verification at their peering borders, the vast majority of accidental route leaks will be permanently eliminated.
2. **Medium-Term (Decentralized Trust Anchors)**: Research into multi-party threshold signatures and transparency logs (analogous to Certificate Transparency in Web PKI) will be integrated into RPKI to prevent unilateral RIR certificate revocation without public audit trails.
3. **Long-Term (SCION and Next-Generation Path-Aware Architectures)**: Clean-slate architectures such as **SCION (Scalability, Control, and Isolation On Next-Generation Networks)** are already running in production across Swiss banking and academic backbones. SCION replaces single-path hop-by-hop forwarding with cryptographically signed, multipath packet routing where senders select verified end-to-end paths through cryptographically isolated failure domains.

A Grounded Assessment: Progress Made, Work Remaining

Building global routing security is not an engineering project with a static finish line; it is an ongoing immune response. The decentralized, sovereign nature of the Internet means that security will always be negotiated at the boundaries between independent networks.

By deploying RPKI Route Origin Validation, enforcing strict IRR-generated customer prefix-lists, rejecting bogons, deploying RFC 9234 roles, and preparing for ASPA path verification, network engineers transform BGP from a fragile, trust-based relic into a hardened, resilient foundation capable of supporting the next century of planetary civilization.